參考用書：Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

# 演算法

## 完整版講義

A4 直式 · 上下兩張 · Lin Cing

---

## 目錄

一、演算法分析與正確性

演算法分析與正確性 3

二、遞迴、遞迴式與分治

遞迴、遞迴式與分治 29

三、排序、選擇與隨機分析

排序、選擇與隨機分析 61

四、資料結構與攤銷分析

資料結構與攤銷分析 138

不相交集合：只合併，不拆分 253

五、動態規劃

動態規劃 273

六、貪心方法

貪婪法 312

七、圖的走訪與連通

圖走訪與連通性 334

八、最小生成樹與最短路徑

最小生成樹與最短路徑 367

九、網路流

網路流 404

十、計算幾何

計算幾何 434

十一、計算複雜度與近似

計算複雜度與近似 455

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 演算法

## 完整版講義

A4 直式 · 上下兩張 · Lin Cing

---

## 目錄

十二、字串比對

字串匹配 502

十三、矩陣運算與 FFT

矩陣運算與 FFT 525

十四、數論演算法

數論演算法 575

十五、線性規劃

線性規劃：模型與證書 613

十六、平行演算法

多執行緒演算法 631

查用附錄

A 求和 653

B 集合、關係、函數、圖與樹 663

C 計數與機率 677

D 矩陣 694

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 演算法分析與正確性

把程式跑完，只回答了「這次發生什麼」。分析演算法還要回答：**所有合法輸入都會得到正確結果嗎？輸入變大時，成本如何增加？**

本章依序建立規格與證明、逐步計數、漸近比較、級數與迴圈分析。插入排序會作為共同案例，從每次搬移一路走到一般成本公式。

先備知識：陣列、迴圈、基本代數與對數；面積與極限推導另用到基礎微積分。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 1–3；求和參閱 Appendix A。

### 閱讀路線

1. 規格與終止性 → 線性搜尋的不變量 → 插入排序的完整分析。
2. 漸近符號與量詞 → 比較成長 → 用級數分析迴圈。
3. 整數邊界與空間 → 成本式的限制 → 數學延伸。

先走完主線，再讀延伸；不熟悉的符號可回查本章定義。



# 先寫規格，再看程式

問題由合法輸入集合 $D$ 與輸出關係 $R(x)$ 描述。輸入 $x \in D$ 可以有不只一個合法答案；演算法則是產生答案的明確程序。

排序的輸入是一列資料，每筆有比較用的鍵值與固定身分。例如 $(3, a), (1, b), (3, c)$ 中，$a, b, c$ 用來辨識原資料，不是額外排序依據。

| 輸出要求 | 檢查方式 |
| --- | --- |
| 鍵值非遞減 | 相鄰鍵值皆滿足 $A[i].key \le A[i+1].key$ |
| 不遺失、不複製資料 | 輸出是輸入資料的排列，重複次數也相同 |
| 若要求穩定排序 | 相等鍵值維持原來的先後 |

只有「有序」仍不夠：刪光資料後的空序列雖有序，卻不是原輸入的排序結果。

## 重複值要靠身分辨認

**輸入 [3a, 1b, 3c] → 鍵值排序；字母表示資料身分**

**有序、保存、穩定**

1b 3a 3c

三項要求全部成立

**有序、保存**

1b 3c 3a

等鍵 a、c 的先後反轉

**有序，但遺失資料**

1b 3a

少了 3c，不是排序結果

穩定性是額外要求。若題目只要求有序且保存全部資料，交換兩筆相等鍵值不會使答案錯誤；若要求穩定，就必須保留 $a$ 在 $c$ 前面。

空輸入與單筆輸入都已排序；演算法仍須合法處理，不能讀取不存在的第一筆。



## 正確結果與一定停下，是兩件事

**部分正確性（partial correctness）**：只要演算法終止，輸出就符合規格。完全正確性（total correctness）還要求每個合法輸入都會終止。

```
while true
    skip
```

這段程式從不交出錯誤答案，也從不交出答案。若只證「終止後正確」，還不能說它完成任務。

證明迴圈會停，常找一個非負整數量：每次迭代嚴格下降，且不會降到負數。有限行程式、有限測試案例，都不能代替這項論證。

## 線性搜尋：先固定返回哪一個

給陣列 $A[1..n]$ 與目標 $x$，返回**第一個**符合 $A[i] = x$ 的索引；若沒有則返回 NIL。允許 $n = 0$，比較與讀取一筆資料各為常數成本。

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

本章陣列索引從 1 開始；$A[a..b]$ 在 $a > b$ 時表示空區間，不會讀取元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 陣列參照與資料身分，要分開理解

本章偽碼以參照存取陣列。把陣列傳給程序後，修改 $A[i]$ 會改到呼叫端看見的資料；只把程序內的變數重新綁到另一個陣列，則不會改綁呼叫端的變數。

| 操作 | 結果 |
| --- | --- |
| $A = [3, 1]$；令 $B = A$；再令 $B[1] = 8$ | 兩個名稱指向同一陣列，$A$ 也變成 $[8, 1]$ |
| $A = [3, 1]$；令 $B = copy(A)$；再令 $B[1] = 8$ | 陣列儲存體不同，$A$ 仍是 $[3, 1]$ |

陣列複製若只複製物件參照，兩個陣列仍可能指向同一批物件；「陣列獨立」不表示「每個物件都深層複製」。排序中標記的身分 $a, b, c$ 則用來辨識資料，不能直接當作其記憶體位址。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 搜尋的不變量，記錄已排除的範圍

每次判斷 `i <= n` 前，維持：$1 \le i \le n+1$，而 $A[1..i-1]$ 都不等於 $x$。

| 階段 | 理由 |
|---|---|
| 初始化 | $i = 1$，已排除範圍為空，條件成立 |
| 維護 | 若本筆不等於 $x$，令 $i \leftarrow i+1$；剛檢查的位置加入排除範圍 |
| 找到時 | $A[i] = x$，且之前沒有匹配，所以返回的是第一個 |
| 掃完時 | $i = n+1$，所有位置都已排除，因此返回 NIL |

未返回的每次迭代都讓非負整數 $n - i + 1$ 減 1，因此必停。$n = 0$ 時直接返回 NIL，不讀取 $A[1]$。



## i 之前已排除，i 之後還不能下結論

在 $A = [6, 2, 6, 5]$ 搜尋 5。到 $i = 3$ 前，只能斷言前兩筆不匹配；即使看見第 3 筆是 6，也要實際比較後才能擴大排除範圍。

到 $i = 4$ 時找到 5，而前段已證明沒有 5，所以返回最早匹配。圖中的「尚未檢查」不等於「不匹配」。

## 同一段程式，三種完整執行

| 輸入與目標 | 依序比較的位置 | 返回值 | 鍵值比較次數 |
|---|---|---|---|
| $[4, 2, 4]$，找 4 | 1 | 1 | 1 |
| $[4, 2, 4]$，找 7 | 1、2、3 | NIL | 3 |
| 空陣列，找 4 | 無 | NIL | 0 |

三例分別檢查最早匹配、搜尋失敗與空輸入；一般正確性仍由不變量證明。

若成功位置均勻分布在 $1..n$，平均比較數為 $(n+1)/2$；若輸入分布不同，不能沿用這個數字。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 計時之前，先說明在數什麼

本章先採 word-RAM 模型：索引與資料可放入固定數量的機器字；讀寫、比較與模型允許的基本算術各計常數成本。這是分析的約定，不是所有實際運算都一樣快。

| 觀察對象 | 常見成本單位 |
|---|---|
| 陣列搜尋 | 鍵值比較次數、讀取次數 |
| 排序 | 比較次數、搬移次數、額外空間 |
| 大整數演算法 | 位元數與多字運算成本 |
| 外部儲存 | 區塊傳輸次數 |

若整數含 $b$ 個位元，任意精度乘法不能不加說明就視為一次常數操作。後面的結論都要連同成本模型閱讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 輸入大小不一定是數值本身

正整數 $N$ 的二進位長度是 $b = \lfloor \lg N \rfloor + 1$，其中 $\lg = \log_2$。以 $N$ 為次數的迴圈，對輸入長度 $b$ 可能是指數成本。

例如 $N = 13$ 寫成 `1101`，只需 4 個位元。若逐一試到 $N$，迭代數為 $\Theta(N)$；對 $b$ 位數輸入，$2^{b-1} \le N < 2^b$，所以是 $\Theta(2^b)$ 次。

陣列通常以元素數 $n$ 為規模；圖常同時用頂點數 $V$、邊數 $E$。把二參數成本寫成 $O(V + E)$，比默默把所有東西叫作 $n$ 更清楚。

### 最好、最壞、平均：先選成本函數

對長度為 $n$ 的輸入集合 $D_n$，令 $C(x)$ 為該輸入的成本。

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

平均成本還需要指定機率分布 $P_n$：

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

**Big-O 並不等於最壞情況。** $B, W, A$ 都是函數，都能各自求上界或下界。隨機演算法還要區分輸入的隨機性與演算法自己擲出的隨機位元。



## 插入排序：把下一筆放進已排序區

輸入 $[(7,a),(2,b),(5,c),(2,d),(1,e)]$，簡記為 $[7a, 2b, 5c, 2d, 1e]$。每輪把一筆資料暫存為 `key`，只把左邊較大的資料往右搬。

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

`and` 由左至右短路求值；$i = 0$ 時不讀取 $A[i]$。$n \le 1$ 時外層不執行。比較採嚴格大於，讓舊的相等鍵留在新資料前面。

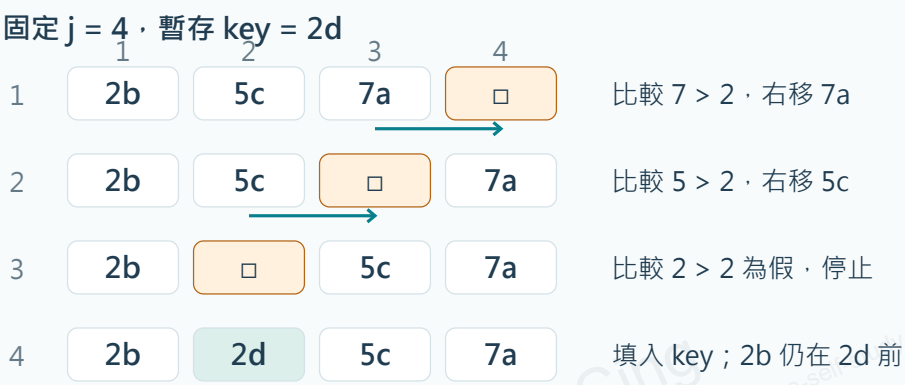
## 每一輪都留下可以核對的結果

| $j$／暫存資料 | 由右向左的鍵值比較 | 右移數 $h_j$ | 填入後的陣列 |
|---|---|---|---|
| 2／$2b$ | $7 > 2$；到邊界 | 1 | $2b, 7a, 5c, 2d, 1e$ |
| 3／$5c$ | $7 > 5$、$2 > 5$ 為假 | 1 | $2b, 5c, 7a, 2d, 1e$ |
| 4／$2d$ | $7 > 2, 5 > 2$、$2 > 2$ 為假 | 2 | $2b, 2d, 5c, 7a, 1e$ |
| 5／$1e$ | $7, 5, 2d, 2b$ 都大於 1 | 4 | $1e, 2b, 2d, 5c, 7a$ |

最後 $2b$ 仍在 $2d$ 前面。共右移 8 次，但鍵值比較是 10 次：第 3、4 輪各多一次失敗比較；到 $i = 0$ 的第 2、5 輪不再比較鍵值。



## 搬移時的重複值，用「洞」來理解




空格是**邏輯洞**，不是寫入的空值。每步由「洞外位置＋暫存 key」保存原資料；第一次搬移後，實體陣列有兩份 $7a$，所以不能把整段實體陣列當成原輸入的排列。

## 外層證明：有序、保存、穩定一起維持

令 $S$ 為開始排序前的輸入副本。每輪開始時：$A[1..j-1]$ 是 $S[1..j-1]$ 的穩定排序，且 $A[j..n] = S[j..n]$。

| 階段 | 不變量如何成立 |
|---|---|
| 初始化 $j = 2$ | 單筆前段已排序，後段未改動 |
| 維護 | 右移所有比新鍵大的尾段，再填入新資料；等鍵不跨越，後段不變 |
| 終止 $j = n + 1$ | 前段就是全部輸入，故全部已穩定排序 |

內層的 $i$ 每次減 1，最多搬 $j - 1$ 筆；外層最多 $n - 1$ 輪，所以必定終止。$n \le 1$ 另外由不執行迴圈成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 內層證明：洞的位置也要寫進條件

固定一輪 $j$，令 $B = A[1..j]$ 為該輪開始的副本，$B[1..j-1]$ 已排序。暫存 $key = B[j]$，令洞在 $i + 1$。

每次內層判斷前：

1. 左段 $A[1..i] = B[1..i]$。
2. 右段 $A[i+2..j] = B[i+1..j-1]$，而這些鍵都大於 $key$。
3. 不計洞內舊值，再加上 $key$，恰好保存 $B$ 的每筆資料。

初始化右段為空；搬移 $A[i]$ 到洞再令 $i \leftarrow i - 1$，三項仍成立。停止時左段皆不大於 $key$，右段皆大於它，填洞便得到有序且保存資料的前段。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把比較、搬移與迴圈判斷分開計數

對 $n \geq 2$，第 $j$ 輪右移 $h_j$ 次；若停在某個 $i \geq 1$ 的失敗鍵值比較，令 $q_j = 1$，否則令 $q_j = 0$。

| 計數對象 | 第 $j$ 輪 | 全程 |
|---|---|---|
| 右移寫入 | $h_j$ | $\sum_{j=2}^{n} h_j$ |
| 最後填入 key 的寫入 | $1$ | $n-1$ |
| 內層完整 guard 判斷 | $h_j + 1$ | $\sum h_j + n - 1$ |
| 鍵值比較 | $h_j + q_j$ | $\sum h_j + \sum q_j$ |

上例 $\sum h_j = 8, \sum q_j = 2$，所以比較 10 次、guard 判斷 12 次、陣列寫入 12 次。暫存 key 的讀取不算在「陣列寫入」裡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼右移數恰好等於逆序對數？

逆序對（inversion）是原輸入中滿足 $p < q$ 且 $S[p].key > S[q].key$ 的位置對。令總數為 $I(S)$。

插入原來第 $j$ 筆時，左側是前 $j-1$ 筆的排序；被右移的恰是其中鍵值大於 $S[j]$ 的資料。因此本輪 $h_j$ 等於以 $j$ 為右端點的逆序對數。

每個逆序對有唯一右端點，所以

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

等鍵不算逆序。上例依右端點分別有 $1, 1, 2, 4$ 對，共 8 對，與右移表一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一演算法的最好、最壞與平均

| 輸入條件 | 逆序對數 | 時間成本 |
|---|---|---|
| 已非遞減排序 | 0 | $\Theta(n)$ |
| 相異鍵嚴格遞減 | $n(n-1)/2$ | $\Theta(n^2)$ |
| 相異鍵的所有排列等機率 | 期望 $n(n-1)/4$ | 期望 $\Theta(n^2)$ |

平均情況的理由：對每對位置 $p<q$，令指示變數 $X_{pq}$ 表示是否形成逆序。在均勻排列模型下，$\mathbb{E}[X_{pq}]=1/2$；線性期望給

$$\mathbb{E}[I]=\sum_{p<q}\mathbb{E}[X_{pq}]=\binom{n}{2}\frac{1}{2}.$$

不需要假設各 $X_{pq}$ 獨立。含重複鍵或非均勻輸入時，必須重新檢查分布。



## 漸近分析：忽略什麼，保留什麼？

我們關心 $n\to\infty$ 時的成長階，允許忽略固定常數倍率與有限多個小輸入。這不代表小輸入的正確性可以忽略。

以下 $n$ 取正整數，$f$ 最終非負，$g$ 最終嚴格正。$O(g),\Omega(g),\Theta(g)$ 都是**函數集合**；$f=O(g)$ 是常用寫法，意思是 $f\in O(g)$。

例如 $n\in O(n^2)$ 正確，只是上界不緊。若要表示已找到上下同階的界，應寫 Θ，不能把所有 Big-O 都讀成「恰好」。

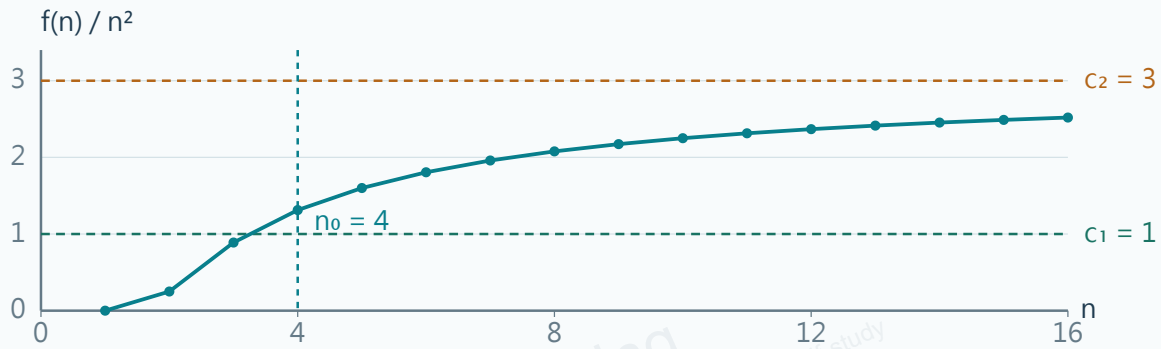
### 上界、下界與緊界的量詞

| 記號 | 正式條件：對所有整數 $n\ge n_0$ |
|---|---|
| $f\in O(g)$ | 存在 $c>0,n_0$，使 $0\le f(n)\le cg(n)$ |
| $f\in\Omega(g)$ | 存在 $c>0,n_0$，使 $0\le cg(n)\le f(n)$ |
| $f\in\Theta(g)$ | 存在 $c_1,c_2>0,n_0$，使 $0\le c_1g(n)\le f(n)\le c_2g(n)$ |

常數可以依固定的 $f,g$ 而定，**不能依輸入大小 $n$ 改變**。門檻不必最小；只要整段尾端都滿足條件即可。

要否定 $f\in O(g)$，必須對任意候選 $c,n_0$，都找得到 $n\ge n_0$ 使 $f(n)>cg(n)$；打敗一組猜測常數還不夠。

## 把常數與門檻畫在同一張圖




n ≥ 4 時整段比值落在 1 與 3 之間；不要求 n₀ 以前也成立。

取 $f(n) = 3n^2 - 8n + 5$，$n \geq 4$ 時，$f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$；$n \geq 1$ 時 $f(n) \leq 3n^2$。

因此 $c_1 = 1, c_2 = 3, n_0 = 4$ 可證 $f \in \Theta(n^2)$。圖顯示比值的位置；涵蓋所有 $n \geq 4$ 的理由是上述不等式，不是有限採樣點。



## 同一個函數，怎麼證明不是線性上界？

已知 $f(n) = 3n^2 - 8n + 5 \geq n^2$ 對 $n \geq 4$ 成立。假設它也在 $O(n)$，則存在固定 $c, n_0$，使 $f(n) \leq cn$ 對所有 $n \geq n_0$ 成立。

選整數 $n > \max\{4, n_0, c\}$，便得到

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

與 $n > c$ 矛盾。因此 $f \notin O(n)$。

這種證明的重點是：先讓對方任選常數，再選足夠大的輸入。不是先固定一個 $n$，再觀察某組常數不適用。

## 兩側的門檻不同，取較大的那一個

若 $f \in O(g)$ 的證據是 $(c_U, N_U)$，$f \in \Omega(g)$ 的證據是 $(c_L, N_L)$，取 $n_0 = \max(N_U, N_L)$，兩側就同時成立。因此

$$\Theta(g) = O(g) \cap \Omega(g).$$

其他常用推論也要保留條件：

| 性質 | 理由或條件 |
|---|---|
| $O$ 的遞移性 | 兩個倍率相乘、兩個門檻取最大 |
| $f \in O(g) \Leftrightarrow g \in \Omega(f)$ | 把同一不等式反向解讀，倍率取倒數 |
| $f + g = \Theta(\max(f, g))$ | 非負時 $\max(f,g) \le f+g \le 2\max(f,g)$ |

若允許正負相消，最後一項不能直接套用，例如 $n + (-n) = 0$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## little-o：每個倍率都能壓過它

$$f \in o(g) \iff \forall c > 0\ \exists n_0(c)\ \forall n \ge n_0(c):\ 0 \le f(n) < cg(n).$$

Big-O 只要求找到一個倍率；little-o 要求**任何正倍率**最終都成立。例如 $7n \in o(n^2)$：給任意 $c > 0$，取整數 $n_0 > 7/c$ 即可。

同理，$f \in \omega(g)$ 要求對每個 $c > 0$，最終都有 $0 \le cg(n) < f(n)$。因此 $n^2 \in \omega(7n)$。

在最終為正的條件下，$f \in o(g)$ 等價於 $f/g \to 0$，$f \in \omega(g)$ 等價於 $f/g \to \infty$。$7n^2$ 與 $n^2$ 同階，不是 little-o。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 比值極限是工具，不是唯一判準

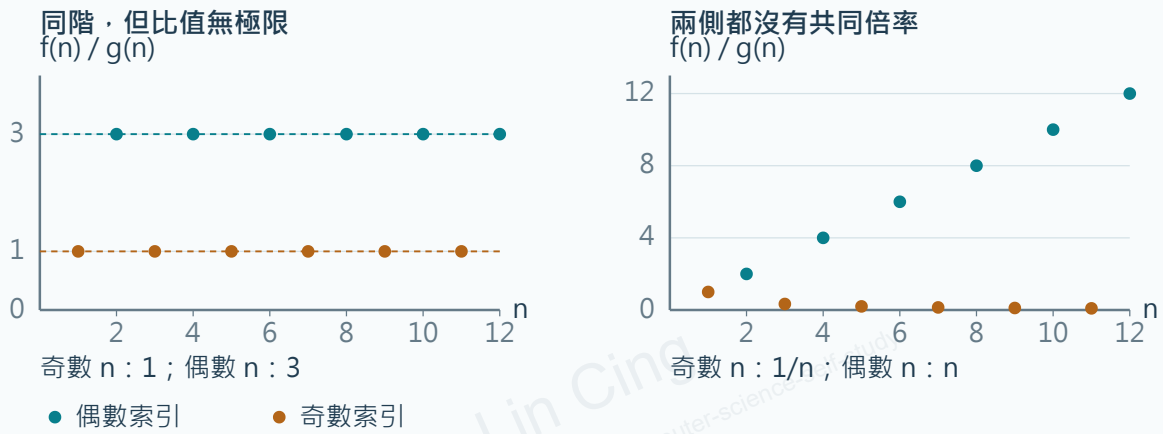
| $\lim f(n)/g(n)$ | 可以得到的結論 |
|---|---|
| 0 | $f \in o(g)$ |
| 有限正數 $L$ | $f \in \Theta(g)$ |
| $+\infty$ | $f \in \omega(g)$ |
| 不存在 | 必須繼續分析，不能直接判定不可比較 |

有限正 $L$ 的理由：取 $\varepsilon = L/2$，最終有 $L/2 < f/g < 3L/2$，所以兩側都能用固定倍率夾住。

只有比值存在時才使用前三列。對數法也需檢查定義域；$\lg f / \lg g \to 1$ 本身**不保證** $f = \Theta(g)$。



## 沒有極限，也可能同階




左圖 $f = n(2 + (-1)^n), g = n$：$n \le f \le 3n$，比值雖無極限仍同階。右圖偶數取 $f = n^2$、奇數取 $f = 1$，$g = n$：偶數比值無界，否定 $O(g)$；奇數比值趨 0，否定 $\Omega(g)$。

## 函數界與輸入界，不要交換量詞

令 $C(x)$ 為輸入成本，$W(n)=\max_{x\in D_n}C(x)$。若 $W(n)\in O(n^2)$，表示存在**共同**常數 $c,n_0$，對所有夠大的 $n$、所有 $x\in D_n$，都有 $C(x)\le cn^2$。

若只說「對每個輸入都找得到一個常數」，而常數可以隨輸入增大，便不能得到共同上界。

插入排序的 $W(n)=\Theta(n^2)$，只保證最壞輸入達平方階；已排序輸入仍是 $\Theta(n)$。因此不能推成「每個輸入都至少需要平方時間」。

### 函數比較的三種實際做法

| 方法 | 適合的情況 | 要避免的跳步 |
|---|---|---|
| 定義夾界 | 多項式、振盪、要給常數 | 只列數值或只證一側 |
| 比值極限 | 正函數且極限可求 | 把極限不存在當成無法比較 |
| 取對數比較 | 正的乘積、冪次、指數 | 把對數同階誤當原函數同階 |

例如 $f=n\lg n, g=n$，雖然 $\lg f/\lg g=1+\lg\lg n/\lg n\to 1$，但 $f/g=\lg n\to\infty$，所以 $f\in\omega(g)$。

對數能協助比較大小；是否達到常數倍率的同階，仍須回到原函數或更精確的對數差。



## 成長階梯上的常數，都必須固定

對固定 $a>1$、$b>0$、$\varepsilon>0$、$k>0$，有

$$1\prec\lg\lg n\prec(\lg n)^b\prec n^\varepsilon\prec a^n\prec n!\prec n^n,$$

其中 $f\prec g$ 表示 $f=o(g)$；只比較充分大的正整數 $n$。

取對數令 $x=\ln n$，則 $(\lg n)^b/n^\varepsilon$ 等於固定常數乘 $x^b/e^{\varepsilon x}$，趨近 0。又 $\ln(n^k/a^n)=k\ln n-n\ln a\to-\infty$，故 $n^k=o(a^n)$。

這裡使用基本極限 $\ln n/n\to0$，下個版面補上理由。若底數或指數依 $n$ 變動，不能套用「固定常數」的結論。

### 為什麼對數會被正次方壓過？

對實數 $x\ge4$，$\sqrt{x}-\ln x$ 的導數為 $(\sqrt{x}-2)/(2x)\ge0$，且在 $x=4$ 時為正。因此 $0\le\ln x/x\le1/\sqrt{x}\to0$。

現在固定 $b,\varepsilon>0$，令 $t=\ln n$，取自然對數比較：

$$\ln\frac{(\ln n)^b}{n^\varepsilon}=b\ln t-\varepsilon t=t\left(b\frac{\ln t}{t}-\varepsilon\right).$$

因 $\ln t/t\to0$，括號最終不超過 $-\varepsilon/2$，整式趨向負無限，原比值便趨 0。這就證出 $(\ln n)^b=o(n^\varepsilon)$；換成固定底數只差常數。

## 對數、指數與底數轉換

| 式子 | 正確解讀 |
| --- | --- |
| $\log_a n = \lg n / \lg a$ | 固定 $a > 1$ 時，不同底數只差常數 |
| $\lg(n^k) = k \lg n$ | 先做冪次再取對數 |
| $\lg^k n = (\lg n)^k$ | 先取對數再做冪次 |
| $2^{\lg n} = n$ | 底數與對數配對 |
| $2^{2 \lg n} = n^2$ | 指數裡的常數不能丟掉 |

$\lg n$ 與 $2 \lg n$ 同階，卻不能推出 $2^{\lg n}$ 與 $2^{2 \lg n}$ 同階。對數底數若隨輸入改變，例如 $\log_n n = 1$，也不是固定底數的換底情況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從迴圈到級數：每層的範圍要重看

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

第 $i$ 輪執行 $i$ 次，因此總次數為

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

不能把內層「平均大約一半」當證明；精確求和已給一般公式。另一種夾界：總數不超過 $n^2$；後半至少 $n/2$ 輪，每輪至少 $n/2$ 次，所以至少 $n^2/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三角形不是少一個成長階

上界：不超過 6 × 6 = 36 格。下界：第 4–6 輪，各取前 3 格。

每格代表一次內層執行。$n = 6$ 共 $1 + 2 + 3 + 4 + 5 + 6 = 21$ 次；第 4–6 輪各取前三次，已有 $3 \times 3 = 9$ 次。這是前面一般平方下界的具體子集合。



## 倍增迴圈：數的是乘了幾次 2

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

若採無溢位的整數算術且 $n \geq 1$，依次造訪 $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$，共 $\lfloor \lg n \rfloor + 1$ 次；$n = 13$ 時是 $1, 2, 4, 8$，共 4 次。

固定寬度實作必須避免 `2*i` 溢位。例如本輪造訪後，若 $i > \lfloor n/2 \rfloor$ 就停止，否則才倍增。數學偽碼的終止證明不能直接涵蓋溢位後的機器行為。

### 幾何級數：最末一層已決定同階

若外層 $i = 1, 2, 4, \ldots, 2^k \leq n$，內層各做 $i$ 次，則總成本

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

因 $2^k \leq n < 2^{k+1}$，有 $n/2 < 2^k \leq n$，所以總成本是 $\Theta(n)$，不是 $\Theta(n \lg n)$。

$n = 13$ 時四輪各做 $1, 2, 4, 8$ 次，共 15 次。雖有 $\Theta(\lg n)$ 輪，但不能把每輪都當成 $n$ 次後，誤認為已得到緊界；那樣只證出寬鬆上界。

## 倍增的是每輪工作量，不是輪數乘最後一輪

左圖每格是一份工作，四輪各做 $1, 2, 4, 8$ 份；右圖的累積量是 $1, 3, 7, 15$。最後一輪的 8 份已占總量一半以上。

對一般最後一輪 $2^k$，$2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$。所以總量與最後一輪同階，而不是再多乘一個輪數。



## 調和級數：用面積取得兩側界

令 $H_n = \sum_{i=1}^{n} 1/i$。$1/x$ 在正半軸遞減，因此

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}.$$

亦即 $\ln(n+1) \le H_n \le 1 + \ln n$。當 $n \ge 3$，$1 + \ln n \le 2 \ln n$，故 $H_n = \Theta(\ln n) = \Theta(\lg n)$。

上界把第一項 1 分開，再用每段左側的函數值蓋住第 2 項以後；下界則在每段用左端矩形蓋住積分。下個版面把兩種矩形分開畫。

### 冪次求和：不用展開也能找緊界

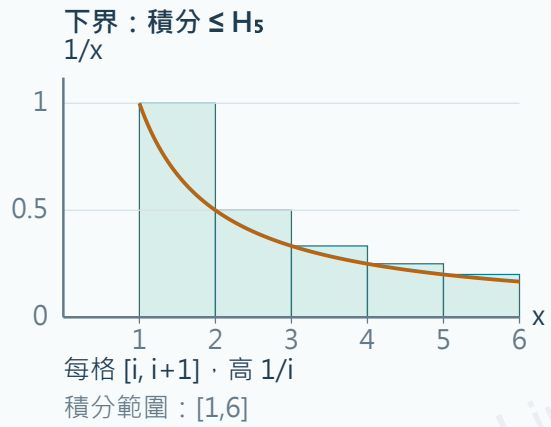
對固定實數 $p \ge 0$，考慮 $S_p(n) = \sum_{i=1}^{n} i^p$。上界直接用 $n$ 項、每項不超過 $n^p$：$S_p(n) \le n^{p+1}$。

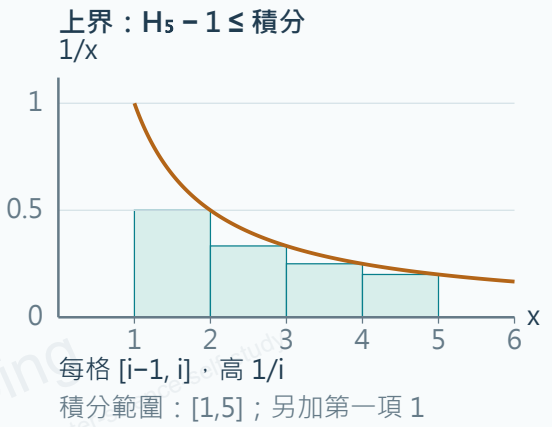
下界只取 $i = \lceil n/2 \rceil .. n$，至少 $n/2$ 項，每項至少 $(n/2)^p$，所以

$$S_p(n) \ge \frac{n}{2} \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

因此 $S_p(n) = \Theta(n^{p+1})$。這個證法保留了固定倍率，也說清楚為什麼 $p$ 必須固定；$p = -1$ 是調和級數，不能套這個結論。

## 同一條曲線，兩種矩形方向




左圖 $n = 5$：每段 $[i, i+1]$ 的矩形高 $1/i$，總面積 $H_5$ 大於 $\int_1^6 dx/x$。右圖第 $i = 2..5$ 項的矩形放在 $[i-1, i]$，位於曲線下，所以 $H_5 - 1 \le \int_1^5 dx/x$。

$H_5 = 137/60 \approx 2.2833$，介於 $\ln 6 \approx 1.7918$ 與 $1 + \ln 5 \approx 2.6094$ 之間。兩圖的座標與面積單位相同。



## 相依的巢狀迴圈，可能是 n log n

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

內層造訪 $i, 2i, \ldots, \lfloor n/i \rfloor i$，所以

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

對 $x \ge 1$，$x/2 \le \lfloor x \rfloor \le x$，故 $nH_n/2 \le S(n) \le nH_n$，得到 $\Theta(n \lg n)$。$n = 6$ 時各輪是 $6, 3, 2, 1, 1, 1$ 次，共 14 次。

## 階乘先取對數，再比較成長

對 $n \ge 4$，$n! \le n^n$，因此 $\lg(n!) \le n \lg n$。

另一方面，後半至少 $n/2$ 個因子都不小於 $n/2$，所以

$$\lg(n!) \ge \frac{n}{2} \lg(n/2) \ge \frac{n}{4} \lg n.$$

故 $\lg(n!) = \Theta(n \lg n)$。這不代表 $n! = \Theta(n^n)$：事實上兩者比值趨近 0。

對固定 $a > 1$，$\ln(n!/a^n) \ge (n/2)\ln(n/2) - n \ln a \to \infty$，可證 $a^n = o(n!)$。若需精確倍率，再使用 Stirling 公式，不能把對數裡的誤差隨意指數化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## floor、ceiling 與餘數的邊界

對實數 $x$，$x - 1 < \lfloor x \rfloor \le x \le \lceil x \rceil < x + 1$。因此整數 $n$ 滿足

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n.$$

對整數 $a$ 與正整數 $m$，本章採數學餘數

$$a \bmod m = a - m\lfloor a/m \rfloor, \qquad 0 \le a \bmod m < m.$$

例如 $-7 \bmod 3 = 2$，因為 $-7 = 3(-3) + 2$。某些語言的 `%` 採截向零的商，會得到負餘數；把公式搬成程式時必須核對語義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 連續整數分割，什麼時候能合併？

對 $x \geq 0$、正整數 $a, b$，有

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

floor 的證明：令 $q = \lfloor x/(ab) \rfloor$，則 $abq \leq x < ab(q+1)$。除以 $a$ 再取 floor，得 $bq \leq \lfloor x/a \rfloor \leq bq + b - 1$；再除以 $b$ 取 floor，結果就是 $q$。

例如 $x = 17, a = 3, b = 2$，兩次 floor 是 $17 \to 5 \to 2$，直接除以 6 取 floor 也是 2。$a, b$ 為正整數是上述整數區間論證的前提。

### ceiling 的端點，要改用嚴格下界

令 $q = \lceil x/(ab) \rceil$，便有 $ab(q-1) < x \leq abq$。除以 $a$ 再取 ceiling，得到

$$b(q-1) + 1 \leq \lceil x/a \rceil \leq bq.$$

再除以 $b$ 取 ceiling，結果也是 $q$。$x = 17, a = 3, b = 2$ 時，$17 \to 6 \to 3$，與 $\lceil 17/6 \rceil = 3$ 一致。

對正整數 $u, v$，另有 $\lceil u/v \rceil \leq (u+v-1)/v$、$\lfloor u/v \rfloor \geq (u-v+1)/v$：寫成 $u = qv + r, 0 \leq r < v$，分 $r = 0$ 與 $r > 0$ 即可核對。實作 `(u+v-1)/v` 時仍須避免加法溢位。



## 空間成本：同時活著多少，才是峰值

額外空間不含輸入本身，但通常包括暫存陣列與遞迴呼叫堆疊；若另排除輸出，須明寫。

| 情境 | 累積配置量 | 同時存活的峰值 |
|---|---|---|
| 依次配置並釋放 $n$ 個單字區塊 | $\Theta(n)$ | $\Theta(1)$ |
| 配置 $n$ 個單字且全部保留 | $\Theta(n)$ | $\Theta(n)$ |
| 共 $K$ 次呼叫，峰值有 $d$ 個常數大小框架同時存活 | $\Theta(K)$ | $\Theta(d)$ |

插入排序只用固定數量的索引與暫存資料，在單筆資料可用常數個字表示的假設下，額外空間為 $\Theta(1)$。若每筆是大型物件，還要說明搬的是物件還是參照。

## 配過多少與同時保留多少：畫成存活區間

依次配置 1、2、3、4 個字：立即釋放時峰值 4，全部保留時峰值 10；兩者累積配置量同為 10。只計圖中存活區塊，不含配置器碎片、快取或作業系統常駐記憶體。



## 常數與交會點：成長階不能回答全部

同一成本單位下，兩種方法的精確模型是 $T_A(n) = 8n^2$、$T_B(n) = 64n \lg n$，$n \geq 2$。$A$ 較快等價於 $n < 8 \lg n$。

| $n$ | $8n^2$ | $64n \lg n$ | 比較 |
|---|---|---|---|
| 16 | 2048 | 4096 | A 較快 |
| 32 | 8192 | 10240 | A 較快 |
| 64 | 32768 | 24576 | B 較快 |

對實數 $n \geq 4$，$n / \lg n$ 嚴格遞增；檢查 $n = 43, 44$ 分別落在兩側，可知正整數 $n \geq 2$ 的切換點在 43 與 44 之間。若模型來自量測，這個結論也只適用於該模型與環境。

## 同階也可能差很多，快硬體也有單位

若 A 的速度是每秒 $10^9$ 個模型步驟，B 是每秒 $10^8$ 步，則應比較

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8,$$

不能把步驟數與秒數直接混在同一個不等式。換硬體改變常數，換演算法可能改變成長階；兩者都需要明確成本模型。

空間上限也會限制可用方法。例如需要 $n^2$ 個項目的表格，即使時間估算可接受，也可能放不進記憶體。實際選擇須同時檢查時間、空間與資料特性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 綜合分析：從程式走到一般結論

分析一段新程式時，可以依下列順序建立答案，而不是先猜一個 Big-O：

1. 固定合法輸入、輸入規模、比較與算術模型。
2. 用小輸入列完整狀態，找到每輪實際做的事。
3. 寫不變量、邊界與遞減量，處理正確性及終止性。
4. 將每次成本寫成一般求和，或建立遞迴式。
5. 分開證上界與下界，註明最好／最壞／平均的對象。
6. 算峰值空間，核對溢位、別名與分布等前提。

本章的插入排序得到 $T(S) = \Theta(n + I(S))$，比只寫最壞 $O(n^2)$ 更能說明輸入特性如何改變成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 延伸：多項式最高次項為什麼主導？

令 $p(n) = a_d n^d + \sum_{i<d} a_i n^i$，固定 $d \geq 1$、$a_d > 0$，並設 $M = \sum_{i<d} |a_i|$。

對 $n \geq 1$，低階項的絕對值總和不超過 $M n^{d-1}$。再取 $n \geq 2M/a_d$，便有

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

因此 $p(n) = \Theta(n^d)$，即使低階係數有負數仍成立；若最高次係數為負，就不符合本章時間函數最終非負的條件。$d = 0, a_0 > 0$ 時另得 $\Theta(1)$。

### 延伸：Big-O 式子的等號不能反推

式子 $n^2 + n = n^2 + O(n)$ 表示右側存在一個屬於 $O(n)$ 的函數，使等式成立；本例選 $h(n) = n$ 即可。

由此可推出 $n^2 + n = \Theta(n^2)$，卻不能從 $f = \Theta(n^2)$ 反推 $f = n^2 + O(n)$：反例 $f = 2n^2$，差值 $n^2$ 不在 $O(n)$。

若要逐項運算，先把匿名函數命名。例如 $u \in O(n), v \in O(n)$，用三角不等式得 $|u + v| \leq (c_u + c_v) n$。若涉及相減，需允許帶符號的誤差界，不能忽略原來非負的定義慣例。



## 延伸：e、ln 與近似的誤差

對 $x > -1$，$\ln(1 + x) \leq x$。因此正整數 $n$ 有 $(1 + 1/n)^n \leq e$。

更精確地，對 $x \geq 0$，

$$x - \frac{x^2}{2} \leq \ln(1 + x) \leq x.$$

下界可由導數得到：$h(x) = \ln(1 + x) - x + x^2/2$，$h(0) = 0$ 且 $h'(x) = x^2/(1 + x) \geq 0$。代入 $x = 1/n$，得到

$$e^{1 - 1/(2n)} \leq (1 + 1/n)^n \leq e.$$

所以不只知道它趨向 $e$，也知道有限 $n$ 時的一側誤差。近似式應附適用範圍，不能任意丟掉高階項。

## 延伸：指數近似也能給明確餘項

Taylor 定理給 $e^x = 1 + x + (e^\xi/2)x^2$，其中 $\xi$ 介於 0 與 $x$。因此 $|x| \le 1$ 時，

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

所以 $e^x = 1 + x + \Theta(x^2)$ 的極限方向是 $x \to 0$，不是 $x \to \infty$。另由凸性，$e^x \ge 1 + x$ 對所有實數成立。

標準冪級數 $e^x = \sum_{k=0}^{\infty} x^k/k!$ 對所有實數收斂；若只取前幾項，還須像上式一樣控制餘項。本節使用基礎微積分的 Taylor 定理，不在此重新推導該定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 延伸：對數界也要涵蓋負的 x

對 $x > -1$，較一般的雙界為

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

上界令 $h = x - \ln(1+x)$，導數為 $x/(1+x)$，最小值在 0，且 $h(0) = 0$。下界令 $q = \ln(1+x) - x/(1+x)$，導數為 $x/(1+x)^2$，也在 0 取得最小值 0；兩個論證都涵蓋正負兩側。

對固定實數 $z$，充分大時代入 $x = z/n$，得 $z/(1+z/n) \le n\ln(1+z/n) \le z$。兩側都趨近 $z$，因此 $(1+z/n)^n \to e^z$。

當 $|x| < 1$，標準展開為 $\ln(1+x) = \sum_{k=1}^{\infty} (-1)^{k+1}x^k/k$；不能省略其適用範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 延伸：Stirling 公式要保留什麼精度？

對正整數 $n$，Stirling 公式為

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling 定理給出主近似的相對誤差。取自然對數可得

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

相對誤差 $O(1/n)$ 經取對數仍為 $O(1/n)$，因 $\ln(1+u) = O(u)$ 對充分小的 $u$ 成立。反過來，若只知道對數相差 $O(n)$，指數化後可能相差指數倍率，不能說階乘值只有常數誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 延伸：把 Stirling 誤差變成數值區間

Stirling 定理較精確的形式為：

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \ge 1).$$

取 $n = 6$，主近似 $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$，小於 $6! = 720$。代入誤差界得到

$$Le^{1/73} < 720 < Le^{1/72}.$$

修正項有明確範圍，不是任意四捨五入。由 $\alpha_n = \Theta(1/n)$ 與零附近 $e^u - 1 = \Theta(u)$，可得 $e^{\alpha_n} = 1 + \Theta(1/n)$。粗略乘積界只決定 $\lg(n!)$ 的成長階，這個定理則進一步控制主近似的相對誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 延伸：反覆取對數，停止條件不能省

令 $\lg^{(0)} n = n$，$\lg^{(k+1)} n = \lg(\lg^{(k)} n)$；只在前一步大於 1 時再取對數。定義

$$\lg^{*} n = \min\{k \ge 0 : \lg^{(k)} n \le 1\}.$$

| 輸入 | 依次得到的值 | $\lg^{*} n$ |
|---|---|---|
| 1 | $1$ | 0 |
| 2 | $2 \to 1$ | 1 |
| 16 | $16 \to 4 \to 2 \to 1$ | 3 |
| 65536 | $65536 \to 16 \to 4 \to 2 \to 1$ | 4 |

$\lg^{k} n$ 是對數值的 $k$ 次方；$\lg^{(k)} n$ 是函數反覆套用。兩者記號不同，成長率也不同。



## 延伸：Fibonacci 的閉式與驗證

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。設 $\varphi = (1+\sqrt{5})/2$、$\psi = (1-\sqrt{5})/2$，兩者都滿足 $r^2 = r + 1$。

因此 $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ 滿足同一遞迴，且 $G_0 = 0, G_1 = 1$。依數學歸納法，$G_n = F_n$ 對每個非負整數成立。

因 $|\psi| < 1 < \varphi$，有 $F_n = \Theta(\varphi^n)$。但 $F_n$ 的**數值大小**不等於計算它的演算法時間：樸素遞迴、表格法與快速倍增必須各自分析，後續遞迴章再比較。

### 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

## 遞迴、遞迴式與分治

遞迴不只是「函式呼叫自己」。要先說清楚每次呼叫負責什麼、為什麼一定停，再把所有呼叫與合併的成本加起來。

本章依序讀：**一條遞迴路徑 → 合併排序 → 遞迴式的證明工具 → 最大子陣列與矩陣乘法 → 不等分割的延伸工具**。

先備知識：第 1 章的不變量、歸納法、漸近符號與級數；矩陣例子另用基本矩陣乘法，延伸定理使用積分。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 2、4；另參考 MIT 6.046J（2005）Lecture 2–3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 子呼叫的答案，是歸納假設的接口

設 $P(n)$ 表示：「對所有大小為 $n$ 的合法輸入，程序會終止，且回傳符合規格的結果。」

1. **基例**：直接核對最小輸入，包括空輸入是否允許。
2. **縮小**：每個子問題都合法，大小嚴格小於 $n$。
3. **組合**：假設較小呼叫的結果正確，證明目前呼叫也正確。

這是強歸納法。不能先假設「目前這次呼叫正確」，也不能只證子問題變小就當作結果正確。

成本另寫 $T(n) = \sum_i T(n_i) + f(n)$：$f$ 只計本次的比較、分割、複製與合併，**不能把子樹的成本再算一次**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 二分搜尋只繼續走一邊

前提：$A[1..N]$ 已非遞減排序，鍵值具有全序；回傳**任一**匹配位置，若沒有則回傳 NIL。整體呼叫為 $(1, N)$，$N = 0$ 時直接返回。

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

本章採包含兩端點的區間。非空呼叫滿足 $1 \le low \le high \le N$；空區間不讀取陣列。這裡沒有保證回傳第一個重複鍵。



## 為什麼可以整段排除？

若 $A[mid] < x$，則所有 $i \le mid$ 都有 $A[i] \le A[mid] < x$，所以答案若存在，必在右半。若 $A[mid] > x$，同理只能留左半；相等就已符合規格。

目前長度 $n = high - low + 1 \ge 1$；繼續搜尋的長度至多 $\lfloor n/2 \rfloor < n$。空區間是失敗基例，故遞迴會終止。以長度強歸納，即得整體正確性。

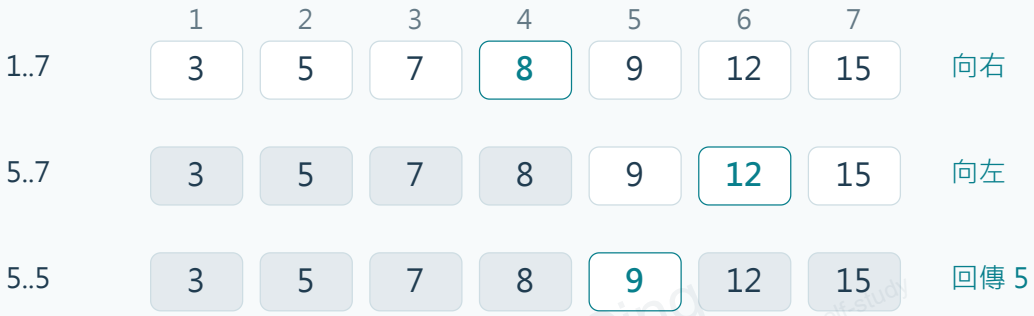
固定成本比較、只傳索引時，最壞成本滿足

$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

沿最長路徑只有 $\Theta(\lg(n+1))$ 個呼叫；最好情況為根即匹配，成本 $\Theta(1)$。順序執行的堆疊峰值為 $\Theta(\lg(n+1))$。

## 相同陣列，找到與找不到各走一遍

找 9：保留白格；目前中點為青框




若找 10：最後再呼叫空區間 6..5 → NIL。

$A = [3, 5, 7, 8, 9, 12, 15]$。找 9：$(low, mid, high)$ 依序為 $(1, 4, 7)$、$(5, 6, 7)$、$(5, 5, 5)$，回傳 5。

找 10 的前三次檢查相同；最後因 $9 < 10$ 呼叫 $(6, 5)$，空區間回傳 NIL。圖中灰格只表示已排除，不是把原陣列刪掉。



## 同一個二分邏輯，複製成本可以差很多

固定原始長度為 $N$，目前區間長度為 $n$。考慮最壞的搜尋路徑：

| 傳參方式 | 每次呼叫的額外工作 | 整次最壞成本 |
|---|---|---|
| 只傳索引或 view | $\Theta(1)$ | $\Theta(\lg(N+1))$ |
| 每次複製目前區間 | $\Theta(n)$ | $\Theta(N)$ |
| 每次複製整個原始陣列 | $\Theta(N)$ | $\Theta(N \lg(N+1))$ |

第二列沿路求和 $N + N/2 + N/4 + \cdots = \Theta(N)$。第三列中，$N$ 不隨遞迴縮小，不能偷換成 $n$。

**子問題只有一個，也能畫遞迴樹**：此時樹退化成一條路。是否適合這個工具，與呼叫數必須大於一無關。

## 快速冪：相同子問題只算一次

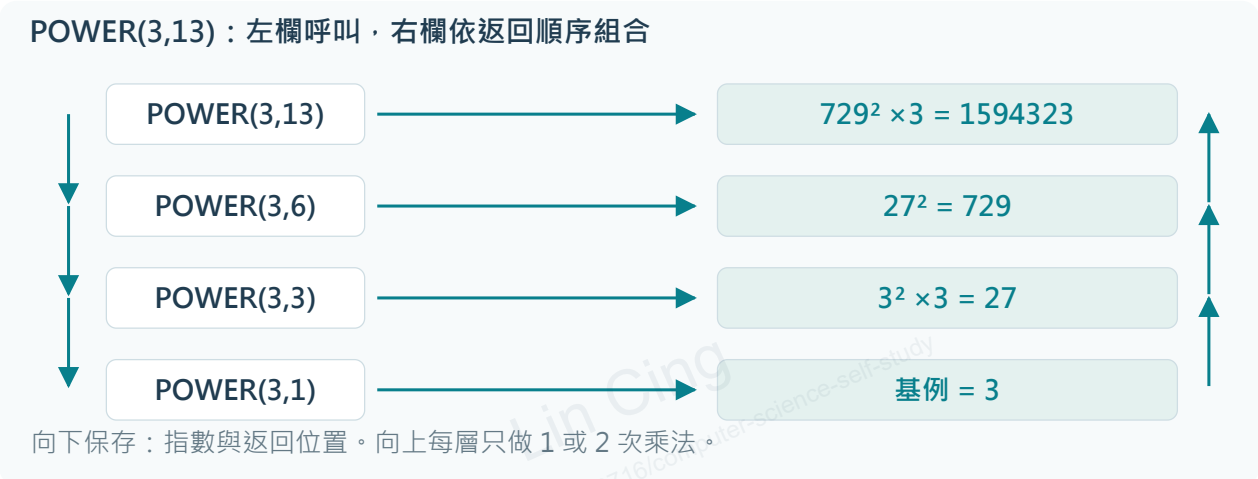
輸入為非負整數 $n$，輸出 $a^n$；約定 $a^0 = 1$。本例先計乘法次數，不把大整數乘法一律視為固定時間。

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

若 $n = 2k$，由 $y = a^k$ 得 $y^2 = a^n$；若 $n = 2k + 1$，再乘 $a$。$\lfloor n/2 \rfloor < n$ 與兩個基例完成強歸納證明。



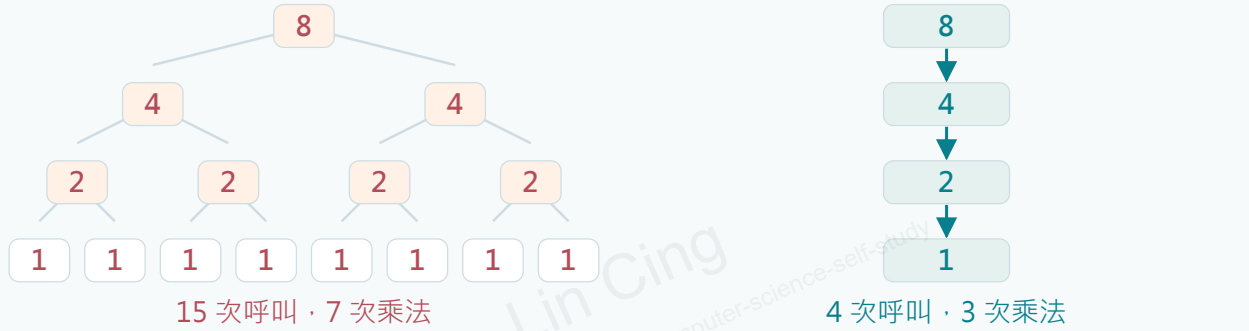
## 往下保留返回位置，往上完成乘法




對 $3^{13}$，向下的指數為 $13 \to 6 \to 3 \to 1$。向上得到 $3 \to 27 \to 729 \to 1594323$；分別做 0、2、1、2 次乘法，共 5 次。

對 $n \geq 1$，平方次數為 $\lfloor \lg n \rfloor$；額外乘 $a$ 的次數為二進位 1 的個數減 1。因此乘法總數為 $\lfloor \lg n \rfloor + \mathrm{popcount}(n) - 1$；$n = 0$ 則為 0。

## 代數式出現兩次，不代表要呼叫兩次




若把 $a^{n/2}a^{n/2}$ 寫成兩次獨立的 POWER 呼叫，即使兩次答案相同，成本仍滿足 $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$，先限 $n$ 為二冪。

保存一次結果再平方，則 $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1)$，在單位成本乘法模型下為 $\Theta(\lg n)$。差別在**實際執行了幾個呼叫**，不是怎麼簡化乘積。

固定寬度實作還要處理乘法溢位；若只需 $a^n \bmod m$，可每次乘法後取模，但乘積本身仍須可安全計算。



## Fibonacci：呼叫樹的重複與輸出大小

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。直接遞迴每次都重算；呼叫總數滿足 $C_0 = C_1 = 1$、$C_n = C_{n-1} + C_{n-2} + 1$。

令 $D_n = C_n + 1$，由同一遞迴與基例可證 $C_n = 2F_{n+1} - 1$。例如算 $F_5 = 5$ 需要 15 次呼叫；時間為 $\Theta(\varphi^n)$，但堆疊深度只有 $\Theta(n)$，其中 $\varphi = (1 + \sqrt{5})/2$。

由小到大只保存前兩項，做 $n - 1$ 次加法即可算 $F_n$（$n \geq 2$）。這是避免重算的基本想法，完整動態規劃設計留到後續章節。

**算術步數與位元成本要分開**：$F_n$ 有 $\Theta(n)$ 個位元；「保存兩個數」是兩個大整數，並非固定數量的機器字。逐項做普通整數加法的位元時間為 $\Theta(n^2)$。

## 用矩陣冪精確計算 Fibonacci

令 $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$。對 $n \geq 1$，

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

$n = 1$ 直接成立。右乘 $Q$ 後，第一欄是原本兩欄之和、第二欄是原第一欄，恰好得到下一組 Fibonacci，歸納完成。

例如 $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$、$Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$，故 $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$，右上角為 $F_5 = 5$。$n = 0$ 直接回傳 0。

對矩陣套快速冪，只需 $O(\lg(n+1))$ 次 $2 \times 2$ 矩陣乘法，且全程可用精確整數。這是**算術操作數**；輸出有 $\Theta(n)$ 個位元，因此位元時間不可能是 $O(\lg n)$。用浮點數計閉式再四捨五入也不能無條件保證正確。



## 合併排序與分治入門

分治（divide and conquer）把問題拆成較小的同類問題，遞迴求解，再合併答案。以下排序包含兩端點的 $A[p..r]$，長度 $m = r - p + 1$；$p > r$ 表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

非空呼叫要求 $1 \leq p \leq r \leq n$；整體呼叫 $(1, n)$，空陣列的 $(1, 0)$ 直接返回。子區間長度為 $\lceil m/2 \rceil$、$\lfloor m/2 \rfloor$，$m \geq 2$ 時都嚴格小於 $m$。

中點寫成差值式，可避免直接計算 $p + r$ 的溢位；索引、長度與加減仍須在實作型別可表示的範圍內。

## 不依賴額外鍵值的 MERGE

前提為 $1 \le p \le q < r \le n$，兩半都已穩定排序。$L, R$ 必須是**獨立副本**，長度分別為 $q-p+1$ 與 $r-q$，不能被後續回寫 $A$ 破壞。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

鍵值相等時先取 $L$：原輸入左半的資料本來就在右半之前。以長度判斷耗盡，不需要鍵值域之外的無窮大哨兵。



## MERGE 的保存、順序與終止證明

令 $m = r - p + 1$。主迴圈每次比較前，維持

$$k - p = (i - 1) + (j - 1).$$

$A[p..k-1]$ 恰是已消耗的 $L[1..i-1]$ 與 $R[1..j-1]$ 的穩定合併，已排序，且其中每個鍵都不大於任何未消耗的鍵。其餘資料仍在獨立副本中。

初始化時輸出為空，條件成立。兩邊都非空時，各自首筆就是該邊最小的剩餘鍵；取較小者、相等先取左，便是全體下一筆。消耗一筆並令 $k$ 加一，維持長度、保存、順序與穩定性；剩餘筆數嚴格減一，故會終止。

主迴圈結束後，一邊耗盡；另一邊已排序，且不小於輸出尾端，可依序接上。最後 $i = |L| + 1, j = |R| + 1, k = r + 1$，全部 $m$ 筆恰輸出一次。若只用原陣列的 view，回寫可能覆蓋尚未讀取的資料，破壞這個證明的前提。

## 有重複 key 的完整合併

令 $L = [1a, 4b, 4c]$、$R = [2d, 4e, 6f]$，左半所有資料原本都早於右半。下表的 $i, j$ 是比較前的索引；只比較鍵值，字母用來辨識資料身分。

| $i, j$ | 比較與選擇 | 輸出前綴 |
|---|---|---|
| 1,1 | $1 \le 2$，取 1a | 1a |
| 2,1 | $4 > 2$，取 2d | 1a,2d |
| 2,2 | $4 \le 4$，取左側 4b | 1a,2d,4b |
| 3,2 | $4 \le 4$，取左側 4c | 1a,2d,4b,4c |
| 4,2 | 左半耗盡，接上右半 | 1a,2d,4b,4c,4e,6f |

共 4 次鍵值比較、6 次輸出寫入，另有 6 筆資料複製到 $L, R$。最壞比較數為 $m - 1 = 5$；$\Theta(m)$ 描述成本階，並不表示每次都恰比較 $m$ 次。



## 有重複 key 的完整合併：圖解

**相等取左：4b、4c 都必須早於 4e**

| | L | | | R | | | 輸出前綴 |
|---|---|---|---|---|---|---|---|
| start | 1a | 4b | 4c | 2d | 4e | 6f | ø |
| 1≤2 | 1a | 4b | 4c | 2d | 4e | 6f | 1a |
| 4>2 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d |
| 4≤4 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b |
| 4≤4 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b, 4c |
| tail | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b, 4c, 4e, 6f |

綠底 = 已消耗；所有未消耗資料仍保存在獨立的 L、R 副本。

## Sentinel 是有前提的另一種表示

哨兵（sentinel）是在每個暫存陣列尾端加一個特殊標記，讓合併可固定輸出 $m$ 次，不必每輪判斷哪邊耗盡。但標記的排序順位必須**嚴格高於所有合法鍵值**，而且不是可輸出的真資料。

不能直接把最大合法整數當作哨兵。若合法鍵包含 $+\infty$，取 $L = [+\infty]$、$R = [+\infty, +\infty]$，再加相同值的哨兵：等值先取左可能依序取到左邊真資料、左哨兵，第三次便讀超過 $L$。

本章的長度檢查版適用於任意全序鍵值域。另一個安全做法是使用帶標籤的哨兵，讓它在比較時嚴格高於全部真資料；仍須保證哨兵不會被輸出成答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從局部合併推到整個排序

對區間長度 $m$ 作強歸納。$m \le 1$ 時已排序且不需修改。$m \ge 2$ 時，兩個子區間都較短；歸納假設保證兩半各自穩定排序，並保留所有原資料。

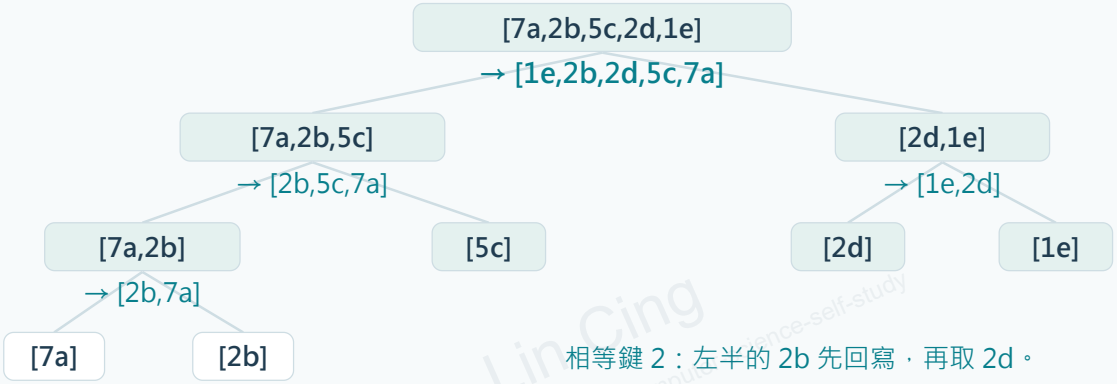
於是 MERGE 的前提成立。合併證明使整段穩定排序且不遺失、不重複資料；區間外未改動，完成目前呼叫的規格。子問題長度嚴格縮小，另保證終止。

此論證不要求 $n$ 是二冪。例如 $[7a, 2b, 5c, 2d, 1e]$ 先分成長 3、2，再分成 2/1 與 1/1。根節點合併遇到相等鍵 2 時，仍先取左邊的 2b，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從局部合併推到整個排序：圖解

往下分割；每個箭頭旁列出該子呼叫的排序結果




相等鍵 2：左半的 2b 先回寫，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 遞迴式的成本不能漏掉合併

對 $m \geq 2$，計算中點為 $\Theta(1)$；兩個子呼叫依序執行，成本相加。MERGE 複製 $m$ 筆並回寫 $m$ 筆，為 $\Theta(m)$，故

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

先看二冪 $m = 2^h$ 上的精確代理式 $U(1) = d$、$U(m) = 2U(m/2) + cm$，固定 $c, d > 0$。深度 $\ell < h$ 有 $2^\ell$ 個內部節點，每個局部成本 $cm/2^\ell$，所以每層總成本 $cm$。

共有 $h$ 個內部層，最底 $m$ 個葉各花 $d$，得到

$$U(m) = cm \lg m + dm.$$

真成本不必恰為同一個 $c$：把各節點的固定上下界分別代入，即得 $\Theta(m \lg m)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 任意長度的界與一個六筆成本表

任意 $m \geq 2$ 平衡拆分後，深度 $\ell$ 的區間長度不超過 $\lceil m/2^{\ell} \rceil$，所以樹高至多 $\lceil \lg m \rceil$。同層區間互不重疊、總長至多 $m$，給出 $O(m \lg m)$ 上界。

若 $\ell < \lfloor \lg m \rfloor$，每段至少有 $\lfloor m/2^{\ell} \rfloor \geq 2$ 筆，尚無葉；這些完整層的總長恰為 $m$。又 $\lfloor \lg m \rfloor \geq \frac{1}{2} \lg m$，故內部工作為 $\Omega(m \lg m)$。另有 $m$ 個單元素基例。

| $m = 6$ 的深度 | 區間長度 | 仍需合併的長度總和 |
|---|---|---|
| 0 | 6 | 6 |
| 1 | 3,3 | 6 |
| 2 | 2,1,2,1 | 4 |
| 3 | 1,1,1,1（另兩葉已停止） | 0 |

本例複製到暫存陣列與回寫各 16 筆，基例共 6 次。不能把每一層都算成 6 筆。



## 峰值空間不是所有配置次數的總和

本版 MERGE 等兩個子呼叫返回後，才建立總長 $m$ 的 $L, R$ 暫存陣列；完成合併就可釋放。因此子呼叫的暫存陣列不會與父層 MERGE 的暫存陣列同時存活。

根層需要 $n$ 筆暫存資料，其他層不會更大，所以暫存陣列峰值為 $\Theta(n)$。另有 $\Theta(\lg n)$ 個呼叫框架，合計仍為 $\Theta(n)$，這裡取 $n \geq 2$。

全程累計配置量可達 $\Theta(n \lg n)$，但那不是同時需要的記憶體。若保留已完成呼叫的暫存陣列不釋放，就改變了生命期模型；也可預先配置 $n$ 筆 scratch 空間重複使用，但仍須維持未讀取資料不被覆寫的前提。

比較第 1 章：插入排序用常數個額外機器字；此合併排序需要線性額外空間。兩者都可穩定排序，前提分別是嚴格大於才右移、等值時先取左。

## 代入法：把猜測證成同一個界

代入法先猜候選界，再用強歸納找出同一組固定常數，使較小輸入的假設推出目前的結論。

令 $T(1)=1$、$T(n)=2T(\lfloor n/2\rfloor)+n$，$n\ge 2$。猜 $T(n)\le cn\lg n$。對 $n\ge 4$，令 $m=\lfloor n/2\rfloor\ge 2$，則

$$\begin{aligned}T(n)&\le 2cm\lg m+n\\&\le cn\lg(n/2)+n\\&=cn\lg n-(c-1)n\le cn\lg n\end{aligned}$$

只需固定 $c\ge 1$。但 $n=1$ 的候選右側為 0，不能作歸納基礎。改用遞迴算出的 $T(2)=4, T(3)=5$；固定 $c=2$ 同時覆蓋兩者，而 $n\ge 4$ 的子引數至少為 2，歸納閉合。

遞迴的基例與證明使用的歸納基礎可以不同，但到達小輸入時的依據都必須交代。



## 同一常數的下界與強化假設

先限二冪 $n$，令 $T(1)=1, T(n)=2T(n/2)+n$。若子問題滿足 $T(n/2)\ge d(n/2)\lg(n/2)$，則

$$T(n)\ge dn\lg n+(1-d)n\ge dn\lg n,\qquad 0<d\le 1.$$

基例右側為 0、左側為 1，成立。對任意整數的原式 $T(n)=2T(\lfloor n/2\rfloor)+n$，先用歸納證單調：$T(2)\ge T(1)$；其餘相鄰輸入的子引數不下降且較小，故 $T(n+1)\ge T(n)$。令 $m=2^{\lfloor\lg n\rfloor}$，則 $n/2<m\le n$，由單調性與二冪下界得 $T(n)\ge T(m)\ge dm\lg m=\Omega(n\lg n)$。這裡比較的是同一遞迴式，並非前述 floor／ceil 分割。

再看 $T(n)=T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+1$，$T(1)=1$。猜 $T(n)\le cn$ 只得 $cn+1$，無法閉合；改猜

$$T(n)\le cn-d.$$

代入得到 $T(n)\le cn-2d+1\le cn-d$，只要 $d\ge 1$。取 $c=2, d=1$，基例也成立，並可歸納得到精確解 $2n-1$。

子問題中保留的低階扣項，正好用來支付父節點的新工作。「$cn+n$ 仍是 $O(n)$」卻沒有證出原先同一個 $cn$，不能作為歸納步驟。

## 換變數與真正縮小的條件

先在 $n = 2^{2^k}$ 上考慮 $T(n) = 2T(\sqrt{n}) + \lg n$、$T(2) = \Theta(1)$。令 $m = \lg n$、$S(m) = T(2^m)$，則

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

還原得 $T(n) = \Theta(\lg n \lg \lg n)$；不是把新變數 $m$ 又當成原輸入 $n$。接下來另證帶 floor 的一般整數版本。

固定平移也要檢查是否縮小。例如 $T(n) = 2T(n/2 + 17) + n$，只有 $n > 34$ 時子引數小於 $n$。令 $n = 34 + x$、$U(x) = T(34 + x)$，得到

$$U(x) = 2U(x/2) + x + 34.$$

採足夠大的固定區間為基例，幾何分層求和得 $\Theta(x \lg x) = \Theta(n \lg n)$。這裡先處理實數引數；整數版還須控制取整誤差。若只給 $T(1)$，$n = 34$ 會自我呼叫，遞迴尚未定義完成。



## 平方根裡的 floor，如何補上一般整數證明？

定義 $T(n) = 1$ 於整數 $1 \le n \le 3$，$n \ge 4$ 時

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

對 $n \ge 4$，$2 \le \lfloor \sqrt{n} \rfloor < n$，所以遞迴有定義且會停止。

先取 $a_k = 2^{2^k}$、$U_k = T(a_k)$。因 $\sqrt{a_k} = a_{k-1}$，這些點沒有取整誤差；$U_0 = T(2) = 1$，且 $U_k = 2U_{k-1} + 2^k$。

除以 $2^k$，得到 $U_k/2^k = U_{k-1}/2^{k-1} + 1$。累加得 $U_k = (k+1)2^k$，也就是這些特殊點上 $T(n) = \lg n(1 + \lg \lg n)$。

這還只涵蓋特殊點，不能到此就宣稱所有整數的界成立；下一步要補單調性與包夾。

## 單調性先證明，再用相鄰特殊點包夾

$T(1)=T(2)=T(3)=1<T(4)$。假設較小引數的相鄰值已非遞減；由 $\lfloor\sqrt{n+1}\rfloor\ge\lfloor\sqrt{n}\rfloor$、兩個子引數均小於 $n$，以及 $\lg(n+1)>\lg n$，得到 $T(n+1)\ge T(n)$。強歸納證明單調性。

選 $k$ 使 $a_k\le n<a_{k+1}$，等價於 $2^k\le\lg n<2^{k+1}$。由單調性，

$$(k+1)2^k\le T(n)\le(k+2)2^{k+1}.$$

此區間內 $\lg n$ 與 $2^k$ 在兩倍內，$1+\lg\lg n$ 與 $k+1$ 也在固定倍數內，因此

$$T(n)=\Theta\big(\lg n(1+\lg\lg n)\big),\qquad n\ge 2.$$

對充分大的 $n$ 可寫成 $\Theta(\lg n\lg\lg n)$。這個包夾使用已證的單調性；不是把 floor 的誤差憑直覺丟掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 遞迴樹：每個節點只記當次工作

遞迴樹每個內部節點只記本次局部成本 $f(\text{size})$；子呼叫由子樹另計。若節點已記完整 $T(\text{size})$，再加子樹就重複計費。

取 $T(n)=3T(n/4)+n^2$、$T(1)=1$，先限 $n=4^L$。深度 $j$ 有 $3^j$ 個節點，每個局部成本 $(n/4^j)^2$，層成本為 $n^2(3/16)^j$。葉數為 $3^L=n^{\log_4 3}$，故

$$T(n)=n^2\sum_{j=0}^{L-1}(3/16)^j+n^{\log_4 3}.$$

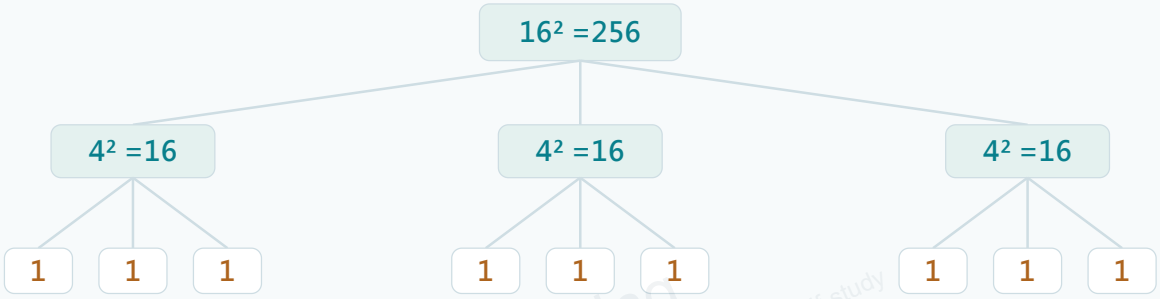
等比和至多 $16/13$，葉成本也不超過 $n^2$，給出上界 $O(n^2)$；根的局部工作給出下界 $\Omega(n^2)$。

$n=16$ 的各層成本為 256、48、9，總計 313。最後的 9 已是九個基例的成本，不能再加九次 $f(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 遞迴樹：每個節點只記當次工作：圖解

T(n)=3T(n/4)+n²，T(1)=1；n=16




層成本：256 → 48 → 9（九個基例）

總成本 313；葉不再另外加上 f(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每層縮成七分之八，還要把葉成本算進去

取整數 $n \geq 8$，

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

並令 $0 \leq n \leq 7$ 時 $T(n) = 1$。內部節點的子問題總長至多 $7n/8$，所以深度 $j$ 的內部成本至多 $n(7/8)^j$，合計至多 $8n$。

若內部節點數為 $I$、葉數為 $L$，每個內部節點都有三個子節點，數邊得 $3I = I + L - 1$，故 $L = 2I + 1$。每個內部節點至少有成本 8，因而 $8I \leq 8n$，給 $L \leq 2n + 1$。

所以 $T(n) \leq 8n + (2n + 1) = O(n)$；根本身提供 $T(n) \geq n$，得到 $\Theta(n)$。**無窮等比和先支付內部工作，葉成本由節點計數另外支付**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 不等分割：先界深度，再界每層質量

令 $a=\lfloor n/3\rfloor$、$b=n-a$，且 $T(n)=T(a)+T(b)+cn$，固定 $c>0$；小輸入的成本有固定正上下界。

沿任一路徑，第 $j$ 層的引數 $q_j$ 滿足

$$\frac{n}{3^j}-\frac{3}{2}\le q_j\le n(2/3)^j+3.$$

這由每步 $q/3-1\le q_{\text{next}}\le 2q/3+1$ 及等比誤差和歸納得到。把大小至多固定 $K\ge 6$ 的整段底層運算視為基例，不改漸近階。右側界給高度 $O(\lg n)$；同層子問題互不重疊，總長度至多 $n$，工作至多 $cn$。葉的正整數長度總和是 $n$，所以至多 $n$ 個葉，合計上界 $O(n\lg n)$。

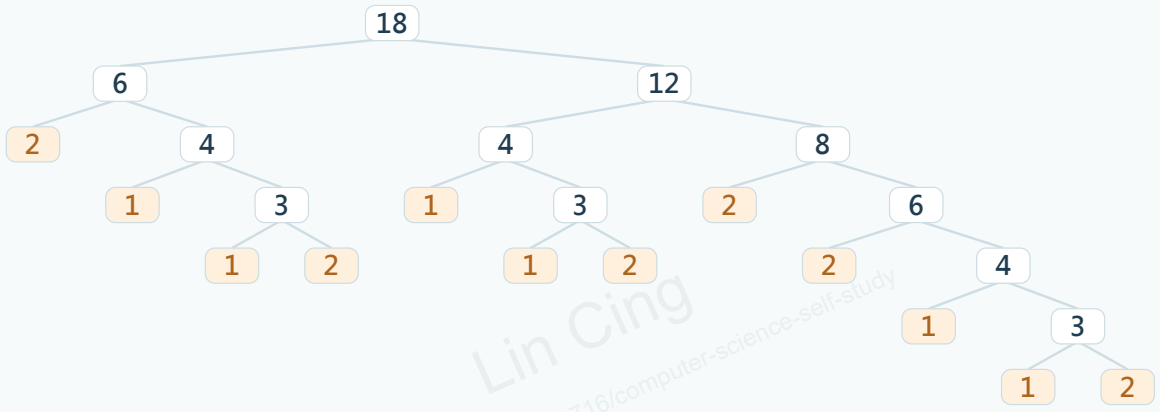
若 $0\le j\le\lfloor\log_3(n/(K+2))\rfloor$，左側界使每個 $q_j>K$，尚無路徑停止。這些完整層都保有總長度 $n$，各成本 $cn$，層數為 $\Omega(\lg n)$，所以有相同下界。

若只知局部工作為 $O(n)$，則仍有上界，卻不能保證下界；零合併成本便可能只剩線性葉成本。



## 不等分割：完整呼叫樹的葉不在同一層

n=18；白底繼續分割，橘底為 n≤2 的基例

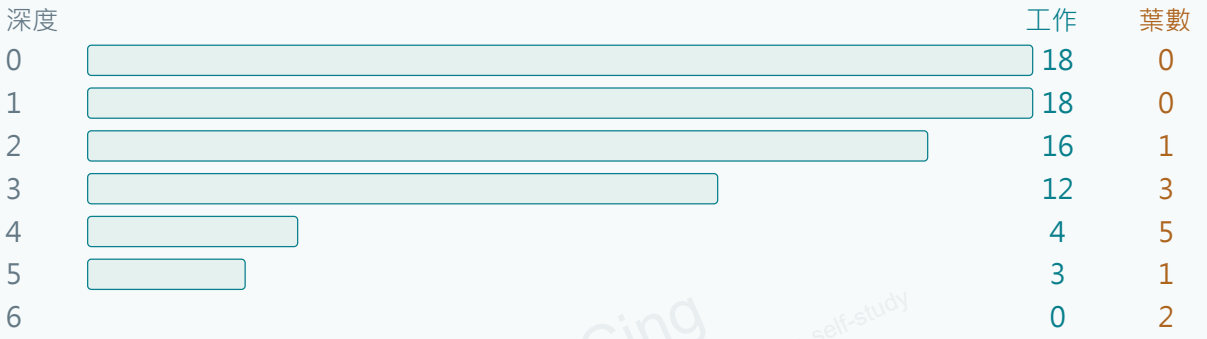



共 11 個內部節點、12 個葉；葉在深度 2 到 6 之間。

每條邊是一次實際子呼叫。橘色葉在不同深度停止；最長路徑為18 →12 →8 →6 →4 →3 →2。

## 不等分割：完整層與提早停止要分清

n=18；n≤2 停止；條長表示該層內部節點的總量




葉節點成本另計；到達基例後不會繼續產生下一層。

本圖固定 $n \le 2$ 為基例，其餘拆成 $\lfloor n/3 \rfloor$ 與 $n - \lfloor n/3 \rfloor$。$n = 18$ 的完整層總量起初都是 18；出現葉之後，**只有仍繼續分割的節點**才支付合併工作。

最短路徑決定下界能保留幾層；最長路徑決定上界要涵蓋幾層。不能用最深路徑的高度，乘上每一層都等於 $n$ 的假想成本來證下界。



## 先看每層工作，再看三種主導方式：圖解

各欄採獨立比例；標籤為每層總成本

## 先看每層工作，再看三種主導方式

三例都取 $n = 16$、$a = b = 2$、$T(1) = 1$。非遞迴項依序為 $1, n, n^2$，各層（含葉）的成本為：

| $f(n)$ | 深度 0、1、2、3、4 | 總成本 |
|---|---|---|
| $1$ | 1、2、4、8、16 | 31 |
| $n$ | 16、16、16、16、16 | 80 |
| $n^2$ | 256、128、64、32、16 | 496 |

這三個數例分別呈現葉端、等高、根端主導；一般定理還需要固定參數、多項式差距與情形 3 的正則性，不能只靠圖的外觀判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 主定理：先核對前提

令固定 $a \geq 1, b > 1$，局部成本 $f \geq 0$，常數大小基例的成本有固定正上下界。考慮

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

先在精確冪上使用下表；取整的延伸條件在後面說明。若 $a$ 是實際呼叫數，它必須為整數；實數 $a$ 可解讀為遞迴權重。

| 情形 | 需要核對的前提 | 結論 |
|---|---|---|
| 1 | 固定 $\epsilon > 0$，$f(n) = O(n^{p-\epsilon})$ | $\Theta(n^p)$ |
| 2 | $f(n) = \Theta(n^p)$ | $\Theta(n^p \lg n)$ |
| 3 | 固定 $\epsilon > 0$，$f(n) = \Omega(n^{p+\epsilon})$；且充分大 $n$ 有 $af(n/b) \leq cf(n)$，固定 $c < 1$ | $\Theta(f(n))$ |

只說「比 $n^p$ 小／大」並不夠：情形 1、3 要有固定的多項式差距，情形 3 還要檢查正則性（regularity）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一張表的完整判定數例

每一列都使用固定正基例。表中的 $p$ 是 $\log_b a$，不是從圖上猜出的指數。

| $(a, b, f(n))$ | 前提的具體核對 | 緊界 |
|---|---|---|
| $(9, 3, n)$ | $p = 2$，取 $\epsilon = 1$ | $\Theta(n^2)$ |
| $(1, 3/2, 1)$ | $p = 0$，$f(n) = \Theta(n^0)$ | $\Theta(\lg n)$ |
| $(3, 4, n \lg n)$ | $p \approx 0.7925$；取 $\epsilon = 0.1$，正則性比值至多 $3/4$ | $\Theta(n \lg n)$ |
| $(8, 2, n^2)$ | $p = 3$，取 $\epsilon = 1$ | $\Theta(n^3)$ |
| $(7, 2, n^2)$ | $p = \lg 7$，取 $\epsilon = \lg 7 - 2 > 0$ | $\Theta(n^{\lg 7})$ |

第三列的正則性由

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4)$$

直接得到。相較之下，$2T(n/2) + n \lg n$ 的對數因子沒有固定多項式差距，不能直接套經典三情形；下一個層和仍能解它。



## 對數邊界必須真的求和

令 $n = b^L, p = \log_b a$，且 $f(n) = \Theta(n^p (\lg n)^k)$，固定實數 $k$。在固定大小的基例以上，把深度 $j$ 改編成離葉端的距離 $r = L - j$，層成本為 $\Theta(n^p r^k)$。

所以總成本是 $\Theta(n^p)$ 的葉成本，加上 $n^p$ 乘一個冪次和：

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

當 $k > -1$，積分比較給 $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$；$k = -1$ 是調和和；$k < -1$ 時和收斂到正常數。若 $k < 0$，須在大於 $1$ 的固定基例之外使用公式，避免對 $\lg 1$ 取負次方。

因此 $2T(n/2) + n/\lg n$ 為 $\Theta(n \lg \lg n)$；換成 $n/(\lg n)^2$ 則為 $\Theta(n)$。這是層和的結論，不是經典三情形中的情形 $1$。

## 正則性缺失的明確反例

只看 $n = 2^L$，令 $T(1) = 1$，且

$$f(n) = \begin{cases} n^2 & L \text{ 為偶數}, \\ n^3 & L \text{ 為奇數}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

每個輸入都有 $f(n) \geq n^2$，對 $a = b = 2, p = 1$ 已有固定多項式差距。但在偶數 $L \geq 2$ 上，子問題的指數 $L - 1$ 是奇數，故

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

所以 $T(n)$ 不屬於 $\Theta(f(n))$。同時計算得 $2f(n/2)/f(n) = n/4$，不存在共同的 $c < 1$ 讓正則性最終成立。

這個例子說明缺少前提會怎樣，沒有推翻主定理。對振盪函數也不能因引數只差一個常數，就假設函數值接近。



## 主定理為什麼成立

先令 $n = b^L, T(1) = t > 0$。展開 $L$ 次可得

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

第 $j$ 層總權重是 $a^j$；$a$ 為整數呼叫數時，它也是節點數。葉項 $tn^p$ 在三種情形中都提供正下界。

情形 1：充分大 $x$ 有 $f(x) \leq Cx^{p-\epsilon}$，因此大規模層的總和至多

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

因 $b^\epsilon > 1$，這是往葉端增大的幾何和。上式可補上任意有限個小引數項來作上界；這些底層的總權重為 $O(n^p)$，固定函數值可吸收到常數。

加上葉成本得到 $O(n^p)$，再用 $tn^p$ 的下界，證得 $\Theta(n^p)$。

## 主定理等高與根主導的兩個證明

**情形 2。** 因 $a = b^p$，充分大引數的每層總成本為

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

有 $L - O(1) = \Theta(\lg n)$ 個這樣的層；將上下界分別相加，得到 $\Theta(n^p \lg n)$，也涵蓋葉成本。

**情形 3。** 正則性可以逐層使用：只要引數仍充分大，就有

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

這些層合計不超過 $f(n)/(1-c)$。剩餘固定大小層與葉的總成本為 $O(n^p)$；因 $f(n) = \Omega(n^{p+\epsilon})$，也屬於 $O(f(n))$。根的局部工作 $f(n)$ 給出下界，合成 $\Theta(f(n))$。

固定小輸入不是整棵樹的常數成本：可能有 $\Theta(n^p)$ 個小節點，必須像這裡一樣計入底層總量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 取整的誤差不會逐層線性累積

沿任一實際呼叫路徑，令 $q_0 = n$，$q_{j+1}$ 為 $\lfloor q_j/b \rfloor$ 或 $\lceil q_j/b \rceil$。兩者都有 $|q_{j+1} - q_j/b| \le 1$。取固定 $C \ge b/(b-1)$，歸納得

$$|q_j - n/b^j| \le C.$$

因下一步誤差至多 $C/b + 1 \le C$，取整誤差不會每層累加成 $j$。

選固定基例範圍 $q \le n_0$，其中 $n_0 > C + 2$，且足夠大使子問題嚴格縮小。上、下誤差界使各路徑進入基例的深度都為 $\log_b n + O(1)$。在 $n/b^j \ge 2C$ 的大規模層，實際 $q_j$ 與 $n/b^j$ 在固定倍數內。

實際有整數 $a$ 個子呼叫時，葉深度只差常數，故葉數為 $\Theta(a^{\log_b n}) = \Theta(n^p)$；固定個底層的總成本為 $O(n^p)$。$a = 1$ 時是一條路，葉與固定個底層的成本都是常數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 從精確冪推到所有整數的成本條件

以下處理整數 $a$ 個實際子呼叫；精確冪上的實數權重版本已由前面的展開式處理。

**情形 1、2：** 大規模層的 $q_j$ 與 $n/b^j$ 在固定倍數內。固定次方即使為負，仍有

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

因此情形 1 可沿用多項式上界與幾何和；情形 2 的每個完整大規模層仍為 $\Theta(n^p)$，層數為 $\Theta(\lg n)$。固定個不完整底層另計 $O(n^p)$。對數擴充在遠離固定基例的層，$\log q_j$ 與離底層深度同階，也可沿用冪次和。

**情形 3：** 正則性必須涵蓋實際取整後的引數。充分條件是

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le c f(x), \qquad c < 1.$$

各層總成本才會以 $c$ 倍收縮；底層仍由多項式差距吸收。對任意 $f$，只證引數相差常數並不足以省略此條件。



## 正則性也能推出多項式分離

在精確冪上，若 $af(x/b) \le cf(x)$、固定 $0 < c < 1$，且某個固定底層點 $f(x_0) > 0$，向上反推 $k$ 層可得

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

因 $\log_b(a/c) = p + \log_b(1/c)$，令 $\epsilon = \log_b(1/c) > 0$，便有沿這條精確冪序列的 $f(n) = \Omega(n^{p+\epsilon})$。

這解釋為什麼有些版本在正且良好的底層條件下，不另外列出多項式差距。本章保留三情形的完整假設，方便逐項核對。

此推導的域是 $n = b^k x_0$；推到全部實數或整數，還需控制最後進入的固定區間，以及該區間上 $f$ 的正下界。不能只證一條序列，就宣稱所有輸入都成立。

## 最大非空連續子陣列

給定 $n \geq 1$ 個有限實數 $A[1..n]$，求非空、連續區間 $[i, j]$，使

$$\sum_{k=i}^{j} A[k]$$

最大，回傳 $(i, j, \text{sum})$。相同最大值可以有多個答案；此規格允許任選其中一個。先以精確加法與比較的操作數分析；浮點誤差、NaN 與溢位不是這個模型的合法輸入行為。

若每天價格為 $P_0, \ldots, P_n$，令 $A_i = P_i - P_{i-1}$，便有 $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$。區間 $i..j$ 對應第 $i-1$ 天買、第 $j$ 天賣。

若允許不交易，才可加入空區間、收益 0；全負輸入時答案會改變。逐一起點、延長終點並累加，可在 $\Theta(n^2)$ 時間與 $O(1)$ 額外空間枚舉；每個區間若重新加總，才變成 $\Theta(n^3)$。



## 跨中線必須由 後綴 與 前綴 組成

對 $low < high$，令 $mid = low + \lfloor (high - low)/2 \rfloor$。任一非空區間恰屬三類：

1. 全在 $low..mid$。
2. 全在 $mid + 1..high$。
3. 跨過中線，滿足 $i \leq mid < j$。

第三類必包含 $mid$ 與 $mid + 1$，可唯一拆成左半的後綴（suffix）$i..mid$ 與右半的前綴（prefix）$mid + 1..j$。

兩側端點可獨立選擇，故最佳跨線和等於**最佳非空左後綴和＋最佳非空右前綴和**。分別掃描兩側即可，不必枚舉全部端點配對。左右兩半各自的最佳子陣列則未必接到中線，不能直接相加。

## 跨中線必須由 後綴 與 前綴 組成：圖解

mid=5：跨線區間必須包含第 5 與第 6 格

左側向外累加：−1, 3, 0, 1, −1 → 最大 3，起點 4

右側向外累加：2, 3, −2, 2 → 最大 3，終點 7

**組合為 A[4..7]=[4,−1,2,1]，總和 6。**



## 跨線掃描的完整程序

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

每次左迴圈更新後，`total` 恰是 $A[i..mid]$ 的和；`left` 是已掃過全部左後綴的最大和。初值 $-\infty$ 是比較標記，不是輸入資料；第一個有限候選一定更新它，並設定起點。

每輪新增恰一個候選、取最大，保持不變量；結束時左後綴已全部納入。右邊的前綴同理，兩者相加得到最佳跨線和。掃描總次數為

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

## 最大子陣列的遞迴與一般證明

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

對區間長度作強歸納。單元素只有自身；較長時兩個子區間均非空且較短，子呼叫分別給前兩類的最大值，CROSS 給第三類最大值。三類互斥且完整，取三者最大即正確；長度嚴格縮小也保證終止。

因此操作成本滿足

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

依序執行子呼叫、只傳索引時，每層僅保存常數個索引與和，堆疊峰值為 $O(\lg n)$。若額外複製子陣列，必須另算配置與存活空間。



## 九元素數例：每次跨線比較都可核對

對 $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$，取 $low = 1, high = 9, mid = 5$。

| 左索引 $i$ | $A[i..5]$ 累計和 | 已知最佳和、起點 |
|---|---|---|
| 5 | $-1$ | $-1$，5 |
| 4 | 3 | 3，4 |
| 3 | 0 | 3，4 |
| 2 | 1 | 3，4 |
| 1 | $-1$ | 3，4 |

右邊 $j = 6, 7, 8, 9$ 的前綴和是 $2, 3, -2, 2$，最佳為 $6..7$、和 3。跨線解為 $4..7$、和 6；左半最佳 $4..4$、和 4，右半最佳 $9..9$、和 4，所以本次回傳 $(4, 7, 6)$。

對全負輸入 $[-8, -2, -5]$，非空規格回傳 $(2, 2, -2)$，不是 0。

## 線性方法的狀態與不變量

令 $E_i$ 為必須以 $i$ 結尾的最佳非空區間和，$B_i$ 為 $1..i$ 中最佳非空區間和。則

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

以 $i$ 結尾的區間不是只有 $A_i$，就是接在以 $i-1$ 結尾的區間後；要延伸時，選先前最佳便足夠。這個分類與基例給出兩個狀態的歸納證明。

若選單元素 $A_i$，起點重設為 $i$；若選延伸，保留舊起點。全域最佳改善時保存該起點與終點 $i$。每輪固定工作，故時間 $\Theta(n)$、額外空間 $O(1)$。

九元素例的 $E_i$ 為 $[-2, 1, -2, 4, 3, 5, 6, 1, 5]$，$B_i$ 為 $[-2, 1, 1, 4, 4, 5, 6, 6, 6]$。分治法不是這題最快的解法；它提供的是另一個清楚的分類與組合證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從純量乘積到區塊矩陣乘法

標準矩陣乘法逐一計算

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

內迴圈在處理第 $k$ 項後，累積值恰為前 $k$ 個乘積之和。$n^2$ 個輸出各做 $n$ 次乘加，算術操作數為 $\Theta(n^3)$；這沒有證明所有矩陣乘法都必須是立方成本。

對偶數階矩陣，寫成區塊

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

每個字母是 $n/2$ 階矩陣，乘法給出

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

這裡保持乘法次序；矩陣區塊一般不交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 八次遞迴乘法的完整成本

先補零到二冪階數，最後擷取原尺寸結果。每次呼叫從自己的矩陣 view 讀出階數，不能一直沿用原始階數。

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

索引 view 只保存起點、stride 與階數，可用常數時間分割。即使真的複製區塊，其 $\Theta(n^2)$ 成本也與固定個區塊加法同階，因此 $T(n) = 8T(n/2) + \Theta(n^2)$，基例為 $\Theta(1)$。

不能把八個子呼叫吸收到一個子呼叫的常數裡。第 $j$ 層有 $8^j$ 次呼叫，葉數是 $8^{\lg n} = n^3$；分支數會改變整棵樹的成本指數。



## Strassen 的七個乘積全部列出

以下各乘積由遞迴 STRASSEN 計算；括號內的加減是逐元素的區塊運算。

$$
\begin{aligned}
M_1 &= (a + d)(e + h), & M_2 &= (c + d)e, \\
M_3 &= a(f - h), & M_4 &= d(g - e), \\
M_5 &= (a + b)h, & M_6 &= (c - a)(e + f), \\
M_7 &= (b - d)(g + h).
\end{aligned}
$$

再組合四個結果：

$$
\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\
C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6.
\end{aligned}
$$

$n = 1$ 時直接做純量乘法；$n > 1$ 時依此分割、計算七個乘積，再組成四塊。這些步驟與基例一起定義完整遞迴程序。

## 四個輸出區塊為何正確

只用分配律與加法消去，保持矩陣乘法的因子順序：

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

基例的純量乘法正確。若七個較小乘積均由歸納假設正確算出，上述恆等式便組成 $AB$ 的全部四塊，完成歸納證明。

這裡需要加法、減法與分配律。只有加法、沒有減法的一般半環（semiring），不能直接套用這七式。



## 七乘積的數值與四塊回收

取 $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$、$B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$。

| 乘積 | 代入 | 值 |
|---|---|---|
| $M_1$ | $(2+4)(5+1)$ | 36 |
| $M_2$ | $(3+4)5$ | 35 |
| $M_3$ | $2(2-1)$ | 2 |
| $M_4$ | $4(-2-5)$ | $-28$ |
| $M_5$ | $(2-1)1$ | 1 |
| $M_6$ | $(3-2)(5+2)$ | 7 |
| $M_7$ | $(-1-4)(-2+1)$ | 5 |

四塊依序為 $36 - 28 - 1 + 5 = 12$、$2 + 1 = 3$、$35 - 28 = 7$、$36 - 35 + 2 + 7 = 10$。直接乘法核對：

$$AB = \begin{bmatrix} 10 + 2 & 4 - 1 \\ 15 - 8 & 6 + 4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

## 七乘積的數值與四塊回收：圖解

數例的四塊位置：C11、C12、C21、C22 不可互換

| A | | | B | | | C | |
|---|---|---|---|---|---|---|---|
| 2 | -1 | × | 5 | 2 | = | 12 | 3 |
| 3 | 4 | | -2 | 1 | | 7 | 10 |

C11: 36−28−1+5=12

C12: 2+1=3

C21: 35−28=7

C22: 36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Strassen 的時間、補零與記憶體

Strassen 的七個子乘積需要 10 次輸入區塊加減，組合四個結果還需 8 次。view 分割為 $O(1)$，所以

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

加減的常數較大，小矩陣未必較快；可在固定門檻改用標準乘法，不改漸近指數。代數恆等式的正確性也不等於浮點誤差保證。

任意 $n \geq 1$ 可補零到 $m = 2^{\lceil \lg n \rceil}$，$n \leq m < 2n$。左上 $n \times n$ 結果仍是原乘積，且 $m^{\lg 7} < 7n^{\lg 7}$，故同階。空矩陣直接返回，不遞迴。

依序完成七個子呼叫、適時釋放暫存時，每層只保留固定個當層區塊，峰值空間滿足

$$S(n) \leq S(n/2) + O(n^2) = O(n^2).$$

另有 $O(\lg n)$ 個框架。若同時保留所有子問題工作區，就必須重建空間遞迴式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 延伸工具：Akra–Bazzi 的適用條件

這裡引用 MIT *Mathematics for Computer Science*（2010）§10.4.3 的定理，採用正且有界的基例作充分條件；**不把下列應用當成定理的完整證明**。

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- $k$ 有限固定，$a_i > 0$、$0 < b_i < 1$ 固定。
- $0 \le x \le x_0$ 時，固定常數 $0 < d_1 \le T(x) \le d_2$。
- $g \ge 0$ 可微，$|g'(x)|$ 至多以多項式成長；$|h_i(x)| = O(x/\log^2 x)$。
- $x_0$ 足夠大，所有子引數都在 $[0, \rho x]$，固定 $\rho < 1$，使遞迴有定義且會停止。

上述條件是本章使用的版本。若基例與 $g$ 全為零，成本恆零，就不能得到帶正葉成本的結論。



## 先求唯一指數，再做積分

令 $p$ 滿足 $\sum_i a_i b_i^p = 1$，則引用定理給出

$$T(x) = \Theta\left( x^p \left[ 1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du \right] \right).$$

因 $0 < b_i < 1$，函數 $F(p) = \sum_i a_i b_i^p$ 連續且嚴格遞減；當 $p \to -\infty$ 時趨向無窮大，$p \to +\infty$ 時趨向 0，所以恰有一個實根。

對 $T(x) = T(x/3) + T(2x/3) + x$，取固定小輸入為正常數，比例方程為 $(1/3)^p + (2/3)^p = 1$，故 $p = 1$。代入得

$$T(x) = \Theta\left( x \left[ 1 + \int_1^x \frac{du}{u} \right] \right) = \Theta(x \log x).$$

這和前面用完整層與最深層得到的兩側界一致；兩種推導的適用前提都要保留。

## 取整與比例改變，是兩種不同的事

floor、ceiling 與固定平移的誤差至多為常數，符合 $O(x/\log^2 x)$；仍須選夠大的基例範圍，確保子問題縮小。

但 $T(n) = T(n/2 + n/4) + T(n/2) + n$ 的第一個子問題其實是 $3n/4$。$n/4$ 不符合小擾動條件；不能沿用兩個 $1/2$ 比例。

正確方程為 $(3/4)^p + (1/2)^p = 1$，唯一根 $p \approx 1.507126591639 > 1$。由

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

得到 $T(n) = \Theta(n^p)$。嚴格指數由方程定義，小數只是近似值。

若其他教材把子問題寫成 $n/c_i$，則本章 $b_i = 1/c_i$。轉換表示法時，方程也必須一起換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 延伸閱讀與定理來源

- Cormen、Leiserson、Rivest、Stein，*Introduction to Algorithms*, 3rd Edition：Chapter 2（合併排序）、Chapter 4（分治、代入法、遞迴樹、主定理及其證明）。
- Demaine、Leiserson，MIT 6.046J（Fall 2005），Lecture 2–3：遞迴式，以及二分搜尋、快速冪、Fibonacci 與矩陣乘法的連結。
- Lehman、Leighton、Meyer，*Mathematics for Computer Science*（2010），§ 10.4.3，Theorem 10.4.1：MIT 官方講義。本章使用正且有界基例的充分條件版本。

主定理的三情形在正文給出證明；Akra–Bazzi 為引用的延伸定理，正文負責核對條件、求指數與計算積分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 排序、選擇與隨機分析

排序要決定所有資料的先後；選擇只問其中一個秩。兩者可以共用分割與堆積，卻不必付出相同成本。

本章依序學：**比較模型的限制 → 堆積與排序 → 隨機分析工具 → 分割與快速排序 → 計數、基數與桶排序 → 次序統計量**。最後用生日問題、負載、等待與線上遴選練習機率建模。

先備知識：第 1 章的規格、不變量、級數，第 2 章的分治與遞迴式。正文先補齊本章需要的機率工具，再使用期望成本。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 5–9；補充參考 MIT 6.046J（2005）Lectures 4–6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 先定義排序的對象與保證

每筆資料由 **key（鍵值）** 與隨它搬動的欄位組成。另加身分標籤 $a, b, c$，方便辨識相同鍵的不同資料；除非特別說明，比較只看 key。

正確排序必須同時保證：輸出鍵值非遞減，而且輸出是輸入資料的同一個多重集合。**穩定性**另要求同鍵資料維持原先順序。

例輸入 $[2_a, 1_b, 2_c]$：$[1_b, 2_a, 2_c]$ 穩定；$[1_b, 2_c, 2_a]$ 排序正確但不穩定；$[1_b, 2_a, 2_a]$ 遺失 $2_c$，連排序規格都不符合。

本章陣列採 **1 起算、閉區間** $A[p..r]$，$p > r$ 表示空區間；鍵值計數表則依鍵值從 $0$ 起算。選擇的 $k$ 永遠是目前子區間內、1 起算的局部秩。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 比較數、執行時間與空間各自計算

預設比較與搬移一筆固定大小資料、讀寫一個索引均為常數成本。若鍵是長字串、大整數或外部儲存物件，必須把實際比較、取位與資料搬移成本加回來。

| 要回答的問題 | 必須固定的條件 |
| --- | --- |
| 最壞時間 $\max_A T(A)$ | 長度與合法輸入集合 |
| 平均時間 $\mathbb{E}_A[T(A)]$ | 輸入的聯合分布 |
| 隨機演算法 $\mathbb{E}_R[T(A,R)]$ | 固定輸入 $A$，對內部亂數 $R$ 平均 |
| 額外空間 | 輸出陣列、暫存區及同時存活的呼叫框架是否計入 |

「分割原地完成」只說分割程序的儲存需求；快速排序仍可能累積遞迴堆疊。「期望很快」也沒有排除某條亂數路徑很慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 比較決策樹：每片葉要辨識一種次序

先限制 $n$ 個鍵互異。比較排序只能由兩鍵比較取得次序資訊，不能讀鍵值當作陣列索引；固定輸入長度後，用二元樹表示所有執行。

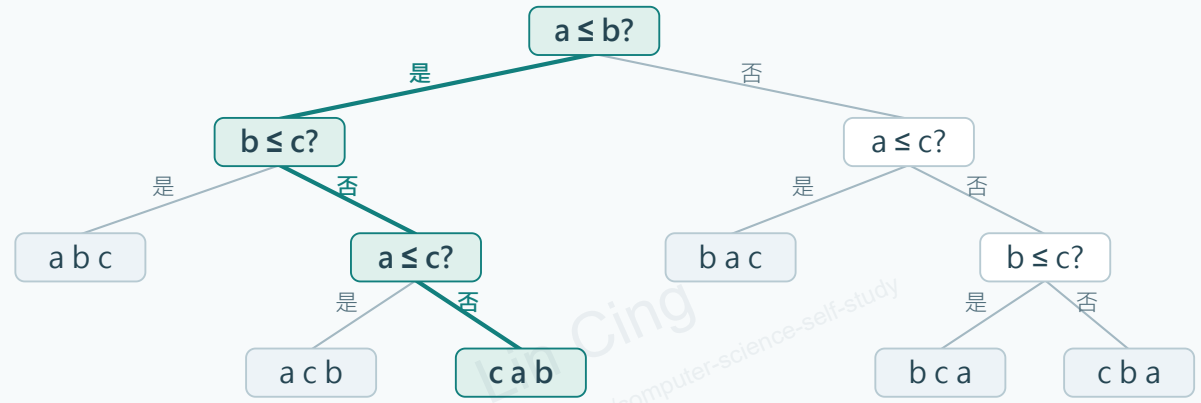
內節點標示被比較的兩個原始資料身分；左右邊分別表示 $\le$、$>$ 的結果。根到葉記錄本次觀察，葉必須決定這 $n$ 個身分的完整排序。

$n!$ 種相對次序必須導向不同的可達葉。若兩種次序到同一葉，演算法觀察完全相同，只能輸出同一個身分排列，至少有一種會錯。因此可達葉數至少 $n!$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 沿著比較結果，走到資料的次序

六種次序，六片可達葉




a=4、b=7、c=1：是 → 否 → 否 → c a b。葉上標示的是資料身分。

圖中的葉標示的是身分次序；例如 $c, a, b$ 表示 $c < a < b$。取 $a = 4, b = 7, c = 1$，依比較結果追蹤到 $c, a, b$，即可核對一條執行路徑。



## 最壞下界說的是「至少有一個輸入」

此處高度 $h$ 計算根到最深葉的邊數；若根為第 1 層，層數為 $h + 1$。高度 $h$ 的二元樹最多有 $2^h$ 片葉，所以

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

對 $n \ge 2$，階乘的後半至少有 $\lfloor n/2 \rfloor$ 個因子，各不小於 $n/2$；另一方面每個因子至多 $n$。因此

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

對充分大 $n$，兩側給出 $\lg(n!) = \Theta(n \lg n)$；小規模另以常數涵蓋。

結論：**每個正確的比較排序，都有最壞輸入需要 $\Omega(n \lg n)$ 次比較。** 這沒有宣稱每個輸入都慢；插入排序遇已排序輸入只做 $n - 1$ 次鍵比較。

允許重複鍵仍包含互異鍵這個困難子集。計數與基數排序則利用鍵值表示，已離開比較模型，必須另外計鍵範圍與位元數的成本。

## 平均下界不能只看最深的葉

令 $N = n!$ 種相對次序均勻，各選一片代表葉，深度為 $d_1, \dots, d_N$。不同葉的根路徑彼此不是前綴。把每個深度 $d_i$ 的葉對應到 $[0,1)$ 中長度 $2^{-d_i}$ 的二元區間，這些區間互不相交，故

$$\sum_{i=1}^{N} 2^{-d_i} \le 1.$$

由幾何平均不大於算術平均，

$$2^{-\frac{1}{N}\sum_i d_i} \le \frac{1}{N}\sum_i 2^{-d_i} \le \frac{1}{N}.$$

取對數得到 $\frac{1}{N}\sum_i d_i \ge \lg N = \Omega(n \lg n)$。這才是均勻輸入的平均下界；不能由最深路徑直接推到平均路徑。

對每條亂數路徑皆正確的排序，固定亂數後仍有此下界。再對亂數平均，因最大值不小於均勻輸入平均，可得 $\max_A E_r[C(A,r)] \ge \lg(n!)$；允許錯答的排序不在此論證範圍。



## 三鍵數例：區分輸入平均與亂數平均

用三鍵數例區分兩種平均。L 採前述的三鍵決策樹；R 採相同排序程序，但將原始身分順序改為 $a, c, b$。排序前等機率選 L 或 R：

| 鍵值遞增的身分次序 | L 比較數 | R 比較數 | 兩者平均 |
|---|---|---|---|
| $a, b, c$ | $2$ | $3$ | $5/2$ |
| $a, c, b$ | $3$ | $2$ | $5/2$ |
| $b, a, c$ | $2$ | $3$ | $5/2$ |
| $b, c, a$ | $3$ | $3$ | $3$ |
| $c, a, b$ | $3$ | $2$ | $5/2$ |
| $c, b, a$ | $3$ | $3$ | $3$ |

各欄對均勻輸入平均皆為 $16/6 = 8/3 \ge \lg 6$，各列期望的最大值為三。均勻輸入平均、單一輸入對亂數的平均、最壞期望成本，三者不可混用。

## 堆積：形狀限制與鍵值限制

二元堆積的形狀是**完全二元樹**：最後一層以前全滿，最後一層由左往右填。1 起算陣列的位置 $i$ 有

$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

只有 $1 \le i \le s$ 的位置屬於目前堆積，$s$ 是 heap-size（有效堆積大小）。陣列容量、陣列長度與 $s$ 是不同的量。

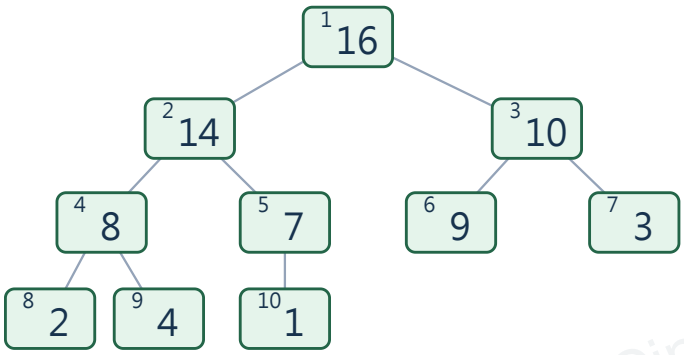
max-heap 要求每個非根位置 $i$ 都有 $A[parent(i)] \ge A[i]$；此處 $A[i]$ 簡寫其鍵值，交換時仍搬動整筆資料。沿祖先鏈可知根為全體最大鍵；同層或左右子樹之間沒有完整排序關係。

高度計最長向下路徑的**邊數**；單節點高度 0，非空 $n$ 節點堆積高 $\lfloor \lg n \rfloor$。若改計層數，則是 $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$。空堆積另行處理，不代入 $\lg 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一棵堆積的樹與陣列表示




節點左上：索引　中央：鍵值

索引固定，鍵值移動

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 |
| **6** | **7** | **8** | **9** | **10** |
| 9 | 3 | 2 | 4 | 1 |

每條邊都滿足：父鍵 ≥ 子鍵
同一列的節點不保證有序

節點旁標示陣列索引。交換時鍵值移動，位置的父子索引關係不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 向下修復的前提不能省略

前提：$1 \le i \le s$，位置 $i$ 的左右子樹已各自為 max-heap，只有 $i$ 與子節點之間可能違規。不存在的子節點不讀取；and 採短路求值。

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

每輪在目前節點與存在的兩個子節點中找最大者，再決定是否交換。只比較兩個子節點、沒有比較父節點，並不能判斷是否需要修復。



## 向下修復為何正確，又為何會停止

若最大者就是目前節點，兩條父子邊合法；兩棵子樹原本合法，因此整棵子樹完成。

否則把較大子節點的根鍵提升。它不小於自己的後代，也不小於另一個子節點的根與原父鍵，所以提升後父節點的兩條邊都合法。唯一可能的新違規，在舊父鍵下移的位置；另一棵子樹保持不變。

對剩餘高度歸納即可證明整棵子樹最後合法。每次 $i$ 變為 $2i$ 或 $2i+1$，嚴格下降一層；高度 $h$ 至多下降 $h$ 次，所以時間 $O(h+1)$，迭代版額外空間 $O(1)$。

例 $A=[16,4,10,8,7,9,3]$、$i=2, s=7$：選子鍵 $8$，交換位置 $2,4$，得 $[16,8,10,4,7,9,3]$。位置 $4$ 的子索引 $8,9$ 已超界，停止。索引 $7$ 的鍵 $3$ 屬於另一棵子樹，這一輪不會拿它比較。

## 由下而上建堆：先讓子樹符合前提

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

不變量：每輪開始時，索引大於 $i$ 的每個節點都是合法 max-heap 的根。

初始化 $i = \lfloor n/2 \rfloor$ 時，較大索引全是葉，成立。兩個子節點索引都比 $i$ 大，因此呼叫滿足 MAX-HEAPIFY 的前提；完成後 $i$ 也合法，而且其後代仍是合法子堆。令 $i$ 減一，保持下一輪條件。

終止 $i = 0$ 時，根 $1$ 也合法，整棵堆積完成。$n = 0, 1$ 時不執行迴圈，直接得到合法空堆或單節點堆。

$\lfloor n/2 \rfloor$ 是最後一個可能有子節點的位置，**不是第一片葉**；例如 $n = 10$ 時位置 $5$ 還有左子節點 $10$，不能漏掉這一輪。



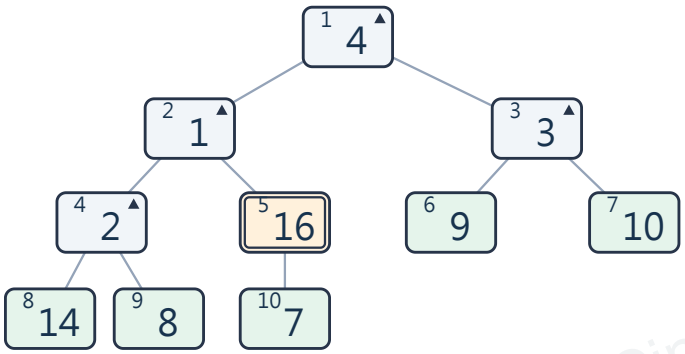
## 由位置五往前建堆：每輪完整狀態

輸入 $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$，有效大小 $s = 10$。表格採 **1 起算索引**；每輪的兩棵子樹已先修復，才處理本輪的根。

| 根 $i$ | 選子節點與交換索引 | 本輪修復後 $A[1..10]$ |
|---|---|---|
| 5 | 唯一子鍵 $7$；$16 \geq 7$，不交換 | $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$ |
| 4 | 子鍵 $14, 8$，選 $14$；$4 \leftrightarrow 8$ | $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$ |
| 3 | 子鍵 $9, 10$，選 $10$；$3 \leftrightarrow 7$ | $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$ |
| 2 | 選 $16$，再選 $7$；$2 \leftrightarrow 5$、$5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| 1 | 選 $16, 14, 8$；$1 \leftrightarrow 2$、$2 \leftrightarrow 4$、$4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

接下來的樹與陣列圖使用同一組資料。橘框為目前處理的根，綠框為已保證合法的子樹根；位置的父子關係固定，交換只移動鍵值及其資料。

## 位置五：只有左子節點




節點左上：索引　中央：鍵值

索引固定，鍵值移動

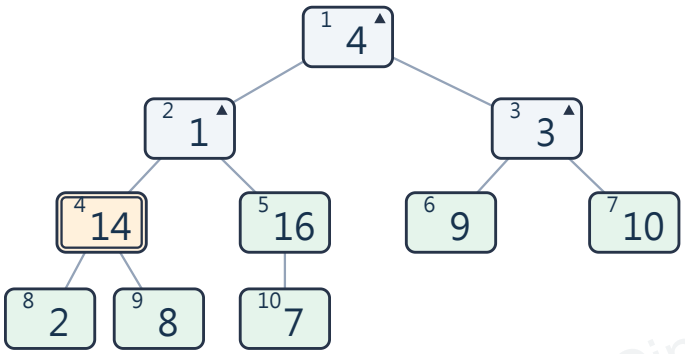
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 ▲ | 1 ▲ | 3 ▲ | 2 ▲ | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 14 | 8 | 7 |

雙框：本輪已完成的子樹根
無記號：先前已合法；▲：待處理

位置 $5$ 的鍵 $16$ 與左子節點 $10$ 的鍵 $7$ 比較，已滿足 $16 \geq 7$，不交換。右子索引 $11 > s$，不能讀取。位置 $6..10$ 都是葉；加上本輪完成的子樹根 $5$，由下往上的不變量繼續成立。



## 位置四：較大的子鍵十四上移




節點左上：索引　中央：鍵值

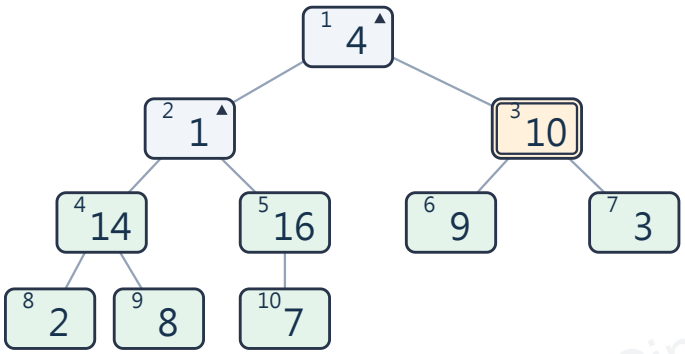
索引固定，鍵值移動

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 ▲ | 1 ▲ | 3 ▲ | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 2 | 8 | 7 |

雙框：本輪已完成的子樹根
無記號：先前已合法；▲：待處理

位置 $4$ 原鍵為 $2$，子索引 $8, 9$ 的鍵為 $14, 8$。先選較大子鍵 $14$，再判斷 $14 > 2$，交換位置 $4, 8$。舊鍵 $2$ 到葉位置 $8$ 後停止，位置 $9$ 的鍵 $8$ 不變。

## 位置三：從兩個子節點中選十




節點左上：索引　中央：鍵值

索引固定，鍵值移動

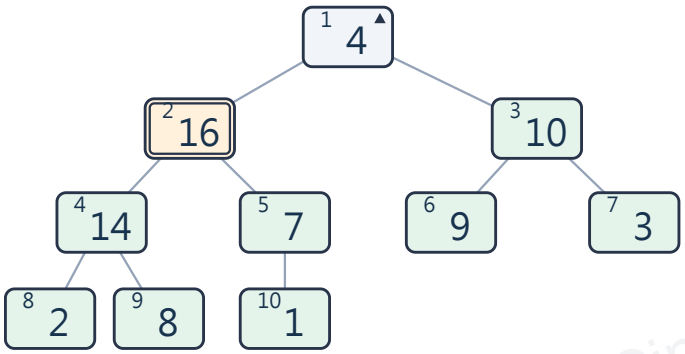
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 ▲ | 1 ▲ | 10 | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 7 |

雙框：本輪已完成的子樹根
無記號：先前已合法；▲：待處理

位置 $3$ 的鍵為 $3$，子索引 $6, 7$ 的鍵為 $9, 10$。選較大鍵 $10$，交換位置 $3, 7$。舊鍵 $3$ 到葉位置 $7$ 後停止，位置 $6$ 的鍵 $9$ 不變。兩個子樹原先合法，交換後以位置 $3$ 為根的整棵子樹也合法。



## 位置二：舊鍵一沿五、十下沉




節點左上：索引　中央：鍵值

索引固定，鍵值移動

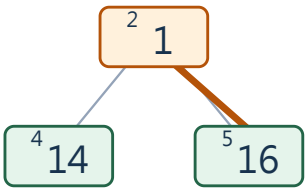
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 ▲ | 16 | 10 | 14 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 1 |

雙框：本輪已完成的子樹根
無記號：先前已合法；▲：待處理

位置 $2$ 的子鍵為 $14, 16$，先與較大的 $16$ 交換。舊鍵 $1$ 到位置 $5$ 後，仍小於唯一子鍵 $7$，再交換位置 $5, 10$。路徑為 $2 \to 5 \to 10$；這輪不改動位置 $4$ 的鍵 $14$。

## 追蹤同一個舊鍵：每一步只選當前子節點

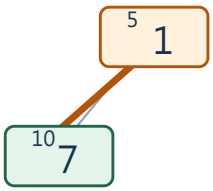
比較 1




16 > 1 → swap(2,5)

A[2]=16; A[5]=1

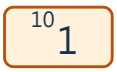
比較 2




7 > 1 → swap(5,10)

A[5]=7; A[10]=1

到葉，停止




20, 21 > heap-size 10

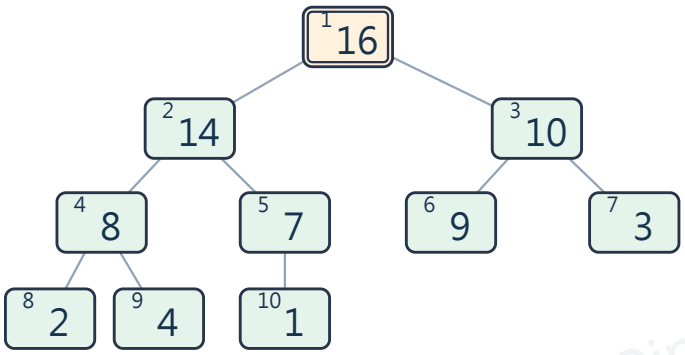
不再讀取 child

舊鍵 1 留在索引 10

橘框始終追蹤舊鍵 $1$；粗線指出本次選中的較大子節點。每個節點左上角是索引、中央是鍵。第一次在子索引 $4, 5$ 中選 $5$，第二次在位置 $5$ 只有子索引 $10$。未畫出的子樹保持合法，不會因為未被走訪就需要重排。

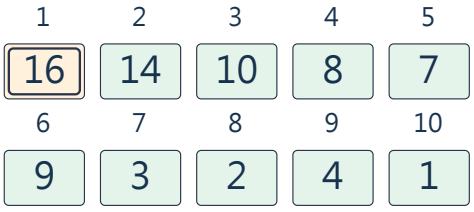


## 位置一：根鍵四沿二、四、九下沉




節點左上：索引　中央：鍵值

索引固定，鍵值移動

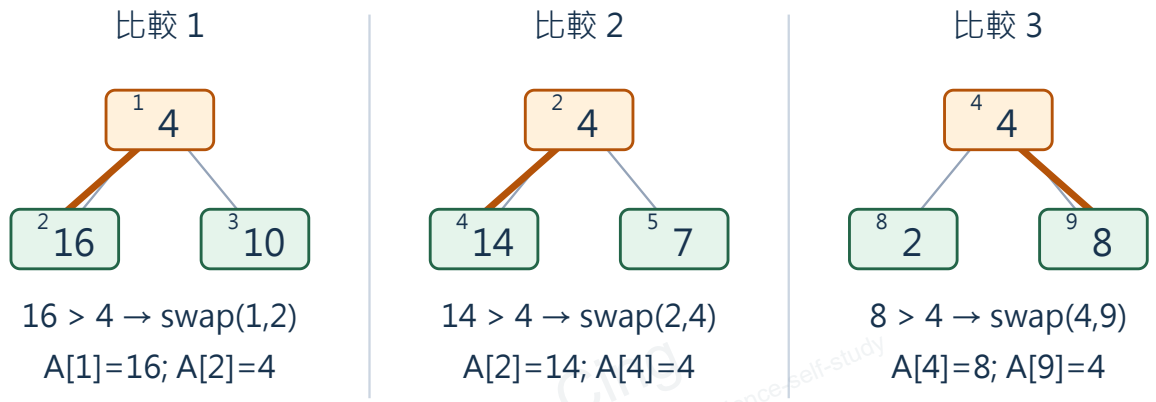



雙框：本輪已完成的子樹根

無記號：先前已合法；▲：待處理

舊根鍵 $4$ 先後與 $16, 14, 8$ 交換，停在葉位置 $9$。最終各非葉的父子關係為 $16 \geq 14, 10$；$14 \geq 8, 7$；$10 \geq 9, 3$；$8 \geq 2, 4$；$7 \geq 1$。這些關係涵蓋所有九條樹邊，因此整棵樹為 max-heap；十筆資料的多重集合不變。

## 根的三次下降：索引路徑與鍵值分開看




每框畫的是交換前的局部狀態，下方列出交換後的鍵值。路徑索引為 $1 \to 2 \to 4 \to 9$，同一個舊鍵始終是 $4$；各步選中的子鍵則為 $16, 14, 8$。到位置 $9$ 後，子索引 $18, 19$ 都超過 $s = 10$，所以停止。



## 逐次交換後的完整陣列

接續位置 $3$ 修復完成後的陣列，所有索引固定為 $1..10$：

| 本輪根 | 交換索引 | 交換後 $A[1..10]$ |
|---|---|---|
| $2$ | $2 \leftrightarrow 5$ | $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$ |
| $2$ | $5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| $1$ | $1 \leftrightarrow 2$ | $[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| $1$ | $2 \leftrightarrow 4$ | $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$ |
| $1$ | $4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

核對方法：每次先在目前節點的有效子節點中選最大，再判斷它是否大於父鍵。一次交換只改兩格，其餘八格與資料多重集合保持不變。要追蹤下降路徑，應保存這些中間狀態；最終陣列本身沒有記錄完整操作歷史。

## 線性建堆：把節點高度逐層加總

先假設 $n \geq 1$。位置 $i$ 至少高 $h$，當且僅當它最左的第 $h$ 代後裔仍存在，也就是 $i2^h \leq n$。所以高度至少 $h$ 的節點數**恰為** $\lfloor n/2^h \rfloor$。

把每個高度拆成 $h$ 個單位，再交換有限求和次序：

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

一次修復花 $O(height(i)+1)$，呼叫共 $\lfloor n/2 \rfloor$ 次，因此總成本 $O(n)$。$n \geq 2$ 時，迴圈本身至少做 $\lfloor n/2 \rfloor$ 次固定工作，給 $\Omega(n)$，合為 $\Theta(n)$。

例如 $n = 15$，高度和為 $7+3+1 = 11$；不能把每個節點都當成根的高度。空堆積的高度和為零，沒有修復呼叫，直接返回；不代入 $\lg 0$，也不聲稱 $0 < 0$。逐筆插入的新鍵可能一路上浮，其最壞總成本 $\Theta(n \lg n)$；那是另一個建堆程序。



## 堆積排序：有效前綴逐步縮小

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

每輪開始時，$A[1..end]$ 為 max-heap；尾端 $A[end+1..n]$ 已遞增，而且尾端每個鍵都不小於堆積中的任一鍵。

交換把前綴最大鍵送到 $A[end]$，使排序尾端再長一格。有效大小改為 $end-1$，原根的兩棵子樹仍合法，所以只需修復新根。交換保留資料多重集合；終止時剩一筆前綴，整體有序。

建堆 $O(n)$，其後至多 $n-1$ 次修復各花 $O(\lg(n+1))$，故時間 $O(n \lg(n+1))$。比較排序下界給最壞 $\Theta(n \lg n)$。本節相等即停的實作，遇全相等鍵可只花 $\Theta(n)$，不能把最壞界套到每份輸入。

迭代修復使額外空間 $O(1)$；若改用遞迴修復，須另計呼叫堆疊。

## 十筆堆積排序：九次取出最大鍵

從建堆結果 $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ 開始。每輪先交換位置 $1$ 與舊末端 $s+1$，把最大鍵送到排序尾端，再修復新大小 $s$ 的前綴。表中的路徑只列後續下沉，不含根與末端的交換。

| 新大小 $s$ | 修復路徑 | 修復後堆積前綴 | 已排序尾端 |
|---|---|---|---|
| $9$ | 1→2→4→9 | $[14, 8, 10, 4, 7, 9, 3, 2, 1]$ | $[16]$ |
| $8$ | 1→3→6 | $[10, 8, 9, 4, 7, 1, 3, 2]$ | $[14, 16]$ |
| $7$ | 1→3→7 | $[9, 8, 3, 4, 7, 1, 2]$ | $[10, 14, 16]$ |
| $6$ | 1→2→5 | $[8, 7, 3, 4, 2, 1]$ | $[9, 10, 14, 16]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 最後五輪：前綴縮到只剩一筆

| 新大小 $s$ | 修復路徑 | 修復後堆積前綴 | 已排序尾端 |
|---|---|---|---|
| $5$ | 1→2→4 | $[7, 4, 3, 1, 2]$ | $[8, 9, 10, 14, 16]$ |
| $4$ | 1→2 | $[4, 2, 3, 1]$ | $[7, 8, 9, 10, 14, 16]$ |
| $3$ | 1→3 | $[3, 2, 1]$ | $[4, 7, 8, 9, 10, 14, 16]$ |
| $2$ | 1→2 | $[2, 1]$ | $[3, 4, 7, 8, 9, 10, 14, 16]$ |
| $1$ | — | $[1]$ | $[2, 3, 4, 7, 8, 9, 10, 14, 16]$ |

例如第二輪，舊末端的鍵 $1$ 到根後沿 $1 \to 3 \to 6$ 下沉，先與 $10$、再與 $9$ 交換。每輪的尾端皆遞增，而且所有尾端鍵都不小於有效堆積的任一鍵；最後合併得到完整遞增序列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 優先權佇列：最大值、取出、升鍵與插入

$Q.A[1..Q.s]$ 是 max-heap，陣列可寫容量為 `Q.capacity`。以下固定容量程序先檢查空／滿；所有交換搬動整筆資料，鍵值只決定優先權。

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

`MAXIMUM` 為 $O(1)$；`EXTRACT-MAX` 保存最大資料，以末筆補根、縮小有效大小，再向下修復，最壞 $O(\lg(s+1))$。單筆取出後大小為零，不讀新的根。



## 升鍵：只沿祖先方向修復

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

升鍵可能破壞的只有祖先方向：新鍵仍不小於原有子鍵。每次交換修好目前父子邊，候選鍵繼續向上；到根或父鍵不小於它時停止，最壞 $O(\lg(s+1))$。`and` 從左往右短路，先確認 $i > 1$。

## 直接插入新資料，不需要哨兵鍵

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

直接把新資料寫在新的最後位置，再上浮，不需額外的負無窮哨兵。固定容量且未滿時，插入最壞為 $O(\lg(s+1))$；若動態陣列需要重新配置，單次搬移可能為 $\Theta(s)$，須另外分析擴容的攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從位置八開始，核對每次上浮

升鍵沿父索引 8 → 4 → 2 上浮

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 原堆積 | 20 | 15 | 18 | 7 | 12 | 9 | 17 | 2 |
| 位置 8 升鍵 | 20 | 15 | 18 | 7 | 12 | 9 | 17 | **16** |
| 交換 8 ↔ 4 | 20 | 15 | 18 | **16** | 12 | 9 | 17 | 7 |
| 交換 4 ↔ 2 | 20 | **16** | 18 | 15 | 12 | 9 | 17 | 7 |

16 先後與父鍵 7、15、20 比較；最後 $20 \geq 16$，停止上浮。

例中先在位置 $8$ 寫入 $16$，依序比較父鍵 $7, 15, 20$；交換 $8 \leftrightarrow 4$、$4 \leftrightarrow 2$ 後停止。位置索引固定，資料身分隨交換移動。

## 降鍵、指定刪除與相等優先權

max-heap 的降鍵只可能破壞往子節點的邊，故向下修復。刪除位置 $i$ 時，以末筆補洞並縮小大小；若 $i$ 已是舊末端，直接縮小即可。

設替補鍵為 $z$。若 $i > 1$ 且 $z > A[parent(i)]$，則因原父鍵不小於原被刪鍵、原被刪鍵不小於其子鍵，$z$ 也支配那些子鍵，只需上浮。否則父邊合法，只需下沉。各操作最壞 $O(\lg(s+1))$，前提是已知位置；搜尋任意鍵仍可能要 $\Theta(s)$。

若用 handle（資料身分代號）定位，交換時必須同步更新 handle 到索引的對照表，移除被刪資料的對照。陣列索引本身不會永遠指向同一筆資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 排序與佇列的同鍵次序，使用不同的序號方向

堆積排序並不穩定：$[2a, 2b, 1c]$ 可排成 $[1c, 2b, 2a]$。若要遞增排序時保留同鍵原順序，可比較 $(key, original_index)$；較晚的同鍵資料較大，會先被放到尾端。若 max-priority queue 要讓同優先權者先到先服務，則比較 $(priority, -arrival_index)$。兩者的序號方向不同，且都改變了比較與儲存規格。

同優先權資料 $a, b$ 依序抵達，原序號分別為 $1, 2$，鍵皆為 $5$：

| 用途 | $a$ 的比較鍵 | $b$ 的比較鍵 | 先取出最大者 | 所需次序 |
|---|---|---|---|---|
| 穩定遞增排序 | $(5, 1)$ | $(5, 2)$ | $b$，先放到尾端 | 輸出 $a, b$ |
| 同優先權先到先服務 | $(5, -1)$ | $(5, -2)$ | $a$，立即服務 | 服務 $a, b$ |

遞增排序會把較大的鍵從右往左放；佇列則直接服務被取出的最大者。使用序號前，先確認「取出的先後」如何對應目標次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 雇用問題：比較成本與替換成本要分開

有 $n$ 位能力相異、可比較排名的候選人；排名越大越好。每次面試後，只要比現任者好就立刻替換。虛擬候選人 dummy 比任何人都差。

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

第 $i$ 輪後，best 是前 $i$ 位中能力最好的候選人：初始無人；新來者較好才替換，否則保留原最佳者。因此最後能找出全體最佳者。



## 同一個程序，三種不同的成本問題

面試費為 $c_I$、每次雇用費為 $c_H$，雇用 $X$ 次的總費用為 $nc_I + Xc_H$。空輸入返回 dummy、$X = 0$。對 $n \geq 1$，排名遞增時 $X = n$，逆序時 $X = 1$。接著區分三種問法：

| 問法 | 隨機來源或量詞 | 要算的量 |
|---|---|---|
| 最壞情形 | 所有合法輸入取最大 | $\max_A X(A) = n$ |
| 平均情形 | 明定輸入分布，例如 $n!$ 種順序等機率 | $\mathbb{E}_A[X(A)]$ |
| 隨機化程序 | 固定任意輸入，由程序均勻洗牌 | $\mathbb{E}_R[X(A, R)]$ |

均勻洗牌使每個固定的相異排名輸入變成均勻排列；前提是能控制處理順序。期望保證仍容許壞執行：遞增排列的機率為 $1/n!$。雇用次數少，只降低雇用費，並未減少面試次數。

## 指示變數：把計數拆成可以求機率的事件

先固定有限樣本空間 $\Omega$，各結果的機率非負、總和為 $1$。隨機變數 $X$ 的期望是 $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$。

事件 $E$ 的指示變數 $I_E$ 在 $E$ 成立時為 $1$，否則為 $0$，故

$$\mathbb{E}[I_E] = 1 \cdot \Pr(E) + 0 \cdot \Pr(E^c) = \Pr(E).$$

若目標計數恰為 $X = \sum_{i=1}^{m} X_i$，交換兩個有限和可得

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

這是**期望線性性**，不要求 $X_i$ 獨立。獨立性可能用在計算單項事件的機率，或將交集機率寫成乘積時。

反例：令 $X_1 = X_2$ 都是同一次公平擲幣的正面指示。仍有 $\mathbb{E}[X_1 + X_2] = 1$，但 $\Pr(X_1 = X_2 = 1) = 1/2$，不是 $1/4$。



## 三個機率工具：聯集界、Markov 界與尾和

先辨認正在求「事件機率」「個數的期望」或「最大值的期望」。以下三個工具都可從指示變數推出。

聯集指示滿足 $I_{\cup_i E_i} \le \sum_i I_{E_i}$，取期望即得

$$\Pr\left(\bigcup_{i} E_i\right) \le \sum_{i} \Pr(E_i).$$

若 $X \ge 0$、$t > 0$，由 $X \ge t I_{\{X \ge t\}}$ 得 Markov 界

$$\Pr(X \ge t) \le \frac{\mathbb{E}[X]}{t}.$$

## 整數尾和：每個結果到達了幾個門檻

若 $L$ 為非負整數，逐個結果皆有 $L = \sum_{j \ge 1} I_{\{L \ge j\}}$，故

$$\mathbb{E}[L] = \sum_{j \ge 1} \Pr(L \ge j).$$

有限上界時可直接交換有限和；無限尾和由非負部分和的單調極限成立，兩側亦可同為無窮大。這三個工具均不需要事件獨立。

獨立性若用在具體例子的機率乘法，仍須另外明列；不能因工具不要求獨立，就省略模型中的獨立條件。

數例：$L = 0, 1, 3$ 的機率分別為 $1/2, 1/4, 1/4$，直接算得期望 $0/2 + 1/4 + 3/4 = 1$。尾機率依序為 $\Pr(L \ge 1) = 1/2$、$\Pr(L \ge 2) = 1/4$、$\Pr(L \ge 3) = 1/4$，加總也等於 $1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 雇用次數：先算每個位置刷新紀錄的機率

假設 $n!$ 種相異排名排列等機率，令 $X_i$ 表示第 $i$ 位是否被雇用。固定前 $i$ 位的候選人集合後，這些人的 $i!$ 種相對順序仍等可能，最高者落在最後位置的機率為 $1/i$。

第 $i$ 位被雇用，恰好等價於「它是前 $i$ 位的最高者」，所以

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

由 $1/x$ 遞減，矩形面積與積分比較給出

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}, \qquad n \ge 1.$$

因此 $\mathbb{E}[X] = \Theta(\ln n)$，而總期望費用仍為 $nc_I + c_H H_n$；面試費這一項並未變成對數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 六人逐輪雇用，與同規模期望值對照

| 位置 $i$ | 排名 | 面試後最佳排名 | 本次雇用？ | 累積次數 |
|---|---|---|---|---|
| 1 | 2 | 2 | 是 | 1 |
| 2 | 5 | 5 | 是 | 2 |
| 3 | 1 | 5 | 否 | 2 |
| 4 | 4 | 5 | 否 | 2 |
| 5 | 6 | 6 | 是 | 3 |
| 6 | 3 | 6 | 否 | 3 |

數例 $[2,5,1,4,6,3]$ 在位置 $1,2,5$ 刷新紀錄，共雇用三次；同規模平均為 $H_6 = 49/20 = 2.45$。只有一次雇用的機率為 $1/n$；每位都雇用只有一種遞增排列，機率為 $1/n!$。空輸入另取 $H_0 = 0$。



## 同一個方法，也能算逆序對與拿回原帽的人數

均勻排列 $n$ 個相異鍵。對位置 $i < j$，令 $I_{ij}$ 表示 $A[i] > A[j]$。交換兩位置的值，將逆序與非逆序結果一對一配對，因此 $\Pr(I_{ij} = 1) = 1/2$。

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

共享位置的逆序事件可能相依；這不影響期望線性性。當 $n = 2$，逆序數只能是 $0$ 或 $1$，但期望為 $1/2$。

另一例：將 $n \geq 1$ 頂帽子均勻地一對一還給原主人們。第 $i$ 人拿回自己的帽子的機率為 $1/n$，固定點數 $F$ 滿足

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

$n = 2$ 時，可能兩人都拿對或都拿錯，各有 $1/2$ 機率。期望為一人，不表示一定恰有一人拿對。$n = 0$ 時直接有 $F = 0$。

## 均勻整數抽樣：拒絕超出範圍的位元結果

RANDOM(a,b) 的契約是回傳包含兩端的整數，各機率 $1/(b-a+1)$，且新呼叫條件於歷史仍均勻。以下以公平、獨立的位元實作；必須有整數 $a \le b$。

令 $m = b - a + 1$。$m = 1$ 直接回傳 $a$；否則令 $k = \lceil \lg m \rceil$，反覆讀取 $k$ 個位元作為 $z \in \{0, \dots, 2^k - 1\}$。若 $z < m$，回傳 $a + z$；否則重抽整組。

每輪接受率 $p = m/2^k > 1/2$。對任一合法值 $v$，回傳它的機率為

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p 2^k} = \frac{1}{m}.$$

重試輪數期望為 $1/p < 2$，因此期望位元成本為 $O(\lg m)$。拒絕抽樣幾乎必然終止，沒有預先固定的重試次數上限。

直接取模會產生偏差：八個三位元結果對 $3$ 取模，餘數 $0, 1, 2$ 的前像數為 $3, 3, 2$，不是等機率。偽亂數產生器與整數取樣方式，也須分別滿足所採模型。



## 原地洗牌：每輪多固定一個位置

每輪從尚未固定的後綴（suffix）中均勻選一筆，與位置 $i$ 交換。即使鍵相同，也把每筆資料視為不同身分。

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

第 $i$ 輪後，後續交換不再碰 $A[1..i]$。空輸入只有一種空排列；單筆輸入只與自己交換。單位成本整數抽樣模型下，時間 $\Theta(n)$、額外空間 $O(1)$；位元抽樣成本另計。

## 均勻洗牌：逐輪延伸指定的前綴

主張：第 $i$ 輪前，任一指定的 $i-1$ 個不同身分序列占據 $A[1..i-1]$ 的機率為 $(n-i+1)!/n!$。初始 $i=1$，空前綴的機率為一。

固定此前綴後，下一個指定身分恰占剩餘 $n-i+1$ 格中的一格。條件均勻抽樣以 $1/(n-i+1)$ 的機率選中它，因此指定長 $i$ 前綴的機率為

$$\frac{(n-i+1)!}{n!}\,\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

歸納至 $i=n$，每個完整排列的機率皆為 $0!/n!=1/n!$。取 $n=4$、指定輸出 $[C,D,A,B]$：

| 固定長度 | $0$ | $1$ | $2$ | $3$ | $4$ |
|---|---|---|---|---|---|
| 指定前綴 | 空 | $C$ | $C,D$ | $C,D,A$ | $C,D,A,B$ |
| 機率 | $1$ | $1/4$ | $1/12$ | $1/24$ | $1/24$ |



## 四筆洗牌：逐輪檢查固定前綴

從 $[A,B,C,D]$ 開始，依序抽到 $j=3,4,3,4$。字母是身分，不是比較用的鍵。

| 輪次 $i$ | 合法索引 | 抽到 $j$ | 交換後完整陣列 | 已固定前綴 |
|---|---|---|---|---|
| 1 | $1..4$ | 3 | $[C,B,A,D]$ | $C$ |
| 2 | $2..4$ | 4 | $[C,D,A,B]$ | $C,D$ |
| 3 | $3..4$ | 3 | $[C,D,A,B]$ | $C,D,A$ |
| 4 | $4$ | 4 | $[C,D,A,B]$ | $C,D,A,B$ |

此路徑機率為 $(1/4)(1/3)(1/2)(1)=1/24$。任意指定輸出也唯一決定一條合法路徑：依次在剩餘後綴找到指定的下一個身分，再將它換到位置 $i$。

## 已固定的前綴，後面不再移動

從 [A,B,C,D] 開始：每輪只從尚未固定的 suffix 選一個

| | 位置1 | 位置2 | 位置3 | 位置4 | 本輪機率 |
|---|---|---|---|---|---|
| i=1, j=3 | C | B | A | D | 1/4 |
| i=2, j=4 | C | D | A | B | 1/3 |
| i=3, j=3 | C | D | A | B | 1/2 |
| i=4, j=4 | C | D | A | B | 1 |

綠格已固定、不再改動；這條路徑機率 1/4·1/3·1/2·1=1/24。

後面的輪次只交換尚未固定的位置；這是前綴能永久固定的原因。單次追蹤用來核對程序，一般均勻性仍由前面的聯合不變量證明。



## 每個位置看似均勻，整個排列仍可能有偏

隨機選一個循環平移量，確實能讓每筆以 $1/n$ 機率落在每個位置；但 $n \geq 3$ 時只有 $n$ 種可能輸出，而非 $n!$ 種。均勻邊際分布不等於聯合均勻。

另一個錯誤程序讓 $i = 1..n$，每輪都從整個 $1..n$ 抽 $j$ 再交換。$n = 3$ 時有 $3^3 = 27$ 條等可能亂數路徑；若六種排列均勻，每種須對應 $27/6$ 條路徑，但路徑數必為整數，故不可能。

錯誤程序會重新移動先前的位置，因此不能沿用「已固定前綴」的證明。證明某個程序不均勻，與只說「看起來不夠隨機」不同。

正確洗牌若只做前 $m$ 輪，$0 \leq m \leq n$，每個有序樣本的機率為 $(n-m)!/n!$；忽略順序後，每個 $m$ 元子集合有 $m!$ 種排列，機率為 $1/\binom{n}{m}$。這省下後續交換；若輸入必須先建成可索引陣列，建表成本仍要計入。

## 隨機優先值：沒有碰撞時才直接得到均勻排列

給 $n \geq 1$ 筆資料獨立均勻的優先值 $P_i \in \{1, \ldots, M\}$，其中整數 $M \geq n$，再按優先值排序。條件於所有優先值不同，交換資料標籤會一對一對應不同次序，且保留機率，因此 $n!$ 種相對順序等可能。

每對碰撞機率為 $1/M$。聯集界（union bound）給

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

取 $M = n^3$、$n \geq 2$，上界為 $(n-1)/(2n^2) < 1/(2n)$。碰撞很少，仍不等於不會碰撞。

若平手時保留原順序，無條件輸出有偏：$n = 2$ 時，第一筆先出的機率為 $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$。

可在任何碰撞時整組重抽，再接受無碰撞結果；上述 $M$ 下接受率有正常數下界，期望輪數為常數。也可在每個平手群中獨立均勻洗牌。用比較排序實作優先值排序需 $\Theta(n \lg n)$ 時間與 $O(n)$ 優先值空間；重抽另須偵測碰撞，不能當成免費。



## 隨機化之後，仍要說清楚正確性與可控制的部分

`RANDOMIZED-HIRE` 先均勻洗牌，再執行 `HIRE`。對每個固定的相異排名輸入，洗牌後皆為均勻排列，故期望雇用數為 $H_n$，期望費用為 $nc_I + c_H H_n$。另有 $\Theta(n)$ 洗牌操作，不能把操作時間與雇用費用混成同一單位。

若候選人只能依外部順序到達、不能重新安排，就沒有這個洗牌控制權。此時 $H_n$ 的分析仍須對外部順序假設均勻，不能用一個無法執行的步驟取消輸入假設。

本章的洗牌與隨機 pivot 排序，隨機的是成本；每次返回都符合規格。這類稱 Las Vegas 演算法。Monte Carlo 演算法允許小機率錯答，必須另列錯誤機率與其適用輸入；只給期望時間不能推得正確率。

後面的線上遴選又是不同目標：策略可能沒有選到全域最佳，但仍按規定選出一人。它的成功機率要依遴選模型另證，不能與排序的正確性保證混用。

## 快速排序：分割後，兩側仍要各自排序

本節使用 1 起算閉區間 $A[p..r]$。選定 pivot（分割基準值）後，分割只建立它與兩側的大小關係，還沒有把兩側內部排好。

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

空區間 $p > r$、單筆區間 $p = r$ 都直接返回。分割回傳 pivot 的位置 $q$；兩個遞迴區間都**排除** $q$，所以長度嚴格縮小。這個回傳契約是排序程序正確與終止的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 尾端分割：保存鍵值，交換整筆資料

尾端版本的分割程序如下。鍵值 $x$ 先保存；交換搬動整筆資料，其他欄位隨鍵一起移動。

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 迴圈邊界：i 是小鍵區尾端，j 是下一筆待分類

每輪開始時，pivot 固定在 A[r]。本章採 **1 起算索引、閉區間 A[p..r]**；i 可以等於 p−1，表示 ≤ 區為空。

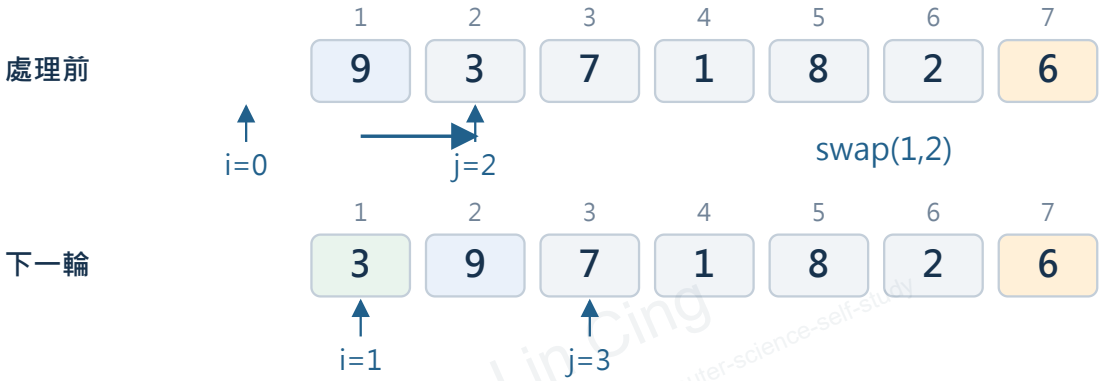
| 區域 | 已知關係 | 初始狀態 i=p−1、j=p |
|---|---|---|
| A[p..i] | ≤ pivot | 空 |
| A[i+1..j−1] | > pivot | 空 |
| A[j..r−1] | 尚未分類 | 除 pivot 外全部 |
| A[r] | pivot | 全程固定到收尾交換 |

**讀圖方式：** 綠色是 ≤ 區，藍色是 > 區，灰色是未知區，橘色是 pivot。以下兩頁都沿用 [9,3,7,1,8,2,6]、pivot=6。



## 小鍵到來：交換後，兩個邊界都前進

### 3 ≤ 6：先讓 i 前進，再交換 A[1] 與 A[2]

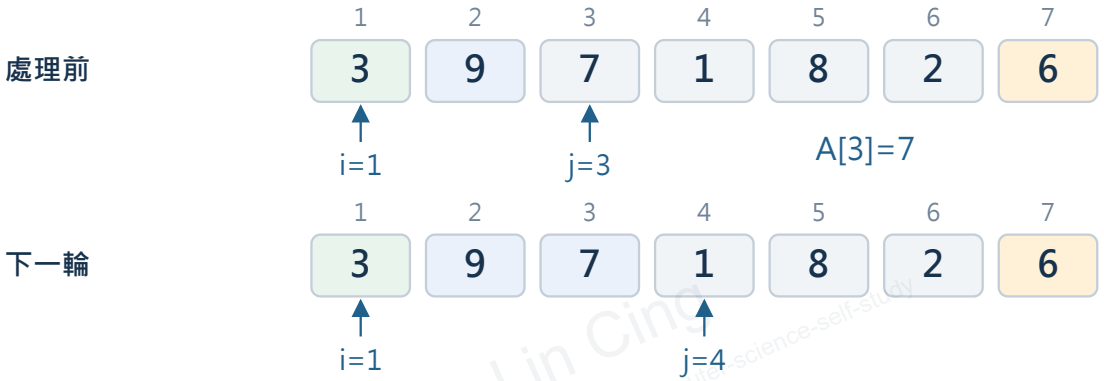



3 加入 ≤ 區；9 回到 > 區。pivot 6 仍在索引 7。

處理 A[2]=3 時，先令 i=1，再交換 A[1]、A[2]。3 加入 ≤ 區，原先的 9 留在 > 區；最後 j 前進。一般情況也可能 i=j，此時是自己與自己交換。

## 大鍵到來：不搬資料，只擴大已分類區

### 7 > 6：不交換、不改 i，只讓 j 前進



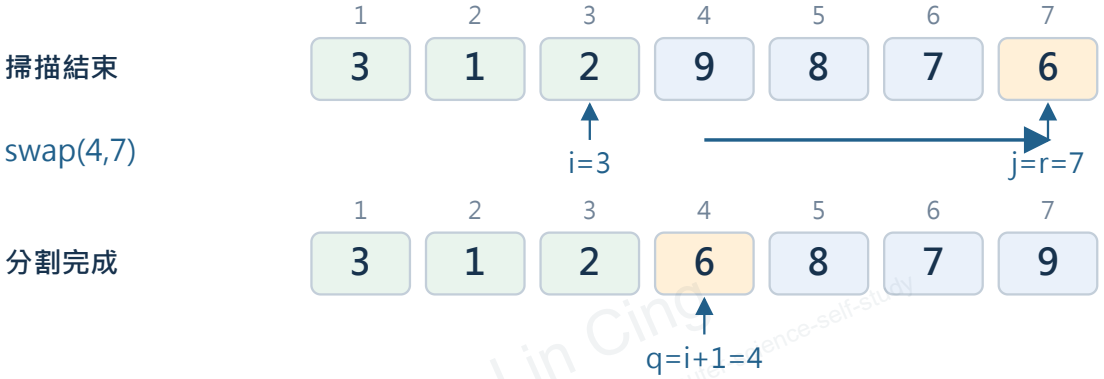

7 從未知區加入 > 區；陣列值不變，但分類邊界改變。

A[3]=7>6，因此 i 不變、陣列不變，只把 j 往右移一格。**不變的是值，改變的是已知資訊：** 7 現在屬於 > 區，不再是未知值。



## 收尾交換：q 才是 pivot 的最終位置

### 掃描結束後，pivot 才移到兩區之間




≤6：[3,1,2] | pivot 6 | >6：[8,7,9]；兩側仍未排序。

每輪未知區長 r−j 減一，所以掃描必終止。結束時 j=r；交換 A[i+1] 與 A[r]，得到 q=i+1。左區 ≤ pivot、右區 > pivot，兩側內部仍未排序。

## 從分割不變量到整體排序正確性

1. **初始化：** i=p−1、j=p，前兩區都是空區間，不變量成立。
2. **保持：** 小鍵交換後擴大 ≤ 區；大鍵只擴大 > 區。兩種情況都使未知區少一筆，pivot 仍固定。
3. **分割終止：** 未知區消失，收尾交換把 pivot 放到 q。交換只重排元素，不會遺失或新增資料。
4. **遞迴歸納：** 空區間、單元素已排序。兩個子區間排除 q，長度都變小；歸納假設使它們各自有序，再接上 pivot，整個區間便有序。

只檢查「左小右大」還不夠：完整正確性同時需要**元素多重集合不變、遞迴嚴格縮小**，以及兩側遞迴回來後確實有序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完整分割追蹤：每次比較與交換都列出

輸入 $A = [9, 3, 7, 1, 8, 2, 6]$、$p = 1, r = 7$。尾端鍵 $x = 6$，初始 $i = 0$；表中 $i$ 是本輪判斷後的值。

| 掃描索引 $j$ | 判斷 | $i$ | 操作後的完整陣列 |
|---|---|---|---|
| 1 | $9 > 6$，不交換 | 0 | $[9, 3, 7, 1, 8, 2, 6]$ |
| 2 | $3 \le 6$，交換 $1, 2$ | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 3 | $7 > 6$，不交換 | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 4 | $1 \le 6$，交換 $2, 4$ | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 5 | $8 > 6$，不交換 | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 6 | $2 \le 6$，交換 $3, 6$ | 3 | $[3, 1, 2, 9, 8, 7, 6]$ |
| 掃描結束 | 交換 $4, 7$ | 3 | $[3, 1, 2, 6, 8, 7, 9]$ |

回傳 $q = i + 1 = 4$。左側 $[3, 1, 2] \le 6$、右側 $[8, 7, 9] > 6$，兩側都還沒排好。掃描恰做 $r - p = 6$ 次鍵比較；連同收尾，時間 $\Theta(r - p + 1)$、額外空間 $O(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最壞時間：一整族輸入達到平方成本

已排序的 n 個相異鍵，每次取最後位置作 pivot。pivot 永遠最大，子問題長度是 n−1 與 0。

| 本輪長度 | 本輪鍵比較 | 下一輪非空長度 |
|---:|---:|---:|
| n | n−1 | n−1 |
| n−1 | n−2 | n−2 |
| … | … | … |
| 2 | 1 | 1 |

比較總數為 $n(n-1)/2$，給出最壞時間的 Ω(n²) 下界。這個結論適用整個輸入族，並非由單次實測推論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 平方上界必須涵蓋每一種分割

令 C(n) 是 n 個鍵的最壞比較數，子問題大小 a+b=n−1。以較小規模的歸納假設可得

$$C(a)+C(b)+(n-1)\le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le\frac{n(n-1)}{2}.$$

C(0)=C(1)=0，故上界涵蓋所有分割；a=0 或 b=0 時恰達界。每個被選作 pivot 的元素不再進入遞迴，所以呼叫總數 O(n)。總時間 O(n+C)，配上下界得到**最壞 Θ(n²)**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 均分時，先按層數清楚計算比較工作

取 n=15，每次選真正的中位數。此表只計 Lomuto 的鍵比較，單元素不呼叫 partition。

| 深度 d | 子問題個數 | 每個長度 | 本層鍵比較 |
|---|---|---|---|
| 0 | 1 | 15 | 14 |
| 1 | 2 | 7 | 2×6=12 |
| 2 | 4 | 3 | 4×2=8 |
| 3 | 8 | 1 | 0 |

共 34 次；同樣 15 個鍵，端點分割則做 105 次。差別來自每層保留下來的子問題大小，與 partition 本身仍是線性掃描並不矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 不必完全平分，固定比例也能保證對數深度

若每次非空子問題的較大側至多為原長度的 ρ 倍，其中 ρ<1 是固定常數，最長分支深度為 O(log n)。同一層的子區間互不重疊，本層總掃描量至多 n，因此總時間 O(n log n)；固定的 1:9 分割也符合。

完全均分且 $n = 2^k - 1$ 時，深度 d 有 $2^d$ 個長度 $2^{k-d} - 1$ 的子問題。本層比較數為

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

前約 k/2 層，每層仍做 Θ(n) 次比較，故這種執行本身就需 Ω(n log n)，合為 Θ(n log n)。這裡的下界來自實際工作量；比較排序的最壞輸入下界並不保證每個輸入都一樣慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 重複鍵：隨機 pivot 不能修好全相等輸入

對 $[5,5,5,5,5]$，每個 <= x 都成立，故每輪 $q=r$。子問題長度依序為 $5,4,3,2$，比較次數 $4+3+2+1=10$；一般 $n$ 筆為 $n(n-1)/2$。

隨機換來的 pivot 仍是 $5$，不能改變這條執行路徑。因此本節稍後對二路版本推導的期望 $\Theta(n\lg n)$，先限定**鍵互異**。本章次序統計量節會完整給出三路分割：小於、等於、大於 pivot；只有兩個嚴格區繼續遞迴。全相等時只分割一次，總時間 $\Theta(n)$。

穩定性是另一項保證。以身分區別輸入 $[2_a,2_b,1_c]$，尾端 pivot 為 $1$，收尾交換得到 $[1_c,2_b,2_a]$；兩筆鍵 $2$ 的順序已反轉。隨機挑 pivot 並不會使這個交換式實作自動變成穩定排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 只遞迴較小側：控制堆疊，沒有省掉分割

原本兩側都遞迴的程序，最壞有 $\Theta(n)$ 層堆疊；分割原地進行，不代表整個快速排序只用常數空間。可改成只遞迴較小側，較大側留在本層迴圈處理。

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 較小側遞迴，限制尚未返回的呼叫鏈

兩側長度 $a + b = n - 1$，真正遞迴的一側至多 $\lfloor (n-1)/2 \rfloor$。沿尚未返回的呼叫鏈，規模至少減半，故堆疊為最壞 $O(\lg(n+1))$。

若 pivot 每次都是最大值，遞迴側為空，迴圈仍依次分割長度 $n, n-1, \ldots, 2$。此時堆疊甚至為常數，時間卻仍是 $\Theta(n^2)$；空間與時間要分別計算。

每次選中位數：一條尚未返回的呼叫鏈

巢狀框架

15 → 7 → 3 → 1：同時存活四個框架

每次選最大值：同一框架的連續迴圈

同一工作框架

每輪的空遞迴呼叫立即返回；較大側留在同一框架中，仍須執行下一次分割。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 隨機化保證的是每個固定輸入的期望成本

每輪在目前閉區間 $p..r$ 均勻抽一個整數 $k$，交換 $A[k]$ 與 $A[r]$，再用同一個尾端分割。未來選擇條件於當時狀態仍須均勻。

鍵互異時，不論目前排列如何，每個局部秩成為 pivot 的機率都是 $1/n$。接下來取期望的隨機來源是**演算法內部選擇**，不要求使用者提供均勻排列。

任何固定輸入仍可能遇到每次都選端點的亂數路徑，花費 $\Theta(n^2)$。期望界既不等於每次執行的上界，也不能直接當成尚未證明的尾機率界。

數學模型的均勻抽樣須與實作分開。固定 seed 可重現某條偽亂數序列，但「程式呼叫了 random」本身不足以證明抽樣均勻；有界整數抽樣可使用本章前面的拒絕抽樣程序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 比較一對鍵，關鍵在秩區間內的第一個 pivot

將相異鍵依大小命名為 $z_1 < \cdots < z_n$。令 $X_{ij}$ 表示 $z_i, z_j$ 是否直接做過一次鍵比較。它們至多比較一次：一次比較必有一方為 pivot，而 pivot 隨即退出遞迴。

考慮 $Z_{ij} = \{z_i, \ldots, z_j\}$。區間外的 pivot 把這整段留在同一側；區間內的 pivot 若是端點 $z_i$ 或 $z_j$，本次便比較兩者；若是內部鍵，兩者分居不同遞迴區，往後不再相遇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 區間外的 pivot，不會分開這一對鍵

**分析排序後的秩：$Z_{2,5}$ = {2,3,4,5}；不是陣列快照**

先選 7：2..5 一起留在左子問題，$X_{2,5}$ 尚未決定。

**區間內先選端點 2（或 5）**

該次比較 2 與 5 → $X_{2,5}$=1

**區間內先選內部 3（或 4）**

2 在左、5 在右 → 永不比較，$X_{2,5}$=0

四個區間鍵等可能先選；其中兩個是端點。$\Pr(X_{2,5}=1)=2/4=1/2$。

因此 $X_{ij} = 1$，恰好等價於「$Z_{ij}$ 內第一個被選為 pivot 的鍵是兩個端點之一」。它不必是全程第一個 pivot。

## 從四種首選情況，推到一般比較機率

追蹤 $z_2, z_5$，只看 $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$ 內誰先成為 pivot。

| 區間內第一個 pivot | $z_2, z_5$ 的去向 | $X_{2,5}$ |
|---|---|---|
| $z_2$ | 本次直接比較 | 1 |
| $z_3$ | 分居兩側，不再相遇 | 0 |
| $z_4$ | 分居兩側，不再相遇 | 0 |
| $z_5$ | 本次直接比較 | 1 |

四個鍵各有 $1/4$ 機率成為區間內首選，因此比較機率為 $1/2$。一般區間長 $m = j - i + 1$，故

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$



## 證明區間內第一個 pivot 確實均勻

等可能性需要證明。固定區間 $Z$ 的 $m$ 個鍵及其中一個指定鍵 $z$，對目前包含 $Z$ 的子問題大小 $N$ 歸納。$N = m$ 時，首個 pivot 均勻，指定鍵機率 $1/m$。$N > m$ 時，本次直接選到 $z$ 的機率 $1/N$；選到其餘 $Z$ 內鍵則失敗。

若先選到 $Z$ 外鍵，含 $Z$ 的後續子問題嚴格縮小，依歸納指定鍵首選機率仍是 $1/m$。對 $N - m$ 個外部鍵求和得

$$\frac{1}{N} + \frac{N - m}{N} \frac{1}{m} = \frac{1}{m}.$$

因此區間內首選確實均勻；分母是 $m$，不是原輸入長度 $n$。其他遞迴分支可穿插執行，但不會移動或比較此子區間內的鍵。

## 期望比較數：把秩區間長度逐項加總

總鍵比較數 $C = \sum_{1 \le i < j \le n} X_{ij}$。期望線性性不需要這些事件獨立。長度 $k = j - i + 1$ 的秩區間共有 $n + 1 - k$ 個，因此

$$\begin{aligned}\mathbb{E}[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)\sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\ &= 2(n+1)H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}.\end{aligned}$$

由 $\ln(n+1) \le H_n \le 1 + \ln n$，得到期望比較數 $\Theta(n \lg n)$。每輪掃描工作與本輪鍵比較次數同階，所有呼叫另有 $O(n)$ 開銷，所以期望時間也是 $\Theta(n \lg n)$。

此精確式適用鍵互異、各輪條件均勻、每個非 pivot 鍵只做一次 <= 的版本。它不計索引測試、交換或產生亂數的成本。比較事件彼此相依，不能據此直接套用獨立變數的變異數或尾界公式。



## 首端 pivot 的分割：先看它自己的不變量

以下是首端 pivot 的單向掃描版本；閉區間仍是 $A[p..r]$，但 $x = A[p]$ 固定到收尾。

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

每輪開始時 $A[p] = x$、$A[p+1..i] \le x$、$A[i+1..j-1] > x$，$A[j..r]$ 尚未分類。小鍵先擴大左區再交換，大鍵只移動掃描邊界；未知區每輪少一筆。

終止時交換 $A[p]$ 與 $A[i]$，回傳位置 $i$。排序遞迴仍須排除該位置。不能把首端版本的初始 $i = p$，直接抄進尾端版本的 $i = p - 1$ 規格。

## 同一個 pivot，兩種分割都能核對

首端版輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，取 $x = 6$、初始 $i = 1$。

| $j$ | 判斷與操作 | 操作後陣列 |
|---|---|---|
| 2 | $10 > 6$，$i = 1$ | $[6, 10, 13, 5, 8, 3, 2, 11]$ |
| 3 | $13 > 6$，$i = 1$ | 同前 |
| 4 | $5 \le 6$，$i = 2$，交換 $2, 4$ | $[6, 5, 13, 10, 8, 3, 2, 11]$ |
| 5 | $8 > 6$，$i = 2$ | 同前 |
| 6 | $3 \le 6$，$i = 3$，交換 $3, 6$ | $[6, 5, 3, 10, 8, 13, 2, 11]$ |
| 7 | $2 \le 6$，$i = 4$，交換 $4, 7$ | $[6, 5, 3, 2, 8, 13, 10, 11]$ |
| 8 | $11 > 6$，$i = 4$ | 同前 |
| 結束 | 交換 $1, 4$，回傳 $4$ | $[2, 5, 3, 6, 8, 13, 10, 11]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 對照兩版的相同 pivot

### 同一個 pivot 鍵，區內排列可以不同

兩版皆 q=4、pivot=6；左側 ≤6、右側 >6，區內不必排序。

尾端版若也要選同一個 $6$，須先交換首尾，再分割 $[11, 10, 13, 5, 8, 3, 2, 6]$，可得 $[5, 3, 2, 6, 8, 10, 13, 11]$。兩版都把 $6$ 放在位置 $4$，各做 $7$ 次掃描鍵比較；兩側的內部排列可以不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 名稱相近時，對照前提、邊界與回傳值

| 版本 | pivot 與初始邊界 | 回傳與遞迴 |
| --- | --- | --- |
| CLRS 尾端單向掃描 | $x = A[r]$，$i = p - 1$，$j = p$ | 回傳單點 $q$；遞迴 $[p, q-1]$、$[q+1, r]$ |
| MIT 2005 首端單向掃描 | $x = A[p]$，$i = p$，$j = p + 1$ | 回傳單點 $q$；仍排除 $q$ |
| 首端雙向掃描、最後定位 | $x = A[p]$，左右掃描遇等值即停 | 指標交錯後交換 pivot，回傳單點 $q$ |
| 本章三路分割 | 先保存 pivot 鍵 $x$，$lt = i = p$，$gt = r$ | 回傳等值閉區間 $[lt, gt]$；整段排除 |

雙向掃描仍須再看回傳契約：經典 Hoare partition 回傳分界線，不保證 pivot 已在最終位置；另一種雙向版本會在收尾交換 pivot，回傳定位點。兩者的遞迴範圍不同，不能只憑「Hoare」名稱照搬。後面的雙向定位程序會明定後者的規格。



## 同一批資料，兩套區間表示

先轉換索引與區間規格，再比較程式。對同一批資料 $[2, 3, 6, 6, 8]$，等值區在 1 起算寫成閉區間 $[3, 4]$，在 0 起算寫成半開區間 $[2, 4)$；兩者都包含兩筆資料。

非空閉區間 $[p, r]$ 對應半開區間 $[p-1, r)$，後者不包含右端點。因此閉區間 $[p, q-1]$ 對應半開區間 $[p-1, q-1)$；兩套規格中的空區間長度都為零。

## 雙向掃描後定位：遇到等值就停下來

以下使用閉區間 $p \le r$，首端 pivot。and 必須由左往右短路求值，先檢查邊界才讀陣列。

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 雙向掃描：不變量與完整交換追蹤

已分類左段 $A[p+1..i-1] \le x$，右段 $A[j+1..r] \ge x$。兩側掃描分別略過嚴格小、嚴格大的鍵；未交錯時交換停住的兩筆，再同時推進。等值鍵不能讓指標永久停住。未知區每輪縮小，最後 pivot 交換到 $j$；回傳兩側為 $[p, j-1]$、$[j+1, r]$。

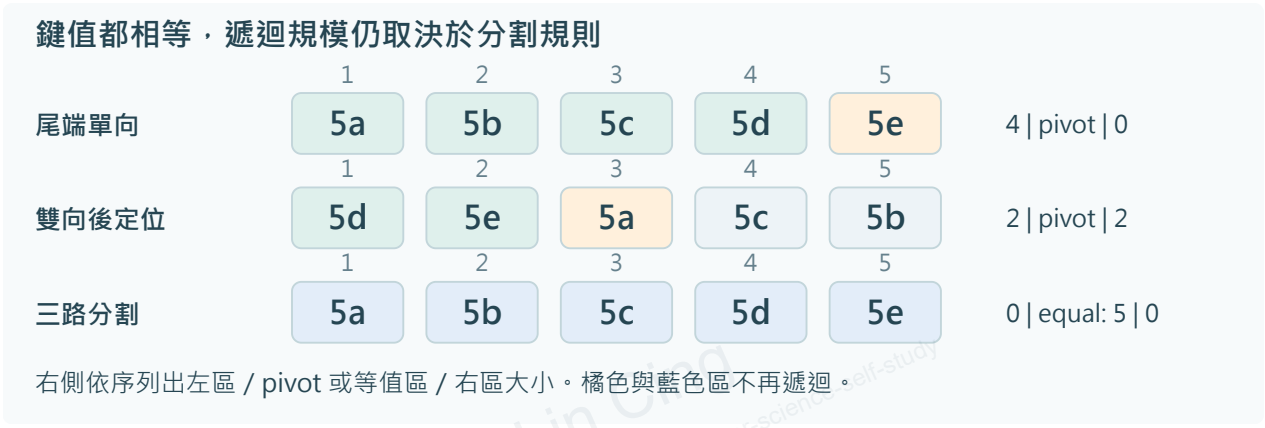
輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，$x = 6$。表中索引是兩側掃描停住、尚未交換的狀態。

| 停住的 $(i, j)$ | 交換位置 | 操作後的完整陣列 |
|---|---|---|
| $(2, 7)$ | $2, 7$ | $[6, 2, 13, 5, 8, 3, 10, 11]$ |
| $(3, 6)$ | $3, 6$ | $[6, 2, 3, 5, 8, 13, 10, 11]$ |
| $(5, 4)$，已交錯 | 收尾 $1, 4$ | $[5, 2, 3, 6, 8, 13, 10, 11]$ |

回傳 $q = 4$，左側皆 $\le 6$、右側皆 $\ge 6$；兩側內部仍不必有序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同樣五個相等鍵，三種分割有三種結果




字母只辨識資料。雙向例依序交換 $2,5$、$3,4$、$1,3$。對全相等輸入族，雙向版每輪線性掃描、兩側約各半，總時間 $\Theta(n \lg n)$；尾端單向版為 $\Theta(n^2)$，三路為 $\Theta(n)$。這些是指定輸入族的成本；雙向版在其他輸入仍可能退化。



## 分類一次，可能做了兩次鍵比較

前面的精確式數的是尾端單向掃描中 `key(A[j]) <= x` 的執行次數，每個非 pivot 鍵恰測一次。三路版本若先測 `< x`，失敗才測 `> x`，一次分類可能做兩次布林比較；遇等值鍵時也會測兩次。

例某輪待分類的鍵為 $[1,4,3]$、pivot 值 $x=3$。只看各鍵與 pivot 的分類判斷，鍵 $1$ 用一次測試，鍵 $4$、$3$ 各用兩次，共五次，而非三次。原地三路版本可能改變處理次序，但每筆的分支測試規則仍要算清楚。

資料結構章節採 0 起算半開區間 $[l,r)$、等值區 $[lt,gt)$；本章三路版本採 1 起算閉區間 $[p,r]$、等值區 $[lt,gt]$。兩者都排除整個等值區，但程式邊界與精確比較常數須各自核對。

鍵互異、pivot 條件均勻時，三路版本的期望時間仍為 $\Theta(n \lg n)$：每輪至多常數倍的線性分類工作，遞迴規模相同。這個漸近結論不表示布林比較數也恰為 $2(n+1)H_n - 4n$。

## 四個秩：比較機率與整次執行的成本不同

四個相異鍵依大小命名為 $z_1, z_2, z_3, z_4$，輸入排列任意。

| 鍵的秩對 | 區間長度 | 直接比較機率 |
|---|---|---|
| $(1,2), (2,3), (3,4)$ | 2 | 各為 $1$ |
| $(1,3), (2,4)$ | 3 | 各為 $2/3$ |
| $(1,4)$ | 4 | $1/2$ |

所以 $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$；代入 $H_4 = 25/12$ 也得到 $2 \cdot 5 \cdot 25/12 - 16 = 29/6$。

單次執行的比較數必為整數。首個 pivot 秩為 $2$ 或 $3$ 時共四次，機率 $1/2$；首選端點、接著三筆中選中位數時共五次，機率 $(1/2)(1/3) = 1/6$；其餘共六次，機率 $1/3$。因此 $4/2 + 5/6 + 6/3 = 29/6$，是對執行成本分布的第二種核對。

相鄰秩必比較，不代表所有鍵對都比較；平均 $29/6$ 也不代表某次真的做了分數次比較。



## 計數排序：從頻率算出每個鍵的區塊

前提：$K \geq 0$ 為整數，每個鍵都在 $0..K$；輸入、輸出位置為 $1..n$。計數表用鍵值當索引，必須能配置 $K + 1$ 格，計數欄位也要能表示 $0..n$。

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

第一輪掃描後 $C[v]$ 是鍵 $v$ 的頻率；累積後是鍵值 $\leq v$ 的資料數，亦即該鍵區塊的**末位置**。後面的回填會消耗計數；若需要保留區間查詢能力，先複製未消耗的前綴表 $P$。

## 末位置回填：逐筆標出寫入位置

取 $A = [2_a, 0_b, 2_c, 1_d, 2_e]$、$K = 2$。鍵 $0, 1, 2$ 的頻率為 $[1, 1, 3]$；累積末位置為 $[1, 2, 5]$。三個輸出區塊依序是位置 $1$、位置 $2$、位置 $3..5$。

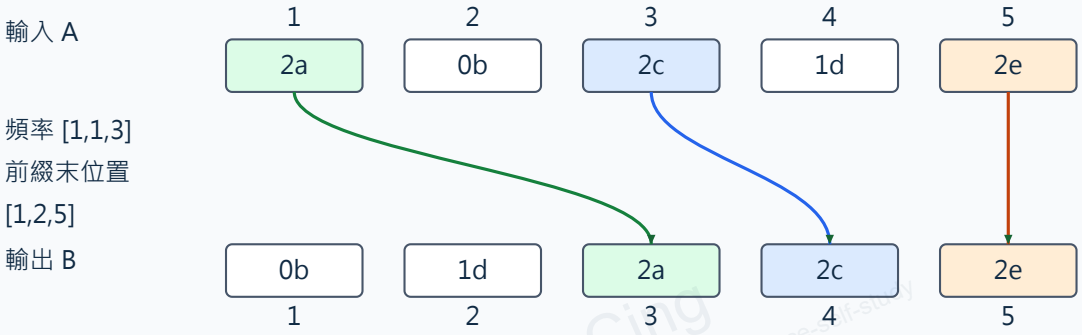
| 掃描位置 $j$ | 資料 | 寫入 $B$ 的位置 | 寫入後的 $C$ |
|---|---|---|---|
| 5 | $2_e$ | 5 | $[1, 2, 4]$ |
| 4 | $1_d$ | 2 | $[1, 1, 4]$ |
| 3 | $2_c$ | 4 | $[1, 1, 3]$ |
| 2 | $0_b$ | 1 | $[0, 1, 3]$ |
| 1 | $2_a$ | 3 | $[0, 1, 2]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同鍵資料的先後順序確實保留下來

### 計數排序：追蹤同鍵資料的寫入位置




反向處理 e→d→c→b→a；放入位置 5→2→4→1→3。

綠a、藍c、橘e是三筆鍵為 2 的資料；輸出仍保持 a→c→e。

輸出 $[0_b, 1_d, 2_a, 2_c, 2_e]$。鍵 2 的身分仍依 $a, c, e$ 出現，這才是穩定性的證據；只看裸鍵 $[0, 1, 2, 2, 2]$ 無法判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 回填不變量、穩定性與成本

固定鍵 $v$，令 $L = P[v-1]$（$v = 0$ 時 $L = 0$）、$R = P[v]$。已處理的輸入後綴中，鍵 $v$ 的資料恰占區塊右端 $[C[v]+1, R]$；$C[v]$ 指向尚未填入的最右位置。

下一筆鍵 $v$ 尚未寫入，所以 $C[v] > L$，寫入合法且不覆蓋既有資料，再減一便保持不變量。終止時 $C[v] = L$，每個區塊恰填滿其頻率大小；區塊依鍵遞增，因此排序正確且資料無增減。

同鍵 $x$ 在輸入先於 $y$ 時，反向掃描先把 $y$ 放到右邊，再把 $x$ 放到左邊，所以穩定。若保留「末位置減一」卻改正向掃描，會反轉同鍵身分；下一節的「起始位置加一」才與正向掃描相配。

四個迴圈分別計初始化、頻率、前綴與輸出，總時間及含輸出的額外空間皆 $\Theta(n + K + 1)$。當 $K = O(n)$ 且 $n \geq 1$ 才是線性；即使只有少數資料，巨大鍵域仍需付表格成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 回填前保存前綴計數，才能回答區間查詢

對鍵值 $0 \leq a \leq b \leq K$，先算鍵值不大於 $b$ 的筆數，再扣除小於 $a$ 的筆數：

$$Q(a, b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

前面五筆資料的三種表格如下：

| 鍵 $v$ | 0 | 1 | 2 |
|---|---|---|---|
| 頻率 $F[v]$ | 1 | 1 | 3 |
| 保存的前綴 $P[v]$ | 1 | 2 | 5 |
| 回填後的計數器 $C[v]$ | 0 | 1 | 2 |

$Q(1, 2) = 5 - 1 = 4$。若誤用回填後的計數器，會得到錯誤的 $2 - 0 = 2$。另有 $Q(0, 0) = P[0] = 1$，實作不必讀取負索引。

必須在回填前複製 $P$；預處理時間為 $\Theta(n + K + 1)$，每次查詢為 $O(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同樣穩定的計數排序，也能從左往右填

末位置版本從右往左掃描，填入後把計數減一。另一個版本先算每個鍵的**起始位置** $S[v]$，從左往右掃描，填入後加一。

設 $F[v]$ 為鍵 $v$ 的頻率、鍵域 $0..K$：

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

初始 $S[v] = 1 + \sum_{u<v} F[u]$，是該鍵區塊的第一格。空鍵區塊可以共用同一起始位置，因為不會有資料寫入它。



## 起始位置的八筆完整追蹤

取 $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$、$K = 6$。鍵 $0..6$ 的頻率為 $[0, 2, 3, 1, 1, 0, 1]$，初始 $S = [1, 1, 3, 6, 7, 8, 8]$。

| $j$ | 資料 | 寫入位置 | 該鍵的下一位置 |
|---|---|---|---|
| 1 | $3_a$ | 6 | $S[3] = 7$ |
| 2 | $1_b$ | 1 | $S[1] = 2$ |
| 3 | $4_c$ | 7 | $S[4] = 8$ |
| 4 | $2_d$ | 3 | $S[2] = 4$ |

## 完成後四筆，核對輸出與最後指標

| $j$ | 資料 | 寫入位置 | 該鍵的下一位置 |
|---|---|---|---|
| 5 | $6_e$ | 8 | $S[6] = 9$ |
| 6 | $1_f$ | 2 | $S[1] = 3$ |
| 7 | $2_g$ | 4 | $S[2] = 5$ |
| 8 | $2_h$ | 5 | $S[2] = 6$ |

輸出 $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$。最後指標可等於區塊末端加一；它表示下一個空位，並不表示這一格還會被存取。



## 同鍵資料沿輸入順序填入同一區塊

$1_b$ 先占位置 1，$1_f$ 再占位置 2；$2_d, 2_g, 2_h$ 依序占位置 3、4、5。顏色連結同鍵區塊，身分標籤用來核對穩定性。

## 已填前綴與下一空位的不變量

令 $S_0[v]$ 是未消耗的起始位置，$c_j(v)$ 是已掃描的前 $j-1$ 筆中、鍵為 $v$ 的資料數。每輪開始時，

$$S[v] = S_0[v] + c_j(v).$$

區塊 $[S_0[v], S[v]-1]$ 恰好裝著已掃描的鍵 $v$ 資料，而且順序與輸入相同；這些區塊互不重疊。

初始化 $c_1(v)=0$，所有區塊為空。讀到鍵 $v$ 時，尚有這筆未放，所以 $c_j(v) < F[v]$，寫入位置仍在 $v$ 的合法區塊內。只寫下一空位再令 $S[v]$ 加一，保持資料與次序；其他鍵的區塊不變。

終止時 $c_{n+1}(v) = F[v]$，每個區塊恰填滿，區塊依鍵排序，故正確且穩定。掃描、計數及起始位置各自都是完整迴圈，時間 $\Theta(n+K+1)$、含輸出的額外空間 $\Theta(n+K+1)$。

**掃描方向與位置定義要成對使用：** 正向配起始位置加一；反向配末位置減一。只改掃描方向，可能仍排對裸鍵，卻反轉同鍵資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LSD 基數排序：穩定性把低位順序保留下來

設每個鍵是固定 $d \ge 1$ 位、基底 $r \ge 2$ 的非負整數；不足位數補前導零。LSD（最低有效位優先）依序處理第 $1, 2, \dots, d$ 位，每輪對目前位做**穩定**排序。

不變量：完成第 $t$ 輪後，序列依最低 $t$ 位代表的數值排序。$t=0$ 無限制。第 $t$ 輪把資料按目前位分組，不同組由該位決定大小；同組內靠穩定性保留前 $t-1$ 位的次序。因此最低 $t$ 位完整有序，$t=d$ 時整體有序。

保留低位順序只發生在**相同高位組內**，不是全域仍依低位遞增。反例：$[12, 11]$ 先按個位得到 $[11, 12]$；十位皆為 $1$，若不穩定地反轉同鍵，便錯誤地回到 $[12, 11]$。

直接對整個序列按百位、十位、個位依次排序，也會把低位變成最後的主序。MSD 必須在同一高位群內遞迴處理，不能只把 LSD 的迴圈倒過來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三輪基數排序，每輪都列出完整序列

百位輪中，205 排在 212 前面，是因為同一百位組內保留了低兩位 $05 < 12$ 的次序。每輪都只比較目前位，完整鍵序由不變量逐步建立。



## 逐輪核對基數排序的完整結果

| 已處理的位 | 完整序列 |
| --- | --- |
| 尚未處理 | $[412, 205, 421, 115, 212, 405]$ |
| 個位 | $[421, 412, 212, 205, 115, 405]$ |
| 個位與十位 | $[205, 405, 412, 212, 115, 421]$ |
| 全部三位 | $[115, 205, 212, 405, 412, 421]$ |

十位輪中，$205, 405$ 的十位皆為 $0$，保留前輪的相對次序。$412, 212, 115$ 的十位皆為 $1$，也保留此前次序；它們的最低兩位依序為 $12, 12, 15$。

## 位元數、每輪基底與成本一起算

每輪用穩定計數排序，時間 $\Theta(n+r)$；$d$ 輪共 $\Theta(d(n+r))$，可重用的額外空間 $\Theta(n+r)$。只有 $d$ 固定且 $r=O(n)$ 才直接得到線性時間。

對 $b \geq 1$ 位元鍵，每輪取 $s$ 位，$1 \leq s \leq b$，則

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) \,.$$

最高一組不足 $s$ 位時可補零。假設一個鍵與一個數位可在常數時間讀取；超過機器字長的大整數，必須另外計取位與搬移成本。

$n \geq 2$ 時取 $s = \min(b, \lfloor \lg n \rfloor)$，便有 $2^s \leq n$，得到 $O(n\lceil b/s \rceil)$；$b = O(\lg n)$ 時是 $O(n)$。

在「每輪成本為 $n+r$」的模型下，32 位元鍵取 $s=8$ 需 $4(n+256)$，取 $s=16$ 需 $2(n+65536)$。後者較小當且僅當 $n > 65024$；等號時相同。這是指定模型的比較，實機還有配置、快取與頻寬成本。



## 分桶是共同技巧，並不代表同一個演算法

| 方法 | 分配依據 | 分配後還做什麼 |
| --- | --- | --- |
| LSD radix sort | 由低位到高位，每次一位 | 每輪對整個序列穩定排序 |
| MSD radix sort | 由最高位開始 | 只在同一高位群內遞迴處理下一位 |
| 區間 bucket sort | $x \in [0,1)$ 映到 $\lfloor nx \rfloor$ | 每桶完成內部排序，再按桶串接 |

三者都可以把資料放進 buckets，但**位數、遞迴規則與成本前提不同**。固定十個數位桶，不等於依 $n$ 分成 $n$ 個等長區間；把每桶直接排序，也不等於繼續按下一位遞迴。

若把 $[312, 305, 121, 118, 214, 207]$ 按百位分組，得到 $[121, 118]$、$[214, 207]$、$[312, 305]$。這一步尚未完成排序：MSD 要在各組處理低位；區間桶方法則須用指定的桶內排序，把每組排好。只串接原分組會得到錯誤次序。

## 區間桶排序：先分配，再把每桶排好

輸入 $n \geq 1$ 個 $x \in [0, 1)$；建立 $n$ 個桶 $B[0..n-1]$，將 $x$ 依序附加至 $B[\lfloor nx \rfloor]$。每桶用插入排序，最後依桶編號串接。空輸入直接返回；$x = 1$ 不在定義域，不能拿來存取 $B[n]$。

正確性不需要隨機分布：若 $x \leq y$，則 $\lfloor nx \rfloor \leq \lfloor ny \rfloor$。不同桶已依不重疊的半開區間排序，同桶則由內部排序完成。依輸入順序附加，再用穩定插入排序，才能另保證穩定性；頭插可能先改變同鍵順序。

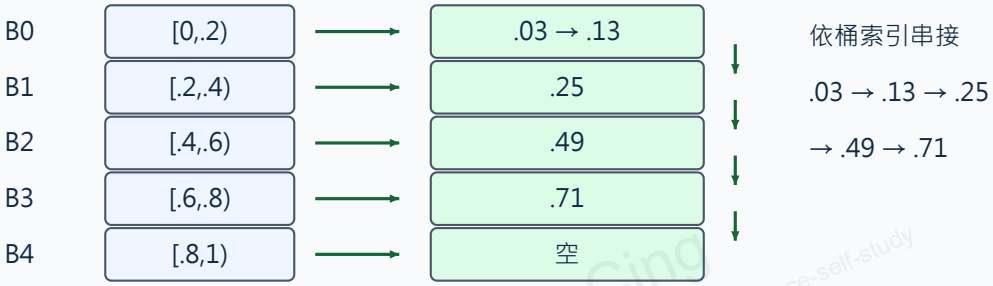
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 五個桶：區間、桶內次序與完整輸出

### 桶排序：區間先決定不同桶的次序

輸入 .13,.25,.03,.71,.49 → 索引 0,1,0,3,2




藍：桶的半開區間；綠：桶內排序結果；跨桶無需再比較。

例 $n = 5$，輸入 $[.13, .25, .03, .71, .49]$，桶編號依序 $[0, 1, 0, 3, 2]$。第 $0$ 桶排成 $[.03, .13]$，第 $1, 2, 3$ 桶分別為 $.25, .49, .71$，第 $4$ 桶為空；輸出 $[.03, .13, .25, .49, .71]$。

## 期望線性：算平方桶大小，而非平方期望

令 $N_i$ 是桶 $i$ 的大小。建立、分配與串接共 $\Theta(n)$，桶內插入排序總成本至多常數倍的 $\sum_i N_i^2$。因此

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

現在才加入輸入假設：各筆鍵**互相獨立且均勻**落在 $[0,1)$。令 $X_{ij}$ 表示第 $j$ 筆落入桶 $i$，則 $N_i = \sum_j X_{ij}$、$X_{ij}^2 = X_{ij}$，且

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

所以 $\mathbb{E}[\sum_i N_i^2] = 2n - 1$，給 $\mathbb{E}[T] = O(n)$；讀取輸入本身給 $\Omega(n)$，合為 $\Theta(n)$。

獨立性用在不同資料 $j, k$ 的落桶事件。不同桶的大小 $N_i$ 彼此相關，因為總和固定為 $n$；這不妨礙期望線性性。僅知 $\mathbb{E}[N_i] = 1$，不能把 $\mathbb{E}[N_i^2]$ 寫成 $1$。



## 分布集中、資料相關與真正的最壞輸入

若各筆仍獨立，但落入桶 $i$ 的機率改為 $p_i$，$\sum_i p_i = 1$，同樣展開得到

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

當 $\sum_i p_i^2 = O(1/n)$ 時仍有線性期望上界。均勻分布給 $1/n$；若所有機率集中單桶，平方桶大小總和就是 $n^2$。

邊際均勻也不夠：先均勻抽 $J \in \{0, \dots, n-1\}$，令每筆 $x_j = (J + 1/2)/n$。各筆對每個桶的邊際機率都為 $1/n$，卻總是全體同桶，$\sum_i N_i^2 = n^2$。這反駁平方成本的錯誤估計；相等鍵的插入排序本身仍可能只需線性時間。

真正達到平方時間的輸入族可取 $n \ge 2$、$x_j = (n - j + 1)/(2n^2)$。它們互異、嚴格遞減，而且全在第 $0$ 桶；插入排序須搬移 $n(n-1)/2$ 次，故最壞時間 $\Theta(n^2)$。

若每桶改用最壞 $O(m \lg(m+1))$ 的排序，總最壞可降為 $O(n \lg(n+1))$；獨立均勻模型仍保留線性期望，因為 $m \lg(m+1) = O(m^2)$。

## 次序統計量與極值：先說清楚要找哪一筆

輸入為 $n \geq 1$ 筆可全序比較的鍵；第 $k$ 小是排序後多重集合的第 $k$ 個值，$1 \leq k \leq n$。重複值各占一個位置，答案卻可相同。例如 $[2, 5, 2, 8]$ 的第 $1, 2$ 小都是 $2$。

本章的下中位數（lower median）取 $k = \lceil n/2 \rceil$。偶數筆時，它是兩個中央值中較小的值，與統計上取兩者平均的定義不同。空輸入與非法秩在入口拒絕。

找最小值可從第一筆開始，依序比較其餘資料；不變量是「目前最小值等於已讀前綴的最小值」，共做 $n - 1$ 次鍵比較。最大值同理。

這也是相異鍵輸入的比較下界：每個非最小值都須至少在一次比較中被證明大於另一個鍵，才能排除最小值資格；一次比較至多排除一個尚未被排除的候選。因此至少 $n - 1$ 次。用相異鍵證明最壞下界，並不要求演算法拒絕重複鍵。



## 成對找極值：八筆資料只做十次比較

同時找最小與最大時，先比較一對資料：較小者只和目前最小值比較，較大者只和目前最大值比較。每對共三次，比兩筆分別挑戰兩個極值少一次。

取 $[14, 3, 9, 1, 8, 5, 12, 6]$。偶數筆先比較第一對以初始化；後面各對維持「兩個極值涵蓋已處理前綴」的不變量。

| 本輪資料 | 組內較小、較大 | 更新後最小值 | 更新後最大值 | 累積比較次數 |
|---|---|---|---|---|
| $14, 3$ | $3, 14$ | 3 | 14 | 1 |
| $9, 1$ | $1, 9$ | 1 | 14 | 4 |
| $8, 5$ | $5, 8$ | 1 | 14 | 7 |
| $12, 6$ | $6, 12$ | 1 | 14 | 10 |

偶數 $n$ 用 $1 + 3(n - 2)/2 = 3n/2 - 2$ 次；奇數 $n$ 先以第一筆同時初始化兩個極值，剩下各對共 $3(n - 1)/2$ 次。統一為 $\lceil 3n/2 \rceil - 2$；$n = 1$ 時為零。平手時仍可任選一方作極值，數值答案正確。

## 為什麼成對法的最壞比較數已經最少

對相異鍵，每筆最初同時具有最小值、最大值兩種候選資格。最後只保留兩種資格各一個，故總共須排除 $2n-2$ 個資格。

對手（adversary）可依下表回答比較。稱未曾比較者為 $U$，只剩最大值資格者為 $H$，只剩最小值資格者為 $L$，兩者都失去者為 $M$。

| 比較對象 | 對手回答與新排除數 |
| --- | --- |
| $U, U$ | 一大一小，各失一種資格；排除 2 個 |
| $H, H$ 或 $L, L$ | 消去一個最大或最小候選；排除 1 個 |
| $H, L$ | 回答 $H > L$；排除 0 個 |
| $U$ 與已分類者 | 若對方是 $H$，令 $U < H$；是 $L$，令 $U > L$；是 $M$，任選一致方向。只排除 $U$ 的一種資格 |
| 涉及 $M$、不涉及 $U$ | 保持 $H > M > L$；$M, M$ 選既有偏序的一個線性延伸，至多排除 1 個 |



## 把候選資格的排除次數算清楚

這些回答維持一致全序：$H$ 從未被證明小於其他鍵，$L$ 從未被證明大於其他鍵，$U$ 沒有比較約束。按 $H > M > L$ 分層，層內採既有比較偏序的線性延伸，不會產生循環。

只有 $U, U$ 比較能一次排除兩個資格，且每次耗掉兩筆未比較資料，至多 $\lfloor n/2 \rfloor$ 次。其他比較至多排除一個資格。若總比較數為 $c$，則

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$

比較前
16 個候選資格
8 × (min + max)

−8 →

四次組內比較後
8 個候選資格
H: 14, 9, 8, 12
L: 3, 1, 5, 6

−6 →

再做六次極值比較後
2 個候選資格
max: 14; min: 1

先集中組內比較：4 × 2 + 6 × 1 = 14 個資格；4 + 3 + 3 = 10 次比較。

## 淘汰賽保存了第二小值的候選名單

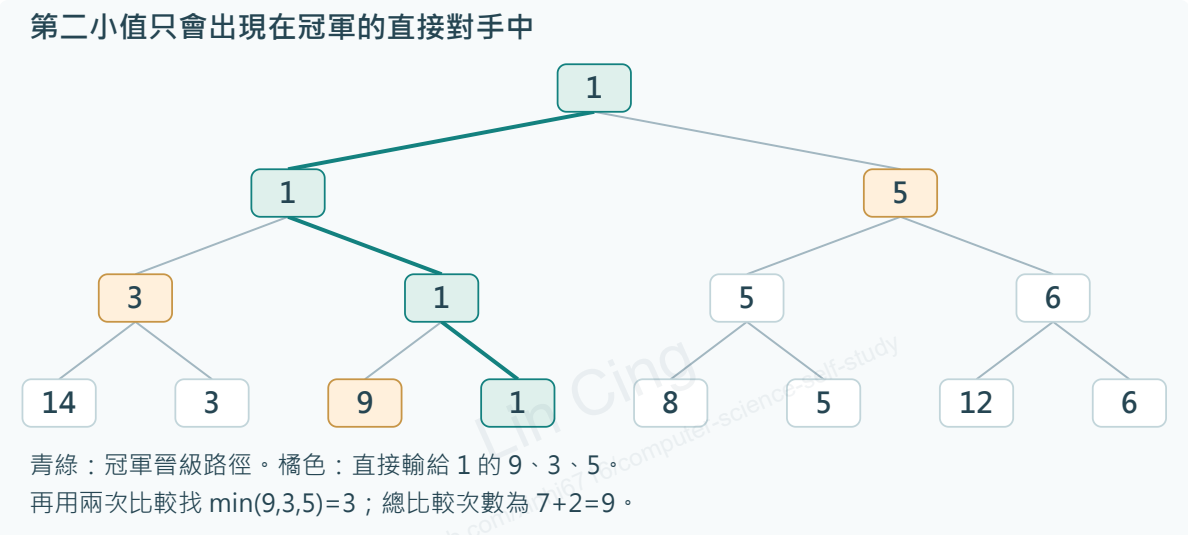
讓較小鍵晉級，並保存每位勝者直接擊敗的對手。第二小值必須直接輸給最小值：若它輸給另一個非最小鍵，已有兩個鍵比它小，產生矛盾。以下先假設鍵相異。

一般 $n \geq 2$ 可用平衡賽程，不足的席位輪空，無須與虛構鍵比較。每次淘汰一人，共 $n-1$ 次；冠軍實際對手數 $d \leq \lceil \lg n \rceil$，清單再比 $d-1$ 次，最壞至多 $n + \lceil \lg n \rceil - 2$。若允許重複鍵，可按 $(key, identity)$ 打破平手，此時求的是第二筆資料，未必是第二個不同值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 八鍵淘汰賽：冠軍直接擊敗了誰




冠軍 $1$ 直接擊敗 $9, 3, 5$，故第二小是這三者的最小值 $3$。淘汰賽共 $7$ 次，候選清單再比 $2$ 次，共 $9$ 次。

## 隨機選擇的三路分割：把等值區一次排除

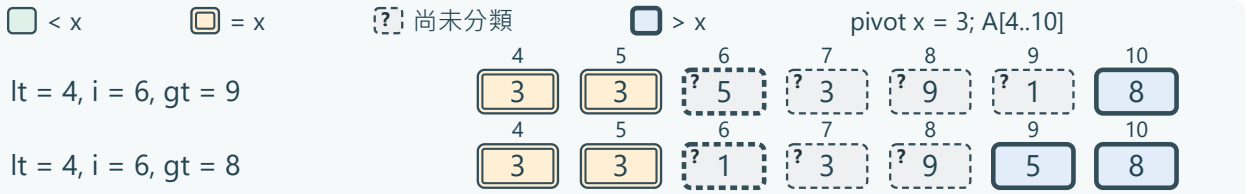
排序會遞迴處理兩側；選擇只保留含答案的一側。本節陣列索引是 **1 起算閉區間** $A[p..r]$；目標 $k$ 是此子陣列內的局部秩，$1 \le k \le r-p+1$。分割比較鍵值、交換整筆資料，$x$ 為子陣列中實際出現的鍵。

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```



## 三路分割：四區不變量與每輪進度

迴圈開始時，$A[p..lt-1] < x$、$A[lt..i-1] = x$、$A[i..gt]$ 未知、$A[gt+1..r] > x$；空區間合法。小鍵分支擴大小區與等值區；等鍵分支擴大等值區。大鍵分支從右端換來的鍵尚未分類，所以不能推進 $i$。每輪未知區長度減一，交換保留多重集合；終止時未知區消失，得到完整三區。




交換 A[6] 的 5 與 A[9] 的 1：gt 減成 8，i 仍為 6。
下一輪必須分類新換來的 A[6] = 1；p = lt = 4，故小於 x 的區間目前為空。

## 隨機選擇：只有往右才要換算局部秩

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



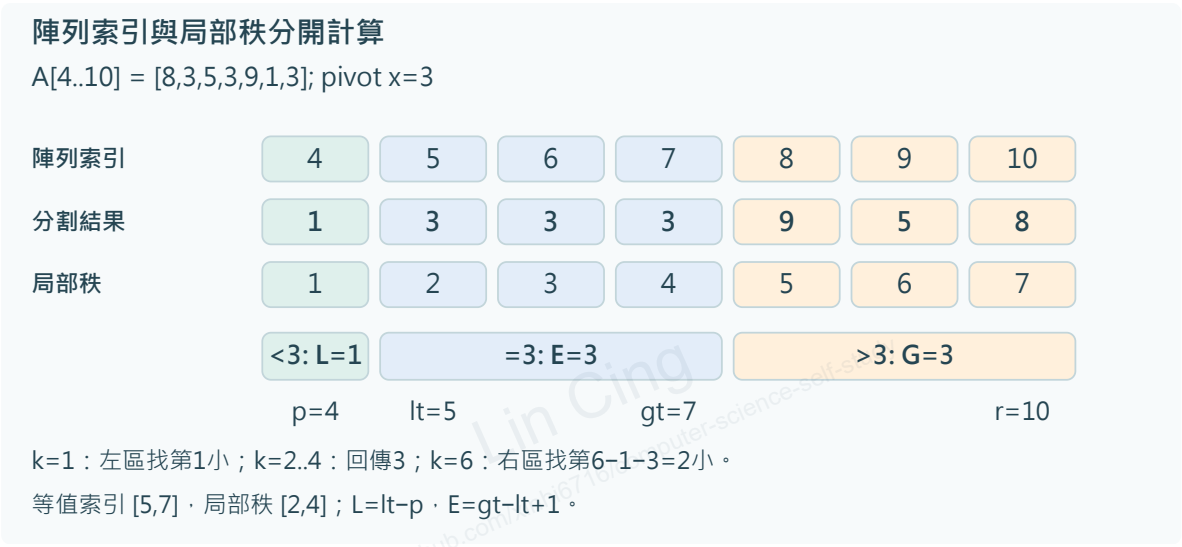
## 選擇迴圈：答案不變，規模持續縮小

$x$ 取自目前資料，因此 $E \geq 1$。答案落在局部秩 $L+1..L+E$ 時直接回傳 $x$；往左保留第 $k$ 小，往右則扣除已確定較小的 $L$ 筆與等值的 $E$ 筆。

迴圈不變量：目前子陣列的第 $k$ 小，等於原問題要求的鍵值。左分支滿足 $1 \leq k \leq L$；右分支滿足 $1 \leq k-L-E \leq r-gt$，故不會保留空區間。每次未返回時，規模至少減少 $E$；最後只剩一筆，回傳該鍵。這同時證明正確性與終止，允許重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 局部秩、區長與絕對索引要分開




$A[4..10] = [8,3,5,3,9,1,3]$，取 $x = 3$，分割後 $lt = 5, gt = 7$，所以 $L = 1, E = 3, G = 3$。局部等值秩為 $2..4$。



# 用區長逐一換算目標局部秩

| 原局部秩 | 判斷 | 下一狀態或答案 |
| --- | --- | --- |
| $1$ | $1 \le L$ | $A[4..4]$ 的第 $1$ 小 |
| $4$ | $L < 4 \le L + E$ | 回傳 $3$ |
| $6$ | $6 > L + E$ | $A[8..10]$ 的第 $6 - 1 - 3 = 2$ 小 |

不能直接比較 $k$ 與 $lt$：前者由目前子陣列的第一筆算起，後者從整個陣列的索引起點算起。若改用絕對目標位置 $t = p + k - 1$，則每輪保留 $t$，不再同時扣局部秩。

## 隨機選擇：對所有後續狀態做強歸納

條件於目前的陣列，pivot 均勻選自其 $n$ 筆資料；未來每輪亦在當時的子陣列內重新均勻選取。先假設鍵相異，pivot 的局部秩 $R$ 均勻分布於 $1..n$。無論目標秩為何，後續規模至多

$$m_R = \max(R-1, n-R).$$

令 $F(A,k)$ 為此狀態的剩餘期望成本。本層掃描與固定開銷至多 $cn$。對**所有較短狀態**假設 $F(B,j) \le a|B|$，條件於本次 pivot 套用歸納，得

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 把所有較大側長度加總，再完成歸納

偶數 $n = 2h$ 時，較大側長度 $h, h+1, \dots, 2h-1$ 各出現兩次，故和為 $2\sum_{t=h}^{2h-1} t = 3h^2 - h$。奇數 $n = 2h+1$ 時，中央秩給 $h$，其餘對稱出現兩次，和為 $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$。除以 $n$ 後均至多 $3n/4$，因此

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

增大 $a$ 以涵蓋 $n = 1$ 基例，即得 $O(n)$。首次分割須掃描 $n > 1$ 筆，故期望為 $\Theta(n)$。

證明沒有假設未知的成本函數單調，也沒有把相依隨機變數的期望拆乘。$3n/4$ 是較大側長度的期望上界，不能當成每次分割的確定上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 重複鍵不破壞三路選擇的期望線性界

為分析替同鍵資料附上固定的身分次序，得到唯一秩 $R \in \{1, \dots, n\}$；實際分割仍只比較鍵，不比較身分。均勻選一筆資料，使分析用的 $R$ 均勻。

若答案在 $< x$ 區，其長度至多 $R - 1$；若在 $> x$ 區，其長度至多 $n - R$；若在等值區便終止。故每種 pivot 身分都有同一上界 $m_R = \max(R - 1, n - R)$，前述對所有較短狀態的強歸納照用，得到期望 $\Theta(n)$。

最壞執行仍可為 $\Theta(n^2)$：相異鍵、找最小值，每輪恰抽到當前最大值，掃描規模為 $n, n-1, \dots, 2$。這條路徑有正機率，期望界不等於最壞界。迭代版加原地三路分割只需 $O(1)$ 額外空間，且會改動輸入。

全等資料則一次掃描就返回。若改用把所有等值都送到同一側的二路 Lomuto，找最小秩時會退化，隨機 pivot 也無法補救。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 確定性選擇：讓完整程序與追蹤使用同一規格

以下列表與排序結果皆為 **1 起算**。SORT-COPY 依鍵排序副本；STABLE-SPLIT-COPY 線性掃描輸入，依序加入小於、等於、大於 $x$ 的三個新列表，保持各區的輸入次序。這是便於自學追蹤的 $O(n)$ 額外空間版本；不把它宣稱為原地程序。

每組至多五筆，取下中位數；組數 $g = \lceil n/5 \rceil$，再對組中位數遞迴找第 $\lceil g/2 \rceil$ 小，作為 pivot。

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 確定性選擇：回傳等值或遞迴單側

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

正確性由分割的三區次序與規模歸納得到。組中位數的遞迴規模 $g < n$；答案側至多 $n-1$，因 $x$ 必出現，等值區非空。兩種遞迴都會停止，時間分析也必須同時計入兩者。



## 五元素組提供的是排除證據，不是精確中位數

### Median-of-medians：每個完整組提供三筆證據

每欄一組，越上方越大；中間一列是組Median。Pivot x=12。

| | G1 | G2 | G3 | G4 | G5 |
|---|---|---|---|---|---|
| | 25 | 24 | ▲ 23 | ▲ 22 | ▲ 21 |
| | 17 | 18 | ▲ 19 | ▲ 20 | ▲ 16 |
| Median→ | ▼ 4 | ▼ 8 | ◆ 12 | ▲ 14 | ▲ 15 |
| | ▼ 2 | ▼ 5 | ▼ 9 | 10 | 13 |
| | ▼ 1 | ▼ 3 | ▼ 6 | 7 | 11 |

組Median [4,8,12,14,15] → 選第3小12；全體真正Median為13。

▼及◆：≤12 共 9 筆　▲及◆：≥12 共 9 筆　◆：共用 pivot

這 $25$ 筆資料恰為 $1..25$。組中位數為 $[4, 8, 12, 14, 15]$，所以 $x = 12$；全體下中位數卻是第 $13$ 小，也就是 $13$。

## 兩側證據，都列出實際鍵值

中位數 $\le 12$ 的前三組，各取其最小三筆，可列出 $[1,2,4]$、$[3,5,8]$、$[6,9,12]$，共九筆 $\le 12$。中位數 $\ge 12$ 的後三組，各取最大三筆，可列出 $[12,19,23]$、$[14,20,22]$、$[15,16,21]$，共九筆 $\ge 12$。

兩份證據在 $12$ 重疊沒有問題：若遞迴找嚴格大側，排除的是全部 $\le x$；若找嚴格小側，排除的是全部 $\ge x$。分析的兩個分支不會同時採用。含重複鍵時仍保證非嚴格的大小關係，三路分割正好把等值區一併排除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 取整與殘組：完成最壞線性時間的歸納

$g = \lceil n/5 \rceil$ 個組中位數的下中位數 $x$，兩側各至少有 $\lceil g/2 \rceil$ 個中位數滿足 $\le x$ 或 $\ge x$。保守扣掉可能不足五筆的殘組，以及含所選 pivot 身分的那一組，剩至少 $\lceil g/2 \rceil - 2$ 個完整組。

每組提供三筆證據，所以兩側各至少有

$$3(\lceil g/2 \rceil - 2) \ge 3n/10 - 6$$

筆非嚴格大小證據，嚴格遞迴側至多 $7n/10 + 6$。小規模下這個下界可能為負，只代表界太鬆；實際等值區非空，子問題仍嚴格縮小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 歸納同時付清兩個遞迴的成本

非遞迴工作至多 $cn$。對所有較短狀態假設成本至多常數 $a$ 乘其長度；$n \ge 140$ 時，

$$\begin{aligned} T(n) &\le a\lceil n/5\rceil + a(7n/10 + 6) + cn \\ &\le (9a/10 + c)n + 7a \le an. \end{aligned}$$

最後一步取 $a \ge 20c$，使 $a/10 - c \ge a/20$，且 $n \ge 140$ 給 $an/20 \ge 7a$。有限的 $1 \le n < 140$ 可增大 $a$ 涵蓋；這是證明門檻，不要求改動程式的五筆基例。初次分組掃描給 $\Omega(n)$，故最壞時間為 $\Theta(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第十六小：逐次列出分組、區長與下一個秩

輸入按以下五組串接，分組前不先排序全體：

$[1,2,4,17,25]$、$[3,5,8,18,24]$、$[6,9,12,19,23]$、$[7,10,14,20,22]$、$[11,13,15,16,21]$。

| 目前 $(n,k)$ | 組中位數 $M$ 與 pivot $x$ | $(\lvert L\rvert,\lvert E\rvert,\lvert G\rvert)$ | 下一步 |
|---|---|---|---|
| $(25,16)$ | $[4,8,12,14,15]\to 12$ | $(11,1,13)$ | 大側第 $4$ 小 |
| $(13,4)$ | $[19,20,16]\to 19$ | $(6,1,6)$ | 小側第 $4$ 小 |
| $(6,4)$ | $[15,16]\to 15$ | $(2,1,3)$ | 大側第 $1$ 小 |
| $(3,1)$ | 基例排序 $[16,17,18]$ | 不再分割 | 回傳 $16$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 由保留的列表，重建下一次分組

第一次大側保留掃描次序，為 $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$。分成五、五、三筆，排序副本後分別是 $[17, 18, 19, 24, 25]$、$[13, 14, 20, 22, 23]$、$[15, 16, 21]$，中位數為 $19, 20, 16$；在這三筆上用基例回傳中位數 $19$。

接下來小側為 $[17, 18, 14, 13, 15, 16]$；前五筆的中位數為 $15$，單筆殘組為 $16$，兩筆取下中位數 $15$。大側為 $[17, 18, 16]$，第 $1$ 小回傳 $16$。這條路徑符合前頁的穩定列表程序；改成原地交換分割，區內次序及後續分組可能不同，不能要求同一條中間路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 改變組大小，時間與空間保證都要重算

固定奇數組大小 $s = 2t + 1$，每個完整組的中位數可提供 $t + 1$ 筆非嚴格證據。忽略取整與常數項時，兩個遞迴規模比例和為

$$\frac{1}{s} + 1 - \frac{t + 1}{2s} = 1 + \frac{1 - t}{2s}.$$

五筆一組時為 $9/10 < 1$，七筆一組時為 $6/7 < 1$，留有線性工作所需的餘裕；有限殘組與歸納基例仍須另算。三筆一組時比例和為 $1$，粗界為

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

此式可證 $O(n \lg n)$，卻不能只憑一個上界便聲稱實際演算法必有 $\Omega(n \lg n)$。下界還須證明相應的壞分割能在各層持續發生。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 副本與尚未返回的呼叫鏈，都要計入空間

本章穩定列表程序的額外空間為 $O(n)$：每次呼叫保存 $O(n)$ 筆副本，兩個遞迴先後執行；對充分大規模，每條尚未返回的呼叫鏈至多按固定小於一的比例縮小，幾何和為 $O(n)$，小規模部分為常數。最壞呼叫深度為 $O(\lg n)$。把分割換成原地版本還不等於整體 $O(1)$ 空間，因為組中位數、副本與遞迴堆疊都必須處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 生日碰撞：用補事件，逐人算不碰撞的機率

模型：$k \geq 0$ 個人各自獨立、均勻選擇 $n \geq 1$ 種日期。這是簡化的機率模型，不直接假設真實生日符合它。

當前 $j$ 人日期都不同，下一人有 $n - j$ 種日期不會碰撞，所以條件機率為 $(n - j)/n$。由條件機率連乘，$k \leq n$ 時

$$\Pr(\text{日期皆不同}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{至少一對同日}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

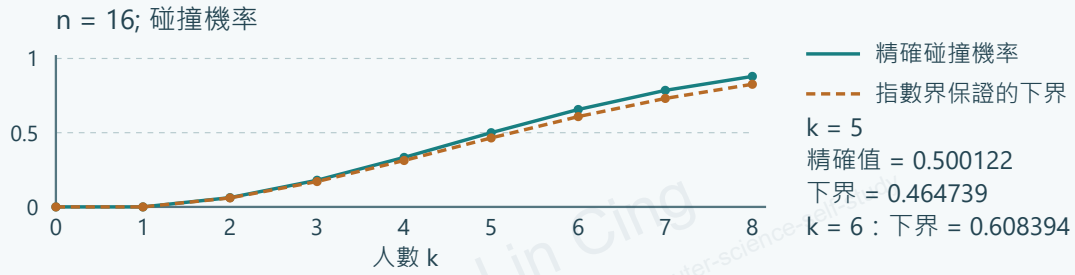
$k = 0, 1$ 無碰撞；空乘積取 $1$。$k > n$ 時由鴿籠原理必碰撞，直接取機率 $1$，不要把連乘繼續延伸成負因子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 充分條件可能比精確門檻保守

對 $2 \le k \le n$，逐項使用 $1 - x \le e^{-x}$，再取補事件：

$$\Pr(\text{碰撞}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$




下界達到 $1/2$ 的條件是 $k(k-1) \ge 2n \ln 2$；這是**充分條件**，並非精確門檻。$n = 16$ 時，精確機率在 $k = 5$ 已超過 $1/2$，這個下界要到 $k = 6$ 才能保證。



## 碰撞對數的期望，不等於發生碰撞的機率

令 $I_{ij}$ 表示第 $i, j$ 人同日。兩人獨立均勻，故

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

碰撞對數 $X = \sum_{i<j} I_{ij}$，所以

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

「有碰撞」是事件 $X \ge 1$。Markov 界只給 $\Pr(X \ge 1) \le \mathbb{E}[X]$，不能反向推論：期望很大，不代表非零機率很大。例如 $X$ 以 $1/M$ 機率等於 $M^2$，否則為零，則期望為 $M$，非零機率卻為 $1/M$。

在生日的獨立均勻模型中，$k = o(\sqrt{n})$ 時由上界得碰撞率趨零；$k \ge C\sqrt{n}$、固定 $C$ 夠大時，由補事件指數界得到固定正的碰撞率。兩個方向合起來，才支持 $\sqrt{n}$ 的機率轉折尺度。

## 十六種日期：把每一個條件機率乘出來

取 $n = 16, k = 5$，逐人累乘不碰撞的機率：

| 加入第幾人 | 本次不碰撞的條件機率 | 到此仍皆不同的機率 |
|---|---|---|
| 1 | $1$ | $1$ |
| 2 | $15/16$ | $15/16$ |
| 3 | $14/16$ | $105/128$ |
| 4 | $13/16$ | $1365/2048$ |
| 5 | $12/16$ | $4095/8192$ |

碰撞機率為 $4097/8192 \approx 0.500122$，碰撞對數的期望則為 $\binom{5}{2}/16 = 5/8$。$k = 5$ 的指數補事件上界為 $e^{-10/16} > 1/2$，尚不能證明碰撞超過半數；$k = 6$ 時才有 $e^{-15/16} < 1/2$。充分條件較保守，與精確答案沒有矛盾。



## 一組樣本的碰撞，與整個模型的平均不同

同一日期樣本與整個機率模型：樣本計數和期望要分開

模型：16日，5人。P(有碰撞)=4097/8192≈0.500122。

E[碰撞對數]=10/16=5/8；這個期望不是上面的事件機率。

圖中的日期為 $[2, 7, 10, 7, 4]$，本次只有第 $2, 4$ 人同日，因此 $X = 1$、$I_{\{X \geq 1\}} = 1$。它們是這一個結果的計數，與對全部結果平均的 $5/8$、$4097/8192$ 不同。

## 兩兩獨立足以算單對，卻不足以連乘全體

令 $U, V$ 獨立均勻於 $\{0, 1, 2\}$，三人的日期為

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

任取兩個日期，皆能唯一還原 $(U, V)$：$B_1, B_2$ 直接還原；$B_1, B_3$ 給 $V = 2B_1 - B_3$；$B_2, B_3$ 給 $U = 2(B_2 + B_3)$，均在模 $3$ 下計算。因此每對日期均勻於九種有序組合，兩兩獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 九種等可能結果，逐一算出碰撞對數

| $U$ | $V$ | $(B_1, B_2, B_3)$ | 碰撞對數 $X$ |
|---|---|---|---|
| 0 | 0 | $(0, 0, 0)$ | 3 |
| 0 | 1 | $(0, 1, 2)$ | 0 |
| 0 | 2 | $(0, 2, 1)$ | 0 |
| 1 | 0 | $(1, 0, 2)$ | 0 |
| 1 | 1 | $(1, 1, 1)$ | 3 |
| 1 | 2 | $(1, 2, 0)$ | 0 |
| 2 | 0 | $(2, 0, 1)$ | 0 |
| 2 | 1 | $(2, 1, 0)$ | 0 |
| 2 | 2 | $(2, 2, 2)$ | 3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 期望相同，碰撞機率仍可能不同

| 模型 | $\Pr(X=0)$ | $\Pr(X=1)$ | $\Pr(X=3)$ | $\Pr(X>0)$ | $\mathbb{E}[X]$ |
|---|---|---|---|---|---|
| 兩兩獨立的構造 | $2/3$ | $0$ | $1/3$ | $1/3$ | $1$ |
| 三個日期完全獨立 | $2/9$ | $2/3$ | $1/9$ | $7/9$ | $1$ |

完全獨立時，$27$ 種等可能三元組中，全異有 $6$ 種、全同有 $3$ 種，恰有一對相同則有 $3\cdot 3\cdot 2=18$ 種：先選哪一對、再選共同日期，最後選另一個不同日期。恰有兩對相同不可能，因為相等具有遞移性。

此模型碰撞率為 $3/9=1/3$；三個完全獨立日期的碰撞率則為 $1-6/27=7/9$。兩者的碰撞對數期望都為 $1$。相同期望不代表相同分布；精確補事件連乘需要更強的聯合假設。



## 球入桶：先固定球數，再問負載與空桶數

$t\ge 0$ 顆球各自獨立、均勻投入 $b\ge 1$ 個桶。固定桶的負載 $N$ 是 $t$ 個指示變數的和，故 $\mathbb{E}[N]=t/b$。恰有 $r$ 球入桶須選定成功的 $r$ 次投球，因此

$$\Pr(N=r)=\binom{t}{r}(1/b)^r(1-1/b)^{t-r},\quad 0\le r\le t.$$

令 $Z_0$、$Z_1$ 分別為空桶數、恰有一球的桶數。對每桶套用指示變數：

$$\mathbb{E}[Z_0]=b(1-1/b)^t,$$

$$\mathbb{E}[Z_1]=b\left[t\frac{1}{b}(1-1/b)^{t-1}\right]=t(1-1/b)^{t-1},\quad t\ge 1.$$

$t=0$ 時直接有 $Z_0=b, Z_1=0$；$b=1$ 的事件直接判定，或以空乘積慣例 $0^0=1$ 解讀邊界式。

例 $t=b=4$：固定桶平均負載為 $1$，但 $\mathbb{E}[Z_0]=81/64$、$\mathbb{E}[Z_1]=27/16$。各桶負載總和固定為四，不獨立；平均一球不代表每桶恰有一球。

## 收集所有桶：每個階段都在等下一個新桶

每次投球獨立、均勻。固定一個桶，令 $T$ 為首次命中所需投球數，含成功的那一次。$\Pr(T > t) = (1-1/b)^t$，$t \geq 0$，故尾和給

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1-1/b)^t = b.$$

四個桶：等到第一次命中指定的第 3 桶

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| 未中 | 未中 | 未中 | 未中 | 未中 | 命中 |

T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

前五次未中，第六次命中；單次等待長度不必等於期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每多收集一個新桶，要再等多久

改問全部桶皆命中。已見 $i$ 個不同桶時，新桶機率為 $(b-i)/b$；等待該次新命中的投球數 $T_i$，條件於目前所見集合，其期望皆為 $b/(b-i)$。未來投球仍獨立且均勻，故不取決於已耗時多久。

總停止時間 $C = \sum_{i=0}^{b-1} T_i$，期望線性性給

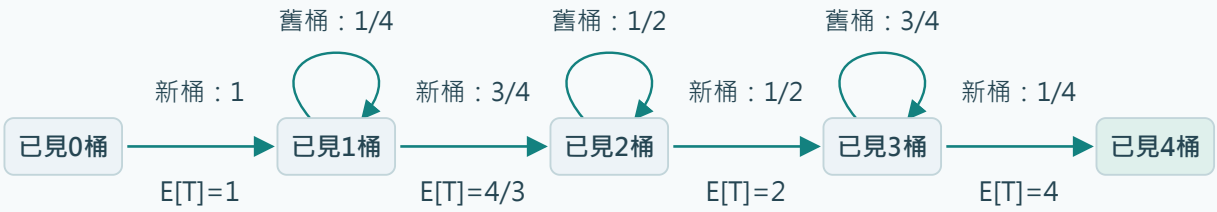
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b-i} = bH_b.$$

$b \geq 2$ 時為 $\Theta(b \ln b)$；$b = 1$ 時恰為一次。這個相加步驟不必另證各階段獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 四桶收集：重複命中時，留在原狀態

### 命中新桶才前進，命中舊桶留在原狀態




實際投球 2,2,4,1,4,3：第1、3、4、6球命中新桶；各階段長度為1、2、1、2。

階段期望相加為 1+4/3+2+4=25/3；上面的單次投球則在第6球集滿。

四桶各階段期望為 $1, 4/3, 2, 4$，合為 $25/3$。圖中實際投球為 $2, 2, 4, 1, 4, 3$，第 $1, 3, 4, 6$ 球命中新桶，各階段實際長度為 $1, 2, 1, 2$，合為六；單次停止時間不必等於期望。



# 第一次碰撞與收集完成：停止條件決定時間尺度

先界收集完成時間 $C$。投完 $t$ 球後，指定桶仍空的機率為 $(1-1/b)^t \le e^{-t/b}$。任一桶空就表示尚未集滿，因此

$$\Pr(C > t) \le \min\{1, be^{-t/b}\}.$$

若整數 $t \ge b(\ln b + c)$、$c \ge 0$，未集滿機率至多 $e^{-c}$。各桶是否為空可以相依；這裡只用了聯集界。

同一串投球，停止條件不同，答案就不同

| t = 1 | t = 2 | t = 3 | t = 4 | t = 5 | t = 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| {2} | {2} | {2,4} | {1,2,4} | {1,2,4} | {1,2,3,4} |

D = 2：首次重複　　C = 6：四桶全部出現

每格下方列出截至當次已見的桶；D 與 C 是同一串投球的兩種停止時間。

## 首次碰撞時間：用尾和證明平方根期望

令 $D$ 為第一次出現重複桶的投球編號。$D > k$ 恰表示前 $k$ 球都不同；$k > b$ 時機率為零，其餘回到生日補事件。對 $k \ge 2$，$k(k-1) \ge k^2/2$，故

$$\begin{aligned}\mathbb{E}[D] &= \sum_{k \ge 0} \Pr(D > k) \\ &\le 2 + \sum_{k \ge 2} e^{-k^2/(4b)} \\ &\le 2 + \int_0^{\infty} e^{-x^2/(4b)}\,dx = 2 + \sqrt{\pi b}.\end{aligned}$$

下界：對整數 $k \le \sqrt{b}/2$，聯集界給碰撞率至多 $k(k-1)/(2b) \le 1/8$，所以前 $\Theta(\sqrt{b})$ 個尾機率至少 $7/8$，得 $\mathbb{E}[D] = \Omega(\sqrt{b})$。

合為 $\mathbb{E}[D] = \Theta(\sqrt{b})$，而 $\mathbb{E}[C] = \Theta(b \ln b)$。同樣是球入桶，問題問「第一次重複」或「全部出現」，答案就不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 連續正面：視窗數與最長長度是不同的量

獨立、公平擲硬幣 $n$ 次，$L$ 為最長連續正面的長度；沒有正面時 $L = 0$。對 $1 \le k \le n$，令 $A_{i,k}$ 表示第 $i$ 到 $i+k-1$ 次全為正面，合法起點為 $1 \le i \le n-k+1$。每個視窗成功率為 $2^{-k}$，故

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n-k+1)2^{-k}.$$

相鄰視窗可能重疊，不能直接當成獨立事件。例如 $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$，因為只需前四次都是正面；兩個邊際機率的乘積卻是 $2^{-6} = 1/64$。期望線性不要求獨立，但求聯集的精確機率需要考慮重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 重疊視窗如何對應同一段連續正面

同一段四連 H 會包含兩個長3視窗，不能當成獨立事件

長3全H視窗：起點1、5、6，共3個。

最長run：5..8，長度4。

重疊視窗可同時成立；第5與第6視窗共享兩次擲幣。

圖中長三的全正面視窗從 $1, 5, 6$ 開始，共 $X_3 = 3$ 個；第 $5..8$ 次的四連正面貢獻了其中兩個視窗。最長長度是 $L = 4$，與視窗數、正面段數都不同。



## 最長連續正面的上界：先求尾機率，再求期望

事件 $\{L \geq k\}$ 是所有長 $k$ 全正面視窗的聯集。對 $1 \leq k \leq n$，聯集界給

$$\Pr(L \geq k) \leq \min\{1, (n-k+1)2^{-k}\} \leq \min\{1, n2^{-k}\}.$$

令 $k_0 = \lceil \lg n \rceil$，其中 $n \geq 1$。由整數尾和及等比級數，

$$\begin{aligned}\mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \geq k) \\ &\leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1.\end{aligned}$$

這裡 $\lg$ 為以 $2$ 為底的對數。若 $k > n$，事件不可能；$n = 0$ 時直接令 $L = 0$，不代入對數。

對 $r \geq 1$ 取整數門檻 $k = \lceil r \lg n \rceil$，可得 $\Pr(L \geq k) \leq n^{1-r}$。例如 $n = 1000, k = 20$ 的精細上界為 $981/2^{20} < 10^{-3}$；$k = 30$ 時可用較鬆的 $1000/2^{30} < 10^{-6}$。這些界控制單次執行出現異常長段的機率。

## 下界：把硬幣切成互不重疊的獨立區塊

對充分大的 $n$，令 $s=\lfloor (\lg n)/2 \rfloor \ge 1$、$m=\lfloor n/s \rfloor$。將前 $ms$ 次切成 $m$ 個長 $s$ 的區塊，尾端不足一塊者不用。每塊全正面的機率為 $2^{-s} \ge n^{-1/2}$；不同區塊使用互不重疊的獨立硬幣，因此這些成功事件相互獨立。

若 $L<s$，所有區塊都必須失敗；反方向未必成立，因為長段可能跨越區塊邊界。因此只有下列不等式：

$$\Pr(L<s) \le (1-2^{-s})^{m} \le e^{-m2^{-s}} \le e^{-m/\sqrt{n}} \longrightarrow 0.$$

由 $m=\Theta(n/\lg n)$，有 $m/\sqrt{n} \to \infty$。再利用非負性，

$$\mathbb{E}[L] \ge s\,\Pr(L \ge s) = \Omega(\lg n).$$

配合上界即得 $\mathbb{E}[L]=\Theta(\lg n)$。這個證明用到了獨立區塊；不能只因重疊視窗的期望很多，就斷言至少一個成功的機率很高。

有限數例：$n=16, s=2, m=8$。至少一個指定區塊為 HH 的機率為 $1-(3/4)^8 = 58975/65536$，故 $\mathbb{E}[L] \ge 117950/65536$。這是可核對的下界，並非精確期望。



## 線上遴選：先固定資訊與決策限制

已知有 $n \ge 2$ 位能力相異的候選人，以均勻隨機排列抵達。每次面試只能與已見者比較；拒絕後不能召回，最多雇用一次。目標為選到全域最佳者，並非讓面試成本最小。

取整數 $1 \le k < n$，前 $k$ 人只觀察、不雇用，記錄最高分 $M$；後段遇到第一位超過 $M$ 者便選取。若始終無人超過，則選最後一人。

第二個迴圈不更新 $M$：在第一個超過門檻者出現以前，其他分數都不高於 $M$；一旦超過便立即返回。最末面試若未通過比較，仍可直接雇用當前第 $n$ 人，並非召回已離開者。$n=1$ 直接選唯一者；$k=0$ 另定義為選第一人。

## 線上遴選：程序與停止條件

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

檢查每位後段候選人之前，先前所有後段分數都沒有超過觀察段最高分 $M$。一旦超過就立即停止；始終沒有超過，才啟用最後一人的備援選擇。



## 成功機率：條件於最佳者的抵達位置

條件於全域最佳者在位置 $i$。若 $i \le k$，它已在觀察階段被拒絕，一定失敗。若 $i > k$，成功恰好要求前 $i-1$ 人中的最高分者落在前 $k$ 位：否則後段已有人先超過 $M$，程序會提早停止；反之，位置 $i$ 就是第一次超過門檻。

最佳者位置均勻，機率為 $1/n$。固定其位置後，前 $i-1$ 人的相對排列仍均勻，因此其中最大者落在前 $k$ 位的條件機率為 $k/(i-1)$。對互斥的位置事件相加，

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

推導使用條件機率，沒有直接假設「最佳者位置」與「先前最大者位置」獨立。

邊界也要單獨定義：$n = 1$ 時成功率 $1$；$k = 0$ 時選第一人，成功率 $1/n$。式中的調和和從 $k \ge 1$ 起才有定義。

## 最佳門檻：有限差分與漸近極限分開求

令 $q_k = \sum_{j=k}^{n-1} 1/j$，並定義空和 $q_n = 0$。因 $q_k = 1/k + q_{k+1}$，對 $1 \le k \le n-2$ 有

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - k q_k}{n} = \frac{q_{k+1} - 1}{n}.$$

$q_k$ 隨 $k$ 嚴格下降，故成功率先增後減。門檻 $k$ 滿足 $q_k \ge 1 \ge q_{k+1}$ 時達最大；等號可能使相鄰門檻並列。$n = 2$ 的唯一正門檻為 $k = 1$，與 $k = 0$ 同為 $1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 最佳成功率為何趨近 1/e

遞減函數 $1/x$ 的積分夾界給

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k,$$

因此 $f(k/n) \le P_k \le f(k/n) + 1/n$，其中 $f(x) = x \ln(1/x)$。由 $f'(x) = \ln(1/x) - 1$、$f''(x) = -1/x < 0$，最大值在 $x = 1/e$，等於 $1/e$。

選離 $n/e$ 最近的合法整數，使 $k/n$ 距 $1/e$ 為 $O(1/n)$；連續可微性給 $f(k/n) = 1/e - O(1/n)$。所有門檻又有 $P_k \le 1/e + 1/n$，故最佳成功率趨近 $1/e$。有限 $n$ 的最佳整數門檻仍應用差分求，不能直接把 $n/e$ 當成精確答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 六人遴選：門檻機率與每次決策都能核對

取 $n = 6$，將各門檻代入 $P_k = (k/6)q_k$：

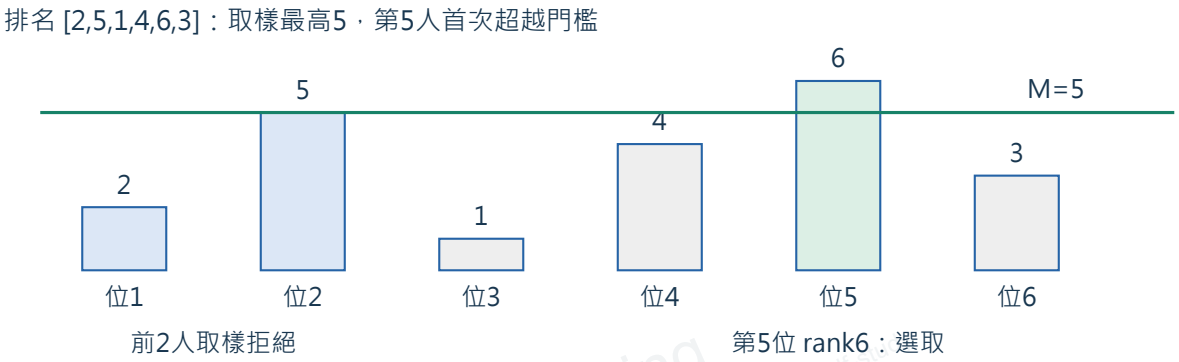
| $k$ | $q_k$ | 精確成功率 $P_k$ |
|---|---|---|
| 1 | $137/60$ | $137/360$ |
| 2 | $77/60$ | $77/180$ |
| 3 | $47/60$ | $47/120$ |
| 4 | $9/20$ | $3/10$ |
| 5 | $1/5$ | $1/6$ |

$q_2 > 1 > q_3$，故最佳正門檻是 $k = 2$；$k = 0$ 成功率只有 $1/6$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從觀察門檻到每一位候選人的決策




排名 $[2, 5, 1, 4, 6, 3]$：前兩人拒絕，$M = 5$；第三人 $1 \le 5$、第四人 $4 \le 5$，均拒絕；第五人 $6 > 5$，立即雇用，第六人無須面試。失敗例 $[5, 6, 1, 2, 3, 4]$ 則在取樣時拒絕了最佳者，後段無人超過 $6$，最後按規則選 $4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 隨機遞迴分割：條件期望與葉數守恆

定義一個額外的成本模型：大小 $n \geq 2$ 的呼叫先輸出 $n$ 個星號，再均勻選 $p \in \{1, \ldots, n-1\}$，處理大小 $p, n-p$ 的兩個子問題；大小 $1$ 輸出一個驚嘆號後停止。每次抽樣在給定先前歷史後仍均勻。

令 $L_n, S_n$ 分別為驚嘆號、星號的期望，$L_1 = 1, S_1 = 0$。固定分割點 $p$ 後，兩側期望相加；再平均所有 $p$，利用 $j \leftrightarrow n-j$ 的對稱，

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

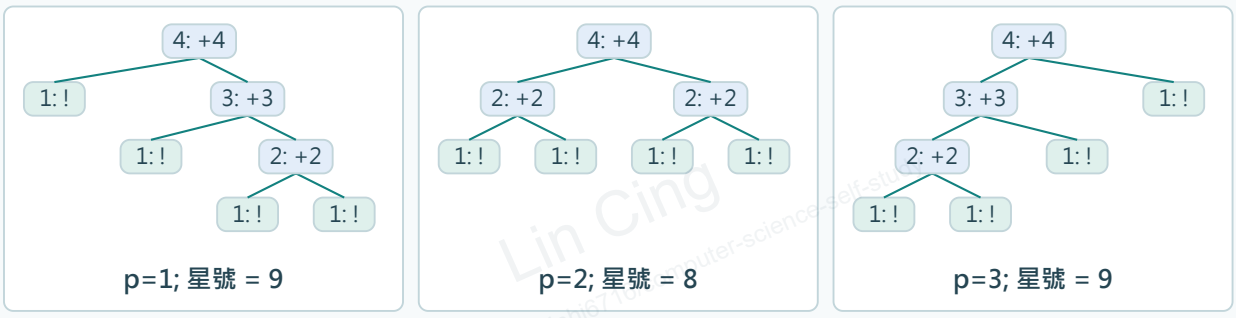
不需要假設兩側成本獨立，因為用的是期望的加法。每條分支的整數規模都嚴格下降，故必然終止。



## 四筆分割：葉數相同，成本卻不同

每次分割保存大小總和，最終每片葉的大小都是 $1$，所以**每次執行都恰有 $n$ 片葉**，不只是期望如此。星號數卻會改變：$n = 4$ 時根成本為 $4$；平衡分成 $2 + 2$，總成本 $4 + 2 + 2 = 8$；分成 $1 + 3$ 時，大小 $3$ 的成本為 $3 + 2 = 5$，總成本為 $9$。因此期望 $S_4 = (9 + 8 + 9)/3 = 26/3$。

三種根分割各機率 1/3；每棵樹都有四片葉



## 解出隨機分割成本：相鄰規模相減

葉數可用強歸納核對：若 $j<n$ 時 $L_j=j$，則 $L_n=\frac{2}{n-1}\sum_{j<n} j=n$。

星號成本先乘開分母：

$$(n-1)S_n=n(n-1)+2\sum_{j<n}S_j,$$

$$nS_{n+1}=n(n+1)+2\sum_{j<n}S_j+2S_n.$$

兩式相減得 $nS_{n+1}-(n-1)S_n=2n+2S_n$，所以 $S_{n+1}=2+\frac{n+1}{n}S_n$。此差分對 $n\ge 2$ 由上式成立，$n=1$ 可由 $S_1=0, S_2=2$ 直接核對。

令 $Q_n=S_n/n$，則 $Q_{n+1}=Q_n+2/(n+1)$。由 $Q_1=0$ 累加，得

$$Q_n=2(H_n-1),\qquad S_n=2n(H_n-1)=\Theta(n\ln n).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



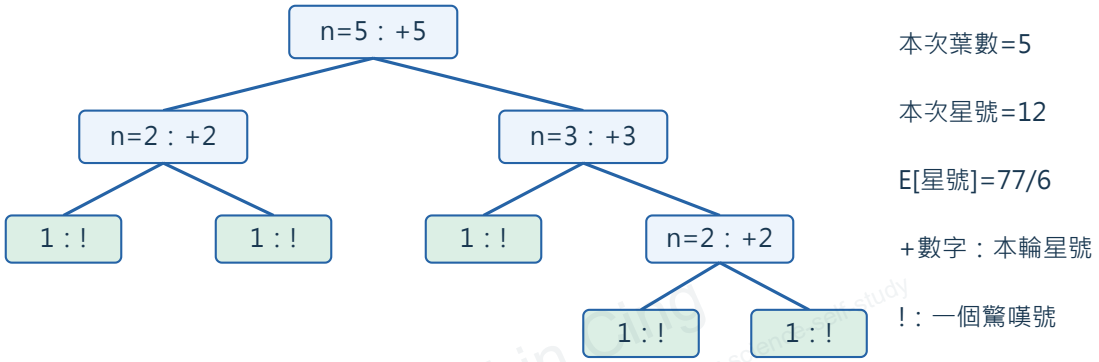
## 用小規模輸入核對封閉式

| $n$ | 每次執行葉數 | $\mathbb{E}[\text{星號數}]$ |
|---|---|---|
| 2 | 2 | $2$ |
| 3 | 3 | $5$ |
| 4 | 4 | $26/3$ |
| 5 | 5 | $77/6$ |
| 6 | 6 | $87/5$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 期望成本與最壞執行仍須分開

五個單位葉完整展開：守恆的是葉數，不是星號成本




圖示 p=2、之後 3=1+2；葉數始終5，但星號總數會隨分割而變。

最壞路徑每次切出 $1$，星號數為 $2+3+\cdots+n=n(n+1)/2-1$；期望的線性對數界並不是每次執行的保證。



## 參考資料與署名

參考用書：Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein，Introduction to Algorithms，第 3 版，第 5–9 章。 補充講義：MIT 6.046J/18.401J，Fall 2005。Lecture 4: Quicksort（Charles E. Leiserson，2005-09-21）；Lecture 5: Sorting Lower Bounds / Linear-Time Sorting（Erik Demaine，2005-09-26）；Lecture 6: Order Statistics（Erik Demaine，2005-09-28）。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

# 資料結構與攤銷分析

資料結構把抽象操作變成可維護的狀態。先問「哪些狀態合法、操作保證什麼」，再問「一串操作總共花多少工作」。

本章依序學習：**表示與操作規格 → 攤銷分析 → 雜湊 → 搜尋樹與擴充 → B-tree → Fibonacci heap → 整數 universe**。

前置知識為第 1 章的規格與不變量、第 2 章的遞迴分析、第 3 章的機率與 heap。這裡先用兩個 stack 和動態陣列，把後面要用的成本工具建立起來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 操作名稱相同，前提可能不同

**抽象資料型態**（ADT）描述可觀察行為；陣列、鏈結與樹則是實作。先固定契約，複雜度才有意義。

| 操作 | 必須先回答 |
| --- | --- |
| DELETE | 給的是 key、陣列索引，還是現成的節點 handle？ |
| SEARCH | 同鍵可否重複？要回傳哪一筆？ |
| POP／DEQUEUE | 空結構回傳錯誤、特殊值，還是根本不允許呼叫？ |
| 指標更新 | 節點物件身分要保留嗎？外部 handle 是否仍有效？ |

雙向鏈結串列刪除已知節點可為常數時間，但「先按 key 找到節點再刪除」通常還要加搜尋成本。後面的每個定理都必須配合它自己的輸入規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 環狀佇列：實體位置不等於邏輯順序

固定容量 $C \geq 1$，使用陣列 $A[0..C-1]$、首筆索引 $h$ 與元素數 $s$。合法狀態滿足 $0 \leq h < C$、$0 \leq s \leq C$，第 $j$ 筆資料位於

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

**空為 $s = 0$，滿為 $s = C$**；不靠兩個相等的索引猜測。下一個插入位置是 $(h + s) \bmod C$，但只有未滿時才能寫入。每個實體格都可使用。

陣列中的舊值不等於仍在 queue 的資料。出列後可清除該格以釋放持有的資源，但成員資格由 $h, s$ 決定。固定容量的 ENQUEUE／DEQUEUE 都是最壞 $O(1)$；配置陣列與元素複製成本須另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先檢查邊界，再更新佇列狀態

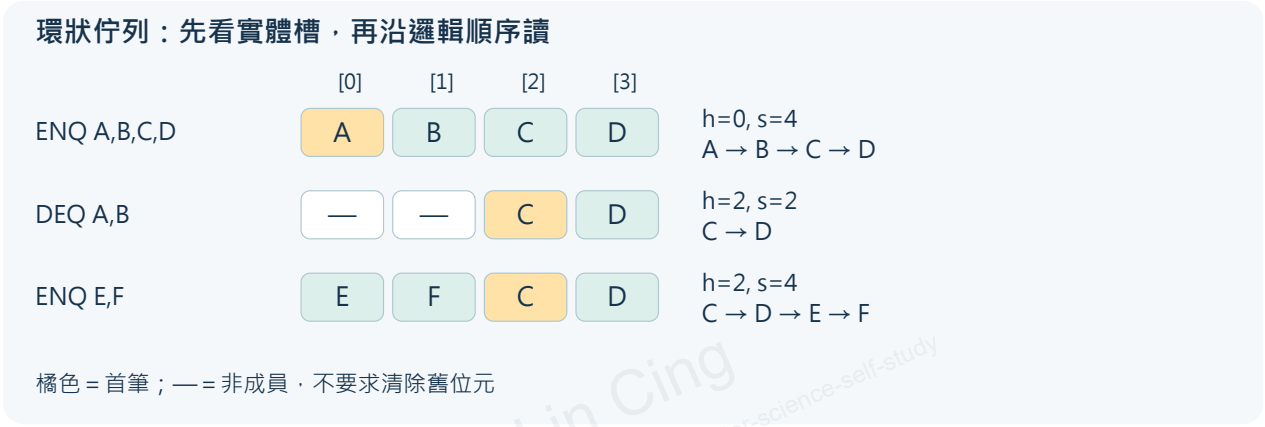
```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

假設元素讀寫成功，操作循序執行。失敗呼叫不改 $h, s$ 或任何有效資料；資料值與錯誤狀態用不同回傳型態區分。此處不是多執行緒 lock-free queue，不能省略同步後直接共用。

## 繞回後，仍然依 C、D、E、F 出列




先放入 A～D，取出 A、B，再新增 E、F。實體為 [E,F,C,D]，但 $h = 2, s = 4$，讀取索引為 $2 \to 3 \to 0 \to 1$。再插入 G 必須拒絕：此時下一個插入位置等於 h，卻是滿態。



## 環狀索引為何維持 FIFO？

插入前 $s < C$，索引 $(h + s) \bmod C$ 不等於任一有效位置 $(h + j) \bmod C$，其中 $0 \le j < s$：若相等，便有 $C \mid (s - j)$，但 $1 \le s - j < C$，矛盾。因此新資料不覆寫舊資料，恰接在邏輯序列尾端。

出列回傳 $A[h] = Q[0]$。更新後 $h' = (h + 1) \bmod C, s' = s - 1$，對 $0 \le j < s'$ 有

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

所以剩餘序列剛好去掉首筆，次序不變。初態 $s = 0$ 合法；成功操作維持不變量，失敗操作保持原態，歸納便涵蓋所有合法呼叫序列。

## 另一種表示法：保留一格，換掉 size

CLRS 第 10.1 節用 head 指首筆、tail 指下一個插入格，保留一格：以本節 0 起算記號，空為 $h = t$，滿為 $(t+1) \bmod C = h$，可存 $C-1$ 筆。

另一常見慣例以 $f$ 指首筆之前、$r$ 指末筆，空為 $f = r$、滿為 $(r+1) \bmod C = f$。兩者換算為 $h = (f+1) \bmod C$、$t = (r+1) \bmod C$。不能只把變數名稱替換，卻保留相同讀寫順序。

本章採 $h, s$，可存滿 C 格；需要時可計算 $t = (h+s) \bmod C$。此時 $t = h$ 同時可能是空或滿，必須查看 $s$。保留一格、額外計數、full 旗標是不同設計，不能混用判斷式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 單向鏈結插入：先保留原本的後繼

給定有效節點 $p$，把尚未加入串列的新節點 $x$ 插在它之後。節點身分固定，key 可重複；$x$ 不得已在同一串列裡。

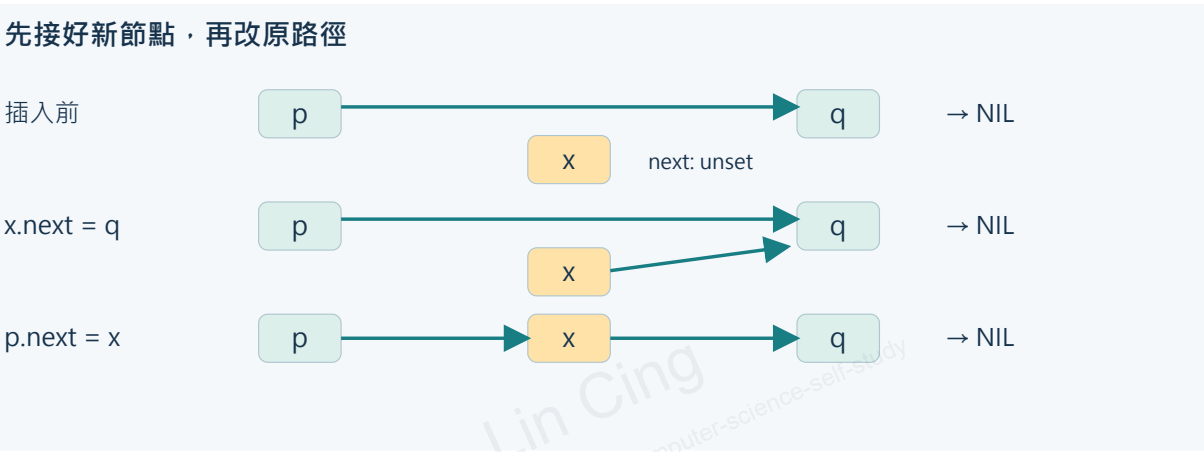
```
q = p.next
x.next = q
p.next = x
```

原路徑 $p \to q$ 變成 $p \to x \to q$，其餘 next 不變，元素只增加 x。若 q 為 NIL，同樣適用；若另存 tail，須把原 tail=p 的情況更新為 tail=x。

若先寫 `p.next=x`，再寫 `x.next=p.next`，會得到 x 指向自己的迴圈，原 q 也可能失去可達性。$O(1)$ 指的是已知 p 的改接；尋找 p、配置 x 及複製非定額資料另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三個狀態看清楚：新增節點如何接回原路徑




先讓 x.next 接到 q，再讓 p.next 接到 x。中間態中，x 還不是由 head 可達的成員；完成後才成為串列的一部分。圖中的節點代號是身分，不以 key 值取代。



## 單向刪除：已知節點不等於已知前驅

刪除 p 後面的節點時，先存 `x=p.next`；若 x=NIL 即無後繼可刪。否則存 `q=x.next`，令 `p.next=q`。若 x 是 tail，令 tail=p；最後才回收 x。只給 x 而不給前驅，刪除尾節點一般需從 head 找前驅，最壞 $\Theta(n)$。

把「把下一筆資料複製到 x，再刪下一筆」當作常數刪除，會改變 x 的資料身分，也不適用尾節點；不符合這裡保留節點身分的規格。

## 原地反轉：先保留尚未處理的後綴

前提是以 NIL 結束、無環的單向串列：

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

若另存 tail，先記 old_head，最後令 tail=old_head。迴圈中 prev 是已處理前綴的反向串列，cur 是尚未處理後綴的首筆；保存 nxt 才能在反向改接後繼續前進。每步少一個未處理節點，故時間 $\Theta(n)$、額外空間 $O(1)$。



## 雙向環狀串列：sentinel 把端點接起來

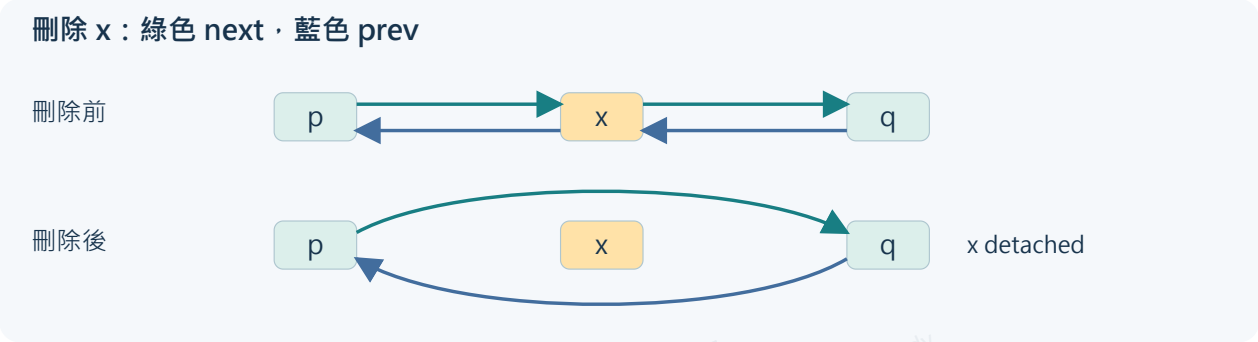
配置固定的虛擬節點 z（sentinel），不算資料元素。空串列滿足 `z.next=z.prev=z`；非空時 z.next 是首筆、z.prev 是末筆。

對每個在環中的節點 v，包含 z，維持 `v.next.prev=v` 與 `v.prev.next=v`。只接受本串列中的有效 handle，禁止刪除 z。檢查陌生 handle 是否屬於本串列的成本，不包含在已知成員的常數改接內。

```
DELETE(x):                  // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

p 或 q 可以是 z，故同一段涵蓋首筆、末筆與唯一資料節點。改接完成才回收 x；外部持有 x 的 handle 隨後失效。這份規格採循序操作，中間指標狀態不可直接視為可供並行讀取。

## 雙向刪除：改兩條鄰居連線，保留其餘路徑




原本 p.next=x、q.prev=x，改成 p.next=q、q.prev=p。其餘成員的相鄰關係不變，x 從環中移除；x 的舊欄位本身不代表它仍是成員。

只有一筆資料時 p=q=z，兩次寫入分別使 z.next=z、z.prev=z，正好回到空態。不要把「兩個指標寫入」誤解為「刪除任意 key 都只花兩步」：先找 key 仍可能掃過整條串列。



## 雙向插入：四個連結都要對得上

把 detached 節點 x 插到已知 p 之後；p 可以是 z。先存 q=p.next：

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

完成後 p↔x↔q，其餘連結不變。原本空時 p=q=z，z.next、z.prev 都會變為 x。若 p 是末筆，q=z，所以 z.prev 自動更新為新末筆。

每條新的雙向相鄰關係都有對應回鏈，故恢復不變量。這裡的寫入順序只保證循序呼叫完成後正確；不提供 concurrent publication 保證。sentinel 簡化邊界，但每個串列多占一個節點，並不改變搜尋的漸近界。

## 用陣列索引表示節點：位置和順序分開

不用原生 pointer，也能用平行陣列 key、next、prev 表示節點；同一索引的三個欄位屬於同一物件。索引 0 保留為 NIL，合法物件使用 1～5。

| 索引 | 狀態 | key | next | prev |
|---|---|---|---|---|
| 1 | free | — | 4 | — |
| 2 | live | B | 3 | 5 |
| 3 | live | C | 0 | 2 |
| 4 | free | — | 0 | — |
| 5 | live | A | 2 | 0 |

head=5，依 next 得 $5 \to 2 \to 3 \to 0$，所以資料順序是 A→B→C；free=1，空格鏈為 $1 \to 4 \to 0$。實體相鄰的索引不是邏輯相鄰的保證。



## 配置與回收：live list 和 free list 不可重疊

固定物件池的槽位分成互斥的三類：live list、free list、呼叫者持有。ALLOCATE 把 free 槽交給呼叫者，初始化並接入後才變成 live。free list 只用 next 欄位，採 stack 方式管理。

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

配置與回收各改常數個欄位，所以在固定物件池內為 $O(1)$；不包含擴充池或配置不定長資料的成本。

## 物件池數例：從配置到回收

沿用前面的五個槽，head 始終為 5：

| 完成的操作 | live 鏈（到 NIL=0） | free 鏈 | 呼叫者持有 |
|---|---|---|---|
| 起態 | 5(A)→2(B)→3(C)→0 | 1→4→0 | 無 |
| ALLOCATE 得 1 | 5→2→3→0 | 4→0 | 1 |
| D 接在 C 之後 | 5→2→3→1(D)→0 | 4→0 | 無 |
| 刪除並 FREE(2) | 5→3→1→0 | 2→4→0 | 無 |

接入 D：key[1]=D、prev[1]=3、next[1]=0、next[3]=1。刪 B：先改 next[5]=3、prev[3]=5，再令 next[2]=4、free=2。

重複 FREE 會破壞 free list；舊 handle 也可能誤指重用後的新物件。索引在範圍內並不足以證明生命週期有效。



## 兩個 stack 如何保持 FIFO

用 $S$ 接收新資料，用 $T$ 提供下一筆出列資料。下列序列記號依指定方向讀取：

$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

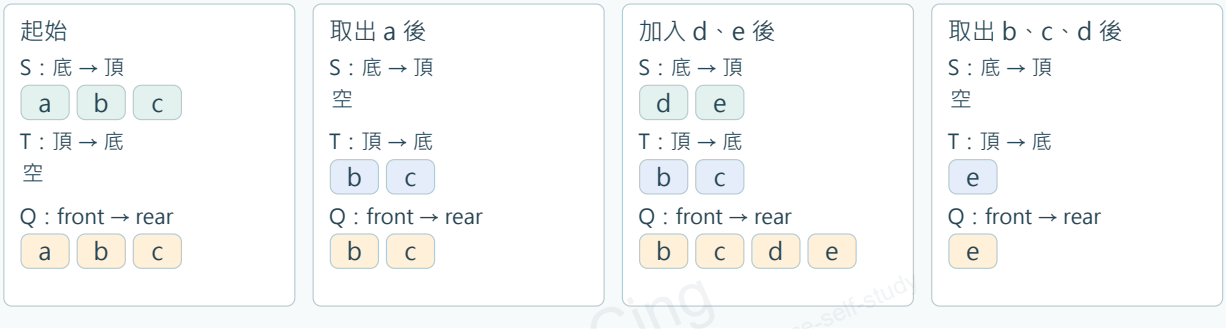
$\parallel$ 表示串接。因此 **$T$ 的 top 是 queue 的 front**；只有 $T$ 空時，才把 $S$ 全部倒進 $T$。

ENQUEUE 把新資料 push 到 $S$，等於接在 $Q$ 尾端。若 $T$ 非空，DEQUEUE pop 它的 top，正好移除 $Q$ 首項。若 $T$ 空，倒轉 $S$ 後，最早進入的資料成為 $T$ 的 top；順序仍等於原來的 $Q$。

兩者皆空時回傳 underflow，狀態不變。**不能每次出列都倒轉 $S$**：$T$ 尚有舊資料時，新資料必須等在它們後面。

## 先看資料順序，再看昂貴的搬移

同一個 queue，兩個 stack：舊資料先出列




圖中的 $S$ 由底讀到頂，$T$ 由頂讀到底。加入 $d, e$ 時，$b, c$ 仍在 $T$ 等待，所以不用搬動。只有 $b, c$ 都已出列，才倒轉 $S$，接著取出 $d$。



## 昂貴一次，不等於每次都昂貴

以一次 stack PUSH 或 POP 為 1 單位；暫不計每次 queue 呼叫固定的判斷開銷。ENQUEUE 成本 1。若 DEQUEUE 需搬移 $k$ 筆，成本是 $2k + 1$：每筆先 pop $S$ 再 push $T$，最後 pop $T$。

從空 queue 出發，每筆資料至多經過四步：**進 $S$ → 離開 $S$ → 進 $T$ → 離開 $T$**。所以 $e$ 次入列、$d$ 次成功出列的總成本至多 $3e + d$。

任意長度 $m$ 的合法操作序列都有 $e + d \leq m$，故總成本至多 $3m$；補回每次固定判斷後仍為 $O(m)$。這個結論沒有假設操作隨機，也不需要「大多數情況便宜」。

單次 DEQUEUE 仍可能是 $\Theta(n)$。攤銷界是整段序列的保證，不能用來宣稱每次呼叫有相同延遲。

## 勢能法：把延後的工作記在狀態上

第 $i$ 次操作把狀態 $D_{i-1}$ 變成 $D_i$，實際成本為 $c_i$。選一個只依狀態決定的函數 $\Phi$，定義攤銷成本

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

$\Delta\Phi > 0$ 表示本次多收一點，為以後存下額度；$\Delta\Phi < 0$ 表示用先前累積的額度支付工作。它不是另一種實際執行時間。

把前 $m$ 次相加，中間勢能成對抵消：

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

若所有可達狀態都滿足 $\Phi(D_i) \geq \Phi(D_0)$，則總實際成本不超過總攤銷成本。常用的充分條件是 $\Phi(D_0) = 0$ 且 $\Phi \geq 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 初始狀態與負攤銷費用，都不能略過

若只證明 $\Phi \geq 0$，卻沒有 $\Phi(D_0) = 0$，正確結論是

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

例如 queue 一開始已有 $n$ 筆資料在 $S$，一次出列就可能搬移 $n$ 筆。不能把這些初始資料當成之前已付費；必須計入建立初態的成本，或保留 $O(n)$ 初始項。

個別 $\hat{c}_i$ 可以為負。只要整體勢能條件成立，望遠鏡等式仍正確；負值表示這次釋放的信用比實際工作還多。

**攤銷**（amortized）針對所有合法序列給總界；**期望**（expected）必須指定亂數或輸入分布。同一資料結構也可能同時有「期望攤銷」界，兩層量詞不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩個 stack 的勢能，只計尚未搬過的資料

令 $\Phi(S,T)=2|S|$，初態兩個 stack 皆空，所以 $\Phi_0=0$。

| 操作 | 實際成本 $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| ENQUEUE | 1 | 2 | 3 |
| $T$ 非空時 DEQUEUE | 1 | 0 | 1 |
| $T$ 空，搬移 $k\geq 1$ 筆後出列 | $2k+1$ | $-2k$ | 1 |
| 空 queue 回報 underflow | 0 | 0 | 0 |

最後一列僅計 stack primitives；若計呼叫開銷，四列各加固定常數。$T$ 中的資料已支付搬移費，不再需要額外勢能。

每個前綴都滿足總成本 $\leq 3e+d$。這與逐筆資料最多做四次 primitive 的 aggregate 證明相符。



## 交錯操作的完整帳目

兩個 stack 均以**底→頂**列出；$\Phi=2|S|$。起終勢能皆為 0，兩種成本均合計 19。

| 操作 | $S$ | $T$ | $c$ | $\Phi$ | $\hat{c}$ |
|---|---|---|---|---|---|
| ENQ a | a | 空 | 1 | 2 | 3 |
| ENQ b | a,b | 空 | 1 | 4 | 3 |
| ENQ c | a,b,c | 空 | 1 | 6 | 3 |
| DEQ → a | 空 | c,b | 7 | 0 | 1 |
| ENQ d | d | c,b | 1 | 2 | 3 |
| ENQ e | d,e | c,b | 1 | 4 | 3 |
| DEQ → b | d,e | c | 1 | 4 | 1 |
| DEQ → c | d,e | 空 | 1 | 4 | 1 |
| DEQ → d | 空 | e | 5 | 0 | 1 |

## 計費法：每個前綴都要付得起

第 $i$ 次操作實付 $c_i$，分析時收取 $a_i$，餘額為

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

從空結構開始，取 $B_0 = 0$。若能對任意合法序列的每個前綴證明 $B_t \geq 0$，便得到 $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$。

**不能先透支，再承諾未來補回**：呼叫者可能在透支當下停止，該前綴的界就已失效。若初態自帶信用 $B_0 > 0$，必須把它列入總成本界，不能當成免費工作。

信用可附在元素上，方便證明它不會被重複花用；若總信用只由目前狀態決定，取 $\Phi(D) = B(D)$，就得到前面的勢能法。



## MULTIPOP：大量刪除的成本由誰支付？

空 stack 支援 PUSH、POP 與 MULTIPOP($k$)，其中 $k \geq 0$；後者最多移除 $k$ 個元素，遇空即停。每次成功 push／pop 的基本成本為 1。

PUSH 收 2：付當次成本 1，另存 1 在新元素上。POP 與 MULTIPOP 收 0，用被移除元素的信用付款。**每個仍在 stack 的元素恰有 1 單位信用**，由歸納可知餘額從不為負。

若 $m$ 次呼叫內有 $p$ 次 PUSH，成功移除總數 $q \leq p$，所以實付 $p + q \leq 2p \leq 2m$。同一物件再次 push 是新一次插入，必須重新付費。

此處只數成功的元素操作。若每次 API 呼叫另收固定的檢查／派送成本 1，總界改為 $m + p + q \leq 3m$；因此空 MULTIPOP 不代表程式完全不用時間。

## Binary counter：先固定寬度與溢位規則

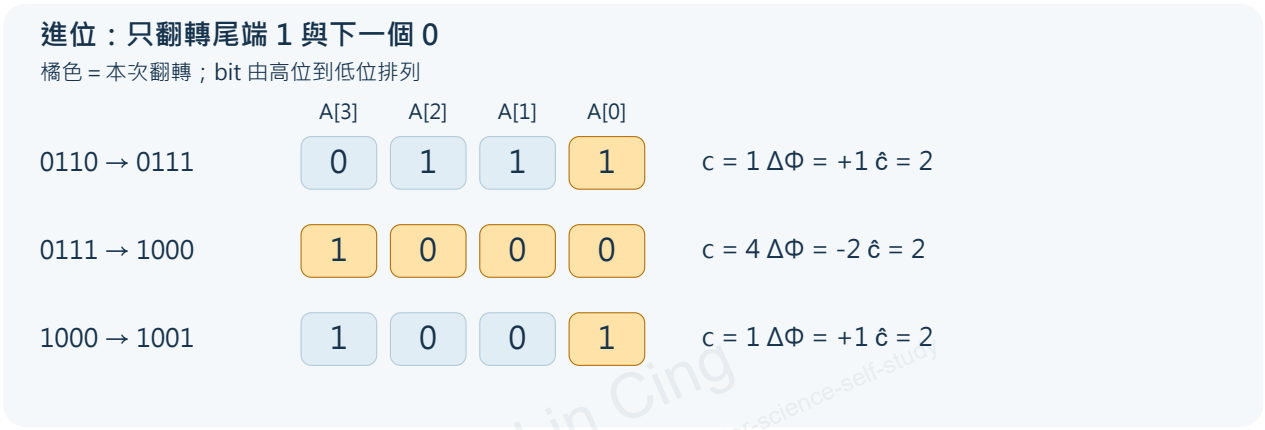
使用固定 $k \geq 1$ 個 bit，$A[0]$ 是最低位，值為 $\sum_{j=0}^{k-1} A[j]2^j$。初值全 0；INCREMENT 計算模 $2^k$ 的加一，故全 1 之後回到全 0。

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

成本 $c_i$ 只計被翻轉的 bit 數。測試與迴圈控制的實際時間為 $\Theta(1 + c_i)$，因此另加每次常數不改變總時間的漸近界。不要把增加位寬的任意精度整數，直接當成這份規格。



## 進位不是同時改掉所有 bit




0111→1000 雖翻轉四個 bit，卻同時用掉先前存下的勢能。下一頁把一般進位與溢位分開計算。

## 一般進位與溢位，分開計算

一般進位若遇到 $t$ 個尾端 1，先把它們清零，再把下一個 0 設為 1：$c = t + 1$。取勢能 $\Phi =$ 目前 1 的個數，便有 $\Delta\Phi = 1 - t$、$\hat{c} = 2$。

溢位是獨立分支：$t = k$，全部清零而沒有新的 1，故 $c = k$、$\Delta\Phi = -k$、$\hat{c} = 0$。不能在這個分支仍寫 $t + 1$ 次翻轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一串加一，三種分析得到同一個界

從全 0 開始做 $m \geq 1$ 次加一。第 $j$ 個 bit 每 $2^j$ 次操作翻轉一次，包含歸零的溢位；故總翻轉數恰為

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

**聚合分析**直接加總各 bit 的次數。**計費法**每次收 2；把 0 設為 1 時，付 1 並留 1 給未來清零，每個 1 都帶有自己的信用。**勢能法**以 1 的個數記錄同一筆信用，普通加一收 2、溢位收 0。

例如 $k = 4, m = 8$：各 bit 翻轉 $8, 4, 2, 1$ 次，共 15；不是 $8 \times 4 = 32$。一般證明涵蓋任意 $m$，這個數例只用來核對計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 完整帳目：從 0000 到溢位歸零

四位元，起態 $\Phi_0 = 0$；bit 字串依最高位到最低位書寫。前 8 次攤銷總額 16，扣除終態勢能 1，實付 15。

| 第幾次 | 操作後狀態 | $c_i$ | $\Phi_i$ | 累積實付 |
|---|---|---|---|---|
| 1 | 0001 | 1 | 1 | 1 |
| 2 | 0010 | 2 | 1 | 3 |
| 3 | 0011 | 1 | 2 | 4 |
| 4 | 0100 | 3 | 1 | 7 |
| 5 | 0101 | 1 | 2 | 8 |
| 6 | 0110 | 2 | 2 | 10 |
| 7 | 0111 | 1 | 3 | 11 |
| 8 | 1000 | 4 | 1 | 15 |



## 完整帳目：最高位也要付清

接續同一串操作。第 9～15 次各收 2，第 16 次溢位收 0。全程實付與攤銷總額均為 30，終態沒有剩餘信用。

| 第幾次 | 操作後狀態 | $c_i$ | $\Phi_i$ | 累積實付 |
|---|---|---|---|---|
| 9 | 1001 | 1 | 2 | 16 |
| 10 | 1010 | 2 | 2 | 18 |
| 11 | 1011 | 1 | 3 | 19 |
| 12 | 1100 | 3 | 2 | 22 |
| 13 | 1101 | 1 | 3 | 23 |
| 14 | 1110 | 2 | 3 | 25 |
| 15 | 1111 | 1 | 4 | 26 |
| 16 | 0000 | 4 | 0 | 30 |

## 哪些改動會破壞常數攤銷界？

**非零初態**：若從 $k$ 個 1 開始，第一次就花 $k$。原勢能仍可用，但只能推出 $\sum c_i \le 2m + \Phi_0 \le 2m + k$；不能省掉初始信用。

**加入 DECREMENT**：對 $k \ge 2$，先從 0 加到 $2^{k-1} - 1$，再反覆交替 INCREMENT、DECREMENT。狀態在 $0\underbrace{11\cdots1}_{k-1}$ 與 $1\underbrace{00\cdots0}_{k-1}$ 間往返，每次都翻轉 $k$ 個 bit。

令準備階段長度 $r = 2^{k-1} - 1$，再做 $2r$ 次交替，總呼叫 $3r$、僅交替部分已花 $2rk$，因此可達 $\Omega(mk)$。每次至多翻 $k$ 個 bit，亦有 $O(mk)$；這個壞序列的成本為 $\Theta(mk)$。

問題在於同一批昂貴進位可以立即被借位反轉，先前「每個 1 預付一次清零」的信用不再足夠。操作集合改了，必須重新證明成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 動態陣列：先把搬移成本算清楚

狀態為 $(n, M)$：$n$ 是有效元素數，$M$ 是容量。從 $(0, 0)$ 開始；首次插入配置 1 格，滿表時先把容量倍增，再加入新資料。

成本只計**寫入新資料 1 次，加上搬移的舊資料數**。配置、釋放與每次判斷的固定開銷暫略。若配置會初始化每一格，必須另計；漸近結論仍可成立，精確常數會改變。

| 第幾次插入 | 插入後 $(n, M)$ | 本次成本 | 累積成本 |
|---:|---:|---:|---:|
| 1 | (1,1) | 1 | 1 |
| 2 | (2,2) | 2 | 3 |
| 3 | (3,4) | 3 | 6 |
| 4 | (4,4) | 1 | 7 |
| 5 | (5,8) | 5 | 12 |

五次插入共搬移 $1 + 2 + 4 = 7$ 筆，**搬移量不一定小於插入次數**。要證明線性總成本，須用完整幾何級數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 只擴張時，aggregate 與勢能給同一總界

插入 $m \geq 2$ 筆，令 $L$ 為小於 $m$ 的最大 2 冪。搬移總數為 $1 + 2 + \cdots + L = 2L - 1$，所以 $T(m) = m + 2L - 1 < 3m$。$m = 0, 1$ 分別有成本 0、1。

勢能取 $\Phi(0, 0) = 0$，非空表取 $\Phi(n, M) = 2n - M$。僅插入的可達狀態滿足 $M/2 < n \leq M$，故勢能非負。

| 分支 | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| 首次插入 $(0, 0) \to (1, 1)$ | 1 | 1 | 2 |
| 未滿，不搬移 | 1 | 2 | 3 |
| 滿表 $(M, M) \to (M + 1, 2M)$ | $M + 1$ | $2 - M$ | 3 |

因此 $T(m) \leq 3m$。**首次插入的攤銷成本是 2**；不能把含 $\log 0$ 的公式直接套用初態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 縮表若太積極，就會反覆搬同一批資料

考慮錯誤策略：滿表時倍增；刪除後使用率一到 $1/2$ 就減半。

取 $k \geq 2$ 為 2 的冪，從滿表 $(k, k)$ 插入一筆，擴成 $(k + 1, 2k)$，搬移 $k$ 筆。再刪除這筆，縮回 $(k, k)$，又搬移 $k$ 筆。之後反覆插入、刪除，每對操作搬移 $2k$ 筆，成本恰為 $2k + 2$。

建好初態需 $O(k)$；接著做 $2r$ 次交替操作，總成本為 $\Omega(kr)$。取 $r = k$，操作次數 $O(k)$，成本卻為 $\Omega(k^2)$。

修正方法是把兩個門檻分開：**滿表才擴張，刪除後至多四分之一滿才縮表**。縮完回到半滿，下一次反向調整前有足夠操作累積信用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 擴縮規格：門檻、空表與資料順序

本例只教 **APPEND 與 DELETE-LAST**。刪除中間元素若要搬動後綴，必須另計該成本。

- 空容量首次 APPEND 配置 1 格；滿表時容量乘 2，再寫入新資料。
- DELETE-LAST 先刪最後一筆。若 $M \geq 2$ 且刪後 $n \leq M/4$，容量減半，搬移剩下的 $n$ 筆；否則容量不變。
- 容量是 2 的冪；用過後最小保留容量 1。空結構刪除回報 underflow，狀態不變。
- 每次成功操作成本 $1+$ 搬移筆數。搬移保持所有存活資料的順序；外部 raw pointer 可能因重新配置失效。

**門檻慣例**：本節採 $n \leq M/4$；原書 §17.4 採 $n < M/4$，在 $M \geq 4$ 時會晚一次刪除才縮表。兩者皆可有常數攤銷界，但不能共用同一張逐步帳目。

在這個規格下，一次操作最多觸發一次 resize。對 $M \geq 4$，刪除觸發縮表時恰有 $n = M/4$；因前一個狀態尚未觸發門檻，而且每次只刪一筆。



## 同一批資料，追蹤到縮表完成

擴縮門檻分開，才能在下一次搬移前存足信用

| | | | 成本 |
|---|---|---|---|
| 滿表 | n=4, M=4 | a b c d | 成本 — |
| APPEND e | n=5, M=8 | a b c d e · · · | 成本 5 = 1 + 4 |
| DELETE e | n=4, M=8 | a b c d · · · · | 成本 1 |
| DELETE d | n=3, M=8 | a b c · · · · · | 成本 1 |
| DELETE c | n=2, M=4 | a b · · | 成本 3 = 1 + 2 |

半滿先保留容量；四分之一滿才縮表，縮完回到半滿。

擴成 8 格後，刪到 4 筆時先保留容量；刪到 2 筆才縮為 4 格。兩個門檻之間的間隔，就是避免 resize 來回震盪的 **hysteresis（遲滯）**。

## 稠密與稀疏狀態，分別存下搬移信用

對 $M \geq 2$ 定義

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

另定義 $\Phi(0,0) = \Phi(0,1) = 0$、$\Phi(1,1) = 1$。兩段在半滿時同為 0，所有可達狀態皆非負。

稠密區的勢能隨插入增加，準備支付擴張；稀疏區的勢能隨刪除增加，準備支付縮表。這不是把同一筆信用算兩次：每個狀態只用其中一段。

下一頁逐一核對分支。$M = 1, 2$ 的空表例外另列，避免把 $M/4$ 當成一定是可達整數元素數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一般容量的四種分支，都有常數攤銷界

下表 $M$ 是操作前容量。未 resize 的分支若跨過半滿邊界，使用兩段公式直接代入。

| 操作情形 | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| 不擴張的插入 | 1 | $-1$ 或 2 | 0 或 3 |
| 滿表擴張 | $M + 1$ | $2 - M$ | 3 |
| 不縮表的刪除 | 1 | $-2$ 或 1 | $-1$ 或 2 |
| $M \geq 4$，刪至 $M/4$ 後縮表 | $1 + M/4$ | $1 - M/4$ | 2 |

最後一列：刪前 $n = M/4 + 1$，勢能 $M/4 - 1$；刪後容量 $M/2$、元素數 $M/4$，正好半滿，勢能 0。

例 $M = 8$：$(3, 8) \to (2, 4)$，搬移 2 筆，$c = 3$；勢能由 1 降為 0，所以 $\hat{c} = 2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最小容量的例外與總成本結論

一般表格之外，再核對以下合法邊界：

| 狀態變化 | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| 首次 $(0,0) \to (1,1)$ | 1 | 1 | 2 |
| 重用空格 $(0,1) \to (1,1)$ | 1 | 1 | 2 |
| 刪空 $(1,1) \to (0,1)$ | 1 | $-1$ | 0 |
| 最小縮表 $(1,2) \to (0,1)$ | 1 | 0 | 1 |

所有成功操作都有 $\hat{c} \le 3$，起態勢能 0、終態勢能非負。因此任意 $m$ 次成功 APPEND／DELETE-LAST 的實際總成本 $\le 3m$。

若把 underflow 呼叫的判斷成本計為 1，其勢能不變、攤銷成本也為 1；總成本仍線性。這個界不適用於任意位置刪除、任意縮容政策，或沒有列入模型的非定額搬移成本。



## Dictionary：先決定同鍵插入的意義

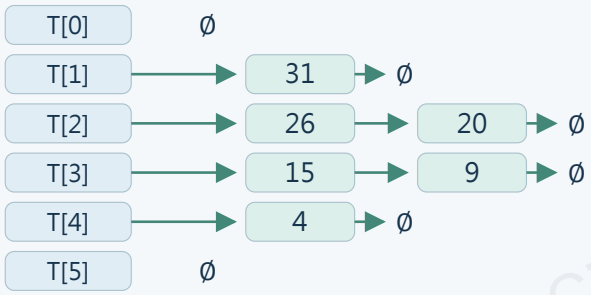
本節維護 key→record 的對應，每個 key 至多一筆；INSERT 遇到既有 key 時更新 value，DELETE 接受 key，找不到則回報不存在。key 的相等判斷與 hash 值相等是兩回事。

若 universe 為 $\{0, \dots, M-1\}$，直接定址表以 key 當索引，三種操作最壞 $O(1)$，空間及一般的全表初始化成本為 $\Theta(M)$。當實際資料只有 $n \ll M$ 筆，才希望把表縮到與 n 相近的大小。

雜湊表使用 $h : U \to \{0, \dots, m-1\}$。$|U| > m$ 時，鴿籠原理保證任何固定 h 都有碰撞；仍須保留完整 key 供核對。以下假設 key、hash 計算與相等比較可在常數個 word 操作內完成；長字串與任意精度整數須另計讀取與運算成本。

## Chaining：同槽資料保留在串列裡

m = 6; h(k) = k mod 6
依序頭插：4, 9, 15, 20, 26, 31




查 20
26 → 20 ✓

查 14
26 → 20 → ø

同桶仍要比完整 key

$m = 6, h(k) = k \bmod 6$，依序頭插 4、9、15、20、26、31。槽 2 為 26→20，槽 3 為 15→9。找 20 檢查兩筆；找不存在的 14 也檢查槽 2 的兩筆。



## 常數頭插，不代表整個 INSERT 都是常數

令 $\alpha = n/m$，chaining 可有 $\alpha > 1$。搜尋掃描 h(k) 指定的整條串列直到找到 key 或到尾端，最壞 $\Theta(n)$。

已知 key 全新時，頭插的改接是 $O(1)$。但本章 INSERT 具有更新既有鍵的語義，必須先搜尋去重，成本為搜尋加常數改接。DELETE(key) 同樣先搜尋；只有持有有效雙向節點 handle 時，才可直接以 $O(1)$ 解鏈。

CLRS §11.2 的常數時間插入與刪除有上述輸入前提：插入假定物件尚未存在，刪除直接給物件。這與本章 dictionary API 不同，並非兩個成本結論互相矛盾。

## 失敗搜尋：把每一筆碰撞的成本加起來

先採 simple uniform hashing：固定相異 key 的槽位獨立且均勻分布。固定不存在的 query q，令 $I_x = \mathbf{1}[h(x) = h(q)]$，則

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

搜尋會檢查這條 chain 的所有記錄，再加常數的 hash 與索引成本，因此期望時間為 $\Theta(1 + \alpha)$。$\alpha = 0$ 時仍要判斷空桶，不是零時間。

線性期望本身不需要碰撞事件互相獨立；上述完整均勻模型是足夠前提，稍後的 universal family 只靠鍵對碰撞上界，就能給出期望時間上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 成功搜尋：平均的是記錄，不是桶

固定相異鍵的插入順序 $x_1, \ldots, x_n$，採頭插、沒有刪除，且成功 query 等機率選其中一筆。找 $x_i$ 要檢查自己，以及晚於它插入的同槽記錄。

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

加入 hash 的常數成本後仍為 $\Theta(1 + \alpha)$。較大的桶含更多可被問到的 key，所以成功查詢並不是等機率選桶。

例如六筆表有兩條長度 2 的 chain、兩條長度 1 的 chain；等機率選 key 的檢查總數為 $(1 + 2) + (1 + 2) + 1 + 1 = 8$，平均 $4/3$。這個確定表的平均，不必等於隨機模型的期望 $1 + 5/12$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 除法與字串 hash：公式不等於分布保證

除法法 $h(k) = k \bmod m$ 很直接；若 $m = 2^p$，只保留最低 p 個 bit，例如 0、8 在 m=8 時都到槽 0。選質數可避開某些輸入規律，但不能保證任意 key 集合均勻。

字串可逐字套 Horner：$H_0 = 0$、$H_{i+1} = (BH_i + c_i) \bmod m$。歸納可知結果等於整個 base-B 整數取模，處理長度 $\ell$ 需 $\Theta(\ell)$ 次定額步驟。

取 B=10、m=7，字元碼序列 [1,2,3] 的中間值是 $1, 5, 4$，對應 $123 \bmod 7 = 4$。若 $B \equiv 1 \pmod m$，結果變成字元碼總和取模，重排字元仍相同；因此正確計算一個公式，不等於它具有低碰撞率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 乘法法：取低 word 裡的高位

$h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$。取 $A = s/2^w, m = 2^p$，則 $r_0 = ks \bmod 2^w$，再取 $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$，其中 $1 \le p \le w$。

寫 $ks = q2^w + r_0$，則 $kA = q + r_0/2^w$，小數部分正是 $r_0/2^w$。乘 $2^p$ 再取 floor，就是把低 word 的最低 w−p 位捨去，留下其中最高 p 位。

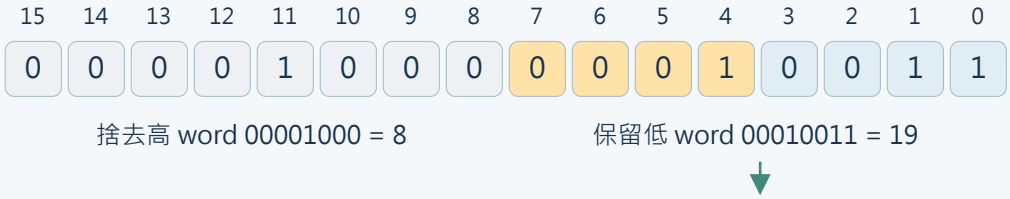
下一頁以八位元完整追蹤：w=8、s=159、k=13、p=4，結果應為 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 乘法數例：完整乘積到槽號

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19




**取低 word 的高四位：0001 → h = 1**

$2067 = 8 \cdot 256 + 19$，低 word 是 00010011，取高四位 0001。不是完整乘積的最高四位；運算必須遵守指定的 word 位寬。



## Universal family：隨機的是函數選擇

有限函數族 H 稱 universal，若對每對固定相異鍵 x、y，均勻抽取 $h \in H$ 時有

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{m}.$$

鍵集合與 query 必須獨立於這次抽樣。這不要求使用者鍵值天生均勻，也不要求所有輸出或碰撞事件互相獨立；更不保證某次抽到的表完全沒有長 chain。

**來源慣例**：本章採 CLRS §11.3.3 的「不超過」。MIT 2005 Lecture 8 的定義寫「恰為」，並用模質數的 dot-product family 示範。恰為的版本也滿足本章定義，但不能把較弱定義反推成相等。

## Universal chaining：完整的期望上界

固定集合 K 與 query q。令 $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$。線性期望與 universal 定義給出

$$E[Y_q] \le \frac{|K \setminus \{q\}|}{m}.$$

q 不存在時，掃描長度期望至多 $n/m$；q 存在時，包含它的整條 chain 長度期望至多 $1 + (n-1)/m$，搜尋不會比掃完整條更貴。所以任意固定 q 的期望搜尋時間為 $O(1+\alpha)$，不需要成功 query 等機率。

若固定操作序列的同時存在筆數一直是 $O(m)$，更新式 INSERT、DELETE(key) 皆先搜尋，總 t 次操作期望 $O(t)$，另加初始化 $\Theta(m)$。若 query 依賴已觀察到的 h 或先前回應而自適應選擇，這份固定鍵對證明不能直接套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Affine family：先證明模質數後的對應

取質數 $P > \max U$；均勻獨立抽 $a \in \{1, \dots, P-1\}$、$b \in \{0, \dots, P-1\}$，令

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

對固定 x≠y，記 $r = (ax+b) \bmod P$、$s = (ay+b) \bmod P$。因 $a(x-y) \not\equiv 0 \pmod P$，必有 r≠s。

反過來，每對 r≠s 都唯一決定 $a = (r-s)(x-y)^{-1} \bmod P$、$b = r - ax \bmod P$。所以均勻的 $(a,b)$ 對應均勻的 $P(P-1)$ 個有序相異 $(r,s)$。P 是質數，才保證非零的 x−y 有乘法逆元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最後取 mod m，碰撞機率仍不超過 1/m

固定 r 後，其餘 P−1 個 s 等可能。與 r 同 mod m 的 s 至多 $\lceil P/m \rceil - 1 \le (P-1)/m$ 個，因此

$$\Pr[h(x) = h(y) \mid r] \le \frac{\lceil P/m \rceil - 1}{P-1} \le \frac{1}{m}.$$

再對 r 取平均，便證明 universal。若 m≥P，取 mod m 不會合併相異的 r、s，碰撞機率為 0。

不等式可由寫 $P = qm + d$、$0 \le d < m$ 核對：d=0 時左側計數為 q−1；d>0 時為 q。m 不必是質數；需要質數的是 P。以 P、乘積模運算能在常數個 word 步驟完成為成本前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## MIT 的 dot-product family 為何也成立？

取質數 m，key 為固定長度向量 $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$。均勻抽係數向量 a，定義 $h_a(x) = \sum_i a_i x_i \bmod m$。

x≠y 時，選一個 $x_j - y_j \neq 0$ 的位置。固定其他係數後，碰撞等價於

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod m.$$

因差值可逆，m 種 $a_j$ 中恰有一種成立，故機率恰為 1/m。這個族的 table modulus m 必須是質數，與前頁 affine family 的任意 table size 不同；長度可變時，hash 計算成本也隨長度改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Open addressing：三種槽位狀態

每槽直接存一筆 record，所以 n≤m。每個 key 的探測序列 $h(k,0),\ldots,h(k,m-1)$ 固定且可重現；要保證有空槽就能插入，這 m 個位置須是全表排列。

| 狀態 | 搜尋動作 | 刪除的影響 |
| --- | --- | --- |
| OCCUPIED | 比完整 key，相等才成功 | 可改為 DELETED |
| DELETED（tombstone） | 繼續探測 | 不是停止點 |
| EMPTY | 判定不存在並停止 | 不能由一般刪除製造 |

EMPTY 是本次建表後未曾使用的槽。若某 key 曾插到更後面，當時不可能跳過 EMPTY；後續刪除只留下 DELETED，才不會切斷搜尋路徑。重建表則重新定義整張表的狀態。



## 搜尋與刪除：最多探測 m 次

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) 先 FIND，找到 j 才把 T[j] 改為 DELETED；不存在則保持原態。key 可以是任何合法值，EMPTY／DELETED 必須有獨立的狀態標記，不能與資料值混淆。

迴圈上限不可省：一張有 tombstone、卻沒有 EMPTY 的表，失敗搜尋必須走完 m 槽後返回；不能無限等待空槽。

## 更新式 INSERT：記住 tombstone，但先查完

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

第一個 DELETED 後面仍可能有同 key；立即插入會製造重複記錄。只有遇 EMPTY 或走完全部 m 槽，才能確認不存在，再使用記住的空位。覆蓋全表是此處的探測前提。



## Tombstone：保留搜尋路徑，再重用空間

m = 7; home(k) = k mod 7

ø = EMPTY; D = DELETED

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |
|---|---|---|---|---|---|---|---|---|
| 插入 1, 8, 15 | ø | 1 | 8 | 15 | ø | ø | ø | 探測：1 → 2 → 3 |
| 刪除 1 | ø | D | 8 | 15 | ø | ø | ø | 查 15：1 → 2 → 3 |
| 更新 8 | ø | D | 8 | 15 | ø | ø | ø | 只更新槽 2 的 value |
| 插入 22 | ø | 22 | 8 | 15 | ø | ø | ø | 探測到 4，再回填槽 1 |

遇到 D 繼續查；確認沒有同 key，才重用 D。

$m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$。插入 1、8、15 時分別占槽 1、2、3。刪 1 後，找 15 仍沿 $1 \to 2 \to 3$，不能在 tombstone 停下。

## 同一份資料的完整探測帳目

承接前圖。最後仍為三筆；INSERT(22) 在槽 4 確認不存在，才回填槽 1。

| 呼叫 | 探測槽 | 寫入／回傳 | 探測次數 |
|---|---|---|---|
| INSERT(1,a) | 1 | 槽 1 存 (1,a) | 1 |
| INSERT(8,b) | 1,2 | 槽 2 存 (8,b) | 2 |
| INSERT(15,c) | 1,2,3 | 槽 3 存 (15,c) | 3 |
| DELETE(1) | 1 | 槽 1=DELETED | 1 |
| FIND(15) | 1,2,3 | 找到槽 3 | 3 |
| INSERT(8,b′) | 1,2 | 更新槽 2，不新增 | 2 |
| INSERT(22,d) | 1,2,3,4 | 重用槽 1 | 4 |
| FIND(29) | 1,2,3,4 | 不存在 | 4 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 探測公式：先確認能否覆蓋全表

Linear probing 用 $(h_0(k)+i) \bmod m$，一定覆蓋 m 槽，但連續的占用區段會吸引更多插入，形成 primary clustering。

Quadratic probing 用 $(h_0(k)+c_1 i+c_2 i^2) \bmod m$，參數不可任意選。例如 m=7、偏移 $i^2$ 的前七項為 $0,1,4,2,2,4,1$，只覆蓋四槽；即使其他槽有空間也可能插不進去。

兩個 key 的 home 相同時，以上兩種方法的整條序列相同，稱 secondary clustering。Double hashing 讓步長也依 key 決定，但仍需證明步長與 m 互質；「多一個函數」本身不是覆蓋證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩種可證的全覆蓋探測

**Double hashing**：$h(k,i) = (h_1(k) + id) \bmod m$。回到 home 等價於 $m \mid id$。設 $g = \gcd(m,d)$，約分後最小正週期是 $m/g$，所以覆蓋全表恰需 g=1。m 為質數可取 1≤d<m；m 為 2 的冪可取奇數 d。

**三角偏移**：對 $m = 2^w$，使用 $f(i) = i(i+1)/2 \bmod m$，先做整數除法。m=8 時為 $0, 1, 3, 6, 2, 7, 5, 4$。

若 f(i)=f(j)，則 $2^{w+1} \mid (i-j)(i+j+1)$。兩因子一奇一偶，故偶因子需被 $2^{w+1}$ 整除。當 $0 \le i, j < 2^w$，有 $|i-j| < 2^w$、$1 \le i+j+1 < 2^{w+1}$，唯一可能為 i−j=0。因此 f 無重複，正好覆蓋全表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Uniform probing 是較強的理想模型

以下期望探測界假設：每個新 key 的探測序列，是相對於目前占用狀態仍均勻的 m 槽排列；無刪除、無 tombstone，且 n<m。它不是每次隨機選一槽的有放回抽樣。

線性、二次與雙重雜湊只有受限的序列集合，不能因為 home 均勻或函數族 universal，就直接宣稱符合 m! 種排列的理想模型。

因此分開三件事：chaining 的碰撞機率、探測是否覆蓋全表，以及探測排列的分布。前兩者成立，仍不足以推出下一頁的期望公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 失敗搜尋界：由尾機率加總

令 X 是遇到第一個 EMPTY 的探測數，$\alpha = n/m < 1$。前 i−1 槽均占用的機率為

$$\Pr(X \ge i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \le \alpha^{i-1}, \quad 1 \le i \le n+1.$$

i=1 為空乘積 1；i>n+1 的機率為 0。各比值不超過 n/m，因 $m \ge n$。由 $X = \sum_{i \ge 1} \mathbf{1}[X \ge i]$ 得

$$E[X] = \sum_{i \ge 1} \Pr(X \ge i) \le \sum_{i \ge 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

新 key 插入的探測同樣適用。m=10、n=8 時上界為 5；它不是每次恰好五次，也不是 linear probing 的無條件保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 成功搜尋界：重現插入時的路徑

固定 n 筆相異鍵的插入順序，無刪除或搬移；成功 query 等機率選記錄。第 i+1 筆插入時只有 i 筆，期望探測至多 $m/(m-i)$；日後搜尋重現其插入路徑。

$$E[C] \le \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$

$$\le \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

因 1/x 遞減，每段 [k−1,k] 的積分至少 1/k。$\alpha = 0$ 時沒有成功 query，不能直接代入 1/α；右式在 α→0 時極限為 1。這裡同時需要理想探測模型與等機率成功查詢，少一項都不能沿用完整公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 低活躍負載，也可能搜尋整張表

m=7，先插入 home 分別為 0～6 的七筆，再全部刪除。此時 n=0、α=0，但七槽全為 DELETED，找任意不存在的 key 都要探測 7 次。

因此不能把無 tombstone 的 $1/(1-\alpha)$ 界套到這張表。可另追蹤 $\beta = (n+d)/m$，d 為 tombstone 數；本例 β=1，才反映沒有 EMPTY 可提早停止。β 本身也不是任意實作的完整成本證明。

重建會清除 tombstone，但必須列入配置、重新計 hash 與插入的成本；在 open addressing 中，重新插入本身也需要探測。若要主張重建的期望攤銷界，還需固定負載門檻、隨機模型與重建規則，不能只說「定期 rehash」就省略成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Perfect hashing：把隨機重試放在建表

FKS 兩層法處理固定的 n 個相異 key。建表後每次查詢最壞 $O(1)$ 個 word 操作，空間 $O(n)$；建表採隨機重試，只有期望線性成本，不能稱最壞線性。

n=0 時直接建空 dictionary，不做 mod 0。以下 n≥1，第一層使用 m=n，均勻抽 universal h。桶 j 大小為 $n_j$，準備配置 $n_j^2$ 個次層槽。

令 Y 為碰撞的無序鍵對數。對每桶用 $n_j^2 = n_j + 2\binom{n_j}{2}$，可得

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

平方空間和的控制來自碰撞對數，不是宣稱每個桶都一定很小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第一層：真的檢查空間，再接受

令 $S = \sum_j n_j^2$。Markov 不等式給出 $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$。每次獨立重抽，直到實際計算得到 S<4n。

每輪只需清空 n 個桶的計數並掃過 n 個 key，成本 O(n)；先檢查 S，**不要在不合格的輪次先配置所有平方表**。接受機率大於 1/2，所以期望試驗輪數少於 2，第一層期望成本 O(n)。

接受後有確定的 S<4n。這比只知道 E[S]<2n 更強：最終成品確實占線性空間，而不是把「期望線性」誤寫成每一次都已線性。極端壞抽樣仍可能重試很多次，不能把期望輪數當硬上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第二層：平方槽數換取無碰撞

條件於已接受的第一層分桶。桶內 s≥2 個 key，配置 $s^2$ 槽，獨立抽 universal $h_j$。碰撞對數 Z 滿足

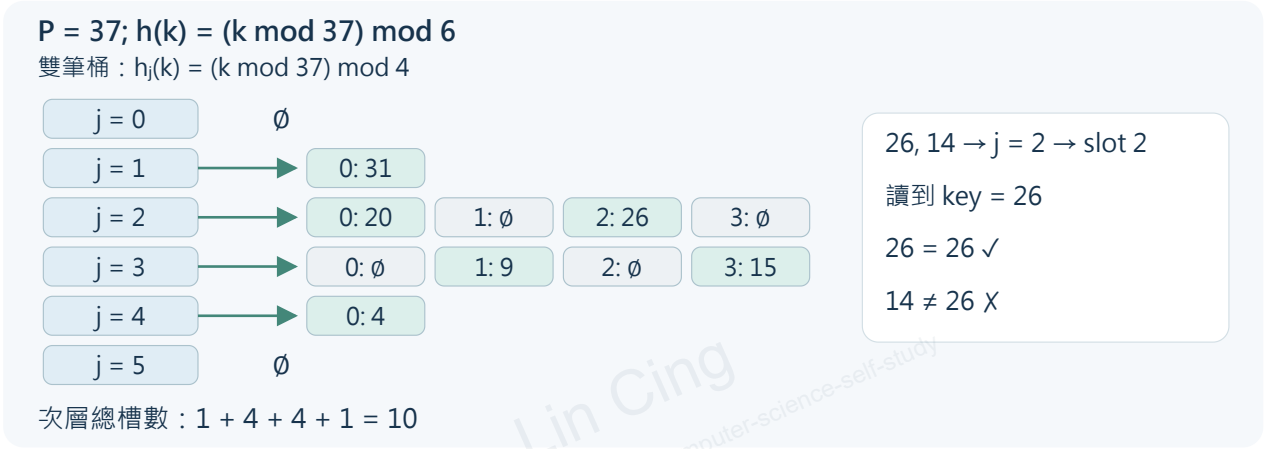
$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

真的檢查無碰撞才接受。每輪清空 $s^2$ 槽、插入 s 筆共 $O(s^2)$，期望少於兩輪。對固定的合格分桶加總，期望成本為 $O(\sum_j s_j^2) = O(n)$；再對第一層取期望，總建表仍 O(n)。

空桶不配置次層；單筆桶可直接存記錄。存每桶大小、hash 參數與指標只另需 O(n)。查詢最壞界依賴定額 hash 與 key 比較，增加新 key 則超出這份靜態規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩層配置：同一組鍵值完整落到槽位




$K = \{4, 9, 15, 20, 26, 31\}$、P=37、$h(k) = (k \bmod 37) \bmod 6$。桶大小 [0,1,2,2,1,0]，平方和 10<4n=24。兩個雙筆桶用 $h_j(k) = (k \bmod 37) \bmod 4$，分別放到槽 (0,2) 與 (1,3)，實際檢查都無碰撞。



## 沒有存入的 key，仍可能撞到已占用槽

查 26：第一層算得 2，次層算得 2，讀到 key=26，相等才成功。查 14：同樣走 2→2，讀到的卻是 26，所以必須回不存在。

查 5：第一層到空桶 5，直接回不存在，不計算不存在的次層函數。查 10：第一層到單筆桶 4，讀到 4≠10，仍回不存在。

**Perfect 只保證已存集合內不互撞，不替未存鍵提供成員資格證明。** Lookup 必須先處理空桶，再檢查次層槽是否占用，最後比完整 key。建表成功後，無論查詢是否存在，都只走常數層；這才是查詢最壞常數時間的保證。

## BST 的限制來自所有祖先

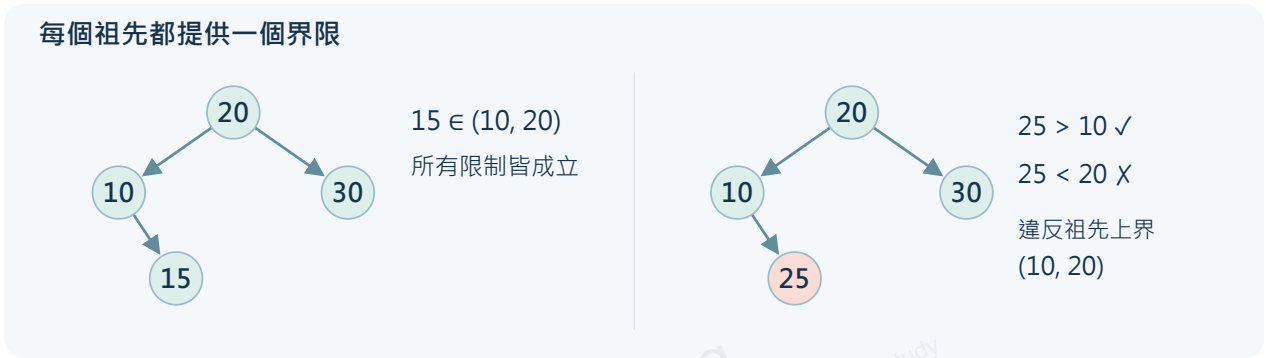
BST（binary search tree，二元搜尋樹）把比較結果變成分支。本節延續唯一 key 的 dictionary：每個節點存 key、value、left、right、parent；空連結為 NIL，root.parent=NIL。同 key 插入只更新 value。

**每個節點的整棵左子樹都小於它，整棵右子樹都大於它。** 只檢查父子大小並不足夠。往左把允許區間的上界縮到目前 key，往右則更新下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 祖先區間：局部大小正確仍可能違規




CLRS §12.1 也容許重複 key，使用左側 ≤、右側 ≥ 的性質。本節採嚴格次序；若要存重複資料，可另存 count 或以 (key,唯一編號) 排序，不能直接混用兩套規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 搜尋：每次比較排除一整棵子樹

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // 三向比較：小於、等於、大於
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

迴圈不變量：若 k 存在，必在目前 x 的子樹中。比較不等時，BST 的全域次序排除另一側；每次嚴格往下一層，有限樹必終止。到 NIL 表示候選子樹已空。

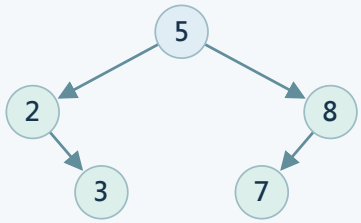
深度 d 按 root 到節點的**邊數**算，root 深度為 0。成功搜尋深度 d 的節點，訪問 d+1 筆；失敗搜尋只計遇到的非 NIL 節點。以下一次三向 key 比較算一次；NIL 測試與比較器內部指令不混入這個數。

高度 h 按最長路徑的邊數算，空樹高度 −1；非空樹搜尋為 O(h+1)，空樹另需 O(1)；統一可寫 O(h+2)。



## 同一棵樹的成功與失敗搜尋

**深度從 0 起；成功比較數 = 深度 + 1**




d = 0　查 7：5 → 8 → 7，3 次

d = 1　查 6：5 → 8 → 7 → NIL

d = 1　失敗也比較 3 次

d = 2　查 9、10 都在 8 的右側失敗

查 7 走 5→8→7，共三次 key 比較；查 6 也走同樣三個節點，最後才到 NIL。查 9 或 10 則走 5→8→NIL，只比較兩次。

成功與失敗都要實際數訪問節點。到 NIL 的最後一次指標測試，不是額外一次 key 比較。

## 平均成本：先決定查詢怎麼抽

若只在五個已存 key 中等機率查詢，成功比較總數為 1+2+2+3+3=11，平均 **11/5**。若 query 改為整數 1～10 均勻抽取，還要加上五個失敗查詢。

| query | 1 | 4 | 6 | 9 | 10 |
|---|---|---|---|---|---|
| 訪問 key | 5,2 | 5,2,3 | 5,8,7 | 5,8 | 5,8 |
| 比較數 | 2 | 3 | 3 | 2 | 2 |

因此混合平均為 (11+12)/10=**2.3**。失敗間隙不是等機率事件：區間 (8,+∞) 在此包含 9、10，權重 2/10；不能把每個 NIL 連結各算同一權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 中序走訪與極值

中序（inorder）依「左子樹 → 目前節點 → 右子樹」列出 key。歸納假設兩個子樹各自輸出遞增序列；BST 性質又讓左側每個值 < 根 < 右側每個值，因此串接後仍遞增。每個節點處理一次、空連結測試 O(n) 次，總時間 Θ(n+1)，遞迴堆疊 O(h+2)。

找 MINIMUM(x) 時，前提為 x≠NIL；沿 left 直到沒有 left child。若還有左子樹，最小值一定在其中；沒有時，目前 key 小於所有右子樹 key，故答案就是 x。MAXIMUM 對稱。空 dictionary 的極值操作應直接回 absent。

**用 BST 排序，不能只算最後的走訪。** 依序插入 1,2,⋯,n 會得到右鏈；建樹比較數為 0+1+⋯+(n−1)=n(n−1)/2，再加 Θ(n) 的走訪，仍是 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 後繼：有右子樹就向下，否則向上找

```
SUCCESSOR(x):                    // x 是有效節點
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                    // NIL 表示沒有後繼
```

有右子樹時，其中最小 key 就是下一個；它比 x 大，且不大於任何其他右子樹 key。沒有右子樹時，向上經過「右孩子 → 父」所遇到的父鍵都更小；第一次由左側回到的祖先才是最小的較大 key。

在樹 5(左 2、右 8)，2.right=3、8.left=7 中：succ(5)=7，路徑 8→7；succ(3)=5，路徑 3→2→5；succ(8)=NIL。PREDECESSOR 交換 left/right 即得。

每次呼叫只沿單一路徑，最壞 O(h+1)。從最小值開始反覆找 successor 可列出全部節點；每條樹邊至多向下、向上各經過一次，整段是 O(n)，不是 n 次最壞界相乘後的必然 Θ(nh)。



## 插入：找到的是空連結，不是任意葉節點

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

迴圈保留 k 的祖先區間；到 NIL 後，新節點恰好落在所有祖先允許的範圍內。既有 key 的分支只改 value，不改 key 或節點身分；在樹內任意改 key 可能破壞次序。

最壞 O(h+2)，包含空樹插入；新增節點空間 O(1)。

## Transplant：先換父節點的連結

```
TRANSPLANT(T,u,v):              // u 必須是樹內有效節點
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

此程序只把「指向 u 的連結」改成指向 v，並更新非空 v 的 parent；**不會替呼叫者接好 v.left、v.right**。v=NIL 合法，但本節的 NIL 不是可寫入 parent 的節點。

刪除 z 若沒有左孩子，直接 TRANSPLANT(T,z,z.right)，也包含 z 是葉節點的情況。若只有左孩子，則改用 z.left。所有被保留的子樹仍滿足 z 的祖先區間。

例如 root=20 只有右孩子 40：移接後 root=40、40.parent=NIL。若 20 是最後一筆，則 root=NIL。這些分支都只改常數個連結；DELETE(key) 尚須先付搜尋成本。

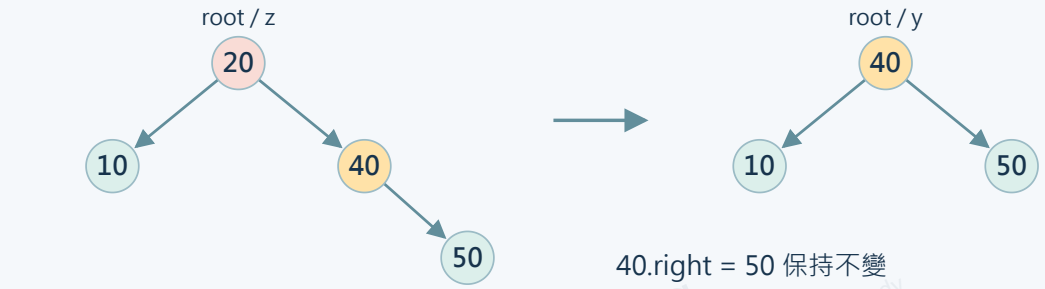


## 兩個孩子：先把後繼從原位取出

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

y 是右子樹最小值，所以沒有左孩子，但**可能有右孩子**。深處的 y 要先由 y.right 接替，才能把整棵原右子樹接到 y。若 y 就是 z.right，則保留原本 y.right；不可執行 y.right=z.right，否則會得到 y.right=y 的自環。

## 後繼就是右孩子：保留它的右連結

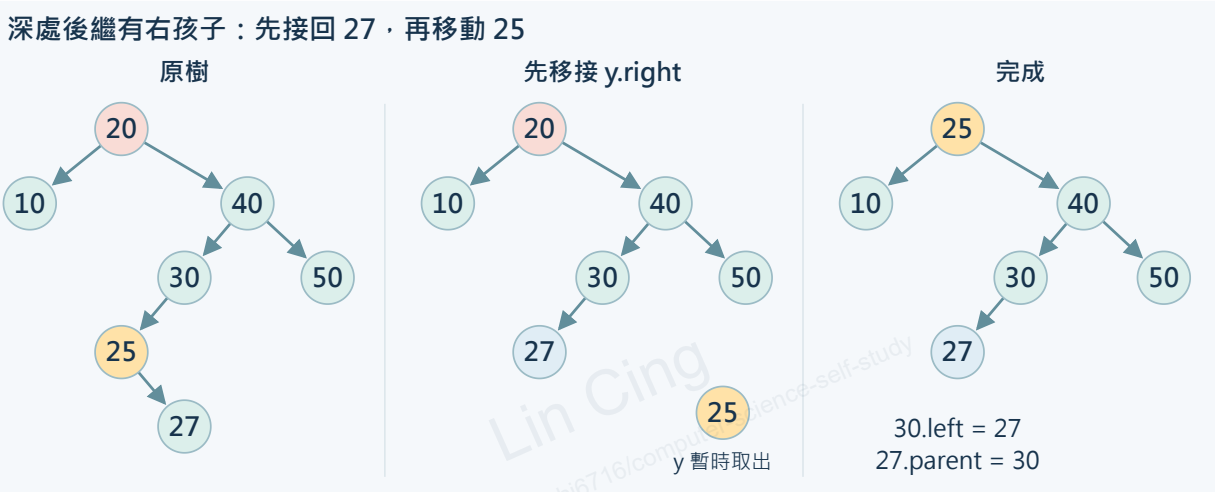



刪 z=20：y=40，且 y.parent=z。先讓 root 指向 y，y.parent=NIL；再接 y.left=10、10.parent=y。**y.right=50 從頭到尾不變。**

中序由 [10,20,40,50] 變為 [10,40,50]。保留下來的所有鍵值、value 與節點身分不變；只有 z 離開樹。非根的 z 也同樣處理，只是改接的是 z 原父的 left 或 right。



## 深處後繼：它的右孩子不能遺失




刪 root z=20，右子樹最小節點 y=25，且 y.right=27。依序更新：① 30.left=27、27.parent=30；② y.right=40、40.parent=y；③ root=y、y.parent=NIL；④ y.left=10、10.parent=y。

## 深處後繼：核對次序與可達性

圖中的中間態把 y 畫成暫時取出的物件；它的舊連結要清理或覆寫，不能在完成前把整個物件池當成一棵合法 BST。完成後 root=25、25.right=40、40.left=30、30.left=27。

原中序 [10,20,25,27,30,40,50] 只刪去 20。若第一步誤寫 30.left=NIL，27 會遺失；若忘了 parent 更新，後續 successor 的向上搜尋也會壞掉。

為何新連結仍符合次序？y=25 是原右子樹的最小值，所以原左子樹的 key 全小於 y，剩下右子樹的 key 全大於 y。27 原在 25 的右側、30 的左側，符合 25<27<30，接成 30.left 仍合法。

移接保留原來除 z 以外的每個物件恰好一次；沒有新建副本。最後還要核對 root.parent=NIL、每個非空 child 的 parent 指回擁有它的節點，且沒有循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪鍵與刪物件：兩種實作的差別

以 z.key=20、y.key=25 為例，兩種做法都能留下正確的鍵集合，但對 handle 的效果不同。

| 做法 | 留在 z 原位的物件 | 離開樹的物件 | 外部 handle 的影響 |
|---|---|---|---|
| 複製後繼 payload，再刪 y | 原 z，內容改成 25 的記錄 | y | z 的意義改變；y 的 handle 失效 |
| 移接後繼節點 y | 原 y，位置改到 z 原位 | z | z 的 handle 失效；y 的身分保留 |

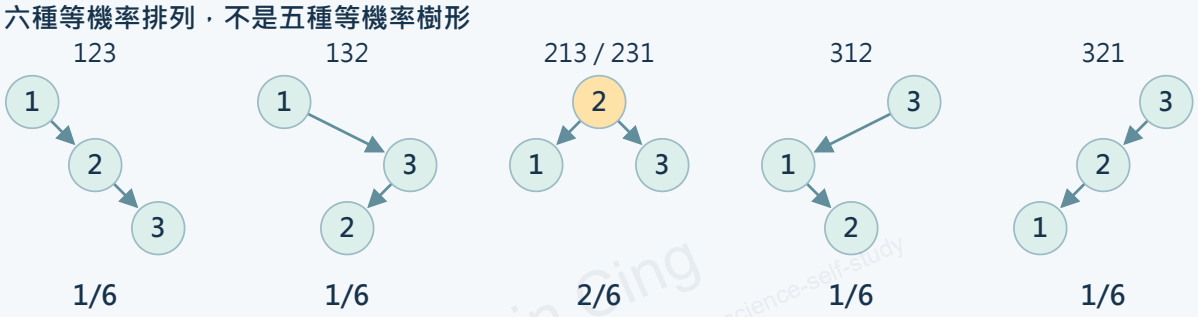
**來源慣例**：紙本筆記示範前者，CLRS 第三版 §12.3 採後者。本節以移接為準。複製法若採 key→value 記錄，必須搬對應 value，不能只抄 key；附加欄位、資源所有權與外部索引也要維護。

兩種方法都可改用前驅（左子樹最大值），但指標方向要整組對稱更換。不要從前驅版取圖、再套後繼版程式的左右分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 隨機順序不等於隨機樹形

固定 n 個相異 key，均勻抽取 n! 個插入排列。首筆為 root；給定 root 秩 R=r，兩側的相對排列仍各自均勻，故子樹可遞迴套同一模型。此處只分析插入建樹。




root=2 的兩個排列得到同一平衡樹，其機率為 2/6；其餘四個排列各得到不同樹形，各為 1/6。若直接把五種形狀各設為 1/5，就換了機率模型。



## 固定秩的期望深度：逐一數祖先

將鍵依大小記為 $x_1<\cdots<x_n$，$D_i$ 是 $x_i$ 的深度。對 $j\neq i$，$x_j$ 成為 $x_i$ 的祖先，**當且僅當閉區間 $[x_{\min(i,j)}, x_{\max(i,j)}]$ 中最先插入的是 $x_j$**。

區間外先插入的 key 只能把兩者一起帶往同側；區間內第一筆若是 $x_j$，它會成為通往 $x_i$ 的路徑節點。若先出現別筆，就會先到 $x_i$，或把 $x_j$ 與 $x_i$ 分開。因此

$$\Pr(x_j \text{ 是 } x_i \text{ 祖先}) = \frac{1}{|j-i|+1}.$$

深度就是祖先數，套線性期望（不需各事件獨立）得

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

其中 $H_t = \sum_{k=1}^{t} 1/k$。n=5、i=3 時，期望深度為 $2H_3 - 2 = 5/3$，成功搜尋的期望比較數再加 1，為 8/3。

## 期望建樹成本：把所有深度加起來

節點插入時比較過的節點，恰好是它在完成樹中的祖先；後續插入不改動既有節點的深度。因此建樹總 key 比較數為 $C = \sum_i D_i$。利用

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

可得

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

常數節點配置與連結另共 Θ(n)。等機率成功查詢的平均比較數，其期望為 $E[C]/n + 1$。n=3 時 E[C]=8/3，平均成功比較數為 17/9；n=1 時建樹比較 0 次、成功查詢 1 次。

這也連到前章的 quicksort：同一相對順序遞迴以首筆為 pivot 時，兩個 key 是否比較，取決於它們之間誰最先被選到。但**平均深度為對數，尚不能推出最大深度也有同樣期望界**。



## 高度分析：先處理 max，再取期望

令 $X_n$ 為高度（邊數），n≥1 時設 $Y_n = 2^{X_n}$；另設 **$Y_0=0$ 作為輔助值**，不是 $2^{X_0} = 2^{-1}$。單節點 $Y_1=1$。n≥2、條件在 root 秩 R=r 時，至少一側非空，故

$$Y_n = 2\max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

令 $a_j=E[Y_j]$。給定 R 後，兩側相對排列均勻，配合 max(u,v)≤u+v 得

$$a_n \leq \frac{2}{n}\sum_{r=1}^{n}(a_{r-1} + a_{n-r}) = \frac{4}{n}\sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

這裡用的是**條件期望**，沒有把隨機大小的子樹與 R 當成無條件獨立。也沒有使用錯誤等式 E[max(U,V)]=max(E[U],E[V])。

例如 $Y_1=1$、$a_2=2$；n=3 的兩個平衡排列高度 1、四個鏈狀排列高度 2，所以 $a_3=(2\cdot2+4\cdot4)/6=10/3$。

## 解出指數高度界，再回到原高度

強歸納證明 $a_n \le n^3$（$n \ge 1$）。$a_1=1$；對 $n \ge 2$，$a_0=0$，且 $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$，所以

$$a_n \le \frac{4}{n}\sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

為何可以從這裡推 $E[X_n]$？令 $\mu = E[X_n]$。由 $e^u \ge 1+u$ 得 $2^x \ge 2^\mu[1+(\ln 2)(x-\mu)]$；取期望，線性項消失，得到 Jensen 在此所需的形式：

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3\log_2 n.$$

$e^u - 1 - u$ 的導數是 $e^u - 1$，在 $u=0$ 取最小值 0，故上述切線界成立。此處高度的支集有限，期望均存在。

另一方面，$h$ 高的二元樹至多 $2^{h+1}-1$ 個節點，故每棵樹 $h \ge \log_2(n+1) - 1$。兩側合起來，隨機建樹的期望高度為 $\Theta(\log n)$，但單次最壞仍為 $n-1$。



## 紅黑樹：用路徑不變量限制高度

紅黑樹仍是嚴格次序的 BST，只多存 color。以下 B 表示黑、R 表示紅；圖中的顏色與 B／R 標籤並用，避免只能靠顏色辨識。

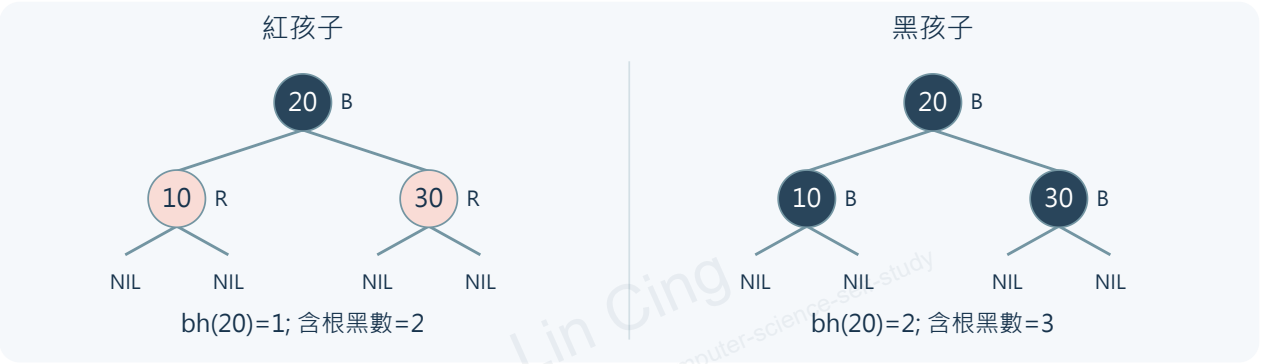
1. 每個節點非紅即黑，根節點 為黑。
2. 所有外部空位置 NIL 為黑；資料節點沒有 孩子，也仍連到 NIL。
3. 紅節點的兩個 孩子 都黑，故不會連續兩個紅節點。
4. **從任一節點往下到各個 NIL 的路徑，黑節點數都相同。** 不能只檢查少數 根節點–NIL 路徑。

用一個共享 哨兵節點 表示所有 NIL，令 NIL.left=NIL.right=NIL、NIL.color=B，root.parent=NIL。一般 child.parent 一致性只檢查非 NIL 的 孩子。同鍵 插入只更新 value，不改形狀或顏色。

前節 BST 不寫 NIL.parent；本節刪除修復將暫借它記錄一個缺黑位置的父節點。它不是所有空位置同時有效的 parent。旋轉遇到 $\beta$=NIL 時仍不寫 NIL.parent。

## 黑高先固定計數範圍

定義 $bh(x)$：從 x 往下到任一 NIL 的黑節點數，**不含 x，含末端 NIL**；另定 $bh(NIL)=0$。紅黑性質保證此數不因選哪條路而變。




若 c 是非 NIL 的 孩子，c 紅時 $bh(c)=bh(x)$；c 黑時 $bh(c)=bh(x)-1$。這是因為父的計數含 c，而 孩子 自己的 bh 不含自己。c=NIL 時同樣有 $bh(x)=1+bh(NIL)$。



## 高度界：先數最少需要多少節點

本章沿用內部節點邊數高度 h：單節點 h=0、空樹 h=−1。另以 H 計 根節點 到外部 NIL 的最長邊數，非空時 H=h+1，空樹 H=0；高度證明使用 H，不偷偷換掉 h 的定義。

**引理：** x 的子樹至少含 $2^{bh(x)}-1$ 個內部節點。對子樹高度歸納：x=NIL 時，$0=2^0-1$。若 x 非 NIL，兩個 孩子 的黑高各至少 bh(x)−1；套歸納假設再加上 x：

$$n_x \geq 2\left(2^{bh(x)-1}-1\right)+1=2^{bh(x)}-1.$$

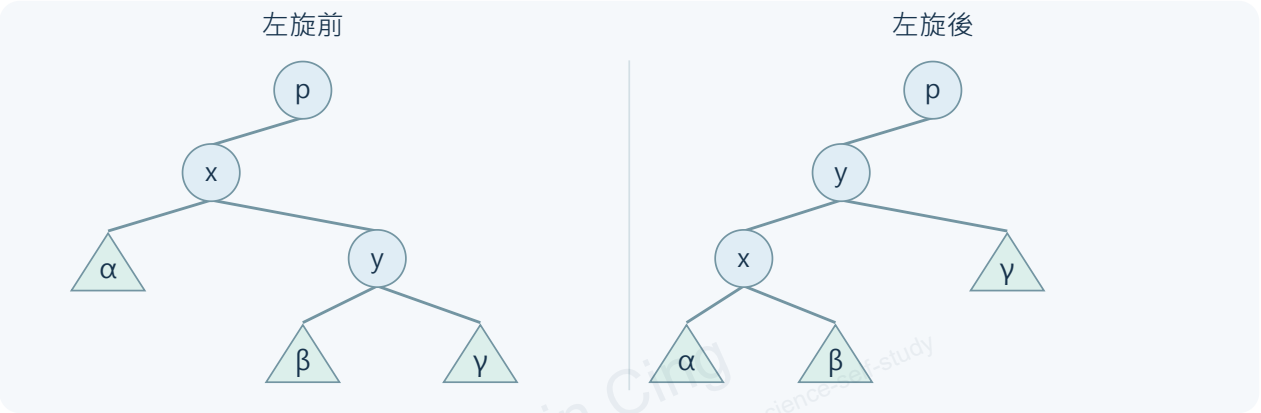
再取一條最長 根節點–NIL 路。含 根節點 與 NIL 共 B 個黑點、R 個紅點。兩端黑，沒有相鄰紅，所以 $R \leq B-1$；路徑邊數

$$H=B+R-1 \leq 2(B-1)=2bh(root).$$

因此 $n \geq 2^{bh(root)}-1 \geq 2^{H/2}-1$，得到 $H \leq 2\log_2(n+1)$。空樹 H=0 也成立。搜尋路徑加終止測試需 O(H+1)，統一寫 O(log(n+2))，才能涵蓋空樹常數成本。

這只保證搜尋高度；插刪還須證明修復會終止且不超出同一時間界。紅黑樹也不要求每個節點左右高度差至多 1，那是 AVL 的條件。

## 左旋：β 換父，舊父改接新根




前提 x≠NIL、y=x.right≠NIL。α、β、γ 表示整棵子樹，允許為 NIL；p 是 x 的**舊父**。旋轉後 y 取代 x 的局部根位置，β 從 y.left 移到 x.right。



## 左右旋轉：完整且對稱的接線

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right           p=y.parent; x=y.left
    beta=y.left                     beta=x.right
    x.right=beta                    y.left=beta
    if beta!=NIL: beta.parent=x     if beta!=NIL: beta.parent=y
    y.parent=p                      x.parent=p
    if p==NIL: T.root=y             if p==NIL: T.root=x
    elif p.left==x: p.left=y         elif p.left==y: p.left=x
    else: p.right=y                 else: p.right=x
    y.left=x                        x.right=y
    x.parent=y                      y.parent=x
```

先保存舊父 p；β=NIL 時不改 sentinel.parent。前提分別為 x.right≠NIL 與 y.left≠NIL。

## 旋轉接線：逐條核對數值與次序

取 p=30、x=10、y=20、β=15，且 p.left=x。其餘子樹內容不變。

| 順序 | 寫入 | 目的 |
|---|---|---|
| 1 | 10.right=15 | 接回 β |
| 2 | 15.parent=10 | 讓 β 認新父 |
| 3 | 20.parent=30 | 新局部根接外父 |
| 4 | 30.left=20 | 更新原父指向局部根的 連結欄位 |
| 5 | 20.left=10 | 舊根變成左 孩子 |
| 6 | 10.parent=20 | 舊根認新父 |

若 x 原為 根節點，第 3 項改為 20.parent=NIL，第 4 項改為 T.root=20。

鍵值區間 α<x<β<y<γ 不變，故中序與 BST 次序保持；節點身分及 鍵值與資料 也不變。**旋轉本身不保證紅黑性質**。



## 插入紅葉：先辨認唯一可能的違規

先依前節 INSERT 搜尋；同鍵 則更新 value 並返回。新節點 z 以 R 插入，left=right=NIL，parent 指向搜尋到的父節點。

原本的一個黑 NIL 被「紅 z 加黑 NIL」取代，任何穿過此位置的路徑黑數不變。若插成黑色，只有經此位置的路增加一黑，通常會破壞黑數相等。

修復迴圈入口保持三件事：**z 紅；若 z.parent 是 根節點，該父必黑；最多只剩根紅或 z 與父同紅其中一種違規。** BST 與各路黑數相等均維持。

若父黑，直接結束；若 z 是新根，最後塗黑即可。只有父 p 紅才進迴圈；p 不可能是根，因此祖父 g 存在且黑。叔 u 是 g 的另一 孩子，可能為 NIL，但 color 仍可安全讀取。

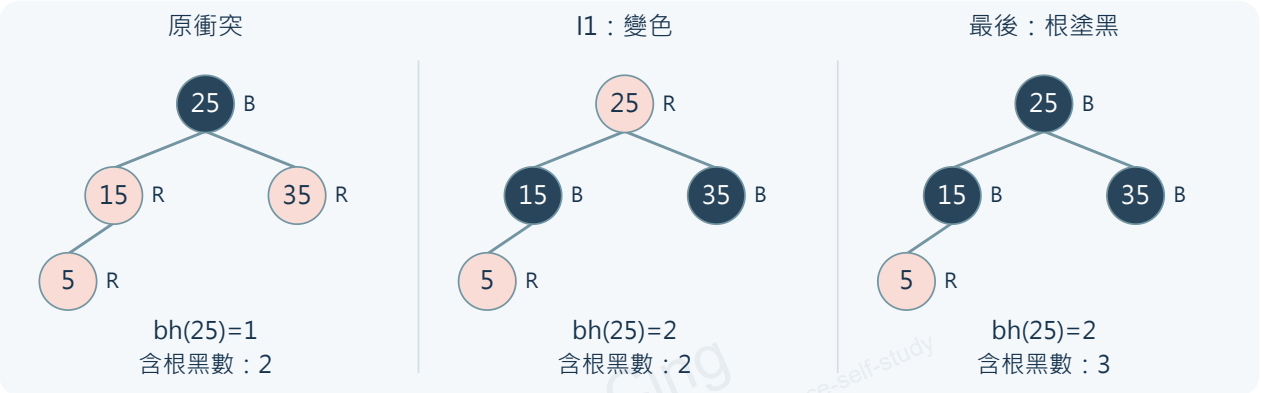
I2 不是另一個互斥結局：執行後必接 I3。旋轉會改 parent；I3 使用的是當下的 z.parent 與祖父，不能沿用旋轉前快取的 p、g。

## 插入修復：父在左側的三個分支

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```



## I1：重新著色不改祖先路徑的黑數




先插 25、15、35，再插 5。新增的 5R 與父 15R 衝突，叔 35R 也紅。把 15、35 塗黑、25 塗紅，z 移到 25；此時兩條子路徑的黑數仍相同。

## I1：哪些黑數改變，哪些不變？

令 p／u 以下（不含 p／u）的每條路有 b 黑點。原先經 g 的完整路有 1+b 黑；改成 gR、p/uB 後仍是 1+b。**穿過 g 的祖先到 NIL 路徑黑數保持不變**，但不含 g 的 bh(g) 從 b 變為 b+1。

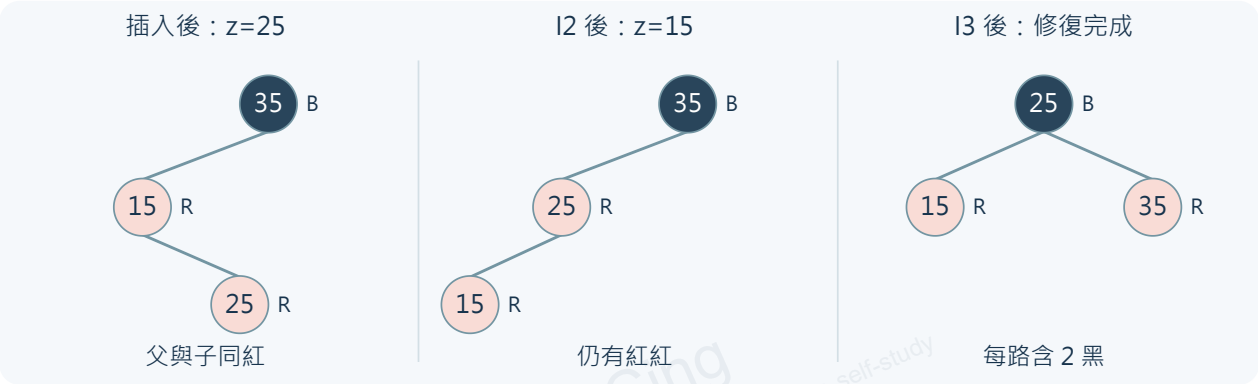
違規只可能移到 g 與它的父。若 g 恰為 根節點，迴圈結束後才把它塗黑，使所有 根節點–NIL 路一起加一黑。這個最後步驟不能套到非根 g。

| 計數範圍 | I1 前 | I1 後 |
|---|---|---|
| 不含 g 的 bh(g) | b | b+1 |
| 含 g 到 NIL | 1+b | 1+b |
| 任一祖先經 g 到 NIL | 外部黑數+1+b | 外部黑數+1+b |

若 g=25 為根且 b=1，I1 後 bh(25)=2、含根黑數=2；最後根塗黑才把含根黑數改為3。



## I2 接 I3：折線先拉直，再修復

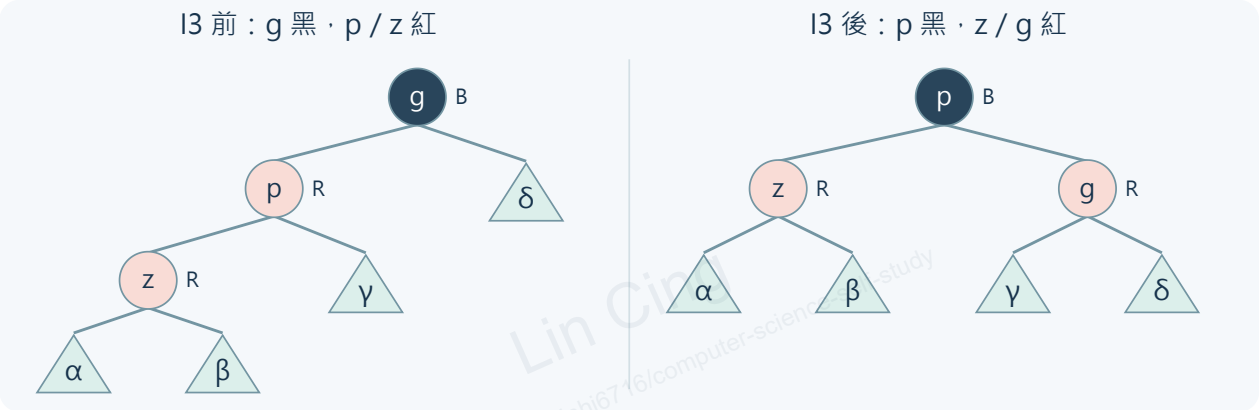



插 35、15、25，叔為黑 NIL，且父 15 在祖父 35 左側、新 25 在父右側。I2 先令 z 指舊父 15，再左旋 15；此時 25R 成為 15R 的父，**紅紅仍存在**。

I3 把目前父 25 塗黑、祖父 35 塗紅，再右旋 35。結果為 25B、15R、35R，每路含 2 黑；I2 後不能返回。

## I3 為何保留黑數，也消除紅紅

設祖父 g 黑、其左孩子 p 紅，而 p 的左孩子 z 也紅；叔 u 黑。z 的兩個孩子為 α、β，p 的原右子樹為 γ，g 的原右子樹為 δ。把 p 塗黑、g 塗紅，再右旋 g：






## I3：逐條驗證修復後的不變量

α、β 因 z 紅而為黑根；γ 因 p 紅而為黑根；δ 就是黑叔。各邊界子樹從根到 NIL（**含子樹根**）的黑數皆為 b；原經 g 的路貢獻 1+b，改後經 p 也貢獻 1+b。

新局部根 p 黑，不會與外父形成紅紅。z 仍紅但父 p 黑；g 改紅，其 孩子 γ、δ 都黑。故局部所有紅黑性質恢復，接回原父也不改外部黑數，修復可以結束。

若原本已是外側鏈（插 35、25、15），跳過 I2，直接 I3 即可；黑數證明完全相同。

| 通往哪棵子樹 | 修復前的局部黑節點 | 修復後的局部黑節點 |
|---|---|---|
| α、β | gB；再加 b | pB；再加 b |
| γ | gB；再加 b | pB；gR 不計；再加 b |
| δ | gB；再加 b | pB；gR 不計；再加 b |

四種目的地前後皆 1+b。子樹內部連結及顏色不變，因此每個內部節點原有的路徑相等性也保留。

## 父在右側：把鏡射動作寫出來

以下取代主修復程式的 else；條件是 z.parent=祖父.right。u 這次取左叔，旋轉方向也交換。

```
u=z.parent.parent.left
if u.color==R:                              // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                     // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                        // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```



## 插入為何至多旋轉兩次？

I1 不旋轉，每次把 z 上移兩層，最多 O(H) 次。I2 接 I3 最多兩次基本旋轉，隨即結束；只做 I3 則一次。因此 CLRS 這套 bottom-up 插入的**旋轉總數至多 2 次**，不是每層各兩次。連搜尋、重新著色一起算，總 O(log(n+2))，新增節點空間 O(1)。

第一種離開迴圈的情況是父黑：沒有紅紅，最後只須確保根黑。第二種是 I3 把局部根塗黑：它與外父也不會衝突。I1 則嚴格往上走，不能無限重複。

| 修復分支 | 基本旋轉 | 下一步 |
| --- | --- | --- |
| I1：叔紅 | 0 | z 上移兩層，可能重複 |
| I2：叔黑、內側 | 1 | 立刻接 I3 |
| I3：叔黑、外側 | 1 | 局部根黑，修復結束 |
| 最後根塗黑 | 0 | 完成 |

可能的旋轉段只有 I3 或 I2→I3；I1 可以出現很多次，但不會增加旋轉數。

## 兩種修復時機，不能混接程式

| 比較項目 | 本節 bottom-up | top-down 2-3-4 分裂式插入 |
|---|---|---|
| 何時處理雙紅 孩子 | 先插紅葉，再從違規處向上修復 | 搜尋途中遇 4-node 先分裂、變色 |
| 何時可能旋轉 | 叔黑的 I2／I3 | 向下處理時若產生相鄰紅便局部修復 |
| 共同不變量 | BST 次序、無相鄰紅、各路黑數相等 | 相同，但修復入口條件不同 |

兩者可得到不同的合法樹形；不能把 top-down 的變色時機插入本節程式卻沿用原證明。「一次雙旋調整」可能包含兩次基本旋轉；比較旋轉數前，先固定計數單位與具體演算法。

也可把紅邊視為紅 孩子 與黑父的連結，收縮後對應 2-3-4 tree；一般紅黑樹允許左紅或右紅。額外要求紅邊左傾的是另一套規格，不能默認。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪除前先分清 z、y、x 與原色

z 是要刪除的節點。若 z 至多有一個孩子，令 y=z；若有兩個孩子，令 y 為右子樹的最小節點，稍後把 **y 這個物件移到 z 的位置**。

x 負責填補 y 的舊位置：後繼 y 沒有左孩子，因此取 x=y.right；若 y=z，則取 z 唯一的非空孩子，沒有孩子時取 NIL。先保存 y 的原色，再讓移來的 y 繼承 z 的顏色。**是否需要修復，取決於 y 搬動前的顏色。**

```
RB-TRANSPLANT(T,u,v):
    若 u.parent==NIL: T.root=v
    否則若 u==u.parent.left: u.parent.left=v
    否則: u.parent.right=v
    v.parent=u.parent              // 即使 v==NIL 也寫
```

本節允許寫入 NIL.parent，只用它保存當前缺黑位置 x 的父節點，供修復時尋找兄弟；它不是所有空連結共用的正確父節點。旋轉遇到 β=NIL 時不可覆寫此欄位。修復完成後才重設 NIL.parent=NIL；NIL 始終為黑色。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## RB-DELETE：先移接，再依原色決定修復

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

刪除物件 z；其餘物件的 鍵值與資料 與身分保持不變。原色為紅時不減黑數；原色為黑時，從填補舊位置的 x 開始處理。若外部 API 接收 key，先搜尋，找不到便回傳 absent，不能對 NIL 呼叫此程序。



## 兩個孩子：移動後繼前先保存原色

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

## 後繼移接：資料跟著物件走

依序插入 40、20、60、10、30、50、70、45、55，得到根 40B：左子樹為 20B(10R,30R)，右子樹為 60R，其左側是 50B(45R,55R)，右側是 70B。

刪除 z=40 時，後繼 y=45R、x=NIL，且 y 的舊父為 50。依序執行：

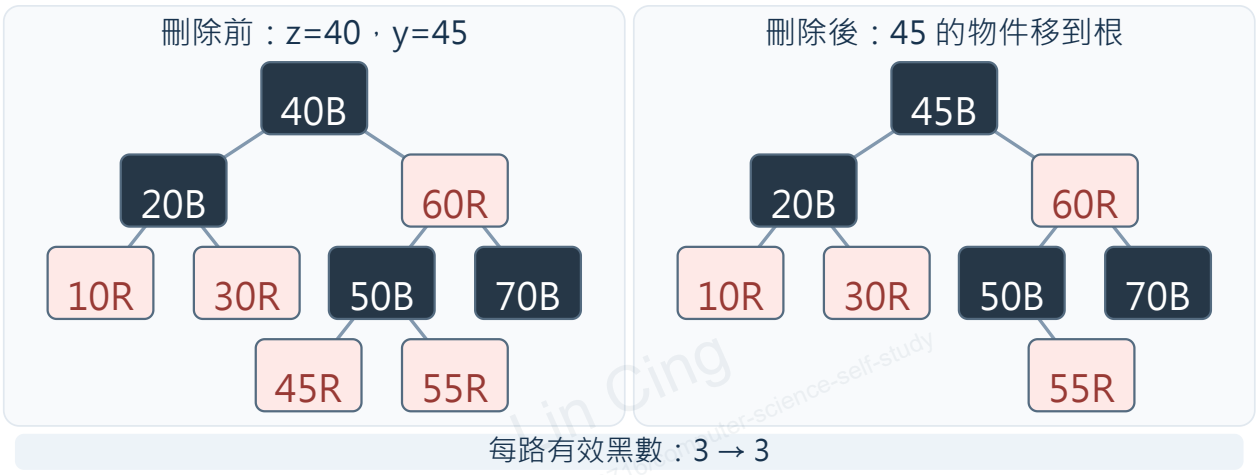
1. 令 50.left=NIL、NIL.parent=50，移出原來的 45。
2. 令 45.right=60、60.parent=45，接上原右子樹。
3. 以 45 取代根 40，令 45.parent=NIL。
4. 令 45.left=20、20.parent=45，並讓 45 繼承 40 的黑色。

結果為根 45B。45 的原色是紅色，舊位置沒有少掉黑節點，因此無須修復；45 原有的資料與物件身分都保留，只有節點 40 被移除。

後繼若就是 z.right，x 的新父應為 y；若後繼更深，第一次移接已保存 x.parent，後續搬動 y 時不可破壞它。這兩種情況對 x=NIL 也必須成立。



## 深處後繼：同一棵樹的移接前後




兩圖是同一數例的移接前後：45 從 50 的左側搬到根。原位置由 x=NIL 補上；第一次移接後 NIL.parent=50，搬動 45 不可改寫它。修復或清理完成後才重設 NIL.parent=NIL。未畫出的空孩子皆為黑 NIL。

## 額外黑記在位置上，不是第三種顏色

若 y 的原色為紅，移除它不會減少路徑黑數；移到 z 位置的節點又已繼承 z 的顏色，所以不必修復。若 y 原為黑，則在填補舊位置的 **x 處暫記一份額外黑（extra black）**，讓經過該位置的路徑仍具有原來的有效黑數。

實際的 x.color 仍只有紅、黑兩種。若 x 紅，最後把它塗黑便吸收額外黑；若 x 已是根，則從全樹所有路徑同時取消這一份額外黑。只有 x 為黑且不是根時，才進入修復迴圈。

此時兄弟 w 不可能為 NIL：若 w=NIL，兄弟側只有末端 NIL 的一份黑；x 側至少已有 NIL 再加額外黑，共兩份，與兩側有效黑數相等矛盾。

x 在父 p 的左側時，w=p.right；稱 w.left 為**近側孩子 near**，w.right 為**遠側孩子 far**。x 在右側時，這兩個角色及旋轉方向全部鏡射。圖中的 x +1 表示額外黑；未畫出的空孩子皆為黑 NIL，路徑計數含起點及末端 NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 修復迴圈：x 表示目前唯一的額外黑位置

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

兩個 輔助程序 回傳下一個修復位置：D2 回傳父節點，D4 回傳 根節點 使迴圈結束。最後的塗黑有兩種意義：x 紅時吸收額外黑；x 已是黑根時，把證明中的額外黑從全部路徑一起去掉，實際 color 不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 左側缺黑：D1～D4 的完整接續

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```



## 右側缺黑：明列鏡射，避免近側與遠側搞反

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

## 先看兄弟，再看近側與遠側顏色

| 分支 | 前提（先處理紅兄弟） | 去向 |
|---|---|---|
| D1 | 兄弟紅 | 變成黑兄弟，接 D2／D3／D4 |
| D2 | 黑兄弟，兩個 孩子 黑 | 缺黑移到父；可能被紅父吸收 |
| D3 | 黑兄弟，near 紅、far 黑 | 轉成 D4 |
| D4 | 黑兄弟，far 紅 | 消除額外黑並結束 |

x 在父左側時，near=w.left，far=w.right。D1／D3 後都要重新取 w；D3 不會返回，必接 D4。D2 的 return 是把父交回外層迴圈，不是宣告整個刪除已完成。

x 在父右側時，near=w.right、far=w.left。幾何角色以「靠近／遠離 x」定義，不能把 near 永遠記成 left。兩個 輔助程序 都不更新 NIL.parent；它由移接步驟保存，直到本次修復結束。

共享 NIL 不會讓此處左右判斷含糊：進入迴圈時兄弟必非 NIL，因此父不可能兩側同時為 NIL。



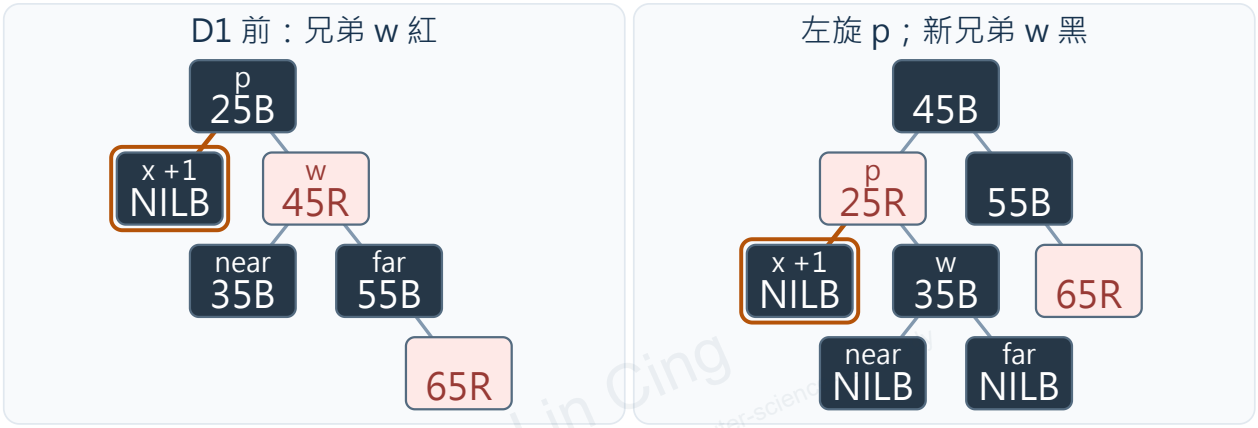
## D1：兄弟轉黑，缺黑位置不動

先以 b 表示 x 子樹從根到 NIL 的**實際黑數**，含 x 與末端 NIL；加上額外黑後，有效黑數為 b+1。D1 的兄弟 w 紅，因此父 p 及 w 的兩個孩子必黑。w 紅而整棵 w 子樹的黑數為 b+1，故它的兩個孩子也各有 b+1 黑。

把 w 塗黑、p 塗紅，再左旋 p。新局部根為黑色 w；其左側的紅色 p 下，x 的有效黑數為 b+1，原近側孩子也為 b+1。w 右側原遠側孩子仍為 b+1。因此旋轉前後，從局部根開始的有效黑數均為 b+2。

額外黑仍留在 x，尚未消除。重新取得 x 的兄弟後，它已是黑色，接著依兩個孩子的顏色進入 D2、D3 或 D4。

## D1 數例：重新取得旋轉後的兄弟




每路有效黑數：3 → 3

依序插入 15、25、35、45、55、65，再刪 15：x=NIL 位於 25B 左側，兄弟為 45R。把 45 塗黑、25 塗紅，左旋 25，得到新根 45B、左側 25R（右孩子 35B）、右側 55B（右孩子 65R）。x 仍是 25.left，**新兄弟改為 35B**；缺黑位置與祖先的有效黑數不變。



## D2：把額外黑交給父節點

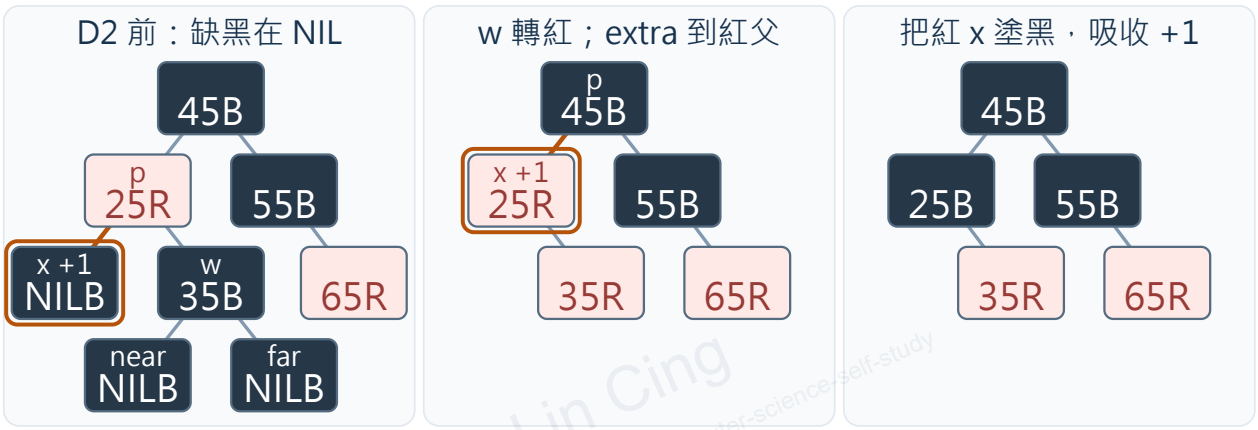
兄弟 w 黑，近側與遠側孩子也都黑。設 x 的實際黑數為 b；含額外黑後為 b+1。w 子樹的完整黑數為 b+1，而 w 的兩個孩子各為 b。

將 w 改為紅色後，w 子樹的黑數降為 b，與 x 側的**實際**黑數相等。但整棵 p 子樹對外仍少一黑，因此改把額外黑記在 p，令 x=p：

| p 原來的狀態 | 接下來的處理 |
| --- | --- |
| 紅色 | 離開迴圈，最後塗黑，吸收額外黑 |
| 黑色且非根 | 沿祖先繼續修復 |
| 根節點 | 從全樹路徑同時取消額外黑，結束 |

D1 剛把 p 塗紅。因此 D1 後若接 D2，會由紅父吸收，不會再向上遇到第二次 D1。

## D2 數例：紅父吸收額外黑




每路有效黑數：3 → 3 → 3

承接 D1：35B 的兩個孩子都是黑 NIL。把 35 改紅，將修復位置從 NIL 上移到 25R；迴圈停止，最後把 25 塗黑。結果為根 45B、左側 25B（右孩子 35R）、右側 55B（右孩子 65R）。額外黑由 25 的紅轉黑補齊，無須再旋轉。



## D2 到達根：所有路徑一起少一黑

原樹為根 25B、兩個黑葉 15B 與 35B。刪除 15 後，D2 把 35 改紅，再令 x=25。x 已是根，因此從全樹所有路徑同時取消額外黑：

每路有效黑數：3 → 3 → 2

最後為 25B，其右孩子為 35R。根的黑高由 2 降為 1，各條路徑仍相等。這裡是**整棵樹一起少一黑**，與上一例「紅父塗黑以補回一黑」是不同的結束方式。

## D3：近側紅，先轉成遠側紅

兄弟 w 黑，近側孩子 N 紅、遠側孩子黑。設 x 的實際黑數為 b。w 的兩個孩子從根到 NIL 均有 b 黑；因 N 紅，N 的兩個孩子也各有 b 黑。

把 N 塗黑、w 塗紅，再右旋 w。新兄弟為 N：左側是原 N.left，黑數為 b；右側是紅色 w，它的兩個孩子為原 N.right 與原遠側孩子，黑數也都為 b。因此新兄弟的完整黑數仍為 b+1，父 p 及祖先看到的有效黑數不變。

新兄弟的遠側孩子就是原來的 w，現在為紅色，故**立即接 D4**。額外黑仍在 x；D3 既沒有把缺黑上移，也沒有完成修復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## D3 數例：右旋兄弟，再接 D4

依序插入 15、25、45、35，得到根 25B、左孩子 15B，以及右孩子 45B（左孩子 35R）。刪除 15 後，x=NIL、兄弟 w=45B、近側為 35R、遠側為黑 NIL。

把 35 塗黑、45 塗紅，再右旋 45，得到根 25B、右孩子 35B，其右孩子為 45R。新兄弟 35 的遠側已紅，下一步必須執行 D4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## D4：兩側補齊，消除額外黑

設父 p 原色的黑色貢獻為 c∈{0,1}，x 的實際黑數為 b。兄弟 w 黑，其近側與遠側孩子的完整黑數皆為 b，且遠側孩子原為紅色。

讓 w 繼承 p 的原色，把 p 與遠側孩子塗黑，再左旋 p。新局部根為 w：

| 新根的哪一側 | 從該側根到 NIL 的黑數 |
|---|---|
| 左側 p | p 黑，加上 x 或近側子樹的 b，得到 b+1 |
| 右側原遠側孩子 | 由紅轉黑，從 b 增為 b+1 |

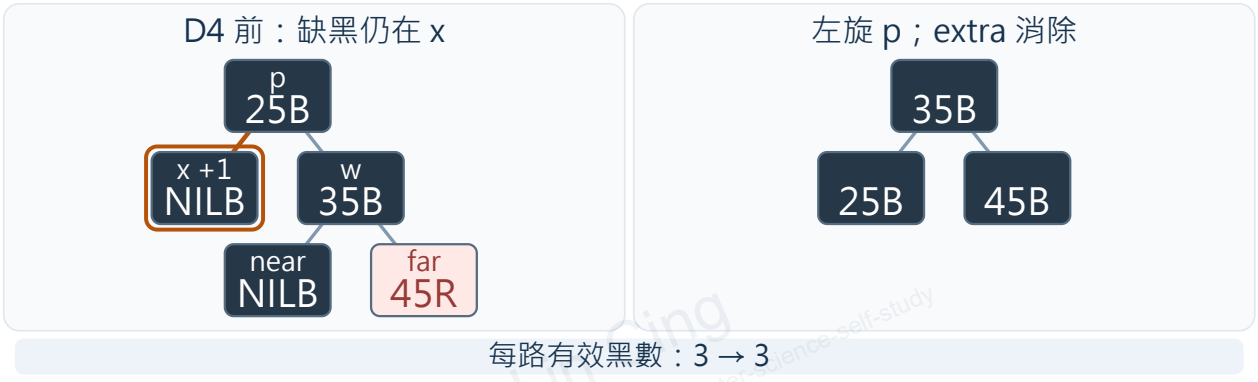
兩側實際黑數已相等，不再需要額外黑。新局部總黑數為 c+b+1，等於原本從 p 開始、含額外黑的有效總數。

若新根 w 紅，它的兩個孩子 p 與原遠側孩子都黑；w 對外又保留 p 的原色，因此不會與外父新增紅紅衝突。令 x=T.root 結束迴圈，最後確保根黑。



## D4 數例：完成同一次刪除

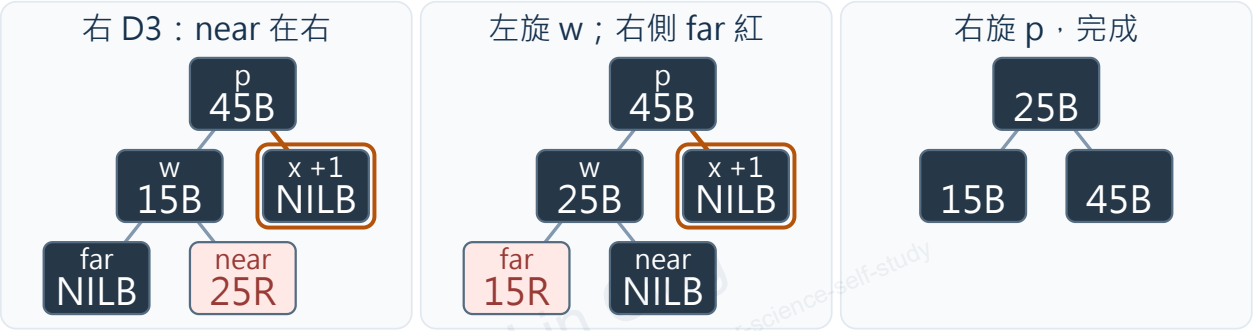
承接 D3 的數例：兄弟 35 繼承父 25 的黑色，25 與遠側孩子 45 也塗黑，再左旋 25。




結果為根 35B、左孩子 25B、右孩子 45B。留下的鍵為 25、35、45；每條根到 NIL 的路徑均有 3 黑，額外黑已完全消除。

## 右側缺黑：把 D3、D4 完整鏡射

依序插入 15、35、45、25，再刪除 35。後繼 45B 就是右孩子，移接後成為新根；x=NIL 位於 45.right，故必須令 NIL.parent=45。




兄弟為 15B，近側是 25R，遠側是黑 NIL。右側 D3 **左旋 15**，接著右側 D4 **右旋 45**；最後為根 25B、兩個黑孩子 15B 與 45B。近側、遠側是相對於 x 的位置，不是固定的左、右方向。



## 同一數例：逐路加總實際黑數與額外黑

以下每條路**含起點與 NIL**。D3 不改 x=NIL 的位置；D4 才消除 extra。

| 階段與路徑 | 實際黑數 | extra | 有效總數 |
|---|---|---|---|
| D3 前：25B → NIL(x) | 2 | 1 | 3 |
| D3 前：25B → 45B → 35R → NIL | 3 | 0 | 3 |
| D3 後：25B → 35B → 45R → NIL | 3 | 0 | 3 |
| D4 後：35B → 25B → NIL | 3 | 0 | 3 |
| D4 後：35B → 45B → NIL | 3 | 0 | 3 |

黑點各加 1、紅點加 0，只有穿過 x 的路再加 extra。未列出的另一側 NIL 路依同一規則也為 3；可對照前後圖逐一驗算。D4 後無須 extra，兩側已各補齊一黑。

## 終止與總成本：旋轉少，不代表刪除為常數

BST 移接先保住鍵序與所有未刪除的節點身分。移除紅色位置不減黑數；移除黑色位置時，先在 x 記額外黑，使祖先看到的有效黑數維持平衡。

| 修復步驟 | 額外黑的位置 | 是否可能繼續向上 |
| --- | --- | --- |
| D1、D3 | 留在原 x | 只轉換局部情況 |
| D2 | 移到父節點 | 黑父且非根時繼續 |
| D4 | 消除 | 結束 |

D2 每次上移一層，最多 O(log(n+2)) 次。D1 後父節點變紅：若接 D2 就由紅父吸收，否則經 D3／D4 結束。因此 D1、D3、D4 各至多出現一次，**整次刪除至多 3 次基本旋轉**。

各分支已分別證明黑數及紅紅關係的維護；最後塗黑處理紅 x 或黑根的結束情況，紅黑性質全部恢復。刪除最後一個節點則得到 T.root=NIL。

搜尋 z、找後繼、向上修復合計仍需 **O(log(n+2))**。至多三次旋轉只限制結構重排；D2 的傳播與重新著色仍可能經過整條祖先路徑。



## 用子樹大小支援動態順序統計

資料反覆插入、刪除後，若還要查「第 k 筆」或「這個節點排第幾」，可以在平衡搜尋樹加入**子樹大小 size**。size 包含節點本身，NIL.size=0：

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

秩（rank）採中序次序的 1-based 位置；select(k) 回傳第 k 個節點。本節數例的鍵互異。若允許重複鍵，可用 (key,identity) 定義全序；只給一個重複值，無法指定唯一節點的秩。

擴充時必須一起說清楚**摘要怎麼算、修改後怎麼修、查詢為何正確**。本節摘要由自身資料與兩個孩子在 O(1) 時間算出，且不依節點顏色；依顏色的摘要，須另算重新著色的維護成本。查詢與維護皆先依樹高 h 分析為 O(h+1)；只有底層維持紅黑性質時，才能換成最壞 O(log(n+2))。加一個欄位不會自動讓普通 BST 平衡。

## Select：每層都使用目前子樹的局部秩

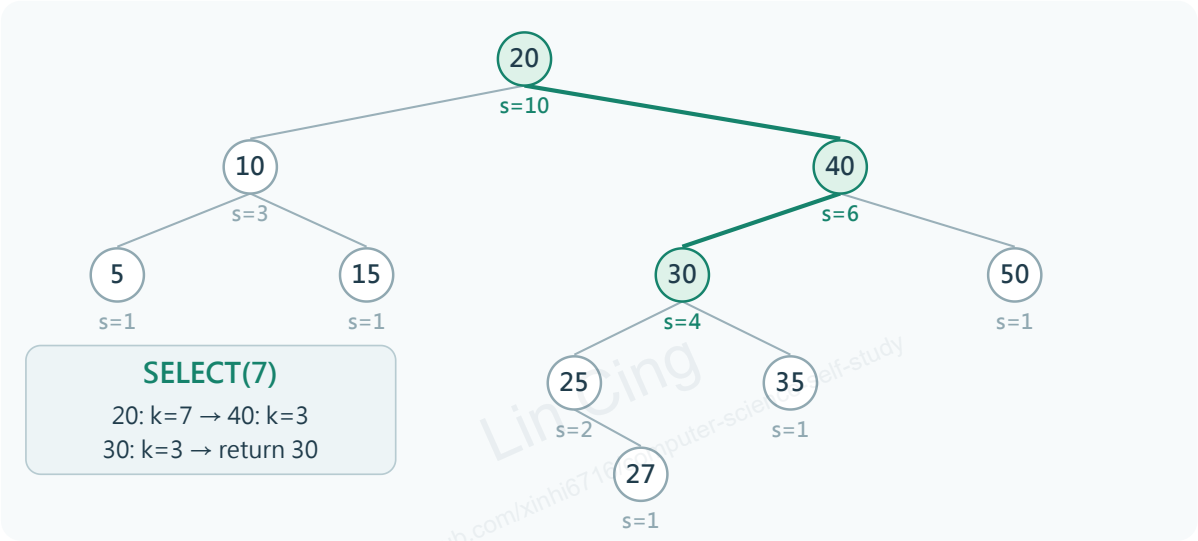
```
SELECT(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

迴圈不變量是「答案仍為目前 x 子樹中的第 k 個節點」。左子樹佔前 r−1 位，x 佔第 r 位；往左時 k 不變，往右時扣掉左子樹與 x，共 r 筆。

合法的 k 保證選到的子樹非空，且新 k 仍合法。每次下降一層，因此會在有限步驟內找到答案，不會先走到 NIL。公開介面須先拒絕 k=0、k>root.size；空樹沒有合法的 k。



## 同一棵樹，同時核對鍵值、size 與路徑




圓內是鍵，圓下 s 是包含自身的子樹大小。綠色路徑追蹤 SELECT(7)：20→40→30。接下來的 select、rank 與刪除數例，都以這棵樹為共同起點。

## Select(7)：往右才扣掉已略過的前綴

| 目前鍵 | 局部 k | left.size | r | 選擇與新 k |
|---|---|---|---|---|
| 20 | 7 | 3 | 4 | 往右到 40；7−4=3 |
| 40 | 3 | 4 | 5 | 往左到 30；仍為 3 |
| 30 | 3 | 2 | 3 | 回傳節點 30 |

中序為 **5,10,15,20,25,27,30,35,40,50**，第 7 筆確為 30。第一步略過 {5,10,15,20}，剩餘子樹中要找的是第 3 筆；不能把全樹的 7 原封不動拿去和每一層的 r 比較。

往左未略過目前子樹中的任何前綴，所以不扣 k；往右略過的正是 r 筆。這也說明為何 k−r 不會是 0：只有 k>r 時才往右。



## Rank：向上累加互不重疊的前綴

```
RANK(T,x):                    # x is a valid node in T
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

不變量：r 是原節點 x 在目前 u 子樹中的中序位置。u 若是左孩子，父節點與右側資料都排在 x 後，不新增前綴；u 若是右孩子，就加上父節點及其整棵左子樹。

新增的資料都在先前 u 子樹之外，又都排在 x 之前，因此不重複。到達根時，r 就是全樹的秩。此介面接受樹內的有效節點，不能傳 NIL、已刪除的物件或另一棵樹的節點。

## Rank(27)：哪些祖先需要加，哪些不用？

初始 r=size(27.left)+1=1。

| u→父節點 | u 位於哪側 | 新增量 | 更新後 r |
|---|---|---|---|
| 27→25 | 右側 | size(NIL)+1=1 | 2 |
| 25→30 | 左側 | 0 | 2 |
| 30→40 | 左側 | 0 | 2 |
| 40→20 | 右側 | size(10)+1=4 | 6 |

最後得到第 6 筆，和中序次序一致。第一次加入節點 25，第二次加入 {5,10,15,20}；加上原本的 27，共 6 筆。這些前綴彼此不重疊，30、40 則都在 27 後面，不能算進來。



## 摘要由下往上重算，旋轉也要同步維護

```
PULL(v):                        # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

此版本同時保存 size 與區間的最大上端點 max；只有順序統計時，可省略 max。NIL.size=0、NIL.max=−∞。

插入新葉時 size=1；接好連結後，從其父節點向根重算。刪除則先完成 BST 移接，再從正確起點沿**最終的父鏈**重算，之後進入紅黑修復。旋轉改好指標後，先 PULL 下移節點，再 PULL 升起節點。

size 與 max 都不依顏色，重新著色不必重算。刪除區間或縮短上端點時，max 可能下降，不能只保留歷史最大值。

## 刪除後，從哪裡開始修摘要？

在移接前先保存起點 a；完成全部連結後，執行 REPAIR-UP(a)，再做紅黑顏色修復。

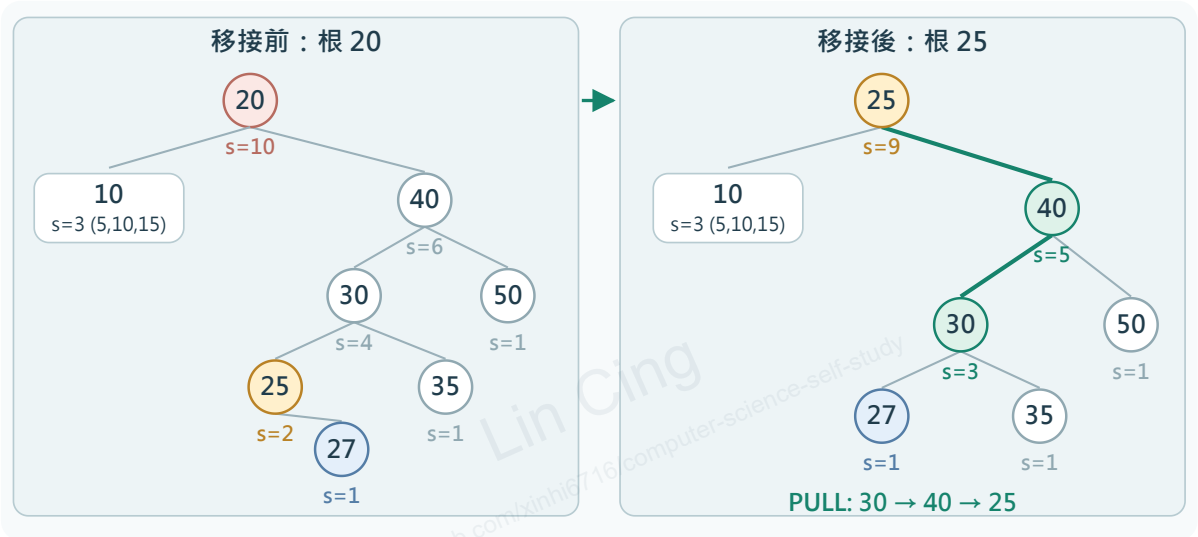
| 刪除分支 | 保存的 a | 最終父鏈涵蓋的變動 |
| --- | --- | --- |
| z 至多一個孩子 | z 的原父節點 | 孩子的子樹不變；祖先失去一筆 |
| 後繼 y 就是 z.right | y | y 接上新的左子樹，再修祖先 |
| 後繼 y 位於更深處 | y 的原父節點 | 原右子樹、搬上去的 y、z 的舊祖先 |

必須保存真正的節點，不能在釋放 z 後再讀 z.parent。共享的 NIL.parent 也不是所有空連結的共同父節點；修復摘要的 a 應另外保存。若 a=NIL，代表沒有祖先需要重算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪除 20、移接 25：連結與摘要一起看




25 的物件身分保留，27 接到 30 的左側；25 再接上原左子樹 10、原右子樹 40。色塊表示移接角色，不是紅黑顏色。圖中先完成 BST 移接與摘要重算，顏色修復依前節程序接續。

## 先算 30，再算 40，最後才算新根 25

移接前保存 a=30。移接後，這條父鏈為 **30→40→25**，依序重算：

| 節點 | 使用已更新的孩子 | 新 size |
|---|---|---|
| 30 | 1+size(27)+size(35)=1+1+1 | 3 |
| 40 | 1+size(30)+size(50)=1+3+1 | 5 |
| 25 | 1+size(10)+size(40)=1+3+5 | 9 |

25 原本的 size=2 已失效。若先重算 25，會讀到尚未更新的 40.size=6，誤得 10。若只從舊根 20 的父節點開始，起點是 NIL，更會完全漏掉這三個需要更新的節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 旋轉後只修兩格，需要額外理由

左旋時 y=x.right 升起，β=y.left 改接給 x。完成指標改接後，**先 PULL(x)，再 PULL(y)**，因為新 y 的摘要依賴新 x。

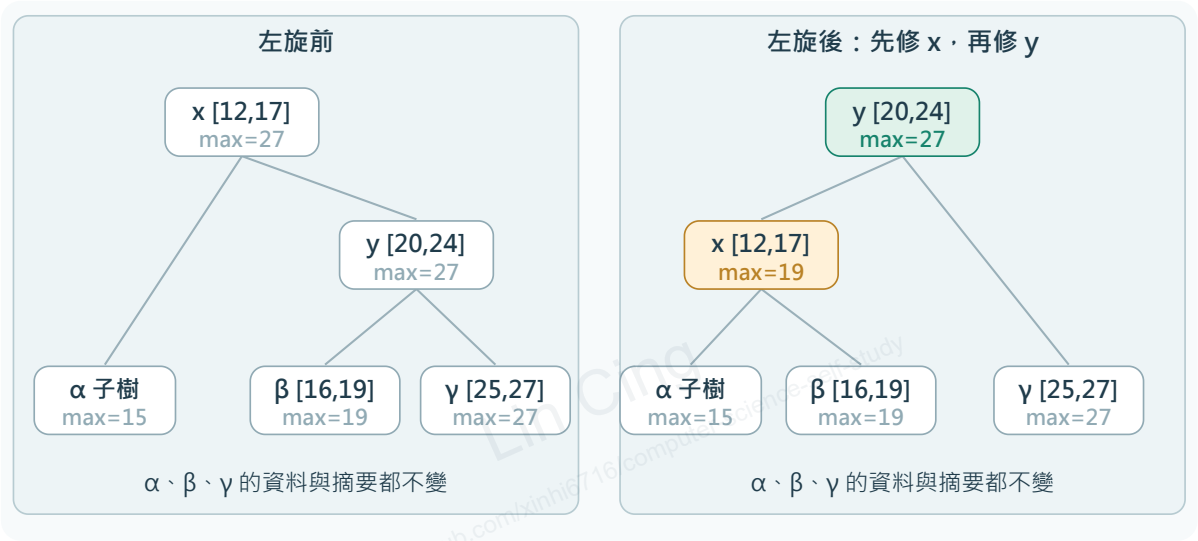
size 與子樹最大 high 都只依「子樹含有哪些資料」。旋轉後，新 y 包含的資料恰為舊 x 的整棵子樹，因此新 y 的摘要等於舊 x，外部祖先看到的摘要不變，只需 O(1) 局部重算。

**由孩子算得出，不代表只影響兩格。** height 也有局部公式，旋轉後局部樹高卻可能改變，仍須向祖先傳播。一般摘要每次旋轉最多增加 O(h+1) 工作；紅黑插入最多 2 次、刪除最多 3 次旋轉，所以仍為 O(log(n+2))。

若直接在每個節點保存全域 rank，插入新最小鍵會讓所有舊節點的 rank 加 1，需要 Θ(n) 次明確寫入。選子樹摘要，正是為了把更新留在少數祖先路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一組區間左旋：x.max 下降，整體 max 不變




先算 x.max=max(17,15,19)=19，再算 y.max=max(24,19,27)=27。新 y 仍涵蓋原局部子樹的全部資料；先前最大上端點 27 沒有消失，只是內部連結改變。



## 區間樹：依下端點排序，保存最大上端點

本節使用閉區間 [l,h]，允許 l=h 的單點區間。兩區間 i、q 重疊，當且僅當：

$$i.low \le q.high \quad \text{且} \quad q.low \le i.high.$$

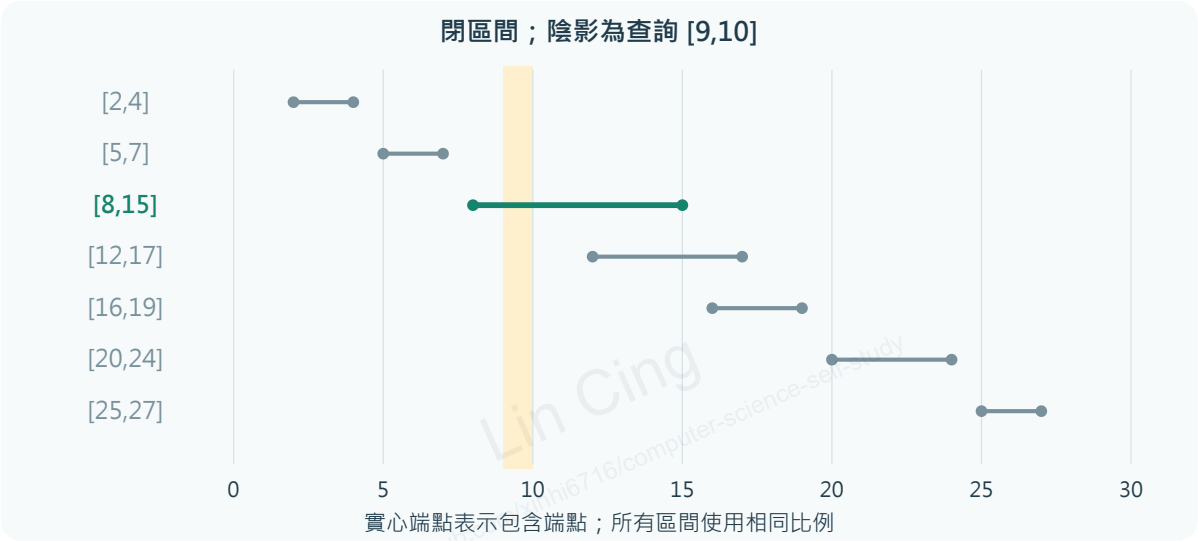
也就是排除 i 完全在 q 左方或右方的兩種情況。

BST 依下端點 low 排序；同 low 可再用物件識別碼決定先後，仍有 left.low≤x.low≤right.low。每節點保存

$$x.max = \max\{x.high,\ x.left.max,\ x.right.max\}, \qquad NIL.max = -\infty.$$

**max 是整棵子樹的最大 high**，不是目前節點的 high，更不是最大的 low。數例的根 [12,17] 雖然 high=17，max 卻是右側後代 [25,27] 提供的 27。

## 把區間放在同一條座標軸上




陰影是查詢 [9,10]。只有 [8,15] 穿過陰影；[5,7] 在它左側，[12,17] 在右側。端點位置與後面的搜尋樹使用的資料完全一致。



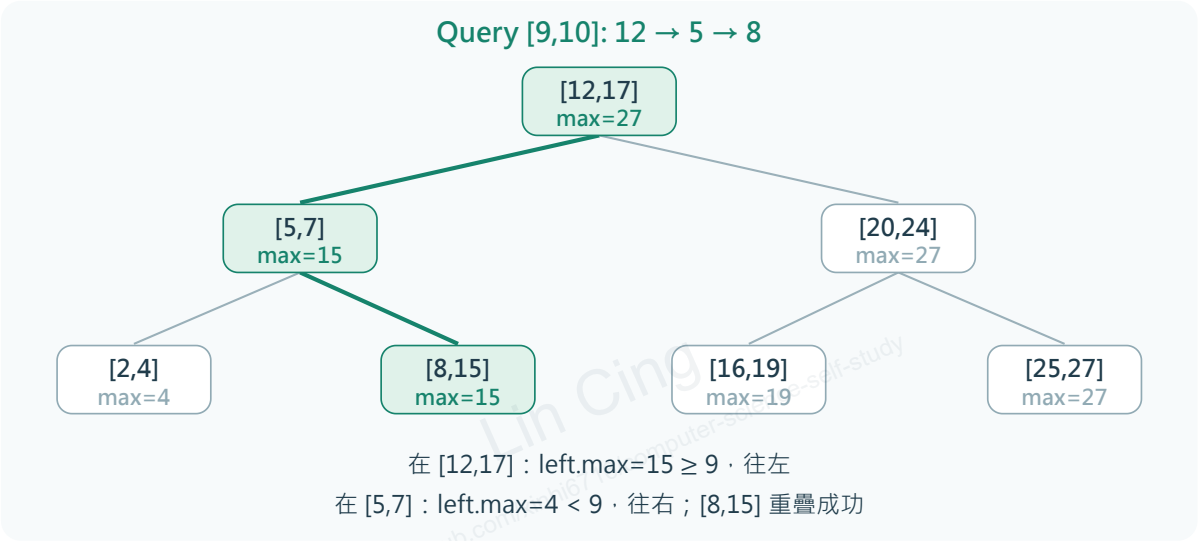
## 重疊搜尋：每一步只選一棵子樹

```
INTERVAL-SEARCH(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

找到目前節點重疊就回傳；否則依左子樹的 max 決定方向。此程序只找**任意一個**重疊區間，不保證 low 最小，也不列出全部答案。

真正需要證明的是：若目前子樹有答案，選出的那一側仍保留至少一個答案。left.max≥q.low 本身不保證左側一定重疊，不能省略後面的正確性論證。

## 同一棵區間樹：綠線追蹤查詢 [9,10]




第一步讀到左側 max=15≥9；到 [5,7] 時，改讀其左側 max=4<9，因此轉向右側，找到 [8,15]。每一步讀的是「當前節點左孩子的子樹摘要」。



## 三次查詢：成功與失敗都能逐步核對

| 查詢 | 訪問順序 | 分支依據／結果 |
| --- | --- | --- |
| [9,10] | [12,17]→[5,7]→[8,15] | 15≥9 往左；4<9 往右；命中 |
| [18,18] | [12,17]→[20,24]→[16,19] | 15<18 往右；19≥18 往左；命中 |
| [28,29] | [12,17]→[20,24]→[25,27]→NIL | 左側 max 依序不足；最後無解 |

查詢 [28,29] 走到 [25,27] 時，目前區間的 high=27<28，且左右孩子皆空，故最後回傳 NIL。**目前節點不重疊**與**某棵子樹可排除**是不同判斷，不能只看 x.high 就決定剪掉左側。

## 往右：先證明左側完全沒有答案

維持不變量：「若原樹存在重疊區間，目前 x 子樹也至少有一個。」起點為根，顯然成立；若 x 重疊就直接回傳。

當左孩子為 NIL，左側沒有候選。否則若 left.max<q.low，對任何左側後代 j 都有

$$j.high \le left.max < q.low.$$

因此它們全在查詢左方，不可能重疊。加上 x 已經不重疊，若仍有答案，只能在右側，往右保持不變量。

這裡用的是**整棵左子樹的 high 上界**。若只將根改成 [12,13]，查詢 [14,14]，雖然根的 high=13<14，左側 [8,15] 仍是答案；單看根的 high 不足以剪枝。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 往左：為什麼不必回頭搜尋右側？

若 left.max≥q.low，左子樹中必有一個區間 j，其 high **恰等於**這個 max。左側若本來就有答案，走左當然安全。

若左側完全無解，j.high≥q.low 卻仍不重疊，只可能是 j.low>q.high。對任一右側後代 k，BST 的鍵序給出

$$k.low \ge x.low \ge j.low > q.high.$$

所以右側也全在查詢右方；加上 x 不重疊，此時目前子樹根本沒有答案。因此不論左側有沒有解，都不會丟掉原本存在的所有解。

走到 NIL 時，由不變量得知原樹也無解。每次下降一層，成本為 O(h+1)，紅黑底層則為 O(log(n+2))。關鍵是**精確 max 對應實際存在的區間**，不是隨意給的上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 過大的舊 max，也可能讓搜尋漏掉答案

將 [8,15] 改成 [8,27] 後再刪掉。左子樹只剩 [5,7]、[2,4]，精確 max 應為 7；若錯留 27，查詢 [22,22]：

| 位置 | 使用的比較 | 錯誤路徑 |
|---|---|---|
| [12,17] | 舊 left.max=27≥22 | 往左，放棄有答案的右側 |
| [5,7] | left.max=4<22 | 往空的右孩子 |
| NIL | 迴圈結束 | 錯誤回報找不到 |

真正答案是右側 [20,24]。精確 left.max=7 會讓第一步往右找到它。**舊上界 27 沒有左側區間作見證**，因此上一頁「左無解則右亦無解」的論證不再成立。修改後必須由下往上重新取 max，允許摘要下降。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 端點慣例與資料修改，也屬於演算法規格

| 區間慣例 | 非空條件 | overlap(i,q) | 左側可能有解 |
|---|---|---|---|
| 閉區間 [l,h] | l≤h | i.low≤q.high 且 q.low≤i.high | left.max≥q.low |
| 半開 [l,h) | l<h | i.low<q.high 且 q.low<i.high | left.max>q.low |

[4,5] 與 [5,7] 在 5 重疊；[4,5) 與 [5,7) 則沒有交集。[5,5) 是空集合，不能把單點閉區間的結論直接套過來。

修改 low 會改變排序鍵，應刪除舊資料再插入；不能留在原位置只改數值。只改 high 且仍滿足非空條件時，鍵序不變，但須從該節點往上重算 max；縮短區間也不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 找一個、找最左與列出全部，是不同問題

本節搜尋只回傳任一重疊區間。若要列出全部，可在 left.max≥q.low 時訪問左側、檢查 x，再於 x.low≤q.high 且 right.max≥q.low 時訪問右側。每個訪問節點 O(1)，最壞仍可為 O(n)，不能直接沿用單一路徑的對數界。

另一個容易完整分析的方法，是反覆搜尋、暫時刪除命中資料，記錄後最後插回。若共有 k 個答案，包含 k 次成功及最後一次失敗搜尋，總成本為 **O((k+1)log(n+2))**，另需 O(k) 儲存結果。這會暫時修改集合，必須符合介面契約。

要得到 O(log n+k)，還需要另一個演算法或更細的訪問節點數證明。size、max 可同時維護，但這裡的 rank 是依 (low,identity) 排序的位置，並非依 high 排序，也不是重疊區間的數量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B-tree：把一頁內的空間換成較少的層數

一個節點保存多個已排序的鍵，以及分隔各鍵範圍的子指標。搜尋先在頁內選出範圍，再讀取下一頁；成本要分成**頁存取（I/O）與頁內比較、搬移（CPU）**。

本節採鍵互異的集合。每個鍵與它的資料或 record pointer 一起搬移；內節點也能保存實際資料。重複插入回傳 EXISTS，不新增一筆。這與 B+ tree「資料集中在葉、父節點另存 separator」的表示不同。

一節點能否放進一頁，取決於頁大小、鍵與資料指標大小、子指標及頁頭。以下的 I/O 分析假設這個配置已成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最小度 t：先固定每種節點的容量

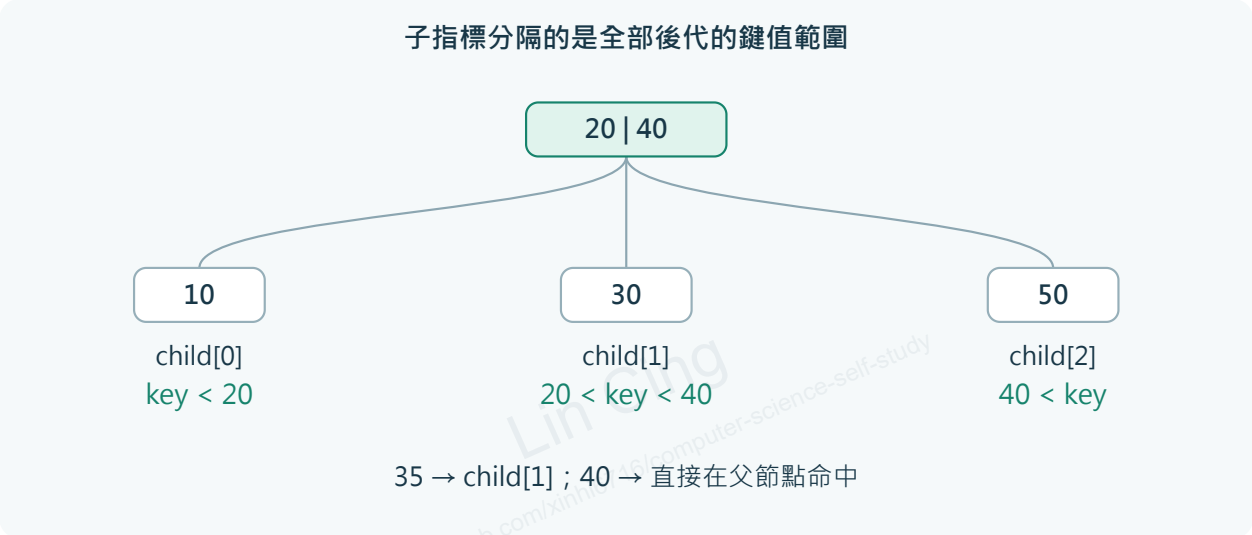
固定整數 t≥2；鍵陣列與子指標陣列皆採 0-based。

| 節點種類 | 鍵數 k | 子指標數 |
| --- | --- | --- |
| 非根節點 | t−1≤k≤2t−1 | 葉為 0；內節點恰為 k+1 |
| 非空根 | 1≤k≤2t−1 | 葉為 0；內節點至少 2 |
| 空樹 | 零鍵葉根 | 0 |

所有葉深度相同；根深度為 0，葉深度為高度 h。非空樹 h=0 表示資料全部放在根；空樹也約定 h=0，但下文的非空高度定理不能直接套 n=0。



## 每個子指標負責整段鍵值範圍




鍵 $a_0<\cdots<a_{k-1}$ 將整個子樹分成 k+1 段。範圍限制適用於**全部後代**，不是只比較子節點的第一個鍵。搜尋 35 要走中間；搜尋 40 直接在父命中。

## order m 與 minimum degree t 不能混用

| 本節採用的定義 | 非根內節點的分支數 | 非根鍵數 |
| --- | --- | --- |
| order m：最大分支數為 m | $\lceil m/2 \rceil \cdots m$ | $\lceil m/2 \rceil - 1 \cdots m-1$ |
| minimum degree t | $t \cdots 2t$ | $t-1 \cdots 2t-1$ |

**m=2t 時容量才完全對應。** m=3 是 2-3 tree，不能令 t=1.5，也不能令 t=2：後者是最多 4 個分支的 2-3-4 tree。

一般「order」的定義並非各書一致；先確認它指最大分支數還是其他容量。上表固定採最大分支數，不從名稱猜公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先分裂與溢位後分裂：前提不同

本節插入先分裂已有 $2t-1$ 鍵的滿節點：提升一鍵，左右各 $t-1$ 鍵，兩邊立即合法。

order m 的溢位後分裂則先讓節點暫存 m 鍵，再提升中位鍵。例如 m=3：葉 [2,8] 插 5，暫成 [2,5,8]，提升 5，留下 [2]、[8]。

若 m=3 卻在滿葉 [2,8] 插入前就分裂，提升任何一鍵都會讓一側零鍵，違反下限 1。因此**容量公式、修復時機與正確性證明必須配套**。刪除同樣有「先修好再下降」與「刪後處理不足」兩種安排。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 高度上界：先數最稀疏的合法樹

假設 n≥1。高度 h 的根至少有 1 鍵；h≥1 時至少有 2 個子節點。深度 i≥1 至少有 $2t^{i-1}$ 個節點，每個至少 t−1 鍵，所以

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

移項並取以 t 為底的對數：

$$h \leq \log_t \frac{n+1}{2}.$$

h=0 時總和為空，得到 n≥1，仍符合。根的最少鍵數是 1，不能誤把根也算成 t−1 鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 高度下界與十億筆資料的頁數

每節點最多 2t−1 鍵、最多 2t 個分支，因此

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

取 t=100、n=$10^9$：h=2 最多 7,999,999 鍵，不夠；h=3 最多 1,599,999,999 鍵，容量足夠。另一方面 h=4 最少 199,999,999 鍵；h=5 最少 19,999,999,999 鍵，太多。故合法樹只能有 h=3 或 4。

一次搜尋走 4 或 5 個節點；若根常駐記憶體、其他路徑頁皆未快取，便是 **3 或 4 次頁讀取**。若根也不在記憶體，再多讀一頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 搜尋：頁內定位，再走唯一可能的子樹

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

若沒有相等鍵，全域範圍限制保證只有 child i 可能包含 k；其餘全部排除。每次下降一層，最後在葉命中或失敗，故終止且正確。lower_bound 可用線性掃描或二分搜尋；兩者不改變頁路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



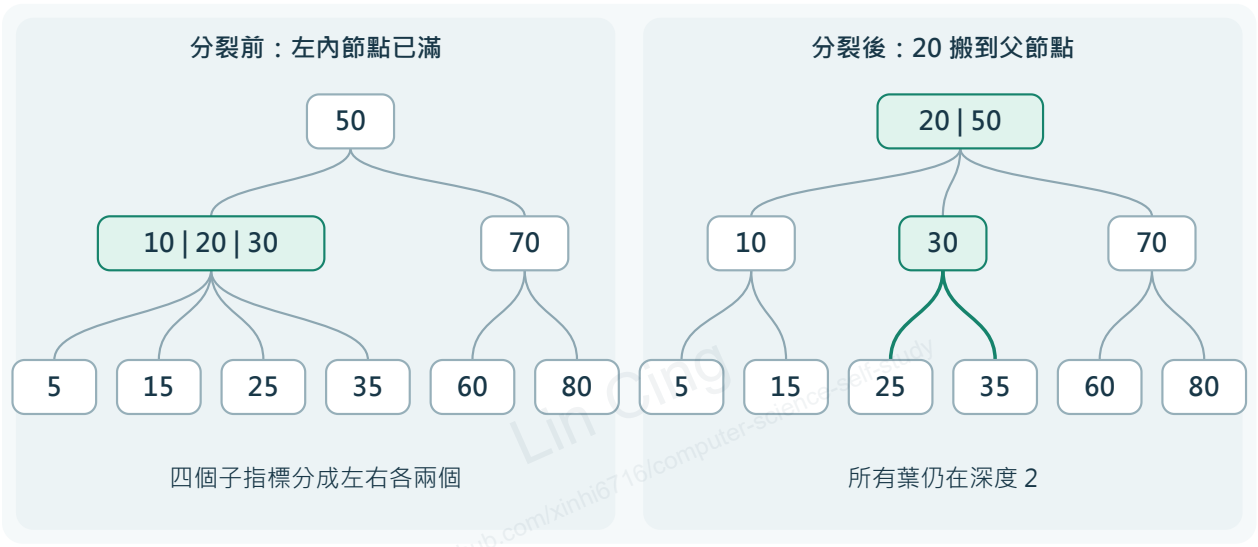
## SPLIT：提升中位鍵，也切開子指標

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

切片採左含右不含，且 old、kids 代表改動前的陣列。左右各 t−1 鍵；若為內節點，各有 t 個子指標。median 搬到父節點，**不在子節點保留副本**。父原先未滿，增加 1 鍵仍合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分裂內節點：四個舊子樹不能遺失



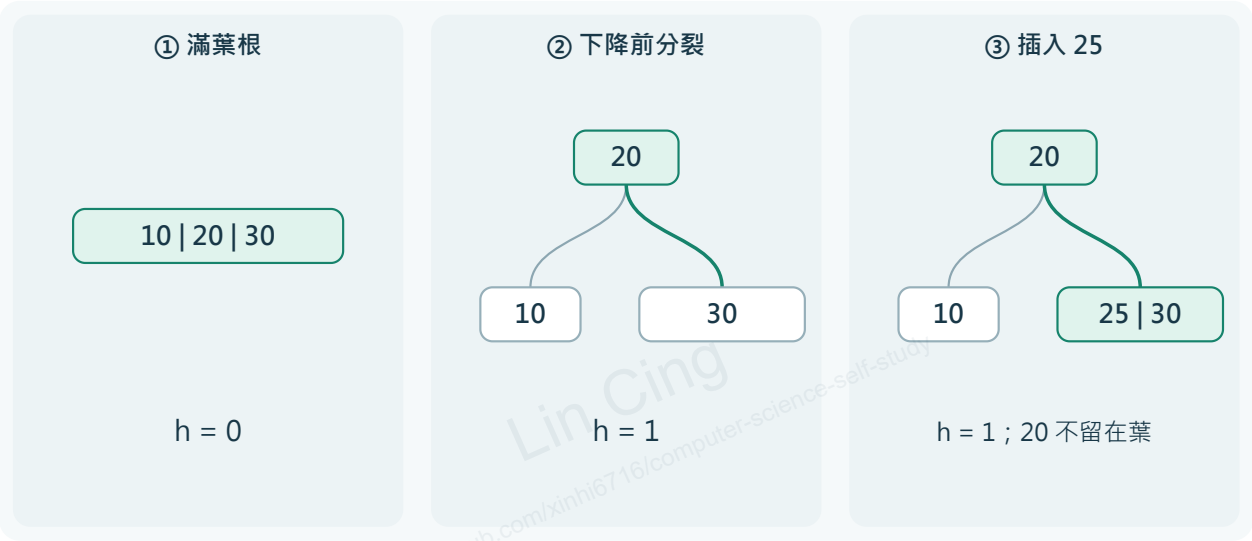

t=2，原父 [50] 的左子節點為 [10,20,30]。提升 20 後，父為 [20,50]；[10] 接 [5]、[15]，[30] 接 [25]、[35]。原父的右子樹 [70] 不變。所有葉仍深度 2。



## 插入：下降時永遠選未滿節點

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

## 根分裂：高度從頂端增加




t=2，滿葉根 [10,20,30] 插 25：先提升 20，再因 25>20 選右葉，得到 [25,30]。只有根分裂新增一層；所有舊葉一起加深，葉深度仍相等。



## 插入正確性：為何不會向上連鎖補救？

迴圈開始時 x 未滿。根若原本滿，先建立新根並分裂，故初始化成立。若下一個子節點滿，因 x 未滿，可安全把一鍵提升到 x；分裂後的兩個子節點各有 t−1 鍵，選中的一側仍未滿。

每次下降一層；到葉後只增加一鍵，最多從 2t−2 變 2t−1，不溢位。SPLIT 保存全部鍵及子樹範圍，最後插入的 k 也落在搜尋選定的範圍，因此鍵集合恰增加 k。

與溢位後分裂相比，這裡把修復提前到下降之前，所以無須回到已離開的父頁處理溢位。插入前查重另走一條路徑，仍不改變漸近成本。

## 刪除：先替即將下降的子問題留一格餘裕

非根節點通常只需 t−1 鍵；本節刪除加強條件：**每次遞迴進入非根節點時，先保證它至少有 t 鍵**。這樣在該層失去一鍵，仍有合法的 t−1 鍵。

| 情況 | 處理 |
| --- | --- |
| 在葉找到 k | 直接刪除 |
| 在內節點找到 k，左側至少 t 鍵 | 用前驅取代，從左側刪前驅 |
| 左側不足 t、右側至少 t | 用後繼取代，從右側刪後繼 |
| 相鄰兩側皆 t−1 | 兩側與父鍵合併，再刪 k |

未找到 k 時，也要先把目標子節點修到至少 t 鍵再下降。根可例外較少；若最後變成零鍵內根，改用唯一子節點當新根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 向左兄弟借：父鍵下降，兄弟最大鍵上升

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

前提：y 有 t−1 鍵、s 至少 t 鍵；兩者皆在同一層。s 的最右子樹介於新、舊分隔鍵之間，必須一起移到 y 的最左側。完成後 y 有 t 鍵，s 仍至少 t−1 鍵；父鍵數不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 向右兄弟借：子指標也要移到正確一側

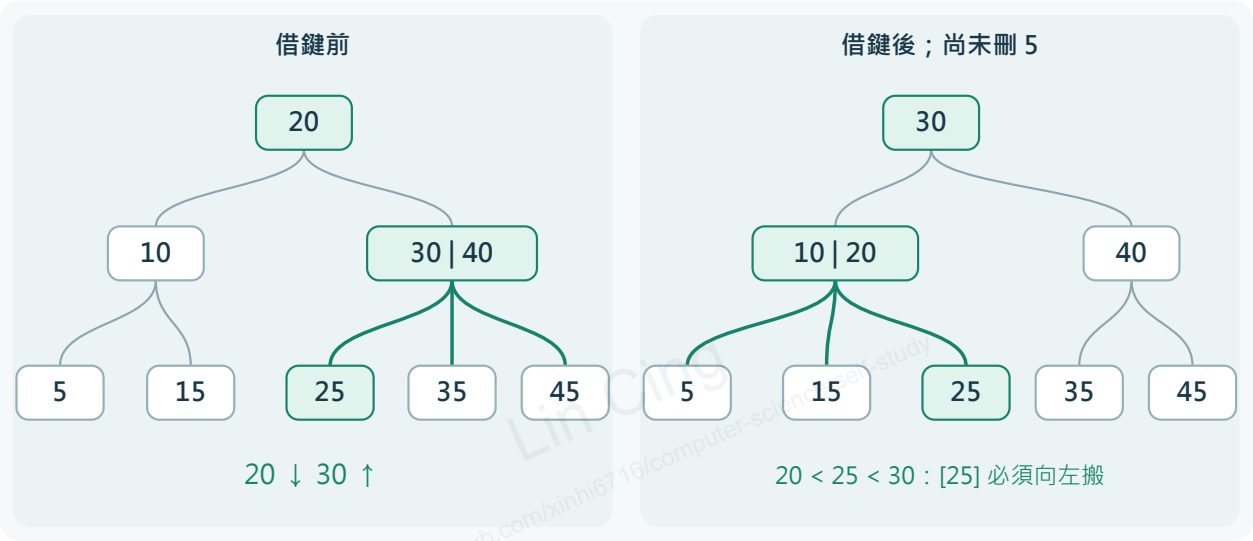
```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

前提同樣是 y 有 t−1 鍵、s 至少 t 鍵。s 的最左子樹移到 y 最右側。只搬鍵、不搬該子指標，會使整段鍵值範圍落在錯誤子樹；即使每個節點內仍排序，也已不再是合法搜尋樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 借鍵數例：25 的子樹為何必須向左搬？




t=2，要刪 5。左內節點 [10] 只有 1 鍵，右兄弟 [30,40] 有 2 鍵。先借：20 下降，30 上升；[25] 介於 20 與 30，必須成為 [10,20] 的最後一棵子樹。圖中只完成下降前修復，尚未刪 5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

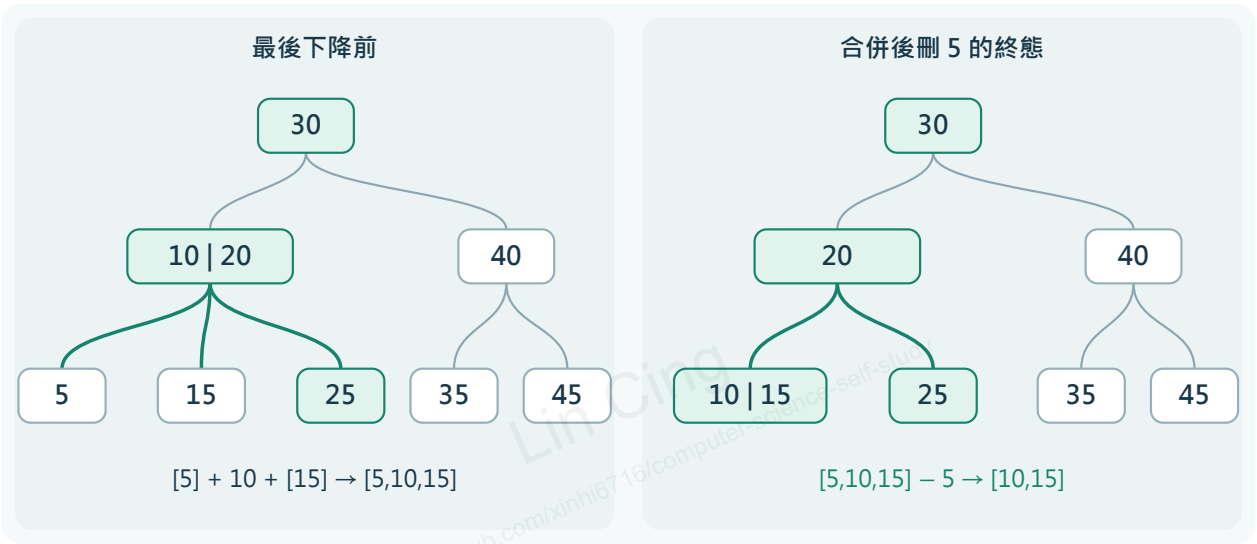
## MERGE：父分隔鍵也是合併內容的一部分

```
MERGE(p,i):                      // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

新 y 有 (t−1)+1+(t−1)=2t−1 鍵，恰好滿而不溢位；若為內節點，有 t+t=2t 個子指標。父少一鍵及一個子指標。非根父進入時至少 t 鍵，現在仍有 t−1；若空的是根，最後統一縮高。



## 接續刪 5：先合併兩個最小葉




在 [10,20] 內，目標葉 [5] 與右兄弟 [15] 各只有 1 鍵。將父鍵 10 一起合入 [5,10,15]，再刪 5 得 [10,15]。左內節點剩 [20]，連 [10,15]、[25]；根 [30] 和右內節點 [40] 不變。中序恰少 5，所有葉仍深度 2。

## 內節點命中：前驅與後繼都搬整筆資料

| t=2 的初始樹：根及兩個葉 | 刪除 | 結果 |
| --- | --- | --- |
| 根 [20]；左 [5,10]、右 [30] | 20 | 前驅 10 上移；根 [10]，葉 [5]、[30] |
| 根 [20]；左 [5]、右 [30,40] | 20 | 後繼 30 上移；根 [30]，葉 [5]、[40] |

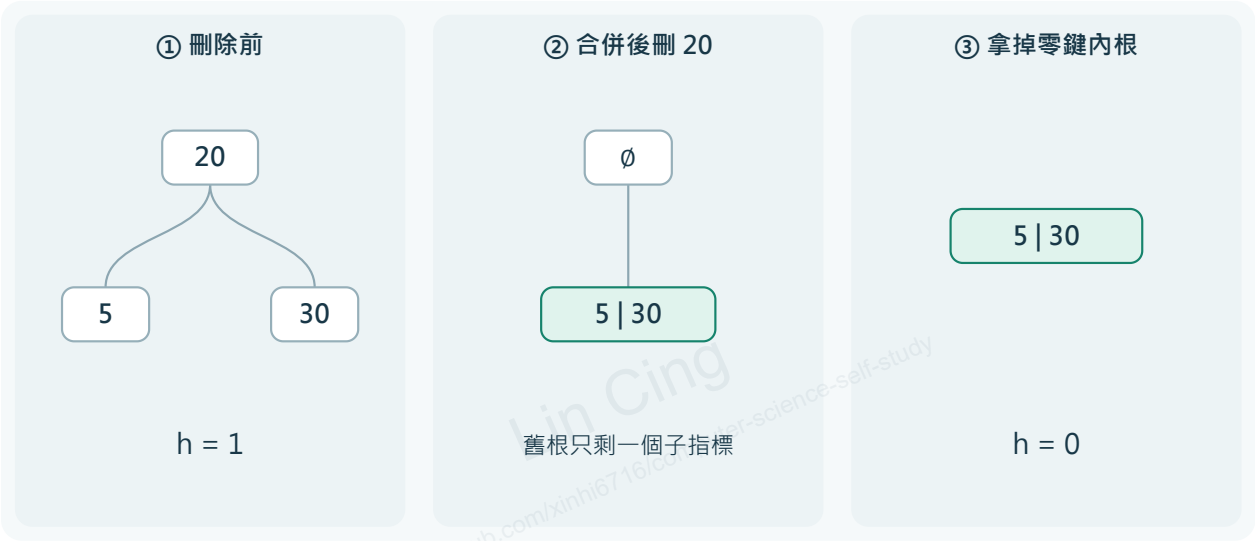
取前驅時，沿左子樹的最右子指標到葉，取最後一鍵；後繼則沿右子樹的最左子指標到葉，取第一鍵。選取後必須從原子樹刪掉那筆資料，不能讓同一鍵在父、葉各留一份。

記錄指標也要同步搬移；只改鍵、保留父的舊資料，會讓查詢回傳錯誤的 record。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 根縮減：只有根能拿掉整層




根 [20] 與葉 [5]、[30] 合成 [5,20,30]，再刪 20 得 [5,30]。舊根零鍵且只有一個子節點，用該子節點取代，h 從 1 變 0。一般內節點不能照做，否則只縮短一側路徑，破壞葉等深度。

## 刪除主程序：命中鍵的三種內部情況

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

此處 keys 陣列存整筆記錄，比較使用記錄的 key；q 是保存下來的記錄值或穩定指標，不可在刪除原位置後讀取已釋放的物件。



## 刪除下降：借鍵優先，合併時修正索引

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

往左合併時，原 child i 被併入 child i−1，必須 i−=1。對外 DELETE(k) 呼叫 DELETE-NODE(root,k)，**不論 FOUND 或 NOT_FOUND，最後都檢查零鍵內根並縮高**，然後回傳原本的搜尋結果。

## 找不到鍵，也可能合法地改變樹形

根 [20]、葉 [5]、[30]，t=2，要求刪 99。先選右子節點；右葉只有 t−1=1 鍵，左兄弟也無法借，所以先合併成 [5,20,30]，索引從 1 改為 0。

在新葉找不到 99，回傳 NOT_FOUND；外層仍須縮掉零鍵內根。最後是葉根 [5,20,30]，集合完全不變，高度從 1 降到 0。

**NOT_FOUND 表示集合沒有刪掉元素，不保證樹形完全不變。** 若 API 要求失敗時連樹形都不動，可在開始前另做存在性搜尋，並將這條額外路徑算入成本。空樹與刪最後一鍵後皆保留零鍵葉根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪除的一般證明與兩種成本

以剩餘高度歸納：葉直接刪除正確；內部替換後遞迴刪掉前驅或後繼，集合只少 k。借鍵與合併保持全域鍵序、子指標數及葉等深度，並讓下一個子問題至少有 t 鍵。非根父即使因合併少一鍵，仍保有 t−1。根例外由外層縮高處理。每次遞迴嚴格下降，故終止。

每層只碰常數個相鄰頁，I/O 為 $O(h+1)=O(1+\log_t(n+1))$。頁內陣列搬移最多 O(t)，插入、刪除 CPU 上界 O(t(h+1))。前驅／後繼若先找再刪，多走的路徑仍總計 O(h+1)，不是每層都重走整棵樹。

純搜尋在頁內二分，可用 O((h+1)log t) 比較；這不會把插入、刪除的陣列搬移也降為 O(log t)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fibonacci heap：把整理留到取出最小值時

最小堆積只要求父鍵不大於子鍵，不要求不同子樹彼此排序。Fibonacci heap 保留一串滿足堆積序的樹：插入或合併時先接上根串列，直到 EXTRACT-MIN 才合併度數相同的根。

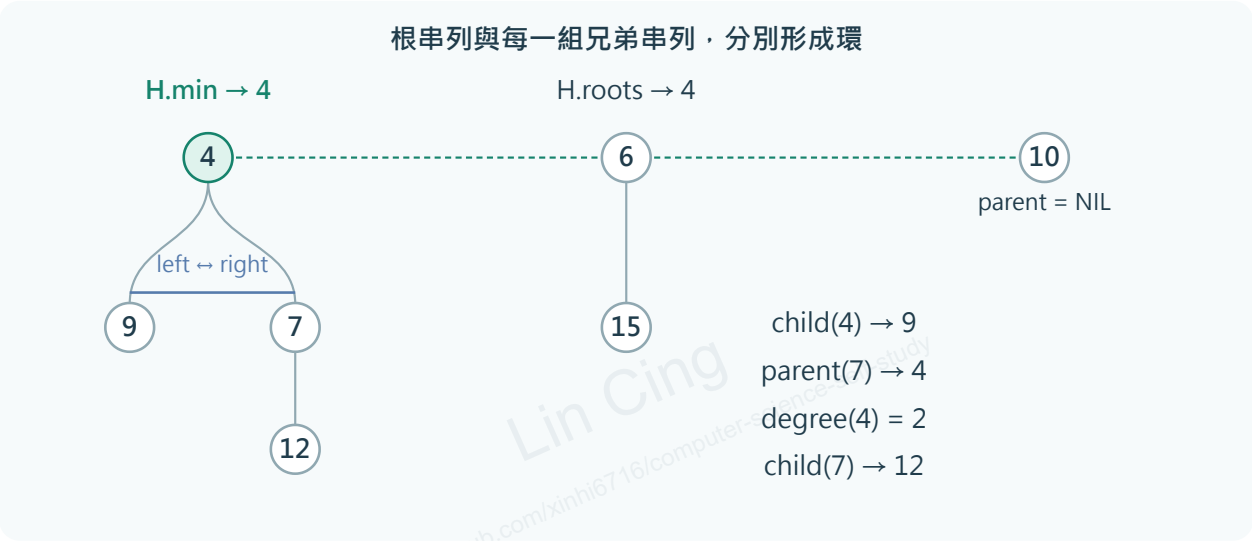
**度數（degree）是直接孩子的個數，不是樹高。** 經過切割，樹不再保證是完整二項樹。後面將證明最大度數為 O(log(n+1))；這個結論不代表根到葉的路徑也只有對數長度。

DECREASE-KEY(H,x,k) 與 DELETE(H,x) 都要求給定有效的節點參照（handle）。只給鍵值、再尋找對應節點，可能需要走遍整個堆積；這筆搜尋成本另計。相同鍵可以共存，但節點身分不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 指標各管一件事：根串列、子串列與最小值




圖中根為4、6、10；4的孩子是9、7，7還接著12。根串列入口與最小值指標用途不同；每組兄弟另有自己的環狀串列。

## 欄位保存什麼？標記從哪一次連結開始算？

節點保存 `key`、`parent`、`child`、`left`、`right`、`degree`、`mark`。`child` 指向任一孩子，其兄弟以環狀雙向串列相連；只有一個孩子時，該節點的 left、right 都指向自己。H.roots 是根串列入口，H.min 指向最小根，H.n 記錄總節點數；空堆積的兩個入口皆為 NIL。

**標記 記錄非根節點在「這一次接到目前父節點之後」，是否已失去一個孩子。** 第一次失去孩子先標記，第二次就切離父節點。這不是終生累積次數；節點切成根、之後重新接到另一個父節點時，紀錄重新開始。

本節統一讓根的 mark=false；提升為根，或重新連結成孩子時，都清除標記。這項慣例也用於後面的勢能計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 雙向環串列：能常數時間合併的實際原因

刪除已知節點 x，只要接 x.left.right=x.right、x.right.left=x.left；若 x 是入口就換入口，若是獨生元素就設 NIL，最後讓 x 左右指回自己。只用 O(1) 次指標更新。

兩個非空環入口 a、b，先存 al=a.left、bl=b.left，再接 al.right=b、b.left=al、bl.right=a、a.left=bl，即把兩環接成一環。空環直接取另一入口。

以下 ATTACH-ROOT(H,x) 將已脫離原串列的 x 接到根串列，設 parent=NIL、mark=false；REMOVE-CHILD(p,x) 維護 p.child、環指標及 度數。這些基本操作都不搜尋鍵，也不走訪全部兄弟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## INSERT、MINIMUM 與破壞式 UNION

| 操作 | 完整處理 | 實際成本 |
|---|---|---|
| MAKE | roots=min=NIL，n=0 | O(1) |
| INSERT(x) | 新節點 degree=0、child=NIL；接根、比較 min、n 加一 | O(1) |
| MINIMUM | 回 min；空堆積回 NIL | O(1) |
| UNION(H1,H2) | 接兩根環，取兩個非空 min 的較小者，n 相加 | O(1) |

UNION 消耗兩個互不重疊的舊堆積；結果取得全部節點，原入口不可再拿來獨立操作。H1=H2 不允許。兩者皆空時結果仍空。這個界不包含逐節點重寫「屬於哪個 heap」欄位；若實作要求那種更新，不能沿用 O(1) 合併界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## EXTRACT-MIN：先提升孩子，再整理根

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

每次取目前 孩子 再移除，避免一邊修改環串列、一邊用失效的 next 繼續走。所有提升節點清 父節點 與 標記；z 被隔離後才交回呼叫者。若只有一個根但它還有孩子，提升後堆積仍非空，不能只憑「原來一個根」就回空。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## CONSOLIDATE：讓每個 度數 最後只留一個根

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LINK 與 度數 表的實作契約

LINK-ISOLATED 把同 度數 的兩個獨立根合成一棵：較大鍵 y 成為 x 的 孩子，x.degree 加一；相等鍵選任一根。A 可用按需延伸的陣列，最多 O(D(n)+1) 槽。上面採逐根取出的版本，避免重複處理已被連結的舊根。

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 整理根的不變量與終止理由

每次外層開始時，尚未處理的根留在 H.roots，已處理節點則分布在 A 指向的互不重疊子樹中；A 每個槽至多存一棵 度數 等於槽號的樹。所有原節點恰好出現在這兩部分之一。

內層維持 d=x.degree；遇到同 度數 的 y 就連結，節點集合與堆積序不變，兩根變一根，d 增一。原有根數有限，每次 link 都讓森林根數減一，故不可能無限循環。

外層每次取掉一個未處理根，因此結束時 A 包含全部剩餘樹，且 度數 各異。重建根串列並逐根比較最小值，得到正確 H.min。注意「度數 各異」只在整理完成後保證；INSERT、UNION、CUT 都可再次造成重複。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



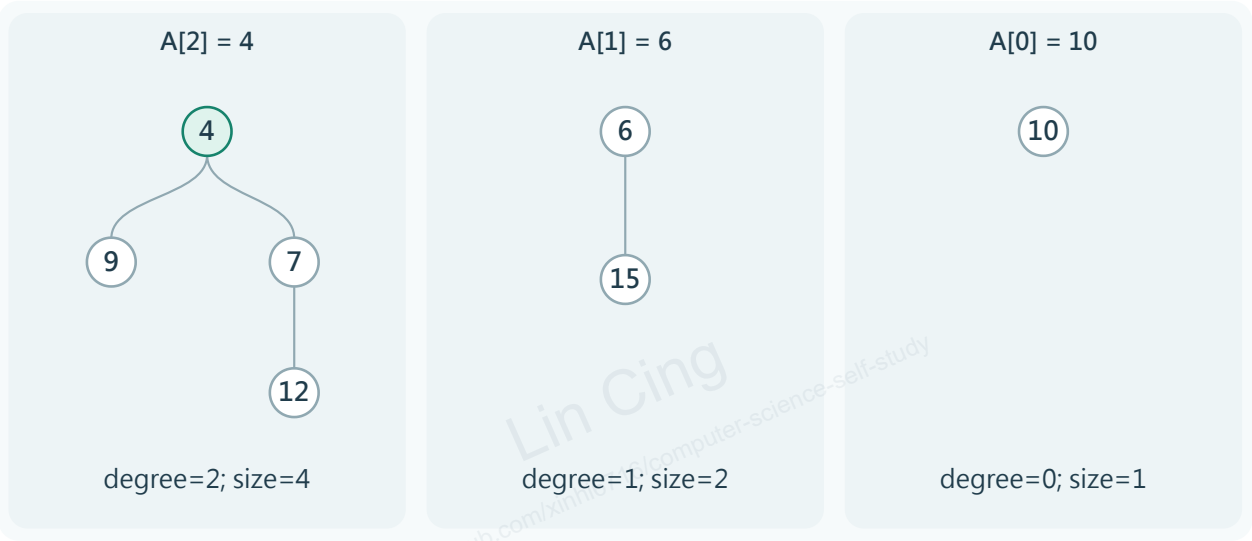
## 數例：四次 link，逐槽追蹤整理過程

從八個單節點根 1、4、9、7、12、6、15、10 開始，先取出最小根 1，再按 4、9、7、12、6、15、10 的順序整理；它們的初始度數皆為 0。

| 取出根 | 這一輪的 link | 非空 A 槽（度數:root） |
|---|---|---|
| 4 | 無 | 0:4 |
| 9 | 9 掛到 4 | 1:4 |
| 7 | 無 | 0:7，1:4 |
| 12 | 12 掛到 7；再把 7 掛到 4 | 2:4 |
| 6 | 無 | 0:6，2:4 |
| 15 | 15 掛到 6 | 1:6，2:4 |
| 10 | 無 | 0:10，1:6，2:4 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 數例終態：樹形、度數 與根數一起核對




根 度數 依序為2、1、0；節點數4+2+1=7，min=4。綠色根4是最小值，其餘根不必按鍵值排序。



## 同一數例：根數下降如何釋出勢能

根 4 的孩子是 9、7，7 的孩子是 12；根 6 的孩子是 15；根 10 沒有孩子。三個根的度數分別為 2、1、0，總節點數為 4+2+1=7，每條父子邊都滿足父鍵≤子鍵。

| 階段 | 根數 r | 標記數 m | 勢能 K(r+2m) |
|---|---|---|---|
| 取出 1 之前：八個單節點根 | 8 | 0 | 8K |
| 移除 1、完成四次連結之後 | 3 | 0 | 3K |

這次勢能下降 5K：移除最小根使根數少一，再由四次連結少四。它把「先前插入時存下的額度」接回後面的攤銷公式；**5K 是勢能變化，不是五條機器指令的實際成本。**

本例恰為未經切割的二項樹。發生 CUT 後，度數 d 不再保證有 $2^d$ 個節點；後面的通用下界改為 $F_{d+2}$。

## DECREASE-KEY：只有違反父子順序才切

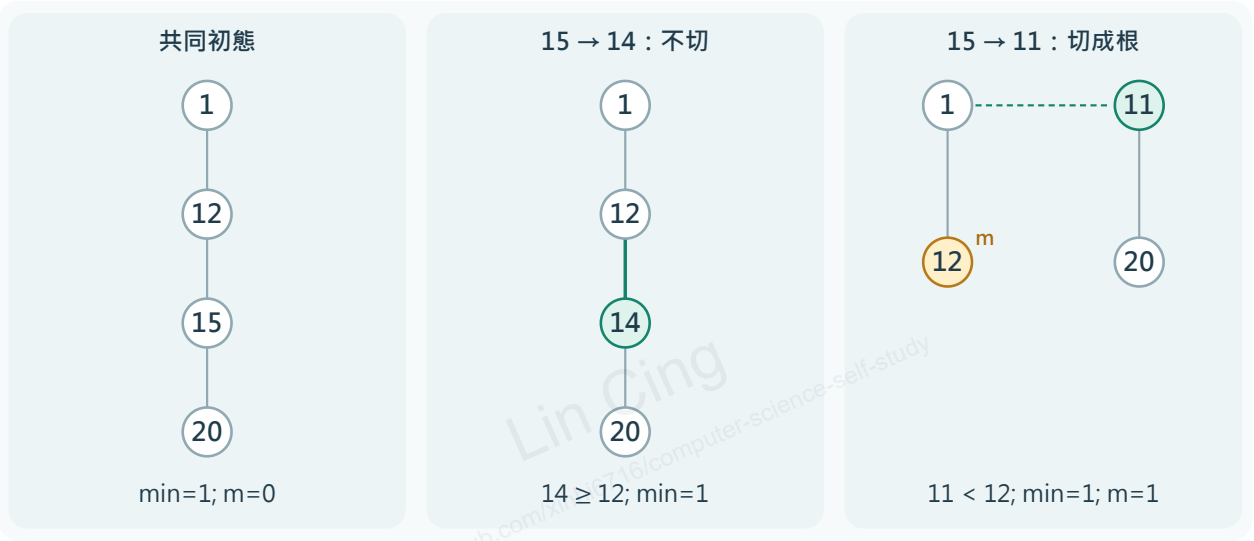
```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

只降低 x 的鍵，不會破壞 x 與孩子的順序；可能壞掉的只有 父節點→x。若 x 已是根，或新鍵仍≥父鍵，皆不切。min 只可能保留舊值或改成 x，其他節點鍵值沒有改變。



## 數例：15→14 不動；15→11 帶著子樹上來




兩個情況都從左圖開始。降低至14不切；降低至11則連孩子20一起提升，並標記第一次失去孩子的非根12。

## 兩種遞減鍵值，從同一初態分開追蹤

根 1 的孩子 12 有孩子 15，15 又有孩子 20。兩個情況各自從這個初態開始：

| 新鍵 | 與父鍵 12 比較 | 指標與標記的變化 |
|---|---|---|
| 14 | 14≥12 | 不切割、不改標記 |
| 11 | 11<12 | 連同孩子 20 一起切成新根；12 若原本未標記，便標記 |

切割搬動的是**以該節點為根的整棵子樹**，不是只取出一個鍵；20 仍是新鍵 11 的孩子。第二種情況中，12 是非根且首次失去孩子，因此留下標記，根 1 不受影響。

兩種結果的最小值都仍是 1。若降低的是既有根，例如把根 3 改為 0，只更新最小值指標，不會切割。



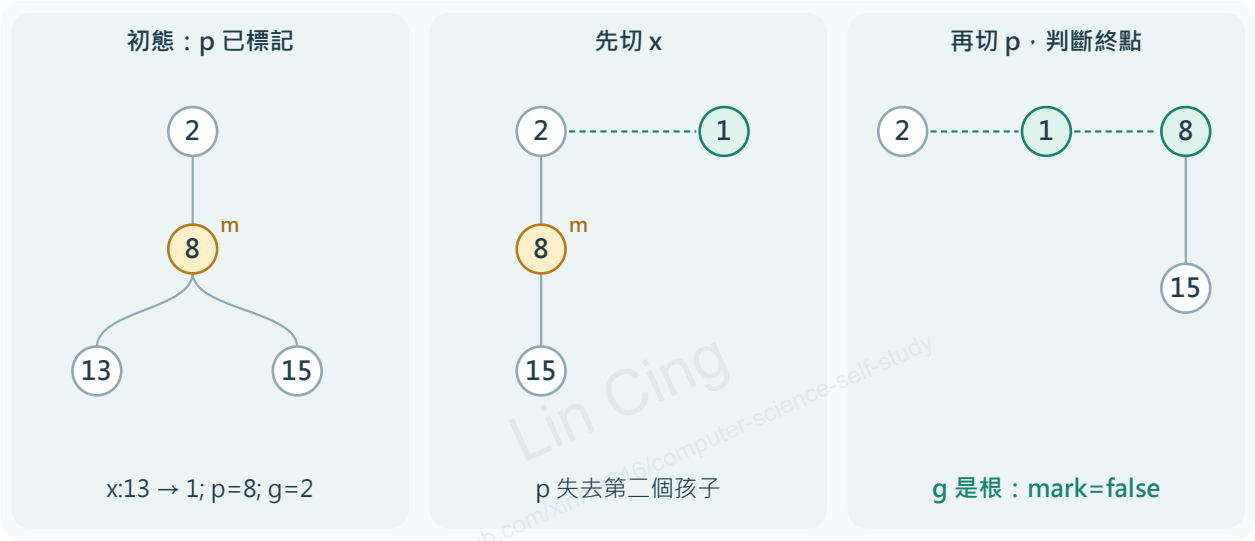
## CASCADING-CUT：第一次失去孩子先標記，第二次才切走

```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

呼叫時 y 剛失去一個 孩子。未標記表示這是本次成為 孩子 後的第一次損失，留下記號便停；已標記表示第二次損失，把 y 切成根並清 標記，父節點 p 也因此失去孩子，所以繼續處理 p。

堆積序仍正確：切掉父子邊不會讓剩餘邊失序。級聯的作用是限制非根能失去多少孩子，保住 度數 與子樹大小之間的關係，而不只是修復鍵值順序。

## 級聯分支一：祖父是根，就停在根




先切x，再切已標記的p。到g時它已是根，所以停止；不新增g的標記。p原有的孩子15仍跟著p。



## 到根就停：根串列與勢能逐項核對

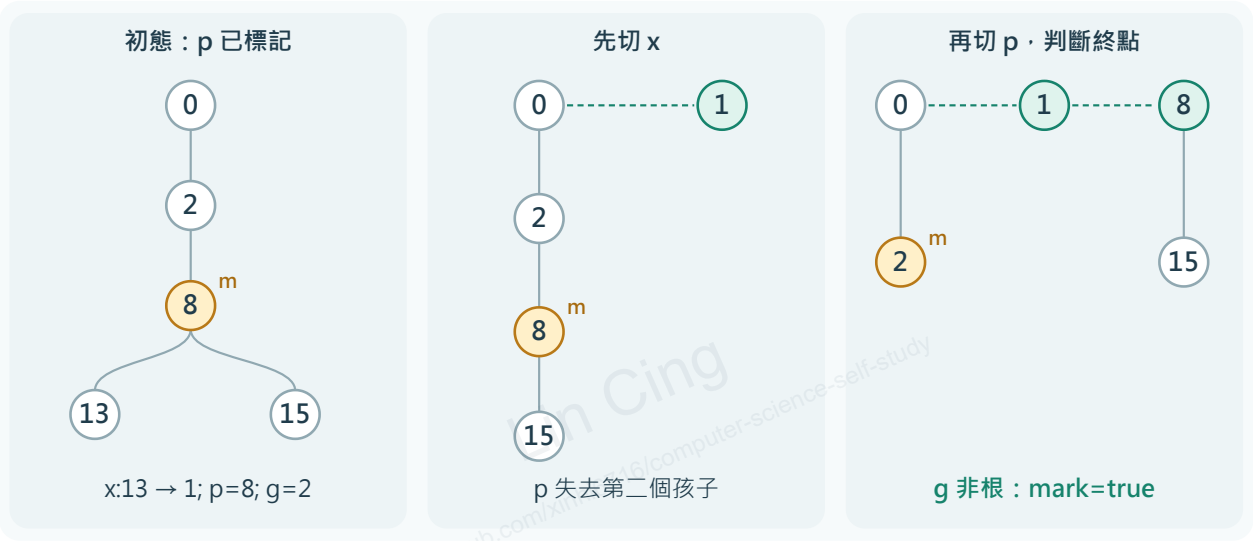
初態只有根 g=2，其孩子 p=8 已標記；p 的孩子為 x=13、s=15。p 的標記表示它在這次成為孩子之後，先前已失去另一個孩子。現在把 x 從 13 降到 1。

先切 x，再切已標記的 p。到 g 時，g.parent=NIL，因此停止，**不把根 g 標記**。最後根為 2、1、8；8 仍帶著孩子 15，最小值改為 1。

| 狀態 | 根數 r | 標記數 m | 未縮放勢能 r+2m |
|---|---|---|---|
| 操作前 | 1 | 1 | 3 |
| 切 x、切 p，抵達根 g 後 | 3 | 0 | 3 |

新增兩個根增加 2，清除 p 的標記減少 2，勢能淨變化為 0。這裡先核對 r+2m；實際採用的勢能還會乘上一個固定常數 K。

## 級聯分支二：祖父非根且未標記，才留下記號




這次g還有父節點q，且原本未標記。切x、p之後，標記g而停；因為根q=0仍最小，min不變。



## 尚未到根：清除 p 的標記，再標記 g

在前一個局部結構上方接根 q=0，令 g=2 為未標記非根，而 p=8 仍已標記。把 x=13 降到 1：先切 x、再切 p；抵達 g 時，g.parent=q 且 g.mark=false，所以標記 g 後停止。

最後根為 q=0、x=1、p=8；q 仍接 g=2，p 仍接 s=15，最小值保持 0。

| 狀態 | 根數 r | 標記數 m | 未縮放勢能 r+2m |
|---|---|---|---|
| 操作前 | 1 | 1 | 3 |
| p 清標記、g 新增標記後 | 3 | 1 | 5 |

若 g 原本也已標記，就把 g 一起切走，繼續處理 q。**停止位置由父節點與標記共同決定**；「祖父被標記」只適用於祖父非根且先前未標記的分支。

## 同一個勢能同時支付所有操作

取固定常數 K>0，對所有現存且互不重疊的堆積定義

$$\Phi = K \sum_{H} \big( r(H) + 2m(H) \big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

K 只需選一次，大到能支付以下常數時間指標操作，之後不可隨操作更換。由空堆積開始 Φ₀=0 且始終非負，故 Σcᵢ≤Σĉᵢ。若從已有森林開始，總界須加初始 Φ₀。

INSERT 增一根、不增 標記，因此 ΔΦ=K，攤銷仍 O(1)。MINIMUM 與 MAKE 不改勢能。UNION 將兩個舊堆積替換成一個，根數與標記數相加，總勢能不變，因此實際與攤銷都 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為何一串級聯切割只收常數攤銷成本？

若沒有切割，實際與勢能變化都 O(1)。否則令 c≥1 為總切割次數，包括最初切 x；其餘 c−1 次都切掉已標記的非根節點，並清除它們的 標記。結尾至多新標記一個非根，x 的舊標記若存在也被清除，因此

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta \Phi \le K(4 - c).$$

存在固定 a，使實際成本≤a(c+1)。選 K≥a，得到

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

這是對任何合法操作序列的保證，不需要隨機輸入。單次實際切割仍可能沿長路徑花線性時間；O(1) 是攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## EXTRACT-MIN 的成本：根多時，用先前存的勢能付款

設操作前有 n 節點、r 個根，最大 度數 上界為 D=D(n)。取出最小根後，待整理根數 R≤r−1+D。每次 link 讓根數減一，所以全部 link 次數≤R；處理孩子、掃根、清空與重建 度數 表，實際成本 ≤a(r+D+1)。

整理後至多 D+1 個根。提升與 link 都只清 標記、不新增，因此

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1)$$

最後一步用同一個 K≥a；空堆積與單節點另為 O(1)。這裡先得到 O(D+1)，接下來必須證明 D=O(log(n+1)) 才能宣稱 O(log(n+1)) 攤銷時間。連續插入 n 個根後第一次取出，實際可達 Θ(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 度數 證明的關鍵：孩子最多只少一個孩子

令 x 現在有 d 個孩子，按它們**最近一次接到 x 的時間**排序為 $y_1, \dots, y_d$。對 i≥2，在 $y_i$ 接上當時，目前仍在的較早孩子 $y_1,\cdots,y_{i-1}$ 都已在 x 下面，因此 x 當時 度數 至少 i−1。

LINK 只合併相同 度數 的根，所以 $y_i$ 當時也至少有 i−1 個孩子。之後若失去第二個孩子就會被切離 x；既然 $y_i$ 現在仍是 x 的孩子，至多只少一個。因此

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

這不是說兄弟的目前 度數 已排序，也不是說每個非根終生只能失去一個孩子；切成根再被連結後，歷史會重新計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從孩子下界推到 Fibonacci 子樹大小

對子樹節點數做強歸納，證明任意 degree=d 的節點 x 都有 size(x)≥$F_{d+2}$。孩子子樹比 x 小，可套歸納假設；F 數列非遞減，配合上一頁的 度數 下界。

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2} \ (j \geq 2).$$

d=0 時 size≥1=F₂；d=1 時至少自己與一個孩子，size≥2=F₃。d≥2 時，第一個孩子至少一節點，其餘依序給出

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

最後的恆等式由 $\sum_{i=0}^{d} F_i$=$F_{d+2}$−1 得到：d=0 為 0=F₂−1；歸納時加上 $F_{d+1}$，再用 Fibonacci 遞迴即可。這個證法以子樹大小歸納，不偷用尚未證明的「最小大小隨 度數 單調」。



## 完成 度數 界：指數大小換成對數 度數

令 ϕ=(1+√5)/2，ϕ²=ϕ+1。證明 $F_{d+2}$≥ϕᵈ：d=0 為 1≥1，d=1 為 2≥ϕ；d≥2 時

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

所以對 n≥1 的堆積中任意節點，n≥size(x)≥$F_{d+2}$≥ϕᵈ，得到 $d \leq \lfloor \log_\varphi n \rfloor$。因此最大 度數 為 O(log(n+1))，度數 表大小、整理後根數與 EXTRACT-MIN 攤銷成本皆為 O(log(n+1))。

| 度數 d | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 子樹節點數下界：$F_{d+2}$ | 1 | 2 | 3 | 5 | 8 | 13 | 21 |

這是保證下界，不宣稱每個 d 都能由某個特定操作序列恰好達到等號。空堆積不取 log 0，另行處理。

## DELETE：指定節點必須真的成為唯一最小

DELETE(H,x) 可先將 x 的比較鍵改成比所有一般鍵都小的專用哨兵，再 EXTRACT-MIN。原本若為非根，DECREASE-KEY 會正確處理父節點失去孩子及級聯切割；取出後連帶提升 x 的所有孩子並整理。

不能只寫「改成負無限大」卻允許其他節點也有相同比較值；否則 min 可能仍指向別的節點，刪錯身分。可把比較鍵擴充為 (tag,key)，平常 tag=1，僅本次目標 tag=0，確保它唯一最小；刪完再把回傳節點的原鍵還原。

以同一勢能把兩操作相加，中間勢能相消，攤銷成本為 O(1)+O(log(n+1))。直接拆除內節點的另一種實作也可以正確，但必須另處理孩子提升、父的 標記 與級聯；不能只把孩子放到根串列就算完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 操作成本總表：先確認輸入與計費範圍

| 操作 | 單次實際上界 | 攤銷上界 |
| --- | --- | --- |
| MAKE、MINIMUM、INSERT、UNION | O(1) | O(1) |
| DECREASE-KEY（已知 handle） | O(n+1) | O(1) |
| EXTRACT-MIN、DELETE（已知 handle） | O(n+1) | O(log(n+2)) |
| 只給任意鍵的 SEARCH | O(n+1) | 堆積序不會自動改善搜尋界 |

表中使用 log(n+2)，讓 n=0 的空堆積也保留常數成本；非空時與慣用的 O(log n) 同階。DELETE、DECREASE-KEY 要求有效節點，因此不能在空堆積提供這種 handle；空 EXTRACT-MIN 則合法，直接回傳 NIL。

n 為相關堆積的節點數。UNION 要求兩個輸入互不重疊，並消耗舊入口。成本不包含配置器、逐節點所有權重標記，或尋找 handle 的額外工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把操作次數換成圖演算法的總成本

圖演算法若有 O(V) 次插入／取出、O(E) 次已知 handle 的遞減鍵值，便得到 O(E+V log(V+1)) 的堆積總成本。是否值得實作仍取決於常數、記憶體與存取模式；漸近界不是實測速度。

例如 V=1000、E=8000，且每個頂點至多插入與取出各一次，每條邊至多觸發一次遞減鍵值：

| 工作 | 最多次數 | Fibonacci heap 的總攤銷上界 |
| --- | --- | --- |
| INSERT | 1000 | O(V) |
| EXTRACT-MIN | 1000 | O(V log(V+1)) |
| DECREASE-KEY | 8000 | O(E) |

相加得到 O(E+V log(V+1))；若使用一般二元堆積，對應上界為 O((E+V)log(V+1))。數字是操作次數上限，並非毫秒或可直接比較的精確成本；兩種結構的常數並不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 利用整數位元，換一種搜尋模型

紅黑樹只比較鍵值；vEB（van Emde Boas tree）還使用鍵的二進位表示。固定集合 $S \subseteq \{0, \ldots, u-1\}$，$u = 2^m$、$m \geq 1$，**n=|S| 與鍵值範圍（universe）大小 u 是不同參數**。本節不存重複鍵。

假設 word RAM 的一個 word 可容納鍵與位址；位移、遮罩、比較、陣列索引皆為常數時間。這不適用於任意精度整數的位元成本模型。

目標是最壞 $O(1 + \log\log u)$ 的查找、插入、刪除、前驅與後繼，最小／最大值為 O(1)。這不違反比較排序下界：演算法使用了額外的整數運算，也須支付初始化與記憶體成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 先看少了什麼：直接定址與遞迴分組

直接定址的 bit vector 能在 O(1) 時間查找、插入與刪除，但若 $S = \{0, u-1\}$，從 0 向右掃描後繼可能花 Θ(u)。在 bit vector 上建立二元摘要樹，可把此查詢降為 O(log u)。

把 universe 分成約 $\sqrt{u}$ 組，再把各組與非空組號都遞迴表示，仍**不保證** O(log log u)：若先遞迴找組號，再遞迴找該組最小值，會有

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

因此成本仍是 Θ(log u)。關鍵不只在縮小 universe，還要讓每層最多留下**一次非定值成本的遞迴**；min/max 快取就是為此而設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 奇數位元寬度：高位與低位不能各除一半

令 $\ell = \lfloor m/2 \rfloor$、$h = \lceil m/2 \rceil$，$L = 2^{\ell}$、$H = 2^{h}$，所以 u=HL。對 u>2：

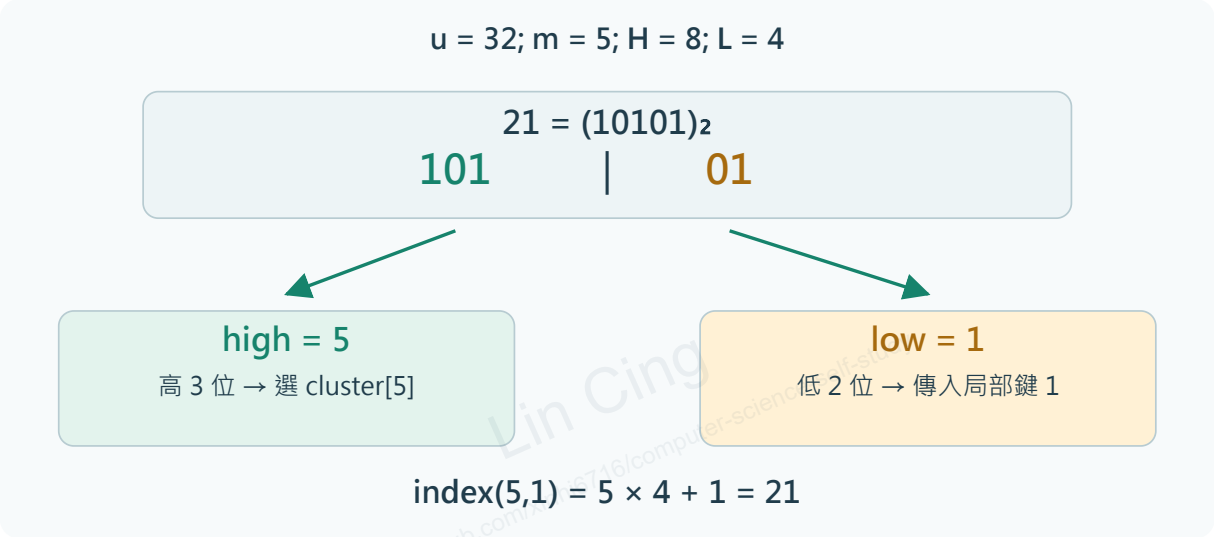
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i, j) = iL + j.$$

有 H 個分組（cluster），每組的 universe 大小為 L；摘要（summary）存 cluster 編號，其 universe 大小為 H。**每層都依該節點的 u 重新計算**，不能沿用頂層遮罩。

u=32 時，m=5、h=3、ℓ=2、H=8、L=4。$x = 21 = (10101)_2$ 切成 $101 \mid 01$，所以 high=5、low=1，還原為 5×4+1=21。u=2 直接處理，不建立下一層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一個鍵的兩個座標




high 選 cluster；low 是傳給該 cluster 的鍵。summary 裡的 5 表示「第 5 組非空」，不是原集合真的存了鍵 5。這兩種資料不能混成同一組鍵值。



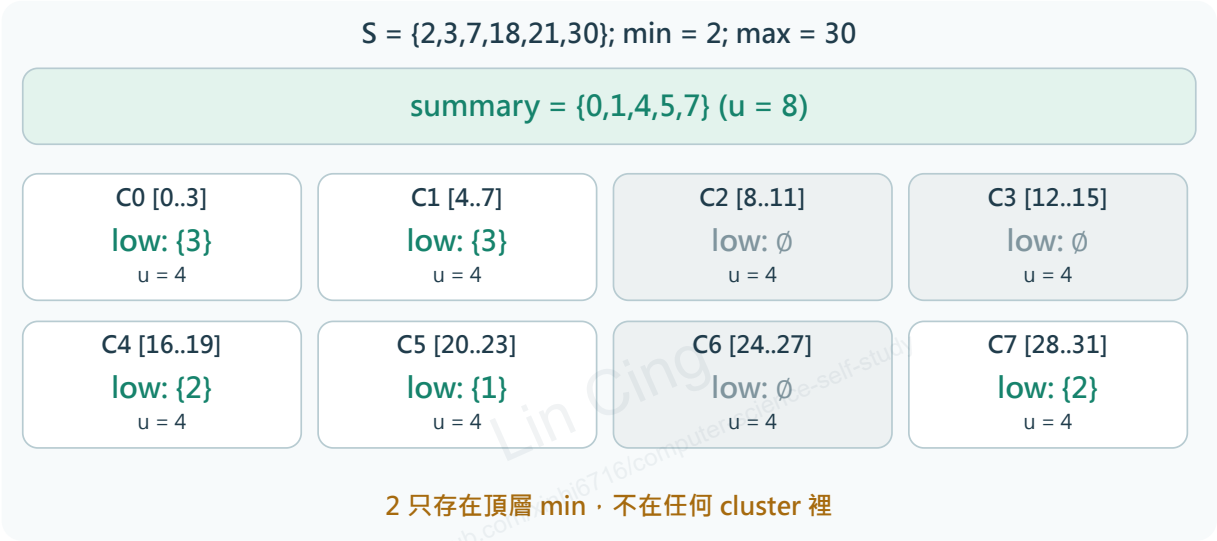
## 表示不變量：最小值不放進任何 cluster

每個節點保存 u、min、max；u>2 時另有 summary 與 cluster[0..H−1]。採用預先建立的完整結構，空節點的 min=max=NIL。

非空時，min 是該集合最小值；所有其他鍵依 high／low 分到 cluster。**min 本身不存入 cluster**，所以單一元素時所有 cluster 與 summary 都空。當至少兩鍵時，max 也存在某個 cluster。

summary 恰存非空 cluster 的編號：$i \in \text{summary} \iff C_i.\min \neq \text{NIL}$。因此 min=max 可在 O(1) 判定單一元素；向空節點插入只需設定 min=max=x，不必往下填滿一條路。

## u=32 的集合：原鍵、組內鍵與摘要分開看




原集合為 {2,3,7,18,21,30}。頂層 min=2、max=30；$C_0$ 只存 low=3，不再存 2。summary={0,1,4,5,7}。圖只展開頂層；每個 $C_i$ 與 summary 仍各自遵守相同不變量。



## 查找與邊界：先保證參數在 universe 內

```
MEMBER(V,x):                  # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high／low 皆使用 V 的尺寸。查找沿一個 cluster 向下；快取命中時提早返回。NIL 必須是獨立狀態，不能用合法鍵 0 代替。

本節後繼定義為集合中**嚴格大於 x** 的最小鍵，前驅為嚴格小於 x 的最大鍵；x 不必在集合中，但須在 universe 內。越界參數由公開介面拒絕，避免索引負值或 H。

## 後繼：用 max 決定遞迴走哪一邊

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

若同組 max 比 j 大，後繼就在同組；否則同組沒有答案，直接問 summary 的下一個非空組。**不用先遞迴失敗一次再換組**。找到下一組後讀 min 是 O(1)。



## 後繼數例：同組與跨組共用一份狀態

同組先看快取 max；跨組只問 summary。兩條路徑互斥。

## 逐步核對後繼查詢的座標

以 {2,3,7,18,21,30} 為準。SUCCESSOR(3)：i=0、j=3，$C_0$.max=3，不可留在同組；summary 的後繼組號是 1，讀 $C_1$.min=3，還原 1×4+3=7。

SUCCESSOR(20)：i=5、j=0，$C_5$.max=1>0，因此只遞迴 $C_5$，得 offset=1，還原為 21。20 不在集合中仍可查詢。SUCCESSOR(30)=NIL，因為沒有嚴格更大的鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 前驅：不能忘記獨立保存的 min

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後的 k=NIL 分支不是無條件回傳 NIL：前面已知 x>V.min，且更小的非空 cluster 不存在，所以答案正是未放入 cluster 的 min。例如 PREDECESSOR(3)=2；即使 $C_0$ 裡沒有更小的 low，也不能漏掉頂層的 2。

## 插入：換出舊 min，並只更新必要的摘要

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

新鍵若小於 min，先交換，接著插入的是**舊 min**。空 cluster 的組號尚未在 summary，故遞迴插入組號，再以 O(1) 建立單一元素 cluster；非空 cluster 的組號已存在，只遞迴該 cluster。兩條昂貴路徑互斥。



## 插入 1：新 min 在外，舊 min 回到 $C_0$

由原集合 {2,3,7,18,21,30} 插入 1：

| 步驟 | 頂層／局部狀態 | 要維護的不變量 |
|---|---|---|
| 交換 | min=1；待插入 x=2 | 新 min 不放 cluster |
| 分組 | high(2)=0、low(2)=2 | 用頂層 L=4 |
| 進 $C_0$ | 原 $C_0$={3}，插入後 {2,3} | $C_0$ 自己的 min=2、max=3 |
| 摘要 | 仍為 {0,1,4,5,7} | $C_0$ 原本就非空 |
| 結束 | max=30 | 原最大值未改 |

$C_0$ 的 u=4，所以它的下一層改用 L=2：舊局部 min=3 被放到其 $C_1$ 的 low=1。不可把頂層 low=2 再當成「整棵樹的鍵 2」而混用座標。

## 刪除：先處理單一元素與 u=2

DELETE-PRESENT 的前提是 x 確實存在。先處理以下兩種狀態，再進入下一頁的一般情況：

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

第一個分支中 min=max 必為有效鍵，因為呼叫契約排除了空集合；刪除後所有 cluster 原本就空。u=2 且不是單一元素時，集合必為 {0,1}，刪除 x 後只剩 1−x。

若直接對空集合或不存在的鍵呼叫，這段程式可能清掉合法資料。公開介面先 MEMBER，未找到就回傳 NOT_FOUND；不能把這項前提藏在解答裡。



## 一般刪除：提升下一個 min，再清掉其 cluster 副本

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

## 刪除時的 x，為什麼會改成新 min？

變數 x 在提升時改成新 min 的值；接下來刪的是它在 cluster 中的那份表示。先更新 cluster 與 summary，再重算 max，不能讀取已經失效的舊最大組號。

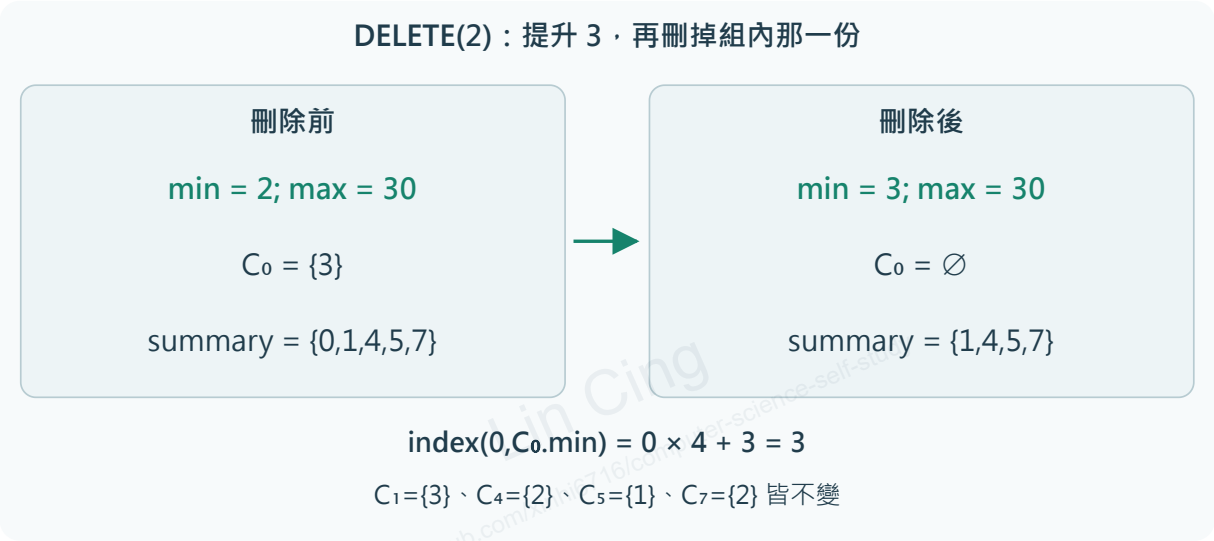
若集合只有 {2,3} 且刪除 2：提升後 x=3、min=3；$C_0$ 的 3 被刪空，summary 也清空。此時 x 等於舊 max=3，所以設定 max=min=3，留下合法的單一元素集合。

若還有 7、18、21、30，提升後的 x=3 不等於 max=30，最大值便不變。這裡比較的是程式目前的 x，不能在某一行偷換回原刪除參數 2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪除 2：提升 3，不是把所有鍵往前搬




3 提升成頂層 min，$C_0$ 清空，summary 刪掉組號 0。

## 逐步核對提升與摘要刪除

從原集合重新開始：summary.min=0、$C_0$.min=3，故新 min=0×4+3=3。刪除 $C_0$ 裡的 3 後 $C_0$ 變空，再從 summary 刪掉組號 0，得到 {1,4,5,7}。max=30 不變。

新 min=3 **只在頂層保存**；若還留在 $C_0$，就破壞表示不變量，也會讓後續刪除／單一元素判斷錯誤。此例中 cluster 的刪除是單一元素分支，只花 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 刪除最大值與最後兩個元素

仍由原集合出發，連續刪除 30、21、18、7、3、2：

| 刪除鍵 | 刪除後 min／max | 頂層 summary |
|---|---|---|
| 30 | 2／21 | {0,1,4,5} |
| 21 | 2／18 | {0,1,4} |
| 18 | 2／7 | {0,1} |
| 7 | 2／3 | {0} |
| 3 | 2／2 | 空 |
| 2 | NIL／NIL | 空 |

刪除 3 後沒有非空 cluster，max 必須回到獨立的 min=2；不能從空 summary 取得組號再索引陣列。刪除最後的 2 只清 min/max，已經沒有其他表示需要清理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 刪除有兩次呼叫，為何仍只剩一條昂貴路徑？

一般刪除確實可能先呼叫 DELETE(cluster)，再呼叫 DELETE(summary)。但第二次只有在 cluster 被刪空時才發生，而刪空表示該 cluster **原本只有一個元素**，第一次呼叫立即走 O(1) 的單一元素分支。

| cluster 原本狀態 | cluster 刪除成本 | 是否刪 summary 組號 |
|---|---|---|
| 單一元素 | O(1) | 是，可能遞迴 |
| 至少兩個元素 | 可能遞迴 | 否，刪除後仍非空 |

因此成本是 $\max\{T(L), T(H)\} + O(1)$，不是 $T(L) + T(H) + O(1)$。這是由表示不變量與分支條件推出的**最壞界**，不需要用機率或攤銷分析掩蓋第二次呼叫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 用位元寬度解遞迴式

設 T(u) 為任一操作的最壞上界，$R(m) = T(2^m)$。每層只有一次非定值遞迴，子節點位元數最多為 $\lceil m/2 \rceil$：

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

對 m≥2，$\lceil m/2 \rceil \le 2m/3$；所以至多 $\lceil \log_{3/2} m \rceil$ 個遞迴層後抵達基例。每層 O(1)，得到 $O(1 + \log m) = O(1 + \log \log u)$。

m=5 的最長位元路徑為 5→3→2→1；cluster 路徑也可能是 5→2→1。**取決於走 summary 或 cluster**，不能把兩種 universe 大小都寫成精確的 $\sqrt{u}$。u=2 時成本是常數，而不是 log log 2=0 次工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 正確性：每個操作如何維持集合表示

以 m 作歸納。m=1 可列出空、{0}、{1}、{0,1} 四種合法集合。較大節點的集合由獨立 min 與各 cluster 還原值的聯集唯一決定。

**查詢**：min/max 守住邊界；同組存在答案時，其餘更高組不可能更小。否則 summary 找到下一個非空組，再讀該組 min。前驅另外納入未放進組內的 min。

**插入**：新 min 與舊 min 交換，只改變保存位置，不遺失鍵；將待插入鍵加入正確 cluster，且只在該組首次非空時加入摘要。

**刪除**：刪 min 時提升目前最小的 cluster 鍵，再刪其組內副本；其餘情況只刪目標鍵。空組同步退出摘要，max 由更新後狀態重建。歸納假設保證子操作正確，因此每步恰好實現目標集合變化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 建立空結構也有成本：Θ(u) 空間與初始化

本節採預先配置的完整樹，不把配置藏進 O(1) 的空節點插入。u>2 時，H 個 cluster 各有 universe L，另有 universe H 的 summary；本層指標與欄位成本至多 cH：

$$P(u) \le HP(L) + P(H) + cH, \qquad P(2) \le p_0.$$

可用強歸納證明 $P(u) \le A(u-1) - B$。選 B≥c、A≥B+p₀，則子項代入後：

$$P(u) \le A[H(L-1) + (H-1)] - B(H+1) + cH \le A(u-1) - B.$$

因此 P(u)=O(u)。只計 cluster 分支的底層節點已需 Ω(u) 空間，故為 Θ(u)；逐節點建立與清空也需 Θ(u) 時間。此推導同時容許 H≠L。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 公開介面、重複鍵與空間取捨

| 情境 | 本節的完整規格 |
| --- | --- |
| 重複 INSERT(x) | 先 MEMBER；存在就回傳 ALREADY_PRESENT |
| 不存在的 DELETE(x) | 先 MEMBER；不存在就回傳 NOT_FOUND |
| x<0 或 x≥u | 在任何索引前拒絕 |
| 需要重複鍵 | 另維護 multiplicity；計數由 0↔1 時才改鍵集合 |
| 需要附帶資料 | 另定義 key→record 對應與所有權 |

多一次 MEMBER 仍是兩個 O(1+log log u) 的相加，不改漸近界。重複鍵的計數假設可在常數時間讀寫、且不溢位；不能直接把重複鍵丟入 INSERT-ABSENT。

完整樹適合可負擔 u 的情境。若改成按需配置與 hash table，需要重新分析空間、配置與期望時間；**不能直接沿用本節的確定性最壞界**。n 很小而 u 很大時，O(n) 空間的平衡樹通常更合理。



## 把攤銷、期望與最壞保證放回各自的前提

| 結構／操作 | 保證 | 主要前提 |
| --- | --- | --- |
| 紅黑樹查詢／更新 | 最壞 O(log(n+2)) | 比較鍵；O(n) 空間 |
| 雜湊查找 | 期望 O(1+α) | 宣告的雜湊機率模型；不支援排序鄰居 |
| Fibonacci 遞減鍵值 | 攤銷 O(1) | 已知 handle；同一序列勢能 |
| 完整 vEB 前驅／更新 | 最壞 O(1+log log u) | 整數 word RAM；Θ(u) 空間／建立 |

若先建立完整 vEB 再做 q 次操作，總上界為 $O(u + q(1 + \log\log u))$；初始化不能被省略。一般鍵的比較模型、整數 universe 模型與隨機模型各有適用條件，沒有一張脫離模型的「最快資料結構」排名。

## 參考資料與署名

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing，個人手寫資料結構筆記：佇列、搜尋樹、紅黑樹、多路搜尋樹與堆積。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 不相交集合：只合併，不拆分

同一組元素分成哪些集合，會隨合併改變；**查詢要回答的是「兩個元素現在是否同組」**。

| 本章要解決的問題 | 對應方法 |
| --- | --- |
| 如何代表一個集合？ | 串列的集合紀錄，或森林的根 |
| 合併時要改多少資料？ | 短串列接長串列；低 rank 根接高 rank 根 |
| 查找為何愈做愈便宜？ | 路徑壓縮，加上整段操作的攤銷分析 |

先備知識：指標、根樹、漸近記號，以及位能法。rank、level、iter 都會在本章定義。



## 三個操作，維護同一份集合分割

集合分割（partition）要求：每個已建立元素恰屬一組；各組非空且兩兩不交。

| 操作 | 輸入前提 | 結果 |
| --- | --- | --- |
| MAKE-SET(x) | x 尚未建立 | 建立單點集合 {x} |
| FIND(x) | x 已存在 | 回傳 x 所在集合的代表 |
| UNION(x,y) | x、y 已存在 | 合併兩者所在集合；同組時不變 |

**代表是識別集合的代號，不保證是最小鍵。** 同組元素的 FIND 必須回傳同一代表；不同組則回傳不同代表。

## 呼叫次數與成功合併，要分開計算

本章先建立 n 個單點。m 計入使用者呼叫的 MAKE-SET、FIND、UNION，所以 m≥n；UNION 內部呼叫的 FIND 不再重複加進 m。

| 名稱 | 規格與計數 |
| --- | --- |
| 成功合併 | 集合數減一，因此最多 n−1 次 |
| UNION 呼叫 | 可以重複，次數不受 n−1 限制 |
| LINK(r,s) | 只接受兩個**不同的根**，不接受任意元素 |

> UNION 先 FIND 兩端；代表相同就返回。**不能再 LINK 同一根，也不能因此增加 rank。**

FIND 可以改 parent，但不改集合分割。刪邊、拆集合與撤銷合併，需要另外的結構與成本分析。



## 無向連通：新邊會合併哪些組？

先為每個頂點 MAKE-SET，再逐邊 UNION。圖解使用這一串輸入：ab、cd、bc、ab、ef、de。

### 邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

| | | | 集合數 |
| --- | --- | --- | --- |
| ab | ⇒ | {a,b} {c} {d} {e} {f} | 5 |
| cd | ⇒ | {a,b} {c,d} {e} {f} | 4 |
| bc | ⇒ | {a,b,c,d} {e} {f} | 3 |
| ab | ⇒ | {a,b,c,d} {e} {f} | 3 |
| ef | ⇒ | {a,b,c,d} {e,f} | 2 |
| de | ⇒ | {a,b,c,d,e,f} | 1 |

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

**第四條 ab 是重複邊：呼叫照算，集合不再改變。** 六次 UNION 中只有五次成功合併。

## 為什麼這個方法恰好得到連通分量？

不變量：處理過的邊所形成的子圖，其連通分量恰好等於目前的集合分割。

| 步驟 | 圖的連通性 | 集合分割 |
| --- | --- | --- |
| 還沒有邊 | 每個頂點獨立 | 每點各成一組 |
| 新邊兩端已同組 | 原本就連通 | UNION 不改分割 |
| 新邊兩端不同組 | 這條邊接起兩個分量 | UNION 恰好合併這兩組 |

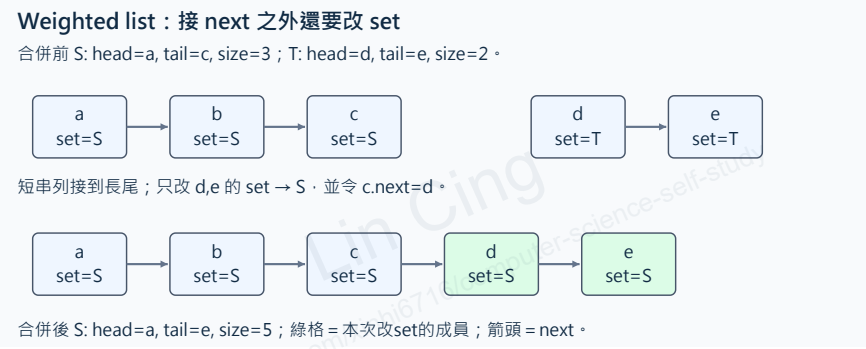
由歸納可知，最後 FIND(u)=FIND(v) 若且唯若存在無向 u–v 路徑。

靜態圖可用 DFS/BFS 在 Θ(n+|E|) 內找分量；DSU 也能處理**逐步加邊的連通查詢**。有向圖忽略方向後只得到弱連通分量，不能用來求 SCC。



## 串列表示：接上尾端，還沒完成合併

每組有一份集合紀錄（set record）：head、tail、size。每個成員另存 next 與 set 指標；FIND(x) 直接回傳 x.set.head，因此是 O(1)。




> a→b→c 與 d→e 合併後，tail 由 c 改為 e、size 由 3 改為 5。**d、e 的 set 也要改指新的紀錄**，FIND 才會回傳 a。

## 加權串列的 UNION，逐項做完

令 S=x.set、T=y.set。同組直接返回；不同組時，讓 S 是較長的串列，同長保留第一組。

| 次序 | 修改 | 目的 |
|---|---|---|
| 1 | S.tail.next=T.head | 把兩串列接起來 |
| 2 | 保存 T 的原 head；更新 S.tail、S.size | 保留走訪起點，更新集合紀錄 |
| 3 | 沿 T 的 next，把每個成員的 set 改為 S | 讓短串列的 FIND 改回傳 S.head |
| 4 | 完成上述改寫後才廢棄 T 紀錄 | 不留下指向失效紀錄的成員 |

走訪時不改 T 內部的 next，因而不會遺失後繼。成本為 $\Theta(1+\min(|S|,|T|))$，**有 tail 只讓接線是 O(1)，不是整個 UNION。**



## 不加權，為什麼會退化成平方成本？

若固定把第二組接到第一組，依序做 UNION($x_2$,$x_1$)、UNION($x_3$,$x_2$)、…，每次新單點都要吸收舊長串列。

| 合併 | 必須重寫 set 的成員數 |
|---|---|
| UNION($x_2$,$x_1$) | 1 |
| UNION($x_3$,$x_2$) | 2 |
| … | … |
| UNION($x_n$,$x_{n-1}$) | n−1 |

總改寫數為 $n(n-1)/2$。外部只有 n 次建立與 n−1 次合併，即 $m=2n-1$，實際成本卻是 $\Theta(n^2)$。

## 每次移短組，同一成員最多搬幾次？

固定一個成員 x。每次它的 set 被改寫，表示它在較短的集合；合併後所在組的大小**至少加倍**。

改 k 次後至少有 $2^k$ 個成員，而總數不超過 n，所以 $k \le \lfloor \log_2 n \rfloor$。加總所有成員的改寫，再加上 m 次呼叫的常數開銷，得到 **$O(m+n \log n)$**；涵蓋小 n 可寫 $O(m+n \log(n+1))$。

| 要分清楚的量 | 結論 |
| --- | --- |
| 整段操作總成本 | $O(m+n \log n)$ |
| 單次合併兩個各 n/2 的集合 | 仍需 $\Theta(n)$ |
| $n=2^k$，同大小集合分輪合併 | 每輪改 n/2 筆，共 $(n/2)\log_2 n$ 筆 |

> 攤銷總界不能直接當成單次最壞界。短組策略也不保證所有序列都只有線性成本。



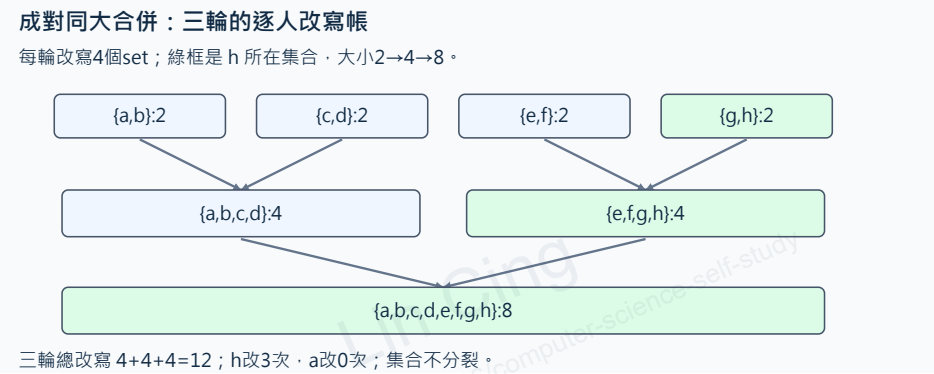
## 八元素：每次合併到底改了誰？

同長保留第一組。這一組操作稍後也會用來比較森林表示。

| UNION | 原大小 | 改 set 的成員 | 新代表／大小 | 累計改寫 |
| --- | --- | --- | --- | --- |
| a,b | 1,1 | b | a／2 | 1 |
| c,d | 1,1 | d | c／2 | 2 |
| e,f | 1,1 | f | e／2 | 3 |
| g,h | 1,1 | h | g／2 | 4 |
| a,c | 2,2 | c,d | a／4 | 6 |
| e,g | 2,2 | g,h | e／4 | 8 |
| a,e | 4,4 | e,f,g,h | a／8 | 12 |

h 的代表依 h→g→e→a，所屬組大小由 1→2→4→8；a 一次也不用搬。**合計 12 次，不代表每人都搬三次。**

## 用同一張圖，對照逐輪成本與個別成員




每輪的總改寫數與「固定一人搬幾次」是不同分母。表格逐輪加總為 12；h 個人則恰好改寫三次。

若反過來每次只把新單點接到大集合，總改寫可以只有 n−1 次；這不影響上界，也不代表所有序列皆線性。



## 森林表示：先找根，再接線

每個節點存 parent、rank。MAKE-SET(x) 設 parent[x]=x、rank[x]=0；根的 parent 指向自己。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                    // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

LINK 只接受兩個不同的根，所以不會形成環，也不會拆開舊集合。不同 rank 不加一；相同 rank 只讓保留根加一。

**串列比較 size；森林比較 rank，後者不是精確的成員數。**

## 兩趟 FIND：先找到根，再沿原路改接

從 x 出發，沿 parent 往上走，遇到自指的根就停。第一趟只找根 r，不改指標；第二趟再把**原查找路徑**上的每個節點直接接到 r。

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

old 保存的是「覆寫前要走的下一站」。若改用 x=parent[x]，讀到的已是剛寫入的根，便跳過中間節點。遞迴版 parent[x]=FIND(parent[x]) 則在呼叫回傳時逐點改接。

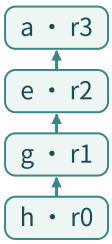
兩趟迭代額外空間為 O(1)；遞迴堆疊與原路徑高度同階。走 s 點需 Θ(s)，只改本次路徑，不改 rank。



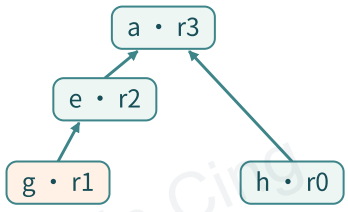
## 找根正確，不代表整條路徑都已壓縮

箭頭由**子指向父**；只畫 h→g→e→a，根 a 自指，其他節點不變。

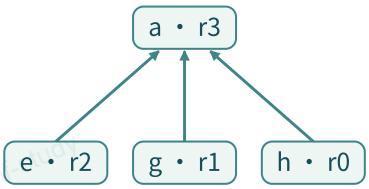
原路徑




覆寫後直接跳根




保存舊父，完成壓縮




| 第二趟目前節點 | 先存 old | 再改 parent | 繼續到 |
|---|---|---|---|
| h | g | a | g |
| g | e | a | e |
| e | a | a（不變） | a，然後停止 |

## 同一八元素的 rank 與壓縮前後

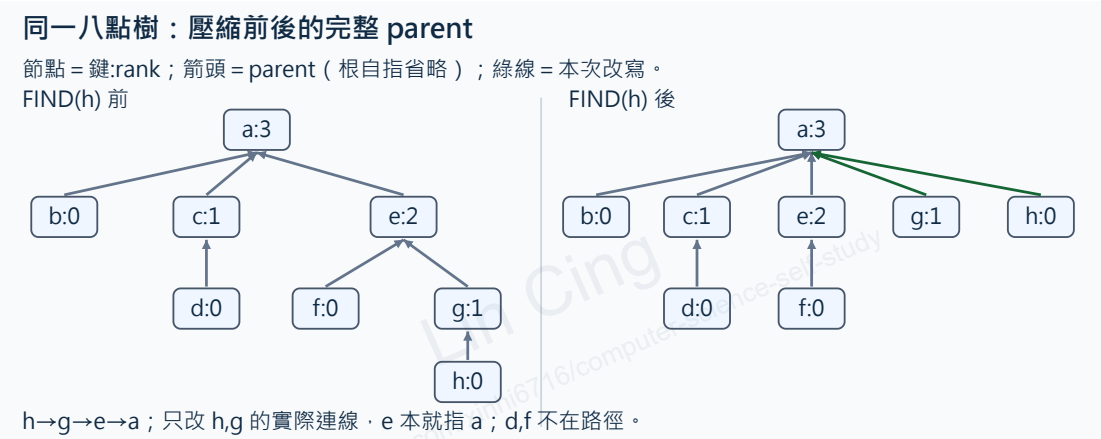
沿用前面七次 UNION；同 rank 時保留第一個根。先記錄每個節點的 rank 與 parent，再看 FIND(h) 改了哪些欄位。

| 點 | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| rank | 3 | 0 | 1 | 0 | 2 | 0 | 1 | 0 |
| FIND(h)前parent | a | a | a | c | a | e | e | g |
| FIND(h)後parent | a | a | a | c | a | e | a | a |

- 第一趟走 h→g→e→a，共三條父邊。第二趟把沿途的 parent 設為 a。
- 真正改變的是 **h、g 的 parent**；e 原本就接 a，其他節點不在路徑上。
- **rank 全部不變**。它記錄合併歷史，不能在壓縮後改填目前高度。



## 看圖核對：改接路徑，保留其餘分支




> 再查 h 只走一條邊；查 d 仍走 d→c→a。**一次 FIND 只壓縮自己的查找路徑。**

## Rank 為什麼能限制樹高？

rank 不是「目前高度」，但能當作它的**上界**。對操作次數歸納：

| 操作 | 樹高如何改變 | 為何仍有 height ≤ rank |
|---|---|---|
| 建立單點 | 高度為 0 | 初始 rank 也是 0 |
| 不同 rank 合併 | 低 rank 的根接到高 rank 的根 | 低 rank 至少差 1，多接一條邊仍不超過大 rank |
| 相同 rank 合併 | 高度最多增加 1 | 保留根的 rank 同時加 1 |
| 路徑壓縮 | 改接到祖先，只會縮短路徑 | rank 不變，上界仍成立 |

> **壓縮後不重算 rank。** 例如根的 rank=2，壓縮後高度可以是 1；兩者不必相等。



## Rank 也記得合併時累積的成員

根的 rank 每升一級，都必須合併兩個同 rank 的不同集合：

- rank=0：至少 1 個成員。
- 兩個 rank=r−1 的根合併：各至少有 $2^{r-1}$ 個成員，新根至少有 $2^r$ 個。
- 不同 rank 合併只增加成員；FIND 不改變集合成員。

非根的 rank 固定在它失去根身分之時。**當時的集合至少有 $2^r$ 個成員**，所以仍有 $2^r \le n$，即 $r \le \lfloor \log_2 n \rfloor$。這不是目前子樹大小的下界。

> 只按 rank 合併，單次 FIND 為 $O(\log n)$；再加路徑壓縮，改進的是**整段操作的攤銷界**。第一個深葉查找仍可能走 $\Theta(\log n)$ 條邊。

## 21.4 Rank 與路徑壓縮的精細分析

### Ackermann 的迭代是函數合成

固定整段序列最終的元素數 $n$，令 $\alpha = \alpha(n)$。分析時不隨目前已建立的元素數重算 $\alpha$。對 $j \geq 1$，定義

$$f^{(0)}(j) = j, \qquad f^{(i+1)}(j) = f(f^{(i)}(j)), \qquad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j)\ (k \geq 1).$$

上標表示**反覆把輸出當成下一次輸入**。由 $A_0^{(i)}(j) = j + i$ 得 $A_1(j) = 2j + 1$；對 $i$ 歸納可得 $A_1^{(i)}(j) = 2^i(j + 1) - 1$，所以

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

各 $A_k$ 對輸入嚴格遞增，且 $A_k(j) \geq j + 1$：由 $A_0$ 開始歸納，較大的輸入會經過更多次、且每次都增加的迭代。又 $j + 1 \geq 2$，故 $A_k(j) > A_{k-1}(j)$。後面的分層界會用到這兩種單調性。



## 本章採用的 inverse-Ackermann 定義

採 Cormen 第三版 §21.4 的單參數版本：

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

| k | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $A_k(1)$ | 2 | 3 | 7 | 2047 | 大於 $10^{80}$ |

所以 $\alpha(2) = 0$、$\alpha(8) = 3$、$\alpha(2048) = 4$。不同教材可能採雙參數定義；不能直接混用其數值。

**完整總界是** $O(m(1 + \alpha(n)))$，其中 $m$ 次外部操作包含 $n$ 次 MAKE-SET。只有 $n \geq 3$ 時才能簡寫為 $O(m\alpha(n))$；$n \leq 2$ 仍有常數操作成本。

## 哪些量會隨操作改變？

對操作數歸納可證：根以 parent 指向自己；非根的 rank 嚴格小於父 rank。相同 rank 的兩根合併時，保留根的 rank 加一；不同 rank 合併時，把小 rank 根接到大 rank 根。壓縮只改 parent，不改 rank。

**節點一旦成為非根，其 rank 就固定。** 後續它所指父節點的 rank 可能增加，或因壓縮改指更高的祖先；兩者都不會降低父 rank。這才是 level 不下降的依據。

rank 為 $r$ 的根至少包含 $2^r$ 個元素。非根沿用失去根身分時的歷史下界，故任一節點的 rank 至多 $\lfloor \log_2 n \rfloor < n$。這不是壓縮後的目前子樹大小。

接下來只對 rank 正、且不是根的節點定義 level 與 iter；rank 0 和根另外計費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## level 決定層級，iter 記錄層內進度

令 $r = \mathrm{rank}(x) \geq 1$、$p = \mathrm{rank}(\mathrm{parent}(x))$，定義

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

正文以 $\ell = \mathrm{level}$、$i = \mathrm{iter}$ 縮短公式。先找父 rank 能跨過的最高層，再算該層可迭代幾次。

因 $p \geq r + 1 = A_0(r)$，有 $\ell \geq 0$。又 $A_\alpha(r) \geq A_\alpha(1) \geq n > p$，故 $\ell < \alpha$。定義給 $i \geq 1$；若 $i \geq r + 1$，就會有

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

與 $\ell$ 的最大性矛盾。因此 $0 \leq \ell < \alpha$、$1 \leq i \leq r$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用非負位能記錄尚可支付的工作

逐節點定義位能，再加總成整座森林的位能：

$$\phi(x)=\begin{cases}\alpha r, & x\text{ 是根或 }r=0,\\ (\alpha-\ell(x))r-i(x), & x\text{ 非根且 }r\ge 1,\end{cases}\qquad \Phi=\sum_{x}\phi(x).$$

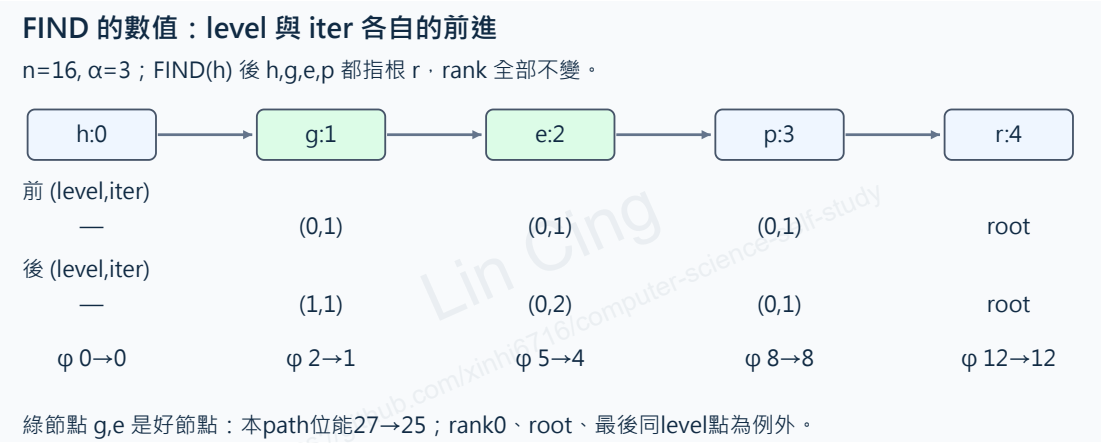
非根的下界為 $(\alpha-(\alpha-1))r-r=0$，上界為 $\alpha r-1$。根的位能是 $\alpha r$，rank 0 的位能為 0；所以 $\Phi$ 永遠非負。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 五點路徑的位能帳

取 $n=16$、$\alpha=3$，壓縮 rank 依序為 0、1、2、3、4 的路徑。圖中的根保持自指。




兩個好節點各釋放 1：g 靠 level 上升，e 靠同層 iter 上升。其餘三點位能不變，所以整條路徑位能由 27 降至 25。

## level 上升時，iter 為何可以下降？

固定非根的 rank $r$。同一層內，父 rank 增加只會讓 iter 不變或增加；每多一次迭代，$\phi$ 就減一。

若層級由 $k$ 升到 $k' > k$，iter 可能由 $i$ 重設為較小的 $i'$。然而

$$\Delta\phi = -(k' - k)r + (i - i') \le -r + (r - 1) = -1.$$

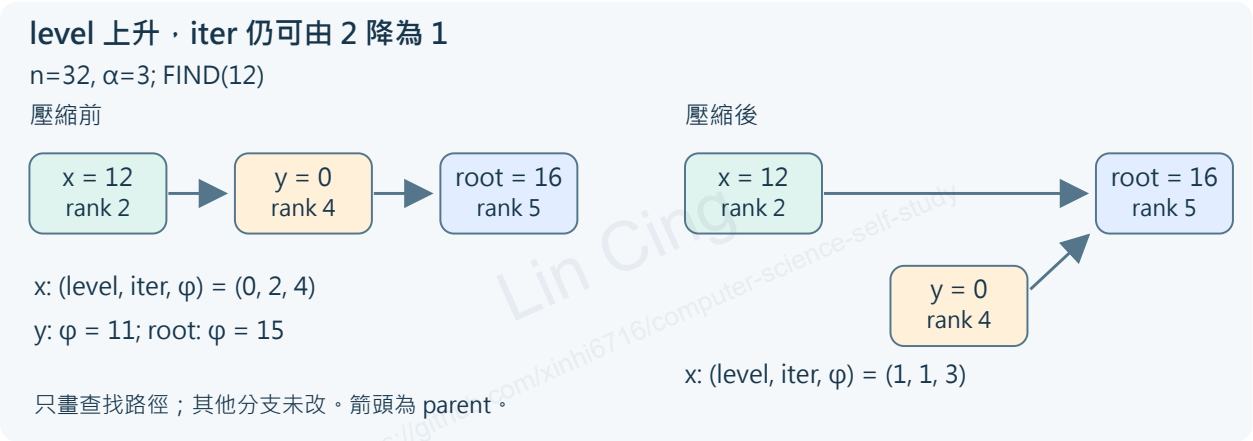
不等式用了 $k' - k \ge 1$、$i \le r$、$i' \ge 1$。**跨層的位能下降，足以支付 iter 重設。** 因此需要單調的是「先比較 level，再比較 iter」的字典序（lexicographic order），不是兩個座標各自遞增。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一個 iter 確實下降的合法 FIND

取 $n = 32$、$\alpha = 3$。查找路徑是節點 $12 \to 0 \to 16$，rank 依序為 $2, 4, 5$。圖只保留這次路徑，其他分支不變。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 跨層數例的逐項計算

對 $x = 12$，固定 rank 為 2；壓縮只把父 rank 由 4 換成 5。

| 量 | 壓縮前 | 壓縮後 |
|---|---|---|
| level | $A_0(2) = 3 \le 4 < A_1(2) = 5$，故為 0 | $A_1(2) = 5 \le 5 < A_2(2) = 23$，故為 1 |
| iter | $A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$，故為 2 | $A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$，故為 1 |
| $\phi(x)$ | $(3-0)2 - 2 = 4$ | $(3-1)2 - 1 = 3$ |

節點 0 的位能保持 11，根的位能保持 15；其餘分支都不變。因此全森林 $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$。這裡只列路徑的加總；並未假設其他 29 個節點的位能為零。



## 如何真的建立這條路徑？

下列只用 MAKE-SET、UNION、FIND；同 rank 保留第一個根。先建立鍵 0 到 31 的單點。

| 操作 | 可核對的結果 |
|---|---|
| 對 0..15 分輪合併相鄰、同大小區塊 | 根 0 的 rank 為 4；節點 12 的 rank 為 2 |
| FIND(12) | 把 12 的 parent 改為 0；不改 rank |
| 同法合併 16..31 | 根 16 的 rank 為 4 |
| UNION(16,0) | 根 16 升為 rank 5；0 的 parent 為 16 |
| 再次 FIND(12) | 沿 $12 \to 0 \to 16$；12 的 parent 改為 16 |

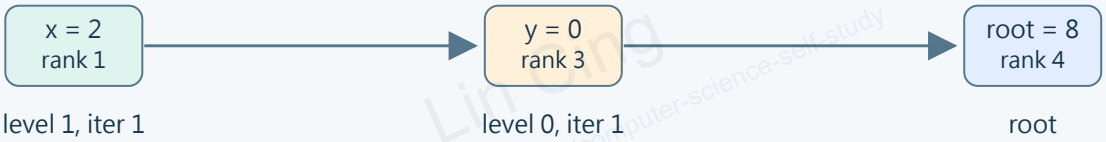
例如前半部的輪次是先合併 (0,1)、(2,3)、…、(14,15)，再合併 (0,2)、(4,6)、…，接著 (0,4)、(8,12)，最後 (0,8)。這說明 rank 數值來自合法歷史，沒有任意指定節點的 rank。

## 沿路的 level 不必遞增

取 $n = 16$。先把 0..7 合成根 0、rank 3 的集合並執行 FIND(2)；再把 8..15 合成根 8、rank 3 的集合，執行 UNION(8,0)。得到以下合法路徑。

**rank 遞增，不代表 level 也遞增**

n=16, α=3; FIND(2)




$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

沿路 rank: 1 < 3 < 4；level: 1 > 0。

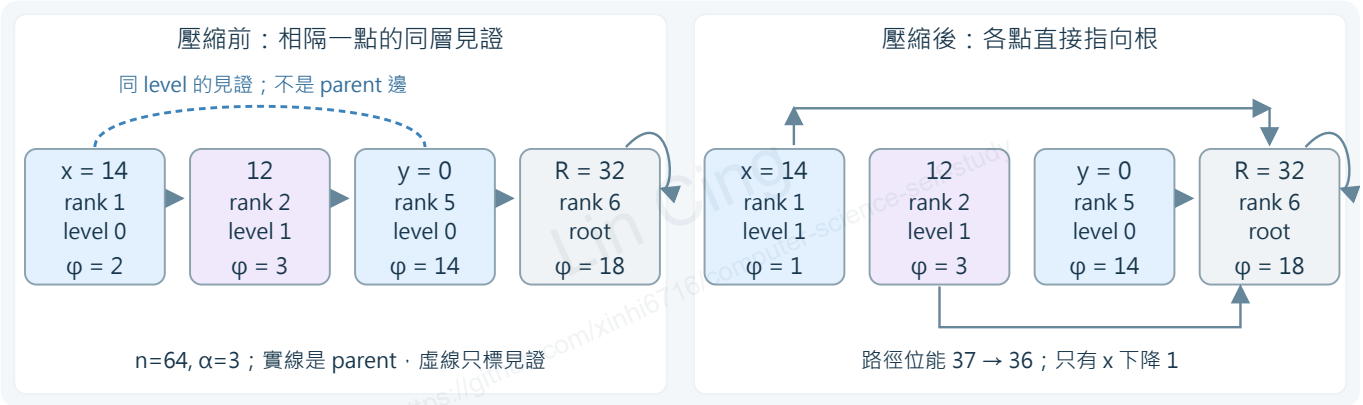
同 level 的最後一點才是例外；不用假設 level 沿路排序。

不同節點的 rank 也變了；不能把沿路比較，混同「同一節點隨時間」的單調性。



## 好節點的同層見證可以隔著其他 level

在壓縮前的路徑上，rank 大於 0 的非根節點 $x$ 的後方若有另一個非根 $y$，且 $\ell(x) = \ell(y)$，就稱 $x$ 為好節點。**找同層的後繼，不是只比較下一個節點。**




路徑 $14 \to 12 \to 0 \to 32$ 的 rank 為 1、2、5、6，level 為 0、1、0（根不定義 level）。14 的見證是 0；中間的 12 不必同層。兩個同層點不必在原樹中相鄰，壓縮後仍能替 14 帶來更高的父 rank。

## 先用合法合併建立這條路徑

取 64 個單點，同 rank 保留 UNION 的第一個根。「分輪建立區間」表示先合併相鄰單點，再合併相鄰同大小區塊。

| 操作 | 能從操作歷史核對的事實 |
|---|---|
| 分輪建立 0..31 | 根 0 的 rank=5；節點 12 的 rank=2，14 的 rank=1 |
| FIND(12) | 原路徑 $12 \to 8 \to 0$；只把 12 改接 0，14 仍接 12 |
| 分輪建立 32..63 | 根 32 的 rank=5 |
| UNION(32,0) | 兩根同 rank，32 升到 6；0 改接 32 |
| 尚未 FIND(14) | 路徑為 $14 \to 12 \to 0 \to 32$，rank 為 1、2、5、6 |

FIND(12) 沿 $12 \to 8 \to 0$ 往上找到根，再把這條路上的父指標直接接到根。它不走進子樹，所以 14 仍接 12。之後的合併才把根 0 接到根 32；這些 rank 都來自合法操作，根 32 保持自指。



## 把每個節點的 level、iter、位能算出來

固定 $n = 64$、$\alpha = 3$。只對rank 大於 0 的非根節點計算 level、iter；根用 $\phi = \alpha r$。

| 節點 | rank | 父 rank：前→後 | level：前→後 | iter：前→後 | $\phi$：前→後 |
|---|---|---|---|---|---|
| 14 | 1 | 2→6 | 0→1 | 1→1 | 2→1 |
| 12 | 2 | 5→6 | 1→1 | 1→1 | 3→3 |
| 0 | 5 | 6→6 | 0→0 | 1→1 | 14→14 |
| 32（根） | 6 | 6→6 | 不適用 | 不適用 | 18→18 |

對 14，壓縮前 $A_0(1) = 2 \le 2 < A_1(1) = 3$，故 level=0、iter=1。壓縮後 $A_1(1) = 3 \le 6 < A_2(1) = 7$，且 $A_1^{(2)}(1) = 7 > 6$，故 level=1、iter=1。位能由 $(3-0)1 - 1 = 2$ 降至 $(3-1)1 - 1 = 1$。

對 12，兩次皆有 $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$，故 level=1、iter=1。對 0，$A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$，故 level=0、iter=1、位能 14。路徑總和 $37 \to 36$；其餘點的 rank、父 rank 與根身分不變，故**全森林位能也恰減 1**。

## 好節點證明：每一步不等式各用了什麼？

所有量先固定在壓縮前。記 $\rho(v) = \text{rank}(v)$、$p_x = \text{parent}(x)$、$p_y = \text{parent}(y)$，根為 $R$。令 $r = \rho(x)$、$k = \ell(x) = \ell(y)$、$i = i(x)$。

| 逐步推導 | 理由 |
|---|---|
| $\rho(R) \geq \rho(p_y)$ | $p_y$ 在通往根的路上，rank 沿非根 parent 邊嚴格增加 |
| $\rho(p_y) \geq A_k(\rho(y))$ | $\ell(y) = k$ 的定義 |
| $A_k(\rho(y)) \geq A_k(\rho(p_x))$ | $y$ 就是 $p_x$ 或其祖先，故 $\rho(y) \geq \rho(p_x)$；$A_k$ 遞增 |
| $A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$ | iter 定義給 $\rho(p_x) \geq A_k^{(i)}(r)$，再用 $A_k$ 遞增 |
| $A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$ | 函數再合成一次，不是數值相乘 |

因此新父的 rank 至少為 $A_k^{(i+1)}(r)$。這裡沒有使用「路徑上的 level 遞增」；只使用 rank 的祖先次序與同一個 $k$ 的函數單調性。



## 跨過下一個門檻，為何一定能支付工作？

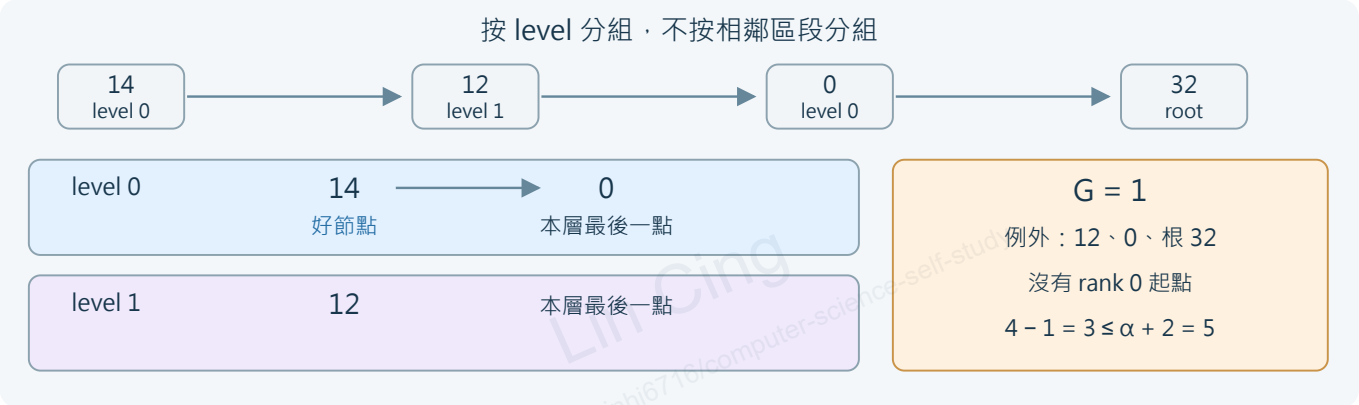
壓縮使 $x$ 的父變成 $R$，但 $r$ 不變。前頁證明的是新父 rank 跨過**舊 level 的下一次迭代門檻**，不保證新的 iter 必定比舊值大；必須分兩種情況。

| 壓縮後 | 位能的差 | 可以支付的原因 |
|---|---|---|
| level 仍為 $k$ | $\Delta\phi = -(i' - i) \leq -1$ | iter 至少變成 $i + 1$ |
| level 升為 $k' > k$ | $\Delta\phi = -(k' - k)r + i - i'$ | $k' - k \geq 1, i \leq r, i' \geq 1$，故 $\Delta\phi \leq -r + (r - 1) = -1$ |

在 64 點例子取 $x = 14, y = 0, k = 0, i = 1$，不等式數值為 $6 \geq 6 \geq 6 \geq 3 \geq 3$。舊層下一個門檻是 $A_0^{(2)}(1) = 3$；新父 rank=6 更跨過 $A_1(1) = 3$，所以落在第二種情況，位能下降 $1$。

這筆帳按「被壓縮的好節點 x」計費，不向見證 y 再收一次，也不要求 y 的位能下降。所有非好節點的位能亦不增加，故可以把各個好節點的下降量相加。

## 每個 level 只留下最後一點




沿原路徑向根看，同一 level 除了最後一點外，每點都有後方的同層見證。因此 14 可計費，12 與 0 分別是 level 1、0 的最後一點；再加根 32，共 3 個例外。若把三個相鄰區段 0／1／0 當成三組，就會漏算好節點 14。



## 從分組計數回到位能總下降

一般路徑含 $s$ 個節點。rank 大於 0 的非根節點的 level 只能是 $0, \dots, \alpha - 1$，故各層最後一點至多 $\alpha$ 個；另加根與可能的 rank 0 起點，例外至多 $\alpha + 2$。rank 沿路嚴格增加，rank 0 起點最多一個。

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

$G$ 是實際好節點數，$g$ 只是保守下界。64 點例子 $s = 4, \alpha = 3$，所以 $g = 0$，實際卻有 $G = 1$、$\Delta\Phi = -1$。短路徑的下界可以不緊；不等式並未要求實際下降量必須等於下界。後面會用這個一般下界支付長路徑的成本。

## 用固定常數把路徑工作付清

設兩趟查找的實際成本至多 $cs$，固定 $C > c$ 並用放大位能 $\Psi = C\Phi$。則攤銷成本

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

| 路徑情況 | 代入 $g$ 後的上界 |
| --- | --- |
| $s \le \alpha + 2$ | $cs \le c(\alpha + 2)$ |
| $s > \alpha + 2$ | $(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$ |

因此每次 FIND 的攤銷成本為 $O(1 + \alpha)$。例如 $s = 7$、$\alpha = 3$、$c = 2$、$C = 3$，下界 $g = 2$，故 $\hat{c} \le 14 - 6 = 8$。$C$ 在整段分析中固定，不能每次臨時換值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LINK 把位能增加限制在新根

MAKE-SET 新增 rank 0 節點，其位能為 0，攤銷成本仍為 $O(1)$。對 LINK，設舊根 $u$ 接到保留根 $v$：

| 可能改變的點 | 位能變化的原因 |
| --- | --- |
| $u$ | 從根變非根；rank 不變，位能不增 |
| $v$ 原有的孩子 | 父 rank 不降；前述單調性保證位能不增 |
| $v$ | 不同 rank 時不變；同 rank 時加一，位能只增 $\alpha$ |

其他點的 rank、父 rank 與根身分皆不變，所以位能不變。於是 $\Delta\Psi \le C\alpha$，LINK 的攤銷成本為 $O(1) + C\alpha = O(1 + \alpha)$。UNION 先做兩次 FIND，根不同才 LINK；最多拆成三次操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從每次攤銷界回到整段實際成本

令第 $j$ 次實際成本為 $c_j$，攤銷成本為 $a_j = c_j + \Psi_j - \Psi_{j-1}$。相鄰位能相消後，

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \le \sum_{j=1}^{m'} a_j.$$

不等號用了初始空森林 $\Psi_0 = 0$ 與終態位能非負。拆開 UNION 後 $m' \le 3m$，所以原本 $m$ 次操作的總成本為 $O(m(1+\alpha(n)))$。

這是整段序列的最壞總界。rank 與壓縮並用時，單次 FIND 仍可能走 $\Theta(\log n)$；先建立平衡的 rank 樹、再首次查找深葉即可達到。線性長鏈需要移除 rank 規則，不能當成本定理的單次反例。

位能分析參考：Cormen 等，第三版 §21.4，印刷頁 576–581。



## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

# 動態規劃

遞迴把問題拆開；動態規劃進一步辨認哪些子問題其實相同，讓每個狀態只求解一次。

本章從狀態與依賴開始，依序處理 **鋼條切割 → 背包 → 最長共同子序列與序列比對 → 矩陣鏈與最佳二元搜尋樹**。

前置知識為遞迴、漸近分析、陣列與樹。重點是把問題定義、遞迴式、填表順序與答案重建連成同一個演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 先決定狀態代表什麼

| 設計問題 | 必須說清楚的內容 |
| --- | --- |
| 狀態是否足夠？ | 相同狀態的未來選擇與成本必須相同。 |
| 遞迴是否完整？ | 每種合法最優解都要落入某個候選分支。 |
| 先算哪些狀態？ | 邊界與依賴決定合法的填表順序。 |
| 如何交回答案？ | 最優值、選擇紀錄與重建路徑各有用途。 |

每個模型都先固定合法解與成本，再比較時間、空間及是否能重建一個最優解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 動態規劃：先決定什麼才算同一問題

動態規劃（dynamic programming, DP）把重複出現的子問題合併為狀態，保存答案，再組合出原問題答案。

- **狀態語義**：此格究竟最佳化哪一組合法解？
- **轉移與證明**：候選能否涵蓋所有最佳解，而且都合法？
- **計算與重建**：依賴先算，並記錄達成最佳值的選擇。

最佳子結構支持正確組合；子問題重疊解釋為何快取能節省重算。兩者不能代替狀態定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 狀態充分性：忘記歷史之前先檢查

若兩段歷史被合併為同一狀態，它們必須具有相同的後續合法選擇、後續成本規則與剩餘目標。

已累積價值可以分開記帳；狀態需保留會改變未來的資訊。

例如切鋼條：每段價格只取決於長度、切割免費、沒有段數上限。剩餘長度同為 $j$，先前如何切不影響未來，因此 $r[j]$ 足夠。

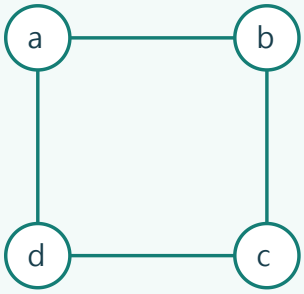
若新增「最多切成 $k$ 段」，只記 $j$ 便不足，還需記剩餘可用段數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最長簡單路：端點狀態會漏掉資源限制

無向圖的邊為 $ab, bc, cd, da$，每條邊長為1；簡單路不可重複頂點。

端點相同，不代表餘下資源相同




最佳整路 / Whole optimum: a → b → c → d

後綴 / Suffix: c → d (1)

獨立最長 / Independent: c → b → a → d (3)

拼接 / Splice: a → b → c → b → a → d

重複 a, b：非法

$a \to b \to c \to d$ 是 $a$ 到 $d$ 的最長簡單路，長度3。但其中 $c \to d$ 並非端點 $c, d$ 間最長路；$c \to b \to a \to d$ 長度3。直接替換得到 $a, b, c, b, a, d$，已不合法。

失敗原因：端點沒有記錄前綴已使用的頂點，不能任意拼接各端點間最佳解。



## 擴充狀態可以修復，但可能付出指數成本

固定起點 $s$，令 $D[S, v]$ 為「恰好使用頂點集合 $S$、從 $s$ 到 $v$ 的最長簡單路長度」，不存在則為 $-\infty$。

$$D[\{s\}, s] = 0, \qquad D[S, v] = \max_{\substack{u \in S \setminus \{v\} \\ uv \in E}} (D[S \setminus \{v\}, u] + 1).$$

其他基底為 $-\infty$；僅計算 $s, v \in S$、$v \neq s$ 的非基底狀態。答案為 $\max_{S \ni s,t} D[S, t]$。

集合大小嚴格增加，形成狀態 DAG；至多 $O(2^{|V|}|V|^2)$ 時間。這是指數 DP，不是多項式解。若原圖本身是 DAG，端點最長路可直接按拓撲序計算。

## 切鋼條：問題條件與狀態

給定整數長度 $n \geq 0$，價格 $p[1..n]$。將整條鋼條切成正整數長度的段，每段都出售；切割免費，同長度可以出現多次。

$$r[j] = \text{恰好用完長度 } j \text{ 的最大總收益}, \qquad r[0] = 0.$$

本例價格（收益單位）：

| 長度 $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p[i]$ | 1 | 5 | 8 | 9 |

求最佳值 $r[4]$，並輸出每段長度。整段出售也是合法方案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第一段選擇導出遞迴式

任何非空切法都有第一段長度 $i$，$1 \leq i \leq j$；第一段直接出售，只有餘段繼續切。

$$r[j] = \max_{1 \leq i \leq j} \{p[i] + r[j-i]\}, \qquad j \geq 1.$$

- $i = j$：不再切割，餘段為空，收益 $p[j] + r[0]$。
- 不把第一段寫成 $r[i]$：此處 $i$ 表示一段成品的長度。
- 只要長度下降，就能以 $j$ 歸納證明計算正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩向證明：不漏解，也不引入假解

令 $OPT(j)$ 為真正最佳收益，並假設所有更短狀態均已正確。

**上界方向**：任一最佳切法第一段為 $i$，其餘段收益不超過 $r[j-i]$。所以 $OPT(j) \le p[i] + r[j-i] \le \max_i(p[i] + r[j-i])$。

**下界方向**：對每個 $i$，一段長度 $i$ 接上長度 $j-i$ 的最佳切法，長度恰為 $j$，全部合法。因此 $p[i] + r[j-i] \le OPT(j)$；取最大仍成立。

兩式合起來得到等號。替換餘段不會改變第一段合法性，是最佳子結構成立的關鍵。

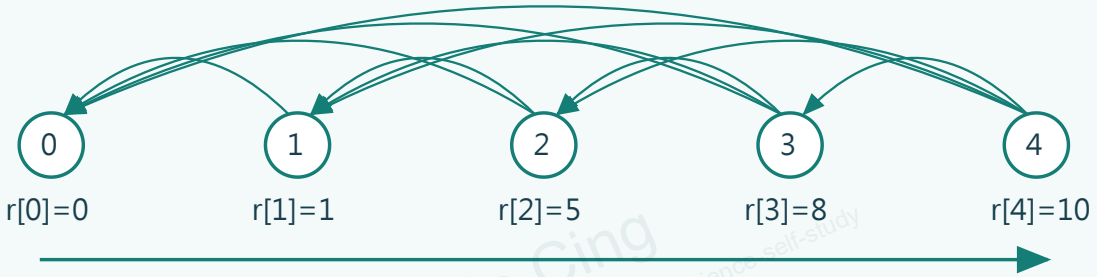
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 重疊子問題與依賴 DAG

箭頭 $j \to k$ 表示計算 $r[j]$ 需要 $r[k]$。

箭頭：狀態 → 所需子問題




計算次序：0 → 1 → 2 → 3 → 4

5 個狀態；10 條依賴；j 嚴格下降

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 先完成較短依賴，再計算較長狀態

所有 $k < j$ 都是 $r[j]$ 的候選依賴。例如 $r[3]$、$r[4]$ 都需要 $r[2]$。合併同長度呼叫後，這份答案只需保存一次。

長度4的依賴 DAG 有5個狀態 $0, 1, 2, 3, 4$。狀態 $j$ 有 $j$ 條向外的依賴邊，因此總數為 $1 + 2 + 3 + 4 = 10$。

依 $0, 1, 2, 3, 4$ 填表，方向與依賴箭頭相反。計算 $r[j]$ 時，所有需要的 $r[k]$ 都已完成。依賴長度嚴格下降，因此不會形成循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 記憶化：每個狀態只展開一次

```
memo[0] = 0; 其餘為 UNKNOWN
solve(j):
    if memo[j] is not UNKNOWN: return memo[j]
    q = -∞
    for i = 1 .. j:
        q = max(q, p[i] + solve(j-i))
    memo[j] = q
    return q
```

呼叫從目標往下展開；重複呼叫直接讀表。以獨立的 `UNKNOWN` 記未計算，不能把合法的負收益當作未知。

本題所有長度都會到達，故仍需 $\Theta(n^2)$ 時間、$\Theta(n)$ 表格與至多 $O(n)$ 呼叫堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 由小到大填表，同時記下第一段

```
r[0] = 0
for j = 1 .. n:
    r[j] = -∞
    for i = 1 .. j:           # i 由小到大
        q = p[i] + r[j-i]
        if q > r[j]:          # 同值保留較小的第一段
            r[j] = q
            first[j] = i
```

迴圈不變量：開始算 $j$ 時，$r[0..j-1]$ 全部正確。每個候選都只讀這些已完成的值。

時間為 $\sum_{j=1}^{n} j = n(n+1)/2 = \Theta(n^2)$；值表與選擇表共 $\Theta(n)$ 空間。



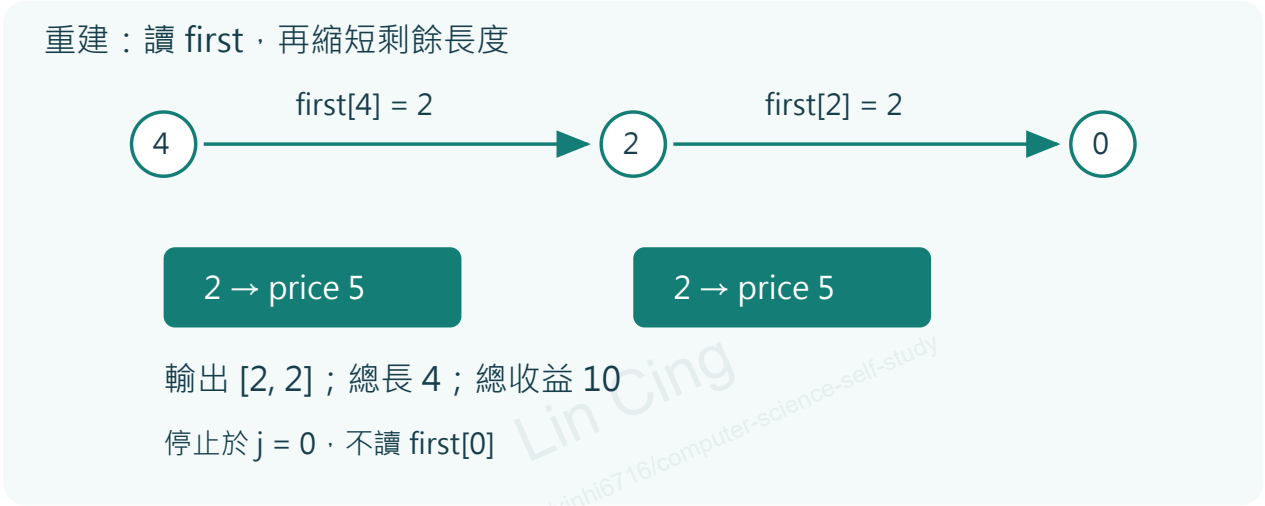
## 完整數例：每個候選都可追查

依 $i = 1, 2, \ldots, j$ 列出候選 $p[i] + r[j-i]$。

| $j$ | 候選收益（依 $i$ 遞增） | 掃描中的最大值 | $r[j]$ | first[$j$] |
|---|---|---|---|---|
| 0 | 空切法 | — | 0 | — |
| 1 | $1+0=1$ | 1 | 1 | 1 |
| 2 | $1+1=2,\ 5+0=5$ | 2, 5 | 5 | 2 |
| 3 | $1+5=6,\ 5+1=6,\ 8+0=8$ | 6, 6, 8 | 8 | 3 |
| 4 | $1+8=9,\ 5+5=10,\ 8+1=9,\ 9+0=9$ | 9, 10, 10, 10 | 10 | 2 |

收益最大的第一段不一定是售價最高的段：$p[4]=9$，而兩段長度2可得10。

## 沿記錄的選擇還原最佳切法




此例 $first[4]=2$、$first[2]=2$。剩餘長度依序為 $4\to2\to0$，輸出 $[2,2]$。核對長度和 $2+2=4$，收益 $5+5=10=r[4]$。



## 記錄的切法為何維持最佳收益

剩餘長度為 $j$ 時，已輸出各段的長度和為 $n-j$，且其收益加上 $r[j]$ 等於 $r[n]$。初始 $j=n$、尚未輸出任何段，因此兩項性質都成立。

令 $i=first[j]$。輸出 $i$ 並把 $j$ 換成 $j-i$，已輸出長度恰增加 $i$。由 $r[j]=p[i]+r[j-i]$，已輸出收益的增加量，恰等於剩餘最佳收益的減少量，因此兩項性質都維持。

到達 $j=0$ 時，已輸出長度為 $n$，且 $r[0]=0$，故已輸出收益為 $r[n]$。本例輸出兩段長度2，總長度4、收益10。

## 重建迴圈在剩餘長度為零時停止

填表完成後，每個正長度 $j$ 的 `first[j]` 都記錄一個最佳第一段。沿這些選擇即可輸出完整切法。

```
j = n; pieces = []
while j > 0:
    i = first[j]
    append i to pieces
    j = j - i
return pieces
```

每個選擇皆滿足 $1 \le first[j] \le j$。每次至少減1，故迴圈至多執行 $n$ 次便在零停止，重建需 $O(n)$ 時間。此時不變量保證總長度為 $n$、總收益為 $r[n]$。

$n = 0$ 時不進入迴圈，直接回傳空串列。



## 記憶化與填表是在排同一張圖

| 面向 | Top-down memoization | Bottom-up tabulation |
| --- | --- | --- |
| 啟動方式 | 從目標遞迴要求依賴 | 按合法順序逐狀態計算 |
| 實際狀態 | 僅展開可達狀態 | 通常填預先指定的範圍 |
| 額外成本 | 呼叫堆疊、快取查詢 | 通常無遞迴堆疊 |
| 本例時間／表格 | $\Theta(n^2)$／$\Theta(n)$ | $\Theta(n^2)$／$\Theta(n)$ |

記憶化不保證比填表快；填表也不必是二維矩形。兩者都必須先有正確轉移與無循環的依賴順序。

## 邊界條件會改變答案

**必須用完整條**：若 $p[1]=-2, p[2]=-5$，則 $r[1]=-2$、$r[2]=\max(-4,-5)=-4$。初始最大值必須為 $-\infty$，不能為0。

**允許丟棄剩餘材料**：改成 $r[j]=\max(0,\max_i(p[i]+r[j-i]))$，同一負價例答案為0；重建也需能記錄「停止並丟棄」。這是另一個問題。

**空輸入**：$n=0$，價格表可為空，收益0、切法空。若 $n>0$，本章假設每個長度 $1..n$ 都有價格；缺價長度不能直接當成0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 換條件時，重新檢查轉移

固定每次實際切割費用 $c\ge 0$：最後整段出售時不收切割費。

$$r[j]=\max_{1\le i\le j}\left(p[i]+r[j-i]-c\,\mathbf{1}_{i<j}\right).$$

原例 $c=2$：兩段2收益 $5+5-2=8$，整段4收益9，因此最佳改為不切。

這仍只需長度狀態；若限制成品段數或價格依先前選擇改變，就應擴充狀態。判斷依據是未來是否合法、成本能否只由狀態決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用可檢查的五件事設計 DP

1. 精確寫出每個狀態的合法解集合與基底。
2. 指定第一個／最後一個決策，枚舉全部合法候選。
3. 證明最佳解落在候選中，而且候選都能組成合法解。
4. 找出嚴格下降的尺度，安排依賴順序。
5. 以「狀態數 × 每狀態候選成本」估算，並另外驗證重建。

本例：長度狀態、第一段決策、兩向證明、遞減長度、二次時間；輸出切法再核對總長度與收益。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 0/1 背包：每件只能選一次

有 $n$ 件物品，重量 $w_i$ 為正整數，價值 $v_i$，容量 $W$ 為非負整數。

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{使} \quad \sum_{i=1}^{n} w_i x_i \le W, \quad x_i \in \{0,1\}.$$

允許空背包，所以最優值至少為 0；負價值物品不必取。物品可有相同重量或價值，但仍是不同的件。 整數容量是這個容量索引 DP 的前提；任意實數重量不能直接當陣列索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 狀態為何要保留物品範圍

定義 $D[i, c]$：只用物品 $1, \ldots, i$、總重至多 $c$ 的最大價值。答案為 $D[n, W]$。 只記「剩餘容量」不夠：相同容量下，可用物品不同，答案也可能不同。以物品前綴限制可用集合，才不會重複拿同一件。 對固定 $(i, c)$，未來選擇只取決於此前綴與容量；不同歷史若到達相同狀態，較差的價值不可能改善任何延伸。這裡假設價值可相加，沒有物品間相依限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



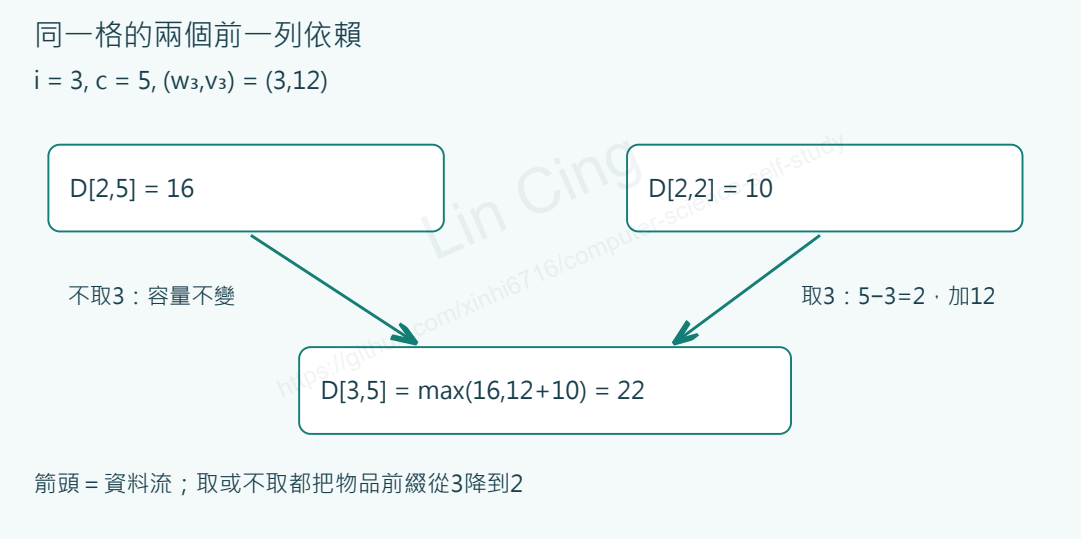
## 兩個互斥分支導出遞迴

$$D[0, c] = 0, \qquad D[i, 0] = 0.$$

若 $c < w_i$，則 $D[i, c] = D[i-1, c]$；否則

$$D[i, c] = \max\{D[i-1, c],\ v_i + D[i-1, c - w_i]\}.$$

不取 $i$ 保留容量；取 $i$ 要扣去 $w_i$，且兩分支都只剩前 $i-1$ 件。



## 遞迴的上下界證明

任取 $(i, c)$ 的可行集合 $S$。若 $i \notin S$，其價值至多 $D[i-1, c]$；若 $i \in S$，移除 $i$ 後只用前 $i-1$ 件，重量至多 $c - w_i$，故價值至多 $v_i + D[i-1, c-w_i]$。因此最優值不超過候選最大值。 反方向：不取分支的最優集合可直接使用；取分支的最優集合不含 $i$，加上 $i$ 後仍不超重，故兩個候選都是可達值。最大候選不超過最優值。 配合 $i = 0$ 的基底，對 $i$ 歸納即得正確性。若移除 $i$ 後不是剩餘問題的最優解，可換成更好的解並加回 $i$，與原解最優矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一數例：輸入與填表順序

容量 $W = 5$；物品 $(w_i, v_i)$ 為 $(2, 10), (1, 6), (3, 12)$。

列是可用前綴，欄是容量上限，並非必須填滿的重量。

每格只讀前一列；依 $i = 1, 2, 3$ 填列，同列容量順序可任意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一數例：完整狀態表

完整表與回溯：W = 5

物品 (w,v)：1=(2,10), 2=(1,6), 3=(3,12)

| 前綴 i / 容量 c | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| i = 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| i = 1 | 0 | 0 | 10 | 10 | 10 | 10 |
| i = 2 | 0 | 6 | 10 | 16 | 16 | 16 |
| i = 3 | 0 | 6 | 10 | 16 | 18 | 22 |

箭頭 = 回溯：取3 → 不取2 → 取1；總重5，總值22

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 看懂表格中的三次比較

完整表給出 $D[3,5]=22$。

物品 1：$D[1,1]=0$（太重）；$D[1,2]=\max(0,10+0)=10$。 物品 2：$D[2,3]=\max(D[1,3],6+D[1,2])=\max(10,16)=16$。 物品 3：$D[3,3]=\max(16,12+D[2,0])=16$，能裝不代表必須取；$D[3,4]=\max(16,12+6)=18$；$D[3,5]=\max(16,12+10)=22$。 每個「取」候選皆是當前物品價值加上可行的前綴最優值，不能加同列任意最大值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 回溯把最優值變成物品集合

從 $(i,c)=(n,W)$ 開始；若 $D[i,c]=D[i-1,c]$，選擇不取並令 $i\leftarrow i-1$。否則取 $i$，令 $c\leftarrow c-w_i$、$i\leftarrow i-1$。相等時固定不取，輸出一組最優解。 本例 $(3,5):22>16$，取 3，至 $(2,2)$；$10=10$，不取 2，至 $(1,2)$；$10>0$，取 1，至 $(0,0)$。 輸出 $\{1,3\}$：重量 $2+3=5$，價值 $10+12=22$。每步 $i$ 減一，回溯 $O(n)$；不要把容量遞減當作唯一停止條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 可直接實作的二維版本

```
D[0..n][0..W] = 0
for i = 1..n:
    for c = 0..W:
        D[i,c] = D[i-1,c]
        if c >= w[i]:
            D[i,c] = max(D[i,c], v[i] + D[i-1,c-w[i]])
```

表格有 $(n+1)(W+1)$ 格，每格常數次運算。時間 $O(n(W+1))$，完整表空間 $O(n(W+1))$；通常在 $W\geq 1$ 時寫作 $O(nW)$。保留表格即可依前頁程序重建。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 壓成一維：倒序維持前一列

```
dp[0..W] = 0
for i = 1..n:
    for c = W down to w[i]:
        dp[c] = max(dp[c], v[i] + dp[c-w[i]])
```

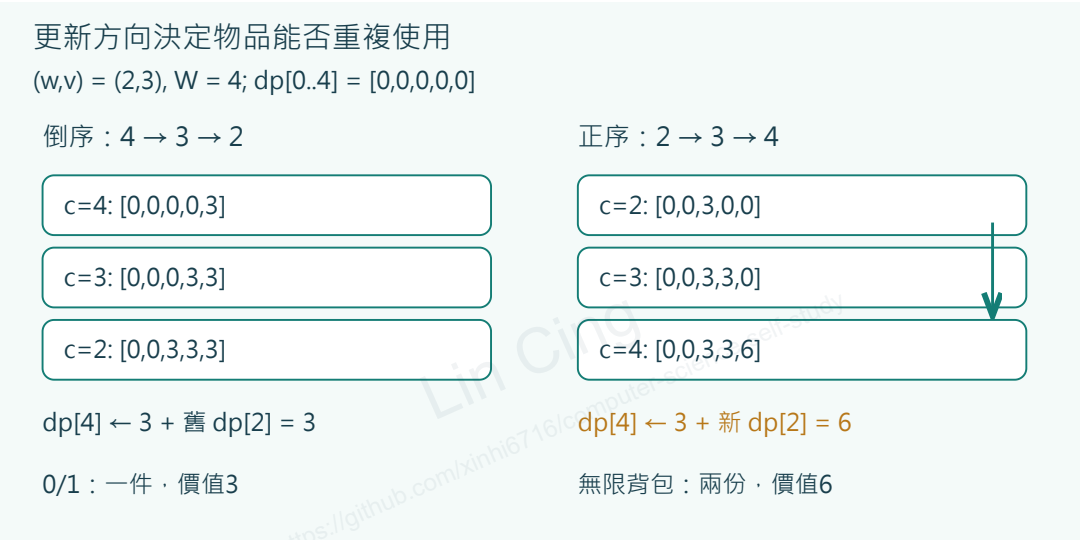
處理 $i$ 時，尚未更新的容量存 $D[i-1,\cdot]$。倒序且 $w_i > 0$，使 $c - w_i < c$ 尚未更新；右側讀到的仍是前一列。更新後 $dp[c] = D[i,c]$。 時間仍 $O(nW)$，值的空間降至 $O(W)$。被覆寫的歷史不足以直接做二維回溯；需要解集合時，可保留二維表或另設帶版本的決策紀錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正序反例：一件物品被拿了兩次

只有一件物品 $(w,v) = (2,3)$，容量 $W = 4$。0/1 最優值應為 3。




正序在 $c = 2$ 寫入 3，之後 $c = 4$ 讀到剛寫的 $dp[2] = 3$，得到 6，等於使用兩份同一物品。更新方向是在選擇依賴哪個版本的狀態。

## 無限背包：同列依賴正好需要正序

若每種物品可取任意非負整數份，定義 $U[i,c]$ 為前 $i$ 種、重量至多 $c$ 的最大價值。可放時

$$U[i,c]=\max\{U[i-1,c],\ v_i+U[i,c-w_i]\}.$$

取走一份後仍能取第 $i$ 種，所以第二項是同列。以容量正序更新一維陣列即可；因 $w_i>0$，同列依賴的容量嚴格較小。 例 $(2,3),W=4$：可取兩份，答案 6；此時前頁的正序結果才符合模型。本節排除零重物品；零重正價值且無限供應會使目標無上界，不能套用此有限遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 恰好裝滿：不可行不是零分

令 $E[i,c]$ 為前 $i$ 件、總重**恰為** $c$ 的最大價值。遞迴形式與 0/1 相同，但

$$E[0,0]=0,\quad E[0,c]=-\infty\ (c>0).$$

不可行候選保持 $-\infty$；實作先檢查可達再相加，避免整數哨兵溢位。正重量使 $E[i,0]=0$。 例單件 $(2,-3),W=2$：至多容量答案為 0（空集合）；恰好容量答案為 $-3$。若 $W=1$，恰好容量不可行。全部初始化為 0 會把不存在的重量誤作可行解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 偽多項式取決於數值大小

$O(nW)$ 對容量數值 $W$ 是多項式，對二進位輸入長度卻未必。記錄容量約需 $\log_2(W+1)$ 位元；若 $W = 2^b$，掃描容量要 $2^b$ 次。 例如同樣 20 件物品，$W = 10^3$ 約有兩萬格更新，$W = 10^9$ 約有兩百億格。整數重量可索引不等於容量索引一定實用。 此分析把每次數值加法與比較視為常數成本；若價值整數位數很大，還需計入算術位元成本。0/1 背包的決定版是 NP-complete；最佳化版稱 NP-hard。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 分數背包的貪心為何不同

若改成 $0 \le x_i \le 1$，每件可切分，對正價值物品按 $v_i/w_i$ 由大到小取；負價值可略過。若較低密度被選而較高密度尚未用完，交換相同重量 $\delta$ 會使價值不減；反覆交換可得到密度貪心解。 用相同三件物品、容量改為 $W = 4$：密度依序是物品 2 的 6、1 的 5、3 的 4。 分數解取 2、1、$1/3$ 件 3，價值 $6 + 10 + 4 = 20$。0/1 密度貪心取 2、1 得 16；0/1 最優取 2、3 得 18。交換重量若不能切分就未必可行。排序實作為 $O(n \log n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分枝限界的上界不是 DP 格值

對最大化問題，分枝限界法（branch and bound）在取／不取的搜尋樹中，保留已找到的可行解價值 $L$。令已取物品價值為 $V$、剩餘容量為 $C$，只用尚未決定物品的分數背包最優值為 $F(C)$。節點的總上界為 $B = V + F(C)$。 因放寬限制只會增加可行集合，節點任何 0/1 完成解價值都不超過 $B$；若 $B \le L$，可剪去該節點（只求一組最優解）。 上一例根節點 $B = 20$，整數最優值為 18：上界不一定可達。DP 的 $D[i, c]$ 則是所定義子問題的**確切**最優值；兩者不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 子序列保留次序，允許跳過字元

子序列的索引必須嚴格遞增；子字串則必須連續。

固定 $X =$ `ABCBDAB`、$Y =$ `BDCABA`。`BCBA` 在 $X$ 使用索引 $(2, 3, 4, 6)$，在 $Y$ 使用 $(1, 3, 5, 6)$。它的長度為 4；`BCA` 雖也是共同子序列，卻不是最長。

目標先求最長共同子序列（LCS）的**長度**，再重建其中一個解。

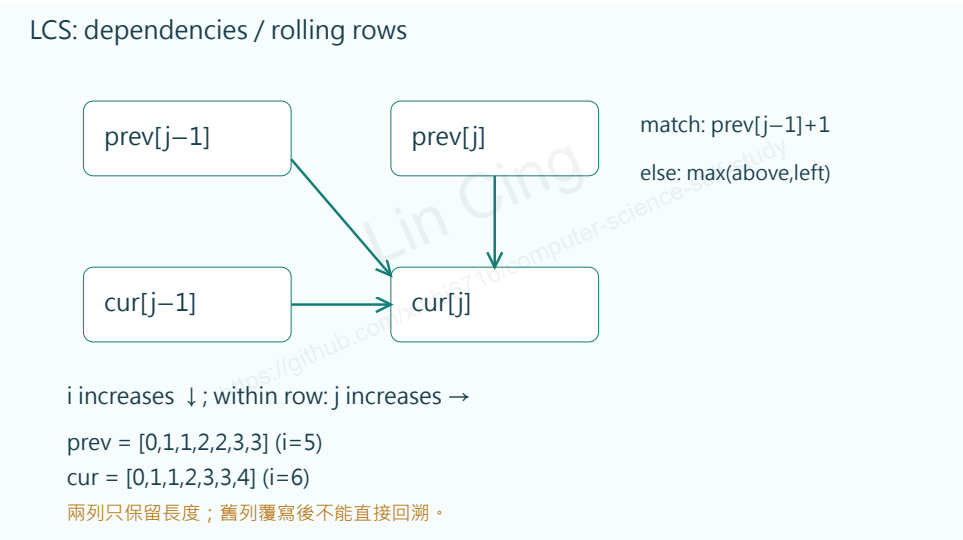
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 狀態必須同時記住兩個前綴

令 $C[i, j]$ 為 $X[1..i]$ 與 $Y[1..j]$ 的 LCS 長度。答案是 $C[7, 6]$。

$C[0, j] = C[i, 0] = 0$：任一前綴為空，只能取空子序列。

依賴只往較小的 $i + j$，形成 DAG。按照 $i = 1..m$、每列 $j = 1..n$ 填表時，所需的左、上、左上格皆已完成。






## 相同末字元：先證明可達下界

若 $x_i = y_j = a$，取較短前綴的一個 LCS，再附加 $a$。索引合法，故

$$C[i, j] \geq C[i - 1, j - 1] + 1.$$

這證明「候選解做得到」。例如 $(6, 6)$ 的末字元皆為 A，$C[5, 5] = 3$，故有長度 4 的共同子序列。單有下界尚不能斷言它最優。

## 相同末字元：再證明沒有更長的解

取任一 LCS $Z$，長度 $k$。其末字元必為 $a$；若不是，兩邊最後位置都未用，可再附加 $a$，與最長矛盾。

刪掉 $Z$ 最後的 $a$，其餘字元在兩邊都可嵌入前 $i-1, j-1$ 個位置。因此 $k-1 \le C[i-1, j-1]$，得到上界。結合前頁：

$$C[i, j] = C[i-1, j-1] + 1.$$

這保證存在使用兩個末位置的最佳嵌入；不代表每一種最佳匹配的索引都必須使用它們。例如 A 對 AA 可匹配第一個或第二個 A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 不同末字元：兩個方向的界

若 $x_i \neq y_j$，共同子序列的一組嵌入不可能同時使用兩邊最後位置：若兩者都使用，必須是同一個末字元。

所以任一最佳解至少略過 $x_i$ 或 $y_j$，其長度至多為 $\max(C[i-1, j], C[i, j-1])$。這是上界。

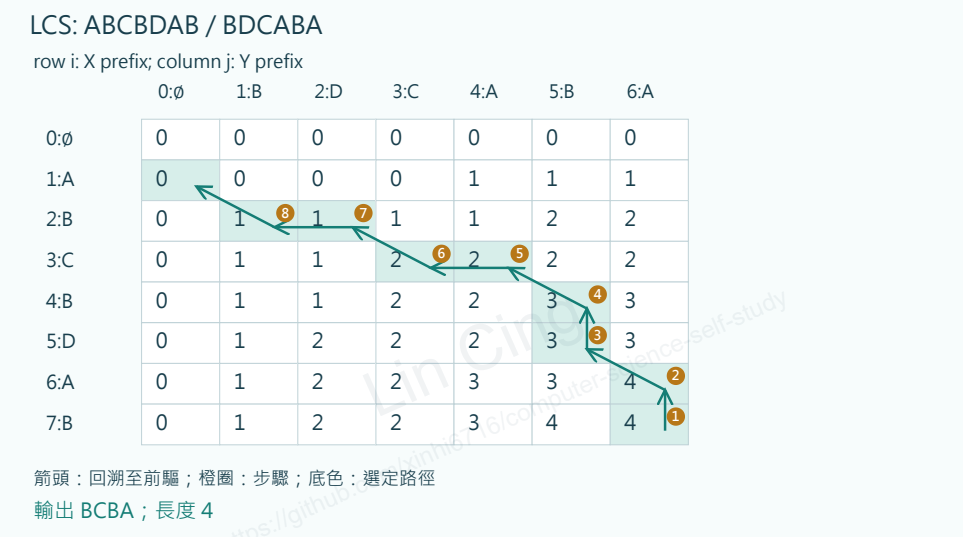
反向看，兩個較短前綴的共同子序列，在完整前綴中仍合法；取較長者即可達到該最大值。故

$$C[i, j] = \max\{C[i-1, j], C[i, j-1]\}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 完整值表：數值代表長度

相同取左上加 1，不同取上、左最大值。零邊界不可省略。




例如 $C[3,3] = C[2,2] + 1 = 2$；$C[5,5] = \max(3,2) = 3$；$C[7,6] = \max(4,4) = 4$。編號箭頭是下一頁逐步使用的回溯路線。



## 逐步回溯：反轉 ABCB 得 BCBA

不同字元且上、左同值時，本例固定先往上；相同則往左上並記錄字元。

| 步 | 目前格 → 前驅格 | 理由／記錄 |
|---|---|---|
| 1 | $(7,6) \to (6,6)$ | 上、左皆 4，先上 |
| 2 | $(6,6) \to (5,5)$ | A=A，記 A |
| 3 | $(5,5) \to (4,5)$ | 上 3 > 左 2 |
| 4 | $(4,5) \to (3,4)$ | B=B，記 B |
| 5 | $(3,4) \to (3,3)$ | 左 2 > 上 1 |
| 6 | $(3,3) \to (2,2)$ | C=C，記 C |
| 7 | $(2,2) \to (2,1)$ | 左 1 > 上 0 |
| 8 | $(2,1) \to (1,0)$ | B=B，記 B；停止 |

## ties 決定選哪一個最佳解

倒序記錄 ABCB，反轉得到 BCBA。

BCBA、BCAB、BDAB 都是長度 4 的 LCS。不同 tie 規則可能回傳不同字串，長度不變。

在 $(7, 6)$ 往左到 $(7, 5)$ 也是合法最佳選擇；往上只是本例的確定性規則。

只存一個前驅只能重建一個解。若要列出所有不同 LCS，必須探索所有合法最佳分支並去重；分支路徑與不同字串的數量也不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 填表與重建有不同成本

共有 $(m+1)(n+1)$ 個狀態，每個內部格只做常數次比較：填完整表時間 $\Theta(mn)$、空間 $\Theta(mn)$。

從 $(m, n)$ 回溯，每一步至少減少一個索引，時間 $O(m+n)$。可存 choice 表，也可保留完整值表，回溯時重新比較鄰格；兩者都能重建一個解。

暴力枚舉一邊的索引子集有 $2^m$ 種候選；許多不同遞迴路徑會重複要求同一對前綴。DP 合併的就是這些重複狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩列滾動：保留長度，丟掉歷史

讓較短的字串放在欄方向。prev 是上一列，cur 是當前列；每列由左至右更新。

```
prev[0..n] = 0
cur[0..n] = 0
for i = 1..m:
    cur[0] = 0
    for j = 1..n:
        if X[i] == Y[j]: cur[j] = prev[j-1] + 1
        else: cur[j] = max(prev[j], cur[j-1])
    swap(prev, cur)
return prev[n]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 兩列滾動與解的重建

時間仍為 $\Theta(mn)$，額外空間 $O(\min(m, n))$。此程式只回傳長度。

每次更新讀取上一列與當前列的左鄰格。交換兩列後，舊歷史可以覆寫，因為後續長度計算不再需要它。

過去列被覆寫後，不能直接照完整表路線回溯。若要輸出序列，保留完整值／choice 表，或另外使用分治重建等演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 全域對齊：每個字元都必須交代

給定 $X = \texttt{ACGTA}$、$Y = \texttt{AGCAC}$。可插入缺口 `-`，使兩列等長；不得有 `-/-` 欄。刪去缺口後須恢復原序列。

| 一欄的類型 | 分數 |
|---|---|
| 相同字元 | +3 |
| 不同字元 | −2 |
| 一個字元對一個缺口 | −3 |

目標是所有欄分數總和最大，開頭與結尾的缺口也照常扣分。這是線性缺口成本的全域對齊；LCS 長度不是本問題的分數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 對齊狀態與完整邊界

$F[i, j]$ 是前綴 $X[1..i]$ 與 $Y[1..j]$ 的最佳全域對齊分數。

$$F[0,0] = 0, \qquad F[i,0] = -3i, \qquad F[0,j] = -3j.$$

只有一邊有字元時，每個字元都必須對缺口，因此邊界為 `0, -3, -6, -9, -12, -15`。

不能把邊界全設 0；那會把未對齊的前綴免費略過，改變問題。也不能加上「與 0 取最大」而仍稱為同一個全域對齊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最後一欄只有三種可能

令 $s(a,b)=3$（相同）或 $-2$（不同）。當 $i,j>0$：

$$F[i,j]=\max\begin{cases}F[i-1,j-1]+s(x_i,y_j) & \text{字元對字元}\\ F[i-1,j]-3 & x_i \text{ 對缺口}\\ F[i,j-1]-3 & \text{缺口對 } y_j.\end{cases}$$

任一合法對齊的最後一欄必屬其中一類；刪去後得到相應前綴對齊，故不會超過右側最大值。反過來，任一前驅最佳對齊附加該欄仍合法，能達到該候選值。兩方向共同證明遞推式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 對齊完整數例：最佳分數為 1

Global alignment: ACGTA / AGCAC

row i: X prefix; column j: Y prefix

| | 0:ø | 1:A | 2:G | 3:C | 4:A | 5:C |
|---|---|---|---|---|---|---|
| 0:ø | 0 | -3 | -6 | -9 | -12 | -15 |
| 1:A | -3 | 3 | 0 | -3 | -6 | -9 |
| 2:C | -6 | 0 | 1 | 3 | 0 | -3 |
| 3:G | -9 | -3 | 3 | 0 | 1 | -2 |
| 4:T | -12 | -6 | 0 | 1 | -2 | -1 |
| 5:A | -15 | -9 | -3 | -2 | 4 | 1 |

箭頭：回溯至前驅；橙圈：步驟；底色：選定路徑

輸出 ACGTA- / A-GCAC；總分 1

$F[2,3]=\max(0+3,-3-3,1-3)=3$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 重建對齊：每一個欄都有代價

從 $(5,5)$ 回溯：

| 目前格 → 前驅格 | 附加欄 | 該步加分 |
|---|---|---|
| $(5,5) \to (5,4)$ | `-/C` | −3 |
| $(5,4) \to (4,3)$ | `A/A` | +3 |
| $(4,3) \to (3,2)$ | `T/C` | −2 |
| $(3,2) \to (2,1)$ | `G/G` | +3 |
| $(2,1) \to (1,1)$ | `C/-` | −3 |
| $(1,1) \to (0,0)$ | `A/A` | +3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正向讀出重建的對齊

反轉欄順序：

```
ACGTA-
A-GCAC
```

總分 $3-3+3-2+3-3=1$。兩個原序列都完整保留。

兩個缺口欄分別交代 $X$ 的 C 與 $Y$ 最後的 C。T/C 不匹配貢獻 $-2$，三個匹配欄貢獻 $3+3+3$。

$F[5,5]=\max(-2-2,-1-3,4-3)=1$，故最後一欄選 `-/C`。即使最終分數為正，中間格仍可為負；全域對齊不把它們重設為 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 相似的表格，需要不同的語義

| 問題 | 狀態值 | 相同字元 | 不同字元 | 邊界 |
|---|---|---|---|---|
| LCS | 最長長度 | 左上 +1 | 上、左最大 | 0 |
| 全域對齊 | 最大總分 | 三候選仍都比較 | 三候選仍都比較 | 累計缺口成本 |

LCS 可以略過字元而不計費；對齊必須把所有字元放到某一欄。對齊即使字元相同，也依既定三分支規則比較候選，不能直接套用 LCS 的 if/else。

兩者都能用前綴 DAG 填表；完整重建則需要足夠的前驅資訊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 區間 DP：最後一次決策

矩陣鏈最後一次相乘決定切點 $k$；BST 的根 $r$ 決定左右鍵區間。兩者都保留原本順序，不能任意重排。

狀態 $[i, j]$ 記錄連續區間；依長度遞增填表，因為每個決策只依賴更短區間。值表回答「多好」，決策表回答「如何做」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 矩陣鏈：維度決定合併成本

給定 $p = [5, 10, 3, 12, 5]$，矩陣 $A_i$ 的維度是 $p_{i-1} \times p_i$。

| 矩陣 | $A_1$ | $A_2$ | $A_3$ | $A_4$ |
|---|---|---|---|---|
| 維度 | $5 \times 10$ | $10 \times 3$ | $3 \times 12$ | $12 \times 5$ |

本節用傳統乘法計純量乘法數：$(a \times b)(b \times c)$ 要 $abc$ 次，結果 $a \times c$。只改括號，不改矩陣順序；DP 規劃成本不包含實際相乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



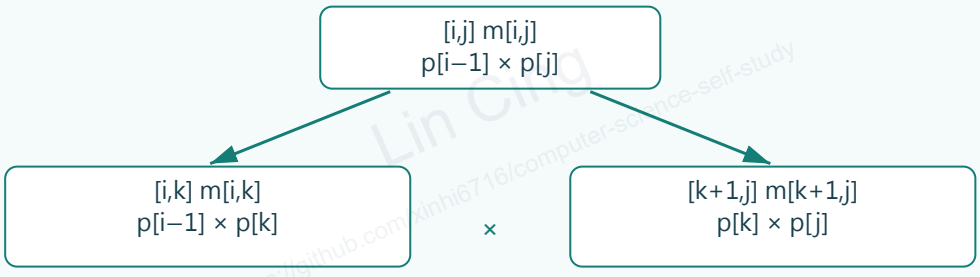
## 矩陣鏈遞迴與狀態充分性

$m[i, j]$ 是計算 $A_i \cdots A_j$ 的最少乘法數，$m[i, i] = 0$。

$$m[i, j] = \min_{i \le k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}.$$

任何括號化的區間結果都為 $p_{i-1} \times p_j$，所以外部只需知道端點與最小成本，不需記內部括號。固定切點後兩個子區間的維度都固定。

**固定切點：兩側輸出維度不隨括號改變**




merge = p[i−1] · p[k] · p[j]

箭頭：父狀態依賴兩個更短區間；替換子解只降低內部成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 切開、替換、再合併的證明

任一完整括號化的最後乘法都有某個 $i \le k < j$。若其中左子括號化非最優，換成更便宜的左解，輸出維度不變，右成本與合併成本也不變，總成本下降，矛盾；右側同理。

反方向，每個 $k$ 的兩個可行子解都能合法相乘，因此每個候選都是可行完整解。涵蓋最優解且沒有加入不可行解，取最小值就是最優。以區間長度歸納，已算子表正確即可推出當前格正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 依長度填表：所有短區間候選

所有對角線 $m[i,i]=0$；表列每個切點的「左＋右＋合併」。

| 長度 | 區間 | 候選與計算 | $m$；$s$ |
|---|---|---|---|
| 2 | $[1,2]$ | $k=1: 0+0+5\cdot 10\cdot 3=150$ | 150；1 |
| 2 | $[2,3]$ | $k=2: 0+0+10\cdot 3\cdot 12=360$ | 360；2 |
| 2 | $[3,4]$ | $k=3: 0+0+3\cdot 12\cdot 5=180$ | 180；3 |
| 3 | $[1,3]$ | $k=1: 0+360+600=960$；$k=2: 150+0+180=330$ | 330；2 |
| 3 | $[2,4]$ | $k=2: 0+180+150=330$；$k=3: 360+0+600=960$ | 330；2 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 完整鏈：三個切點都必須比較

| $[1,4]$ 的切點 | 左成本 | 右成本 | 最後相乘 | 候選總成本 |
|---|---|---|---|---|
| $k=1$ | 0 | 330 | $5\cdot10\cdot5=250$ | 580 |
| $k=2$ | 150 | 180 | $5\cdot3\cdot5=75$ | **405** |
| $k=3$ | 330 | 0 | $5\cdot12\cdot5=300$ | 630 |

最小合併成本本身不是一般性的貪婪規則，必須加上兩邊的最佳成本。此例巧合由 $k=2$ 同時得到最便宜合併與最便宜總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 完整鏈：三個切點都必須比較 （續）

矩陣鏈完整表：m[i,j] / s[i,j]

| | j=1 | j=2 | j=3 | j=4 |
|---|---|---|---|---|
| i=1 | 0 / – | 150 / 1 | 330 / 2 | 405 / 2 |
| i=2 | | 0 / – | 360 / 2 | 330 / 2 |
| i=3 | | | 0 / – | 180 / 3 |
| i=4 | | | | 0 / – |

填表順序：L=1 → 2 → 3 → 4；每格只讀更短區間

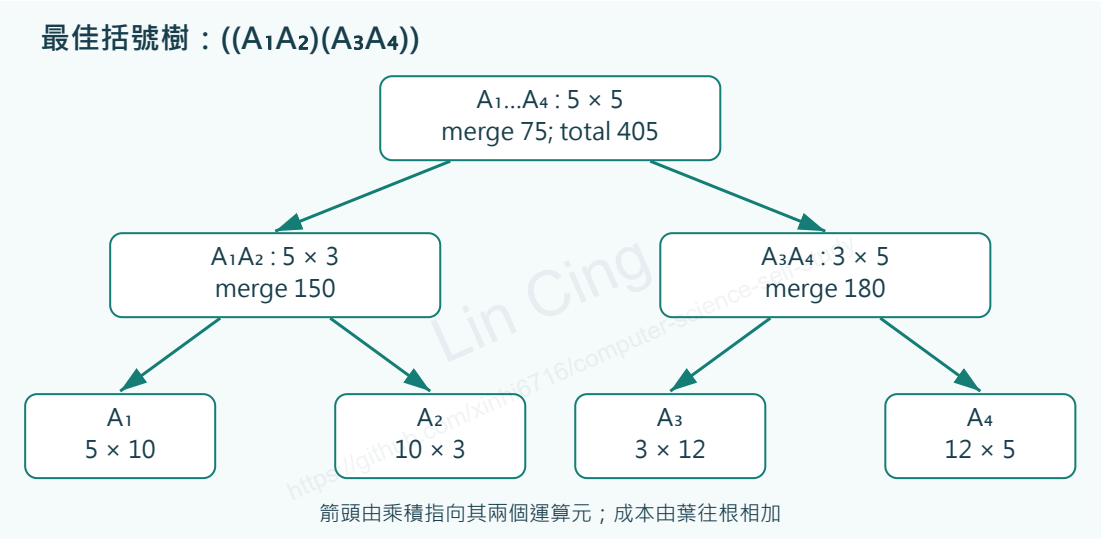
L=4: 580, 405, 630 → min = 405

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 由切點表還原括號樹

$s[1,4]=2$，接著 $s[1,2]=1$、$s[3,4]=3$，得 $((A_1A_2)(A_3A_4))$。

先算 $A_1A_2$：$5\times 3$，成本150；算 $A_3A_4$：$3\times 5$，成本180；最後得到 $5\times 5$，成本75，總計405。




遞迴：若 $i=j$ 輸出 $A_i$；否則輸出括號內的 `build(i,s[i,j])` 與 `build(s[i,j]+1,j)`。每個葉與合併節點各訪問一次，重建 $\Theta(n)$。



## 矩陣鏈演算法與精確工作量

```
for i=1..n: m[i,i]=0
for L=2..n:
  for i=1..n-L+1:
    j=i+L-1; m[i,j]=infinity
    for k=i..j-1:
      t=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
      if t<m[i,j]: m[i,j]=t; s[i,j]=k
```

長度 $L$ 有 $n-L+1$ 格、各有 $L-1$ 個候選，所以候選數

$$\sum_{L=2}^{n}(n-L+1)(L-1)=\binom{n+1}{3}=\frac{n^3-n}{6}.$$

故時間 $\Theta(n^3)$；值表與切點表各 $\Theta(n^2)$ 空間。這是規劃的複雜度，不是實際矩陣乘法耗時。

## OBST：成功與失敗都要建模

有序真鍵 $k_1 < \cdots < k_n$，成功查詢權重 $p_1, \ldots, p_n$。外部葉 $d_0, \ldots, d_n$ 代表查無資料的間隙，權重 $q_0, \ldots, q_n$。

$d_0$ 表示小於 $k_1$；$d_i$ 表示 $k_i < x < k_{i+1}$；$d_n$ 表示大於 $k_n$。一棵含 $n$ 個真鍵的擴充 BST 有 $n+1$ 個外部葉。

機率模型有 $\sum p + \sum q = 1$；非負次數權重也能直接使用，總成本除以全域總權重才得到每次查詢期望。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 兩種成本：差一個固定常數

根深度為0。$C(T)$ 只數實際比較的真鍵；$E(T)$ 額外數到達外部葉的一次終止檢查。

$$C(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j \operatorname{depth}(d_j).$$

$$E(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j(\operatorname{depth}(d_j) + 1).$$

所以 $E(T) - C(T) = \sum_j q_j$，與樹形無關，最優樹集合相同。空段只有 $d_{i-1}$：$C[i, i-1] = 0$，$E[i, i-1] = q_{i-1}$。初始化與報出的成本必須用同一慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## OBST 區間：保留全域機率質量

區間 $[i, j]$ 包含鍵 $k_i, \ldots, k_j$ 與外部葉 $d_{i-1}, \ldots, d_j$。

$$w[i, j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

子問題仍用原始全域權重，不除以區間質量。因為子樹降低一層時，每個屬於此子樹的查詢都多一次檢查，增加的是全域質量 $w$。

若改存條件期望，合併時必須重新乘回各子樹質量，不能把不同分母的條件期望直接相加。空段的 $w[i, i-1] = q_{i-1}$ 也不能漏掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## OBST 遞迴與替換證明

$$E[i, j] = w[i, j] + \min_{i \le r \le j} \{E[i, r-1] + E[r+1, j]\}.$$

$C$ 也滿足相同形狀的遞迴，但空段成本為0。根 $r$ 的左右鍵範圍由中序順序唯一決定；合併增加 $w[i, j]$。

任一最優樹都必有某個根。若其中子樹非最優，以同一鍵／間隙集合的較便宜樹替換，仍為 BST；子樹全域質量不變，所以移至相同深度所增成本不變，得到更小總成本而矛盾。反向，每個候選根搭配兩棵可行子樹都形成可行 BST，因此枚舉所有根後取最小即為最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三鍵例 B：初始化與單鍵區間

固定 $p = (.20, .30, .10)$、$q = (.10, .05, .15, .10)$，總質量1。

空段 $E[1,0] = .10$、$E[2,1] = .05$、$E[3,2] = .15$、$E[4,3] = .10$。

| 區間 | $w$ | 唯一候選 $E_L + E_R + w$ | $E$ | $C$ |
|---|---|---|---|---|
| $[1,1]$ | .35 | $.10 + .05 + .35$ | .50 | .35 |
| $[2,2]$ | .50 | $.05 + .15 + .50$ | .70 | .50 |
| $[3,3]$ | .35 | $.15 + .10 + .35$ | .60 | .35 |

單鍵區間的 $C$ 已含其左右間隙的失敗比較，故不是單純 $p_i$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 例 B：兩鍵與三鍵所有根候選

| 區間；質量 | 候選根與完整計算 | 最小 $E$；$C$ |
|---|---|---|
| $[1,2]$；.80 | $r=1: .10 + .70 + .80 = 1.60$；$r=2: .50 + .15 + .80 = 1.45$ | 1.45；1.15 |
| $[2,3]$；.70 | $r=2: .05 + .60 + .70 = 1.35$；$r=3: .70 + .10 + .70 = 1.50$ | 1.35；1.05 |
| $[1,3]$；1.00 | $r=1: .10 + 1.35 + 1 = 2.45$；$r=2: .50 + .60 + 1 = 2.10$；$r=3: 1.45 + .10 + 1 = 2.55$ | **2.10；1.70** |

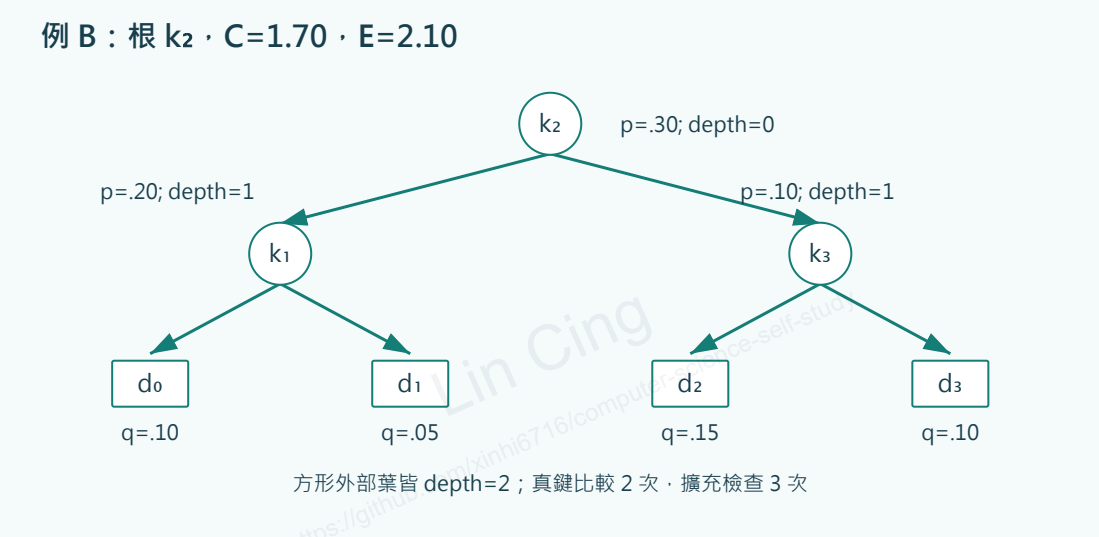
這三個區間的最佳根皆為 $r = 2$。

各區間 $C = E - \sum_{t=i-1}^{j} q_t$；扣除的是該區間的間隙總質量。全表共10個根候選，不能只算最後3個。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 例 B：重建後直接驗算期望

根為 $k_2$，左子 $k_1$，右子 $k_3$。真鍵深度分別1、0、1；四個外部葉深度皆2。






## 例 B：逐項核對查詢成本

真鍵比較次數依序為2、1、2，成功查詢的加權成本為 $.20 \cdot 2 + .30 \cdot 1 + .10 \cdot 2 = .90$。

四個失敗間隙的總機率為 $.10 + .05 + .15 + .10 = .40$。每次失敗查詢比較2個真鍵；若將外部葉的終止檢查也計入，則成本為3。

$C = .90 + (.10 + .05 + .15 + .10) \cdot 2 = 1.70$；$E = .90 + .40 \cdot 3 = 2.10$。兩值差.40；這是每次全域查詢的期望，不是「成功條件下」期望。

## OBST 填表、重建與精確成本

```
for i=1..n+1: E[i,i-1]=w[i,i-1]=q[i-1]
for L=1..n:
  for i=1..n-L+1:
    j=i+L-1; w[i,j]=w[i,j-1]+p[j]+q[j]
    E[i,j]=infinity
    for r=i..j:
      t=E[i,r-1]+E[r+1,j]+w[i,j]
      if t<E[i,j]: E[i,j]=t; root[i,j]=r
```

候選數 $\sum_{L=1}^{n}(n-L+1)L=\binom{n+2}{3}=n(n+1)(n+2)/6$，因此基本演算法 $\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。`build(i,j)` 若空則回傳 $d_{i-1}$，否則以 `root[i,j]` 接左右子樹；輸出 $2n+1$ 節點，$\Theta(n)$ 時間。

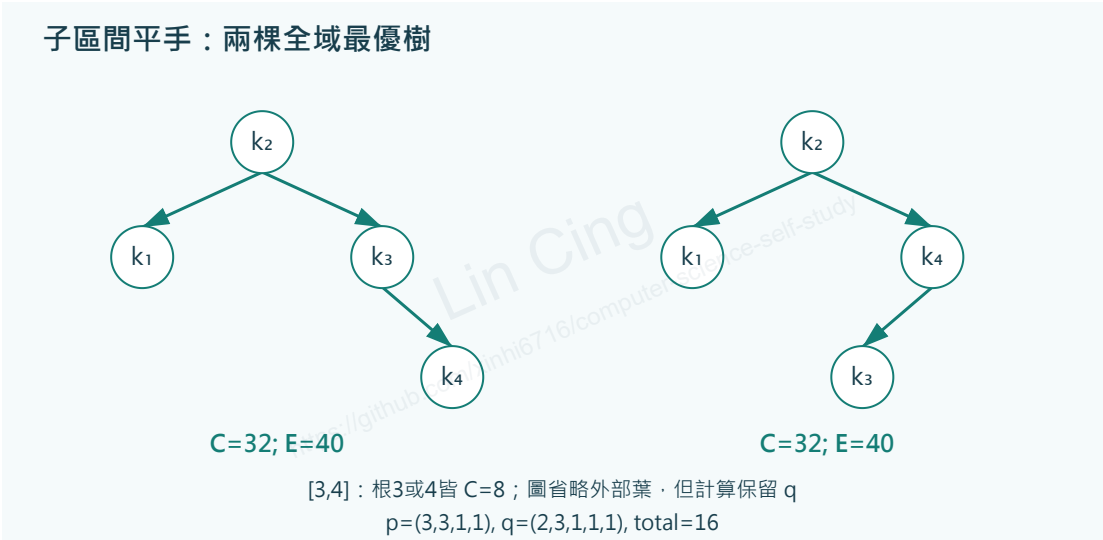
Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 整數例 C：相同最小成本可以有兩棵樹

取次數權重 $p=(3,3,1,1)$、$q=(2,3,1,1,1)$，全域總權重16。

區間 $[3,4]$ 的 $w=5$；$C$ 候選根3為 $0+3+5=8$，根4為 $3+0+5=8$，兩者皆最優。全域根2的候選為 $C[1,1]+C[3,4]+16=8+8+16=32$；其他根1、3、4的候選分別35、38、41。




全域 $C=32$、$E=40$；除以16得期望2、2.5。只存第一個最小根可重建一棵；要求所有最優樹時須存所有平手根，輸出數量可能指數成長。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 僅成功查詢與完整模型如何銜接

| 模型 | 權重與邊界 | 最佳化目標 |
| --- | --- | --- |
| 資料結構課第10章成功版 | 只有 $p$；空段0；$W = \sum p$ | 成功查詢加權真鍵比較數 |
| 本節完整模型 | $p, q$；$C$ 空段0，$E$ 空段 $q$；$w = \sum p + \sum q$ | 成功與失敗的全域加權成本 |

現行資料結構教材也另外列出完整 $p/q$ 模型與 $E - C$ 差；不能把「成功版」當成它唯一的慣例。原始資料結構筆記使用真鍵成本 $C$；CLRS §15.5 使用擴充成本 $E$，本節明確連接兩者。

將所有 $q$ 設0才退化為僅成功查詢的模型；直接刪去非零 $q$ 會改變最佳化問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 區間 DP 的檢查清單

矩陣鏈：端點決定輸出維度；切點必須 $i \le k < j$；最小的是完整候選總成本；重建沿記錄的切點。

OBST：鍵順序固定；間隙隨子區間保留；空段成本與 $C/E$ 慣例一致；保留全域權重；所有候選根都比較；平手策略只影響選哪棵最優樹。

矩陣鏈與 OBST 的「同為區間 DP」不代表可互換合併成本或邊界。此處分析基本立方演算法；更快的 OBST 最佳化須另外證明根單調性，不能套到任意區間 DP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. 講者 Charles E. Leiserson。 MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. 講者 Charles E. Leiserson；投影片署名 Erik D. Demaine、Charles E. Leiserson。 原始掃描筆記《演算法_wjungle》動態規劃相關頁；《資料結構》實體第 166–170 頁用於比較最佳搜尋樹模型。原始筆記涵蓋演算法與資料結構，並非單一本教科書全文。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 貪婪法

每一步選擇眼前最有利的候選，能否保證整體最優？關鍵是證明這個選擇仍能延伸成最優解。

本章依序研究 **活動選擇 → Huffman 編碼 → 擬陣與工作排程**，從具體交換論證走向可重用的結構條件。

前置知識為排序、優先佇列、樹、集合與歸納證明。學習目標是分清局部選擇、可行性與最優性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 從選擇規則走到正確性

| 每次設計都要回答 | 所需證據 |
| --- | --- |
| 什麼是合法解？ | 固定相容性、前綴碼或獨立集合的限制。 |
| 眼前選哪一項？ | 清楚定義候選、排序與平手處理。 |
| 為何不失去最優解？ | 交換、縮約或擬陣交換性質。 |
| 改了目標還成立嗎？ | 用反例檢查加權或限制改變後的規則。 |

活動選擇、Huffman 與擬陣各有自己的證明，不可只憑「看起來合理」套用同一條規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用最早結束保留後續選擇

一間教室、固定活動時間、每個活動價值相同。目標是選最多個互不衝突的活動。

貪心規則是每次選「與已選活動相容、且最早結束」的活動。正確性來自可交換的最優解，不是因為例子剛好成功。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 端點相接算相容

活動 i 使用半開區間 $[s_i,f_i)$，假設 $s_i < f_i$。

兩活動相容當且僅當 $f_i \le s_j$ 或 $f_j \le s_i$。已選活動最後在 L 結束時，候選的判斷式是 $s_i \ge L$。

[0,3) 與 [3,5) 相容；[0,3) 與 [2,5) 衝突。若場地需要清場時間 δ，條件須改成 $s_i \ge L+\delta$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

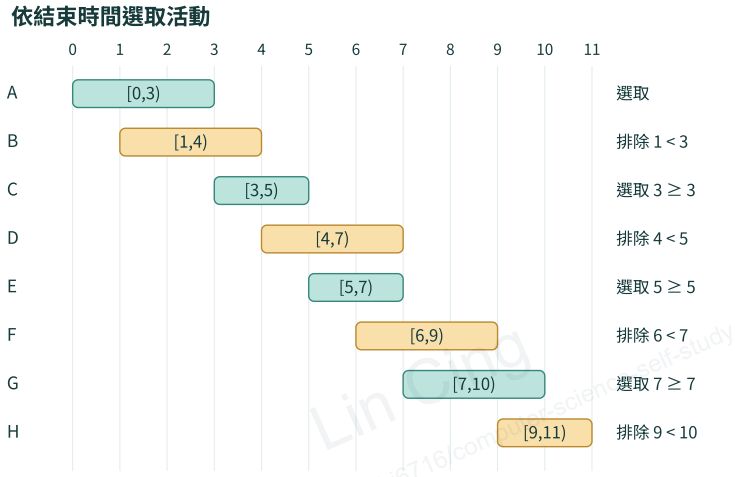
## 先按結束時間排序再掃描

```
selected = []
last_finish = −∞
for (id, start, finish) in sort_by_finish(activities):
    if start >= last_finish:
        append id to selected
        last_finish = finish
return selected
```

同結束時間可任意固定順序。本例採 ID 順序。空輸入回傳空集合；拒絕活動時不更新 last_finish。



## 八個活動的時間線




時間（小時）· 綠色選取，橙色因重疊排除

時間以小時計。輸入 A[0,3)、B[1,4)、C[3,5)、D[4,7)、E[5,7)、F[6,9)、G[7,10)、H[9,11)。綠色選取 A、C、E、G；橙色標出與當時最後活動衝突者。

## 每一步只比較最後結束時間

| 候選 | 掃描前 L | 判斷 | 動作 | 掃描後 L |
|---|---|---|---|---|
| A [0,3) | $-\infty$ | $0 \geq -\infty$ | 選 A | 3 |
| B [1,4) | 3 | $1 < 3$ | 排除 B | 3 |
| C [3,5) | 3 | $3 \geq 3$ | 選 C | 5 |
| D [4,7) | 5 | $4 < 5$ | 排除 D | 5 |
| E [5,7) | 5 | $5 \geq 5$ | 選 E | 7 |
| F [6,9) | 7 | $6 < 7$ | 排除 F | 7 |
| G [7,10) | 7 | $7 \geq 7$ | 選 G | 10 |
| H [9,11) | 10 | $9 < 10$ | 排除 H | 10 |

輸出 4 個活動。最大個數不保證答案唯一，例如 A、C、E、H 也是 4 個。



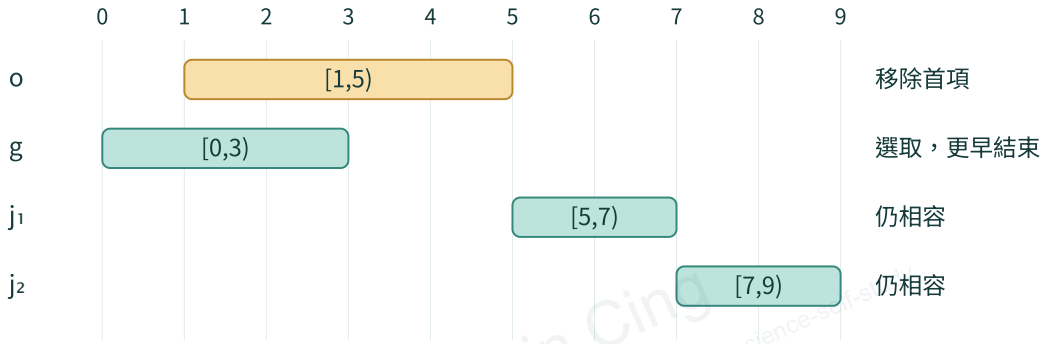
## 最早開始與最短時間都可能失敗

| 規則 | 反例區間 | 規則得到 | 更佳答案 |
|---|---|---|---|
| 最早開始 | X[0,8)、A[1,3)、B[3,5)、C[5,7) | X，1 個 | A、B、C，3 個 |
| 最短時間 | A[0,4)、X[3,5)、B[4,9) | X，1 個 | A、B，2 個 |

最短時間法在這裡指反覆從仍相容的活動中選最短者。X 長度 2，嚴格短於 A、B，但跨越兩者接點。最早開始不能用來推論最早結束也失敗。

## 交換第一個活動而不損失數量

### 以 g 替換 o，後續活動完全保留




$f_g = 3 \le f_o = 5 \le s_{j_1} = 5$

考慮任意非空剩餘問題，所有候選均在目前邊界 L 之後開始。令 g 為候選中最早結束者，O 為一個最大可行解，o 為 O 的第一個活動。因 g 最早結束，$f_g \le f_o$。



## 交換後仍可行，且仍最優

若 g=o，O 已包含貪心選擇。否則令 $O' = (O - \{o\}) \cup \{g\}$。

1. g 的開始時間 $\ge$ L，所以不衝突於已固定的前綴。
2. O 中其餘活動 j 都在 o 之後，故 $s_j \ge f_o \ge f_g$，換成 g 後仍相容。
3. g 不會原本就在 O 的後段，因正長度與相容性會推出 $f_g > f_o$，與 $f_g \le f_o$ 矛盾。
4. 所以 $|O'|=|O|$。$O'$ 仍最優，且以 g 開頭。

結論是「存在一個包含 g 的最優解」，不是「每個最優解都包含 g」。

## 剩餘問題與歸納完成整體證明

選 g 後只留下 R={i ∣ $s_i \geq f_g$}。沒有活動能排在 g 前面，因所有候選的結束時間 $\geq f_g > s_g$。

若包含 g 的最優解，其後綴不是 R 的最優解，就能用更大的後綴替換而得到更大的原解，矛盾。因此 OPT(S)=1+OPT(R)。

空集合是歸納基底。交換論證保證第一步安全，對更小的 R 重複同一論證，得到整個貪心解最優。單有最優子結構不足以證明貪心選擇安全。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 時間成本與證明適用範圍

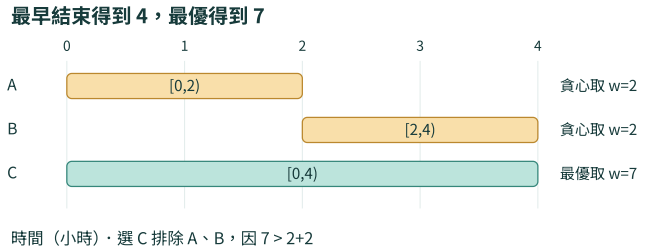
排序 O(n log n)，掃描 Θ(n)，總時間 O(n log n)。若輸入已按結束時間排序，則總時間 Θ(n)。

除排序與輸出外，掃描只需 O(1) 額外空間；輸出至多 n 個 ID。

本證明假設單一資源、固定正長度區間、最大化個數。由右往左每次選最晚開始的相容活動，也可作對稱交換證明；不能把它誤寫成由左往右最早開始。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 加入權重後交換不再保值




A[0,2) 權重 2，B[2,4) 權重 2，C[0,4) 權重 7。最早結束選 A、B，總值 4；最優選 C，總值 7。

交換保住「個數」卻不能保住「總值」。這推翻原規則於加權版本的正確性，不代表已證明所有可能的貪心規則均失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 加權區間用取或不取的動態規劃

依結束時間排序，p(j)=最大的 i<j 且 $f_i \le s_j$；不存在則 p(j)=0。令 M(j) 為前 j 個活動的最大總權重。

M(0)=0，M(j)=max(M(j−1), $w_j$+M(p(j)))。

不取 j 時答案在前 j−1 個中；取 j 時，其前綴只能來自前 p(j) 個。兩種情況涵蓋所有最優解。

排序 A、B、C，p=(0,1,0)，M=(0,2,4,7)。回溯比較兩支可還原 {C}。二分搜尋求 p，總時間 O(n log n)，DP 與回溯 O(n)，空間 O(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把頻率轉成最短的總位元數

Huffman 解有限字母表的二元、逐符號、無前綴碼最佳化。輸入是非負權重 $w_c$，輸出是每個符號的碼字。主要證明先設 $n \geq 2$ 且權重正值，最後擴充邊界。

| 符號 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| 頻率，千次 | 45 | 13 | 12 | 16 | 9 | 5 |

六種符號的定長碼需每次 3 bits。本例 Huffman 平均 2.24 bits，共 224 千 bits，未計碼表與框架。

原掃描第20頁只把 Huffman 列為貪心例子。本節補上 CLRS 的構造與證明；MIT 兩年度講義提供貪心選擇與最優子結構框架。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 無前綴性讓串接可即時解碼

無前綴 prefix-free 意指任一碼字都不是另一個碼字的前綴。二元樹中只有葉子代表符號，左邊標 0，右邊標1。

例如 $\{0, 10, 110, 111\}$ 無前綴；$\{0, 01\}$ 不成立。讀到葉子就輸出符號並回到根，不需要分隔符。

互異碼字仍可能無法唯一解碼，例如 $\{0, 01, 10\}$ 中，位元串010可分成0|10或01|0。無前綴性是充分條件，但不是唯一可解碼的必要條件。

這裡的最佳化範圍是逐符號無前綴碼，不宣稱可勝過使用符號間相依性的區塊壓縮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Kraft 不等式限制的是碼長

對有限二元無前綴碼，碼長 $\ell_i$ 是非負整數，必有

$$\sum_i 2^{-\ell_i} \le 1.$$

深度 $\ell$ 的節點對應 $[0,1)$ 中長度 $2^{-\ell}$ 的二進位區間。無前綴使這些區間互不重疊，因此總長不超過 $1$。

反向也成立。將碼長由短到長排列，依序取尚未使用的最左 dyadic 區間。先前較短區間占滿深度 $\ell$ 的整數格；總長限制保證仍有一格可取，得到所需無前綴碼。

例如碼長1,2,3,3的和為1；1,1,2的和為1.25，故不可能。$n \ge 2$ 時碼長至少1；空碼長0只容許單一符號。這個判準不能直接用在任意僅碼字互異的碼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 加權外部路徑長就是成本

對樹 $T$，葉深度 $d_T(c)$ 等於碼長。目標是

$$B(T) = \sum_c w_c d_T(c).$$

若 $w_c$ 為出現次數，$B$ 的單位是 bits；若為機率，$B$ 為 bits/符號。正權重下，單子女內部節點可刪除，受影響葉子深度下降且成本嚴格下降，所以最優樹必為滿二元樹。

每個內部節點的權重是其葉子權重總和。每個葉權重被其所有祖先各算一次，因此

$$B(T) = \sum_{v \text{ internal}} w(v) = \text{所有合併權重之和}.$$

這是兩種獨立核算成本的方法；根權重則只有 $\sum_c w_c$，不等於總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 每輪取出兩棵最輕的樹

佇列元素是整棵子樹，key為子樹權重。每次合併都把兩個元素變成一個，故恰好 $n-1$ 輪。

```
Q ← min-priority queue of symbol leaves
while |Q| > 1:
    x ← extract-min(Q)
    y ← extract-min(Q)
    z ← Node(weight=x.weight+y.weight, left=x, right=y)
    insert(Q,z)
return extract-min(Q)
```

不變量是佇列中的子樹葉集合互斥且聯集為原字母表，所有佇列權重之和保持100。新節點必須放回佇列，不能只掃原始頻率。



## 五輪佇列，逐輪可追查

表中佇列僅為顯示而排序，heap內部不必完全排序。$z_i$ 是第 $i$ 輪新節點。

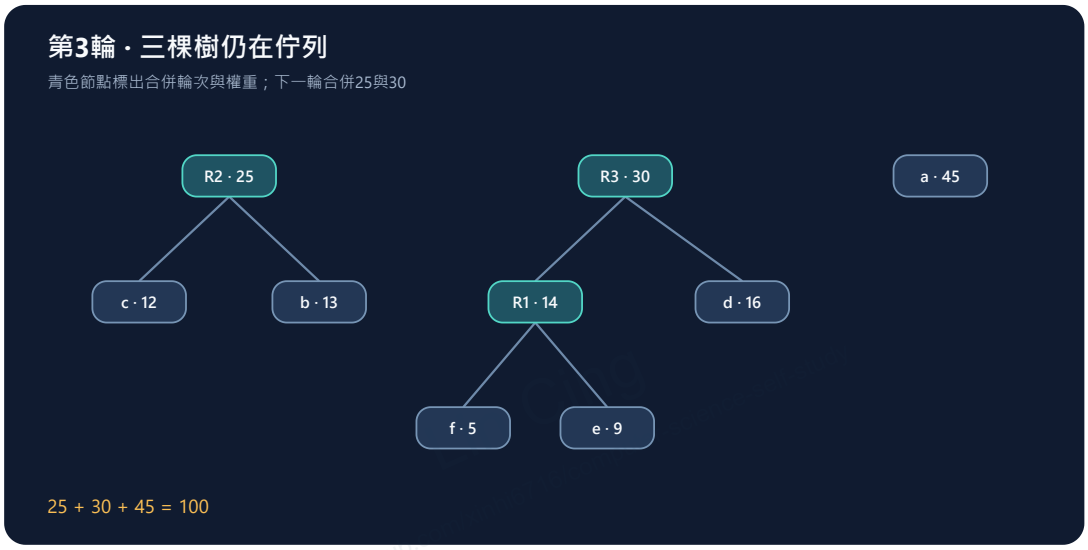
| 輪 | 取出 | 插入 | 合併後佇列，依權重排序 |
|---|---|---|---|
| 0 | | | f5,e9,c12,b13,d16,a45 |
| 1 | f5,e9 | $z_1$14 | c12,b13,$z_1$14,d16,a45 |
| 2 | c12,b13 | $z_2$25 | $z_1$14,d16,$z_2$25,a45 |
| 3 | $z_1$14,d16 | $z_3$30 | $z_2$25,$z_3$30,a45 |
| 4 | $z_2$25,$z_3$30 | $z_4$55 | a45,$z_4$55 |
| 5 | a45,$z_4$55 | $z_5$100 | $z_5$100 |

# 五輪佇列，逐輪可追查 （續）

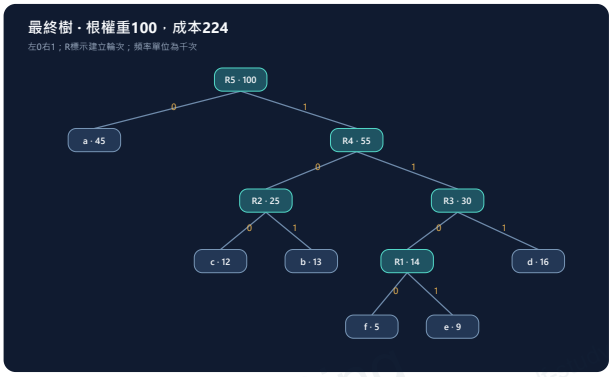
第三輪合併後，佇列只有三棵樹，權重25、30、45。下一輪要合併25與30，不是將新節點30直接接到45。



# 中途的森林也是演算法狀態



## 由根讀碼，再用兩種方式核算




| 符號 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| 碼字 | 0 | 101 | 100 | 111 | 1101 | 1100 |
| 深度 | 1 | 3 | 3 | 3 | 4 | 4 |

合併權重14、25、30是之後成本的已定部分。子樹往上一層時，其每個葉子都增加1 bit，新增成本剛好是該子樹權重。

$B = 45 + 39 + 36 + 48 + 36 + 20 = 224$ 千 bits。

合併核算為 $14 + 25 + 30 + 55 + 100 = 224$。根權重100千次。Kraft和 $1/2 + 3/8 + 2/16 = 1$。例如abc編成0101100。



## 交換論證先證明有最優樹接受貪心選擇

設 $x, y$ 是兩個最小權重。取一棵最優滿二元樹，選最大深度 $D$ 的兄弟葉位置 $a, b$。最大深度葉的兄弟必也是葉，否則會有更深的葉。

先把 $x$ 與 $a$ 交換。若 $x$ 本來在 $b$，先交換這對兄弟位置，這不改成本。交換成本差為

$$B(T_{new}) - B(T) = (w_a - w_x)(d_T(x) - D) \le 0.$$

接著把 $y$ 與位置 $b$ 的葉交換。$x$ 已固定在 $a$，剩下的 $y$ 是未固定葉中的最小權重，因此相同不等式成立。若符號已在目標位置就略過交換。

兩次都不增成本，原樹已最優，故新樹仍最優。結論是「存在」最優樹使$x, y$為最深兄弟，不是所有最優樹都必須長成此形。

## 收縮與歸納把局部選擇接到全域最優

把兄弟 $x, y$ 收縮成葉 $z$，令 $w_z = w_x + w_y$。其他深度不變，兩葉各少走一條邊。

$$B(T) = B(T') + w_x + w_y.$$

若最優樹收縮後的 $T'$ 不最優，用較便宜的縮小樹替代，再展開 $z$，增加的仍是固定 $w_x + w_y$，就會比原最優樹更便宜，矛盾。

歸納以一個葉子成本0為基底。交換引理保證某個最優解含本輪最小兄弟；歸納假設保證 Huffman 解好 $n - 1$ 個符號的收縮問題；展開恢復固定成本，所以得到原問題最優解。

局部「本次合併最便宜」本身不構成證明。真正的保證是交換不增成本與收縮後保留最優子結構。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同權重、單一符號與零權重

同權重可任意打破平手。左/右互換也不改碼長成本；要重現同一碼表，可用固定的建立序號當第二key。

單一符號的樹模型用空碼 $\epsilon$，成本0。實作若強制非空碼可指定0，成本 $w$；這是不同的碼長限制。空碼解碼需要外部給定符號個數。空字母表則直接回傳空結果。

零權重若仍在字母表內，必須分配碼字。Huffman與不增成本證明仍適用，但刪單子女可能只是不增成本，所以不能宣稱每棵最優樹都滿。例如a權重1、b權重0，用00與01成本2，比0與1成本1差；全零時任何合法碼成本皆0。

權重必須非負且有限。若總權重0，不能除以總權重計算平均碼長。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 複雜度與可驗證的交付

二元 min-heap 建立需 $O(n)$。每輪兩次extract-min與一次insert各 $O(\log n)$，共 $n-1$ 輪，所以建樹時間 $O(n \log n)$、節點空間 $O(n)$，共有 $2n-1$ 個節點。

若權重預先排序，可用原葉佇列與合併節點佇列，在兩個佇首中取最小值，建樹 $O(n)$。排序成本另計。

這些是建樹成本。若直接儲存所有碼字字串，輸出時間與空間還需 $\Theta(\sum_c \ell_c)$，最壞可達 $\Theta(n^2)$；編碼訊息還需處理實際輸出的位元。

逐一檢查碼字互非前綴，並計算葉深成本 45+39+36+48+36+20=224 與合併成本 14+25+30+55+100=224。最優性由交換與縮約證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 何時「選最重」能保證最佳？

活動選擇有專用的最早結束規則。Matroid 則用集合結構保證任意非負權重的排序貪心。

本單元把「可行解」寫成獨立集合，逐步證明交換，再把單位作業的逾期罰款改寫為保住的權重。

閱讀主線為 CLRS §16.4–16.5。無向圖森林只用來追蹤集合操作，MST 演算法與 cut property 留第 8 章。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 有限 ground set 與兩個公理

令 M=(E,$\mathcal{I}$)，E 是有限 ground set，$\mathcal{I}$⊆2^E，且 ∅∈$\mathcal{I}$。

- 遺傳性。A∈$\mathcal{I}$ 且 B⊆A，則 B∈$\mathcal{I}$。
- 交換性。A,B∈$\mathcal{I}$ 且 |A|<|B|，則存在 x∈B\A，使 A∪{x}∈$\mathcal{I}$。

滿足這些條件才是 matroid。只有遺傳性的結構稱獨立系統。交換公理量化的是每一對獨立集合，不是只檢查一組成功例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 極大與最大不能混用

極大獨立集合無法再加入元素。最大獨立集合有最多元素。

在 matroid 中，若極大集合 A 比獨立集合 B 小，交換性立即給 A 一個可加入元素，矛盾。因此所有極大獨立集合等大，稱為基底，其大小為 rank r。

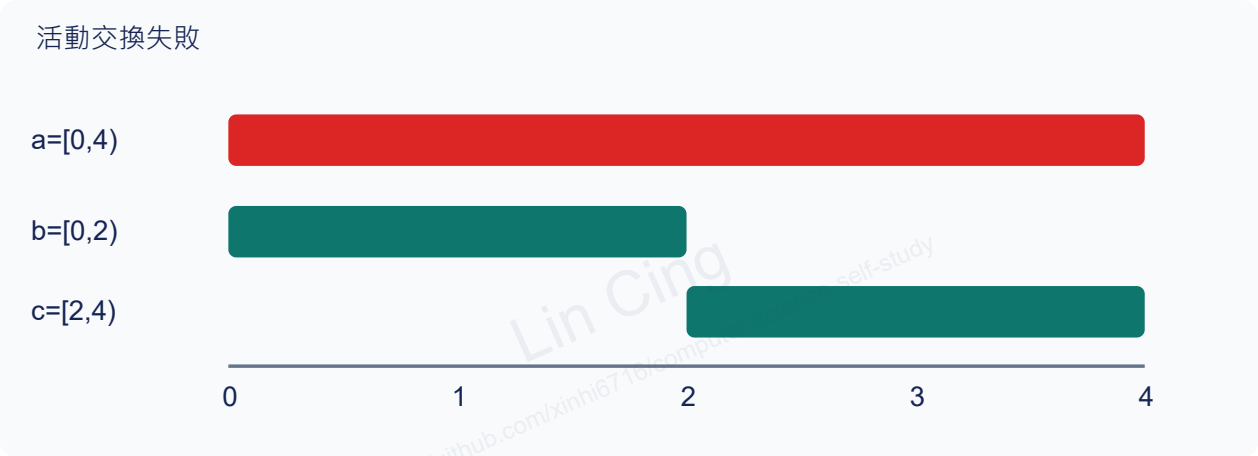
Uniform matroid $U_{2,3}$ 有 E={a,b,c}，$\mathcal{I}$={X⊆E:|X|≤2}。三個二元素集合都是基底，單元素集合都不是極大。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 活動選擇的獨立系統不是 matroid

採半開區間。a=[0,4)、b=[0,2)、c=[2,4)。獨立集合是兩兩不衝突的活動集合。

A={a} 與 B={b,c} 都獨立，|A|=1<2=|B|。但 A∪{b} 和 A∪{c} 都衝突，交換性失敗。




遺傳性仍成立。最早結束的專用證明不表示活動獨立系統是 matroid。



## 加權貪心與負權重的界線

目標為 max $\Sigma_{x\in A} w(x)$，條件 $A\in\mathcal{I}$。先按 w 遞減排列，A←∅。依序掃描 x，若 $A\cup\{x\}\in\mathcal{I}$ 就加入。

當 w≥0，這個版本求得最大權重獨立集合，零權重可加入，輸出一個基底。排序 O(n log n)，n 次獨立性查詢，每次 T_ind，總計 O(n log n+nT_ind)。

允許負權重且可自由選子集時，跳過所有 w(x)<0。例 $U_{2,2}$，權重 5,−7，強制加滿得 −2，而只取第一個得 5。若題目要求基底，則不能跳過負權重，原排序規則仍求最大權重基底。

## 交換證明 1｜先準備一個最佳基底

先考慮非負權重。任一最佳獨立集合可擴成基底而不減權重，因此存在最佳基底 O。

維持不變量「某個最佳基底 O 包含目前貪心集合 G」。初始 G=∅ 成立。下一個接受元素為 g。若 g∈O，不需改 O。

若 g∉O，先令 $I_0=G\cup\{g\}$，再設 H=$I_0$。因 |H|<|O| 時可用交換性，反覆從 O\H 加入元素直到 |H|=|O|=r。每步都保留獨立性與 G∪{g}。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



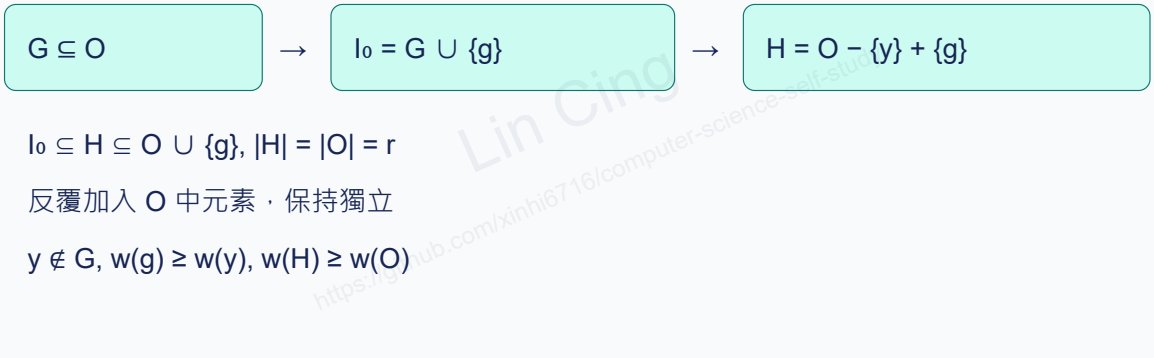
## 交換證明 2｜替換一項且不減權重

完成後 H⊆O∪{g}、|H|=|O| 且 G∪{g}⊆H，故 H=(O{y})∪{g}，其中 y∈O\G。

因 G∪{y}⊆O，y 在目前 G 下可行。y 不可能已在更早被拒絕，否則當時的較小前綴加 y 不獨立，與遺傳性矛盾。因此排序保證 w(g)≥w(y)。

w(H)=w(O)−w(y)+w(g)≥w(O)。O 已最佳，故 H 也是最佳，並包含新的 G。

以一次交換保留貪心前綴




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

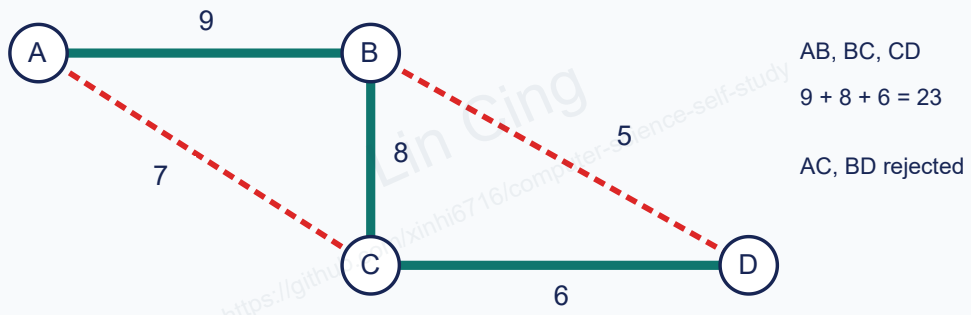
掃描結束時 G 是基底，包含它的最佳基底只能等於 G。負權重的自由子集版在非負元素的限制 matroid 上套用同一證明。

## Graphic matroid｜邊集合的可行性是無環

給無向圖 V={A,B,C,D}，E={AB,BC,AC,CD,BD}。$\mathcal{I}$ 是不含無向環的邊集合。

刪除森林的邊仍是森林。若森林 F 比森林 H 邊少，H 必有一條邊跨越 F 的兩個連通分量，否則 H 在每個 F 分量內最多只能有 $|V_i|-1$ 條邊，合計不超過 $|F|$。跨分量邊可加入 F 而不成環，故有交換性。

最大權重森林的接受與拒絕




本例是最大權重森林，不是最小生成樹的逐步教學。



## Graphic matroid｜追蹤每次接受與拒絕

沿用上圖，權重 AB=9、BC=8、AC=7、CD=6、BD=5。

| 掃描邊 | 判定 | 掃描後的 G | 總權重 |
|---|---|---|---|
| AB | 接受，連接 A 與 B | {AB} | 9 |
| BC | 接受，連接 C | {AB,BC} | 17 |
| AC | 拒絕，形成 A–B–C–A 環 | {AB,BC} | 17 |
| CD | 接受，連接 D | {AB,BC,CD} | 23 |
| BD | 拒絕，形成 B–C–D–B 環 | {AB,BC,CD} | 23 |

三條接受邊形成基底，權重 23。此處的查詢只問加邊後是否仍為森林。

## 單位作業｜最小罰款等於最大保住權重

所有作業在時間 0 可用，單一處理器，每件恰需 1 單位時間，不搶先。deadline d_i 為 1 到 n 的整數。完成時間 C_i≤d_i 免罰，否則支付非負 penalty p_i。每件作業最終都要執行。

獨立集合 A 是「能全部準時完成的作業子集」。令 N_t(A)=|{i∈A:d_i≤t}|。

min Σ_late p_i = Σ_all p_i − max_(A∈ℐ) Σ_i∈A p_i。

| 作業 | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ |
|---|---|---|---|---|---|---|---|
| d_i | 4 | 2 | 4 | 3 | 1 | 4 | 6 |
| p_i | 70 | 60 | 50 | 40 | 30 | 20 | 10 |

數據採 CLRS 圖 16.7，總 penalty 為 280。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 可行性判準與排程方法

A∈ℐ ⇔ 對每個整數 t∈[0,n]，N_t(A)≤t ⇔ A 按 deadline 遞增排序後全部準時。

必要性。若 N_t>t，必須在前 t 格完成超過 t 件單位作業，不可能。

充分性。把 deadline 排成 d_(1)≤⋯≤d_(k)。若第 j 件逾期，即 d_(j)<j，則 N_{d_(j)} ≥j>d_(j)，矛盾。故第 j 件在時間 j 完成且 j≤d_(j)。

遺傳性由 N_t(B)≤N_t(A) 得到。可行性與 penalty 無關，penalty 只決定貪心掃描順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 作業交換性｜每一個時間前綴都要成立

設 A,B 可行且 $|A|<|B|$。取 $k=\max\{t:N_t(B)\le N_t(A)\}$。因 $N_0(A)=N_0(B)=0$，$k$ 存在。因 $N_n(B)>N_n(A)$，$k<n$。

在 $k$ 處差值 $\le 0$，在 $k+1$ 處差值 $>0$，故 B 比 A 有更多 deadline 恰為 $k+1$ 的作業。可選 $x\in B\setminus A$ 且 $d_x=k+1$。

- $t\le k$ 時，$N_t(A\cup\{x\})=N_t(A)\le t$。
- $t\ge k+1$ 時，整數差值給 $N_t(A)+1\le N_t(B)\le t$。

所以 $A\cup\{x\}$ 可行。連同遺傳性與空集合可行，得到 deadline matroid。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完整選取表｜拒絕時給出違反前綴

依 penalty 遞減掃描，N 向量依序列出 $t=1,\dots,7$，接受者才更新 A。

| 考慮 | 候選 $N_1,\dots,N_7$ | 決定 | 保住 penalty |
|---|---|---|---|
| $a_1$ | (0,0,0,1,1,1,1) | 接受 | 70 |
| $a_2$ | (0,1,1,2,2,2,2) | 接受 | 130 |
| $a_3$ | (0,1,1,3,3,3,3) | 接受 | 180 |
| $a_4$ | (0,1,2,4,4,4,4) | 接受 | 220 |
| $a_5$ | (1,2,3,5,5,5,5) | 拒絕，$N_4=5>4$ | 220 |
| $a_6$ | (0,1,2,5,5,5,5) | 拒絕，$N_4=5>4$ | 220 |
| $a_7$ | (0,1,2,4,4,5,5) | 接受 | 230 |

準時集合為 $\{a_1,a_2,a_3,a_4,a_7\}$，最小總罰款 $280-230=50$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 完整排程表｜選取順序不是執行順序

把接受的作業按 deadline 遞增排，再接上拒絕作業。相同 deadline 任意破同分。

| 執行格 | 作業 | 完成 C_i | d_i | 狀態 | 罰款 |
|---|---|---|---|---|---|
| [0,1) | $a_2$ | 1 | 2 | 準時 | 0 |
| [1,2) | $a_4$ | 2 | 3 | 準時 | 0 |
| [2,3) | $a_1$ | 3 | 4 | 準時 | 0 |
| [3,4) | $a_3$ | 4 | 4 | 準時 | 0 |
| [4,5) | $a_7$ | 5 | 6 | 準時 | 0 |
| [5,6) | $a_5$ | 6 | 1 | 逾期 | 30 |
| [6,7) | $a_6$ | 7 | 4 | 逾期 | 20 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完整排程表｜選取順序不是執行順序 （續）

先排準時集合，再排逾期作業

Σ saved = 230; Σ late penalty = 30 + 20 = 50

若改成不同處理時間、release time 或多處理器，須重新檢查可行集合的公理。本證明只涵蓋上述單位作業模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 16, Greedy Algorithms. 原始手寫掃描筆記《演算法_wjungle》貪婪法相關頁，涵蓋活動選擇、Huffman、擬陣與排程。 MIT 6.046J, Fall 2004, Lecture 13, Greedy Algorithms and Minimum Spanning Trees；Fall 2005, Lecture 16。兩份 MIT 講義用於貪婪法與 MST 證明框架，不作為 Huffman 原文來源。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 圖走訪與連通性

從 BFS 的距離層、DFS 的時間區間，走向拓樸排序、強連通分量與單點或單邊故障分析。

本章依序研究 **BFS → DFS → 拓樸排序 → SCC → low、割點、橋與雙連通區塊**。

前置知識為佇列、堆疊、遞迴、集合與歸納證明。每次走訪都要先固定圖模型與鄰接順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 先問輸出，再選擇走訪資訊

| 問題 | 保留的資訊 | 證明核心 |
| --- | --- | --- |
| 最少幾條邊到達？ | BFS 距離與 parent | FIFO 層次不變量 |
| 何時進入與完成？ | DFS 顏色、森林與時間 | 括號與 white-path 定理 |
| 哪些工作可先做？ | DAG 的完成逆序或剩餘入度 | 每條邊遵守順序 |
| 哪些頂點互相可達？ | SCC 與縮點圖 | 完成時間與分量邊界 |
| 移除頂點或邊會斷開嗎？ | 無向 low 與邊堆疊 | 子樹能否回到祖先 |

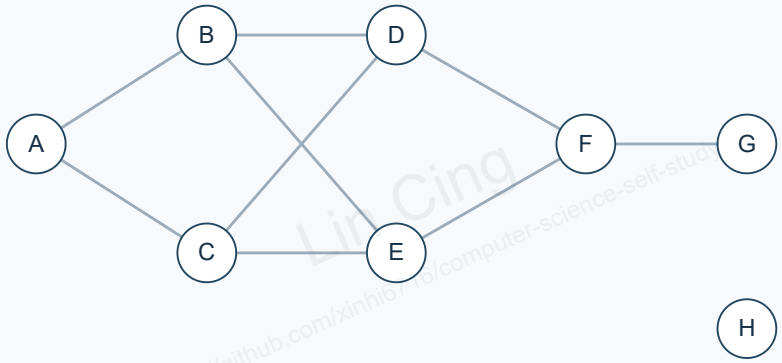
BFS 的 d 是距離；DFS 的 d 是發現時間。SCC lowlink 與無向 low 的更新規則必須分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 圖的模型先決定邊如何計數

本節固定無權、無向、無自環且無平行邊的圖。$V = \{A, B, C, D, E, F, G, H\}$，$|V| = 8$，$|E| = 9$。邊集合為 $\{AB, AC, BD, BE, CD, CE, DF, EF, FG\}$。

固定圖 G，起點 A，H 孤立




8 個頂點，9 條無向邊。鄰居按字母順序。

問題是從 A 出發，求到各點的最少邊數與一條對應路徑。H 沒有任何邊。



## 鄰接串列與矩陣記錄相同關係

串列 `Adj[u]` 列出 u 的鄰居。矩陣 $M[u, v] = 1$ 當且僅當有邊 $uv$。本例矩陣對稱、對角線為 0。

同一張圖的兩種表示

鄰接串列

A: B, C

B: A, D, E

C: A, D, E

D: B, C, F

E: B, C, F

F: D, E, G

G: F

H: ∅

| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| D | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| F | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| G | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| H | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

串列共 18 個鄰接項目。矩陣 8 × 8 = 64 格，且 $A = A^{\mathsf{T}}$。

無向每條邊存兩個鄰接項目，所以總數為 $2|E| = 18$。有向圖只將弧 $u \to v$ 存在 `Adj[u]`，總數為 $|E|$。

## 查邊與列舉鄰居需要不同成本

令 $n = |V|$，$m = |E|$，使用普通未排序串列與直接索引矩陣。

| 操作 | 鄰接串列 | 鄰接矩陣 |
|---|---|---|
| 儲存 | $\Theta(n + m)$ | $\Theta(n^2)$ |
| 查詢 $u \to v$ | 最壞 $O(1 + \deg^+(u))$ | $\Theta(1)$ |
| 列出 u 的所有鄰居 | $\Theta(1 + \deg^+(u))$ | $\Theta(n)$ |

無向圖以 $\deg(u)$ 代替出度。常數 1 包含空串列的處理。有向矩陣不必對稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 自環和平行邊必須說清楚表示規則

若改採多重圖，每條有向平行弧各占一個串列項目，自環 $u \to u$ 也占一項。無向圖採每條邊兩個端點 incidence 的慣例，自環在同一串列存兩項，所以總數仍是 $2m$。

二元矩陣只保存「是否相鄰」，會丟失平行邊數。可改存重數，但邊身分或各別權重還要額外結構。若無向自環改只存一次，總項目數是 $2m - \ell$，其中 $\ell$ 是自環數。

BFS 只在白點首次發現時入隊。自環和平行邊不使頂點重複入隊，卻會增加串列掃描次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 顏色、距離與 parent 各有一個角色

白色表示未發現，灰色表示已發現但尚未掃描完鄰居，黑色表示鄰居全部掃描完。只有 `WHITE → GRAY → BLACK`。

$d[v]$ 是從 A 到 v 的邊數估計，初值為 $\infty$。$\pi[v]$ 記首次發現 v 的點，初值為 NIL。起點設 `d[A]=0`、灰色、入隊。

在每次 while 條件測試時，Q 恰包含所有灰點。剛出隊但仍在掃描的 u 也是灰色，因此不能把這項等式宣稱為每個程式瞬間都成立。筆記 PDF p34 的 DFS $d$ 表示發現時間，本節 BFS $d$ 表示距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先標灰，再入隊，才能只發現一次

```
for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
color[A]=GRAY; d[A]=0; Q=[A]
while Q is not empty:
    u=dequeue_front(Q)
    for v in Adj[u] in alphabetical order:
        if color[v]==WHITE:
            color[v]=GRAY; d[v]=d[u]+1; π[v]=u
            enqueue_back(Q,v)
    color[u]=BLACK
```

佇列需用 O(1) 的頭尾操作。用陣列每次刪第一格並搬移其他元素，會失去這項成本假設。

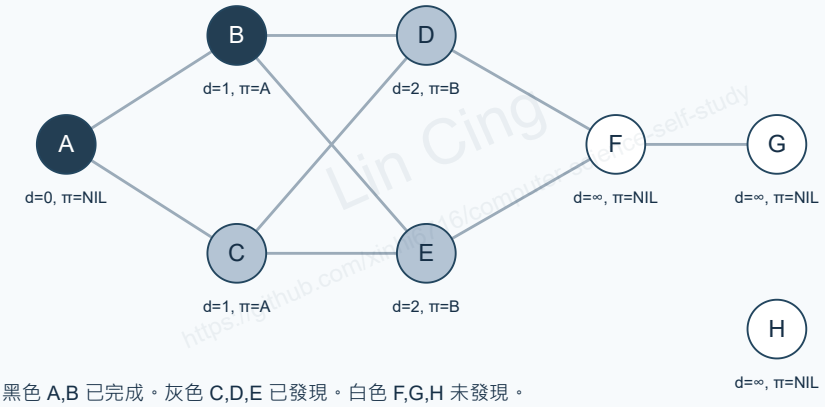
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前兩次掃描確定距離 1 與 2

初始 `Q=[A]`。掃描 A 的 B、C，兩點由白轉灰，距離均為 1、parent 均為 A。完成後 A 變黑，`Q=[B,C]`。

接著出隊 B。A 已黑，D、E 仍白，所以令 `d[D]=d[E]=2`、`π[D]=π[E]=B`。完成後 `Q=[C,D,E]`。

掃描 B 完成後，Q = [C, D, E]




黑色 A,B 已完成。灰色 C,D,E 已發現。白色 F,G,H 未發現。

C 雖已在隊內，仍要先於 D、E 出隊。



## 完整軌跡記錄每次掃描的結果

完整 FIFO 佇列軌跡

| 完成掃描 | 本次新發現 | Q 隊首 → 隊尾 | 已完成黑點 |
|---|---|---|---|
| init | A | [A] | ∅ |
| A | B,C | [B, C] | A |
| B | D,E | [C, D, E] | A,B |
| C | ∅ | [D, E] | A,B,C |
| D | F | [E, F] | A,B,C,D |
| E | ∅ | [F] | A,B,C,D,E |
| F | G | [G] | A,B,C,D,E,F |
| G | ∅ | [] | A,B,C,D,E,F,G |

每列均在 while 測試前，Q 恰好包含灰點。H 始終白色。

## 佇列軌跡的逐步解讀

掃描 C 時，D、E 已灰，不更改距離或 parent。掃描 D 首次發現 F，令 `d[F]=3`, `π[F]=D`。掃描 E 時 F 已灰。掃描 F 發現 G，令 `d[G]=4`, `π[G]=F`。

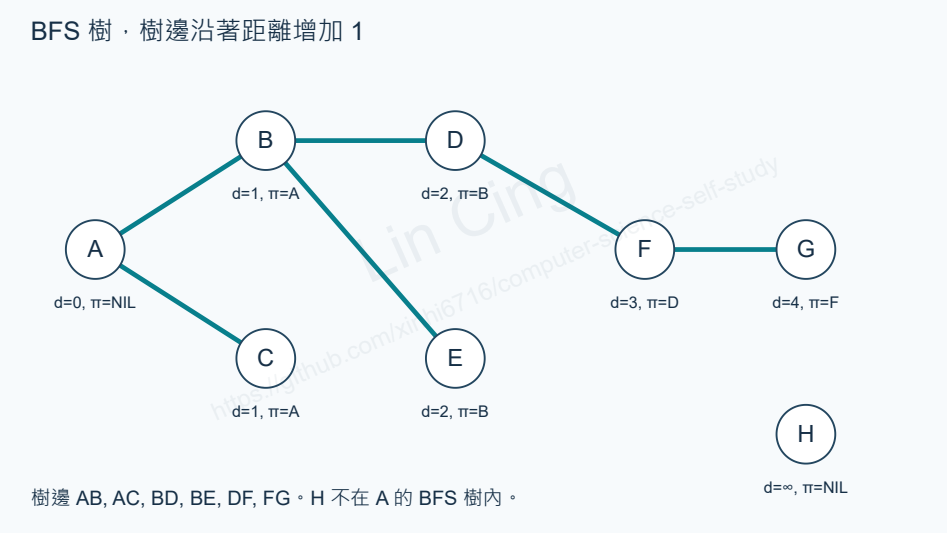
G 掃描完後 Q 為空。七個可達點各入隊一次，H 從未入隊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## parent 樹保留一條最短路徑

| v | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| d[v] | 0 | 1 | 1 | 2 | 2 | 3 | 4 | ∞ |
| π[v] | NIL | A | A | B | B | D | F | NIL |




從 G 沿 parent 回溯得到 `G,F,D,B,A`，反轉後為 `A,B,D,F,G`，共 4 條邊。`π=NIL` 也可能代表起點，須先檢查目標是否就是 A，再判斷不可達。

## FIFO 保持非遞減的距離順序

若 Q 非空，隊內距離非遞減，且隊尾距離至多比隊首多 1。Q 只會同時含相鄰兩層。

理由是出隊刪除最前面的最小值。掃描距離 k 的 u 時，新點一律標成 k+1，附在隊尾。當層尚未處理的點留在前方，下一層的新點留在後方。

例如掃描 B 後 Q 的距離是 `[1,2,2]`。掃描 D 後是 `[2,3]`。每個距離 k 的點，都會先於距離 k+1 的點被掃描。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 最短邊數需要上下兩個界

令 $\delta(A, v)$ 為圖中最少邊數。對已發現點，parent 鏈提供一條長度 $d[v]$ 的實際路徑，所以 $\delta(A, v) \le d[v]$。

再按真實距離 k 歸納。k=0 的 A 正確。若 $\delta(A, v) = k > 0$，最短路徑上前一點 u 的距離為 k−1。由歸納假設 u 正確，且會出隊掃描。

掃描 u 時，若 v 白，就設 $d[v] = k$。若 v 已被發現，FIFO 保證先前掃描的點距離至多 k−1，所以 $d[v] \le k$。結合第一個界，$d[v] = k$。因此所有可達點都會被發現，且得到最短邊數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 鄰居順序改變樹，距離不變

把各點鄰居改成反字母順序。A 先發現 C 再發現 B。C 先發現 E、D，因此 `π[D]=π[E]=C`。E 先於 D 掃描，所以 `π[F]=E`。

出隊順序變為 `A,C,B,E,D,F,G`。到 G 的樹路徑變為 `A,C,E,F,G`。距離向量仍是 `(0,1,1,2,2,3,4,∞)`。

兩種樹都通過最短路徑證明。$\delta$ 是原圖與起點決定的數值，parent 是演算法在同長路徑間的選擇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一次 BFS 不會自動走完所有分量

從 A 結束後，H 仍白、$d[H] = \infty$，因為沒有 A 到 H 的路徑。

若要列出無向圖的連通分量，先全體初始化一次。再依序檢查頂點，每遇白點就以它為根執行 BFS，寫入新的分量編號。本例分量是 `{A,B,C,D,E,F,G}` 與 `{H}`。

每棵樹內的距離應解讀為到各自根的距離。對有向圖，這個外層迴圈只產生依順序而變的可達搜尋森林，不能據此宣稱強連通分量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 表示方式決定 BFS 掃描成本

設 $V_R$ 為從起點可達的點。串列版本初始化 $\Theta(n)$，每個可達點只出隊一次，每個對應鄰接項目只掃描一次。總時間 $\Theta(n + \sum_{u \in V_R} |Adj[u]|)$，上界 $O(n+m)$。

矩陣版本每個出隊點掃一整列，時間 $\Theta(n + n|V_R|)$，最壞 $\Theta(n^2)$。兩者額外的顏色、距離、parent 與 queue 空間皆為 $O(n)$，不含圖儲存。

本例串列掃 18 個項目，矩陣掃 $7 \times 8 = 56$ 格，兩者都初始化 8 點。若連 H 一起完成全森林，矩陣掃 64 格，串列仍掃 18 項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## BFS 最佳化的是邊數

若每條邊權重皆為同一個正數 c，最少邊數路徑也最小化總權重，路徑成本為 $c\,d[v]$。不同權重時不能套用此結論。

例如有向邊 s→t 權重 10、s→x 權重 1、x→t 權重 1。BFS 選 1 條邊的直接路徑，但最小總權重路徑有 2 條邊、成本 2。

本節固定圖與五張 SVG 是原創例子。CLRS §22.1–22.2 提供表示、演算法與證明依據。MIT04 L14 與 MIT05 L17 p43–46、59 以單位權重與 FIFO 說明 BFS。筆記 p40 在 Edmonds–Karp 中使用 BFS 選最少邊擴充路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DFS 的輸出是森林與時間區間

給定圖 G=(V,E)，完整 DFS 對每個頂點恰好發現一次、完成一次。輸出 parent、發現時間 d、完成時間 f。樹邊給出可達路徑，時間區間給出祖先關係。DFS 樹路徑通常不是最短路徑。

本節先完整追蹤一張有環有向圖，再刪邊得到 DAG，用同一組時間驗證拓樸排序。

來源 · note-original; CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先固定圖模型與走訪順序

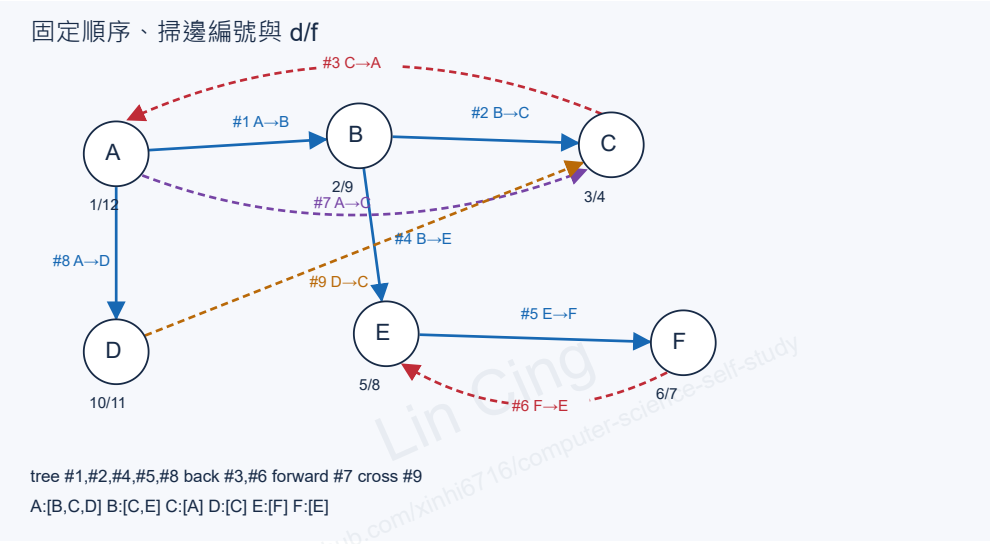
本例為有限有向圖，沒有平行邊或自迴路。完整 DFS 的外層順序 A,B,C,D,E,F。鄰接表順序固定如下，括號內的先後不可互換。

```
A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]
```

箭頭表示一條原圖邊。樹邊以實線藍色、back 以紅色、forward 以紫色、cross 以橙色顯示。分類屬於這一次 DFS，換順序可改變分類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 先固定圖模型與走訪順序 （續）




來源 • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22。



## 三色表示遞迴生命週期

WHITE 尚未被發現。GRAY 已發現但呼叫尚未返回。BLACK 已掃完所有出邊。探索邊時的來源一定是目前最深的 GRAY 頂點。所有 GRAY 頂點恰好形成呼叫堆疊的一條祖先鏈。

時間只在發現、完成時加一，不在每次掃邊時加一。每點滿足 $1 \le d[u] < f[u] \le 2|V|$，全圖共 $2|V|$ 個互異時間。

來源 • note-original; CLRS3-22 § §22.3–22.4。

## 完整 DFS 與 VISIT 的責任

```
DFS(G):
  all colors = WHITE; all parents = NIL; time = 0
  for u in fixed vertex order:
    if color[u] == WHITE: VISIT(u)
VISIT(u):
  time += 1; d[u] = time; color[u] = GRAY
  for v in Adj[u] in fixed order:
    if color[v] == WHITE:
      parent[v] = u; VISIT(v)
  color[u] = BLACK; time += 1; f[u] = time
```

VISIT(u) 只探索從當下白色 u 可延伸的白色頂點。外層迴圈補齊未訪問頂點，因此有向圖可能得到多棵樹。不同 DFS 樹不代表強連通分量；跨樹仍可能有指向先前 BLACK 頂點的邊。

來源 · note-original; CLRS3-22 §§22.3–22.4。



## 逐輪堆疊就是時間線

| t | event | stack after |
|---|---|---|
| 1 | discover A | A |
| 2 | discover B | AB |
| 3 | discover C | ABC |
| 4 | finish C | AB |
| 5 | discover E | ABE |
| 6 | discover F | ABEF |
| 7 | finish F | ABE |
| 8 | finish E | AB |
| 9 | finish B | A |
| 10 | discover D | AD |
| 11 | finish D | A |
| 12 | finish A | ∅ |

每次發現先 push，每次完成再 pop。t=4 返回 B 後，B 的鄰接掃描器才繼續處理 E。t=9 返回 A 後，A 還要掃 C、D。完成不等於整個演算法結束。

# 逐輪堆疊就是時間線 （續）

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 每條邊都要掃，遇到已訪問也不跳表

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 1 | AB | W | tree | A | 1 |
| 2 | BC | W | tree | AB | 2 |
| 3 | CA | G | back | ABC | 3 |
| 4 | BE | W | tree | AB | 4 |
| 5 | EF | W | tree | ABE | 5 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 掃描餘下邊並比較分類

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 6 | FE | G | back | ABEF | 6 |
| 7 | AC | B | forward | A | 9 |
| 8 | AD | W | tree | A | 9 |
| 9 | DC | B | cross | AD | 10 |

clock 是掃邊當刻已發生的最後一個時間事件。target 的 W/G/B 是掃邊當刻的顏色。A→C 在 C 完成後才掃到，因此是 forward；D→C 也到 BLACK，但 C 不是 D 的後代，因此是 cross。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



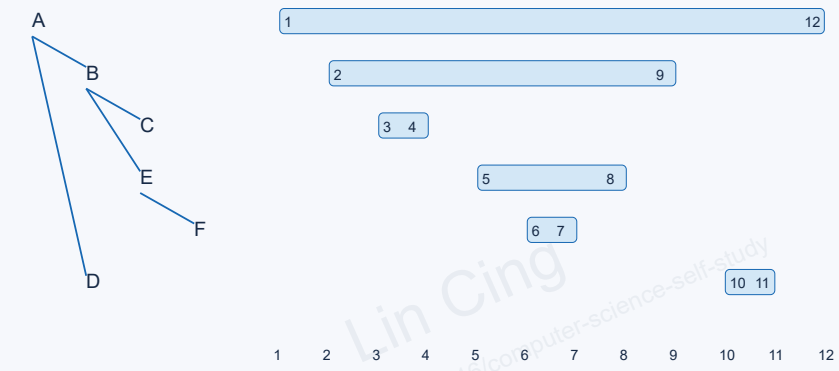
## 最後時間與森林對齊

| vertex | d | f | parent |
|---|---|---|---|
| A | 1 | 12 | NIL |
| B | 2 | 9 | A |
| C | 3 | 4 | B |
| D | 10 | 11 | A |
| E | 5 | 8 | B |
| F | 6 | 7 | E |

樹邊為 A→B、B→C、B→E、E→F、A→D。發現順序 A B C E F D；完成順序 C F E B D A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 最後時間與森林對齊 （續）




(A (B (C C) (E (F F) E) B) (D D) A)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 括號定理排除交錯區間

對相異頂點 u,v，區間 [d[u],f[u]] 與 [d[v],f[v]] 只能互斥或嚴格包含。包含當且僅當內層頂點是外層頂點的真後代。

證明。假設 d[u]<d[v]。若 f[u]<d[v]，u 已返回，兩區間互斥。否則 v 在 u 尚為 GRAY 時發現，v 的呼叫嵌套於 u 內。遞迴必須先完成 v 才能返回並完成 u，因此 d[u]<d[v]<f[v]<f[u]。交換 u,v 處理另一種發現順序。

本例括號為 `(A (B (C C) (E (F F) E) B) (D D) A)`。`d[u]<d[v]<f[u]<f[v]` 不可能。

來源 • CLRS3-22 § § 22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## white-path 定理的時間前提

在 u 剛被發現的瞬間，v 成為 u 的後代，當且僅當存在 u→⋯→v 的路徑，除已轉 GRAY 的起點 u 外，其餘頂點此刻全是 WHITE。若取染色前瞬間，整條路徑均為 WHITE。此處包含 u 自己作零長路徑。

必要性。DFS 樹的 u 到 v 路徑上每個真後代都比 u 晚發現，故此刻為白。充分性。若白路徑存在，取路徑上第一個不是 u 後代的 x，其前驅 w 是 u 或 u 後代。掃 w→x 時，x 若白就立刻遞迴；若非白，它必在 u 發現後、w 完成前被發現。兩者都讓 x 的發現落在 u 活躍期內，矛盾。

只有「原圖有路徑」不夠。C→A→D 可達，但在 d[C]=3 時 A 是 GRAY，D 不會成為 C 後代。

來源 · CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有向圖的四種邊

| 類型 | 掃邊時判別 | 完成後的關係 |
| --- | --- | --- |
| tree | 目標 WHITE | parent[v]=u |
| back | 目標 GRAY | v 是 u 的祖先，含自迴路 |
| forward | 目標 BLACK 且 d[u]<d[v] | 真後代但非樹邊 |
| cross | 目標 BLACK 且 d[v]<d[u] | 無祖先關係 |

對相異頂點，tree/forward 均滿足 d[u]<d[v]<f[v]<f[u]，因此只看時間不能分辨它們，還需要 parent。back 滿足 d[v]<d[u]<f[u]<f[v]。cross 滿足 d[v]<f[v]<d[u]<f[u]。

來源 · note-original; CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 無向邊只有 tree 或 back

無向簡單圖中，一條實體邊在兩端鄰接表各出現一次，必須按第一次遇到的方向分類。已用來發現子節點的那條 parent 邊，反向掃描時不是另一條 back edge。

證明。令 d[u]<d[v] 且 {u,v} 為邊。u 完成前必掃到 v，因此 v 是 u 後代。若第一次從 u 掃向 v，v 尚白，為 tree；若第一次由 v 掃向 u，u 尚灰，為 back。不會有跨子樹邊。

無向環檢測忽略同一條 parent 邊的反向項。多重圖要用 edge ID，平行的另一條邊仍可形成長度 2 的環。自迴路形成長度 1 的環；定理的環定義須與圖模型一致。

來源 · note-original; CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有向圖有環當且僅當有 back edge

本命題使用有限有向圖，允許自迴路，環是沿箭頭可走回起點且內部點互異的非空路徑。必須對全圖跑完整 DFS。

若存在 u→v back edge，樹中 v 到 u 的有向路徑加上 u→v 成環，自迴路本身就是環。反方向取某個環最早發現的頂點 v，以及環上指向 v 的前驅 u。發現 v 時其餘環頂點皆白，沿環存在白路徑 v→⋯→u。white-path 定理使 u 成為 v 後代，所以 u→v 是 back edge。

本例 C→A 證明 A→B→C→A 成環，F→E 證明 E→F→E 成環。只看到 BLACK 不能推論成環。

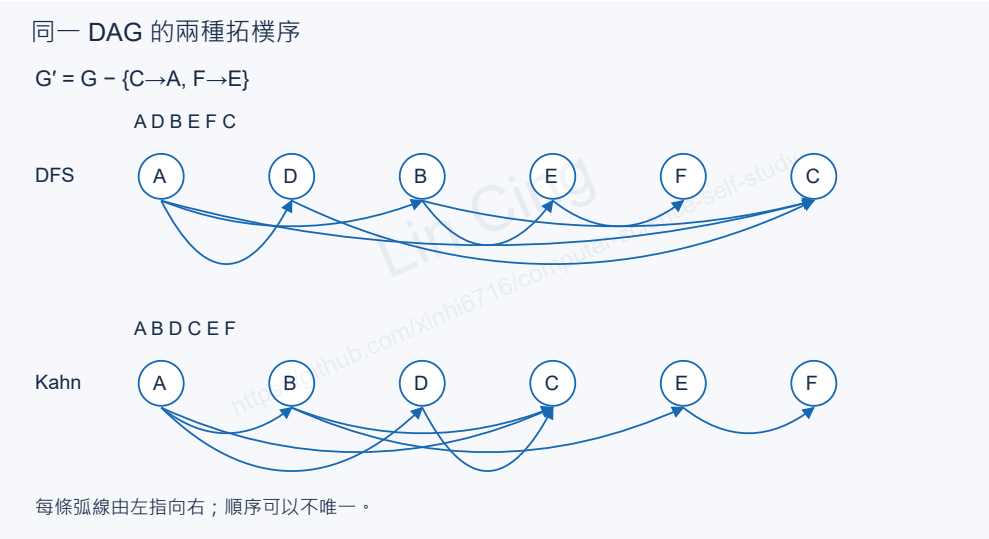
來源 · note-original; CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 刪除兩條 back edge 得到 DAG

令 G′=G−{C→A,F→E}。鄰接順序不變，所有 d/f 與樹邊也不變。完成逆序為 `A D B E F C`。逐條邊檢查位置均由左向右，所以這是 G′ 的拓樸序。

原圖相同逆序不是拓樸序，C→A 與 F→E 都逆向。任何圖都能產生完成逆序，只有 DAG 保證其為拓樸序。






## 為何完成逆序可作拓樸排序

DAG 的每條邊 u→v 在掃描時，v 不可能是 GRAY，否則產生 back edge 與有向環。若 v 是 WHITE，遞迴先完成 v 再完成 u。若 v 是 BLACK，v 已完成而 u 尚未完成。因此每條邊均有 f[u]>f[v]。按 f 遞減排列便保證 u 排在 v 前。

實作在完成時 append，再整體 reverse，或完成時插到鏈結串列前端。不要用陣列反覆插入最前方，否則可能多花 $\Theta(V^2)$。若碰到 back edge，回報環並拒絕輸出拓樸序。

來源 • CLRS3-22 § § 22.3–22.4。

## Kahn 用剩餘入度看同一個 DAG

Kahn 重複取剩餘入度 0 的點，刪除它的出邊。若還有點卻沒有入度 0 的點，就有環。以字母最小者優先，G′ 的過程如下。

| 取出 | 刪邊後可選集合 |
|---|---|
| A | B,D |
| B | D,E |
| D | C,E |
| C | E |
| E | F |
| F | ∅ |

輸出 A B D C E F，與 DFS 不同但同樣合法。每次取出者沒有尚未處理的前驅，這是不變量。若剩餘每點入度≥1，反覆沿前驅走，有限性保證重複頂點，得到環。FIFO 佇列可達 Θ(V+E)，字母最小堆則需 O(E+V log V)。



## 成本與自學檢核

鄰接表完整 DFS 的時間 Θ(V+E)，每頂點呼叫一次，每有向邊掃一次。無向邊掃兩次仍是 Θ(V+E)。鄰接矩陣每頂點掃 V 格，時間 Θ($V^2$)。三色、parent、d/f 及最深遞迴鏈共 Θ(V) 額外空間。

用明確堆疊模擬時，frame 應存 (u, 下一個鄰接索引)，掃完後才能寫 f[u]。只把所有鄰居 push 進頂點堆疊，未必重現本例的遞迴樹、完成時間與分類。

練習 A07-D01 至 D06 依序檢驗時間、分類、定理前提、無向模型、環證明與拓樸序。

來源 ・ MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22。

## 連通問題的輸出不同

無向圖中，連通分量是可互相到達的極大頂點集合。有向圖中，SCC 要求兩方向皆有路徑；單向可達不足以合併。

| 問題 | 輸出 | 刪除測試 |
| --- | --- | --- |
| 連通／強連通 | 頂點分割 | 無 |
| 割點 | 頂點集合 | 刪頂點及其相接邊後，分量數增加 |
| 橋 | 邊集合 | 刪邊後，分量數增加 |
| 點雙連通區塊 | 邊分組，割點可重複出現 | 區塊內無割點 |

本節無向輸入固定為有限、無自環、無平行邊的 simple graph。鄰接串列與根皆按字母序，時間從 1 起。來源 [S1, S2]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 無向連通分量只需一個外層迴圈

對每個未標記頂點 s 開始 DFS 或 BFS，這次走訪全部標成同一個 component_id。 若存在從 s 到未標記 v 的路徑，路徑上第一個未標記頂點必由已走訪前驅發現，矛盾。因此走訪包含整個分量；走訪又只沿邊移動，不會跨越分量。 孤立點仍須由外層迴圈發現，不能只迭代邊。鄰接串列時間 Θ(V+E)，額外空間 O(V)。MIT 兩年份的 BFS 範例可支持這個遍歷背景，未提供 SCC 或割點演算法 [S3, S4]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

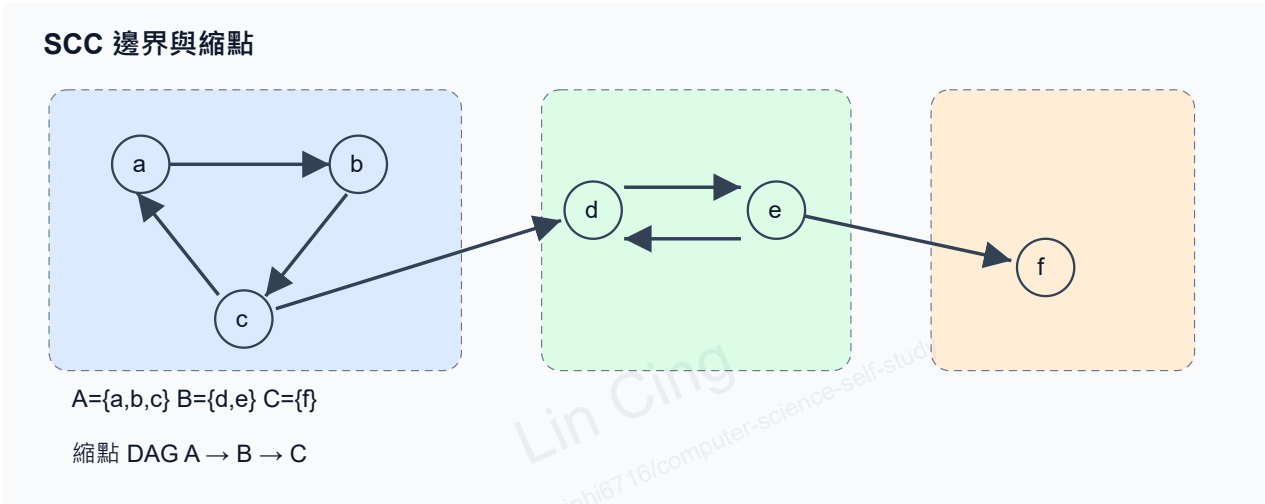
## SCC 的極大性與縮點圖

u~v 定義為 u 可達 v 且 v 可達 u。長度零路徑給自反性，定義給對稱性，路徑串接給遞移性，所以 SCC 是等價類且分割 V。 每個 SCC 縮成一點，保留跨分量有向邊，刪去自環並可去重，得到 condensation graph。 若它有長度至少二的有向環，環上各分量都可互相到達，會合成一個更大的 SCC，違反極大性。因此縮點圖是 DAG。單點 SCC 不必有自環 [S1 §22.5]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 數例先固定邊與分量邊界




G 的邊為 ab, bc, ca, cd, de, ed, ef。此處 xy 表示 x→y。 SCC 為 A={a,b,c}、B={d,e}、C={f}；縮點為 A→B→C。DFS 在原圖從 a 一次可走遍六點，故「一棵原圖 DFS 樹就是一個 SCC」是錯的。

## Kosaraju 第一趟要記完成時間

第一趟在 G 做完整 DFS 森林，每點完成時附加到 finish list。完成順序為 f,e,d,c,b,a；第二趟根候選順序取反向 a,b,c,d,e,f。

| 點 | 發現 d | 完成 f | 父點 |
|---|---|---|---|
| a | 1 | 12 | ∅ |
| b | 2 | 11 | a |
| c | 3 | 10 | b |
| d | 4 | 9 | c |
| e | 5 | 8 | d |
| f | 6 | 7 | e |

時間戳在發現與完成時各加一；不要把此 f 與頂點 f 混淆 [S1]。



## 轉置保留 SCC，反轉跨分量邊

$G^T$ 把每條 u→v 改為 v→u。原圖路徑 u⇝v 對應轉置路徑 v⇝u；故互相可達的等價類不變。 轉置串列為 a:[c], b:[a], c:[b], d:[c,e], e:[d], f:[e]。縮點為 C→B→A。 鄰接串列轉置先建 V 個空串列，再掃每條邊加入反向串列，成本 Θ(V+E)。第二趟必清空 visited，保留第一趟的完成順序 [S1]。

## 完成時間引理分兩種先發現情況

設不同 SCC 間有 C→D，並定義 F(C)=max{f(v):v∈C}、d(C)=min{d(v):v∈C}。要證 F(C)>F(D)。

1. 若 d(C)<d(D)，C 最先被發現的 x 當時可沿全白路徑到 C 的每點，也可經跨邊到 D 每點。這些點皆在 x 返回前完成，所以 F(C)=f(x)>F(D)。
2. 若 d(D)<d(C)，D 最先被發現的 y 可走遍 D，但不可到達 C，否則 C→D 與 D⇝C 會合併兩個 SCC。故 D 的所有點在 C 任一點被發現前完成，F(D)<d(C)<F(C)。

此引理比較的是分量最大完成時間，不保證每個 C 中頂點都比每個 D 中頂點晚完成 [S1 Lemma 22.14]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第二趟恰好收一個 SCC 的證明

依第一趟 f 遞減選尚未訪問的根 u，設它在 C。歸納假設先前樹已各收完整 SCC，故 C 此時全白，而且 F(C) 在剩餘分量中最大。 在 $G^T$ 若有 C→D，原圖就有 D→C，完成引理給 F(D)>F(C)。因此 D 必已在第二趟訪問。從 C 不能走進其他未訪問分量。 另一方面，C 在 $G^T$ 仍強連通，且全白，故 DFS 從 u 必走遍 C。這棵樹既不漏 C，也不包含別的分量。空森林為歸納基礎，逐樹成立直到全圖完成。 兩次 DFS 加轉置總時間 Θ(V+E)，額外空間 O(V+E)，含轉置 [S1 Theorem 22.16]。

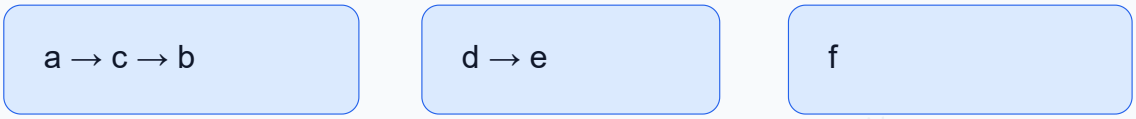
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 第二趟的根、堆疊與跳過

### Kosaraju 兩趟方向與順序

第一趟 G 的完成順序 f,e,d,c,b,a

反轉作根順序 a,b,c,d,e,f




第二趟在 $G^T$，每框是一棵樹 / 一個 SCC

已訪問的 b,c,e 根候選皆跳過

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 第二趟的根、堆疊與跳過 （續）

| 根候選 | $G^T$ 遞迴堆疊最深狀態 | 新發現順序 | 樹輸出 |
|---|---|---|---|
| a | [a,c,b] | a,c,b | {a,b,c} |
| b,c | 已訪問，跳過 | ∅ | ∅ |
| d | [d,e]；邊 d→c 已黑 | d,e | {d,e} |
| e | 已訪問，跳過 | ∅ | ∅ |
| f | [f]；邊 f→e 已黑 | f | {f} |

每棵樹返回後呼叫堆疊清空；分量輸出不依第二趟完成時間重排。使用 G 而不轉置、但維持 f 遞減時，根 a 將錯誤收進所有六點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Tarjan 使用尚未定案的頂點堆疊

disc[u] 是第一次發現序號；low[u] 是此版遞迴更新的 lowlink。S 儲存已發現但尚未輸出 SCC 的點；onstack[u] 表示 u∈S，與 DFS 灰色不同。

```
visit(u):
  disc[u] = low[u] = ++clock; push(S,u); onstack[u]=true
  for v in Adj[u]:
    if disc[v] is unset:
      visit(v); low[u]=min(low[u],low[v])
    else if onstack[v]: low[u]=min(low[u],disc[v])
  if low[u]==disc[u]:
    repeat: x=pop(S); onstack[x]=false; emit x
    until x==u
```

外層依序 visit 所有未發現點。已輸出點不再降低 low；對已發現且仍在 S 的邊使用 disc[v]，不可任換 low[v]。來源 [S5] 的現代記號改寫。



## Tarjan 的完整返回與彈出紀錄

**Tarjan 分量堆疊與彈出邊界**

堆疊 S 由底到頂；最右為 top

[a,b,c,d,e,f] → pop f

[a,b,c,d,e] → pop e,d

[a,b,c] → pop c,b,a

low = [1,1,1,4,4,6] SCC = {f} | {d,e} | {a,b,c}

下表 S 由底到頂。disc 按 a,b,c,d,e,f 為 1,2,3,4,5,6。

## Tarjan 發現頂點與更新 low

| 事件 | 當下更新 | S |
|---|---|---|
| 進 a | disc[a]=low[a]=1 | [a] |
| 進 b | disc[b]=low[b]=2 | [a,b] |
| 進 c | disc[c]=low[c]=3 | [a,b,c] |
| c→a | low[c]=1 | [a,b,c] |
| 進 d | disc[d]=low[d]=4 | [a,b,c,d] |
| 進 e | disc[e]=low[e]=5 | [a,b,c,d,e] |
| e→d | low[e]=4 | [a,b,c,d,e] |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Tarjan 返回與輸出分量

| 事件 | 當下更新 | S |
|---|---|---|
| 進 f | disc[f]=low[f]=6 | [a,b,c,d,e,f] |
| f 返回，6=6 | 彈 f；輸出 {f} | [a,b,c,d,e] |
| e 收 f，返回 | low[e]=min(4,6)=4；不彈 | [a,b,c,d,e] |
| d 收 e，4=4 | low[d]=4；彈 e,d；輸出 {d,e} | [a,b,c] |
| c 收 d，返回 | low[c]=min(1,4)=1 | [a,b,c] |
| b 收 c，返回 | low[b]=1 | [a,b,c] |
| a 收 b，1=1 | low[a]=1；彈 c,b,a；輸出 {a,b,c} | [] |

最後 low 為 1,1,1,4,4,6；各點 onstack 最終皆 false。b 返回時已非灰色，但尚在 S，所以呼叫堆疊與 S 不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## SCC lowlink 與無向 low 的分界

| 項目 | 有向 Tarjan SCC | 無向割點／橋 |
|---|---|---|
| 非樹邊目標 | 必須仍 onstack | 必須是祖先，排除父邊反向 |
| 狀態堆疊 | 未定案頂點 | 雙連通用邊堆疊 |
| 邊界測試 | low[u]=disc[u] | low[v]≥disc[u] 或 > |
| 輸出 | 頂點分割 | 割點、橋、邊區塊 |

SCC 邊可指向已完成呼叫但仍在 S 的點。例 a→b,b→a,a→c,c→b，b 呼叫返回後 c→b 仍影響 low[c]。若只用 gray 判定會誤分 c。 若從 x 先輸出單點 SCC，再從 y 看見 y→x，onstack[x]=false，這條邊不能把 y 與 x 合併。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 無向 low 是子樹的逃生能力

對 DFS 頂點 u，low[u] 是從 u 子樹沿零或多條向下樹邊，再至多一條回祖先邊可達的最小 disc；包含 disc[u]。 初始化 low[u]=disc[u]。樹子點 v 返回後取 min(low[u],low[v])。非父邊連到已發現祖先 w 時取 min(low[u],disc[w])。 無向 DFS 無跨子樹邊，否則先走的子樹會經那條邊發現另一子樹。故判斷子樹能否繞過 u，只需知道能否由回邊到達 u 的真祖先 [S1 Problem 22-2]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 割點判定必須分根與非根

非根 u 是割點，當且僅當存在 DFS 子點 v 滿足 low[v]≥disc[u]。此時 v 子樹無法到達 u 的真祖先，刪 u 便與 u 的父側分離；若每個子樹都可回真祖先，刪 u 後仍可經祖先側連接。 根 r 是割點，當且僅當 DFS 樹子點數至少 2。不同根子樹無邊相連，刪根會拆開；只有一棵子樹時，刪根仍由那棵子樹連通。零個子點也不是割點。 根不能套非根公式。只有 a—b 的圖也有 low[b]≥disc[a]，但刪 a 不增加分量數 [S1 22-2(a,b)]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

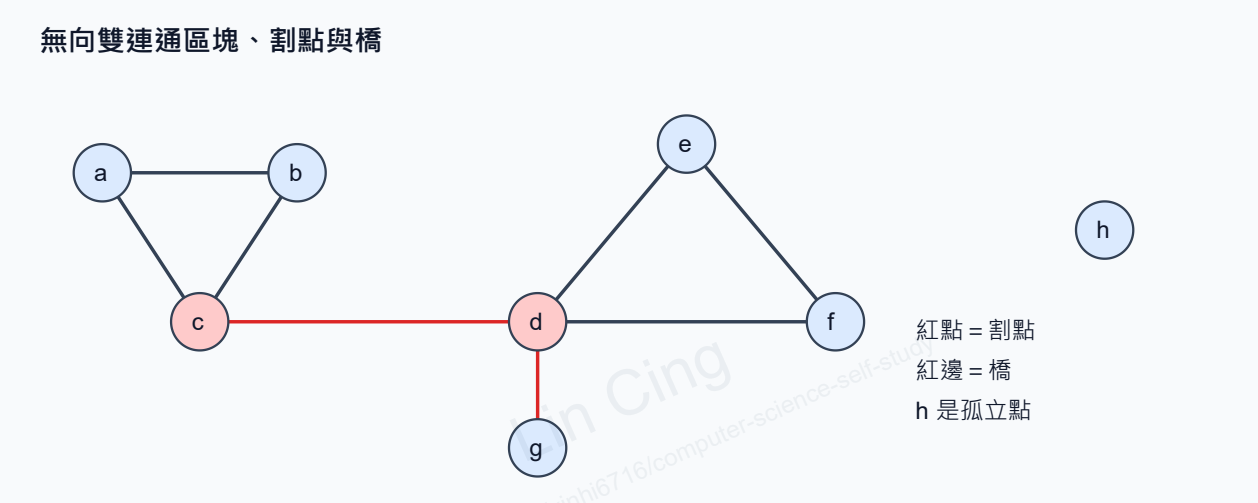


## 橋使用嚴格大於

樹邊 (u,v) 是橋，當且僅當 low[v]>disc[u]。沒有回邊到 u 或更早祖先，刪除該邊後兩側分離。 等號 low[v]=disc[u] 表示子樹能回到 u，形成包含 (u,v) 的環，故該邊不是橋。但刪掉 u 同時刪回邊終點，非根 u 仍可能是割點。 在無向 simple graph，非樹邊總在環中，因此不會是橋。若擴充到多重圖，必以 edge_id 跳過恰好那條 parent edge，不能跳過所有通往父點的邊；兩條平行邊就不是橋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 無向數例與完整 disc／low 表




邊集合 ab,bc,ca,cd,de,ef,fd,dg；h 孤立。字母序 DFS。



# 無向 DFS 的 disc／low：a 至 d

| 點 | disc | low | 父點 | DFS 子點 |
|---|---|---|---|---|
| a | 1 | 1 | ∅ | b |
| b | 2 | 1 | a | c |
| c | 3 | 1 | b | d |
| d | 4 | 4 | c | e,g |

## 無向 DFS 的 disc／low：e 至 h 與判定

| 點 | disc | low | 父點 | DFS 子點 |
|---|---|---|---|---|
| e | 5 | 4 | d | f |
| f | 6 | 4 | e | ∅ |
| g | 7 | 7 | d | ∅ |
| h | 8 | 8 | ∅ | ∅ |

回邊 c→a 降 low[c] 至 1，f→d 降 low[f] 至 4，再逐層向上傳遞。割點 {c,d}，橋 {cd,dg}。a 只有一個樹子點，不是割點；d 有多個子點但它是非根，仍套非根規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 雙連通區塊需要邊堆疊

採 block 慣例，橋獨立成二點一邊區塊，孤立點另記 singleton vertex block。大小至少三的點雙連通區塊為極大無割點子圖。這與 CLRS 22-2 的只分割非橋邊的 cycle-based bcc 慣例不同，輸出時須說明。 走樹邊 (u,v) 前 push 該邊；看到往祖先的回邊時 push 一次。子點返回且 low[v]≥disc[u] 時，持續 pop 並把 (u,v) 本身也包含進輸出的區塊，然後停止。 這個邊界適用根的每個子樹，與根是不是割點分開判定。每邊最多 push/pop 一次，總時間 Θ(V+E)。區塊分割邊，割點可以同時屬於多個區塊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 邊堆疊的每個分量邊界

下列堆疊由底到頂；ca 與 fd 是回邊，父邊反向不再 push。

| 事件 | 邊堆疊 | 輸出 |
|---|---|---|
| 依序 push ab,bc,ca,cd,de,ef,fd | [ab,bc,ca,cd,de,ef,fd] | ∅ |
| f 返回 e，4<5 | 不變 | ∅ |
| e 返回 d，4≥4 | [ab,bc,ca,cd] | pop fd,ef,de |
| push dg，g 返回，7≥4 | [ab,bc,ca,cd] | pop dg |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 邊堆疊回到根並輸出區塊

| 事件 | 邊堆疊 | 輸出 |
|---|---|---|
| d 返回 c，4≥3 | [ab,bc,ca] | pop cd |
| c 返回 b，1<2 | 不變 | ∅ |
| b 返回 a，1≥1 | [] | pop ca,bc,ab |
| 以 h 為新根 | [] | 孤立點 {h} |

區塊為 {de,ef,fd}、{dg}、{cd}、{ab,bc,ca}，加孤立點 {h}。只 pop 一條邊、或在遇到 de 之前停止，會把三角形拆錯。空堆疊是每棵完整 DFS 樹結束的檢查條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把公式用在故障問題上

本例原有兩個連通分量，主圖與 {h}。刪 c 後為 {a,b}、{d,e,f,g}、{h} 共三個；刪 d 後為 {a,b,c}、{e,f}、{g}、{h} 共四個。刪橋 cd 後為三個，刪 de 仍為兩個。 刪掉割點所增加的分量數不必是 1。橋表示單一連線故障，割點表示節點及所有相接線路一起故障；SCC 則描述方向約束下的互相可達，三者不能互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 來源與適用範圍

S1 為 CLRS 第三版 §22.5，印刷頁 615–620；割點與橋補充用 Problem 22-2，印刷頁 621–622。S2 為原手寫 note P34–36 的 DFS、完成時間與邊分類，這些已讀頁無 SCC 或割點 low 公式。 S3 為 MIT 2004 L14，PDF 45–59；S4 為 MIT 2005 L17，同範圍為 BFS，僅補遍歷背景。S5 為 Tarjan 1972 原始論文，SCC 部分以現代 disc/low/onstack 記號改寫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

CLRS, Introduction to Algorithms, 3rd ed., §§22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 最小生成樹與最短路徑

MST 最小化連通所有頂點的總邊權。最短路徑最小化指定端點之間的路徑成本。兩者的狀態與安全選擇條件不同。

依序研究 **cut 與 MST → relaxation、DAG、Bellman–Ford、Dijkstra → APSP、min-plus、Floyd–Warshall、Johnson**。

前置知識為同組的不相交集合單元（原第21章）與優先佇列操作契約，以及第7章圖、路徑與拓樸排序。原第21章介紹 MAKE-SET、UNION、FIND、rank 與路徑壓縮，並證明精細的反阿克曼成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 先固定問題，再選演算法

| 問題與條件 | 正確性依據 |
| --- | --- |
| 無向圖的最小連通成本 | cut property 與安全邊交換。 |
| DAG 的單源最短路徑 | 拓樸序讓前驅先完成。 |
| 含負邊的單源最短路徑 | Bellman–Ford 的邊數界與負環檢查。 |
| 非負邊的單源最短路徑 | Dijkstra 抽出後定案的不變量。 |
| 所有有序端點對 | DP 狀態或 Johnson 勢能重加權。 |

每一步都對照距離定義、圖的假設、狀態軌跡與成本模型。負邊與負環的影響須分開判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-01｜要最小化的是連通成本

輸入為無向加權圖 G=(V,E)，w:E→ℝ。連通圖的生成樹有 |V|−1 條邊，MST 最小化 Σe∈T w(e)。它不要求每個根到頂點的路徑最短。

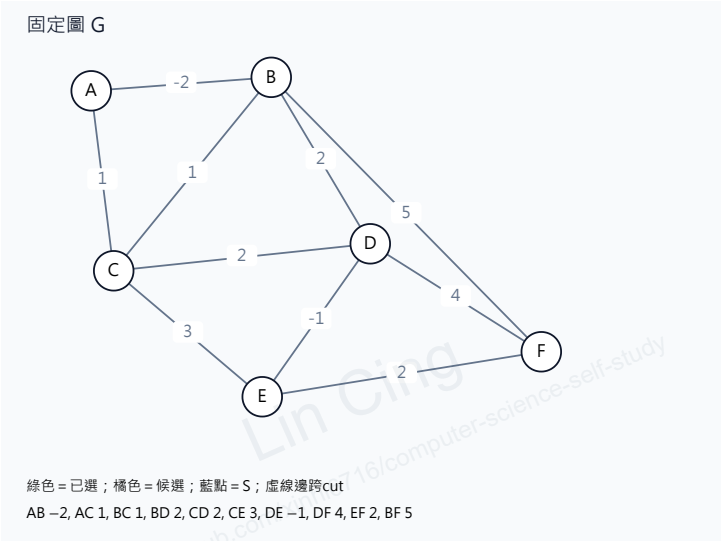
前置為同組的不相交集合單元（原第21章）的 MAKE-SET / UNION / FIND、rank 與路徑壓縮，以及 heap 操作契約與第7章圖、路徑、cycle、cut。DSU 的精細攤銷分析見原第21章。

來源 [CLRS23 §23.1; MIT04 L13.8–9; MIT05 L16.8–9]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# A08-MST-02｜同一張圖，兩種貪心




V={A,B,C,D,E,F}。AB=−2，AC=BC=1，BD=CD=EF=2，CE=3，DE=−1，DF=4，BF=5。

## A08-MST-03｜安全邊保留至少一個最優解

Kruskal 同權依邊名字典序；Prim 從 A 開始，以 (key,頂點名) 抽最小，只有嚴格變小才換 parent。所有輪次使用這些規則。

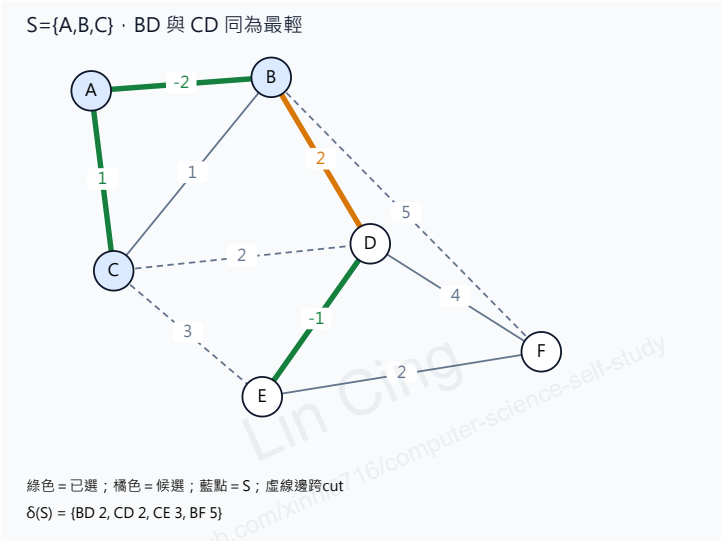
維持不變量 A ⊆ 某棵 MST，並非 A 屬於所有 MST。若 A∪{e} 仍包含於某棵 MST，稱 e 對 A 安全。

cut (S,V−S) 尊重 A，意指 A 沒有邊跨越 cut。跨 cut 的最小權重邊稱 light edge，可能不只一條。

來源 [CLRS23 Theorem 23.1]。



## A08-MST-04｜cut property 的 ties 條件




若 A 可延伸成 MST，cut 尊重 A，任一 light edge 都對 A 安全。此例 δ(ABC)={BD 2,CD 2,CE 3,BF 5}，BD 或 CD 都可選。

## A08-MST-05｜交換證明的每一步

當 e 是某 cut 唯一最輕邊，e 屬於每棵 MST。有 ties 時只保證可納入某棵 MST，不能推成每棵。

MIT04/05 為簡化假設所有權重相異；本章採 CLRS 的一般條件。

取含 A 的 MST T。若 e∈T 已完成；否則 T+e 出現唯一 cycle。e 跨 cut，因此 T 中連接其端點的路徑至少有另一條跨 cut 邊 f。

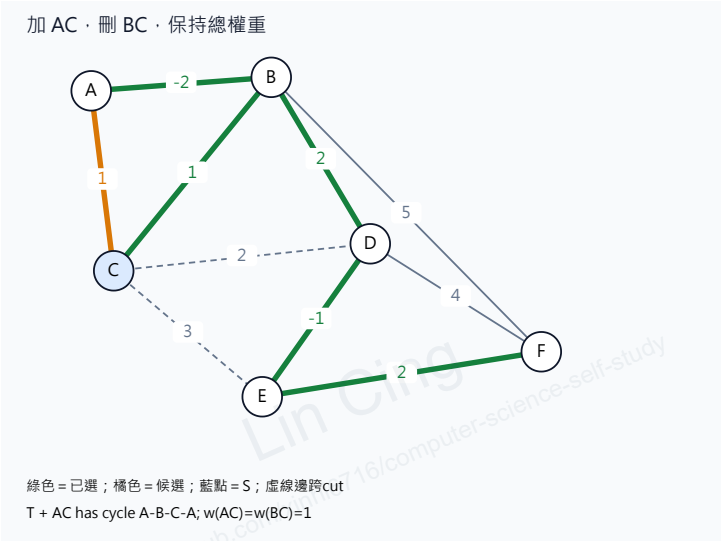
cut 尊重 A ⇒ f∉A。e 最輕 ⇒ w(e)≤w(f)。T′=T−f+e 連通且無 cycle，仍含 A∪{e}，且 w(T′)≤w(T)。

T 已最優，故必有 w(T′)=w(T)。等號容許 ties；若 e 唯一最輕而 e∉T，則嚴格變小會矛盾。

來源 [CLRS23 pp627–628; MIT04 L13.22–25; MIT05 L16.22–25]。



## A08-MST-06｜圖上交換能保住 A




令 A={AB,DE}，T={AB,BC,BD,DE,EF}。cut S={C} 尊重 A；AC 和 BC 都是最輕邊，權重 1。加 AC 產生 A–B–C–A；刪除不在 A 的 BC。

## A08-MST-07｜cycle property 不能省略嚴格二字

得到 {AB,AC,BD,DE,EF}，總權重同為 2。此圖也展示「safe」不代表「原本就在指定 T 裡」。

若 e 是某 cycle 唯一最重邊，e 不屬於任何 MST。證明時假設 T 含 e，刪 e 形成 cut；cycle 的其他邊有跨 cut 的 f，且 w(f)<w(e)，交換就更便宜。

若 e 只是並列最重，只能說存在某棵 MST 不含 e。三角形三邊都為 1 時，每邊都可能被選，也都可能被省略。不可同時刪光並列邊。

來源 [CLRS23 exercise 23.1-5，推導]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A08-MST-08｜Kruskal 用分量決定能否選

依 (weight,edge-name) 遞增掃邊。FIND(u)≠FIND(v) 才選 uv 並 UNION；相同即有已選路徑，加入會成 cycle。

每次接受邊是連接某個當前分量與外界的 light edge，其分量 cut 尊重 A。DSU 只管當前 A 的連通性，不是原圖 G 的連通性。

來源 [CLRS23 §23.2]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A08-MST-09｜Kruskal 全部十輪

| 輪次 / Round | 邊 / Edge | 判定 / Decision | DSU 分量 / Components after round |
|---|---|---|---|
| 0 | ∅ | 初始化 / initialize | A / B / C / D / E / F |
| 1 | AB, −2 | 選 / accept | AB / C / D / E / F |
| 2 | DE, −1 | 選 / accept | AB / C / DE / F |
| 3 | AC, 1 | 選 / accept | ABC / DE / F |
| 4 | BC, 1 | 拒 / reject, B–A–C 已連通 / connected | ABC / DE / F |
| 5 | BD, 2 | 選 / accept | ABCDE / F |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



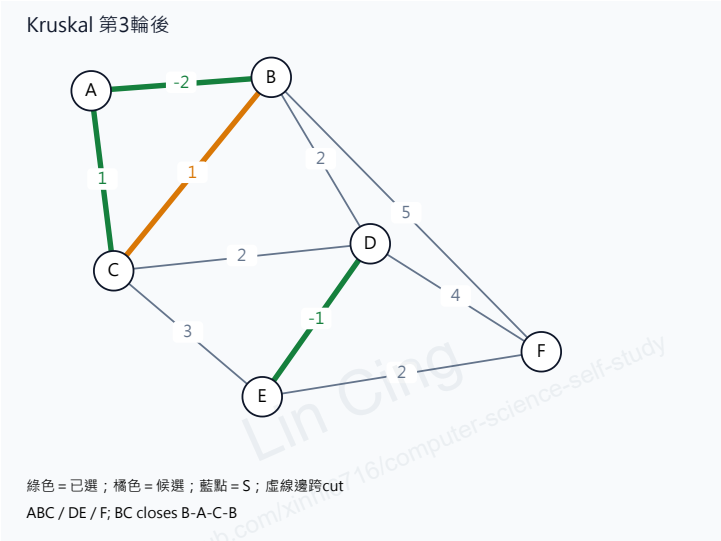
## A08-MST-09B｜Kruskal 後五輪

| 輪次 / Round | 邊 / Edge | 判定 / Decision | DSU 分量 / Components after round |
|---|---|---|---|
| 6 | CD, 2 | 拒 / reject, C–A–B–D 已連通 / connected | ABCDE / F |
| 7 | EF, 2 | 選 / accept | ABCDEF |
| 8 | CE, 3 | 拒 / reject, C–A–B–D–E 已連通 / connected | ABCDEF |
| 9 | DF, 4 | 拒 / reject, D–E–F 已連通 / connected | ABCDEF |
| 10 | BF, 5 | 拒 / reject, B–D–E–F 已連通 / connected | ABCDEF |

第7輪已取得5條邊可提早停止。表格仍掃完全部10邊，讓每個拒邊原因可核對。DSU欄列分量，不聲稱列出內部parent樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

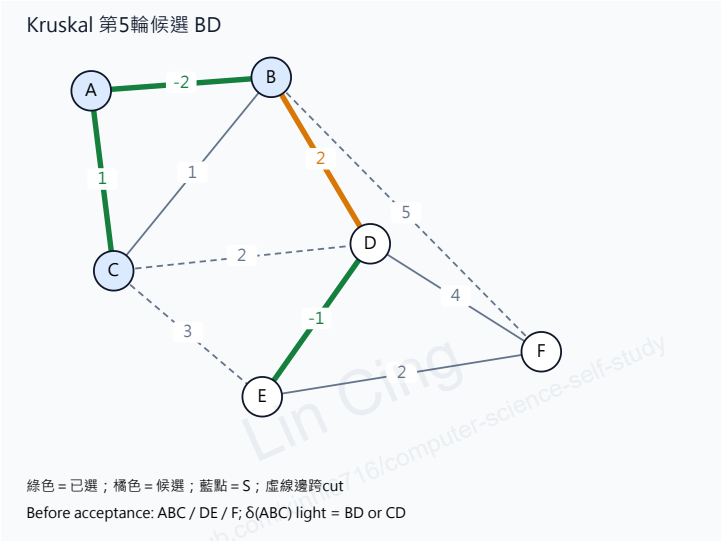
# A08-MST-10｜拒 BC，接 BD




第4輪 BC 不是因權重大才拒絕，因 B、C 已同分量。



# A08-MST-10｜拒 BC，接 BD （續）




第5輪 BD 的 cut 證據正是卡04。CD 同權可代替 BD，但排在 BD 後，之後已成環。

## A08-MST-11｜Prim 的 key 是一條邊的成本

S 是已抽出頂點。對 v∉S，key[v]=min{w(u,v):u∈S}，不存在則為 ∞；parent[v] 記錄達到最小值的 u。

抽 key 最小的 v，parent 邊是跨 (S,V−S) 的 light edge。只在 v 還在 Q 且 w(u,v)<key[v] 時更新。初始化 key[A]=0 僅為選根，並非實際邊。

來源 [CLRS23 pp634–636; MIT04 L13.26; MIT05 L16.26]。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



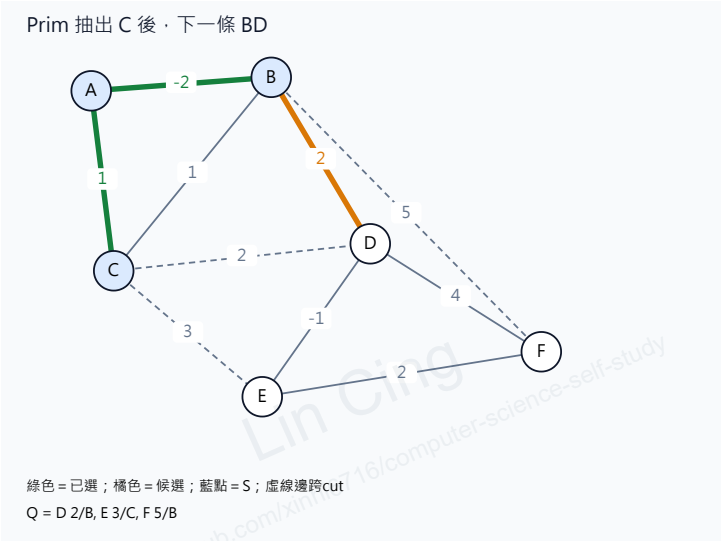
## A08-MST-12｜Prim 全部 key、parent 與 PQ

| 抽出 / Extract | 抽出前 S / S before | 加入 / Add | 掃鄰邊後 Q，key/parent，優先序 / Q after relaxation, in priority order |
|---|---|---|---|
| 初始 / Initial | ∅ | ∅ | A 0/∅, B ∞/∅, C ∞/∅, D ∞/∅, E ∞/∅, F ∞/∅ |
| A | ∅ | 根 / root | B −2/A, C 1/A, D ∞/∅, E ∞/∅, F ∞/∅ |
| B | A | AB | C 1/A, D 2/B, F 5/B, E ∞/∅ |
| C | AB | AC | D 2/B, E 3/C, F 5/B |
| D | ABC | BD | E −1/D, F 4/D |
| E | ABCD | DE | F 2/E |
| F | ABCDE | EF | ∅ |

Q列出邏輯優先序，不是binary heap的陣列位置。未出現在Q的頂點已在S，最終parent為B←A,C←A,D←B,E←D,F←E。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

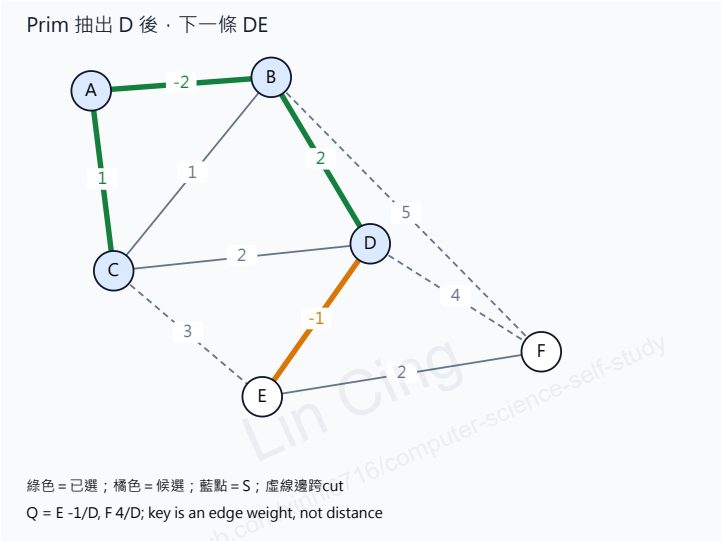
## A08-MST-13｜每次抽出都有一個 cut




C抽出後，S=ABC，Q={D,E,F}；δ(S) 為 BD 2,CD 2,CE 3,BF 5。Q中D的key=2，parent=B。CD=2沒有嚴格更小，因此保留B。

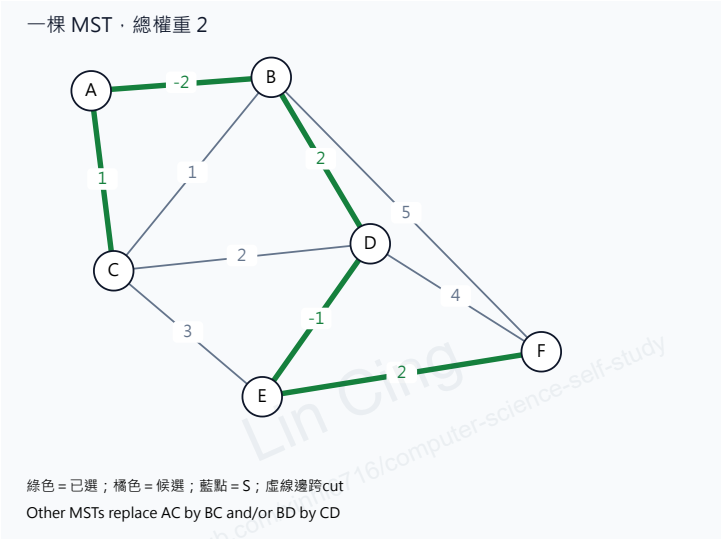


## A08-MST-14｜負權不破壞 MST 貪心




D抽出前key[D]=2；掃D後key[E]=−1。抽出key序列0,−2,1,2,−1,2並不單調。

## A08-MST-15｜共同答案與非唯一




T={AB,AC,BD,DE,EF}，w(T)=−2+1+2−1+2=2。獨立枚舉全部 C(10,5)=252 個五邊子集，篩連通樹後確認最小值2，共4棵MST。



## A08-MST-15A｜跨 cut、負權與最優解

其餘抽出前cut依序為 S={A}、AB、ABCD、ABCDE，最輕候選分別 AB −2、AC/BC 1、DE −1、EF 2。根A沒有parent邊。

交換證明只比較邊權，不要求非負。Prim不做 key[u]+w(u,v)，因此不是Dijkstra的距離鬆弛。

全部最優集合為 {AB,DE,EF}∪{AC或BC}∪{BD或CD}。權重相異足以保證唯一；有重複權重不必然不唯一。

## A08-MST-16｜不連通時交付 minimum spanning forest

若圖有 c 個連通分量，跨所有頂點的生成樹不存在。分量各自的 MST 合成 MSF，邊數 |V|−c。

固定圖若只保留 {AB −2,AC 1,BC 1,DE −1,DF 4,EF 2}，分成 ABC 與 DEF。MSF={AB,AC,DE,EF}，權重0，另一棵以BC換AC。

Kruskal 掃完即得 MSF。Prim 當剩餘最小key=∞，以該頂點為新根，parent=∅，不加入∞假邊，再繼續掃鄰邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A08-MST-17｜成本跟實作一起寫

令 n=|V|、m=|E|。Kruskal比較排序 O(m log m)，DSU初始化與操作 O((n+m)(1+α(n)))，沿用原第21章的界。簡單連通圖可合寫 O(m log n)。

| Prim 實作 | 抽最小 | 降低key | 整體時間 |
|---|---|---|---|
| 陣列掃描＋鄰接矩陣 | O(n) | O(1) | $O(n^2)$ |
| indexed binary heap＋鄰接串列 | O(log n) | O(log n) | O((n+m) log n) |
| Fibonacci heap＋鄰接串列 | O(log n) 攤銷 | O(1) 攤銷 | O(m+n log n) |

indexed heap需維護頂點位置，Q成員測試O(1)。lazy heap每次改善push新紀錄，pop時丟棄過期紀錄，可能有O(m)個項目，界為O(n+m log(m+1))、額外O(n+m)空間；不能宣稱其heap只有n項。

來源 [CLRS23 pp633,636; MIT04 L13.40–50; MIT05 L16.40–50]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A08-MST-18｜把證明、狀態、答案互相核對

選邊先指出尊重A的cut與light條件；拒邊先給已選路徑。Prim每輪讀 key/parent/Q，Kruskal每輪讀 DSU分量。ties只決定哪棵最優樹，負權不改安全性。

練習 A08-M01–08 分別檢查cut、交換、cycle、DSU、PQ、MSF、唯一性與成本。`check.py`獨立枚舉所有生成樹並驗證這兩張完整追蹤表。

原筆記PDF pp36–37提供交換思路與第三輕邊反例；本章用CLRS23補足cut尊重A與ties的精確敘述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 單源最短路徑的三種答案

給有限有向圖、有限實數邊權與來源 s。定義 δ(s,v) 為所有 s→v walk 權重的下確界。不可達為 ∞；可沿途繞可達負環再到 v 為 −∞；其餘可達點存在有限最短路。空路使 δ(s,s)≤0。

本節以同一圖追蹤 d、π、佇列，分清演算法「何時可承諾答案」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 負邊與負環的差別

若到 v 的 walk 不能經過負環，刪去正環使成本降低，刪去零環不改成本，因此存在至多 $|V|-1$ 邊的簡單最短路。

全圖無負環是充分條件。對單一 v 的精確條件是沒有「s 可達、且可再到 v」的負環。別支的負環未必破壞 v；不可達負環不影響來源 s。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 共同核心是 relaxation

```
initialize: d[s]=0; d[v≠s]=∞; π[v]=NIL
RELAX(u,v,w):
  if d[u] ≠ ∞ and d[v] > d[u]+w:
    d[v]=d[u]+w; π[v]=u
```

∞ 以獨立標記表示。對有限 w，∞+w=∞。整數程式先判斷可達，再用足夠寬的數值加法。不可拿最大整數直接相加，也不能任意向下截斷負值。嚴格改善才改 π；平行邊時 π 應記錄邊 ID。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## relaxation 不變量與收斂

每個有限 d[v] 都是某條實際 s→v walk 的成本。因此 d[v]≥δ(s,v)，估計值只降不升，不可達點永遠為 ∞。

若最短路 s⇝u→v 的前綴已收斂，relax(u,v) 後 v 也收斂。依路徑順序 relax 每一條邊即可傳遞正確值；其間穿插其他 relax 不妨礙。到達有限 δ 後不再改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

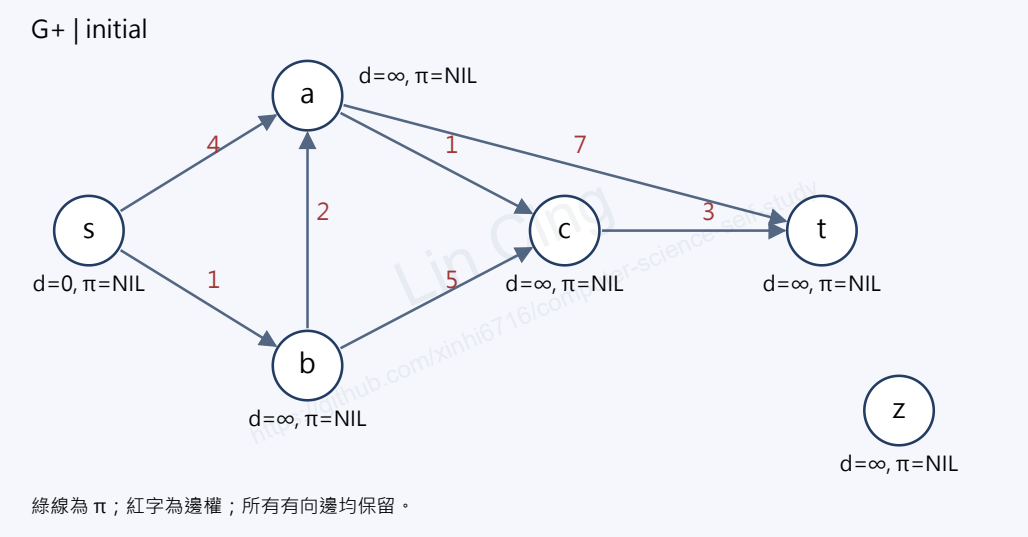


## 固定主圖 G+ 與初始化

頂點順序固定 (s,a,b,c,t,z)，z 孤立。全部邊如下。

s→a:4, s→b:1, b→a:2, a→c:1, b→c:5, c→t:3, a→t:7。

初始化 d=(0,∞,∞,∞,∞,∞)，π 全 NIL。邊及數值表均為自製例，並非教材圖的複製。



## DAG 允許負邊

令 G− 與 G+ 相同，僅 a→c 改成 −2。它仍為 DAG。固定拓序 s,b,a,c,t,z。每個 u 被處理前，其所有前驅已完成，所以 d[u] 已正確；無須比較當前最小鍵。

拓序加掃邊為 Θ(V+E)。拓序需先驗證，不能把有環圖任意排一排就套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## DAG 的單次傳遞

以下每列完成該點出邊。π 仍按 (s,a,b,c,t,z) 排列。

| u | d | π |
|---|---|---|
| 初始 | (0,∞,∞,∞,∞,∞) | (NIL,NIL,NIL,NIL,NIL,NIL) |
| s | (0,4,1,∞,∞,∞) | (NIL,s,s,NIL,NIL,NIL) |
| b | (0,3,1,6,∞,∞) | (NIL,b,s,b,NIL,NIL) |
| a | (0,3,1,1,10,∞) | (NIL,b,s,a,a,NIL) |
| c | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |
| t,z | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |

注意 a 的 3 大於 c 最終的 1，卻必須先處理 a。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Bellman-Ford 固定掃邊順序

在 G− 不使用拓序，固定每 pass 掃描以下七邊，直接寫回 d。

c→t:3, a→t:7, a→c:−2, b→c:5, b→a:2, s→b:1, s→a:4。

執行 |V|−1=5 passes，再檢查有沒有可改善的邊。若某整個 pass 無更新，可提前停。順序影響中途值與收斂輪數，但不影響無可達負環時的最終答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bellman-Ford 逐 pass 的 d、π

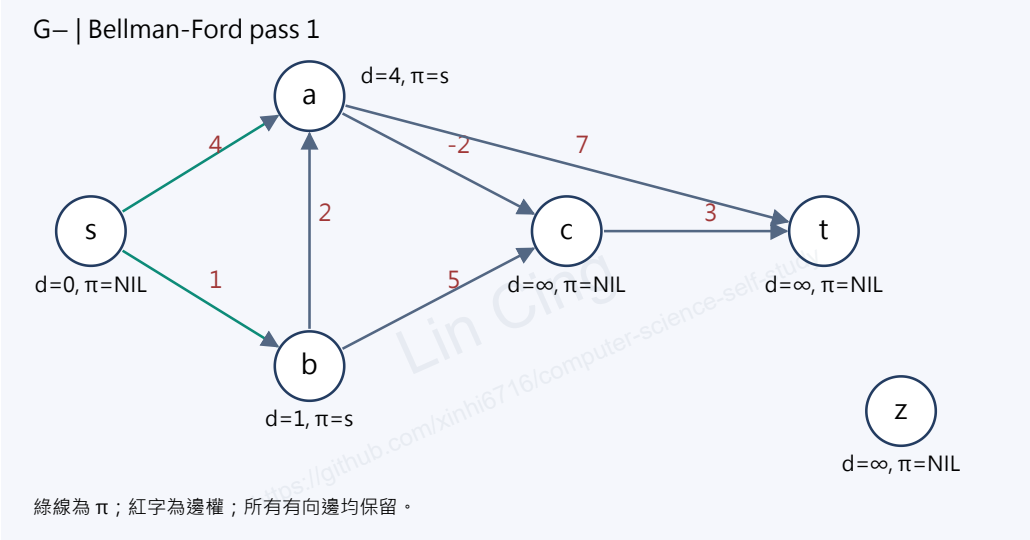
| pass | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) |
|---|---|---|
| 0 | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) |
| 1 | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) |
| 2 | (0, 3, 1, 2, 11, ∞) | (NIL, b, s, a, a, NIL) |
| 3 | (0, 3, 1, 1, 5, ∞) | (NIL, b, s, a, c, NIL) |
| 4 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |
| 5 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |

第 1 pass 的尾端才處理 s 出邊。第 2 pass 先用 a=4 令 c=2、t=11，尾端才令 a=3。第 3 pass 先令 t=5 再令 c=1；第 4 pass 才令 t=4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

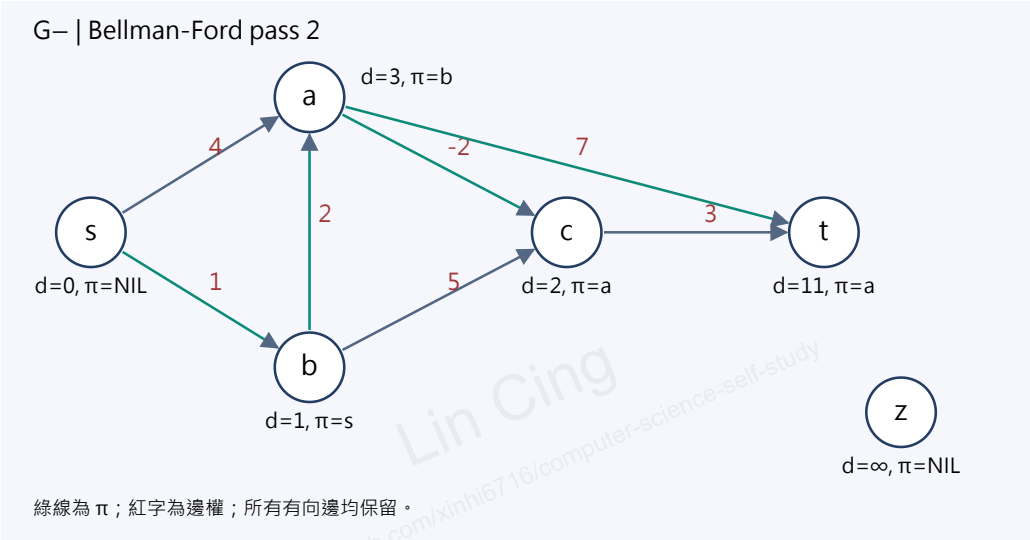
## 逐 pass 圖必須保留邊與暫存值

圖與上一表是同一個 G−、同一順序、同一取樣點。第 2 pass 的 π 雖已形成 s→b→a→c，d[c]=2 仍不是目前父鏈的成本 1。父鏈與 d 可能短暫不同步。

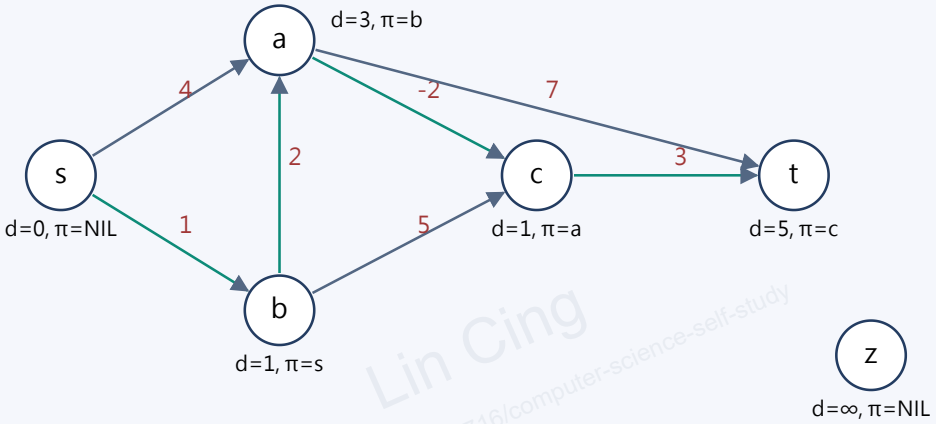





## 逐 pass 圖必須保留邊與暫存值 （續）



## 逐 pass 圖必須保留邊與暫存值 （續）




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bellman-Ford 為何只需 V−1 passes

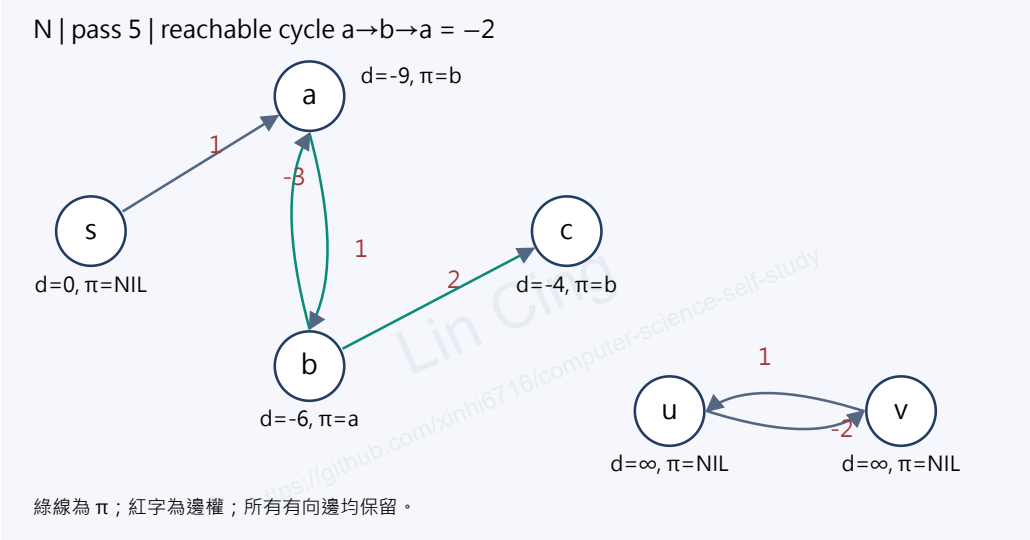
第 k 個完整 pass 後，每個可用 ≤k 邊的最短路已收斂。理由是第 k pass 一定掃到該路的第 k 邊，而其前綴至遲前一 pass 已收斂。

in-place 實作可能一輪傳過多邊，故不可宣稱 d「恰等於限制 k 邊的最短距離」。無可達負環時存在 ≤V−1 邊的簡單最短路，因此結果正確。時間 O(VE)，含初始化寫成 O(V+VE) 可涵蓋 E=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 可達負環與受影響後繼

圖 N 的全部邊為 s→a:1,a→b:1,b→a:−3,b→c:2,u→v:−2,v→u:1。u,v 不可達。固定此順序，頂點 (s,a,b,c,u,v)。






## 負環檢查與受影響頂點

pass 0 到 5 的 d 依次為 (0,∞,∞,∞,∞,∞)、(0,−1,2,4,∞,∞)、(0,−3,0,2,∞,∞)、(0,−5,−2,0,∞,∞)、(0,−7,−4,−2,∞,∞)、(0,−9,−6,−4,∞,∞)。

第 6 次只讀檢查發現 a→b 可改善，因 −9+1<−6。從所有這類目的點做 DFS/BFS，標記 a,b,c 為 −∞；s 保持 0，u,v 為 ∞。不能把全圖全標成 −∞。負環檢測正確性來自沿環把「不可改善」不等式相加會得到 0≤負環權重的矛盾。

## Dijkstra 的演算法合約

所有邊權必須非負。S 為已確定頂點，Q=V−S。每輪 extract-min u，加入 S，依鄰接串列 relax 所有 u 的出邊並 decrease-key。

本例 Q 含所有頂點，鍵同值以頂點名稱排序。表中 Q 已排序顯示邏輯內容，不表示 heap 陣列。若最小鍵為 ∞，可提前停止，剩餘點全不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dijkstra 每輪完整 d、π、Q

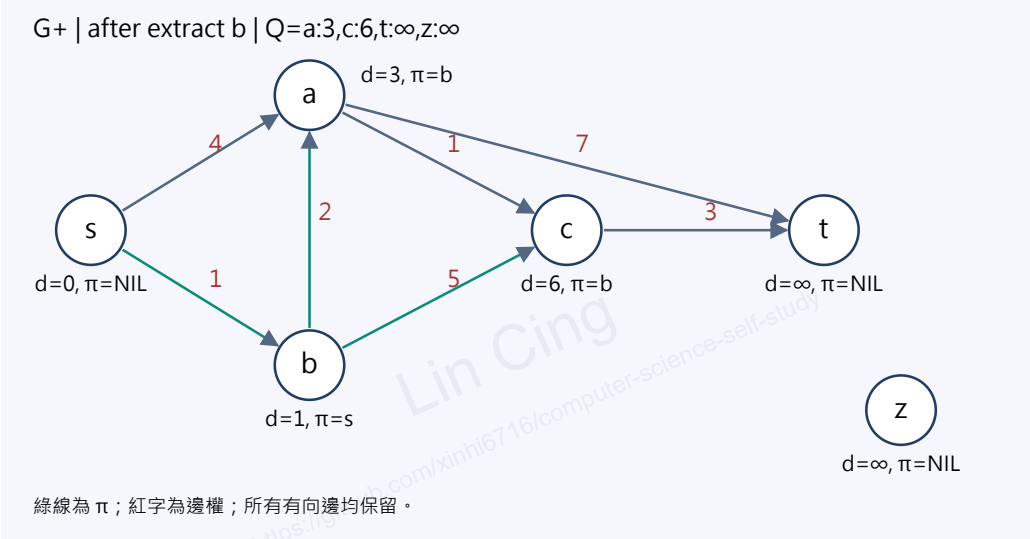
| extract | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) | Q after relaxation |
|---|---|---|---|
| init | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) | s:0, a:∞, b:∞, c:∞, t:∞, z:∞ |
| s | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) | b:1, a:4, c:∞, t:∞, z:∞ |
| b | (0, 3, 1, 6, ∞, ∞) | (NIL, b, s, b, NIL, NIL) | a:3, c:6, t:∞, z:∞ |
| a | (0, 3, 1, 4, 10, ∞) | (NIL, b, s, a, a, NIL) | c:4, t:10, z:∞ |
| c | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | t:7, z:∞ |
| t | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | z:∞ |
| z | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | |

每列在該輪出邊全部 relax 後取樣。初始化到 extract s，先得到 a=4,b=1。extract b 才把 a 改成 3，並首次令 c=6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩次關鍵更新與可見佇列

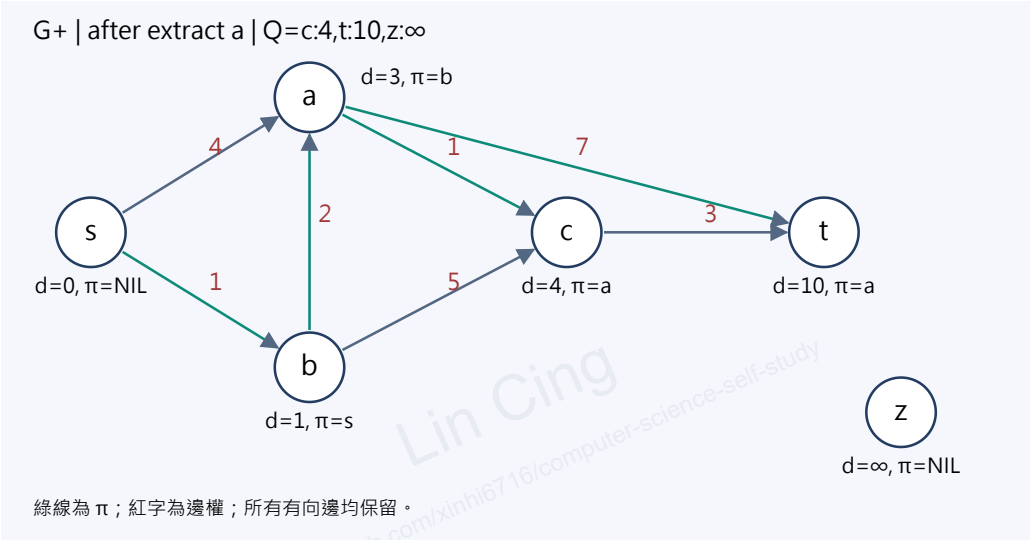
extract b 後 a=3、c=6；extract a 後 c=4、t=10。因此 c 的父點從 b 改為 a，t 首次由 a 接入。




綠線為 π；紅字為邊權；所有有向邊均保留。



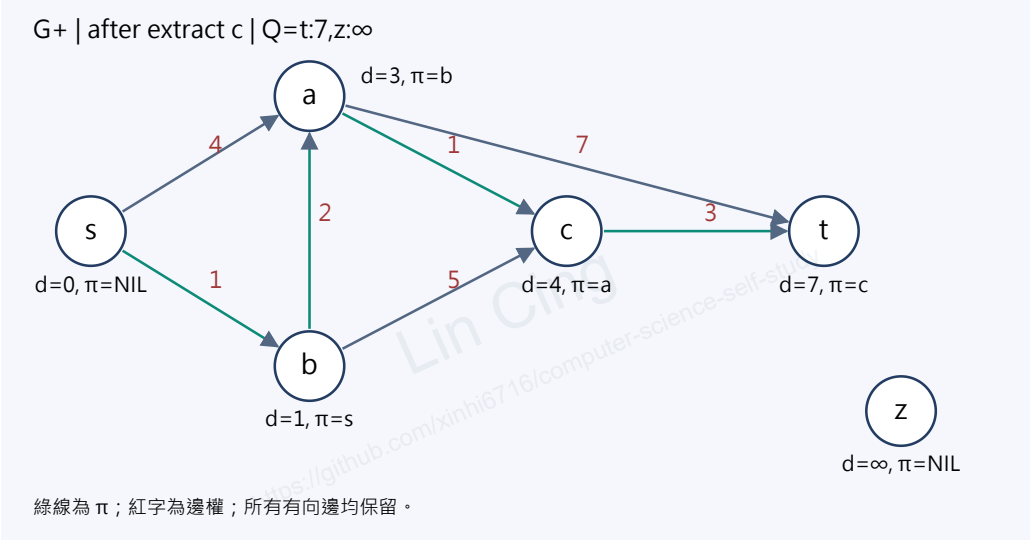
## 兩次關鍵更新與可見佇列 （續）




綠線為 π；紅字為邊權；所有有向邊均保留。

## 最後改善與不可達點

extract c 後 t 從 10 降為 7，π[t] 從 a 改為 c。extract t 不改值；最後 z 的 ∞ 不是尚未算出的有限答案。




最短路 s→b→a→c→t 成本 1+2+1+3=7。



## 為何 extract-min 可確定答案

不變量為每個 x∈S 都有 d[x]=δ(s,x)。設下一個 u 可達，取一條 s→u 最短路，y 為第一個不在 S 的點，x 為其前驅。x 出邊早已 relax，故 d[y]=δ(s,y)。

非負的 y⇝u 後綴給 δ(s,y)≤δ(s,u)。由 extract-min 與上界性，
δ(s,u)≤d[u]≤d[y]=δ(s,y)≤δ(s,u)，全部相等。不可達 u 用 no-path 性質處理。

## 負邊如何打破 settled 證明

反例 H 為 s→a:2, s→b:5, b→a:−4, a→t:2。抽取順序 s,a,t,b，抽 a 時得到 2，實際 δ(s,a)=1；t 被定為 4，實際為 3。

若禁止修改 S，a 也錯；若允許改 a 卻不重新處理其出邊，t 仍錯。可重開點的修正版不再具這個 Dijkstra 的 settled 證明與成本。失效處正是 δ(s,b)≤δ(s,a) 不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dijkstra 的優先佇列成本

|V| 次 extract-min，至多 |E| 次成功 decrease-key，每邊掃描一次。

| Q 實作 | 總時間 |
| --- | --- |
| 無序陣列 | $O(V^2+E)$ |
| 二元 heap | $O((V+E)\log V)$ |
| Fibonacci heap | $O(E+V\log V)$ 攤銷 |

lazy heap 每改善插入新紀錄、抽出時跳過過期值，可用 $O((V+E)\log(V+E))$ 時間與 $O(V+E)$ 空間；不可把它寫成 decrease-key 版本而沿用 $O(V)$ heap 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 路徑重建與選法

僅對有限且已確認的距離重建。從目標沿 π 回追到 s，再反轉；若目標 d=∞ 回傳不可達，標為 −∞ 則回傳無有限最短路。加上最多 |V| 步的防護可揭露損壞或尚未定案的父鏈。

G− 最終 t←c←a←b←s，反轉得 s→b→a→c→t，成本 1+2−2+3=4。來源 s 的答案是空路。

| 圖條件 | 選法 |
| --- | --- |
| DAG，可有負邊 | 拓序 relaxation |
| 一般圖，全非負 | Dijkstra |
| 一般圖，可能負邊 | Bellman-Ford 與可達負環檢查 |

資料來源為原筆記 p42、CLRS 第24章與 MIT04 L14/L15、MIT05 L17/L18；頁碼映射見 sources.json。

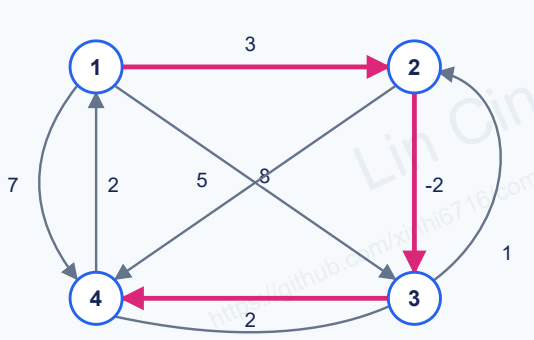


## APSP 要交付每一對的距離與路徑

輸入有向圖 G=(V,E)，n=|V|，m=|E|。邊可為負。距離矩陣每列是一個來源。無路徑為 +∞；可穿越負環的點對為 −∞，沒有有限最短路徑。

先在無負環條件推導有限距離。W 的直接邊取最小權，無邊為∞，對角線取 min(0,自環權重)，不能抹掉負自環。

**四點共用圖 G**




8 條邊；唯一負邊 2→3

2→3→4→2 = 1

1→2→3→4 = 3

1→4 = 7

3 < 7

粉紅邊標示本次重點路徑或負環；箭頭表示方向。

本段數例為自製四點圖，並非原書圖。來源 [S1 §25.1–25.3]、[S2 p4–23]、[S3 p4–23]。

## 邊數 DP 的狀態是「最多」m 條邊

$L^{(t)}[i,j]$ 是最多 t 條邊的 walk 最小權重。$L^{(0)}=I$，I 對角為0，其餘∞。

$$L^{(t+1)}[i,j] = \min_k \{L^{(t)}[i,k] + W[k,j]\}.$$

沿最後一條邊切開得到候選；每個候選也能接成可行 walk。W[j,j] 取 0 與自環權重的最小值，故 W[j,j]≤0。對最多 t 邊的 walk，接上此對角項：若為 0 就接空路，否則接負自環。結果最多 t+1 邊且成本不增加，故即使有負環也涵蓋較短的 walk；固定預算不一定等於無限制距離。

無負環時，刪掉非負圈不增加成本，最短路可取簡單路，最多 n−1 邊。這是停止在 n−1 的依據。[S1 p687–688；S2 p6–9；S3 p6–9]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Min-plus 乘法把邊數預算相加

定義 $(A\otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$。若 $A=L^{(a)}$、$B=L^{(b)}$，則 $A\otimes B=L^{(a+b)}$。任一最多 a+b 邊的 walk 可在第 min(a,實際邊數) 條邊後切開；反向可直接串接。

結合律來自同一個最小值。

$$(A\otimes B)\otimes C = A\otimes(B\otimes C), \qquad \min_{p,q}(A[i,p]+B[p,q]+C[q,j]).$$

I 是單位矩陣。這裡只用有限數與 +∞，避免 ∞+(−∞) 未定義。普通矩陣乘法的「加總、相乘」換成「取最小、相加」；不能直接沿用 Strassen 的減法。[S1 p688–691；S2 p10–14；S3 p10–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Slow 與 faster 改的是計算層數

```
Extend(A,B):
  C = new n×n matrix filled with ∞
  for i,j,k: C[i,j] = min(C[i,j], A[i,k]+B[k,j])
  return C
Slow(W):
  L = I
  repeat n−1 times: L = Extend(L,W)
  return L
Faster(W):
  L = W; t = 1
  while t < n−1: L = Extend(L,L); t = 2*t
  return L
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Min-plus 成本與負環偵測

一次 Extend 為 $\Theta(n^3)$。Slow 共 n−1 次，$\Theta(n^4)$；faster 的預算 1,2,4,…，$\Theta(n^3 \log n)$。只保存輸入與新矩陣即可用 $\Theta(n^2)$ 工作空間，不能邊讀邊覆寫同一乘法的輸入。

上述答案假設無負環。若要靠 min-plus 對角線偵測負環，預算須至少 n，因負的簡單圈可能有 n 條邊。n−1 的停止條件只用於求無負環距離。[S1 p689–692；S2 p13–14；S3 p13–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同圖核算 $L^{(2)}$ 與 $L^{(3)}$

$L^{(1)}=W$。$L^{(2)}$ 把每一對可能的最後一個中繼點全部試過。

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | 7 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(3)}$ 已是最短距離，因 n−1=3。

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(2)}[1,4]=\min(7,3+5,8+2)=7$；$L^{(3)}[1,4]=1+2=3$，路徑1→2→3→4。Faster 算 $L^{(2)}$、$L^{(4)}$，後者等於 $L^{(3)}$。



## Floyd–Warshall 改用允許中繼集合

$D^{(k)}[i,j]$ 的所有中繼點屬於 $S_k=\{1,\cdots,k\}$，端點 i,j 不受此限制。$D^{(0)}=W$。

$$D^{(k)}[i,j]=\min\{D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j]\}.$$

無負環時可取簡單最短路。若路徑不經 k，用第一項；若經 k，就在唯一出現的 k 切開，兩段的中繼點都在 $S_{k-1}$。每個候選串接後也是允許的 walk，刪掉非負圈仍可行。兩方向證明轉移等式。

原筆記用「最短路的子路徑仍最短」引出最優子結構；這裡再加上允許集合，才得到正確可執行的 DP。[S0 p18–19；S1 p693–695；S2 p15–17；S3 p15–17]

## k=0，尚未允許任何中繼點

$D^{(0)}$ 只有直接邊及空路徑。Π[i,j] 存 j 的前一點，並非中繼點編號。i=j 或無路徑為 NIL；直接邊的前驅為 i。

| $D^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | ∞ | 0 |

| $\Pi^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | NIL | NIL |



## k=1，只開放中繼點 1

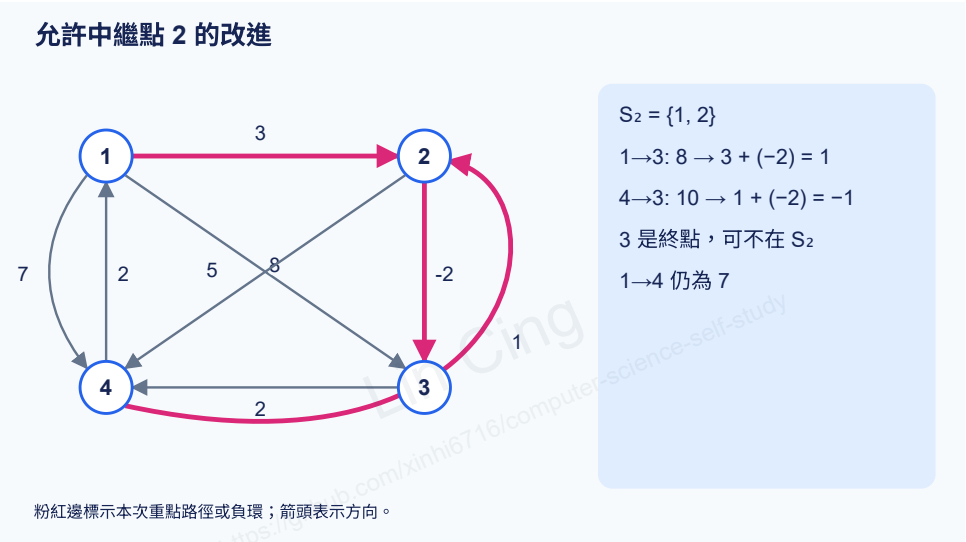
唯一改進為 4→3。原本∞，現在 4→1→3 的權為 2+8=10。$\Pi^{(1)}[4,3]=\Pi^{(0)}[1,3]=1$。端點3不必在{1}。

| $D^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | 10 | 0 |

| $\Pi^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 1 | NIL |

# k=2，透過負邊改進兩對

1→3 由8降為1，4→3 由10降為−1。兩者最後一段均為2→3，所以兩個前驅都改成2。





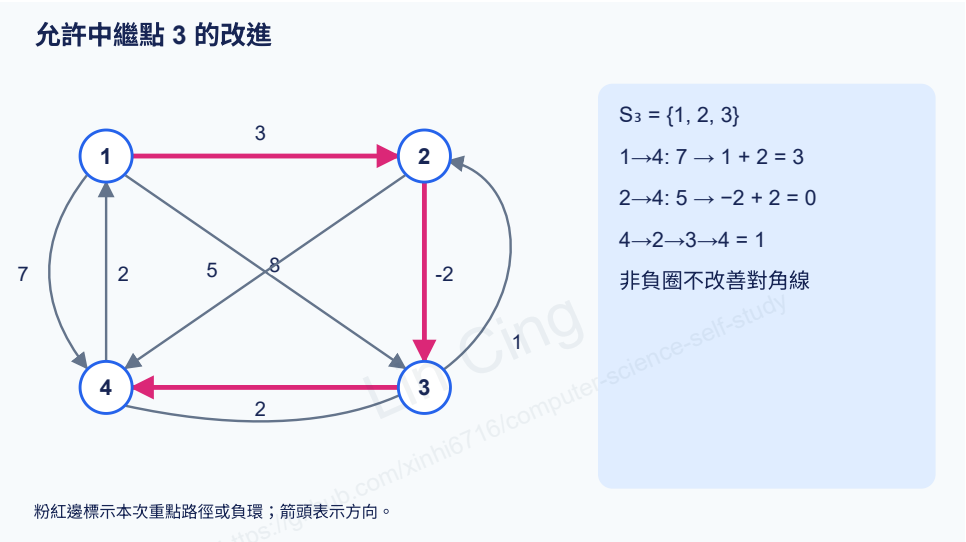

# k=2，透過負邊改進兩對 （續）

| D⁽²⁾ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| Π⁽²⁾ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

# k=3，三邊路徑第一次出現

1→4 由7降為3；2→4 由5降為0。Π[1,4]=Π[3,4]=3，不能一律填入 k 當作一般規則。






# k=3，三邊路徑第一次出現 （續）

| $D^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | ∞ | 0 | -2 | 0 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | NIL | NIL | 2 | 3 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

## k=4，完成距離並重建路徑

新增 2→1=2、3→1=4、3→2=3，其餘不變。對來源1，從終點4反向查 Π[1,4]=3、Π[1,3]=2、Π[1,2]=1，再反轉得1→2→3→4，權3。

嚴格改善才更新 Π[i,j]=Π[k,j]，相等保留舊路。先檢查可達與負環影響，才重建；零權圈下相等也更新可能抹掉有效前驅。[S1 p695–697]

| $D^{(4)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(4)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | 4 | NIL | 2 | 3 |
| 3 | 4 | 4 | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |



## k 必須在外圈，才能維持 DP 層

```
D = W
for k = 1..n:
  for i = 1..n:
    for j = 1..n:
      if D[i,k]+D[k,j] < D[i,j]:
        D[i,j] = D[i,k]+D[k,j]
        Π[i,j] = Π[k,j]
```

外圈 k 完成後，每一對都已納入 $S_k$。無負環時 D[k,k]=0，所以本層的第 k 列與第 k 欄不變；原地讀取仍是前一層所需值。i、j 可互換，任意把 k 移到內圈則無此保證。

反例只有 1→4=1、4→3=1、3→2=1。依遞增 i,j,k 只跑一遍，算(1,2)時 D[1,3]還是∞，結果維持∞；稍後才算出 D[1,3]=2，卻不回訪(1,2)。正確距離是3。[S1 p695、699 ex25.2-4；S2 p17；S3 p17]

## 負對角線是警訊，−∞ 要按點對標記

完成 Floyd 後，存在 D[t,t]<0 當且僅當存在負環。負對角線代表一條負的封閉 walk，其中必有負圈；反向而言，負圈的某個頂點在允許全部中繼後會得到負對角線。

用可達矩陣 R 判定受影響點對。

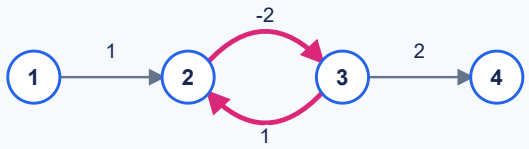
$$\delta(i,j) = -\infty \iff \exists t:\ R[i,t] \wedge D[t,t] < 0 \wedge R[t,j].$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 負對角線是警訊，−∞ 要按點對標記 （續）

**負環只影響可穿越它的點對**




2→3→2 = −1

i ∈ {1, 2, 3}

j ∈ {2, 3, 4}

這 9 對距離為 −∞

δ(4,4)=0

δ(4,1)=+∞

粉紅邊標示本次重點路徑或負環；箭頭表示方向。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

上圖是獨立反例，不是共用圖 G。只有 {1,2,3}×{2,3,4} 的9對受影響。先完成有限運算，再做標記；有負環時 Floyd 的有限輸出不是無限制距離。Johnson 選擇整體回報負環並停止。[S1 ex25.2-6、§25.3]

## Johnson 用超級來源求每個頂點的位勢

新增 s，對每個 v 加入 s→v 權0。s 沒有入邊，因此不創造新環，也不改變原頂點間的路徑。每個原頂點都可從 s 到達，任何負環都會被 Bellman–Ford 偵測。

無負環才設 h(v)=δ(s,v)。共用圖的 h=(0,0,−2,0)，因 s→2→3 給 −2，其餘不低於0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bellman–Ford 各層求得位勢

同步 Bellman–Ford 的邊數層，順序為(s,1,2,3,4)。

| 最多邊數 | 距離向量 |
|---|---|
| 0 | (0,∞,∞,∞,∞) |
| 1 | (0,0,0,0,0) |
| 2 | (0,0,0,−2,0) |
| 3、4 | (0,0,0,−2,0) |

完成 |V′|−1=4 層後，第5層無改善才通過。實作若逐邊原地鬆弛，中間向量會受邊順序影響，最終位勢相同。[S1 p702–704；S2 p22–23；S3 p22–23]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 最短路三角不等式保證重加權非負

Bellman–Ford 的位勢滿足 h(v)≤h(u)+w(u,v)。移項即得

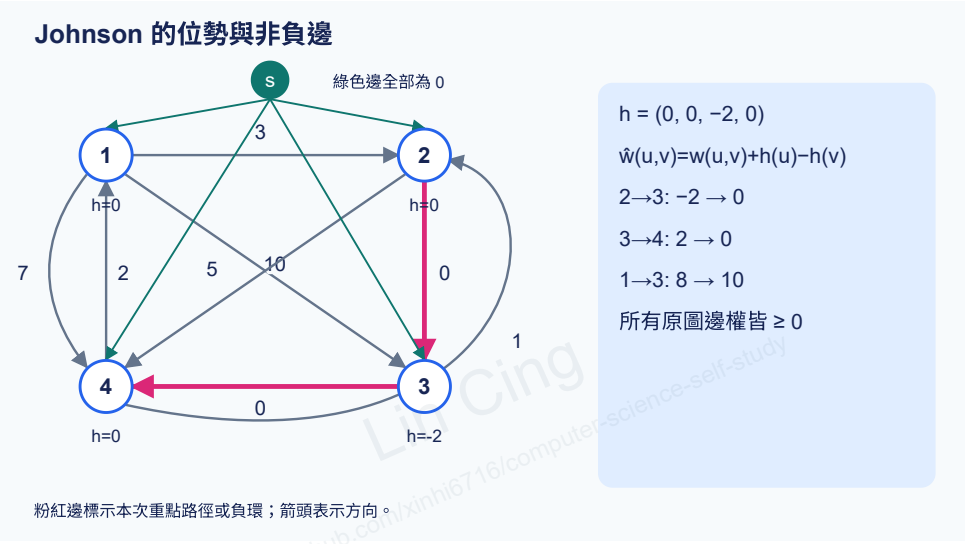
$$\widehat{w}(u,v) = w(u,v) + h(u) - h(v) \geq 0.$$

| u→v | w | h(u)−h(v) | ŵ |
|---|---|---|---|
| 1→2 | 3 | 0 | 3 |
| 1→3 | 8 | 2 | 10 |
| 1→4 | 7 | 0 | 7 |
| 2→3 | -2 | 2 | 0 |
| 2→4 | 5 | 0 | 5 |
| 3→4 | 2 | -2 | 0 |
| 4→1 | 2 | 0 | 2 |
| 4→2 | 1 | 0 | 1 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 最短路三角不等式保證重加權非負 （續）




任意 h 都能保序，只有滿足差分限制的 h 才保證非負。每邊加同一常數無法替代位勢，因路徑邊數不同。[S1 lemma25.1、p702；S2 p19–23；S3 p19–23]

## 望遠鏡消去證明同端點路徑保序

對 $p=\langle v_0,\cdots,v_r\rangle$，

$$\widehat{w}(p)=\sum_{a=1}^{r}(w(v_{a-1},v_a)+h(v_{a-1})-h(v_a))=w(p)+h(v_0)-h(v_r).$$

內部位勢逐項相消。同一對端點的所有路徑都加上同一常數，所以大小關係與平手都保留，最短路集合不變。圈的端點相同，故圈權不變，也不可能把負環消成非負圈。

例如1→3直接權8，重加權10；1→2→3原權1，重加權3，兩者都加2。1→2→3→4與1→4的端點位勢差為0，權重仍為3與7。

因此 $\delta(u,v)=\hat{\delta}(u,v)-h(u)+h(v)$。先檢查可達，∞ 保持∞，不要對無窮值做有符號整數算術。[S1 lemma25.1；S2 p19–21；S3 p19–21]



## 每個來源跑 Dijkstra，再還原原距離

來源1在重加權圖依序定案1、2、3、4。抽出1後估計為(0,3,10,7)；抽出2後為(0,3,3,7)；抽出3後為(0,3,3,3)。

所有來源得到的重加權距離如下。

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 3 | 3 |
| 2 | 2 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 | 0 |
| 4 | 2 | 1 | 1 | 0 |

每格套 $\delta=\delta-h(\text{來源})+h(\text{終點})$，還原成前面的 $D^{(4)}$。例如 $\delta(1,3)=3-0-2=1$，$\delta(3,1)=2-(-2)+0=4$。Dijkstra 的前驅路徑直接保留，因為保序證明已保證路徑相同。

檢核程式另以簡單路徑枚舉驗算全部16格，不以兩套 DP 一致當唯一證據。

## 圖密度與堆的選擇決定成本

| 方法 | 時間 | 適用條件 |
| --- | --- | --- |
| Slow min-plus | $\Theta(n^4)$ | 教學用邊數 DP |
| 重複平方 | $\Theta(n^3 \log n)$ | 教學用 min-plus 結合律 |
| Floyd–Warshall | $\Theta(n^3)$ | 矩陣輸入、稠密圖 |
| Johnson，binary heap | $O(n(m+n)\log n)$ | 稀疏鄰接串列 |
| Johnson，Fibonacci heap | $O(nm+n^2\log n)$ | 攤銷 decrease-key 為 O(1) |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 比較總成本與工作空間

顯式超級來源的 Bellman–Ford 為 $O(n(m+n))$，加上 n 次 Dijkstra 與 $\Theta(n^2)$ 還原。Fibonacci 版本稠密 $m=\Theta(n^2)$ 時 $O(n^3)$，稀疏 $m=O(n)$ 時 $O(n^2\log n)$；binary 版本稠密上界 $O(n^3\log n)$。完整輸出本來就需 $\Theta(n^2)$ 空間。

Floyd 工作空間 $\Theta(n^2)$，若保留全部教學層則 $\Theta(n^3)$。Johnson 除輸出外，鄰接串列與單來源工作空間 $O(m+n)$。

來源鍵 S0=原筆記 p18–19；S1=CLRS 第三版 §25.1–25.3，印刷 p684–705；S2=MIT 2004 lecture16 p1–23；S3=MIT 2005 lecture19 p1–23。完整路徑、讀取頁碼與內容對應見 sources.json。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapters 23–25. MIT 6.046J, Fall 2004, Lectures 13–16; Fall 2005, Lectures 16–19. Original handwritten algorithms notes (演算法_wjungle). These notes provide selected proof ideas and context; complete worked traces and diagrams are original educational synthesis.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 網路流

網路流以容量與流量守恆描述資源如何從來源傳到匯點。殘餘網路記錄增加流量與撤銷既有決策的可能性。

本章依序研究 **增廣路徑、最大流最小割與 Edmonds–Karp → push–relabel → 匹配與下界建模**。

前置知識為圖、廣度優先搜尋、不變量與漸進分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 從流量到證明與建模

| 問題 | 要檢查的條件 |
| --- | --- |
| 流量是否合法？ | 容量限制與中間頂點的流量守恆。 |
| 如何改進或證明最優？ | 殘餘路徑、割容量與最大流最小割定理。 |
| 局部操作如何收斂？ | 前流、頂點高度與 push–relabel 不變量。 |
| 如何表示新限制？ | 匹配、頂點容量與有下界的可行流。 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A01 | 網路流要交付可驗證的最優證書

給定有向網路 G=(V,E)、源點 s、匯點 t 與有限非負容量 c。求可行流 f 的最大淨流出量。

本節由原筆記六點例推進，最後同時交付值為11的流與容量11的割。來源為 [CLRS26] §26.1–26.2、[MIT04-20]、[MIT04-21]、[NOTE] 38–40頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A02 | 容量限制與守恆是不同條件

每條原邊 e 滿足 0≤f(e)≤c(e)。每個 v∈{s,t} 滿足 Σ入邊f=Σ出邊f。

定義 |f|=Σ(s,v)f(s,v)−Σ(v,s)f(v,s)。若源點有入邊，不能只算流出量。對所有內點求和，源點淨流出恰為匯點淨流入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A03 | 固定流模型，避免反平行邊混淆

本節 f 是每條原邊的非負流，不採 MIT04 的反對稱淨流記法。原例無反平行原邊。

若同時有 u→v 與 v→u，每條原邊各有自己的正向與反向殘餘邊，以 edge_id 配對。u→v 方向可同時含 c(u,v)−f(u,v) 的加流通道與 f(v,u) 的取消通道。可合併容量，但更新時必須分攤。另一做法是把一條原邊拆成 u→q→v。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A04 | 殘餘邊是允許的流量修改

原邊 u→v 的正向殘餘為 c−f，反向 v→u 的殘餘為 f。只把正殘餘放進 G_f。

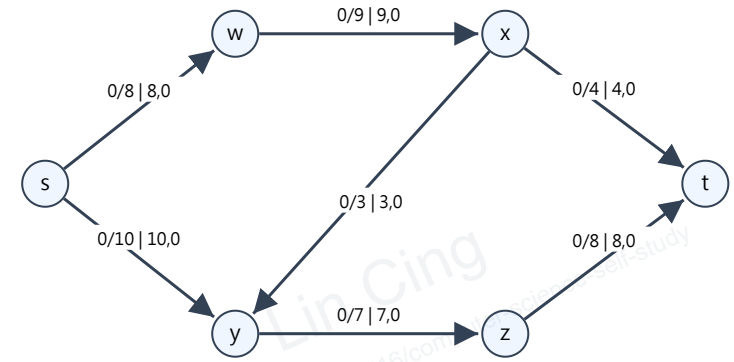
沿反向邊送 δ，意思是原邊 f 減 δ。它不是新增一條實體運輸線，也不受不存在的原邊容量0阻擋。增廣前必須保留反向邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A09-A05 | 六點網路與零流

容量依序為 sw=8、sy=10、wx=9、xy=3、yz=7、xt=4、zt=8。所有邊起始 f=0。

第0輪之後，流量 0




邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

圖中標籤為 f/c | 正向剩餘,反向剩餘。箭頭是原邊方向，反向剩餘由標籤給出。



# A09-A06 | 增廣保留可行性並增加流值

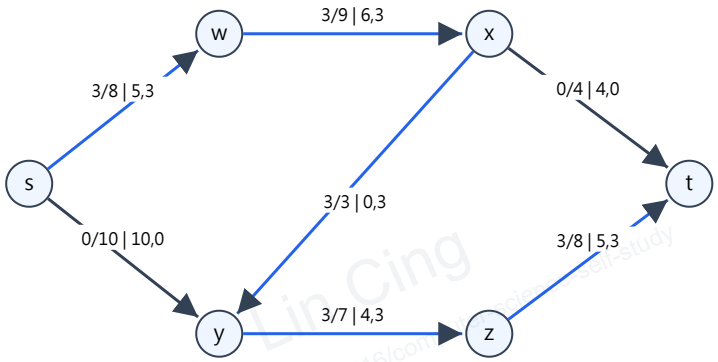
選 G_f 中簡單 s→t 路 P。δ=min_{e∈P}r(e)>0。正向原邊加δ，反向原邊減δ。

容量由 δ≤r 保證。路徑內點恰有一單位入流修改與一單位出流修改，淨平衡仍為0。s 淨流出增 δ，t 淨流入增δ，因此新流值為 |f|+δ。至少一條殘餘邊飽和。

## A09-A07 | 第1輪選一條較長路

選 $P_1$=s→w→x→y→z→t，殘餘為8,9,3,7,8，所以 $\delta_1$=3。流值3。

第1輪之後，流量 3




邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

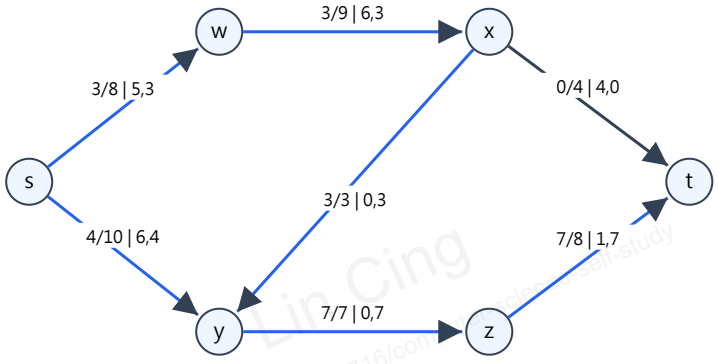
邊 xy 已飽和，卻產生 y→x 的反向殘餘3。這是一般 Ford–Fulkerson 的合法選路，並非 Edmonds–Karp。



## A09-A08 | 第2輪先填滿 yz

選 $P_2$=s→y→z→t，殘餘10,4,5，所以 $\delta_2$=4。流值7。

第2輪之後，流量 7

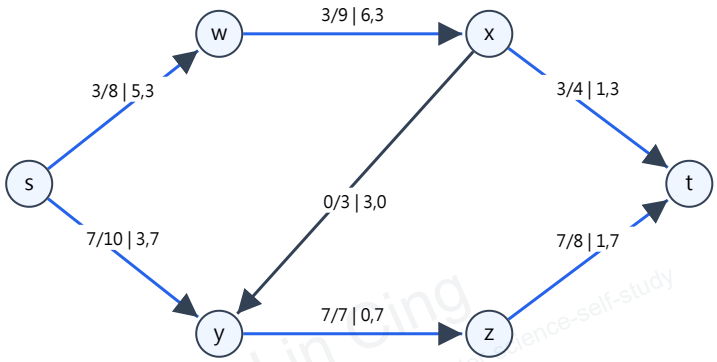



邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

此時 x 的入流3=出流3；y 的入流3+4=出流7。單看原圖從 s 到 t 的未飽和路，會錯過下一輪的取消。

# A09-A09 | 第3輪使用逆邊取消

第3輪之後，流量 10




邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

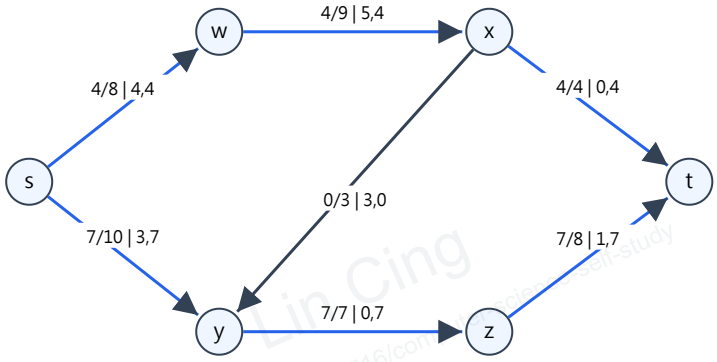
選 $P_3$=s→y→x→t，殘餘6,3,4，所以 $\delta_3$=3。y→x 是 xy 的反向邊，令 f(xy) 由3降為0。流值10。取消後 x 的入流3=出流3；y 的入流7=出流7。流值增加，單邊流量可以減少。



# A09-A10 | 第4輪與所有內點的守恆驗算

選 $P_4$=s→w→x→t，殘餘5,6,1，所以 $\delta_4$=1。最終流值11。

第4輪之後，流量 11




邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

最終 w 為4入4出，x 為4入4出，y 為7入7出，z 為7入7出。源點4+7=11，匯點4+7=11。所有邊流依 sw,sy,wx,xy,yz,xt,zt 為 (4,7,4,0,7,4,7)。

## A09-A11 | 每一輪都可逐項重算

| 輪次 | sw | sy | wx | xy | yz | xt | zt | 值 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 3 | 0 | 3 | 3 | 3 | 0 | 3 | 3 |
| 2 | 3 | 4 | 3 | 3 | 7 | 0 | 7 | 7 |
| 3 | 3 | 7 | 3 | 0 | 7 | 3 | 7 | 10 |
| 4 | 4 | 7 | 4 | 0 | 7 | 4 | 7 | 11 |

每列驗 w:sw=wx；x:wx=xy+xt；y:sy+xy=yz；z:yz=zt。check.py 對全部輪次與16種 s–t 割逐一驗算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A12 | 割的容量與淨流不能混算

割 (S,T) 滿足 s∈S、t∈T、T=V\S。

c(S,T)=Σ_{u∈S,v∈T}c(u,v)，只加 S→T 原邊容量。f(S,T)=Σ_{S→T}f−Σ_{T→S}f。

將 S 內頂點的出流減入流加總，內邊兩兩消去，內點守恆只留下 s，所以 f(S,T)=|f|。再用容量與非負性得 |f|≤c(S,T)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A13 | 最優流與最小割的雙向證明

若存在割使 |f|=c(S,T)，任何可行流 g 都有 |g|≤c(S,T)=|f|，故 f 最大且此割最小。

反方向，若 f 最大，存在增廣路就能增加流值，矛盾，故沒有增廣路。令 S 為 G_f 中從s可達的點。t不可達，所以形成合法割。每條 S→T 原邊飽和，否則可跨割；每條 T→S 原邊流為0，否則其反向殘餘可跨割。因此 |f|=f(S,T)=c(S,T)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A14 | 例子的停止證書

最終 G_f 從 s 可達 S={s,w,x,y}，T={z,t}。sw,sy,wx,xy 的正殘餘維持這些可達性，但 yz、xt 正殘餘均為0。

割容量 c(S,T)=c(yz)+c(xt)=7+4=11。沒有 T→S 正流。已構造可行流11且任一流≤11，故最大流=最小割=11。不能把 zt 的剩餘1誤當可增廣量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A15 | Ford–Fulkerson 的整數終止性

```
f[e] = 0 for each original edge e
while find_path(G_f, s, t) returns P:
    delta = min(residual[arc] for arc in P)
    update each paired original edge along P by +delta or -delta
return f, reachable(G_f, s)
```

初始化0；只要有增廣路就依瓶頸增廣。整數容量使每輪殘餘與δ為整數，δ≥1。流值受有限割容量限制，因此至多 F*=|f*| 輪。

以鄰接串列找路並更新 O(E) 時，時間 O(E(1+F*))，通常寫 O(EF*)，另有 O(V+E) 初始化。這是偽多項式界，因 F* 的數值可對容量輸入位數呈指數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A16 | 選路可使常數大小網路跑 2U 輪

四點網路 su,ut,sv,vt 容量均為 U，uv 容量1。交替選 s→u→v→t 與 s→v→u→t。每輪δ=1，後一條取消 uv，直到2U輪才達最大值2U。

有理容量可乘公分母化整數，會終止但此界仍非輸入位數多項式。任意實數容量且任意選路的 FF 可能不終止，不能由「每輪增加」推出有限輪，也不能保證非終止序列的極限最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A17 | Edmonds–Karp 固定使用 BFS

每輪在正殘餘圖用 BFS 找邊數最少的 s→t 路，不是找容量最大、權重最小或瓶頸最大的路。

六點例可選 s→w→x→t，δ=4；再選 s→y→z→t，δ=7；兩輪值11。這與前四輪 FF 的合法路徑選擇不同。保留反向邊仍是必要的通用實作規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A18 | BFS 距離不下降的完整理由

令 d,d′ 是一輪前後從s的邊數距離，不可達為∞。反設有點距離下降，沿新最短路選第一個下降點v及前驅u，故 d(u)≤d′(u)=d′(v)−1。

若 (u,v) 舊已存在，d(v)≤d(u)+1≤d′(v)，矛盾。若舊不存在，它只能是本輪最短增廣路上 (v,u) 新生成的反向邊；因此 d(u)=d(v)+1，推出 d(v)=d(u)−1≤d′(v)−2，仍矛盾。故所有距離不下降，變為∞後也不會重新可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A19 | 關鍵邊的再次飽和至少升兩層

一輪至少飽和一條關鍵殘餘邊 (u,v)，其 r(u,v)=δ 且 d(v)=d(u)+1。飽和後它消失。

在它重新飽和前，必先沿 (v,u) 增廣使它再出現。此時 d_new(u)=d_new(v)+1≥d_old(v)+1=d_old(u)+2。距離單調確保之後也不下降。能出現在增廣路上的有限距離至多 V−1，故每個方向／edge_id 只有 O(V) 次關鍵事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A20 | 將事件數換成完整時間界

殘餘邊方向至多2E，每輪至少一個關鍵事件，每邊 O(V) 次，總增廣 O(VE)。

每輪 BFS 與路徑更新 O(V+E)。先移除孤立點或用可達子圖使相關 V=O(E)，得到每輪O(E)，總 $O(VE^2)$，外加O(V+E)初始化。容量大小不影響輪次界，在精確實數算術模型也成立；位元運算成本另計。

停止時回傳可行流與可達割。這兩個物件提供獨立於選路歷程的最優驗收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P01｜暫存流量，再恢復守恆

Ford–Fulkerson每次維持合法flow；push-relabel允許中間點暫存excess。局部push只看一條殘餘邊，relabel提高一個點。

本節以CLRS第3版§26.4–26.5為準。note掃描p38提供逆流取消與殘餘容量的前置概念，不把它當作push-relabel的來源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P02｜preflow與excess的精確語義

對原圖每邊維持0≤f(u,v)≤c(u,v)。令e(u)=Σ_v f(v,u)−Σ_v f(u,v)。preflow要求所有u≠s有e(u)≥0。active只指u∈{s,t}且e(u)>0。

正向殘餘容量為c−f，反向為f。本例無反平行原邊，反向push直接減少對應原邊flow。source excess可以為負；sink雖可有正excess，永不discharge。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P03｜height不是即時最短距離

合法height滿足h(s)=n、h(t)=0，且每條殘餘邊(u,v)都滿足h(u)≤h(v)+1。它是證明用標籤，不保證等於到t的距離。

若存在簡單殘餘s→t路徑，長度k≤n−1。沿路相加得n=h(s)≤h(t)+k=k，矛盾。因此合法height下始終沒有殘餘s→t路徑。此時preflow仍可能不守恆，不能直接宣告完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P04｜初始化先飽和source出邊

設所有flow、excess與height為0，改h(s)=n，將每條(s,v)飽和並更新e(s)、e(v)。這個步驟是初始化，不受一般PUSH的height差1條件限制。

source正向邊已無殘餘容量；反向邊v→s符合0≤n+1，其他殘餘邊兩端height為0。因此同時建立preflow與合法height。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P05｜push的適用條件與更新

PUSH(u,v)要求u active、c_f(u,v)>0且h(u)=h(v)+1。設δ=min(e(u),c_f(u,v))，更新原邊flow或取消逆向flow，再令e(u)−=δ、e(v)+=δ。

δ不超過excess與容量，故preflow保持。新出現的逆邊v→u滿足h(v)=h(u)−1≤h(u)+1。飽和push使該殘餘邊消失；非飽和push使e(u)=0。兩者同時成立時歸類為飽和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P06｜沒有合法出邊才relabel

RELABEL(u)要求u active，且所有殘餘鄰居v都有h(u)≤h(v)，等價於沒有admissible出邊。設h(u)=1+min{h(v):c_f(u,v)>0}。

excess>0意味存在正流入邊，其逆邊提供至少一個殘餘鄰居，所以min不是空集合。新height嚴格增加；所有出邊仍合法，入邊右側height變大只會放鬆限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P07｜generic正確性與終止分開證明

只要存在active點，若有admissible邊就push，否則relabel適用。故停止時所有非s/t點excess為0，preflow成為flow。合法height排除殘餘s→t路徑，由max-flow min-cut得最大流。

這是「若終止則正確」。後面的操作次數上界才保證必定終止。證明不依容量是否整數，也不以容量值當迭代上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P08｜height界與relabel成本

任一active u都有殘餘路徑回s。反證可取由u可達集合S，若s∉S且無殘餘出邊，跨入S的原flow必為0，而Σ_{v∈S}e(v)≤0，與u active矛盾。

取簡單u→s路徑，長≤n−1，得h(u)≤h(s)+n−1=2n−1。height單調且只對active點relabel，因此每點O(n)次、總O(n²)次。每次掃N(u)求min需O(deg(u))，全部relabel為Σ_u O(n deg(u))=O(nm)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P09｜飽和push為何只有O(nm)

一次u→v飽和後，再次同向飽和前必先有v→u的push補回殘餘容量。第一次h(u)=h(v)+1，反推時h′(v)=h′(u)+1≥h(u)+1。

再次u→v可push時h″(u)=h″(v)+1≥h(u)+2。因此每方向重複飽和會強迫height至少增加2，每個殘餘方向O(n)次。原圖m條邊產生至多2m個方向，合計O(nm)。每次push本身O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P10｜非飽和push的位能帳

令Φ=Σ_{u active}h(u)，初值0且非負。非飽和push移除u，至多新增v。因h(u)=h(v)+1，Φ至少降1；若v為s/t則不加入。

一次relabel或飽和push使Φ增加至多O(n)。relabel $O(n^2)$次，飽和push O(nm)次，故非飽和push $O(n^3+n^2m)$。刪除與s/t無關的孤立成分後n=O(m)，得到$O(n^2m)$。容量大小不在此上界中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P11｜從操作數轉為實際時間

不能把「有$O(n^2m)$次操作」直接當時間證明。以雙向殘餘arc配對實作O(1)的push；保存excess與active容器以O(1)選active點。

每點保留current arc，掃過整個N(u)才relabel並重設。每點O(n)輪，總掃描與relabel成本O(nm)。active點可任選並discharge，所以仍是generic的合法排程。加上push次數得到$O(n^2m)$，初始化O(n+m)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P12｜relabel-to-front保存所有中間點

L是V−{s,t}全部頂點的鏈結表，包含e=0的點。N(u)包含原圖所有入邊與出邊的另一端，固定順序，不隨殘餘容量刪除。每點有跨呼叫保留的current指標。

DISCHARGE(u)重複至e(u)=0。current=NIL時relabel並重設表首；current admissible時push且不前進；其餘情形current移到下一鄰居。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P13｜前移規則與current arc的合法性

記old=h(u)，DISCHARGE(u)後若h(u)>old，才把u移到L表首。下一個u必為移動後的u.next。未升高時照原順序前進；不把零excess點刪除。

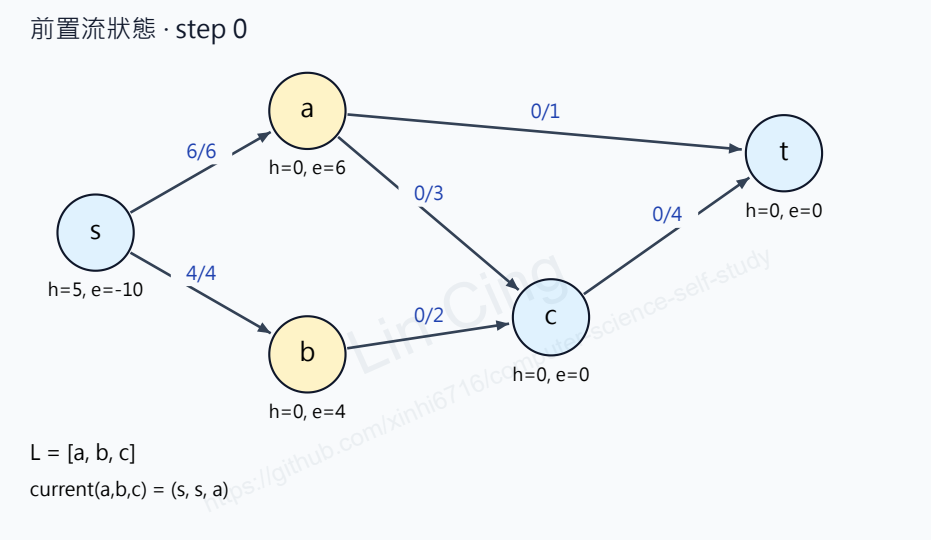
current跨呼叫保留為何安全？push不創造admissible邊；別點relabel後沒有admissible入邊。故u尚未relabel的一輪內，已跳過的u出邊不會變可用。掃到NIL才可斷定relabel適用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P14｜固定五點例與初態

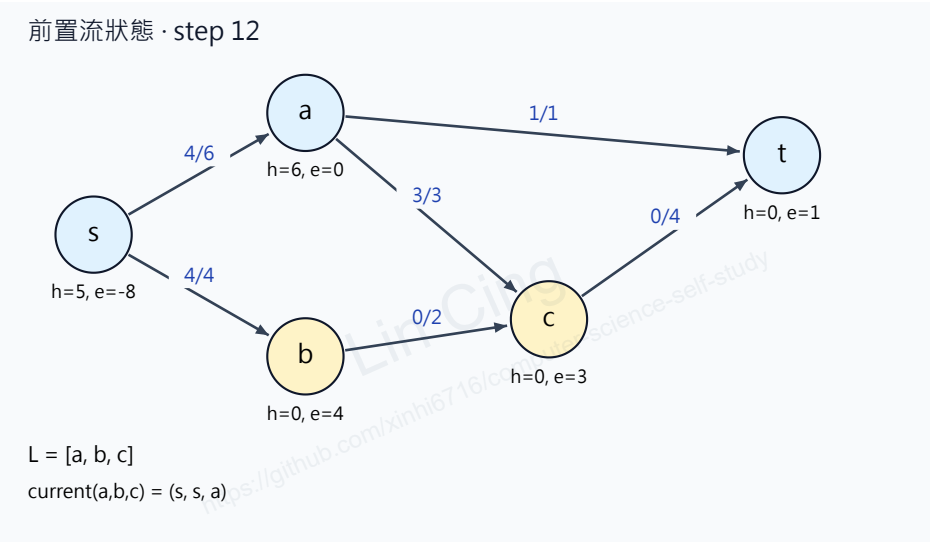
V=(s,a,b,c,t)，n=5。原邊容量為sa=6、sb=4、ac=3、at=1、bc=2、ct=4。固定N(a)=[s,c,t]、N(b)=[s,c]、N(c)=[a,b,t]，L=[a,b,c]。




step 0的h=(5,0,0,0,0)、e=(-10,6,4,0,0)、current=(s,s,a)、L=[a,b,c]。

## P15｜a先送出，再退還source

a在height 1送3給c、送1給t，仍有excess 2。此時只有a→s殘餘出邊，relabel到6後退2給s。source高度固定5，故退流的height差仍為1。



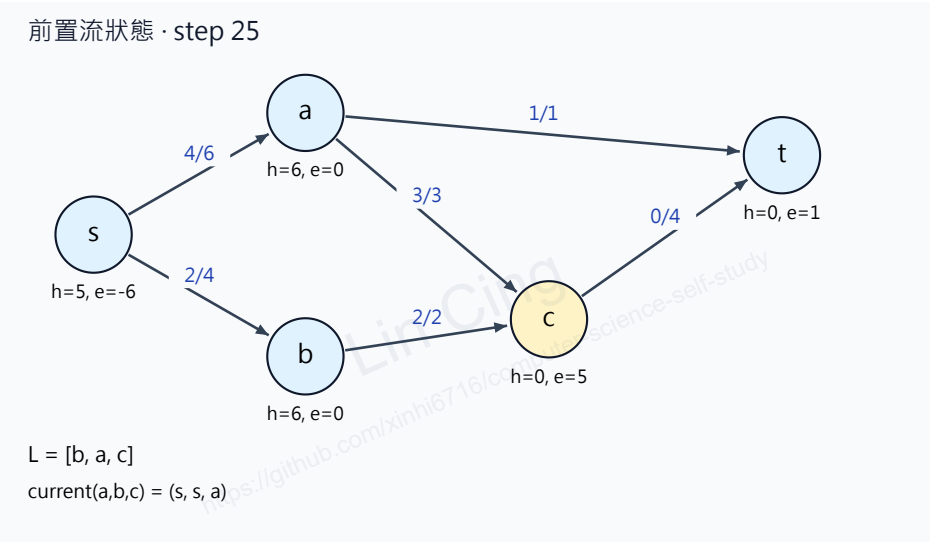



## P15｜a先送出，再退還source （續）

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 5 | relabel並重設arc a | 5,1,0,0,0 | -10,6,4,0,0 | s,s,a | a,b,c |
| 7 | push a a→c δ=3 | 5,1,0,0,0 | -10,3,4,3,0 | c,s,a | a,b,c |
| 9 | push a a→t δ=1 | 5,1,0,0,0 | -10,2,4,3,1 | t,s,a | a,b,c |
| 11 | relabel並重設arc a | 5,6,0,0,0 | -10,2,4,3,1 | s,s,a | a,b,c |
| 12 | push a a→s δ=2 | 5,6,0,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |
| 14 | 移至表首 a | 5,6,0,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |

# P16｜b前移，c再把多餘量退給a

b送2給c、退2給s後前移，L=[b,a,c]。a的零excess discharge立即返回，之後處理c。c先送4給t，再升到7退1給a；c前移後L=[c,b,a]。






# P16｜b前移，c再把多餘量退給a （續）

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 18 | relabel並重設arc b | 5,6,1,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |
| 20 | push b b→c δ=2 | 5,6,1,0,0 | -8,0,2,5,1 | s,c,a | a,b,c |
| 22 | relabel並重設arc b | 5,6,6,0,0 | -8,0,2,5,1 | s,s,a | a,b,c |
| 23 | push b b→s δ=2 | 5,6,6,0,0 | -6,0,0,5,1 | s,s,a | a,b,c |
| 25 | 移至表首 b | 5,6,6,0,0 | -6,0,0,5,1 | s,s,a | b,a,c |
| 32 | relabel並重設arc c | 5,6,6,1,0 | -6,0,0,5,1 | s,s,a | b,a,c |
| 35 | push c c→t δ=4 | 5,6,6,1,0 | -6,0,0,1,5 | s,s,t | b,a,c |
| 37 | relabel並重設arc c | 5,6,6,7,0 | -6,0,0,1,5 | s,s,a | b,a,c |
| 38 | push c c→a δ=1 | 5,6,6,7,0 | -6,1,0,0,5 | s,s,a | b,a,c |
| 40 | 移至表首 c | 5,6,6,7,0 | -6,1,0,0,5 | s,s,a | c,b,a |

# P17｜終態、割證書與完整操作表

a保留current=s，以height 6再退1給s。最後e=(-5,0,0,0,5)，流值5。割S={s,a,b,c}、T={t}容量at+ct=1+4=5，與流值相等。

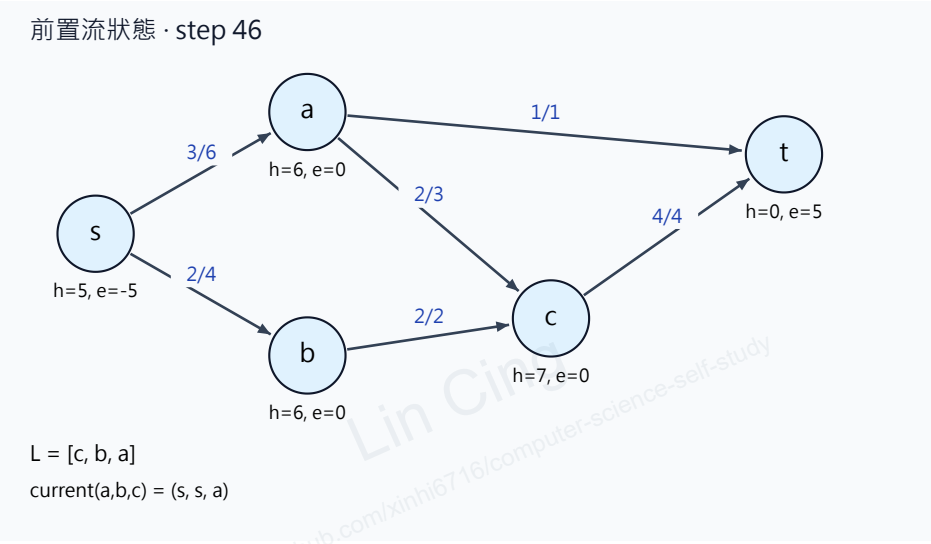
本章詳解附47列完整表，含每次arc前進、discharge起訖與list前移。圖中step直接對應同編號表列。

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 44 | push a a→s δ=1 | 5,6,6,7,0 | -5,0,0,0,5 | s,s,a | c,b,a |
| 46 | 終止 | 5,6,6,7,0 | -5,0,0,0,5 | s,s,a | c,b,a |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# P17｜終態流量與守恆




終態邊流為sa=3、sb=2、ac=2、at=1、bc=2、ct=4；各中間點的入流等於出流。

## P18｜relabel-to-front為何是O($n^3$)

外層迴圈每次測試時，L是中間點admissible network的拓樸序，且目前u之前皆為零excess。邊的height降1，故圖無環。初態所有中間點height相同，且u在表首。

若discharge沒有relabel，push不創造admissible邊，因此不會送至u之前；u清空後前進，兩項仍成立。若u曾relabel，最後u無admissible入邊，且push不創造此邊；移u至表首後，原有其他點的邊順序不變，u已清空，下一點之前只有u，兩項恢復。掃到表尾時全體中間點excess為0；結合合法height排除增廣路徑，得到最大流。

這些性質不要求在discharge中成立：step 37時c→a admissible但c在a之後；step 38時a的excess為1；step 40將c前移後恢復。

相鄰relabel間至多O(n)次DISCHARGE，共O($n^2$)次relabel，故O($n^3$)次呼叫。relabel、arc前進及飽和push共O(nm)；每次DISCHARGE至多一個非飽和push，因它清空u。總時間O($n^3$+nm)=O($n^3$)，簡單圖有m=O($n^2$)。



## 把需求寫成流的限制

本節用非負弧流 x(e) 表示原始有向邊上的運量，0≤x(e)≤c(e)。若主章用反對稱淨流 f，則 f(u,v)=x(u,v)−x(v,u)。帶下界時必須保留每條弧的 x，不能只用淨流抹去兩方向的義務。

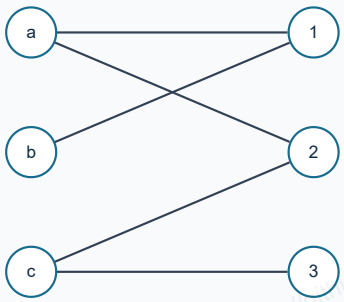
每次歸約都證明原解能轉成流、目標值相等、流能還原原解。對離散選擇，還要確認整數流。

## 二分匹配的建網

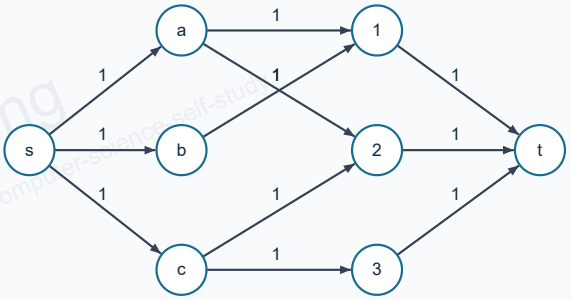
給定 G=(L∪R,E)，L、R 不交且每條邊跨兩側。加新點 s,t，把每條無向邊定向為 L→R，加入 s→u 與 v→t。所有容量為 1。

例 L={a,b,c}，R={1,2,3}，E={a1,a2,b1,c2,c3}。轉換後有 8 點、11 弧。圖左為原問題，圖右為容量網。

原二分圖




轉換網路，全部容量 1




解 {a2,b1,c3}，流值 3，割 {s} 容量 3。



## 匹配到流的映射

取 M={a2,b1,c3}。每條 uv∈M 沿 s→u→v→t 送 1，其餘弧送 0。匹配的端點不重複，所以每條 source 弧與 sink 弧至多使用一次。

內點每次收到 1 就送出 1，容量及守恆都成立。跨 L→R 的總流為 |M|=3，故流值是 3。割 S={s} 的容量為 3，因此這個流及匹配都最優。

## 整數流到匹配的映射

給定整數流 x，取 M={uv∈E:x(u,v)=1}。因 s→u 容量為 1，守恆迫使 u 至多一條出弧有正整數流。v→t 同理使每個 v 至多被選一次。故 M 是匹配且 |M|=|x|。

從零流執行 Ford–Fulkerson，整數容量使每次殘餘容量、瓶頸及更新後流都為整數。這證明存在整數最大流，不表示每個最大流都是整數。例 $K_{2,2}$ 的四條中間弧各送 1/2，仍是值 2 的最大流，但正流弧不能直接當匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 匹配失敗的割證書

若 L={a,b,c}、R={1,2,3} 且 E={a1,b1,c2,c3}，a 與 b 只能爭取 1。可取 {a1,c2}，大小 2。

令 S={s,a,b,1}。從 S 到補集只有 s→c 與 1→t，各容量 1，所以割容量 2，最大匹配也是 2。集合 A={a,b} 滿足 |N(A)|=1<2，顯示不能匹配整個 L。不能把「極大」匹配誤當「最大」匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 頂點容量必須限制總通過量

原網 s→v 容量 5、v→t 容量 5，但 v 每單位時間最多處理 3。單獨限制每條鄰邊為 3，不能一般化處理多條入邊。正確限制是 Σ_u x(u,v)≤b(v)。

把 v 拆為 v_in,v_out，以容量 b(v)=3 的內部弧相連。每條入弧改接 v_in，每條出弧從 v_out 出發。例中最大流 3，割 {s,v_in} 的容量為 3。

原網．v 的容量為 3

拆點．以內部弧限制總量

最大流 3。割 {s,v_in} 的容量 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 拆點的雙向證明與尺寸

原可行流在每條改接弧維持原值，並令內部弧流等於 v 的總流入。守恆及頂點容量保證轉換可行。反過來，v_in 與 v_out 的守恆迫使內部弧等於總流入及總流出；合併兩點就還原原流。

若全部 n 點都拆開，得到 2n 點、m+n 弧。只拆 k 個受限內點則為 n+k 點、m+k 弧。若 source 或 sink 也有總容量，必須拆開並把外部 source 接到 s_in、外部 sink 接在 t_out。

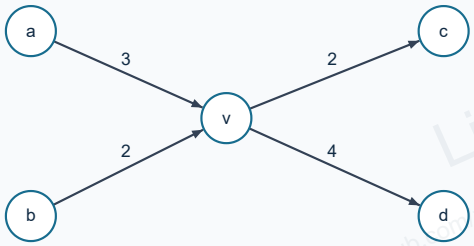
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 多源多匯加入超點

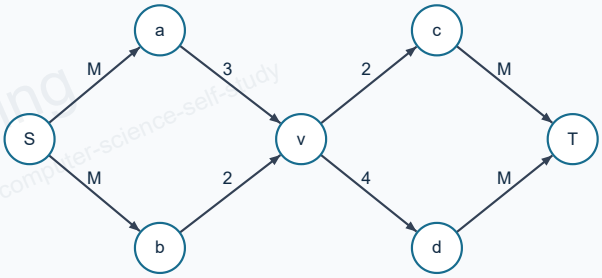
原問題有供應點 a,b、需求點 c,d，中間點 v。弧容量為 a→v=3、b→v=2、v→c=2、v→d=4。源點沒有原始入弧，匯點沒有原始出弧。

加入 S→a,S→b,c→T,d→T。取大容量 M=11，等於原弧容量總和。送 a→v→c 2、a→v→d 1、b→v→d 2，總量 5。割 {S,a,b} 容量 3+2=5，證明最優。

原多源多匯網




超點．人工弧容量 M=11




流值 5．送到 c 為 2、d 為 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 總量最大與配額可行不同

原解延伸時令 S→a 等於 a 的淨流出、c→T 等於 c 的淨流入，超點之外便全部守恆。移除超點則還原相同總量。這個證明要求各供應點淨流出非負、需求點淨流入非負。

若 a,b 必須供應 3,2，c,d 必須接收 2,3，新增弧容量分別設為 3,2,2,3，並檢查最大流是否等於 P=5。總新增 source 容量與 sink 容量皆為 5，值 5 等價於每一配額弧飽和。上一例可行。若 c 的配額改為 3、d 改為 2，則 v→c 只有 2；割 {S,a,b,v,d} 容量 2+2=4，配額失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 有限大容量必須有上界依據

最大流模型中，新增弧只需不壓低原問題最佳值。原弧有限非負容量時，M=Σ_e c(e) 是安全但可能寬鬆的選擇。也可用已證明的原網總輸送上界，例如本例 3+2=5。M=4 放在每條超點弧仍足夠此例，但不能憑感覺選常數。

帶需求的輔助最大流只需送 D 單位，故不應限制需求的人工連接弧可取容量 D。若為了用割嚴格排除人工弧，須選 M 大於已知候選割上界，例如整數模型用 U+1。M≥U 足夠保最佳值，未必能排除同值人工割。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 下界先轉成每點不平衡量

可行 circulation 在每個點都守恆，且 ℓ(e)≤x(e)≤u(e)，其中 0≤ℓ≤u。令 y(e)=x(e)−ℓ(e)，新容量為 u−ℓ。定義 d(v)=Σ_in ℓ−Σ_out ℓ。

逐點代入原守恆得到 Σ_in(y+ℓ)=Σ_out(y+ℓ)，因此 Σ_out y−Σ_in y=d(v)。若 d(v)>0，y 必須淨流出 d(v)；若 d(v)<0，y 必須淨流入 −d(v)。每條下界在一個頭點加、尾點減，故 Σ_v d(v)=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 超點方向由守恆推導

加新源 S*、新匯 T*。若 d(v)>0，加 S*→v 容量 d(v)；若 d(v)<0，加 v→T* 容量 −d(v)。設 D=Σ_{d>0}d=Σ_{d<0}(−d)。

當需求弧飽和，d>0 的點滿足 Σ_in y+d=Σ_out y；d<0 的點滿足 Σ_in y=Σ_out y−d；d=0 的點照常守恆。三種情形都導回 Σ_out y−Σ_in y=d。

可行的充要條件是輔助最大流值 D，等價於所有 S* 出弧飽和。T* 入弧總容量同為 D，所以它們也全部飽和。刪超點並還原 x=ℓ+y。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

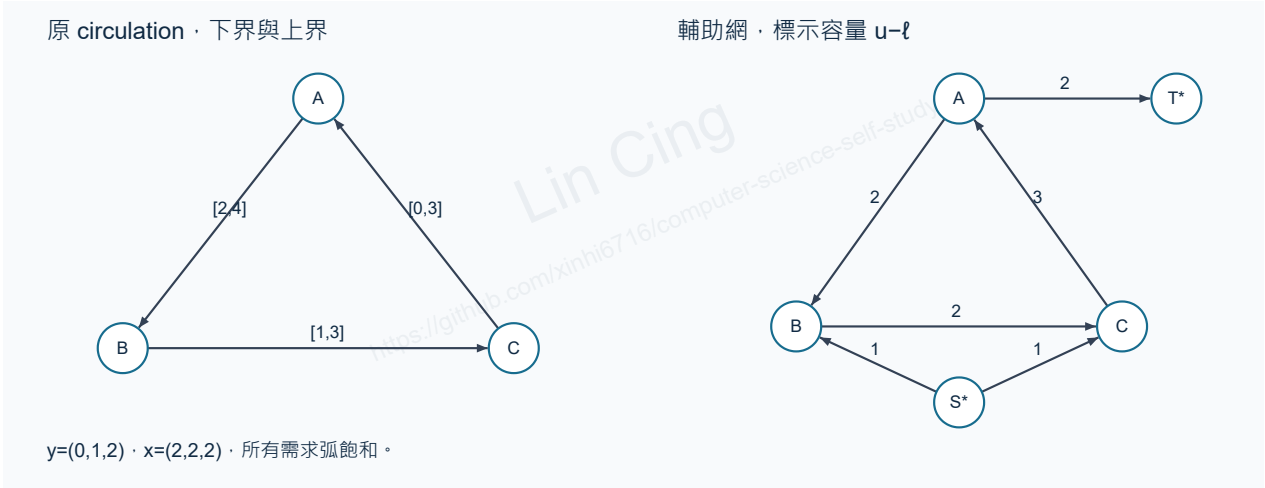


## 有下界循環的完整可行例

原圖 A→B:[2,4]、B→C:[1,3]、C→A:[0,3]。方括號為 [ℓ,u]。d(A)=0−2=−2，d(B)=2−1=1，d(C)=1−0=1，D=2。

新容量依序是 2,2,3，並加 S*→B=1、S*→C=1、A→T*=2。沿 S*→B→C→A→T* 送 1，再沿 S*→C→A→T* 送 1。得到 y=(0,1,2)，還原 x=(2,2,2)。

A、B、C 各入 2、出 2，且三條弧均符合上下界。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 不可行例要展示不足的割

保持上一例，但把 C→A 改成 [0,1]。d 與 D 不變。S*→C→A→T* 可送 1，之後 C→A 飽和。

取輔助割 S={S*,B,C}，補集為 {A,T*}。唯一出割弧 C→A 容量 1，故最大流至多 1<D=2。原網也能直接看出 A 至少必須送出 2，卻最多只能收到 1，違反守恆。不能以「所有剩餘容量非負」判斷下界可行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從帶下界 s-t 流到 circulation

先限於要求流值非負的 s-t 流。加回弧 t→s，範圍 [0,M]，使所有點都守恆；回弧流量等於原 s-t 流值。若要測試恰好 F，改用 [F,F]，再套前述需求歸約。

例 s→a:[1,3]、a→t:[1,2]，要求 F=2。加入 t→s:[2,2]，得到 d(s)=1、d(a)=0、d(t)=−1。輔助路 S*→s→a→t→T* 送 1，原兩弧還原為 2，回弧為 2。若 F=3，a→t 上界 2，無可行解。

只找可行流值時，可用 M=Σ 原弧上界。若問題允許負流值，單一非負回弧未涵蓋該情況，必須另建模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 還原答案時逐項驗收

先核對原問題的選擇或配額，再核對每條原弧的上下界與每點守恆。最後給出等值割、需求弧飽和，或失敗割作證書。整數性只在所有轉換後容量為整數且求得整數流時使用。

一般實數容量的可行流可以是分數。即使所有容量為整數，任意可行流或任意最大流也可能分數。匹配讀出必須選整數最大流。帶整數上下界的可行 circulation 可透過整數輔助流還原成整數解。

來源對照採 CLRS 26.1、26.3 與習題 26.1-7、26.2-5、26.2-6、26.3-2。下界歸約及本節數例為自行推導，check.py 以枚舉及割逐一核對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 26, Maximum Flow. Original handwritten algorithms notes (演算法_wjungle), network-flow pages. MIT 6.046J, Fall 2004, Lectures 20 and 21, Maximum Flow and Edmonds–Karp.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 計算幾何

如何用精確的幾何判定，設計有效率且正確的演算法？

本章依序學習 **方向判定 → 線段相交與掃描線 → 凸包 → 最近點對**。前置知識包括排序、平衡搜尋樹、分治法與漸近分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 從幾何圖形到演算法

| 問題 | 學習重點 |
| --- | --- |
| 方向與線段相交 | 行列式符號、邊界與數值精度。 |
| 掃描線 | 事件順序、狀態與相鄰性。 |
| 凸包 | 排序、堆疊不變量與退化情況。 |
| 最近點對 | 分治、帶狀區域與幾何打包。 |

每個結果都依賴明確的輸入假設與正確性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用局部符號回答幾何問題

本節以平面點與閉線段為輸入。先固定座標與相交語義，再以方向判定組成線段相交測試，最後用掃描線回答「是否存在一對相交線段」。

所有數值例使用精確整數或有理數。時間界先按每次精確算術為常數的模型計算；大整數位元成本另計。來源為 CLRS 33.1–33.2 與 MIT 2004 L17；原筆記 PDF43–47 提供幾何章脈絡與轉向圖。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 左轉的正號取決於公式順序

固定 x 軸向右、y 軸向上。

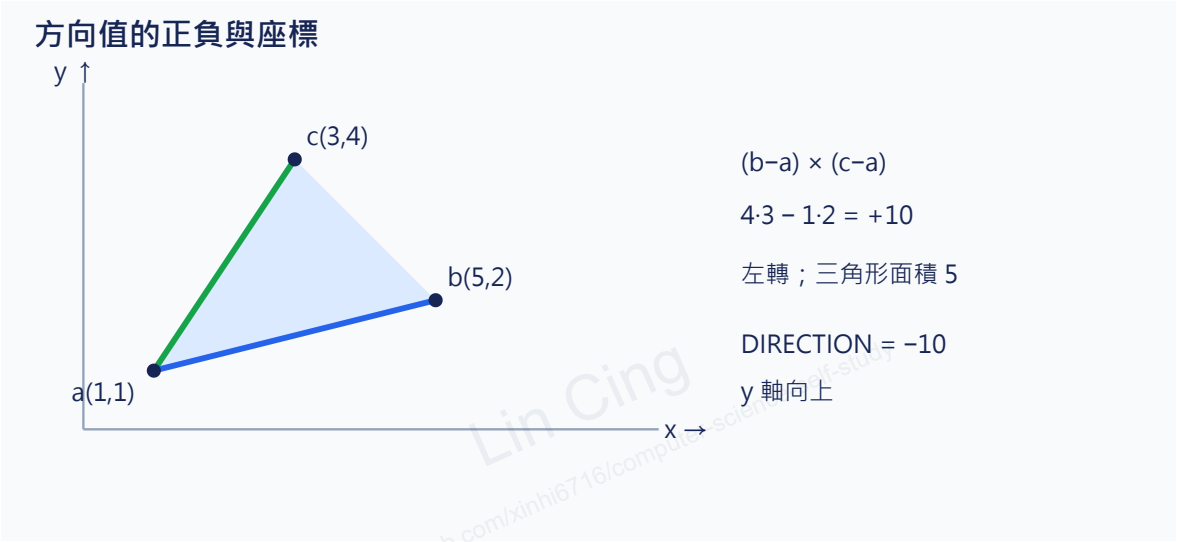
$$\operatorname{orient}(a,b,c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

正值表示從 a 經 b 到 c 左轉，負值為右轉，零表示共線。絕對值是平行四邊形面積，三角形面積為其一半。

CLRS 的 `DIRECTION(a,b,c)` 使用 $(c-a)\times(b-a)$，恰為本式負值。引用書中不等號時必須一起換號。SVG 畫面向下的 y 軸由繪圖轉換處理，不改變數學慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 左轉的正號取決於公式順序 （續）




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 實算一次行列式

令 a=(1,1)、b=(5,2)、c=(3,4)。差向量 b−a=(4,1)，c−a=(2,3)。

$$\operatorname{orient}(a,b,c) = 4 \cdot 3 - 1 \cdot 2 = 10 > 0.$$

因此 c 在有向直線 ab 的左側，三角形面積為 5。交換 b,c 得 −10；平移三點不改變差向量。若 c=(9,3)，則差向量為 (8,2)，行列式 4・2−1・8=0，僅能推得共線，不能推得 c 在閉線段 ab 內。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 閉線段相交與嚴格交叉

閉線段 $ab = \{a + t(b - a) : 0 \le t \le 1\}$ 含端點，也允許 a=b 的零長度線段。

本節將「嚴格交叉」定義為兩個非共線線段的內部單點交會。若題目把共線內部重疊也稱為嚴格相交，必須另訂規格。

| 幾何情形 | 閉線段相交 | 嚴格交叉 |
| --- | --- | --- |
| 兩內部穿越 | 是 | 是 |
| 端點接觸、T 形接觸 | 是 | 否 |
| 共線重疊 | 是 | 否 |
| 點線段落在另一段上 | 是 | 否 |
| 僅延長線相交 | 否 | 否 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 閉線段相交與嚴格交叉 （續）

## 閉線段邊界案例

嚴格交叉　端點接觸　共線重疊　點線段

藍、紅為輸入；綠為共同線段；金點為接觸點。

四例皆為閉段相交；只有第一例是嚴格交叉。

端點、共線與零長度必須由規格明確涵蓋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## on 必須同時檢查共線與兩軸界限

```
on(a,b,p):
    return orient(a,b,p) == 0 and
           min(ax,bx) <= px <= max(ax,bx) and
           min(ay,by) <= py <= max(ay,by)
```

a=(2,1)、b=(2,5)、p=(2,8) 雖共線且通過 x 界限，仍不在 ab 上，因此不能刪掉 y 界限。若 a=b，兩軸界限共同要求 p=a，故點線段也有定義。

CLRS 的 ON-SEGMENT 將「已知共線」交給呼叫端保證。本講義將它合併到 on，避免單用包圍盒就誤判。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 四個方向值完成測試

記 o1=orient(a,b,c)、o2=orient(a,b,d)、o3=orient(c,d,a)、o4=orient(c,d,b)。

```
opposite(u,v): return (u<0 and v>0) or (u>0 and v<0)
intersect(a,b,c,d):
    compute o1,o2,o3,o4
    if opposite(o1,o2) and opposite(o3,o4): return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

第一個分支恰為嚴格交叉；第二分支處理端點、共線重疊與零長度。比較正負，不直接乘 o1・o2，避免再增加乘積溢位風險。垂直線段不必求斜率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 必須是雙向跨越

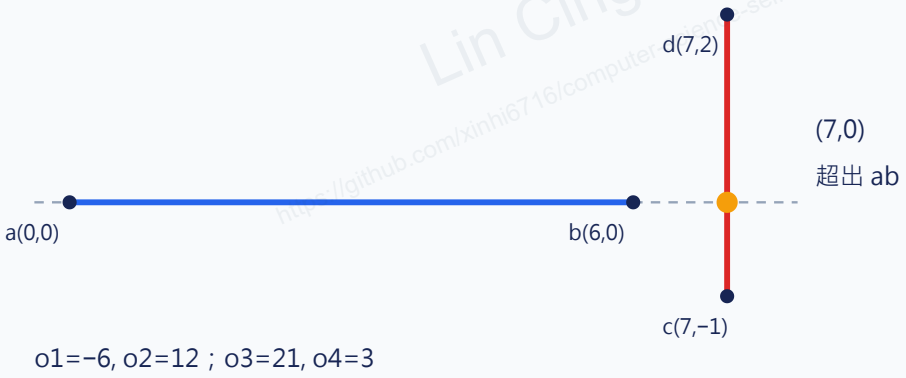
ab 為 (0,0)→(6,0)，cd 為 (7,−1)→(7,2)。

$$(o_1, o_2, o_3, o_4) = (-6, 12, 21, 3).$$

c,d 跨過 ab 所在直線，但 a,b 同在 cd 所在直線一側。交點 (7,0) 超出 ab，故閉線段不相交。

一般位置下，第一組異號使直線交點落在 cd 內，第二組異號使同一交點也落在 ab 內。兩組條件缺一不可。共線時，兩個閉區間若重疊，必有某個端點落在另一區間內，因而四個 on 測試足夠。

**跨越延長線仍可能不相交**




o1=−6, o2=12；o3=21, o4=3



## 精確符號需要足夠位元

若每個座標絕對值至多 B，差值至多 2B，保守界為 $|\,\mathrm{orient}\,| \le 8B^2$。先將座標擴寬，再做減法與乘法；計算完成才轉型已太晚。

a=(0,0)、b=(100000,0)、c=(0,100000) 的方向值為 $10^{10}$，已超出有號 32 位元範圍。完整有號 32 位元座標也不保證有號 64 位元中間值安全。

浮點相減可能消去有效位數。縮放座標 λ 倍會令行列式乘 $\lambda^2$，固定 epsilon 因此不具尺度無關保證。將「差距小於 epsilon」當排序相等，還可能破壞傳遞性。需要精確算術，或有誤差界的過濾器加精度提升；本節不把任意 epsilon 當證明。

## 從所有配對轉成事件

n 段逐對測試花 $\Theta(n^2)$ 次判定。存在性問題只需找到一對；列舉問題須輸出全部 k 對。k 可達 $\Theta(n^2)$，故不能把存在性 O(n log n) 誤寫為列舉所有交點的時間。

掃描線以 x 當時間。事件佇列保存線段端點；狀態 T 保存目前穿過掃描線的線段，以下到上排列。

先講清簡化版本的前提。每段非垂直、非零長度，端點 x 互異，且無三段同點相交，也無共線重疊。閉線段測試本身沒有這些限制；這些是掃描狀態與證明的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 插入與刪除都可能產生新鄰居

```
T = empty balanced search tree
sort the 2n endpoints by x
for endpoint p:
    if p is the left endpoint of s:
        insert s in T
        test s against predecessor and successor; if any intersects: return true
    else:
        test predecessor(s) against successor(s); if they intersect: return true
        delete s from T
return false
```

測試的是完整線段是否相交，不只測試當下掃描線位置。故可以在實際交點之前回傳。刪除時必須測原本隔著 s 的兩段，否則會漏掉新相鄰配對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用座標逐事件追蹤

a:(0,0)→(10,5)，c:(1,2)→(3,2)，b:(2,6)→(9,0)。下到上的 T 如下。

| 事件 | 高度與狀態 | 新測試 |
|---|---|---|
| x=0 插 a | [a] | 無 |
| x=1 插 c | a=1/2,c=2；[a,c] | a,c 不交 |
| x=2 插 b | a=1,c=2,b=6；[a,c,b] | c,b 不交 |
| x=3 刪 c 前 | a=3/2,c=2,b=36/7；[a,c,b] | a,b 相交，回傳 |

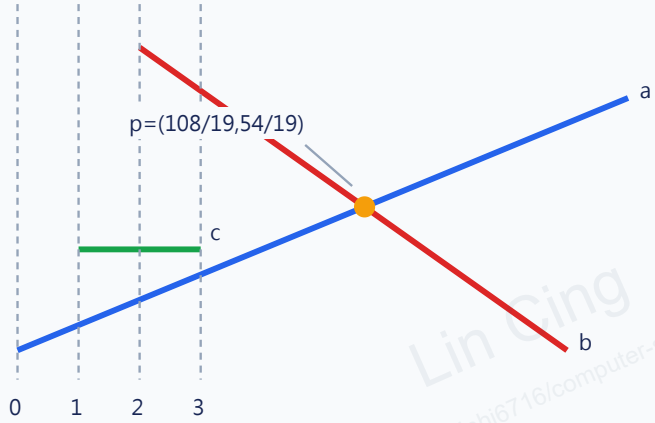
a 與水平 y=2 在 x=4 才相遇，超出 c；b 與 y=2 在 x=20/3 才相遇，也超出 c。解 $x/2 = (54 - 6x)/7$ 得 a,b 交點 $(108/19, 54/19)$。刪 c 的事件早於交點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 用座標逐事件追蹤 （續）

### 刪除 c 時偵測未來相交




| 事件 x | T 下→上 |
|---|---|
| 0 | [a] |
| 1 | [a,c] |
| 2 | [a,c,b] |
| 3 | a,b → true |

x=3 < 108/19

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何只看相鄰段就能回答存在性

假設仍未回傳，考慮最左交點 p 的兩段 a,b。p 左側尚無交叉，所以狀態的上下次序不會自行改變。無三段同點時，a,b 在 p 之前或 p 處必成相鄰。

它們首次成相鄰只會由插入其中一段，或刪掉中間最後一段造成。演算法恰在兩種事件檢查完整配對，故應在 p 之前或 p 處回傳。若先在別處回傳，存在性答案也已正確。

平衡樹每事件做常數次 O(log n) 操作，共 2n 個端點，排序 O(n log n)，所以總時間 O(n log n)、空間 O(n)。此論證依賴找到第一對就停止，不能移植成列舉證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同 x 事件不可任意排序

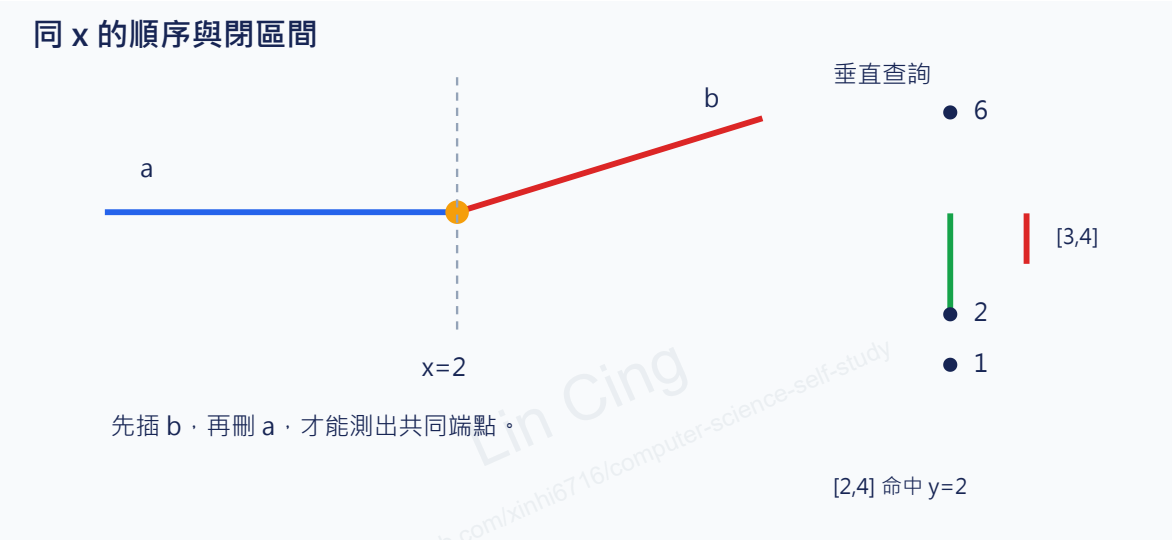
放寬端點 x 互異時，非垂直版本可依 (x,e,y,id) 排序，e=0 是左端點、e=1 是右端點。相同 x 的所有插入先於刪除，再以 y 由低到高；id 僅使同類同點事件可重現。

a:(0,1)→(2,1)、b:(2,1)→(4,2) 在共同端點相交。先刪 a 再插 b 會讓兩段從未同在 T。先插 b 則測得到。

同高度可對應真交點，不能用任意 id 取代相交檢查。同點有多段時，需將事件與同高度群組一起處理並驗證邊界，或保留簡化前提；不能只說「隨便打破平手」。CLRS 33.2 的原始證明排除三段同點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 同 x 事件不可任意排序 （續）




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 垂直段是區間查詢

垂直段 v:(x,ylo)→(x,yhi) 不應以單一高度放進一般狀態樹。它可能接觸該區間中的任一非垂直段。

在同 x 批次先插非垂直左端點，接著對垂直閉區間查詢狀態高度，再刪非垂直右端點。存在性可找第一個高度 ≥ylo，若 ≤yhi 即命中。同 x 的垂直段另按 ylo 排序檢查區間重疊；零長度視為單點區間。相同位置的非垂直端點也須檢查。

例 x=2 時活動高度為 1、2、6。v 的 [3,4] 查詢沒有命中，[2,4] 則在邊界 y=2 命中。這描述一般化所需的操作；前頁簡化偽碼未實作完整同點與重疊處理，不能宣稱它已支援所有退化輸入。

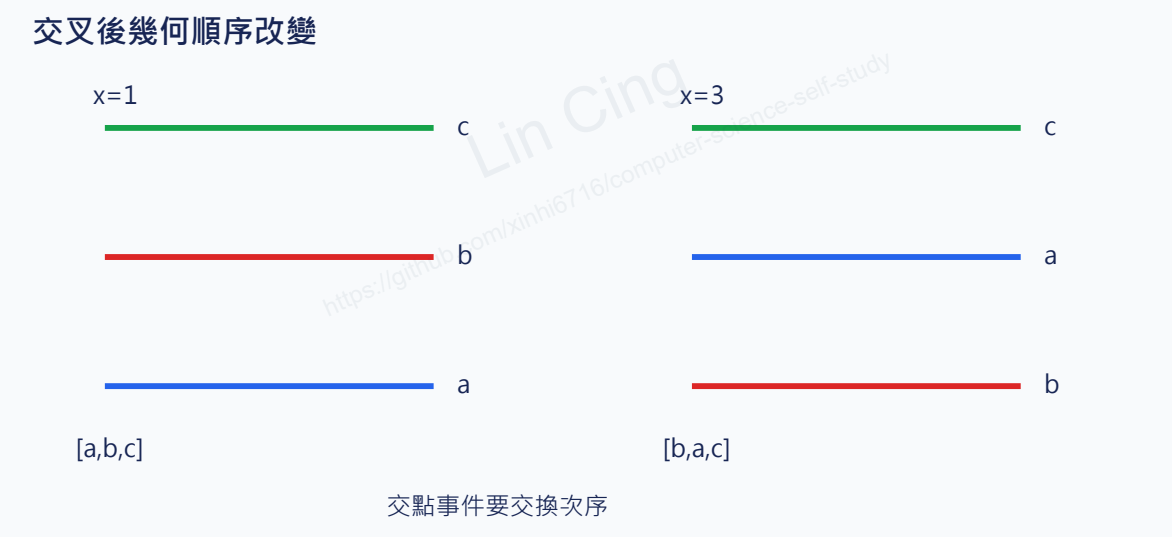
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 印出後繼續會破壞狀態

a:(0,0)→(4,4)，b:(0,4)→(4,0)，c:(0,5)→(4,5)。在 x<2 時次序為 [a,b,c]，x>2 則為 [b,a,c]。

若在插入時發現 a,b 相交只印出再繼續，端點事件之間的樹次序沒有交換，便不再代表幾何鄰居。首次被測出的交點也未必最靠左，因為測試的是整段。

MIT04 的列舉版本另外維護預測交點事件，於交點交換鄰居並更新事件，處理重複與過期事件；在一般位置下為 O((n+k) log n)。共線重疊時須先定義輸出是相交配對、點或重疊段，因為共同點可有無限多個。






## 讓判定語義與資料結構一致

| 層次 | 必須固定的規格 | 本節驗證例 |
| --- | --- | --- |
| 方向判定 | 座標軸、外積順序、算術 | 行列式 10 與反序 −10 |
| 兩段相交 | 閉段或嚴格交叉、點線段 | 重疊、端點、延長線反例 |
| 掃描存在性 | 事件次序、狀態鄰居、停止條件 | 刪 c 才測到 a,b |
| 一般化 | 同點、垂直、共線的處理 | 插入後查詢再刪除 |
| 列舉 | 交點事件與輸出定義 | 交叉後要更新上下次序 |

練習 A10-P01–P06 分別檢查數值符號、邊界、數值可靠性、逐事件狀態、同座標處理與偵測／列舉差異。方向判定將在凸包的保留或彈出決策中再次使用。

## 凸包與最近點對使用不同刪減證據

凸包刪除不能成為極端頂點的點。最近點對刪除不可能改善目前距離的候選配對。兩者都以幾何不變量減少搜尋。固定點集與完整驗算見 data.json 與 check.py。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 明定輸出政策

凸包只輸出極端頂點，逆時針且不重複首點。共線邊上的中間點不輸出；全共線回傳兩端，單點回傳該點，空集合回傳空。凸包先合併相同座標；最近點對保留不同 id 的重複座標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 先看固定點集

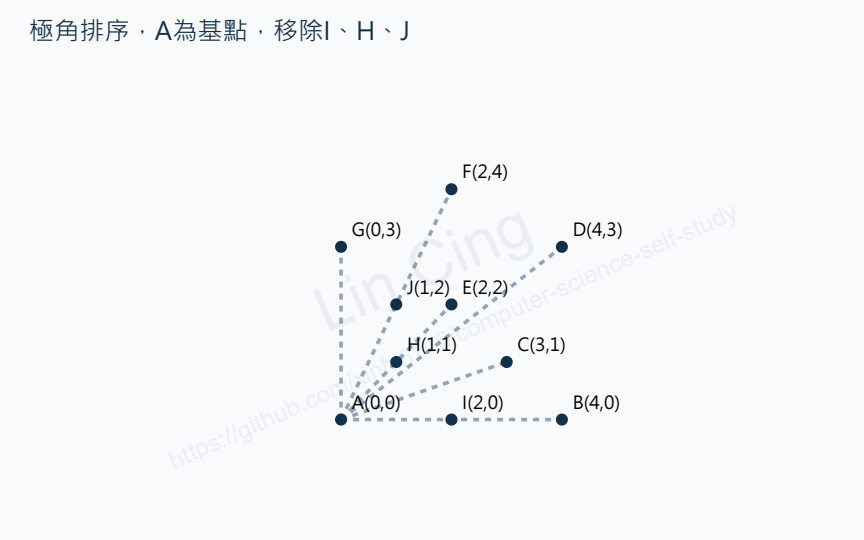
A(0,0), B(4,0), C(3,1), D(4,3), E(2,2), F(2,4), G(0,3), H(1,1), I(2,0), J(1,2)。A 是最低 y 的最左點。orient(a,b,c)=(b−a)×(c−a)，正值為左轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 極角同值必須有政策

繞 A 依極角遞增，未刪除為 I,B,C,D,H,E,J,F,G。同射線只留最遠點，因此移除 I、H、J，保留 B,C,D,E,F,G。J 與 F 也在同一射線。不可只對浮點 atan2 做相等判定，精確版本用叉積與平方距離。



## 掃描規則處理非左轉

令 S 為 stack。依角度讀入 p。只要 |S|≥2 且 orient(S[-2],S[-1],p)≤0 就 pop，然後 push p。≤0 配合只留極端點；改成 <0 會改變共線輸出，但仍須一起調整角度同值與收尾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每輪 stack 可重播

| Point | Popped | Stack bottom to top |
|---|---|---|
| B | - | A,B |
| C | - | A,B,C |
| D | C | A,B,D |
| E | - | A,B,D,E |
| F | E | A,B,D,F |
| G | - | A,B,D,F,G |

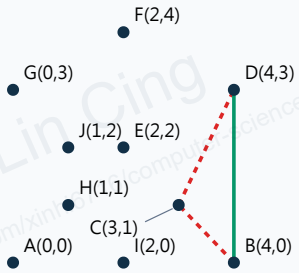
表中起始 stack=[A]，每次列出所有 pop。它與先 push 首三點的標準版本在此非退化資料上等價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## pop 的局部幾何理由

讀 D 時 orient(B,C,D)=−3，C 被 pop，stack 成 A,B,D。讀 F 時 orient(D,E,F)=−4，E 被 pop，stack 成 A,B,D,F。讀 G 不 pop。被移除點位於 A、先前保留點與新點形成三角形內或邊上，不再是已處理集合的極端點。

讀入D，pop C，保留A、B、D

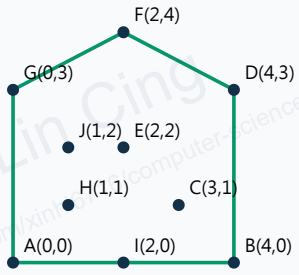





## 正確性不只檢查最後轉向

不變量為處理每一個角度前綴後，stack 正好是該前綴與 A 的凸包極端頂點，依逆時針排列。初始化成立；pop 不改變該前綴凸包，push 新極角的點補上外側邊。終止涵蓋全部點。最後 A,B,D,F,G，全部輸入點位於每條有向邊左側或邊上。

凸包極端頂點 A、B、D、F、G



## 複雜度與 Jarvis march

Graham 排序 O(n log n)，每點至多 push、pop 各一次，掃描 O(n)，空間 O(n)。Jarvis 每輪掃描全部 n 點選下一條支撐邊，h 個極端頂點共 O(nh)。共線候選選最遠點，起點重訪時停止。h 很小有利；h=n 時 $O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 最近點對的完整輸入

固定八點 A(−4,0), B(−3,4), C(−1,1), D(0,5), E(0,1.5), F(2,3), G(4,0), H(5,5)。以平方距離比較，最後才開根號。若不同 id 具有同座標，立即回傳距離 0 的配對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 預排序保持兩種順序

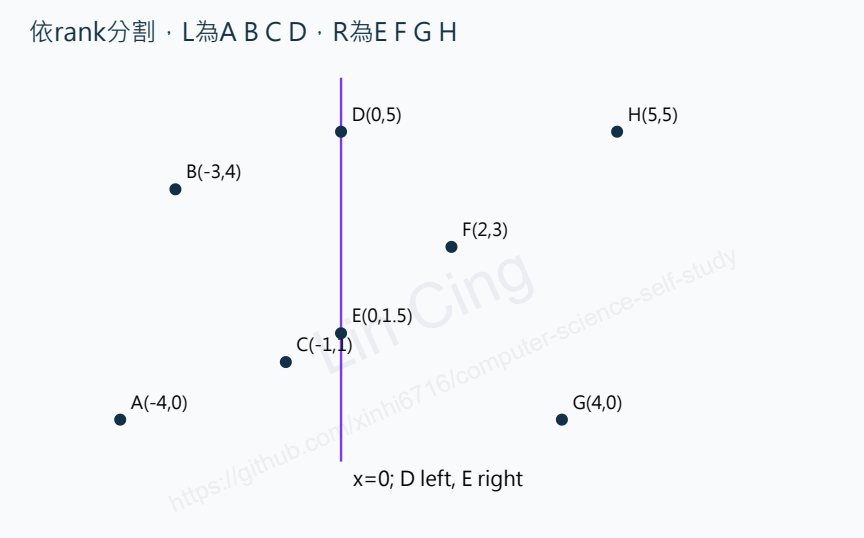
只在最外層排序 X=(x,id)、Y=(y,id)。遞迴輸入兩陣列必須具有相同 id 集合。X 依中間索引切成 L 與 R，再以 L 的 id 集合線性穩定過濾 Y。不能每層重新排序 Y。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 重複 x 不用座標判成員

此例 X=A,B,C,D,E,F,G,H，L=A,B,C,D，R=E,F,G,H，分割線 x=0。D、E 的 x 都是 0，依 id/rank 分到不同子問題。若把所有 x≤0 放左邊，子問題可能失衡。



## 遞迴答案與跨線答案

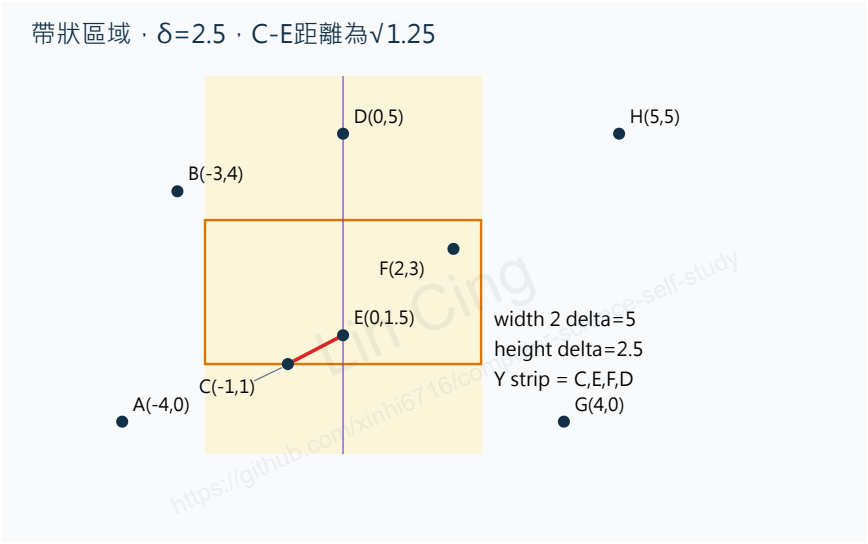
|P|≤3 時窮舉。左側最小平方距離 10，A-C 與 B-D 同值；右側為 6.25，E-F。因此 δ=min(√10,2.5)=2.5。跨線改善配對的兩點必有 |x−0|<δ 且 |Δy|<δ。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## strip 必須依 y 次序掃描

從 Y 穩定過濾 |x−線|<δ，得到 C(−1,1),E(0,1.5),F(2,3),D(0,5)。C-E 的平方距離為 1.25，距離約 1.118；C-F 為 13，E-F 為 6.25，F-D 為 8。只有 C-E 改善 δ。

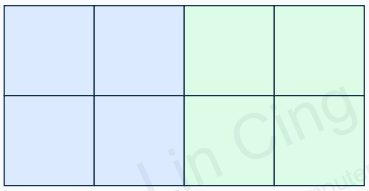



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 常數界來自同側分離

假定 δ>0 且遞迴已證明 L 內、R 內任意兩點距離≥δ。對 strip 中 p，只考慮其上方 y 差小於 δ 的點，落在寬 2δ、高 δ 矩形。左右各分四個 δ/2×δ/2 格，同一側每格最多一點，因格對角線 δ/√2<δ。線上點仍按 id 所屬側計數。

每側四格，每格邊長δ/2



2 delta by delta; cell diagonal = delta/sqrt(2)
At most 4 left + 4 right = 8 including p

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為何最多看後七個

每側最多 4 點，矩形合計最多 8 點，包含 p，因此後方候選最多 7 個。Y 必須有一致的 y、id 次序；可用半開格規則唯一分配邊界。此界不假設左右跨側點已互相分離。δ=0 先結束，不能再套正方形裝填。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 遞迴式取決於資料流

預排序 O(n log n)，每層分割 X、過濾 Y、建 strip、每點常數比較總共 O(n)。
T(n)=2T(n/2)+O(n)=O(n log n)。若每層排序 strip，變成 O(n $\log^2$n)。深度 O(log n)，循序遞迴釋放陣列可用 O(n) 額外空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 來源與獨立驗算

CLRS 第三版 §33.3–33.4，印刷頁 1029–1043，PDF 頁 1050–1064。原掃描 note PDF 頁46–47支持凸包與 Graham；沒有從這兩頁推定最近點對內容。MIT 2004 L18 PDF頁3–11支持分治與裝填；2005本地21份清單無計算幾何講次。check.py窮舉固定點集全部45/28個配對並驗證支撐邊、stack與分治答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 33. MIT 6.046J Fall 2004 computational geometry lecture slides, segment intersection and closest pair. Original handwritten algorithms notes (演算法 _wjungle), computational geometry pages.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 計算複雜度與近似

當問題規模增加，如何判斷求解是否有效率？我們先以輸入編碼長度衡量時間，再研究判定問題、驗證與多項式時間歸約。

接著研究最佳化問題的近似演算法，以可行解、上下界及反例理解保證的適用條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 本章路線

判定問題 → P 與 NP → 歸約方向 → NP 完全性 → 近似比與具體演算法。

區分證明困難度的歸約與求近似解的方法；NP 完全性本身不證明 P ≠ NP，也不給指數時間下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 先把問題寫成語言

本節從輸入位元數出發，建立 P、NP 與 polynomial-time many-one reduction。目標是能獨立證明 3SAT ≤p CLIQUE，而不是背歸約箭頭。

前置知識為布林公式、無向簡單圖、漸近界與全稱／存在量詞。

學完應能回答「證書驗得快」為何不表示「證書找得快」，並指出每項 NP-completeness 證明的兩個義務。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 決策、搜尋與最佳化先分清

| 形式 | CLIQUE 例子 | 輸出 |
| --- | --- | --- |
| 決策 decision | 是否存在至少 k 個頂點的 clique？ | yes／no |
| 搜尋 search | 找出 k-clique，如存在 | 頂點集合或不存在 |
| 最佳化 optimization | 求最大 clique 大小 ω(G) | 整數 |

本節 P、NP 以決策語言定義。最佳化的多項式演算法能解決決策版，只需比較 ω(G) ≥ k。反向還需額外構造，不能只說兩者是同一個問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 輸入長度不是數值大小

設 x 為完整編碼，N = |x| 為位元數。整數 t ≥ 1 的二進位長度為 ⌊log₂ t⌋ + 1。圖的編碼也要包含頂點與邊，門檻 k 也在輸入內。

若 t = 2^m，二進位只需 m+1 位。執行 t 次迴圈需 Θ(2^m) 步，不能因為寫成 O(t) 就稱為多項式時間。若改用 unary，輸入有 t 個符號；那已是不同編碼。

圖的 adjacency matrix 與顯式 adjacency list 可互相以多項式時間轉換。允許這類轉換不代表允許把二進位數值展成 unary。

**同一數值，不同輸入長度**

t = 2^m

| 二進位 binary | 一元 unary |
| --- | --- |
| 1000...000 | 1111...111 |
| m + 1 位元 | 2^m 個符號 |
| t 次迴圈 = Θ(2^m) | t 次迴圈 = Θ(輸入長度) |

多項式必須相對於完整編碼長度 N = |x|。

O(t) 不代表 O(poly(log t))。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 從編碼集合定義 P

決策語言 L ⊆ {0,1}* 收集所有 yes instance 的編碼。約定錯誤編碼不在 L，演算法必須在多項式時間拒絕。

$$L \in P \iff \exists M\ \exists p \text{ polynomial } \forall x:\ M(x) \text{ halts within } p(|x|) \text{ and } [M(x) = 1 \iff x \in L].$$

M 是 deterministic decider。時間界須對每個輸入成立，包含 no instance。PATH = {⟨G,s,t,k⟩ | 存在至多 k 條邊的 s 到 t 路徑} 可用 BFS 求最短距離後比較，故在 P。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## NP 的定義包含證書長度

存在 deterministic verifier V 與多項式 p、q，使

$$x \in L \iff \exists c,\ |c| \le p(|x|) : V(x,c) = 1.$$

V 在 O(q(|x|+|c|)) 時間內停止。因為證書長度也受 p 限制，合法候選的驗證時間才是 |x| 的多項式。可先拒絕超長證書。

NP 是 nondeterministic polynomial time，不是 non-polynomial。定義保證 yes instance 有短證書，沒有保證能在多項式時間找到它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完備性與健全性是兩個量詞

| 義務 | 正確敘述 | 不能換成 |
|---|---|---|
| 完備性 completeness | x ∈ L ⇒ 存在短 c 使 V 接受 | 每份證書都接受 |
| 健全性 soundness | x ∉ L ⇒ 所有允許的 c 都使 V 拒絕 | 存在一份被拒絕的 c |

每份合法輸入與允許長度的證書都必須讓 V 停止。許多 validator 只檢查資料格式；NP verifier 還要證明證書確實支持該 yes 答案。

**Verifier 的完備性與健全性**

|c| ≤ p(|x|)，驗證時間 ≤ q(|x| + |c|)，p、q 為多項式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## CLIQUE 的 verifier 寫到可執行

輸入為簡單無向圖 G 與二進位非負整數 k。若 k > n，直接拒絕。k = 0 時接受空證書。其餘證書是恰好 k 個不同頂點編號。

1. 檢查編碼、證書長度、編號範圍與互異性。
2. 對所有 i < j，檢查 $\{c_i,c_j\} \in E$。
3. 全部通過才接受。

證書長度 $O(k \log(n+1)) \leq O(n \log(n+1))$。建好 adjacency matrix 後需 $O(n^2+k^2)$ 次基本圖操作，位元操作成本仍為輸入長度的多項式。若有 clique，可取其 k 個頂點；若 verifier 接受，所有頂點對均相鄰，確實是 clique。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 已知 P ⊆ NP；等號仍未知

若 $L \in P$，令 V(x,c) 直接執行 L 的 decider 並忽略 c。每個 yes instance 皆可用空證書，no instance 不會被任何證書誤接受。因此 $P \subseteq NP$。

目前未解的是 $NP \subseteq P$ 是否成立，亦即 P = NP 是否成立。不能畫出 P 與 NP-complete 分離的圖而不註明假設 $P \neq NP$。

NP-complete 的已知歸約定理不等於已證 $P \neq NP$；指數時間暴力演算法也不能證明所有演算法都需要指數時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## many-one 歸約保留 yes 與 no

A ≤p B 指存在總函數 f，可在輸入長度的多項式時間計算，且對所有 x，

$$x \in A \iff f(x) \in B.$$

f 必須在多項式時間計算，不能假設可免費求解 A。若 A∈P，且 B 非空且非全集，可先判定 A，再依結果輸出 B 的固定 yes 或 no 字串。這是輸出單一字串的 many-one reduction，不是多次 oracle 查詢。

所有字串均須滿足等價式。若來源拒絕錯誤編碼且目標有固定 no 字串，可將錯誤編碼映至它；一般目標未必有 no 字串，例如 B={0,1}*。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



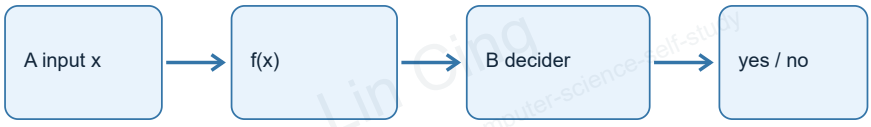
## 歸約方向由求解器接線決定

若 A ≤p B，則「會解 B」可推出「會解 A」。演算法為 x → f(x) → B-decider → A 的答案。

因此 B ∈ P ⇒ A ∈ P。取逆否可得 A ∉ P ⇒ B ∉ P。不能由 A ∈ P 推得 B ∈ P，也不能由 B 很難推得 A 很難。

要證明目標 B 很難，應由已知難的 A 歸約到 B。若只證 B ≤p A，僅表示 B 可借用 A 的求解器。

**A ≤p B 讓 A 使用 B 的求解器**




x ∈ A ⇔ f(x) ∈ B

B ∈ P ⇒ A ∈ P

證明 B 很難，從已知難的 A 指向 B。

f 必須在多項式時間直接建構 instance。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何時間與輸出長度都要追蹤

設 f 的時間為 O(N^a)，且輸出長度 |f(x)| ≤ C N^a。若 B-decider 在長度 M 的輸入上需 O(M^b)，則組合時間為

$$O(N^a) + O((CN^a)^b) = O(N^a + N^{ab}).$$

a、b、C 為固定常數，故仍為多項式。關鍵是標準逐位元輸出模型下，輸出不可能比執行時間還長。

若允許把指數大小的 B instance 以「壓縮描述」當作同一輸入，B 的原時間界不能直接套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## many-one 歸約的傳遞性

若 A ≤p B 經 f，且 B ≤p C 經 g，定義 h = g ∘ f。

語義上，x ∈ A ⇔ f(x) ∈ B ⇔ g(f(x)) ∈ C。兩個雙條件缺一不可。

若 f 的時間 O(N^a)，g 的時間 O(M^b)，則 |f(x)| = O(N^a)，h 的時間 O(N^a + N^(ab))。因此 A ≤p C。

這證明容許串接固定數量的多項式歸約。若重複 N 次且每次長度平方，最終長度可能 N^(2^N)，不能直接套「多項式次數的呼叫仍為多項式」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## NP-hard 與 NP-complete 的條件

| 名稱 | 條件 |
| --- | --- |
| B 為 NP-hard | 對每個 $A \in NP$，$A \leq_p B$ |
| B 為 NP-complete | B 為 NP-hard 且 $B \in NP$ |

若已知 A 是 NP-hard，證 $A \leq_p B$ 即可由傳遞性證 B NP-hard。再獨立給 B 的短證書與多項式 verifier，才能得 NP-complete。

NP-hard 本身不保證 $B \in NP$，亦不保證 B 可判定。本節以決策語言定義；談最佳化問題 NP-hard 時，須先說明採何種決策或求解歸約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3SAT 到 CLIQUE 的建構

令 $F = C_1 \wedge \cdots \wedge C_m$，每個子句含三個 literal occurrences，本節允許重複。CLRS 的三個相異 literals 版本是其子集合，故 NP-hardness 仍適用，assignment verifier 也仍為多項式。3SAT 的 NP-completeness 採用 CLRS34 已知定理，此處證明目標 CLIQUE。

- 每個 occurrence 建一個頂點 $v_{ij}$，記錄其子句 i 與 literal $\ell_{ij}$。
- 同一子句的頂點完全不連邊。
- 不同子句的兩頂點，當且僅當 literals 不是互補的 x 與 ¬x 才連邊。
- 輸出 $\langle G, k=m \rangle$。

重複出現的相同 literal 仍是不同頂點。跨子句的相同 literal 彼此相容，必須連邊。

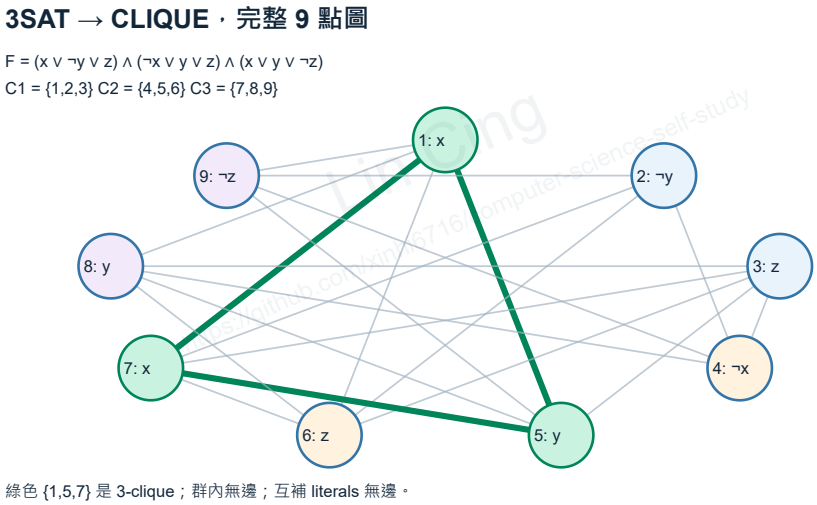
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把實際公式畫成完整圖

取 F = (x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ z) ∧ (x ∨ y ∨ ¬z)。

頂點編號依子句順序為 1:x、2:¬y、3:z；4:¬x、5:y、6:z；7:x、8:y、9:¬z。完整邊集如下，圖中每條灰線或綠線都是真實邊。

E = {15,16,24,26,34,35,36,17,18,19,27,29,37,38,48,49,57,58,59,67,68}，共 21 條。x=y=z=true 時，可選 {1,5,7}，三條邊 15、17、57 都存在。






## 正向證明由 assignment 選 clique

假設 F 可滿足，固定一個滿足 assignment。每個子句至少有一個真 literal，從每個子句各選一個對應頂點，得到 m 個頂點 S。

任意兩個選中頂點來自不同子句，且其 literals 同時為真，不可能是互補的 x 與 ¬x。依建構規則兩者相鄰。

所以 S 中每一對頂點都相鄰，|S|=m，為 m-clique。這證明 F ∈ 3SAT ⇒ ⟨G,m⟩ ∈ CLIQUE。歸約建圖本身沒有使用這個 assignment，它只在正確性證明中出現。

## 反向證明由 clique 還原 assignment

假設 G 有大小 m 的 clique S。每個子句群內沒有邊，所以 S 在每群最多選一點。共 m 群而有 m 點，故每群恰選一點。

S 的任兩點相鄰，所以所選 literals 不含互補對。令選中的正 literal 之變數為 true，負 literal 之變數為 false。沒有變數被要求同時為 true 與 false。未選到的變數任意賦值。

每個子句都含一個選中的真 literal，故 F 可滿足。這證明 $\langle G,m\rangle \in \text{CLIQUE} \Rightarrow F \in \text{3SAT}$，完成雙條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 建圖成本與邊界必須相容

輸入長度 N 至少與 occurrence 數同階。m 個三文字子句產生 3m 個頂點，至多 $9 \cdot m(m-1)/2$ 條邊。比較所有跨群 pairs，連同變數編號的位元比較，加上輸出邊編號的成本，$O(N^2 \log(N+1))$ 位元操作足夠；顯式邊串列大小 $O(m^2 \log(m+1))$，也是 N 的多項式。

| 邊界 | 約定與理由 |
| --- | --- |
| m=0 | 空 conjunction 為 true；G 空、k=0，空 clique 存在 |
| 同變數正負跨群 | 不連邊，避免矛盾 assignment |
| 單子句含 x 與 ¬x | 群內仍不連邊；只須挑一個真 literal |
| 重複 literal | 保留 occurrence 頂點，不合併 |
| 不合法 3CNF | 映到固定 no instance ⟨空圖,1⟩ |

若 CLIQUE 規格只允許 $k \geq 1$，m=0 應另映到固定 yes instance $\langle K_1,1\rangle$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 現在才可宣告 CLIQUE NP-complete

1. 已知 3SAT 是 NP-complete。
2. 已構造多項式時間總函數 f，並證 F 可滿足 ⇔ f(F) 有 m-clique。
3. 所以 3SAT ≤p CLIQUE。由歸約傳遞性，CLIQUE 為 NP-hard。
4. 前述頂點證書與 verifier 證 CLIQUE ∈ NP。
5. 合起來得 CLIQUE 為 NP-complete。

若有人提出 CLIQUE 的多項式時間演算法，便由歸約得所有 NP 語言皆在 P，配合 P ⊆ NP 得 P=NP。這是條件定理，沒有證明 P≠NP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 決策 oracle 如何找最佳值與證書

對 CLIQUE，固定圖 G，謂詞「有至少 k 個頂點的 clique」隨 k 單調。於 [0,n] 二分搜尋，用 O(log(n+1)) 次 decision 呼叫可得 ω(G)。這是多次呼叫的求解方法，不是剛才的一次 many-one reduction。

已知 G 有 k-clique。依序試刪頂點 v，若 G−v 仍有 k-clique 就刪除，否則保留。至多 n 次呼叫後圖為 inclusion-minimal 的 yes instance，恰有 k 個頂點，得到證書。

理由是任取一個 k-clique，若尚有 clique 外的頂點，它仍可刪除。某頂點一旦不可刪，往後刪除其他頂點也不會讓它變得可刪。此處每次子圖長度不增，故若 decision 在 P，整個搜尋亦為多項式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從驗證器到第一個 NP 完全問題

給定布林電路 $C$，CIRCUIT-SAT 詢問是否存在輸入 $y$ 令 $C(y) = 1$。它屬於 NP：把輸入指派作為 witness，依拓撲順序計算每個 gate，時間與編碼長度成多項式。

要證明 NP-hard，不能只從某個尚未知為 NP-hard 的問題歸約。固定任意語言 $L \in \mathrm{NP}$，取其確定性多項式時間 verifier $V$、多項式界 $p, T$，使 $x \in L \iff \exists y \in \{0,1\}^{p(|x|)} : V(x, y) = 1$。長度不足的 witness 可用長度欄位與 padding 編碼。

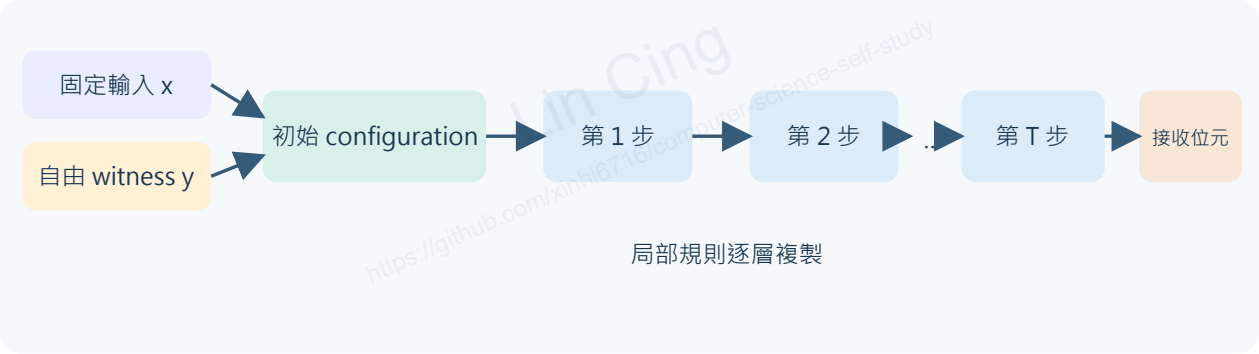
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 把一次執行攤開成電路

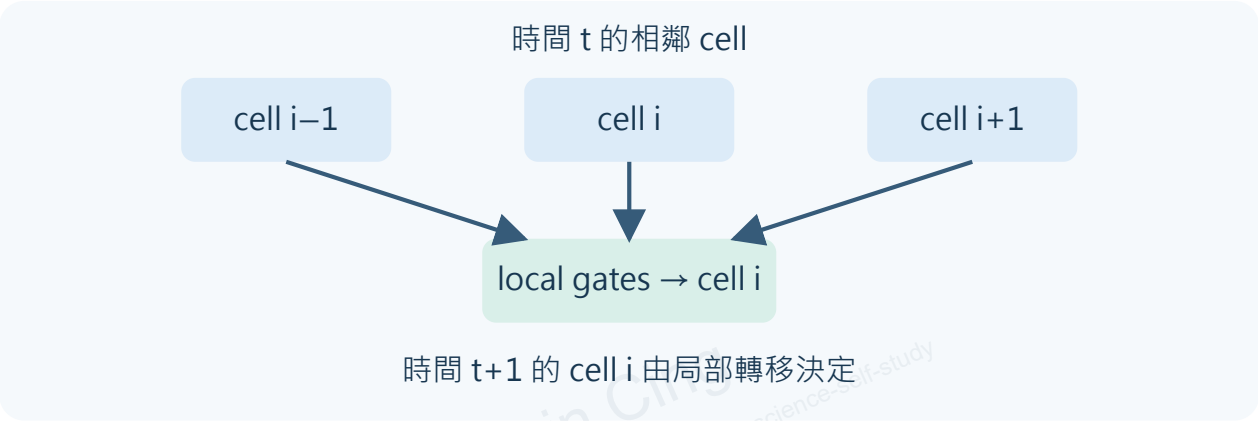
固定 $x$，令 $n = |x|$，使用固定且對所有輸入一致的編碼方式。把 verifier 的記憶體、讀寫頭、控制狀態與輸入寫成長度 $s(n) = \mathrm{poly}(n)$ 的 configuration；最多 $T(n) = \mathrm{poly}(n)$ 步。接收後以吸收狀態維持接收，恰好執行 $T(n)$ 步也不改變答案。

選用單帶圖靈機模型時，每個 cell 的下一步狀態由其自身及固定範圍鄰居決定；有限的局部轉移表可由常數大小的布林 gate 電路實作。對每一步複製一層、對每個 cell 複製局部電路，得到 $O(T(n)s(n))$ 個 gate。其他合理模型也須先給出多項式模擬，不能把任意 RAM 操作視為免費常數大小電路。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 局部更新如何接成下一層




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正確性：逐層維持同一個 configuration

初始層將 $x$ 的每一位固定為常數，$y$ 的 $p(n)$ 位保留為自由輸入，其餘位置寫入初始空白與控制狀態。若採不同長度的 witness，固定長度編碼內含合法長度檢查；非法編碼輸出 0。

對任意自由輸入 $y$，第 0 層就是 $V(x, y)$ 的初始 configuration。假設第 $t$ 層等於實際第 $t$ 步，局部 gate 依同一轉移規則給出第 $t+1$ 步，故歸納成立。最後 output 取接收狀態位，因此 $C_x(y) = 1 \iff V(x, y) = 1$；存在滿足輸入恰好等價於 $x \in L$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為什麼這是一個多項式時間歸約

$L$ 一旦固定，verifier、局部規則、常數及多項式上界也固定；它們可寫進建構器。對每個 $x$，建構器算出 $T(n), s(n)$、逐層輸出 gate 描述，並把 $x$ 硬接為常數；時間和輸出長度皆為 $\mathrm{poly}(n)$。這是 uniform construction：不是僅宣稱「每個長度存在一個小電路」。

因此任意 $L \in NP$ 都有 $L \leq_p \mathrm{CIRCUIT\text{-}SAT}$；連同 membership，CIRCUIT-SAT 是 NP-complete。電路可含共享 wire；不必展開為可能指數大的公式樹。歸約輸出的是電路本身，而不是先求出 witness。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從電路到布林可滿足性

*CIRCUIT-SAT 到 SAT 的局部 gate 約束*

為每條 wire（可共享同一 gate output）設一變數，並把輸出 wire $z_{\mathrm{out}}$ 本身作為 unit clause，要求它為真。每個 gate 加入輸出等於 gate 函數的條件：

| Gate 關係 | 等價 CNF clauses |
|---|---|
| z = u AND v | (¬z∨u) AND (¬z∨v) AND (z∨¬u∨¬v) |
| z = u OR v | (z∨¬u) AND (z∨¬v) AND (¬z∨u∨v) |
| z = NOT u | (z∨u) AND (¬z∨¬u) |

例如 AND 的前兩 clauses 強迫 z=true 時 u,v 皆真；最後一 clause 強迫 u=v=true 時 z=true。OR 與 NOT 可同樣分真假兩向直接檢查。常數 wires 用 unit clause 固定。

電路接受輸入可延伸為所有 wire 的真值，滿足每個局部 clause 與 output clause。反向滿足公式時，局部等價式沿拓撲序迫使每個 wire 等於實際計算值，output=true，所以原電路可滿足。每 gate 至多三 clauses，總長 O(gates+wires)，不把共享子電路反覆展開成指數長樹狀公式。

## SAT 到 3-CNF 的 parse-tree gadget

任意公式先解析成 NOT、一輸入或 AND/OR 二輸入樹；例如 implication 可用 ¬a∨b 改寫，其他固定 arity connective 亦用固定大小 truth-table gadget。為每個內節點設新變數，使用前頁 gate clauses，根變數設真。若同一節點輸入重複，先在所得 clauses 去除重複 literal；含 l 與 ¬l 的 clause 是 tautology，可整個刪掉。

此時 clauses 長度至多三。若要求每 clause 恰含三個相異變數，逐 clause 使用新的 padding variables：

- 兩 literal (a∨b) 改為 (a∨b∨p) AND (a∨b∨¬p)。a∨b 真則任意 p 都可；若假，兩式分別強迫 p 與 ¬p，矛盾。
- 一 literal a 改為四式 (a∨p∨q)、(a∨p∨¬q)、(a∨¬p∨q)、(a∨¬p∨¬q) 的 AND。a 真則皆真；a 假時，任何 p,q 指派都令其中一 clause 為假。
- 空 clause 表示不滿足，直接輸出上述 padding 過的 z AND ¬z；空 AND 表示恆真，輸出固定可滿足的三變數 clause。

原 assignment 可用每個 parse node 的真值延伸；反向沿樹恢復所有節點語義。padding 保留每 clause 的真假要求，因此整體只保留 equisatisfiability，而非把加入新變數後的公式當作原變數的逐字同一函數。每節點常數 clauses，每 clause 至多擴成四個，所以大小與建構時間皆多項式。



## 長 clause 拆分與可追蹤 gate 例

若已得到長 clause $l_1 \vee \cdots \vee l_r$，$r > 3$，也可用 chain：

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \\ \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

若所有 $l_i$ 都假，首式逼 $y_1 = \text{true}$，中間逐次逼下一個 $y_j = \text{true}$，末式卻要求最後 $y_{r-3} = \text{false}$，矛盾。若至少一個 $l_i$ 真，令 $y_j$ 表示前 $j+1$ 個 literals 是否全假，即可逐式滿足。此拆分用 $r-3$ 新變數、$r-2$ clauses；不能直接用 distributivity 把任意公式轉 CNF，那可能產生指數長度。

例 F=(a∨b) AND ¬a，設 u=a∨b、v=¬a、z=u AND v，給三組 gate clauses 並加 z。a=false,b=true 時 u=v=z=true，滿足所有約束。若 a=true，NOT gadget 強制 v=false，而 z 與 AND gadget要求 v=true，故這類指派不能假造接受。這同時示範 witness 延伸與反解。

原書 3-CNF 定義為三個相異 literals；本教材的主要 reduction 進一步可規格化為三個相異變數。出現重複 literal 的輸入仍可先依上述去重、padding 規則處理，不影響後續 CLIQUE occurrence 構造。

## CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER

independent set S 是圖中任兩點都沒有 edge 的集合。vertex cover C 是每條 edge 至少一端在 C 的集合。

對 n 點、無 self-loop 的無向圖 G=(V,E)，令 $\overline{G} = (V, \{uv : u \neq v,\ uv \notin E\})$。同一 pair 在 G 有 edge 若且唯若在 $\overline{G}$ 沒有 edge。因此：

$$C \text{ is a clique in } G \iff C \text{ is an independent set in } \overline{G}.$$

若 S 是 independent set，則 $V \setminus S$ 是 vertex cover。假設有 $\overline{G}$ 的 edge 未被 $V \setminus S$ 覆蓋，它的兩端都在 S，違反 S independent。反向若 $V \setminus S$ 是 vertex cover，任何 $\overline{G}$ edge 不可能兩端都在 S；所以 S independent，也是在 G 的 clique。

令 $k \in \{0, \ldots, n\}$。把上面兩步串起，得到

$\text{CLIQUE}(G, k) \iff \text{VERTEX-COVER}(\overline{G}, n - k)$。

$(G, k) \mapsto (\overline{G}, n - k)$ 與反方向的 $(H, \ell) \mapsto (\overline{H}, n - \ell)$ 都以 $O(n^2)$ 次配對檢查建立補圖。這裡CLIQUE可說「至少k點」或「恰k點」，因較大clique任取k點仍是clique；VC採「至多n−k點」，較小cover可補點至恰n−k點，所以同一iff成立。這需0≤k≤n；範圍外門檻先直接判定並映射至固定yes/no實例，不能產生負大小certificate。



## CLIQUE 與 VERTEX-COVER 的可追蹤例

令 $G$ 有頂點 {1,2,3}，edges {12,23}，取 k=2。{1,2} 在 G 是 clique。

補圖 $\overline{G}$ 只有 edge {13}。$\{1, 2\}$ 在 $\overline{G}$ 是 independent set，因為 1 與 2 沒有 edge。其補集 $\{3\}$ 是 $\overline{G}$ 的 vertex cover，大小為 $3 - 2 = 1$，且它覆蓋唯一 edge 13。

轉換是 $\text{VC}(\overline{G}, n - k)$。所以 $\text{CLIQUE}(G, k) \iff \text{VERTEX-COVER}(\overline{G}, n - k)$。這個等價直接給出雙向約化：complement 本身就是 polynomial-time、可逆的 encoding transformation。

# 從頂點覆蓋到 Hamiltonian cycle

*VERTEX-COVER 到 HAMILTONIAN-CYCLE 的端口 gadget*

HC certificate 是全部頂點的排列，逐邊與閉合檢查給 NP membership。下面完整建立 NP-hardness，不需要平面 crossing gadget；圖中交叉畫線不代表交叉處是頂點。

對原圖每邊 {u,v} 建十二個新點 a1,...,a6、b1,...,b6，其中 ai=[u,v,i]、bi=[v,u,i]。內邊共十四條：

- 兩條鏈 a1-a2-a3-a4-a5-a6 與 b1-b2-b3-b4-b5-b6，共十條。
- 四條交叉連結 a1-b3、a3-b1、a4-b6、a6-b4。

只有四個端口 a1,a6,b1,b6 可接 gadget 外部。其他八點在最後 HC 中必有兩條內邊。a2,b2,a5,b5 在 gadget 本身 degree=2，故它們的八條相鄰鏈邊被迫選入。

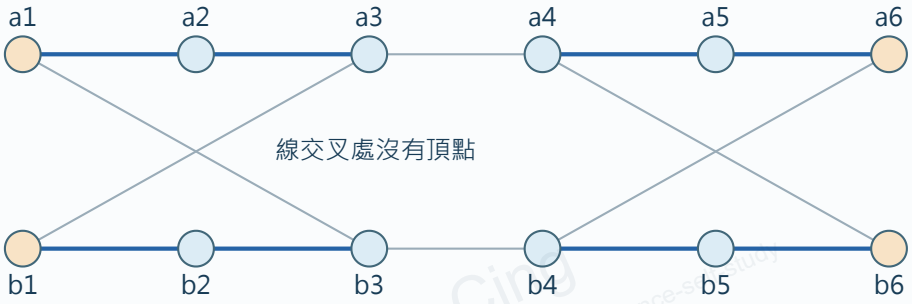
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 從頂點覆蓋到 Hamiltonian cycle （續）

HC gadget：只有 a1、a6、b1、b6 能接外部

$a_i$=[u,v,i]；$b_i$=[v,u,i]




藍色8邊由 a2、a5、b2、b5 的 degree=2 強迫選入；灰色6邊仍待決定。
兩條中邊 a3−a4、b3−b4 決定其餘4條cross edges；橘色為4個端口。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## gadget 只有三種合法的覆蓋方式

餘下選擇由中間兩邊 a3-a4、b3-b4 決定。每個內部點仍需 degree=2，故這兩邊一旦選或不選，四條交叉連結的使用亦被迫確定。

| 中邊選擇 | 對全部十二點的路徑覆蓋 |
|---|---|
| 兩條都選 | a1-a2-a3-a4-a5-a6，以及 b1-b2-b3-b4-b5-b6，兩條 disjoint paths |
| 只選 b3-b4 | a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6，一條 a1 到 a6 的完整 path |
| 只選 a3-a4 | 對稱的一條 b1 到 b6 的完整 path |
| 兩條都不選 | 上下各形成一個封閉六點 cycle，與外部無法連成單一 HC，因此禁止 |

這四種窮盡所有可行內度配置，故合法 HC 只能使用前三種。尤其不能從 a 端口進來卻沿 b 的外部鏈離開；也不能讓十二點完全無人覆蓋。此局部事實正好表示原邊至少有一個端點被選。



## 中邊如何強迫端口的外部使用

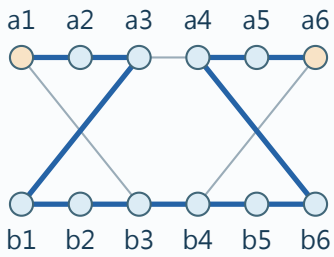
令x表示是否選a3−a4，y表示是否選b3−b4，值為0或1。內點a3已有a2−a3，所以a3−b1必取1−x；a4同理使a4−b6取1−x。b3,b4則使a1−b3與a6−b4取1−y。

因此a1,a6的內部度都是2−y，還需要的外部度為y；b1,b6的外部度為x。端口不可能一邊連外、一邊封閉：同一原頂點側的兩端總是同時接外或同時不接外。

x=y=0時，下列兩個六點cycle已各自用滿度2：a1−a2−a3−b1−b2−b3−a1，以及a4−a5−a6−b4−b5−b6−a4。它們不能接到selector，故不可能是整張圖的單一HC。其餘三種配置的全部路徑如下圖，逐點可核對沒有漏點或換側離開。

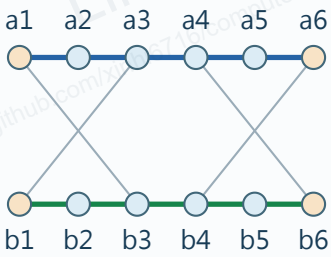
三種合法模式都不會跨原頂點身分離開

只走u側：12點



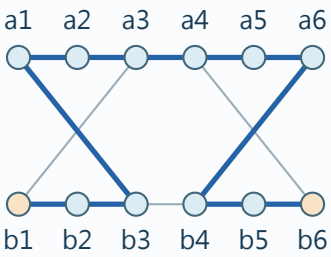

x=0，y=1

u、v各走6點




x=1，y=1

只走v側：12點




x=1，y=0

灰邊未選；藍 / 綠是各條路徑；橘色端口各需一條外邊補到 degree=2。

## 串接原頂點的 incident gadgets 與 selectors

VERTEX-COVER 輸入採「大小至多 k」。先移除孤點，不影響最小 cover。k<0 或 k=0 且有邊，輸出固定 no HC instance；無邊或 k 大於等於剩餘頂點數，輸出固定 yes triangle。以下可假設無孤點且 $1 \le k < |V|$；一個較小 cover 可以補入其他非孤點成恰 k 點。

對每個原頂點 u，任排其鄰居 u(1),...,u(d)。把各 gadget 的 u 側依序串接：加入 [u,u(i),6]-[u,u(i+1),1]，i=1,...,d-1。這些連結使一條外部路徑一旦選 u，便沿它全部 incident gadgets 走到末端。

另建 k 個 selector s1,...,sk。每個 selector 都接到每個 u 鏈的首端 [u,u(1),1] 與末端 [u,u(d),6]，沒有其他外邊。圖共有 12|E|+k 點；十四內邊乘 |E|，串鏈至多 2|E|，selector 連邊 2k|V|，全為多項式規模。原圖有向無向不可混淆，此構造與目標皆為無向圖。



## 為何啟用會傳遍整條鏈

若 u 側兩個端口都使用外邊，稱此側為**啟用**。排序後的 u 鏈中，每個內部端口恰有一條可用外邊，即接相鄰 u 側的串鏈邊。若第 i 側啟用，它必使用該邊；相鄰側的端口因此使用外邊，由同一側兩個端口同時使用外邊的性質，該側另一端也必須使用外邊。向兩個方向歸納，u 的所有 incident gadgets 及鏈首尾都啟用。反之，一側未啟用，即不可能有任何一側啟用。

鏈的兩個端點在 gadget 外只接 selector；啟用的鏈因此在兩端各用一條 selector 邊。局部路徑連接同一 u 側兩端，故沿 cycle 行走時，抵達 selector 之前不會轉換原頂點身分。未啟用側的點仍被訪問：每個邊 gadget 至少有一側啟用；僅另一側啟用時，另一側的十二點完整路徑覆蓋所有點，包括未啟用側。

例：u–v–w、k=1、cover={v}

此覆蓋使兩個 v 側啟用，以 U.b6 → V.a1 連接

未啟用 u、w 側的點仍在十二點完整路徑中

## VC 到 HC 的完整 iff

若 C={u1,...,uk} 是 cover，將每個被選 u 的 incident-gadget 鏈作一條 cover path。原邊若只有 u 被選，使用從 u 側進出的十二點完整 path；若兩端都選，使用兩條各六點 path。因每條原邊至少一端被選，每個 gadget 的十二點恰好各被訪問一次。依序用 s1、u1 鏈、s2、u2 鏈、...、sk、uk 鏈、s1 串成一個 cycle，亦各訪 selector 一次，故為 HC。

反向由啟用沿 incident 串鏈傳遞的性質，任何啟用 u 側都強迫整條 u 鏈啟用；此鏈接兩個 selector，途中維持同一原頂點身分。取所有啟用的 u。每個 selector 在 cycle 中度數為二，每條啟用鏈消耗兩個 selector 端點，因此恰有 k 條鏈；k=1 時鏈首尾以兩條不同邊接回同一 selector。每條 u 鏈只有一對端點，不能重複計數。每個邊 gadget 的零啟用模式形成兩個封閉小環，因此至少一個端點的鏈啟用。所得 k 個頂點覆蓋所有原邊。

因此 VC(G,k) iff HC(G')；構造、反解與規模皆多項式。固定 yes/no 例處理前處理的退化情況，不需用不存在的空鏈湊 selectors。



## VC gadget 的小例

原圖為 path u-v-w、k=1，可選 cover {v}。建兩個十二點 gadgets $W_{uv}$、$W_{vw}$；把 v 側第一個 gadget 的端口 6 接第二個 gadget 的端口 1，再用唯一 selector 接整條 v 鏈的首尾。

兩條原邊只有端點 v 被選：U 的 v 側為 b 側（x=1,y=0），V 的 v 側為 a 側（x=0,y=1）。串鏈邊 U.b6–V.a1 使兩側同時啟用；未啟用的 u、w 側頂點則包含在這兩條十二點完整路徑中。selector 與兩個 gadgets 共二十五點形成 HC。反之若選 u 的鏈，$W_{vw}$ 沒有任何被選端點，無法覆蓋，因此不會假造 cover {u}。

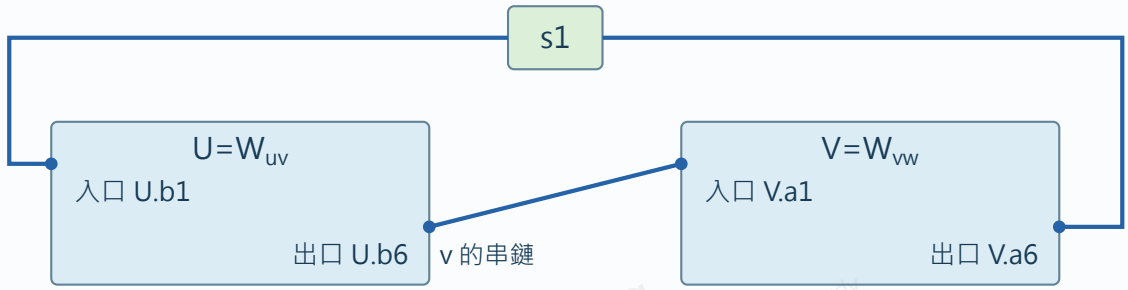
完整二十五點cycle可由以下兩條十二點路徑接成。記U=$W_{uv}$、V=$W_{vw}$；U的b側代表v，V的a側代表v：

- U：b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6。
- V：a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6。

由s1→U.b1開始，走完U後經串接邊U.b6→V.a1，再走完V，最後V.a6→s1閉合。U與V的同名點是不同頂點，故24個gadget點加s1恰25點，每點只出現一次。每個相鄰pair都能在十四內邊、串鏈或selector邊中逐項查得。

## VC gadget 的小例 (續)

原圖 u–v–w、cover={v}：唯一 selector 串成25點 cycle




各框內是已列出的完整12點路徑，U與V的同名點互不相同。

此圖顯示選定的HC；完整外邊依selector與incident-chain構造定義。



## HAMILTONIAN cycle $\leq_p$ TSP decision

給 HC instance $G = (V, E)$，$n = |V|$。建立完全圖 $K_n$，邊權重：

$w(u, v) = 1$ 若 $\{u, v\} \in E$，否則 $w(u, v) = 2$。令門檻 $B = n$。

**HC 到 tour。** G 的 Hamiltonian cycle 有 n 條原圖 edges，每條 weight 1，因此在 $K_n$ 是 cost n 的 tour。

**Tour 到 HC。** $K_n$ 任一個 tour 恰有 n 條邊。若其 cost $\leq n$，且每條 weight 至少 1，則每條邊都必須 weight 1。這些邊都在 G，tour 因而也是 G 的 Hamiltonian cycle。

CLRS 原書將原邊設為 0、缺邊設為 1，門檻為 0；此教材對每條邊加 1。每個巡迴恰有 n 條邊，總成本都增加 n，因此門檻改為 n，yes/no 不變。

構造是 $O(n^2)$。所以 $\mathrm{HC}(G) \iff \mathrm{TSP}(K_n, w, n)$，且 TSP decision的 certificate 是一條 tour，檢查排列與總成本即可，故 TSP decision in NP。

## TSP 實例追蹤

令G是三角形，edges={12,23,31}，n=3。完全圖的三條邊全部權重1，因此tour 1→2→3→1的成本3，門檻B=3。

門檻 B=3。此 tour cost 恰好等於 B，對應 G 的 Hamiltonian cycle。

若原圖刪去31，構造出的完全圖仍有31，但權重改為2。任何三點tour都含三條邊，成本1+1+2=4>B，所以不存在合格tour，與原圖沒有HC一致。對n<3的簡單無向圖，本章HC無解，直接映射至固定no TSP instance；上述每tour恰n邊的主要論證採n≥3。

optimization 版 TSP 要求真正最小總成本；decision 版只問是否存在 cost $\leq B$。若 optimization algorithm 在 polynomial time 回傳最佳值，決策版可直接比較 B；決策版 NP-hard 因而支持 optimization 版也難以精確 polynomial-time 求解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SUBSET-SUM、bit encoding 與 DP

decision 版 SUBSET-SUM 問：給正整數集合 S 與 t，是否存在子集合元素和恰為 t。

Certificate 是一組 selected indices；verifier 檢查每個索引在範圍內、不重複，且所選整數和恰為 t。長度與檢查時間 polynomial，所以 SUBSET-SUM in NP。

DP 狀態 $dp[i, s]$ 表示前 i 個 elements 可否達和 s。轉移為

$dp[i, s] = dp[i-1, s] \vee (a_i \leq s \wedge dp[i-1, s-a_i])$。

計算量 $O(nt)$。這是 pseudopolynomial，不是以 t 的 binary bit length 衡量的 polynomial。NP-hardness 的完整無進位數位表與兩向證明接在下頁；其巨大數值仍有多項式 bit 長，與此 DP 的 pseudopolynomial 界限一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3-CNF-SAT 到 SUBSET-SUM 的數位表

先把公式規格化為 n 個變數、k 個 clauses，每 clause 三個相異變數、無互補 tautology，每個變數至少出現一次；前面的去重/padding 處理可達成。恆真與恆假公式另輸出固定 yes/no 整數實例。

建立十進位 n+k 欄，前 n 欄對應變數，後 k 欄對應 clauses。這是緊湊建立整數的方式，最終仍以 binary 編碼交付。

- 每變數 $x_i$ 建兩數 $T_i$、$F_i$。兩者的第 i 個變數欄為 1，其他變數欄為 0。$T_i$ 在包含正 literal $x_i$ 的 clause 欄為 1，$F_i$ 在包含 $\neg x_i$ 的欄為 1，其餘為 0。
- 每 clause j 加 slack 數 $S_j$、$S'_j$，只在該 clause 欄分別放 1、2，其他欄為 0。
- target 的每個變數欄為 1，每個 clause 欄為 4。

總共 2n+2k 個正整數，每數 n+k 位十進位，故每數 binary 長度 $O(n+k)$，輸出總 bit 長 $O((n+k)^2)$，建構為多項式。若輸入要求集合而非 multiset，上述非退化假設保證兩個同變數數字不相同；不同變數的高位欄不同，slack 欄也不混同。



## 為何無進位，以及兩向解碼

任意子集合在每個變數欄至多加到 2；每個 clause 含三個 distinct literals，對應 $T_i/F_i$ 數最多提供三個 1，slack 最多加 1+2=3，因此 clause 欄至多 6<10。任何子集合都不會進位，不只是正確 witness 才不進位。

若原式可滿足，每變數依真假選 $T_i$ 或 $F_i$，使變數欄和全為 1。每 clause 的真 literals 數 r 為 1、2 或 3，可用 slack 子集合湊 4-r：r=1 選 1+2，r=2 選 2，r=3 選 1。所有欄符合 target，得到 subset-sum yes。

反向若數字子集合和為 target，無進位迫使每變數欄恰為 1，所以 $T_i$、$F_i$ 恰選一個，定義一致 assignment。clause target 為 4，但兩 slack 合計至多 3，所以至少一個被選 $T_i/F_i$ 在該欄有 1，對應原 clause 中一個真 literal。每 clause 都滿足。兩向與多項式規模一起完成 NP-hardness，再加索引 certificate 的 NP membership，SUBSET-SUM 為 NP-complete。

## 可手算的無進位例子

取 F=(x1∨x2∨x3) AND (¬x1∨x2∨¬x3)，欄序 x1,x2,x3,C1,C2：

| 數 | 十進位欄表示 |
|---|---|
| T1,F1 | 10010，10001 |
| T2,F2 | 01011，01000 |
| T3,F3 | 00110，00101 |
| S1,S1' | 00010，00020 |
| S2,S2' | 00001，00002 |
| target | 11144 |

assignment x1=true,x2=true,x3=false 選 T1+T2+F3=11122。每個 clause 真 literal 數都是二，再選 S1'=20、S2'=2，得 11144。沒有進位；高位每欄的 1 也直接證明每個變數只選一個真值數。

數值可能與 $10^{n+k}$ 同階，但 binary 表示僅 $O(n+k)$ bits。這正是 reduction 可多項式輸出，而 $O(nt)$ 精確 DP 仍非輸入 bit 長度多項式的原因。



## 可手算的無進位例子 (續)

T1+T2+F3=11122；再加 slack 00022，得到 target 11144

| | x1 | x2 | x3 | C1 | C2 | |
|---|---|---|---|---|---|---|
| T1 | 1 | 0 | 0 | 1 | 0 | 變數欄各恰為1 |
| T2 | 0 | 1 | 0 | 1 | 1 | 兩個 clause 各有 |
| F3 | 0 | 0 | 1 | 0 | 1 | 2個真 literal |
| slack | 0 | 0 | 0 | 2 | 2 | 各以 slack 2 補到4 |
| target | 1 | 1 | 1 | 4 | 4 | |

## 所有子集合都不能進位，才可反向解碼

若只對「依滿足指派選數」證明無進位，尚不足以證明反向：某個錯誤子集合也許靠進位偽造 target。本構造對任意子集合都有變數欄≤2、clause欄≤6，故十進位下絕不進位。進位排除是所有候選的全稱命題。

每個數的位元長度O(n+k)，把十進位欄轉成binary可用逐欄乘10加digit的$O((n+k)^2)$簡單bit演算法；2n+2k個數也仍多項式。驗證子集合和時，至多加O(n+k)個O(n+k)位元整數，總和多出O(log(n+k))位元，亦多項式。

例如只取x1=false,x2=false,x3=false，兩clause的真literal數為0、2。第一欄即使兩個slack都取，也只有3，不能達4；不會向其他欄借出或借入一個位值來掩飾不滿足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 近似保證比較的是所有輸入

NP-hard 最佳化仍可有多項式時間的近似解。本章每個演算法都先輸出可行解，再證明它與未知 OPT 的關係。單一測資接近最佳值不能證明近似比。

核心工具為 matching 下界、MST 下界、元素收費與動態規劃狀態修剪。前置知識為貪婪法、MST、歸納法及二進位輸入長度。

來源對照為 CLRS 3e §§35.1–35.3、35.5；MIT 2004 lecture 24；手寫筆記 PDF 54–56。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最小化與最大化的界不能倒置

設 A 為可行解值，OPT 為最佳值，ρ≥1。正值目標下的統一比率為 max(A/OPT,OPT/A)≤ρ。

| 問題 | 可行性給的界 | 近似保證 |
| --- | --- | --- |
| 最小化 | OPT≤A | A≤ρOPT |
| 最大化 | A≤OPT | A≥OPT/ρ |

例如最小化 OPT=10、A=15 是 1.5 倍；最大化 OPT=15、A=10 也是 1.5 倍。最大化寫 A≤ρOPT 幾乎沒有內容。OPT=0 時避免除法，直接用乘法式；subset sum 無可用正整數時回傳 0。

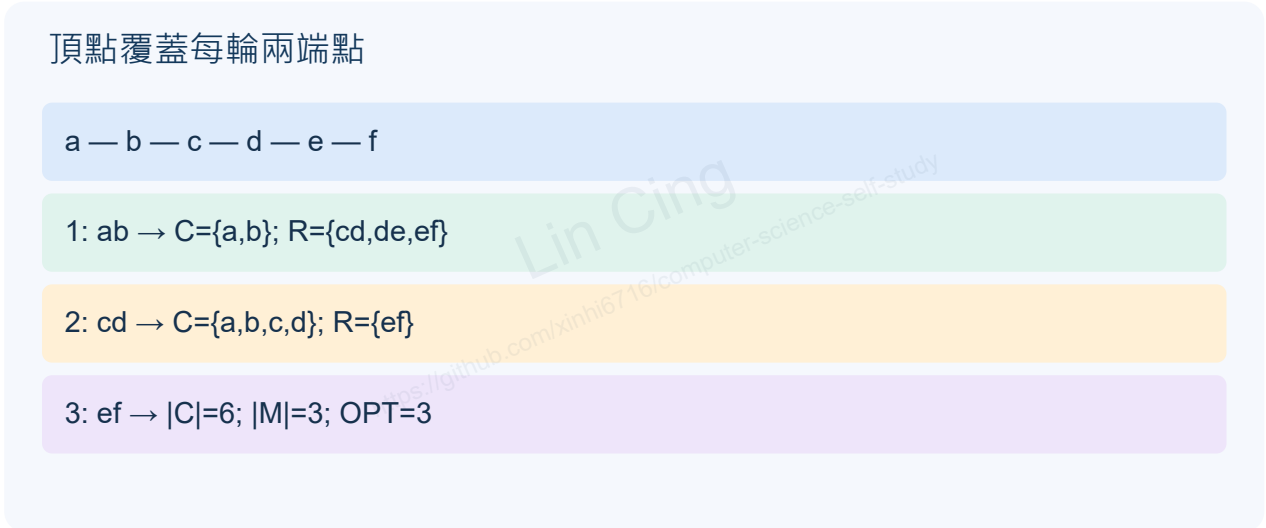
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 頂點覆蓋每輪選邊的兩端

無向圖 G=(V,E) 的 vertex cover C 必須碰到每一條邊，目標最小化 |C|。令 C=∅、R=E。當 R 非空，任選 uv∈R，把 u,v 加入 C，刪掉 R 中所有與 u 或 v 相接的邊。

終止時每條原始邊都因為已有端點在 C 才被刪除，因此 C 可行。可用一次掃描產生 maximal matching，再取全部端點，時間 O(|V|+|E|)。此演算法不直接處理帶權頂點。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐輪記錄把演算法與證明接起來

取 E={ab,bc,cd,de,ef}，依序挑 ab、cd、ef。

| 輪 | 挑邊 | 新增 C | 刪除的剩餘邊 | 輪後剩餘 |
|---|---|---|---|---|
| 1 | ab | a,b | ab,bc | cd,de,ef |
| 2 | cd | c,d | cd,de | ef |
| 3 | ef | e,f | ef | ∅ |

M={ab,cd,ef} 是 matching；C={a,b,c,d,e,f}。覆蓋 {b,d,f} 的大小為 3。三條互不相接的邊又要求任何覆蓋至少 3 點，故 OPT=3，A/OPT=6/3=2。這是確實達到 2 的例子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2 倍證明依靠 matching 下界

令 M 為各輪挑出的邊。選 uv 後立即刪掉所有相接邊，所以往後不可能再選含 u 或 v 的邊，M 的邊兩兩不共端點。

任一覆蓋 C* 都要覆蓋 M 每條邊。因端點互不重疊，這些邊不能共用同一個覆蓋頂點，故 $|C^*| \geq |M|$。演算法每條邊恰選兩個新端點，所以 $|C| = 2|M| \leq 2|C^*|$。

M 為 maximal，因終止後沒有邊可再加入；不必是 maximum。路徑 a-b-c-d 中 {bc} maximal 但大小 1，maximum 為 {ab,cd}、大小 2。只取每條 matching 邊任意一端，不能保證覆蓋其他邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Metric TSP 先明定圖與成本

輸入為至少三個頂點的完整無向圖，成本非負、對稱，且任意 u,v,w 滿足 $c(u,w) \le c(u,v)+c(v,w)$。目標為每點恰訪一次再回起點的 Hamiltonian cycle。

計算 MST T，任選根，沿樹做完整 DFS 走訪 W，每條樹邊去回各一次。保留每個頂點第一次出現的次序，最後回根，得到 tour H。

完整走訪 W 常重訪根或分支點，因此 W 只是 closed walk，尚未是可行的 TSP tour。完整圖確保所有捷徑邊存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 樹走訪到 tour 的數值範例

四點正方形 A=(0,0)、B=(1,0)、C=(1,1)、D=(0,1)，使用 Euclidean 距離。MST 可選 AB、BC、CD，成本 3。

完整走訪 A,B,C,D,C,B,A 成本 6；保留第一次出現並回根，得 A,B,C,D,A，成本 4。任何閉合巡迴至少有四條邊，每條距離至少 1，故 OPT≥4；此 tour 又達 4，OPT=4。

### 正方形的樹走訪與巡迴

A(0,0), B(1,0), C(1,1), D(0,1)

T={AB,BC,CD}; c(T)=3

W=A,B,C,D,C,B,A; c(W)=6

H=A,B,C,D,A; c(H)=4=OPT

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Double-tree 的三個不等式

從最佳 tour $H^*$ 刪去任一邊，得到 spanning tree。非負成本使刪邊不增成本，MST 最小性給 $c(T) \le c(H^*)$。

$W$ 每條樹邊走兩次，所以 $c(W)=2c(T)$。對 $W$ 中兩次保留頂點間的路段 $u=v_0,v_1,\dots,v_k=w$，反覆用三角不等式可得 $c(u,w) \le \Sigma_j c(v_{j-1},v_j)$。所有這些路段把 $W$ 分割，故捷徑後 $c(H) \le c(W)$。

合併即 $c(H) \le c(W)=2c(T) \le 2\mathrm{OPT}$。密集完整圖可用 $O(n^2)$ 的 Prim，走訪與取首次出現需 $O(n)$，總計 $O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 少了三角不等式，捷徑可能更貴

令四點完整圖 $AB=BC=CD=DA=1$，$AC=BD=M>4$。MST 可選以 A 為根的 AB、AD、BC，先訪 D 再訪 B 的 preorder 為 A,D,B,C，tour 成本 $2+2M$；最佳 tour A,B,C,D,A 成本 4。

完整走訪 A,D,A,B,C,B,A 成本 6。shortcut D→A→B 改為 D→B，成本由 2 增至 M；最後 C→B→A 改 C→A 也增加。比率 $(2+2M)/4$ 可任意大。$M=10$ 時為 $22/4=5.5$。

這證明 double-tree 保證會失效。更強的「任意常數近似皆不存在，除非 P=NP」另需 gap reduction，不能只靠此反例推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 集合覆蓋貪婪選最多新元素

給 universe U 與集合族 F，假設 ∪F=U；選最少集合覆蓋 U。每輪選 |S∩R| 最大的 S，R 為尚未覆蓋元素，直到 R=∅。平手按 A,B,C 的名稱順序。

例 U={1,⋯,6}，A={1,2,3,4}、B={1,2,5}、C={3,4,6}。先選 A 新增 4 點，接著 B 新增 5，最後 C 新增 6。貪婪共 3 組；B∪C=U 且沒有單一全集，故 OPT=2。

### 首次覆蓋的收費帳本

A={1,2,3,4}; B={1,2,5}; C={3,4,6}

A → 1,2,3,4: each 1/4

B → 5: 1; C → 6: 1

Total=3; OPT=2; H4=25/12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每輪成本平均分給首次覆蓋元素

一輪新增 k 個元素，就給這 k 個元素各收 $c_x=1/k$。每元素只收一次，因此貪婪集合數 $|G|=\Sigma_x c_x$。

上例 $c_1=c_2=c_3=c_4=1/4$，$c_5=c_6=1$。總收費 $4\times 1/4+1+1=3$。最佳組 B 的收費 $1/4+1/4+1=3/2$，C 也是 3/2。

對任意最佳覆蓋 F*，每元素至少被一組包含，且收費非負，故 $\sum_x c_x \le \sum_{S\in F^*}\sum_{x\in S} c_x$。重複計費只使右側更大。剩下要證明每組的總收費有 harmonic 上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Harmonic charging 的完整局部論證

固定任一 S∈F，令 s=|S|。依被首次覆蓋的先後排列其元素 $x_1,\dots,x_s$，同輪任意排序。

$x_j$ 被覆蓋那輪開始時，至少 s−j+1 個 S 元素未被覆蓋。即使 $x_j$ 前面有同輪元素，這個數仍是下界。貪婪選中的集合新增 k≥|S∩R|≥s−j+1 點，否則改選 S 更好。

所以 $c_{x_j}=1/k\le 1/(s-j+1)$。加總得 $\sum_{x\in S}c_x\le\sum_{j=1}^{s}1/(s-j+1)=H_s$。這個界適用所有候選集合，而不只貪婪選到的集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 由局部收費得到全域近似比

若 U=∅，貪婪與最佳解皆為空集合族，OPT=0；成本不等式成立，不計算 0/0，也不取空集合的最大值或 ln 0。以下假設 U≠∅，令 $H_0=0$、$d=\max_{S\in F}|S|\ge 1$。上一頁與最佳覆蓋的重複計費不等式給出

$|G|=\sum_x c_x\le\sum_{S\in F^*}\sum_{x\in S}c_x\le\sum_{S\in F^*}H_{|S|}\le H_d|F^*|$。

因 $H_d=1+\sum_{j=2}^{d}1/j\le 1+\int_1^d dx/x=1+\ln d\le 1+\ln|U|$，這是 $H_d$ 近似。例中 d=4、$H_4=25/12$，保證 3≤(25/12)×2=25/6；實際比率為 3/2。

簡單重掃實作每輪 O(mN)、至多 N 輪，為 $O(mN^2)$，m=|F|、N=|U|。這已足夠證明多項式時間；不把此界誤寫為 log N 無常數項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Subset sum 的精確列表為何爆炸

輸入正整數 $x_1,\cdots,x_n$ 與整數目標 $t\geq 1$，找總和不超過 $t$ 的最大值。允許空集合。精確狀態 $P_0=\{0\}$，$P_i=P_{i-1}\cup(P_{i-1}+x_i)$，只保留 $\leq t$ 的值。

每個值都能追溯到實際子集合，但列表可能有 $2^n$ 個值。$O(nt)$ 的布林 DP 只是 pseudo-polynomial，因二進位 $t$ 的長度是 $\Theta(\log t)$。

固定 $0<\varepsilon<1$，令 $\delta=\varepsilon/(2n)$。近似列表 $L_i$ 只保留相鄰數值相差足夠大的代表，目標為 $OPT/(1+\varepsilon)\leq \max L_n\leq OPT$。若 $n=0$ 或沒有任何 $x_i\leq t$，直接回傳 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trim 保留較小代表以維持可行性

對已排序去重列表，先保留最小值。掃描 $y$ 時，僅在 $y>(1+\delta)\text{last}$ 才保留 $y$ 並更新 last；否則由 last 代表 $y$。每個原值 $y$ 都有保留值 $z$ 滿足 $y/(1+\delta)\leq z\leq y$。

每輪先合併 $L_{i-1}$ 與 $L_{i-1}+x_i$，再 TRIM，最後刪掉 $>t$ 的值。始終保留 0。正整數的 0 需獨立處理，不能拿 $\log 0$。

用較小代表確保未來加相同的非負 $x_i$ 後仍不會超過對應真實和。保留值本身仍是實際子集合和；若需輸出子集合，可附帶前驅狀態與選物索引。

修剪列表 $\delta=0.05$, $t=308$

$L_1=\{0,104\}$

$L_2=\{0,102,206\}$

$L_3=\{0,102,201,303\}$

$L_4=\{0,101,201,302\}$; $OPT=307$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 修剪小例逐輪驗算

取 x=(104,102,201,101)、t=308、ε=0.4，n=4、δ=0.05。用精確比較避免浮點邊界。

| i | 合併後 | 修剪並刪掉 >308 後 |
|---|---|---|
| 1 | 0,104 | 0,104 |
| 2 | 0,102,104,206 | 0,102,206 |
| 3 | 0,102,201,206,303,407 | 0,102,201,303 |
| 4 | 0,101,102,201,203,302,303,404 | 0,101,201,302 |

第二輪 104≤102×1.05=107.1 被刪；第三輪 206≤201×1.05=211.05 被刪；第四輪 303≤302×1.05=317.1 被刪。結果 302=201+101，最佳為 307=104+102+101。比率 307/302≈1.01656≤1.4。



## 誤差沿 n 輪相乘的歸納證明

主張每個 $y\in P_i$、$y\le t$，都有 $z\in L_i$ 使 $y/(1+\delta)^i\le z\le y$。i=0 僅有 0，成立。

若 y 未使用 $x_i$，由歸納假設取上一輪代表 q。若 $y=p+x_i$，取 p 的代表 q，有 $q+x_i\ge p/(1+\delta)^{i-1}+x_i\ge (p+x_i)/(1+\delta)^{i-1}$，因 $x_i\ge 0$。兩種情況都在合併列表找到 $w\in[y/(1+\delta)^{i-1},y]$。

本輪修剪留下 $z\in[w/(1+\delta),w]$，得到主張。又 $z\le y\le t$，所以不會被容量刪除。套用 y=OPT，最大保留值 $A\ge \mathrm{OPT}/(1+\delta)^n$。

$\ln(1+\delta)\le\delta$，故 $(1+\delta)^n\le e^{\{\varepsilon/2\}}\le 1+\varepsilon$。最後一步由 $f(\varepsilon)=\ln(1+\varepsilon)-\varepsilon/2$，$f(0)=0$、$f'(\varepsilon)=1/(1+\varepsilon)-1/2\ge 0$，適用 $0\le\varepsilon\le 1$。故乘法式 $A\ge \mathrm{OPT}/(1+\varepsilon)$ 對所有情形成立。若 OPT=0，由可行性得 A=0，不取比率；若 OPT>0，前述下界給出 A>0，才可除以 A 得 $\mathrm{OPT}/A\le 1+\varepsilon$。

## FPTAS 必須對位元長度與 1/ε 多項式

修剪後的正值相鄰比 >1+δ，且全在 [1,t]。所以 $|L_i| \le 2+\ln t/\ln(1+\delta)$。因 $\ln(1+\delta) \ge \delta/(1+\delta)$，$\delta=\varepsilon/(2n)<1/2$，故 $|L_i| \le 2+3n \ln t/\varepsilon$。

設 $K=O(1+n \log t/\varepsilon)$。用兩個有序列表線性合併，每輪 $O(K)$ 次算術操作，n 輪為 $O(nK)$。若改用一般排序，額外 $O(\log K)$ 因子仍為多項式。

二進位輸入長度 B 包含各 $x_i$、t 與有理 ε 的編碼。列表整數需要 $O(\log t)$ 位元，合併前至多 $t+\max x_i$；有理比較可交叉相乘，因此位元運算也對 B 與 1/ε 多項式。ε 不視為免費的無限精度實數。

PTAS 只要求固定 ε 時對 B 多項式，例如 $B^{1/\varepsilon}$；FPTAS 另要求對 1/ε 多項式。本章修剪法滿足後者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 加權集合覆蓋：按新增元素的單價選擇

給定集合族覆蓋有限 universe $U$，每個集合成本為正有理數。令 $R$ 是尚未覆蓋元素；每輪選 $w(S)/|S \cap R|$ 最小且分母不為零的集合。空 $U$ 回傳空解；若集合聯集不含 $U$，回報無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐輪追蹤與反例

$U=\{1,2,3\}$；$A=U$、$B=\{1,2\}$、$C=\{3\}$，成本分別為 9、2、1。比值為 3、1、1。若先選 $B$，它的兩元素各付 1；再選 $C$，元素 3 付 1，總成本 3。$B, C$ 合起來成本 3，含 $A$ 的方案至少成本 9，故 OPT=3。單純選新增元素最多會先選 $A$，成本 9；若將其成本換成任意大的 $M$，比值即無界。

加權覆蓋：集合與逐輪收費

A: {1 2 3} 成本 9

B: {1 2} 成本 2

C: {3} 成本 1

第 1 輪：選 B | 2 / 2 = 1

元素 1 收費 1；元素 2 收費 1

第 2 輪：選 C | 1 / 1 = 1

元素 3 收費 1

演算法成本 ALG = 3；最優 OPT = 3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每個元素的收費構成局部上界

每輪選集合 $S$、新覆蓋 $k$ 個元素，將 $w(S)/k$ 分給這些元素。每個元素恰首次被覆蓋時付費，所以收費總額等於 ALG。固定任意集合 $A$，將其中元素按首次覆蓋排序為 $e_1, \dots, e_t$。覆蓋 $e_i$ 的那一輪開始時，$A$ 至少還有 $t-i+1$ 個未覆蓋元素。即使幾個元素同輪覆蓋，順序任排也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 把局部上界加到最優解

因貪婪法在該輪可選 $A$，$charge(e_i) \le w(A)/(t-i+1)$。加總得 $\sum_{e\in A} charge(e) \le w(A) H_{|A|}$。設 $d = \max_S |S|$，最優覆蓋至少含每個元素一次。收費非負，所以重複計費只放大右側：$ALG \le \sum_{A\in OPT}\sum_{e\in A} charge(e) \le H_d \cdot OPT$，其中 $H_d = \sum_{i=1}^{d} 1/i \le 1 + \ln d$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## MAX-3-CNF：期望保證

每句含三個相異變數；各變數獨立公平取值。每句八種取值中只有一種使它為假，故滿足機率為 $7/8$。設 $Y$ 為滿足句數、$m$ 為總句數，期望線性給 $E[Y] = 7m/8 \ge 7\,OPT/8$；句子之間不必獨立。單句仍有 $1/8$ 機率不滿足，故此界不是每次輸出的保證。重複文字 $(a \vee a \vee a)$ 的滿足率只有 $1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐一固定變數，不降低條件期望

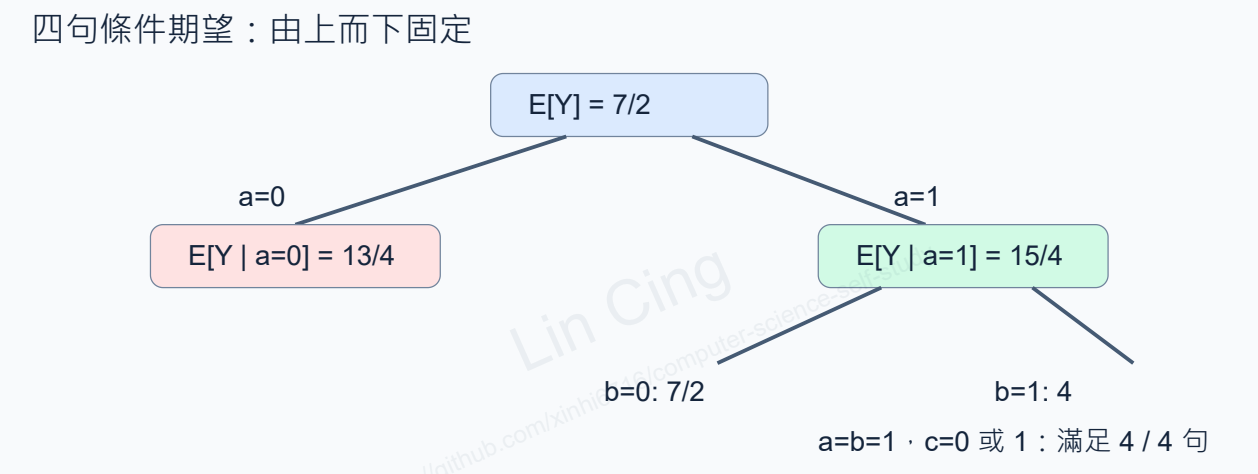
在部分指派下，已滿足句的貢獻為 1，全指派且為假的貢獻為 0；其餘若有 $r$ 個未定變數，貢獻為 $1-2^{-r}$。下一變數固定為 0 或 1 的兩個條件期望平均等於目前期望。選較大者，再重複；固定完畢後，條件期望就是實際滿足句數，至少 $7m/8 \geq 7OPT/8$。每步直接重算全部句子需 $O(nm)$ 時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 條件期望的四句數例

四句依序為 $(a \vee b \vee c)$、$(\neg a \vee b \vee c)$、$(a \vee \neg b \vee c)$、$(a \vee b \vee \neg c)$。初始期望 $7/2$。固定 $a=0$ 或 $1$ 得 $13/4$ 或 $15/4$，選 $1$；接著固定 $b=0$ 或 $1$ 得 $7/2$ 或 $4$，選 $1$；$c$ 任取均滿足四句。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 加權頂點覆蓋：整數模型與 LP

無向圖頂點具有非負有理權重。以 $x_v \in \{0,1\}$ 表示選取，最小化 $\sum_v w_v x_v$，每條邊 $uv$ 要求 $x_u + x_v \geq 1$。改為 $0 \leq x_v \leq 1$ 得 LP；每個整數解仍可行，故 $LP^* \leq OPT$。以多項式時間有理 LP 方法求精確最優解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 門檻捨入為何可行且不超過兩倍

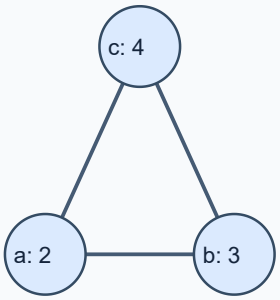
選所有 $x_v \geq 1/2$ 的頂點。若某邊兩端皆未選，兩端值都小於 $1/2$，總和小於 $1$，與 LP 限制矛盾。逐頂點 $1_{v \in C} \leq 2x_v$；乘非負權重並加總，得 $w(C) \leq 2LP^* \leq 2OPT$。單邊兩端權重為 1 和 $M$ 時，直接取 matching 兩端成本 $1 + M$、OPT=1，不能套用未加權法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三角形：下界證書與捨入

三角形 $a, b, c$ 權重為 2、3、4。半值解成本 $9/2$。將 $ab, ac, bc$ 三邊限制分別乘 $1/2, 3/2, 5/2$ 再相加，得到任何 LP 可行解成本至少 $9/2$，所以半值解為最優。門檻捨入選三點成本 9；整數最優取 $a, b$，成本 5，實際比值 $9/5$。

三角形 LP 與證書




$x_a = x_b = x_c = 1/2$

$ab * 1/2 + ac * 3/2 + bc * 5/2$

$2x_a + 3x_b + 4x_c \geq 9/2$

LP 最優值 = 9/2；捨入門檻 1/2

{a,b,c} 成本 9；{a,b} 最優成本 5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 隨機捨入：先處理漏覆事件

令 $N = |U| \geq 2$、成本非負，LP 最優解滿足每元素 $\sum_{S \ni e} x_S \geq 1$。取 $k = 3\lceil \log_2 N \rceil$，每集合獨立以 $p_S = \min(1, k x_S)$ 選取。若某包含 $e$ 的集合有 $p_S = 1$，它必被覆蓋；否則 $\Pr(e\text{漏覆}) = \prod_{S \ni e}(1 - p_S) \leq e^{-k} \leq N^{-3}$。聯集界給任一元素漏覆的機率至多 $N^{-2} \leq 1/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 成本與可行性要同時檢查

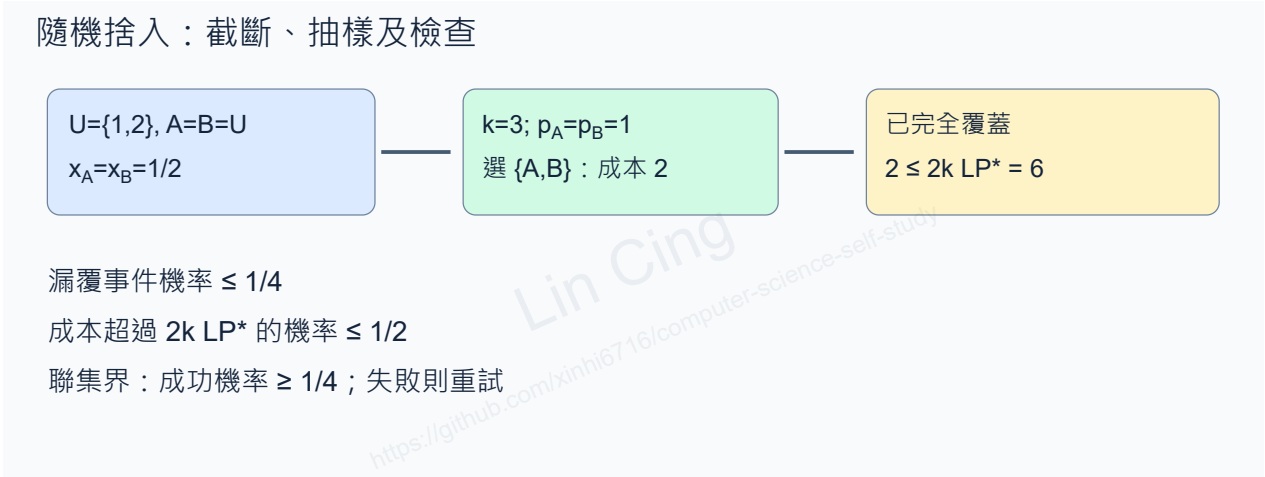
期望成本 $\sum_S w_S p_S \le kLP^*$。若 $LP^* > 0$，Markov 不等式給成本超過 $2kLP^*$ 的機率至多 $1/2$。加上漏覆機率，單次任一失敗機率至多 $3/4$，所以成功機率至少 $1/4$。檢查兩條件，失敗就獨立重試；期望試驗次數至多 4，且成功成本 $\le 2kOPT$。$LP^* = 0$ 時成本必為 0，僅需重試覆蓋；$N = 0$ 回空解，$N = 1$ 選最便宜的覆蓋集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一個隨機抽樣步驟

令 $U = \{1, 2\}$，$A = B = U$ 且各成本 1。取最優 LP 解 $x_A = x_B = 1/2$，$LP^* = 1$，$k = 3$，故 $p_A = p_B = 1$。兩者都選，成本 2 且完整覆蓋，滿足 $2 \le 2kLP^* = 6$。此例示範截斷至 1；不是失敗機率界的緊例。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 非 metric TSP：建構有間隙的輸入

假設存在固定 $\rho \geq 1$ 的確定性多項式近似法，取固定整數 $R \geq \rho$。把 $n$ 點 Hamiltonian cycle 輸入 $G$ 轉為完全圖：原邊成本 1，非邊成本 $M = Rn + 1$。若 $G$ 有 Hamiltonian cycle，$OPT = n$，近似輸出成本至多 $Rn$。若沒有，任何 tour 至少使用一條非邊，成本至少 $M + n - 1 > Rn$。

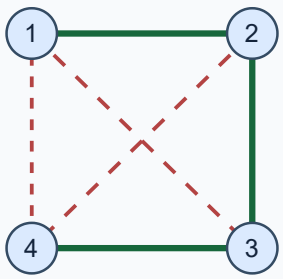
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 成本門檻會判定 Hamiltonian cycle

用近似法輸出是否 $\leq Rn$ 判定原圖有無 Hamiltonian cycle，所以該近似法存在將推出 $P = NP$。$M$ 的二進位長度為 $O(\log n)$，完全圖輸出長度多項式。$n = 4, R = 2$ 時 $M = 9$；若原圖是四點路徑，缺邊成本 9，每個 tour 至少一條缺邊，成本至少 12，高於門檻 8。這些成本可能違反三角不等式；證明也不直接排除僅有期望保證的隨機演算法。

路徑變完全圖：成本落差




n=4, R=2, M=Rn+1=9

綠色實線：原邊成本 1

紅色虛線：非邊成本 9

路徑 1-2-3-4-1：成本 12

無 Hamilton 圈：每條巡迴 ≥ 12 > 8

有 Hamilton 圈：近似輸出 ≤ 8

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三文字子句：期望與單次成功機率

設 $m \geq 1$，每個 clause 恰有三個相異變數（可有否定）。各變數獨立均勻賦值，每 clause 恰有一種使它為假的局部指派，故 $\mathbb{E}[X] = 7m/8$，其中 $X$ 是滿足子句數。子句之間即使共享變數，期望的線性仍成立；這裡並未假設子句滿足事件獨立。

設 $q = \Pr(X \geq 3m/4)$。由 $0 \leq X \leq m$，在成功時用 $m$ 上界、失敗時用 $3m/4$ 上界：

$$\frac{7m}{8} = \mathbb{E}[X] \leq qm + (1-q)\frac{3m}{4}, \qquad q \geq \frac{1}{2}.$$

因 $OPT \leq m$，成功時同時有 $X \geq 3OPT/4$。這是成功機率下界，不是每次都成功；若有一文字或重複變數子句，$7/8$ 的前提可能失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 可檢查的重試與小例子

獨立重做 $r$ 次並保留最多滿足子句的指派，達不到 $3m/4$ 的機率至多 $2^{-r}$；每次都能直接計數檢查，無須知道 OPT。例：一個 clause $(a \vee b \vee c)$，8 種指派中 7 種滿足，因此 $X \geq 3/4$ 的機率是 $7/8$，高於通用下界 $1/2$。$m = 0$ 直接回傳任意指派；不存在比值分母問題。

這是「以高機率得到至少 $3OPT/4$」的保證。期望近似比、單次成功率與每次必然的保證不同；若需要確定性結果，可以採用逐變數最大化條件期望的另一路做法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Christofides：目標與輸入假設

輸入是 $n \geq 3$ 的完整無向圖；邊成本非負、對稱，且對所有頂點滿足三角不等式 $c(u, w) \leq c(u, v) + c(v, w)$。允許不同頂點距離為零。目標是最小成本 Hamilton 巡迴；令其成本為 OPT。以下演算法回傳可行巡迴，成本至多 $3OPT/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從樹到偶度連通多重圖

1. 求最小生成樹 $T$，收集其中奇度頂點集合 $O$。
2. 在 $O$ 的完整子圖求**最小權完美配對** $M$。
3. 將 $T$ 和 $M$ 合併成多重圖，相同邊若同時出現在兩者，保留兩份。
4. 沿所有邊走一次求 Euler 閉行走，之後略過已見頂點並回起點。

握手定理給 $|O|$ 為偶數。每個 $O$ 頂點在配對中恰加一條邊，度數由奇轉偶；其他點不變。因 $T$ 連通，合併圖連通且每點偶度，故存在 Euler 閉行走。一般圖最小權完美配對可多項式時間求得；任意配對不保證以下成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第一個下界：樹不比最佳巡迴貴

由最佳巡迴刪除任一邊得到生成樹。邊成本非負，刪邊不增加成本；最小生成樹不比該樹貴，所以 $c(T) \leq OPT$。若容許負邊，這一步不能照用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第二個下界：從最佳巡迴抽出奇度點

按照最佳巡迴中遇到 $O$ 的順序，將其餘頂點逐一略過。完整圖提供每條捷徑，反覆使用三角不等式得 $O$ 上的封閉循環成本至多 OPT。生成樹在 $n \geq 3$ 時至少有兩個奇度點。若 $|O| = 2$，這個循環視為兩點間往返，各方向各一份邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 交替拆成兩個完美配對

奇度點數為偶數。把上述循環的邊按順序交替分成兩組；每組都讓每個奇度點恰好出現一次，因此是完美配對。兩組成本之和不超過 OPT，所以較便宜的一組成本至多 $OPT/2$。最小權配對 $M$ 不比它貴，故 $c(M) \le OPT/2$。兩點時兩份往返邊各構成同一條邊的配對，論證仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 捷徑保持可行且不增成本

Euler 閉行走總成本恰為 $c(T) + c(M)$。按首次出現順序保留每個頂點，其餘走訪由直接邊取代；每次略過一段路都可由三角不等式證明不增成本。最後回起點，得到每個頂點恰好一次的巡迴。故

$c(\text{tour}) \le c(T) + c(M) \le OPT + OPT/2 = 3OPT/2$。

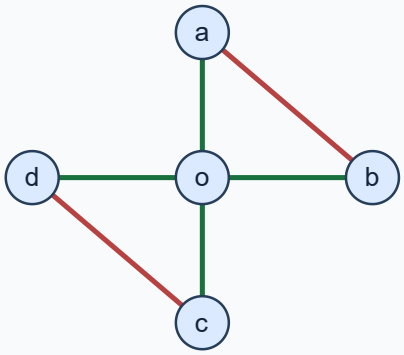
零成本 OPT 時，這個乘法不等式仍成立，不需除以 OPT。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 四葉星形 metric：樹、配對與實際巡迴

五點 $o, a, b, c, d$：中心到各葉距離 1，兩葉距離 2。這是樹路徑距離，滿足三角不等式。星形四邊總成本 4，是 MST：每個生成樹有四邊且每邊成本至少 1。中心度 4，四葉度 1，所以 $O = \{a, b, c, d\}$。各葉對距離均為 2，選 $M = \{ab, cd\}$ 成本 4。

星形 MST、配對與 Euler 行走




綠線：MST，四邊各成本 1

紅線：配對 ab、cd，各成本 2

Euler：o → a → b → o → c → d → o

成本 1+2+1+1+2+1 = 8

略過第二次 o：o → a → b → c → d → o

巡迴成本 8 = OPT

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 數例的 Euler 行走與 OPT

合併圖可走 $o \to a \to b \to o \to c \to d \to o$，成本 $1+2+1+1+2+1=8$。略過第二次出現的 $o$ 得 $o \to a \to b \to c \to d \to o$，成本 $1+2+2+2+1=8$。任何五點巡迴在中心有兩條成本 1 的邊，其餘三邊連兩葉、各成本 2，所以 OPT=8。此例輸出最優；$3/2$ 是對所有符合假設的輸入的上界，不是此例的實際比值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, chapters 34 and 35. MIT 6.046J Fall 2004, Lecture 24, approximation algorithms. Original handwritten algorithms notes, complexity and approximation pages.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 字串匹配

找出樣式在文字中的所有出現位置，包含重疊。比較直接搜尋、滾動雜湊、有限自動機與 KMP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 本章路線

先定義輸出與邊界情況，再追蹤篩選、狀態轉移與前綴回退；區分正確性與執行時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 字串比對問題

文字 T 長 n，樣式 P 長 m。以 0 起算，輸出每個使 T[s:s+m]=P 的位置 s，包含重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 邊界契約

空樣式匹配 n+1 個邊界 0..n。非空樣式若 m>n 則沒有匹配。以下先假設 1≤m≤n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 四種方法

樸素法直接比較。Rabin–Karp 以雜湊篩選。DFA 預先建立轉移表。KMP 用前綴函數按需回退。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 樸素比對

對每個 s=0..n−m，依序比較 P[j] 與 T[s+j]。全部相等則輸出 s。每次只將 s 加一，保留重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 樸素法逐字示例

T=ababa，P=aba。s=0 比較 a、b、a，成功；s=1 首字 b≠a；s=2 比較 a、b、a，成功。輸出 0、2。

**Naive / 樸素**

```
T   a b a b a
P   a b a     → 0
P     a b a   → fail
P       a b a → 2
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 樸素法成本

最多 (n−m+1)m 次字元比較，最壞時間 O((n−m+1)m)，額外空間 O(1)。重複字元可逼近最壞情況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rabin–Karp 指紋

令字元值在 0..d−1，長度 m 的字串雜湊為 Σ x[j]d^(m−1−j) mod q。相等字串必有相等雜湊，反向不保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 滾動雜湊

令 h=d^(m−1) mod q。移出字元 a、加入 b，更新 t′=(d(t−a h)+b) mod q。將負餘數正規化到 0..q−1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 滾動公式從哪裡來？

視窗 W=T[s:s+m]、新字元 c=T[s+m]。把字元視為數值，令未取模的多項式 H(W)=Σ(j=0..m−1) W[j]d^(m−1−j)。

扣除最高位 W[0]d^(m−1)，乘 d，再加 c：

H(T[s+1:s+m+1]) = d(H(W)−W[0]d^(m−1))+c。

逐項移位得到下一視窗。對 q 取模，代入 t_s=H(W) mod q、h=d^(m−1) mod q，即得 t_(s+1)=(d(t_s−W[0]h)+c) mod q；q≥2 不必是質數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 滾動掃描的迴圈不變量

處理視窗 s 之前，t_s 等於當前視窗 T[s:s+m] 的 hash；p 是 P 的 hash。初始化時用 Horner 法算出 p 和 $t_0$。若 s<n−m，前頁恆等式保證更新後的不變量對 s+1 仍成立；最後一格不讀 T[n]。

相等字串必有相同 hash，故真匹配不會被篩掉；hash 相等時再逐字確認，故碰撞不會造成誤報。例中中間差可為負，最後正規化至 0..q−1。

### d=10、q=13、m=3：視窗 123 → 234

h=100 mod 13=9；(10·(6−1·9)+4) mod 13=0。

負數餘式須正規化；234 mod 13=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 滾動雜湊示例

d=10、q=13、m=2，T=1234、P=34。視窗 12、23、34 的雜湊依序為 12、10、8。樣式雜湊為 8，最後逐字確認。

**視窗向右移動一格**

移除 1 → 乘 10 → 加入 3
$(10\cdot(12 - 1\cdot10) + 3) \bmod 13 = 10$

移除 2 → 乘 10 → 加入 4
$(10\cdot(10 - 2\cdot10) + 4) \bmod 13 = 8$

h = 10；先前雜湊可取模，結果不變。



## 雜湊碰撞與成本

當 $1 \le m \le n$，且模運算為常數時間，令 H 為所有雜湊相等視窗數，包含真匹配。逐字確認的時間為 $O(n+m+mH)$，最壞為 $\Theta(n+m+m(n-m+1))$。

$T=a^n$、$P=a^{\lfloor n/2 \rfloor}$ 時，每個視窗都真匹配；即使沒有假碰撞，確認仍需 $\Theta(nm)$。

若隨機雜湊使每個不相等視窗的碰撞機率至多 1/q，真匹配數為 V，期望時間為 $O(n+m+mV+mn/q)$。接近線性還須限制真匹配的確認成本，例如 $V=O(1)$ 且 $q \ge m$。

## 隨機 base 雜湊：隨機化對象與條件

固定輸入後，在質數 q 的有限域 F_q 均勻選 b；字元編碼在 F_q 內互異。長度 m 的字串 X 使用 H_b(X)=Σ(j=0..m−1) X[j]・b^(m−1−j)。採用 b⁰=1，包括 b=0。

對固定相異 X、Y，差 D(b)=H_b(X)−H_b(Y) 是非零多項式，次數至多 m−1。域上非零多項式最多有其次數個根，所以碰撞機率 ≤min(1,(m−1)/q)，一般不能直接寫成 1/q。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 多項式根數與期望成本

根數理由：若 r 是根，因式定理可寫 D(x)=(x−r)Q(x)；其餘不同根都必為 Q 的根。反覆提出根，根數不能超過次數。

令 N=n−m+1、真匹配數 z。對每個非匹配窗口設假命中指示變數，期望線性相加，無須窗口碰撞事件獨立：E[f]≤(N−z)min(1,(m−1)/q)。逐字確認版本的期望時間為 O(n+m+mz+m(N−z)min(1,(m−1)/q))，假設域運算 O(1)。真匹配驗證成本 mz 不會消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一次假命中與兩次真匹配

取數字編碼、q=11、b=10、m=2，P=12、T=122312。H_b(xy)=10x+y mod 11；P 的 hash 是 1。

五個窗口依序為 12、22、23、31、12，hash 為 1、0、1、9、1。窗口 23 的 hash 與 P 相等卻字串不同，逐字確認後拒絕；位置 0、4 是真匹配。

### q=11，b=10：雜湊相同仍需逐字確認

| s = 0 | s = 1 | s = 2 | s = 3 | s = 4 |
|---|---|---|---|---|
| 12 → 1 | 22 → 0 | 23 → 1 | 31 → 9 | 12 → 1 |
| 真匹配 | 略過 | 假命中 | 略過 | 真匹配 |

P=12 的 hash=1；核對後輸出 0、4，拒絕 23。



## 同一例子的確切碰撞機率

現在不固定 b=10，而在 $F_{11}$ 均勻選 b。三個非匹配窗口 22、23、31 與 P=12 相碰的方程分別為 b=0、b=10、b=6；每個窗口碰撞機率都是 1/11。

令各窗口碰撞指示變數為 $I_1$、$I_2$、$I_3$；$E[f]=E[I_1]+E[I_2]+E[I_3]=3/11$。事件是否互相獨立不影響這個加總。當 b=10，只有 23 假命中；仍須驗證兩個真匹配。

## 自動機狀態

狀態 q 是已讀文字的最長後綴同時為 P 前綴的長度，0≤q≤m。讀入 c 後更新 δ(q,c)。若 q=m，輸出目前結尾位置減 m 加一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



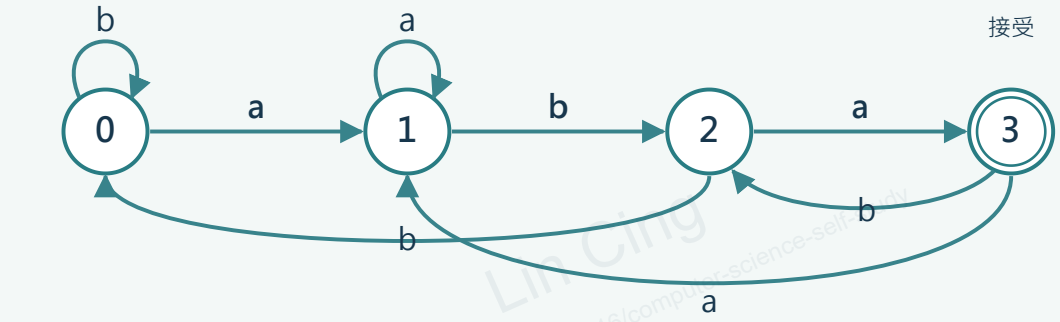
## DFA 轉移定義

δ(q,c)=max{k≤m：P[:k] 是 P[:q]+c 的後綴}。包含接受狀態 m 的轉移，才能找出重疊匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DFA 轉移表

P=aba，字母表 {a,b}。各狀態的 (a,b) 轉移為 0:(1,0)、1:(1,2)、2:(3,0)、3:(1,2)。掃描 O(n)，表佔 O(m|Σ|)。






## 從最長後綴得到自動機狀態

令 P⟨k⟩ 表示長度 k 的樣式前綴，P⟨0⟩=ε；P[i] 則表示從 0 起算的第 i 個字元。σ(x) 是同時為 x 後綴的最長樣式前綴長度。狀態 q=σ(x)，轉移定義 δ(q,a)=σ(P⟨q⟩a)。

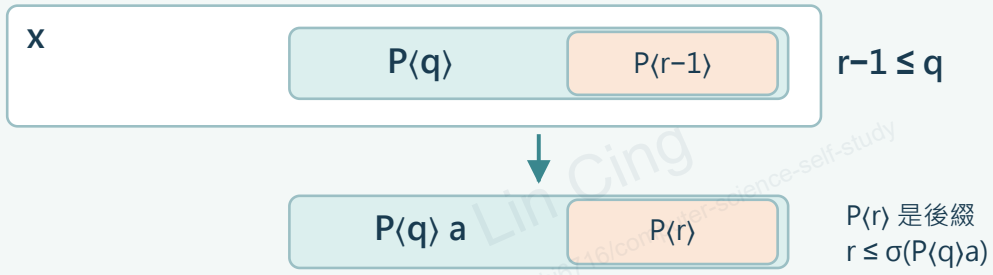
核心等式是 δ(σ(x),a)=σ(xa)：不必保留整段 x，就能決定加入 a 後的狀態。

## 狀態等式：第一個方向

設 q=σ(x)、r=σ(xa)。若 r>0，從 P⟨r⟩ 去掉末字後，P⟨r−1⟩ 是 x 的後綴，故 r−1≤q。

P⟨q⟩ 與 P⟨r−1⟩ 都是 x 的後綴，較短者是較長者的後綴；因此 P⟨r−1⟩ 是 P⟨q⟩ 的後綴，再接 a 得 P⟨r⟩ 是 P⟨q⟩a 的後綴。故 r≤σ(P⟨q⟩a)。若 r=0，此不等式顯然成立。

先比較 x 的兩個後綴，再把同一字元 a 接回




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 狀態等式：反向與匹配正確性

P⟨q⟩ 是 x 的後綴，所以 P⟨q⟩a 是 xa 的後綴；P⟨q⟩a 的任何後綴也是 xa 的後綴。故 σ(P⟨q⟩a)≤σ(xa)=r。合併得兩者相等。

初始 σ(ε)=0；每讀一字用等式歸納，狀態恰為 σ(已讀文字)。狀態 m 當且僅當 P 是已讀文字的後綴，這恰好對應一個匹配。接受後仍需轉移，才能找到重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用 π 建立整張 DFA 表

令 π[q−1] 是 P⟨q⟩ 的最長真 border 的長度。對有限字母表 Σ，m≥1：

• q=0：若 a=P[0]，δ(0,a)=1；否則為 0。 • 1≤q≤m：若 q<m 且 a=P[q]，δ(q,a)=q+1；否則 δ(q,a)=δ(π[q−1],a)。

注意 q=m 時不能讀 P[m]，直接使用第二個分支；π[q−1]<q，所以依賴的表列已建立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼建表可以沿 border 回退？

當直接延伸失敗，長度 q+1 的候選不成立；任何較短且非空的匹配 P⟨k⟩，移去末字後，P⟨k−1⟩ 必是 P⟨q⟩ 的真 border。所有真 border 都沿 π 鏈出現，因此以最長真 border 開始重新處理同一字元 a 不會漏掉候選；k=0 永遠可行。

計算 π 用 O(m) 時間及空間，m+1 列、每列 |Σ| 個常數時間格子合計 O(m|Σ|) 時間與空間；假設可 O(1) 存取字母與表格。掃描另需 O(n) 與輸出成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前綴函數

π[i] 是 P[:i+1] 的最長真前綴且同時是後綴的長度，0≤π[i]≤i。真前綴排除整個字串。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 前綴函數示例

P=ababaca，索引 0..6；π=[0,0,1,2,3,0,1]。字元 a b a b a c a 對應 0 0 1 2 3 0 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐字建立前綴函數

i=0:a→0；1:b 與 a 不同→0；2:a=a→1；3:b=b→2；4:a=a→3；5:c≠b，3→1，c≠b，1→0，c≠a→0；6:a=a→1。

**i = 5：比較 c 時，候選邊界反覆回退**

| 索引 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |
|---|---|---|---|---|---|---|---|---|
| | a | b | a | b | a | c | a | |
| k = 3 | | | a | b | a | b ≠ c | | 3 → 1 |
| k = 1 | | | | | a | b ≠ c | | 1 → 0 |
| k = 0 | | | | | | a ≠ c | | π[5] = 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 回退為何有效

若已有長度 k 匹配但下一字不符，較短候選必須是 P[:k] 的邊界。最長候選為 π[k−1]。反覆回退直到匹配或 k=0。例：P=ababaca、k=5、下一字元 x，回退鏈是 5→3→1→0，不能只回退一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## border 鏈為什麼列盡所有候選？

設 B 是字串 X 的最長真 border。任一較短真 border C 同時是 X 的前綴與後綴，也是 B 的前綴與後綴，因此是 B 的 border；反過來 B 的 border 也是 X 的 border。

從長度 |X| 開始反覆取 π[length−1]，每次嚴格變小，必以 0 結束；由上述雙向包含，此鏈恰列出所有真 border 的長度，沒有漏項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

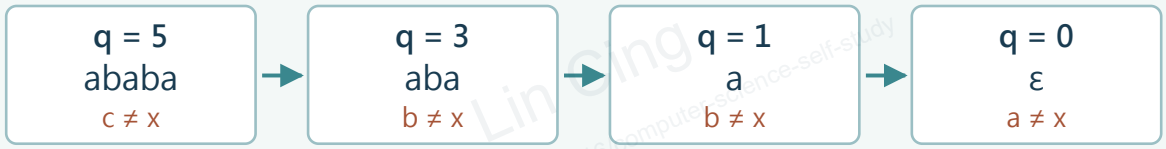


## 前綴函數：不變量與正確性

若 m=0，π 為空表，不讀取樣式字元。若 m≥1，先設 π[0]=0：單一字元沒有非空真 border，這是歸納基例。接著對 i=1,⋯,m−1，已知 π[0..i−1]，令 k=π[i−1]。當 P[k]≠P[i] 且 k>0，沿 k←π[k−1] 反覆回退；同一 P[i] 不動。

每次回退留下的候選依長度遞減且列盡，因此第一個能延伸者給最長 border，令 π[i]=k+1。若到 k=0 仍不相等，答案是 0；若 P[0]=P[i]，答案是 1。這完成對 i 的歸納。

### P = ababaca：對同一字元 x 連續回退




5 → 3 → 1 → 0；在 q=0 比較 P[0]，不符就維持 0。

較短 border 必為最長 border 的 border。

## 為什麼建 π 只需線性時間？

以 k 為非負勢能。每個 i 最多增加 k 一次、幅度至多 1；每次 while 回退使 k 至少減 1。從 k=0 出發，累計回退次數不超過累計增加量 ≤m−1。

每個 i 還有至多一次終止比較與常數工作，因此建表 O(m)，KMP 掃描同理 O(n)。這是跨整趟執行的界，單一字元仍可能觸發多次回退。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## KMP 預處理

π[0]=0。i 從 1 到 m−1，令 k=π[i−1]；只要 k>0 且字元不符，就反覆令 k=π[k−1]；停止回退後若相符則 k 加一；令 π[i]=k。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP 掃描

令 q=0。逐字 c：只要 q>0 且 P[q]≠c，就反覆令 q=π[q−1]；停止回退後若 P[q]=c，q 加一。q=m 時輸出 i−m+1，再設 q=π[m−1]，保留重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## KMP 掃描的兩種狀態

假設 m≥1，令 σ(x) 是 x 的後綴中最長樣式前綴的長度（可為 m）；令 ρ(x) 是最長長度小於 m 的樣式前綴後綴長度。

每輪開始的 q=ρ(x)，其中 x 是已讀文字；初始 q=ρ(ε)=0。若 σ(x)<m，則 ρ(x)=σ(x)；若 σ(x)=m，則 ρ(x)=π[m−1]。因此接受後儲存的是最長真 border，不是 m。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一次掃描等價於 DFA 轉移

對同一新字元 a，while 沿 border 鏈從 q 遞減嘗試；第一個可延伸的候選最長，全部失敗則為 0。因此暫存狀態 r=δ(q,a)。

若 σ(x)<m，q=σ(x)；若 σ(x)=m，q=π[m−1] 且 δ(m,a)=δ(π[m−1],a)。由狀態等式 δ(σ(x),a)=σ(xa)，兩種情況均有 r=σ(xa)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼輸出後回退不漏重疊？

r=m 當且僅當樣式為已讀文字 xa 的後綴，因而輸出位置 i−m+1 恰好正確；接著 q←π[m−1]=ρ(xa)。若 r<m，令 q←r=ρ(xa)。兩種情況均恢復下輪不變量。

對 P=aba、T=ababa，位置 2 與 4 分別輸出 0 與 2；兩次接受後都保留 q=1。後續字元從該 border 繼續，因此不漏重疊。

**P=aba、T=ababa：先接受，再保留 border**

| i | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| T[i] | a | b | a | b | a |
| 輸入前 q | 0 | 1 | 2 | 1 | 2 |
| r = σ | 1 | 2 | 3 | 2 | 3 |
| 輸出後 q | 1 | 2 | 1 | 2 | 1 |
| 輸出 | – | – | 0 | – | 2 |

r=3 時輸出，然後 q=π[2]=1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP 逐字追蹤

T=ababa，P=aba，π=0,0,1。各 i、字元、q：0 a→1；1 b→2；2 a→3，輸出 0 並回退 1；3 b→2；4 a→3，輸出 2 並回退 1。

**P = aba：完整匹配後保留長度 1 的邊界**

| T 的索引 | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 讀入字元 | a | b | a | b | a |
| 狀態 q | 1 | 2 | 3 | 2 | 3 |
| 匹配位置 | | | 0<br>3 → 1 | | 2<br>3 → 1 |

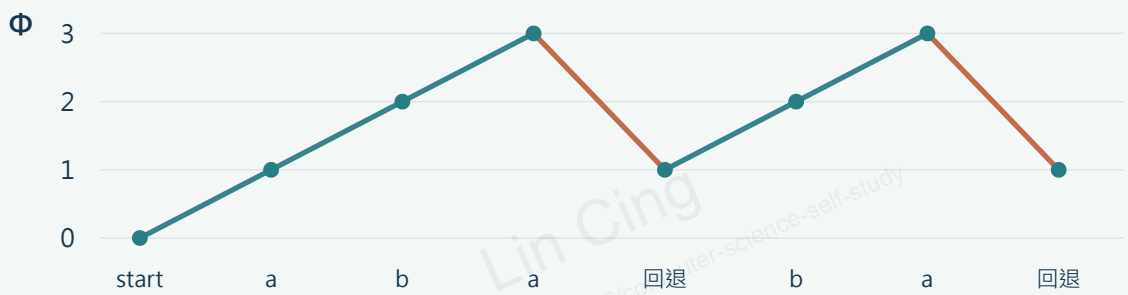
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 線性時間的攤銷證明

預處理及掃描各自用 q 或 k 為勢能。每個輸入位置至多使勢能增加 1；每次非零回退至少減少 1，故總回退不超過總增加。其他比較每位置 O(1)，總時間 O(m+n)。

**勢能 Φ = q：上升至多一次，回退必下降**




每次下降至少 1；總回退次數不超過總上升次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP 時間與空間

預處理 O(m)，掃描 O(n)，π 表 O(m)。儲存 z 筆輸出另需 O(z)；若串流輸出則不計入工作空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 四法成本比較

在 1≤m≤n 下，樸素法最壞 O(nm) 時間及 O(1) 空間；Rabin–Karp 最壞 O(nm) 及 O(1) 工作空間；DFA 掃描 O(n)、表 O(m|Σ|)；KMP O(n+m) 及 O(m)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 實作陷阱

明確指定字元與索引單位：位元組、Unicode code point 與 grapheme cluster 不同。先處理空樣式；雜湊命中須確認；完整匹配後不可直接重設為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 驗算策略

用直接枚舉當 oracle，窮舉小字母表所有短 T、P，比較 Rabin–Karp、DFA、KMP 的完整輸出。測試碰撞、空樣式、m>n 和重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 核心概念

已匹配前綴的邊界組成可行回退鏈。KMP 沿此鏈回退，DFA 預算狀態更新，雜湊法以確認步驟保證正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

CLRS, Introduction to Algorithms, 3rd ed., chapter 32, pp. 990–1006. MIT 6.046J Fall 2004, Lecture 22.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 矩陣運算與 FFT

本章從矩陣分塊與乘法出發，經消去、PA=LU 和最小平方，再以 DFT 與 FFT 求多項式卷積。

矩陣運算以分塊與因子分解組織計算；FFT 以偶奇拆分重用單位根。留意維度、主元、轉換慣例與補零長度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 矩陣運算把同一組係數重用

方程 $Ax = b$ 的重點是把係數與右端分開。固定 $A$，換 $b$ 時重用分解。全節採列向量、實數矩陣、$PA = LU$，$L$ 的對角線為 1。最小平方的殘差固定為 $r = b - A\hat{x}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 內維度相同才能相乘

$A \in \mathbb{R}^{m \times k}$，$B \in \mathbb{R}^{k \times n}$，則 $C = AB \in \mathbb{R}^{m \times n}$，$c_{ij} = \sum_{t=1}^{k} a_{it} b_{tj}$。每格做 $k$ 次乘法，合計 $mkn$ 次乘法與 $mn(k-1)$ 次加法。$AB$ 可定義不代表 $BA$ 可定義。

### 矩陣尺寸與一格內積

A：m × k

B：k × n

AB：m × n

共同索引 t = 1…k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 逐格乘法保留索引的意義

取 $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$，$B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$。$c_{12} = 1(0) + 2(1) + 3(1) = 5$，故 $AB = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$。$BA$ 是 $3 \times 3$，不能與 $AB$ 相等。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分塊相乘仍須逐塊檢查維度

把內維度分成 $k = k_1 + k_2$。令 $A = [A_1\ A_2]$，$B = [B_1; B_2]$，其中 $A_s$ 為 $m \times k_s$，$B_s$ 為 $k_s \times n$。則 $AB = A_1B_1 + A_2B_2$。四象限分塊時 $C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$。區塊不能當可交換純量。

### 內維度切成 k₁ + k₂

$A_1 : m \times k_1$

$B_1 : k_1 \times n$

$A_1B_1 + A_2B_2 : m \times n$

$A_2 : m \times k_2$ ; $B_2 : k_2 \times n$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 分治減少乘法數才改變指數

一般正方分塊有 $8T(n/2) + \Theta(n^2) = \Theta(n^3)$。Strassen 以 7 個乘積重組四塊，得到 $7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$。可補零到二次冪再裁切。這是算術成本分析，未保證任意小矩陣更快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen：七個乘積如何構成？

將方陣分成同尺寸四塊 $A=[A_{11},A_{12};A_{21},A_{22}]$、$B=[B_{11},B_{12};B_{21},B_{22}]$，以七次區塊乘法計算：

$P_1=(A_{11}+A_{22})(B_{11}+B_{22})$；$P_2=(A_{21}+A_{22})B_{11}$；$P_3=A_{11}(B_{12}-B_{22})$；$P_4=A_{22}(B_{21}-B_{11})$。

$P_5=(A_{11}+A_{12})B_{22}$；$P_6=(A_{21}-A_{11})(B_{11}+B_{12})$；$P_7=(A_{12}-A_{22})(B_{21}+B_{22})$。區塊乘法不可交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Strassen：重組與正確性

$C_{11}=P_1+P_4-P_5+P_7$，$C_{12}=P_3+P_5$；$C_{21}=P_2+P_4$，$C_{22}=P_1-P_2+P_3+P_6$。

例如展開 $C_{11}$，其餘交叉項相消，留下 $A_{11}B_{11}+A_{12}B_{21}$；其餘三塊同樣展開得到一般分塊乘法。每層七次半尺寸乘法與 $\Theta(n^2)$ 加減，遞迴式 $T(n)=7T(n/2)+\Theta(n^2)=\Theta(n^{\log_2 7})$，可補零至二次冪。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen：2×2 數值追蹤

取 $A=\begin{bmatrix}1&2\\3&4\end{bmatrix}$、$B=\begin{bmatrix}5&6\\7&8\end{bmatrix}$。七個純量乘積依序是 $P_1=65$、$P_2=35$、$P_3=-2$、$P_4=8$、$P_5=24$、$P_6=22$、$P_7=-30$。

重組得 $C_{11}=65+8-24-30=19$、$C_{12}=-2+24=22$、$C_{21}=35+8=43$、$C_{22}=65-35-2+22=50$，與直接相乘一致。這是算術次數分析；有限精度與較小規模的常數成本仍須另評估。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 消去先決定乘數再更新尾塊

在第 $k$ 步，以 $u_{kk}\neq 0$ 為 pivot，令 $l_{ik}=a_{ik}/u_{kk}$，執行 $R_i\leftarrow R_i-l_{ik}R_k$。消去後的上三角係數形成 $U$，乘數形成 $L$。分塊尾部更新是 Schur complement $A_{22}-a_{21}a_{12}/a_{11}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 可逆矩陣也可能有零 pivot

$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ 的行列式為 $-1$，但不換列的第一步會除以 0。交換兩列，$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$，可得 $PA = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = IU$。零 pivot 只證明目前選擇失敗，不能直接宣告原矩陣奇異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第一欄換列與消去的完整數值

沿用 CLRS28.1 的 $A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$。部分 pivoting 選欄內絕對值最大者，先 $R_1 \leftrightarrow R_3$ 得 $[5, 6, 3; 3, 4, 4; 1, 2, 0]$。乘數 $l_{21} = 3/5, l_{31} = 1/5$。更新 $R_2 \leftarrow R_2 - \frac{3}{5}R_1$、$R_3 \leftarrow R_3 - \frac{1}{5}R_1$ 後為 $[5, 6, 3; 0, 2/5, 11/5; 0, 4/5, -3/5]$。

第 1 輪：比較、換列、消去

A

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$$

P₁A

$$\begin{bmatrix} 5 & 6 & 3 \\ 3 & 4 & 4 \\ 1 & 2 & 0 \end{bmatrix}$$

第 1 輪後工作矩陣

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

|1| < |3| < |5| → R₁ ↔ R₃

l₂₁ = 3/5, l₃₁ = 1/5; R₂ ← R₂ − (3/5)R₁; R₃ ← R₃ − (1/5)R₁

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第二次換列也要交換已存乘數

第二欄 $4/5 > 2/5$，交換工作矩陣 $R_2, R_3$。同時交換 $L$ 已存的第一欄乘數，變成 $l_{21} = 1/5, l_{31} = 3/5$。此時工作矩陣為 $[5, 6, 3; 0, 4/5, -3/5; 0, 2/5, 11/5]$。用 $l_{32} = 1/2$ 消去後最後一格是 $11/5 - (1/2)(-3/5) = 5/2$。

第 2 輪：工作矩陣、P 與 L 同步換列

U₁

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

P₁

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

L：已完成第 1 欄

$$\begin{bmatrix} 1 & 0 & 0 \\ 3/5 & 1 & 0 \\ 1/5 & 0 & 1 \end{bmatrix}$$

|4/5| > |2/5| → R₂ ↔ R₃

圖示為交換前；L 只交換已完成欄，未動對角線。



## 以 PA=LU 檢查分解慣例

$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，$L = \begin{bmatrix} 1 & 0 & 0 \\ 1/5 & 1 & 0 \\ 3/5 & 1/2 & 1 \end{bmatrix}$，$U = \begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$。兩邊相乘都等於 $[5, 6, 3; 1, 2, 0; 3, 4, 4]$。$P$ 作用於方程列，解未知量的順序不變。

消去後逐項核驗 PA = LU

U after R₂ ↔ R₃

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 2/5 & 11/5 \end{bmatrix}$$

U

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$$

PA = LU

$$\begin{bmatrix} 5 & 6 & 3 \\ 1 & 2 & 0 \\ 3 & 4 & 4 \end{bmatrix}$$

l₃₂ = (2/5)/(4/5) = 1/2; R₃ ← R₃ − (1/2)R₂

LU rows: (5, 6, 3); (1, 6/5+4/5, 3/5−3/5);

(3, 18/5+2/5, 9/5−3/10+5/2) = (3, 4, 4)

## 為何非奇異矩陣的 LUP 能持續選主元？

先看一個可逆的當前尾塊 $K$。其第一欄必有非零元素，否則 $K$ 有全零欄而奇異。交換尾塊中的列，把非零元素移到 pivot 位置 $a$；列交換不改變可逆性。

令換列後的尾塊為 $K' = PK$。分塊消去後 $\det K' = a \det S$，而 $\det K' = \pm \det K$。因 $\det K \neq 0$ 且 $a \neq 0$，新的 Schur complement $S$ 仍可逆，可重複相同步驟直至最後。後續換列時也須交換 $L$ 已儲存的對應乘數，最後得到 $PA = LU$。實數運算中選最大絕對值的 partial pivot 有助控制局部乘數，但不保證所有問題都數值穩定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 原地 LUP 的輸入、輸出與記憶體契約

輸入為實數 $n \times n$ 矩陣 $A$。工作陣列 $W$ 初始複製 $A$，排列向量 $p = [1, \dots, n]$；最後 $W$ 的對角線及上三角存 $U$，嚴格下三角存 $L$ 的乘數，$L$ 的對角線隱含為 $1$。排列向量滿足 $(Pb)_i = b_{p_i}$，輸出關係是 $PA = LU$。

下列演算法採精確算術及一基索引；對 $n = 0$ 可約定輸出空分解。所謂「原地」指覆寫工作矩陣；若仍需保留原始 $A$，複製需另用 $\Theta(n^2)$ 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 原地 LUP：選主元、換整列、更新尾塊

對 $k = 1, \dots, n$，先在 $i = k, \dots, n$ 選使 $|W_{ik}|$ 最大的列 $q$。若 $W_{qk} = 0$，回報矩陣奇異並停止；否則交換 $W$ 的第 $k$ 與 $q$ 整列，也交換 $p_k, p_q$。

對每個 $i = k+1, \dots, n$，先存 $W_{ik} \leftarrow W_{ik} / W_{kk}$；再對 $j = k+1, \dots, n$ 更新 $W_{ij} \leftarrow W_{ij} - W_{ik} W_{kj}$。整列交換包含 $j < k$ 的既有乘數；只交換未處理的尾塊會破壞 $PA = LU$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 已完成區塊不變量與更新證明

完成前 $k$ 步後，將目前排列的原矩陣寫為

$PA = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & S \end{bmatrix}$，其中左上兩個三角塊皆為 $k \times k$，$S$ 是尚未消去的 Schur complement。已處理欄中的 $L_{11}, L_{21}$ 和已完成列中的 $U_{11}, U_{12}$ 都由 $W$ 保存。

下一步換列時，同步重排 $L_{21}$ 已儲存的舊乘數；把新 pivot 之下各值除以 pivot 得新乘數，再以外積更新 $S$。這正是矩陣相乘的尾塊公式，因而維持等式。初始 $k = 0$ 時 $S = A$；$k = n$ 時 $S$ 為空，得到 $PA = LU$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 零主元回報的意義與限制

精確算術下，若當前欄的剩餘元素全為 0，尾塊含零欄，故尾塊奇異；先前 pivot 均非零且排列、消去步驟可逆，所以原 $A$ 奇異。反之，若 $A$ 可逆，每步尾塊可逆，故總有非零候選，演算法完成。

在浮點算術中，用閾值把小 pivot 視為零，只表示按所選尺度與容忍值拒絕分解，不能直接證明數學上的矩陣奇異；結果還受捨入、縮放與條件數影響。若需數值秩，應使用合適的 rank-revealing 方法並說明閾值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 原地 LUP 追蹤：列交換與一個尾塊

取 $A=[1,2;3,4]$、$b=(5,11)^T$。第一欄最大絕對值為 3，交換兩列並設 $p=(2,1)$，工作矩陣為 $[3,4;1,2]$。

計算 $l_{21}=1/3$，更新右下元素 $2-(1/3)4=2/3$，所以 $W=[3,4;1/3,2/3]$，它同時存 $L=[1,0;1/3,1]$、$U=[3,4;0,2/3]$。檢查 $LU=[3,4;1,2]=PA$；$Pb=(11,5)^T$。

列交換要包含先前儲存的乘數；本例第一步尚無舊乘數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前代依序得到中間右端

取 $b=(3,7,8)^T$，先排列成 $Pb=(8,3,7)^T$。解 $Ly=Pb$。$y_1=8$，$y_2=3-(1/5)8=7/5$，$y_3=7-(3/5)8-(1/2)(7/5)=3/2$。一般式 $y_i=(Pb)_i-\sum_{j<i}l_{ij}y_j$。每個右端前代為 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 回代從最後一列向上求解

解 $Ux=y$。$x_3=(3/2)/(5/2)=3/5$。$x_2=[7/5+(3/5)(3/5)]/(4/5)=11/5$。$x_1=[8-6(11/5)-3(3/5)]/5=-7/5$。代回原矩陣得 $Ax=(3,7,8)^T$。一般式 $x_i=(y_i-\sum_{j>i}u_{ij}x_j)/u_{ii}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分解一次再處理多個右端

一般稠密消去的尾塊更新總量是 $\sum_{k=1}^{n-1}(n-k)^2=\Theta(n^3)$。已有分解時，每個新右端只需兩次三角解 $\Theta(n^2)$。$q$ 個右端總成本 $\Theta(n^3+qn^2)$。部分 pivoting 仍可能有元素成長，不是任意問題的精度保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 行列式與逆矩陣由分解導出

若共換列 $s$ 次，$\det A=(-1)^s\prod_i u_{ii}$。本例兩次換列，故 $\det A=5(4/5)(5/2)=10$。求逆解 $Az_j=e_j$，各 $z_j$ 是逆矩陣第 $j$ 欄。分解後 $n$ 次解共 $\Theta(n^3)$；單次 $Ax=b$ 直接解即可，先求逆多做了工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前代與回代的迴圈不變量

對單位下三角 $L$ 求 $Ly = Pb$：第 $i$ 步令 $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$。進入第 $i$ 步前，$1, \dots, i-1$ 列已成立且這些 $y_j$ 不再改變；新式使第 $i$ 列成立，故歸納後 $Ly = Pb$。

對非奇異上三角 $U$ 求 $Ux = y$：由 $i = n$ 往 1 計算 $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$。進入第 $i$ 步時 $i+1, \dots, n$ 列成立；上三角使這些列不依賴 $x_i$，本步又使第 $i$ 列成立。非零對角主元保證除法成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一 LUP 例子的兩次三角解

前代：$y_1 = 11$，$y_2 = 5 - (1/3)11 = 4/3$。回代：$x_2 = (4/3)/(2/3) = 2$，$x_1 = (11 - 4 \cdot 2)/3 = 1$。最後 $A(1, 2)^T = (5, 11)^T = b$。

第 2 列回代先成立；計算 $x_1$ 時不改變 $x_2$，因此第 2 列仍成立。這就是回代不變量在數值例子中的具體作用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

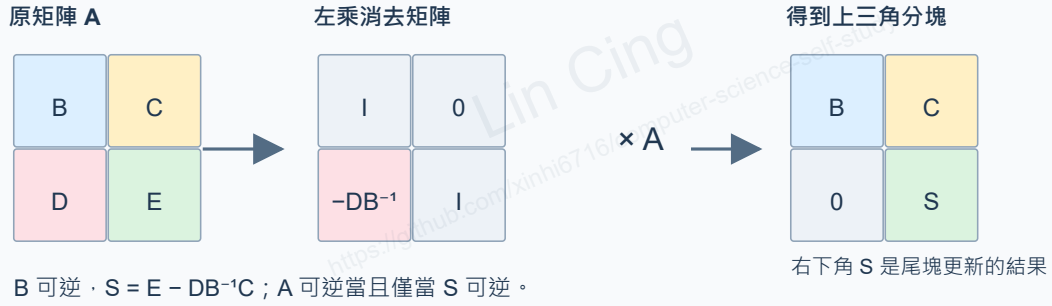
## 從消去看分塊反矩陣

令 $A=\begin{bmatrix}B & C\\ D & E\end{bmatrix}$，$B$ 為可逆方陣，其餘區塊的維度使乘法有意義。定義 Schur complement（舒爾補）$S=E-DB^{-1}C$。左乘分塊消去矩陣，可得

$\begin{bmatrix}I & 0\\ -DB^{-1} & I\end{bmatrix}A=\begin{bmatrix}B & C\\ 0 & S\end{bmatrix}$。

先消去左下區塊，再處理 $S$；區塊順序不可隨意交換。

**Schur complement：左下消去，右下跟著更新**




B 可逆，S = E − DB⁻¹C；A 可逆當且僅當 S 可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 分塊反矩陣：公式與存在條件

若 $B$ 與 $S$ 都可逆，將分塊消去的三個因子逆序相乘，得到

$A^{-1}=\begin{bmatrix}B^{-1}+B^{-1}CS^{-1}DB^{-1} & -B^{-1}CS^{-1}\\ -S^{-1}DB^{-1} & S^{-1}\end{bmatrix}$。

實際做題時可直接檢查 $AA^{-1}=I$。在 $B$ 可逆的前提下，$A$ 可逆當且僅當 $S$ 可逆；若 $B$ 不可逆，這個公式不能使用，但不代表 $A$ 不可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一矩陣分塊求逆：先算 Schur complement

取 $A=\begin{bmatrix}4&2&2\\2&5&1\\2&1&3\end{bmatrix}$，切在前兩列與欄：$B=\begin{bmatrix}4&2\\2&5\end{bmatrix}$，$C=(2,1)^T$，$D=C^T$，$E=3$。

$B^{-1}=\frac{1}{16}\begin{bmatrix}5&-2\\-2&4\end{bmatrix}$，$B^{-1}C=(1/2,0)^T$，$S=3-(2,1)(1/2,0)^T=2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一矩陣分塊求逆：逐塊組裝

$S^{-1}=1/2$，右上塊 $-B^{-1}CS^{-1}=(-1/4,0)^T$，左下塊為其轉置。

左上塊 $B^{-1}+(1/2,0)^T(1/2)(1/2,0)=\begin{bmatrix}7/16&-1/8\\-1/8&1/4\end{bmatrix}$。

因此 $A^{-1}=\begin{bmatrix}7/16&-1/8&-1/4\\-1/8&1/4&0\\-1/4&0&1/2\end{bmatrix}$。例如第一列乘第一欄為 $4(7/16)+2(-1/8)+2(-1/4)=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 正定性保證遞迴不會遇到零主元

實對稱矩陣 $A$ 為正定（SPD），指每個非零 $z$ 都有 $z^TAz>0$。分塊 $A=\begin{bmatrix}B & C\\ C^T & D\end{bmatrix}$ 時，$B$ 也是 SPD：對非零 $x$，取 $z=(x,0)$，得 $x^TBx>0$。

$S=D-C^TB^{-1}C$ 同樣 SPD：對非零 $y$，取 $z=(-B^{-1}Cy,y)$，則 $y^TSy=z^TAz>0$。因此 $B,S$ 均可逆，能遞迴使用分塊公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky 分解：平方根與尾塊

實對稱正定矩陣有唯一分解 $A=LL^T$，其中 $L$ 是對角線為正的下三角矩陣。把第一列欄寫成 $A=\begin{bmatrix}a & r^T\\ r & D\end{bmatrix}$：因 $a>0$，令 $\ell_{11}=\sqrt{a}$、$v=r/\sqrt{a}$，剩餘矩陣 $S=D-vv^T$ 是 SPD。

對 $S$ 遞迴分解 $S=L_2L_2^T$，便有 $L=\begin{bmatrix}\sqrt{a} & 0\\ v & L_2\end{bmatrix}$。每一步的平方根都為正；以相同對角線符號由第一欄遞迴決定，可證唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Cholesky 的逐元素遞迴公式

設實對稱正定矩陣 $A = LL^T$，$L$ 為正對角下三角矩陣。由 $a_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} l_{jk}$，按欄 $j = 1, \dots, n$ 依序求：

$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$；對 $i > j$，$l_{ij} = \dfrac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}}{l_{jj}}$。

空和取 $0$。先求對角線，再求同欄下方元素；公式中的舊欄 $k < j$ 都已計算完成，上三角元素固定為 $0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼平方根與除法都合法？

第 $j$ 步的根號內是前 $j-1$ 欄消去後剩餘 Schur complement 的第一個對角元素。SPD 的主區塊和 Schur complement 都是 SPD，故此對角元素嚴格為正；因此 $l_{jj} > 0$，欄內的除法有定義。

逐欄代回 $a_{ij} = \sum_{k \le \min(i,j)} l_{ik} l_{jk}$，可見每一步構造的元素都使對應矩陣條目相等，最後 $A = LL^T$。相同的正對角條件迫使每一步的根號及同欄元素唯一，因此分解唯一。若輸入並非實對稱正定，不能保證根號為正或除數非零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用遞迴式逐項算同一個 3×3 例子

對 $A=[4,2,2;2,5,1;2,1,3]$，第一欄：$l_{11}=\sqrt{4}=2$，$l_{21}=2/2=1$，$l_{31}=2/2=1$。

第二欄：$l_{22}=\sqrt{5-l_{21}^2}=2$，$l_{32}=(1-l_{31}l_{21})/l_{22}=0$。第三欄：$l_{33}=\sqrt{3-l_{31}^2-l_{32}^2}=\sqrt{2}$。

所以 $L=[2,0,0;1,2,0;1,0,\sqrt{2}]$；核對 $l_{31}l_{21}+l_{32}l_{22}=1=a_{32}$，並以 $LL^T=A$ 驗算所有元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky 算例：每次更新都看得見

沿用 $A=\begin{bmatrix}4&2&2\\2&5&1\\2&1&3\end{bmatrix}$。第一步 $\ell_{11}=2$，其下兩數都是 $1$，尾塊更新為 $\begin{bmatrix}5&1\\1&3\end{bmatrix}-\begin{bmatrix}1&1\\1&1\end{bmatrix}=\begin{bmatrix}4&0\\0&2\end{bmatrix}$。

因此 $L=\begin{bmatrix}2&0&0\\1&2&0\\1&0&\sqrt{2}\end{bmatrix}$。相乘 $LL^T$ 回到 $A$；對任意非零 $x$，$x^TAx=\|L^Tx\|^2>0$，也驗證本例正定。

## 已知 Cholesky，怎麼解方程？

$LL^Tx = b$ 分成 $Ly = b$ 的前代與 $L^Tx = y$ 的回代。取 $b = (8, 8, 6)^T$：前代得 $y = (4, 2, \sqrt{2})^T$；回代依序得 $x_3 = 1$、$x_2 = 1$、$x_1 = 1$。代回可確認 $A(1, 1, 1)^T = b$。

一般稠密矩陣的 Cholesky 分解需 $\Theta(n^3)$ 算術運算；每個右端的兩次三角解為 $\Theta(n^2)$。SPD 是必要使用前提，浮點輸入近似正定時仍需處理捨入與失敗。

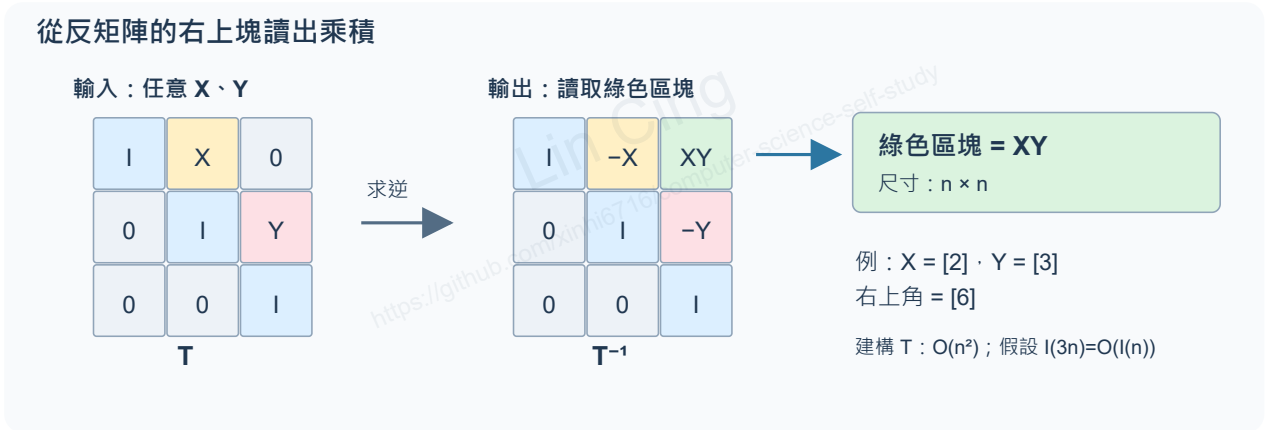
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 反矩陣可以算出矩陣乘積

欲計算兩個 $n \times n$ 矩陣 $X, Y$ 的乘積，建立上三角分塊矩陣 $T = \begin{bmatrix} I & X & 0 \\ 0 & I & Y \\ 0 & 0 & I \end{bmatrix}$。

$T^{-1} = \begin{bmatrix} I & -X & XY \\ 0 & I & -Y \\ 0 & 0 & I \end{bmatrix}$：直接做分塊乘法，右上角 $XY - XY = 0$，其餘非對角塊也抵消。讀出右上塊便得到 $XY$；$T$ 對任意 $X, Y$ 均可逆。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從反矩陣讀出乘積：成本條件

若 $I(n)$ 是 $n \times n$ 可逆矩陣的反矩陣演算法上界，且 $I(n) = \Omega(n^2)$、$I(3n) = O(I(n))$，建立 $3n \times 3n$ 的 $T$ 並讀取右上塊只需 $O(n^2)$，故矩陣乘法成本 $M(n) = O(I(n))$。

這是算術操作模型的歸約。倍增規律不可省略；若只知某個規模的反矩陣成本，不能直接把 $I(3n)$ 換成 $I(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從快速乘法到快速反矩陣

先討論 SPD 矩陣。分塊公式需要遞迴求 $B^{-1}$ 和 $S^{-1}$，以及常數次半尺寸乘法與 $O(n^2)$ 次加減，故 $I_{SPD}(n) \le 2I_{SPD}(n/2) + O(M(n) + n^2)$。

若 $M(n) = \Omega(n^2)$、$M(n/2) \le cM(n)$ 對某固定 $c < 1/2$ 成立，則各層總成本以比率至多 $2c < 1$ 遞減，總計 $I_{SPD}(n) = O(M(n))$。這不是所有任意形狀函數 $M$ 無條件成立的結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一般可逆矩陣與算術模型限制

對實數可逆 $A$，$A^TA$ 是 SPD：$x^TA^TAx = \|Ax\|^2 > 0$。先形成並求逆 $A^TA$，再乘 $A^T$，即 $A^{-1} = (A^TA)^{-1}A^T$。若補零至二次冪，使用 $\mathrm{diag}(A, I)$；還需假設 $M(n+k) = O(M(n))$（$0 \le k \le n$）。

在上述正規性及精確算術成本模型下，乘法與求逆互相歸約。這不是浮點穩定性的保證：$A^TA$ 的條件數會平方，實務求解通常採用有主元消去或適當分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LDLᵀ 與 Cholesky 的關係

對 SPD 矩陣，也可把消去的主元留在對角矩陣：$A = LDL^T$，其中 $L$ 為對角線全 1 的下三角矩陣，$D$ 為正對角矩陣。每一步的主元是目前 SPD Schur complement 的左上角，故必為正。

若 $D = \mathrm{diag}(d_1, \dots, d_n)$，設 $D^{1/2} = \mathrm{diag}(\sqrt{d_i})$，則 $A = (LD^{1/2})(LD^{1/2})^T$，這就是正對角 Cholesky 因子。前例 $L = [1,0,0;\, 1/2,1,0;\, 1/2,0,1]$、$D = \mathrm{diag}(4,4,2)$；乘 $D^{1/2}$ 得 $[2,0,0;\, 1,2,0;\, 1,0,\sqrt{2}]$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 稠密分解的主導運算與換列成本

假設稠密 $n \times n$、標量加減乘除各算 1 次 flop，且無提前奇異停止。LUP 的尾塊更新以乘法加減法計約 $2\sum_{s=1}^{n-1} s^2 = \frac{2}{3}n^3 + O(n^2)$ 次 flop；乘數除法僅 $O(n^2)$。一個右端的前代加回代共約 $2n^2 + O(n)$ 次 flop。

Cholesky 的逐元素乘加約 $\frac{1}{3}n^3 + O(n^2)$ 次 flop；除法與平方根分別為 $O(n^2)$ 和 $n$ 次，若平方根成本不同應另列。這兩個領先係數適用所述 flop 慣例，不是執行時間比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 選主元、記憶體及數值成本另計

LUP 的 partial pivot 每欄掃描絕對值並比較，合計 $O(n^2)$ 次比較；至多 $n$ 次整列交換，最多移動 $O(n^2)$ 個矩陣元素及 $O(n)$ 個排列索引。這些比較、記憶體讀寫與平方根通常不放進上述乘加 flop，實際執行時間可能由記憶體或實作決定。

工作矩陣 $W$ 使用 $\Theta(n^2)$ 儲存，額外排列向量 $\Theta(n)$；覆寫與否及保存原矩陣會改變額外空間。稀疏矩陣、低秩捷徑、平行化與提早停止不適用上述稠密完整流程的直接時間結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三個不共線點無法全數插值

資料 $(t,b)=(0,1),(1,2),(2,2)$，擬合 $b\approx\alpha+\beta t$。$A=\begin{bmatrix}1&0\\1&1\\1&2\end{bmatrix}\in\mathbb{R}^{3\times 2}$，$x=(\alpha,\beta)^T$，$b=(1,2,2)^T$。相鄰斜率為 1 與 0，故沒有一直線通過三點。目標固定為 $\min_x\|b-Ax\|_2^2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正規方程把導數設為零

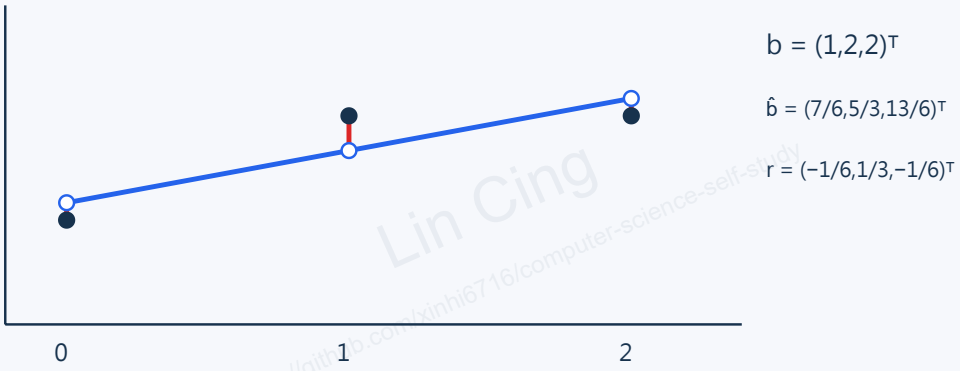
$\nabla\|b-Ax\|^2=-2A^T(b-Ax)=0$，故 $A^TA\hat{x}=A^Tb$。本例 $A^TA=\begin{bmatrix}3&3\\3&5\end{bmatrix}$，$A^Tb=(5,6)^T$。$A$ 滿欄秩，$z^TA^TAz=\|Az\|^2>0$ 對所有非零 $z$ 成立，故解唯一。若欄相依，這個唯一性結論不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 解出係數再逐點算預測與殘差

正規方程為 $3\alpha + 3\beta = 5$、$3\alpha + 5\beta = 6$。相減得 $\beta = 1/2$，再得 $\alpha = 7/6$。預測 $A\hat{x} = (7/6, 5/3, 13/6)^T$，殘差 $r = (-1/6, 1/3, -1/6)^T$。平方和為 $1/36 + 1/9 + 1/36 = 1/6$。

### 垂直殘差，不是最短點線距離



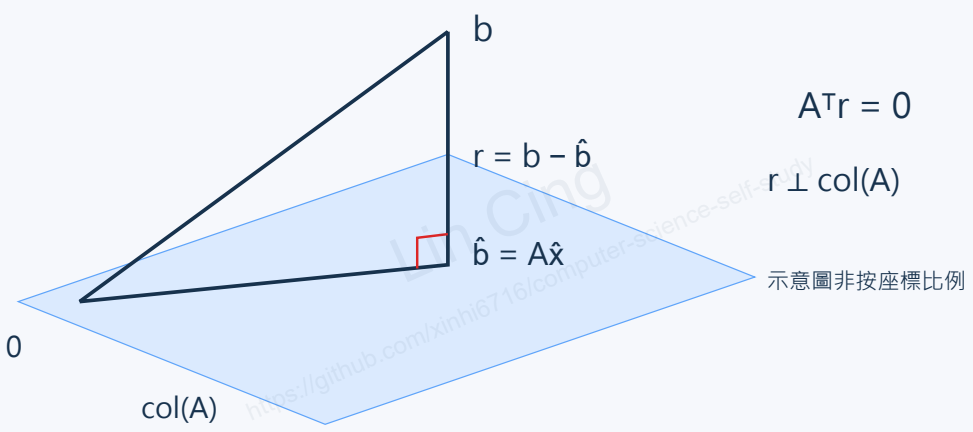

紅線固定 t；藍線 b̂ = 7/6 + t/2



## 垂直殘差與資料空間正交是兩種圖

散點圖在固定 $t_i$ 上畫垂直線段，長度 $|b_i - \hat{b}_i|$。它不是點到直線的最短距離。在 $\mathbb{R}^3$ 資料空間，$\hat{b} = A\hat{x} \in \text{col}(A)$，$r \perp \text{col}(A)$。直接檢查 $A^T r = (-1/6 + 1/3 - 1/6,\ 1/3 - 2/6)^T = (0, 0)^T$。

### 資料空間 R³ 的正交投影示意



## 正規方程會平方條件數

滿欄秩時 $\kappa_2(A)=\sigma_{\max}/\sigma_{\min}$，而 $A^TA$ 特徵值為 $\sigma_i^2$，所以 $\kappa_2(A^TA)=\kappa_2(A)^2$。例如 $A=\mathrm{diag}(1,10^{-4})$ 的條件數為 $10^4$，$A^TA$ 為 $10^8$。浮點形成 $A^TA$ 會使弱方向更難保留；不是每題都固定損失相同位數。

## QR 避免先形成平方條件數的矩陣

薄 QR 為 $A=QR$、$Q^TQ=I$。最小平方改解 $R\hat{x}=Q^Tb$。本例 $q_1=(1,1,1)^T/\sqrt{3}$、$q_2=(-1,0,1)^T/\sqrt{2}$，$R=[\sqrt{3},\sqrt{3};0,\sqrt{2}]$，$Q^Tb=(5/\sqrt{3},1/\sqrt{2})^T$，仍得 $(7/6,1/2)^T$。實務用 Householder QR；秩疑慮用 pivoted QR 或 SVD。QR 不會消除資料本身病態。

## 一般 rank：正規方程是必要條件

令 $f(x) = \|b - Ax\|_2^2$。若 $\hat{x}$ 是全域最小解，對任意方向 $z$，一元函數 $g(t) = f(\hat{x} + tz)$ 在 $t = 0$ 達到最小值，故 $g'(0) = -2z^T A^T (b - A\hat{x}) = 0$。

此式對所有 $z$ 成立，因此 $A^T(b - A\hat{x}) = 0$，也就是 $A^T A\hat{x} = A^T b$。推導不要求 $A$ 滿欄秩或 $A^T A$ 可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一般 rank：正規方程也是充分條件

反過來，若 $\hat{x}$ 滿足 $A^T(b - A\hat{x}) = 0$，設 $r = b - A\hat{x}$。對任意 $z$ 展開：

$\|b - A(\hat{x} + z)\|^2 = \|r - Az\|^2 = \|r\|^2 - 2z^T A^T r + \|Az\|^2 = \|r\|^2 + \|Az\|^2 \geq \|r\|^2$。

故 $\hat{x}$ 為全域最小解；等號恰在 $Az = 0$，所以全部解恰為 $\hat{x} + \ker(A)$。又 $A\hat{x} \in \mathrm{col}(A)$ 且 $r \perp \mathrm{col}(A)$，故它就是 $b$ 的唯一正交投影。有限維欄空間的投影存在，因而至少有一個 $\hat{x}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 滿欄秩時的投影矩陣

若 $A \in \mathbb{R}^{m \times n}$ 滿欄秩，$A^T A$ 為 SPD，$\hat{x} = (A^T A)^{-1} A^T b$ 是唯一最小平方解。定義 $P = A(A^T A)^{-1} A^T$，則預測 $p = Pb$。

由 $P^T = P$ 與 $P^2 = P$，$P$ 是到 $\mathrm{col}(A)$ 的正交投影；$A^T(b - Pb) = 0$ 直接給出殘差正交。欄不滿秩時不能使用 $(A^T A)^{-1}$，應採上一節的 SVD／廣義逆表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 欄不獨立時：預測唯一，係數可能不唯一

對任意實矩陣 $A$ 與向量 $b$，最小平方問題 $\min_x \|Ax - b\|_2^2$ 一定有解；所有解都滿足 $A^T Ax = A^T b$。原因是 $b$ 在 $\mathrm{col}(A)$ 的正交投影 $p$ 存在且唯一；任何滿足 $Ax = p$ 的 $x$ 都是解。

若 $A$ 的欄相依，$\ker(A)$ 含非零向量。只要 $x_0$ 是解，全部解恰為 $x_0 + \ker(A)$；因此 $Ax = p$ 仍唯一，但係數不唯一。$A = 0$ 時所有 $x$ 均達到相同殘差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 欄不獨立的具體算例

取 $A=\begin{bmatrix}1&1\\2&2\\0&0\end{bmatrix}$、$b=(2,3,1)^T$，令 $u=(1,2,0)^T$。$p=u(u^Tb)/(u^Tu)=(8/5,16/5,0)^T$，$r=b-p=(2/5,-1/5,1)^T$，可驗 $A^Tr=0$。

$Ax=p$ 等價於 $x_1+x_2=8/5$，故所有解為 $(4/5,4/5)^T+t(1,-1)^T$。這些係數給出相同預測；其中 $t=0$ 的範數最小，為 $(4/5,4/5)^T$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SVD 選出唯一的最小範數解

設 $A=U\Sigma V^T$ 的非零奇異值為 $\sigma_1,\dots,\sigma_r$，定義 $A^+=\sum_{i=1}^{r}\sigma_i^{-1}v_iu_i^T$。$x_*=A^+b$ 是唯一的最小範數最小平方解；全部最小平方解為 $x_*+z$，其中 $z\in\ker(A)$。

由 $x_*\in\ker(A)^\perp$，$\|x_*+z\|^2=\|x_*\|^2+\|z\|^2$，所以 $z=0$ 唯一最小。秩為零時空和為零，$A^+=0$；數值實作須依資料尺度選擇秩截斷閾值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何要計算多項式的點值？

長度 $m$ 與 $n$ 的係數向量直接相乘需 $O(mn)$ 次算術運算。先在共同點求值，逐點相乘，再內插回係數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 點值能否唯一還原係數？

取域 $F$ 中 $n$ 個點 $x_0, \dots, x_{n-1}$，係數向量 $a$ 與點值 $y$ 滿足 $y = Va$，其中 $V_{ij} = x_i^j$（$0 \le i, j < n$）。這個線性系統何時有唯一解？關鍵是以下行列式，而不是僅憑「方陣」判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Vandermonde 行列式

$\det V = \prod_{0 \le i < j < n} (x_j - x_i)$。因此在域上，點兩兩相異當且僅當 $V$ 可逆；若兩個點相同，對應兩列相同。

例：點 $1, 2, 4$ 的行列式為 $(2-1)(4-1)(4-2) = 6$。在實數域它非零；在 $\mathbb{F}_3$ 中 $1$ 與 $4$ 是同一個點，行列式為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 證明：把最後一個點看成變數

對 $n$ 歸納。$n = 1$ 時行列式和空乘積都是 $1$。先把 $x_0, \dots, x_{n-2}$ 視為互異的不定元，令 $D(t)$ 是最後一列為 $[1, t, \dots, t^{n-1}]$ 的行列式。

$D(t)$ 是次數至多 $n-1$ 的多項式。當 $t = x_i$ 時有兩列相同，故 $D(x_i) = 0$；在多項式環 $F[x_0, \dots, x_{n-2}, t]$ 中，這些兩兩不同的 monic 一次因子皆為不可約元素，因此它們的乘積整除 $D(t)$，得到 $D(t) = c \prod_{i=0}^{n-2} (t - x_i)$。最後可將不定元代入任意域元素，即使代入後有重複點也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 證明：決定最高次係數

沿最後一列展開，唯一含 $t^{n-1}$ 的項是右下角項，其餘子式是前 $n-1$ 個點的 Vandermonde 行列式，符號為正。

因此 $c = \det V_{n-1} = \prod_{0 \le i < j < n-1} (x_j - x_i)$。代回 $t = x_{n-1}$ 即得所需乘積，歸納完成。此證明也涵蓋重複點，無須先假設各點不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Lagrange 插值：不只知道解存在

若 $x_0, \dots, x_{n-1}$ 兩兩相異，定義 $L_i(z) = \prod_{j \ne i} \frac{z - x_j}{x_i - x_j}$。每個分母在域中非零，因此可逆；$L_i(x_k) = 1$ 當 $i = k$，否則為 $0$。

給定任意點值 $y_i$，$P(z) = \sum_i y_i L_i(z)$ 的次數小於 $n$ 且 $P(x_i) = y_i$，這直接構造解。若另一個次數小於 $n$ 的 $Q$ 有相同點值，則 $P - Q$ 有 $n$ 個不同根，只能是零多項式；故插值唯一。這也與 Vandermonde 可逆結論一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 如何在 $O(n^2)$ 求插值多項式的係數？

先逐因子相乘求 $G(z)=\prod_{j=0}^{n-1}(z-x_j)$，第 $k$ 次更新長度 $k+1$ 的係數列，總共 $O(n^2)$ 次域運算。

對每個 $i$，以 synthetic division（綜合除法）用 $O(n)$ 算出 $Q_i(z)=G(z)/(z-x_i)$；以 Horner 法在 $x_i$ 評估 $d_i=Q_i(x_i)=\prod_{j\neq i}(x_i-x_j)\neq 0$。再將 $y_i d_i^{-1}Q_i$ 的 $n$ 個係數加到答案。$n$ 次各花 $O(n)$，所以總共 $O(n^2)$ 次域運算；假設域的四則運算是單位成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼少於 $n$ 個點不能唯一決定？

給定 $m<n$ 個互異點 $x_0,\dots,x_{m-1}$，零多項式 $P(z)=0$ 和 $Q(z)=\prod_{i=0}^{m-1}(z-x_i)$ 都在這些點取值零，卻是不同多項式，且次數都小於 $n$。

例如要決定二次以內的多項式，只知道 $P(0)=P(1)=0$，則 $P(z)=0$ 和 $P(z)=z(z-1)$ 都符合。再多一個不同點的值，才有機會利用前述 $n$ 點唯一性定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Horner 法：單點求值的不變量

對 $A(z)=\sum_{j=0}^{n-1}a_jz^j$，固定求值點 $x$，令 $b=0$；依 $k=n-1,n-2,\dots,0$ 更新 $b\leftarrow a_k+xb$。完成第 $k$ 次後，迴圈不變量是 $b=\sum_{j=k}^{n-1}a_jx^{j-k}$。初始化時 $b=0$ 對空和成立；每次更新把冪次多乘一個 $x$ 並加入 $a_k$；結束時 $k=0$，得到 $A(x)$。

例如 $A(z)=2+3z+4z^2$、$x=2$：$b$ 依序為 $4$、$3+2\cdot4=11$、$2+2\cdot11=24$。此實作對 $n$ 個係數使用 $\Theta(n)$ 次域運算；對 $n$ 個點逐一重做 Horner 則為 $\Theta(n^2)$，是此方法的成本，不是所有多點求值方法的下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 固定變換慣例

令 $n=2^s$、$\omega_n=e^{2\pi i/n}$，定義 $X_k=\sum_{j=0}^{n-1}x_j\omega_n^{jk}$。此處正向 DFT 使用正指數；索引皆為 $0$ 至 $n-1$。

$\omega_4 = i$

k: 0 1 2 3

$\omega_4^k$: 1 i −1 −i

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 單位根與週期

偶數 $n$ 下，$\omega_n^n = 1$、$\omega_n^{k+n} = \omega_n^k$，且 $\omega_n^{k+n/2} = -\omega_n^k$。$n = 4$ 時依序為 $1, i, -1, -i$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 拆成偶數與奇數係數

令 $A(z) = E(z^2) + zO(z^2)$，其中 $E(t) = \sum_r x_{2r} t^r$，$O(t) = \sum_r x_{2r+1} t^r$。兩個半長 DFT 共用平方後的單位根。

A(z) = E(z²) + z O(z²)

E: x₀, x₂, x₄, …

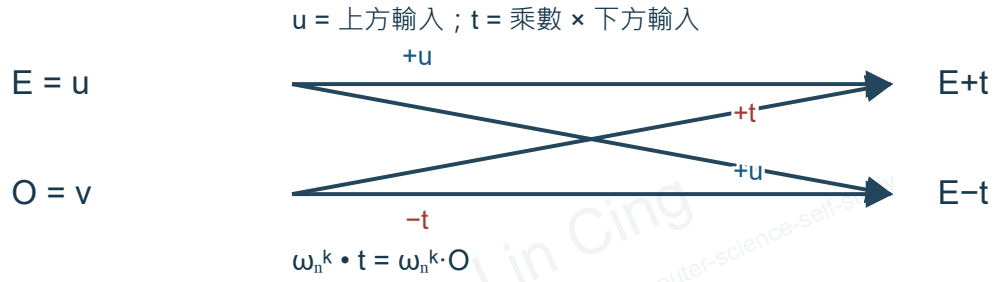
O: x₁, x₃, x₅, …

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 合併一對輸出

對 $0 \le k < n/2$，令 $t = \omega_n^k O_k$，則 $X_k = E_k + t$、$X_{k+n/2} = E_k - t$。這一對運算稱蝶形，乘數稱旋轉因子。

### 一個蝶形




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為何成本是 $O(n \log n)$

$T(n) = 2T(n/2) + O(n)$ 有 $\log_2 n$ 層，每層 $O(n)$。原地迭代版每階段有 $n/2$ 個蝶形，共 $\log_2 n$ 階。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 輸入索引做位元反轉

$n = 4$ 時，兩位元索引 $00, 01, 10, 11$ 反轉為 $00, 10, 01, 11$。因此 $[1, 2, 0, -1]$ 重排為 $[1, 0, 2, -1]$，供時間抽取的迭代階段使用。

索引： 00 01 10 11

反轉： 00 10 01 11

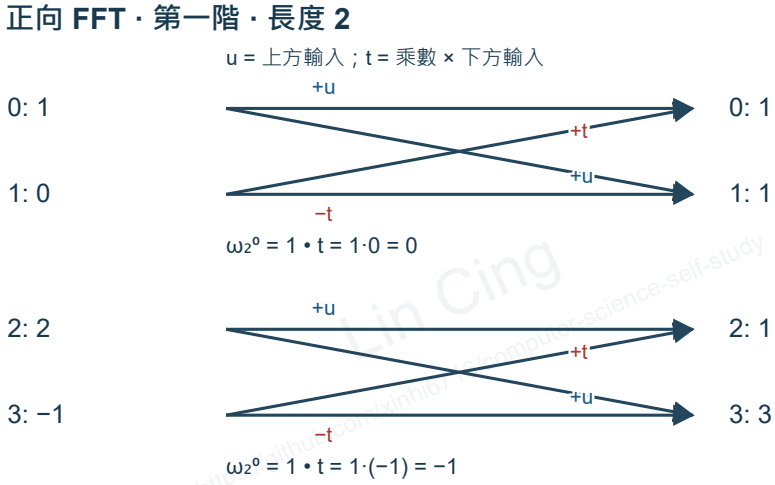
數值： 1 0 2 −1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第一階，長度 2

輸入 $[1, 0, 2, -1]$，兩個乘數都是 $\omega_2^0 = 1$。索引 $(0, 1)$ 得 $(1 + 0, 1 - 0) = (1, 1)$；$(2, 3)$ 得 $(2 - 1, 2 - (-1)) = (1, 3)$。狀態為 $[1, 1, 1, 3]$。

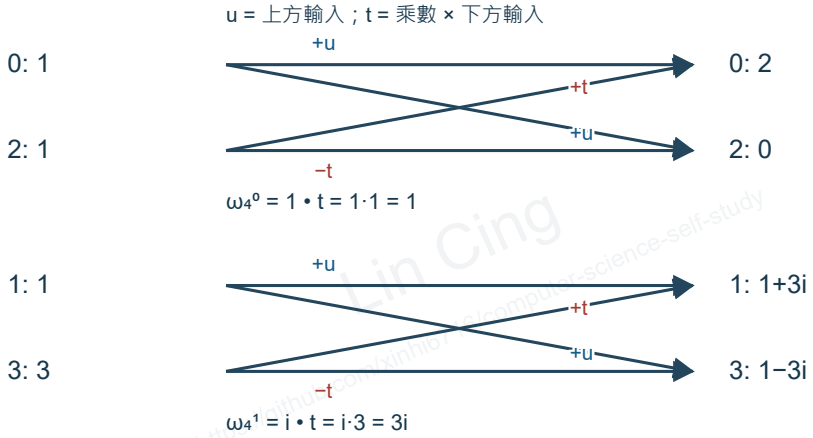



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第二階，長度 4

輸入 $[1,1,1,3]$。$(0,2)$ 用 $\omega_4^0=1$，得 $(2,0)$；$(1,3)$ 用 $\omega_4^1=i$，得 $(1+3i,1-3i)$。按索引排列 $X=[2,1+3i,0,1-3i]$。

正向 FFT · 第二階 · 長度 4






## 用定義直接核對

$X_0=1+2+0-1=2$；$X_1=1+2i+(-1)(-i)=1+3i$；$X_2=1-2+0+1=0$；$X_3=1-2i+(-1)i=1-3i$。此計算獨立核對索引與符號。

## 迭代蝶形：每一階完成了什麼？

令 $N = 2^{\ell}$，第 $s$ 階（$0 \le s \le \ell$）完成後，每個連續長度 $2^s$ 的區塊都保存一個長度 $2^s$ 的 DFT，依自然頻率順序排列。對第 $b$ 個區塊，輸入原序列為 $x_{r+tN/2^s}$，其中 $0 \le t < 2^s$，$r$ 是把 $b$ 的 $\ell - s$ 位元反轉得到的值。

框內列出 DFT 所處理的原序列索引；輸出在各框內依自然頻率排列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 不變量如何由蝶形維持？

$s = 0$ 時，每個單元素區塊就是原始資料的位元反轉排列，不變量成立。設某階已成立：下一階每個區塊的前、後半恰好分別是同一子序列偶數及奇數位置的 DFT。對每個 $k$，用 $E_k \pm \omega_{2^{s+1}}^{k} O_k$ 組成位置 $k$ 與 $k + 2^s$ 的點值，故新區塊成為完整 DFT。

兩個半區塊的原索引間隔均為 $N/2^s$，其起始索引差為 $N/2^{s+1}$，正是新子序列的偶數／奇數位置。歸納至 $s = \ell$，得到整個輸入的自然頻率順序 DFT。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 位元反轉不是免費操作

若 $N = 2^{\ell}$ 且 $i$ 以 $\ell$ 位元表示，$\mathrm{rev}_{\ell}(i)$ 是其反向位元所代表的索引。最直接的方法逐一掃描每個索引的 $\ell$ 位元，總工作量 $O(N \log N)$。若採用反向計數器，每次更新只翻轉受進位影響的位元，所有 $N$ 次更新合計 $O(N)$ 次位元翻轉，可用 $O(N)$ 工作產生全部索引。

原地重排只在 $i < \mathrm{rev}_{\ell}(i)$ 時交換，以免交換兩次。前面平行 FFT 的 $O(1)$ 重排深度假設映射已提供；若映射要自己生成，其成本必須另計。不同字長和指令模型也會影響位元操作成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 迭代演算法

先位元反轉。依序令 $L = 2, 4, \dots, n$；對每個起點為 $b$ 的區塊及 $0 \le j < L/2$，令 $u = a_{b+j}$、$t = \omega_L^{j} a_{b+j+L/2}$，再同時寫入 $(u + t, u - t)$。寫入前先保存 $u, t$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逆變換

$x_j = \frac{1}{n}\sum_{k=0}^{n-1} X_k \omega_n^{-jk}$。以逆單位根執行同一 FFT，再將每項除以 $n$。

位元反轉 → 逆根 FFT → 除以 n

[2, 1+3i, 0, 1−3i] → [1, 2, 0, −1]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



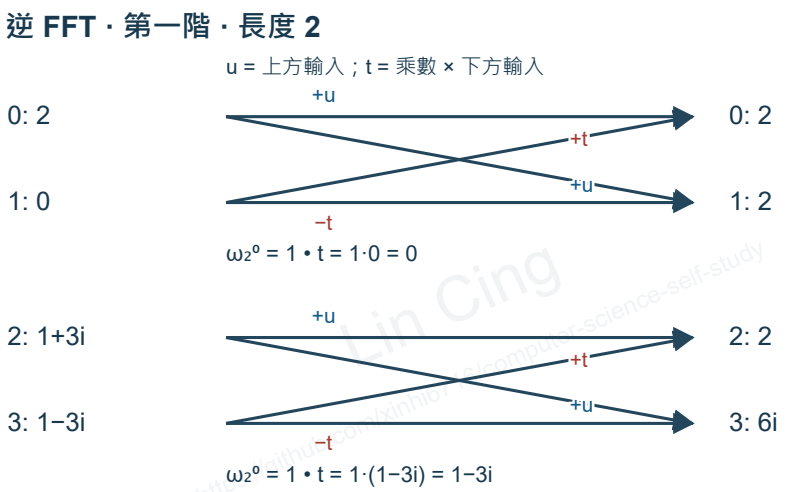
## 為何要除以 $n$

代入 $X_k$ 得 $\frac{1}{n}\sum_r x_r \sum_k \omega_n^{k(r-j)}$。因 $0 \le r, j < n$，內層等比級數在 $r = j$ 時為 $n$，其他時為 $0$，故只剩 $x_j$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逆變換例子，第一階

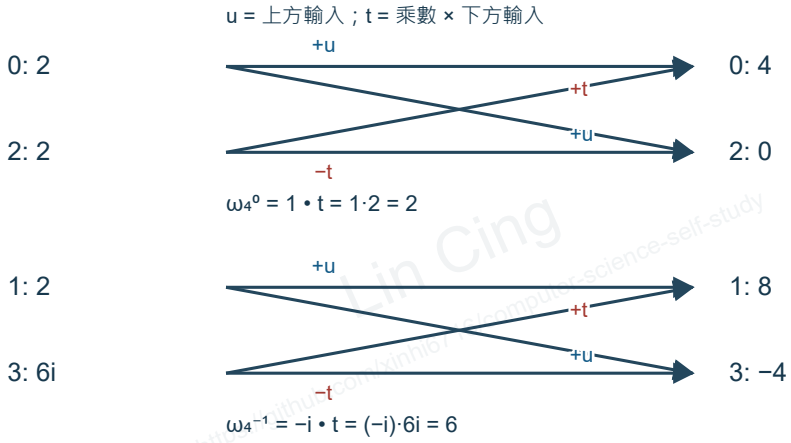
將 $[2, 1+3i, 0, 1-3i]$ 位元反轉為 $[2, 0, 1+3i, 1-3i]$。長度 2 的乘數皆為 1，兩對運算得到 $[2, 2, 2, 6i]$。






## 逆變換例子，第二階

使用 $\omega_4^{-1} = -i$。$(0, 2)$ 的乘數為 1，結果 $(4, 0)$；$(1, 3)$ 的乘積為 $(-i)(6i) = 6$，結果 $(8, -4)$。將 $[4, 8, 0, -4]$ 除以 4 得 $[1, 2, 0, -1]$。



## 遞迴 FFT 的空間：看同時活著的資料

順序深度優先實作若每層配置長度與該子問題成正比的暫存，並在完成後釋放，沿單條遞迴路徑同時存活的容量至多 $N + N/2 + N/4 + \cdots < 2N$；加上輸入／輸出和 $O(\log N)$ 呼叫堆疊，峰值額外空間為 $O(N)$。

若有意保留每一階全部節點的陣列供追蹤或重用，每階合計 $N$ 個元素，共 $\log_2 N + 1$ 階，儲存量則為 $\Theta(N \log N)$。兩者的運算量都可為 $\Theta(N \log N)$；空間差異來自保留策略，非單純由遞迴層數決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼正向 DFT 做兩遍不等於逆變換？

在正向慣例 $X_k = \sum_j x_j \omega^{jk}$ 下，再做一次正向變換並除以 $N$，可交換求和得到 $\frac{1}{N}\sum_k X_k \omega^{k\ell} = \frac{1}{N}\sum_j x_j \sum_k \omega^{k(j+\ell)} = x_{(-\ell) \bmod N}$。

內層和僅在 $j + \ell \equiv 0 \pmod N$ 時為 $N$。所以結果是循環索引反轉；逆變換須使用 $\omega^{-1}$ 再除以 $N$，或正向做第二次後再反轉索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一個實際反例

令 $N=4$、$\omega=i$、$x=[1,2,0,0]$。一次正向 DFT 為 $[3,1+2i,-1,1-2i]$；第二次仍採正向根，再除以 $4$，得到 $[1,0,0,2]$，不是原輸入 $[1,2,0,0]$。錯誤不是少除一個 $N$，而是頻率指數方向不對；改用逆根 $-i$ 即可還原。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 平行 FFT 的工作量與深度

以同步、隨機存取的 EREW PRAM 為模型，假設 $N=2^s\ge 2$、位元反轉位址映射已給定，每階段可以配置 $N/2$ 個處理器。每個處理器讀一對獨立輸入、算一個蝶形並寫回該對；不同蝶形不讀寫同一格，階段之間有同步屏障。旋轉因子可預先給定。

位元反轉搬移用 $O(N)$ 工作與 $O(1)$ 時間；每一階用 $\Theta(N)$ 工作、$\Theta(1)$ 時間，共 $\log_2 N$ 階。因此 $W=\Theta(N\log N)$、深度 $D=\Theta(\log N)$，額外儲存 $O(N)$。這不是宣稱任何實機均可達到；若生成索引或旋轉因子也需計費，必須另加其成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 線性與循環卷積

長度 $N$ 的變換逐點相乘得到循環卷積 $c_k = \sum_{j=0}^{N-1} a_j b_{(k-j) \bmod N}$。補零不足時，高次係數會回繞並加到前面。

m + n − 1 ≤ N

[1, 2, 0, 0] × [3, 4, 0, 0]

線性輸出: [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先補零再相乘

對長度 $m, n \geq 1$，選取二的冪 $N \geq m + n - 1$，兩向量補零至 $N$，FFT 後逐點相乘，再做逆 FFT。前 $m + n - 1$ 項即線性卷積。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 卷積例子

$[1,2]*[3,4]=[3,10,8]$。取 $N=4$，補成 $[1,2,0,0]$、$[3,4,0,0]$，逐點乘積的逆變換為 $[3,10,8,0]$。若取 $N=2$，係數 $8$ 會回繞至常數項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 複數 FFT 卷積：先做兩次正轉換

以 $A(z)=1+2z$、$B(z)=3+4z$ 為例；線性卷積長度為 $2+2-1=3$，因此補零至 $N=4$。採正向 $\omega=i$：$A=[1,2,0,0]$、$B=[3,4,0,0]$。

由 $X_k=\sum_j x_j i^{jk}$，依序在 $1,i,-1,-i$ 求值，得到 $\widehat{A}=[3,1+2i,-1,1-2i]$ 和 $\widehat{B}=[7,3+4i,-1,3-4i]$。例如 $A(i)=1+2i$、$B(-1)=3-4=-1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐頻相乘，再算逆變換

逐點乘法得 $C = [21, -5+10i, 1, -5-10i]$；例如 $(1+2i)(3+4i) = -5+10i$。逆變換採 $i^{-1} = -i$，每個輸出是 $c_j = \frac{1}{4}\sum_{k=0}^{3} C_k(-i)^{jk}$。

四個未除以 4 的和依序為 $[12, 40, 32, 0]$，因此 $c = [3, 10, 8, 0]$。這與直接相乘 $(1+2z)(3+4z) = 3+10z+8z^2$ 一致。圖中依同一個頻率相乘，不能交叉配對。

| 頻率 | A 的點值 | B 的點值 | | 逐頻相乘 |
|---|---|---|---|---|
| 1 | 3 | 7 | × | 21 |
| i | 1+2i | 3+4i | × | −5+10i |
| −1 | −1 | −1 | × | 1 |
| −i | 1−2i | 3−4i | × | −5−10i |

逆變換： [21, −5+10i, 1, −5−10i] → [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有限域版 Fourier 變換

選質數 $p$，令 $N = 2^s$ 且 $N \mid (p-1)$。選 $\omega \in \mathbb{F}_p$，使它的乘法階恰為 $N$：$\omega^N = 1$，而 $1 \le k < N$ 時 $\omega^k \neq 1$。有限域非零元素形成階 $p-1$ 的循環群，故此條件足以保證存在。

定義 $X_k = \sum_{j=0}^{N-1} x_j \omega^{jk} \pmod p$。所有乘加都在 $\mathbb{F}_p$ 中計算；複數的 $i$ 在此由合適的 $\omega$ 取代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為什麼逆變換有效？

因 $N \mid p-1$，所以 $p \nmid N$，$N^{-1}$ 存在。對 $0 < r < N$，$\omega^r \neq 1$，且 $1-\omega^r$ 在域中可逆，故幾何和 $\sum_{k=0}^{N-1} \omega^{kr} = 0$；$r = 0$ 時其值為 $N$。

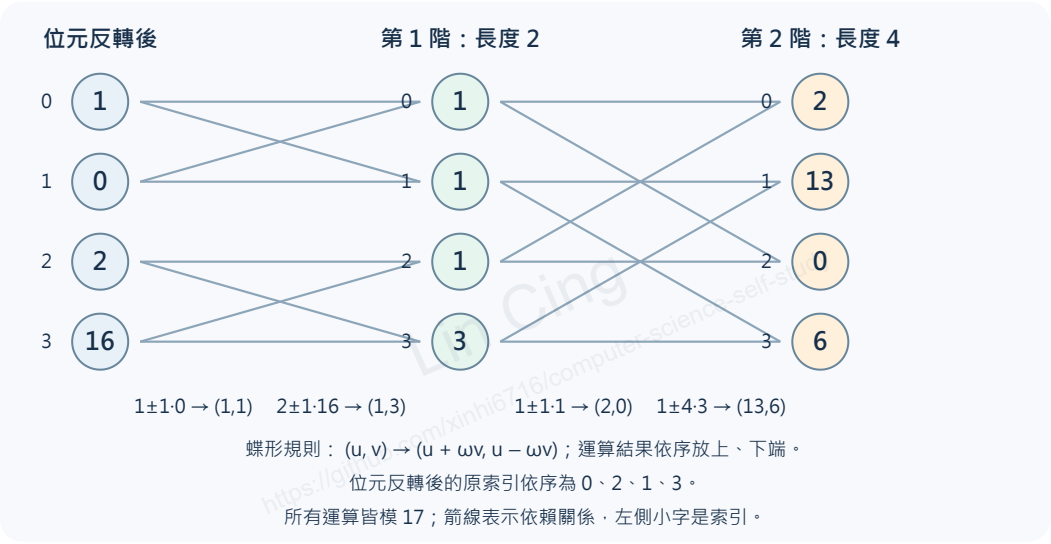
代入並交換求和可得 $x_j = N^{-1} \sum_{k=0}^{N-1} X_k \omega^{-jk} \pmod{p}$。這個論證仰賴域中非零元素可逆；不能把它直接推廣到任意合數模數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 具體 NTT：模 17 的兩層蝶形

$N = 4$、$p = 17$、$\omega = 4$：$4^2 = 16 = -1$、$4^4 = 1$，所以階恰為 4；$4^{-1} = 13$。輸入 $[1, 2, 0, -1]$，在模 17 中為 $[1, 2, 0, 16]$。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

依一般二次求值也能核對輸出 $[2, 13, 0, 6]$。每個負數都先按模 17 解讀。

## 點乘後的結果與整數還原

對 $A=[1,2]$、$B=[3,4]$，補零到 $N=4$。它們的 NTT 分別為 $[3,9,16,10]$、$[7,2,16,4]$；逐點乘得 $[4,1,1,6]$，逆變換為 $[3,10,8,0]$，對應 $3+10z+8z^2$。

要得到線性卷積而非循環卷積，須 $N \geq r+s-1$。對整數輸入，模 $p$ 結果只決定餘數；若已知每格係數絕對值至多 $M$ 且 $p>2M$，才可從對稱代表元唯一還原。此例 $r=s=2$、$|a_i|,|b_i| \leq 4$，取 $M=2\cdot 4^2=32$，而 $17 \not> 64$；例如餘數 $10$ 也代表整數 $-7$；若未另給足夠的範圍資訊，不能從模數結果判定實際係數是 10 還是 -7。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有號整數卷積：界限足夠的例子

令 $A=[1,2]$、$B=[1,-1]$，仍用 $p=17$、$N=4$、$\omega=4$。補零後，兩個 NTT 分別是 $[3,9,16,10]$、$[0,14,2,5]$，逐點相乘為 $[0,7,15,16]$；逆變換得到 $[1,1,15,0]$。

因輸入兩邊係數的絕對值均不超過 $B=2$，每個卷積係數至多含 $\min(2,2)=2$ 項乘積，故 $M=2B^2=8$。$p=17>2M=16$，其對稱代表元落在 $[-8,8]$：$15$ 還原為 $-2$，因而得到 $[1,1,-2,0]$。

長度 $N$ 的 FFT 需 $\Theta(N\log N)$ 次模 $p$ 算術操作；若 $p$ 隨輸入大小增長，模乘、模加的成本還取決於 $\log p$ 位元的運算方法，不能一律當成固定時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 精度與成本

三次變換與 $N$ 次逐點乘法需 $O(N \log N)$ 次算術運算及 $O(N)$ 空間。這不是位元複雜度保證。浮點誤差會累積；只有證明每項絕對誤差小於 $1/2$ 才能安全四捨五入為整數，否則須提高精度或採精確方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 選擇適合的方法

小輸入直接乘法可能較簡單。FFT 需要足夠補零與數值誤差預算；精確整數結果可考慮合適模數的數論變換與有界重建。介面應核對符號、索引和正規化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

CLRS, Introduction to Algorithms, third edition, chapters 28 and 30. Matrix: PDF pages 834–861. FFT: PDF pages 919, 921, 923, 928–936, 939–941, 944–945 (PDF page numbering, third edition). MIT 6.046J Fall 2004 lecture 3 slides, PDF pages 27 and 35–43, and Fall 2005 lecture 3 notes support matrix multiplication and Strassen, not FFT.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 數論演算法

從整除、不變量與同餘出發，建立最大公因數、模方程、快速冪、質數測試與 RSA。

### 整除與同餘

a≡b (mod n) 意味 n | (a−b)，n>0。例 29≡5 (mod 12)，因 29−5=2・12。加法與乘法可逐項取餘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 歐幾里得算法

例 gcd(252,198)：252=1・198+54；198=3・54+36；54=1・36+18；36=2・18+0，所以 gcd=18。

$$252 = 1\times198 + 54$$

$$198 = 3\times54 + 36$$

$$54 = 1\times36 + 18$$

$$36 = 2\times18 + 0$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最大公因數不變量

a=qb+r 時，a、b 的公因數恰為 b、r 的公因數，因 r=a−qb 且 a=qb+r。非雙零輸入先取絕對值，最後非零餘數就是 gcd。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 擴展歐幾里得表

保持 r=252s+198t。各列 (r,s,t)：(252,1,0)，(198,0,1)，(54,1,−1)，(36,−3,4)，(18,4,−5)，(0,−11,14)。因此 18=4・252−5・198。

$$252 = 1\times252 + 0\times198$$

$$198 = 0\times252 + 1\times198$$

$$54 = 1\times252 - 1\times198$$

$$36 = -3\times252 + 4\times198$$

$$18 = 4\times252 - 5\times198$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 係數更新與終止

若前兩列為 $(r_0,s_0,t_0)$、$(r_1,s_1,t_1)$，商 $q=\lfloor r_0/r_1 \rfloor$，下一列是 $(r_0-qr_1,s_0-qs_1,t_0-qt_1)$。僅在 $r_1>0$ 時更新，此時 $0\le r_0-qr_1<r_1$。每列維持線性組合，正餘數遞減而終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 整數輸入與 Bézout 係數

若 $(a,b)=(0,0)$，定義 $\gcd=0$，取 $x=y=0$。其餘情況從 $r_0=|a|$、$r_1=|b|$ 開始，對 $|a|$、$|b|$ 的係數列為 $(1,0)$、$(0,1)$。$r_1=0$ 時先停止，不做除法；最後非零 $r_0$ 是 $\gcd(a,b)$。

若 $r_0=s|a|+t|b|$，回填 $x=s\cdot\operatorname{sign}(a)$、$y=t\cdot\operatorname{sign}(b)$，其中 $\operatorname{sign}(0)=0$。則 $ax+by=\gcd(a,b)$，包含任一輸入為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 帶負號的例子

(7,−3) 取 (7,3)：7=2・3+1；3=3・1+0。回代 1=7−2・3=1・7+2・(−3)，故 (x,y)=(1,2)。

(−12,18) 取 (12,18)：12=0・18+12；18=1・12+6；12=2・6+0。回代 6=18−12=1・(−12)+1・18，故 (x,y)=(1,1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 零輸入的例子

(0,−5) 取 (0,5)：0=0・5+0，接著停止。gcd=5=0・0+(−1)・(−5)，故 (x,y)=(0,−1)。

(0,0) 不做除法。依約定 gcd(0,0)=0=0・0+0・0，取 (x,y)=(0,0)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Euclid 的最壞情況

令 a≥b>0，Euclid 使用帶餘除法。若有 k 次非零餘數，逆推最小可能輸入得到相鄰 Fibonacci 數；因此 $b \geq F_{k+1}$，而 $F_j \geq \varphi^{j-2}$（j≥2，φ=(1+√5)/2）。所以除法次數是 O(log(b+1))；相鄰 Fibonacci 數在輸入增大時達到 Θ(log b)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼 Fibonacci 出現？

設 g=gcd(a,b)>0，先把所有餘數除以 g，則最後兩個標準化餘數為 g/g=1 和 0。從標準化序列的終點 0,1 向前，每一步 $r_{i-1} = q_i r_i + r_{i+1}$，商 $q_i \geq 1$，所以 $r_{i-1} \geq r_i + r_{i+1}$。用商 1 反推得到 Fibonacci 下界。原始餘數最後是 g,0，故不假設 g=1。例 34=1・21+13、21=1・13+8、13=1・8+5、8=1・5+3、5=1・3+2、3=1・2+1、2=2・1+0。

**反推最小餘數**

每個值至少是右側兩值之和

商為 1 時取等號

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 除法次數與 bit 成本

令兩輸入最多各有 B 位。若採用傳統長除法，一次 B 位除法 $O(B^2)$ bit operations，加上 $O(B)$ 次除法，得到簡單上界 $O(B^3)$。這是保守上界，不等於最佳實作的複雜度；以除法運算為單位的 $O(B)$ 不能直接當作 bit 成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Euclid 兩步減半與逆元證書

對 $a \geq b > 0$，第一個餘數 $r = a \bmod b$。若 $r \leq b/2$，次參數一步已至少減半；若 $r > b/2$，因 $b/2 < r < b$，下一次商為 1，下一餘數 $b \bmod r = b - r < b/2$。因此每至多兩次除法，新的次參數不超過原來的一半，或演算法先終止。這給出 $O(\log(b+1))$ 次除法，不把大整數除法的 bit 成本當成常數。

例 $391 = 299 + 92$、$299 = 3 \cdot 92 + 23$、$92 = 4 \cdot 23 + 0$，所以 $\gcd = 23$。回代 $23 = 299 - 3(391 - 299) = 4 \cdot 299 - 3 \cdot 391$。因 $\gcd(299, 391) = 23 \neq 1$，299 沒有模 391 的乘法逆元；若存在 $299u \equiv 1 \pmod{391}$，則 $299u - 391v = 1$ 的左側卻被 23 整除，矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 線性同餘的可解性

ax≡b (mod n) 等價於存在整數 y 使 ax+ny=b。令 d=gcd(a,n)。n>0 時，可解的必要且充分條件是 d|b，且可解時模 n 恰有 d 個不同解：必要性由公因數整除左側；充分性由 Bézout 等式乘以 b/d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 列出全部同餘解

解 14x≡30 (mod 100)。d=2，化為 7x≡15 (mod 50)，$7^{-1}$≡43，故 x≡45 (mod 50)。模 100 恰有兩解 45、95；代入皆餘 30。

14x ≡ 30 (mod 100)

7x ≡ 15 (mod 50)

x ≡ 45 (mod 50)

x = 45, 95 (mod 100)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 互逆元素

a 在模 n 下有逆元當且僅當 gcd(a,n)=1。例 7・43=301≡1 (mod 50)。擴展歐幾里得找出的 Bézout 係數即逆元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 模線性方程：壓縮表示與輸出成本

求 ax≡b (mod n)，n≥1。令 g=gcd(a,n)。若 g∤b 無解；若 g|b，將方程與模數除以 g。若 step=n/g>1，此時 a/g 與 step 互質，可用逆元求一個 $x_0$；若 step=1，直接取 $x_0=0$，不需模1逆元。所有模 n 解恰是 $x_0+j\cdot step$（$0\le j<g$），其中 step=n/g，$x_0$ 取 $0\le x_0<step$。用三元組 $(x_0,step,g)$ 可表示全部解。

例 24x≡36 (mod 60)：g=12，約分得 2x≡3 (mod 5)，$x_0=4$、step=5，壓縮結果 (4,5,12)，代表 4、9、…、59 共12個解。0x≡0 (mod n) 對所有 n 個剩餘類成立，其三元組為 (0,1,n)，包含 n=1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 壓縮與逐一列出的成本

若 a、b 先模 n 化，三元組中每個數至多 n，所需空間是 O(log(n+1)) bits。這只是輸出表示大小，不包含求 gcd 或逆元的運算成本。

若要求逐一列出 g 個解，至少需 Ω(g) 次輸出；對 $0x \equiv 0 \pmod n$，g=n，若用一般二進位逐一寫出所有代表，總輸出長度為 Θ(n log(n+1)) bits。當 n 以 $\beta = \lceil \log_2(n+1) \rceil$ bits 給出時，n 可對 β 指數增長；因此壓縮輸出和逐一列舉不是相同成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 模算術的除法代表乘逆元

模 n 只有與 n 互質的元素才有乘法逆元。若 $b \in U(n)$，記 a/b 為 $a \cdot b^{-1} \pmod n$，它不是先做整數除法再取餘。例模 15 下，$7 \cdot 13 = 91 \equiv 1$，故 $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$。如果分母不是 unit，就不能直接取消；應解線性同餘並檢查 gcd 的可解條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 中國剩餘定理

若模數兩兩互質，給定各餘數有唯一解模各模數乘積。例 x≡2 (mod 3)、x≡3 (mod 5)：3 的倍數加 2 依序 2、5、8，故 x≡8 (mod 15)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 非互質模數的相容性

x≡a (mod m)、x≡b (mod n) 可解當且僅當 gcd(m,n)|(b−a)，解唯一模 lcm(m,n)。令 x=a+mt，轉成 mt≡b−a (mod n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 非互質例子

$x\equiv 2 \pmod 6$、$x\equiv 5 \pmod 9$：$\gcd=3$ 整除差 3，$6t\equiv 3 \pmod 9$，得 $t\equiv 2 \pmod 3$，故 $x\equiv 14 \pmod{18}$。若第二餘數為 4，差 2 不可被 3 整除，無解。

$$x = 2 + 6t$$

$$6t \equiv 3 \pmod 9$$

$$t \equiv 2 \pmod 3$$

$$x \equiv 14 \pmod{18}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 中國剩餘定理：互質模數

給定兩兩互質 $m_1,\cdots,m_k>1$ 與任意 $a_i$，$M=\prod m_i$，$M_i=M/m_i$。因 $\gcd(M_i,m_i)=1$，取 $y_i=M_i^{-1} \bmod m_i$；解為 $x\equiv \Sigma a_i M_i y_i \pmod M$。

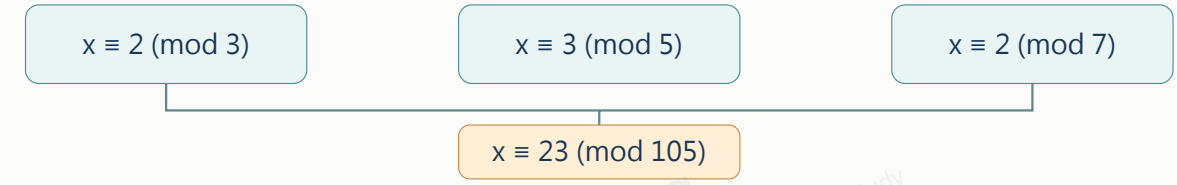
對第 i 個模數，只有第 i 項是 $a_i$，其餘項均為 0，因此構造確實滿足全部條件。若 $x,x'$ 都是解，每個 $m_i$ 均整除 $x-x'$；兩兩互質故 M 整除差，故解模 M 唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## CRT 例子

$x\equiv 2 \pmod 3$、$x\equiv 3 \pmod 5$、$x\equiv 2 \pmod 7$。$M=105$，$(M_1,M_2,M_3)=(35,21,15)$，其逆元分別為 $(2,1,1)$。$x\equiv 2\cdot 35\cdot 2+3\cdot 21+2\cdot 15=233\equiv 23 \pmod{105}$。檢查 23 的餘數分別是 2、3、2。

**同一個整數滿足三條限制**




23 除以 3、5、7 的餘數依序是 2、3、2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 不互質的 CRT

兩條同餘 $x\equiv a \pmod m$、$x\equiv b \pmod n$ 有解，若且唯若 $d=\gcd(m,n)$ 整除 $b-a$。必要性來自 $x-a$ 為 $m$ 的倍數且 $x-b$ 為 $n$ 的倍數。充分性：令 $x=a+mt$，解 $(m/d)t\equiv (b-a)/d \pmod{n/d}$；因 $m/d$ 與 $n/d$ 互質，$t$ 唯一模 $n/d$，故 $x$ 唯一模 $\operatorname{lcm}(m,n)$。多條限制可依序合併，遇不相容即無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 模冪演算法

m>1、e≥0。以 accumulator=1 開始，從最高位到最低位掃描 e 的全部位元：先 acc=acc² mod m，位元為 1 再 acc=acc・a mod m。e=0 時結果 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 模冪逐位追蹤

算 $7^{13} \bmod 23$，$13=(1101)_2$。acc 從 1 開始，第一位 1 先平方得 1，再乘 7 得 7；第二位 1 算 $7^2 \bmod 23=3$，再乘 7 得 21。各位後依序為 7、21、4、20。故結果 20。

bits: 1 1 0 1

acc: 1 → 7 → 21 → 4 → 20

$7^{13} \bmod 23 = 20$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 模冪不變量

已處理前綴代表整數 k 時，acc≡$a^k$ (mod m)。新位 b 把前綴變成 2k+b；平方後乘 $a^b$ 恰得新指數。掃描 O(log(e+1)) 位，這是模乘次數而非位元運算成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 右到左模冪與不變量

輸入 A∈ℤ、B≥0、n≥1。初始化 r=1 mod n、a=A mod n、b=B；當 b>0，若 b 為奇數先令 r←ra mod n，再令 a←$a^2$ mod n、b←⌊b/2⌋。不變量是 r・$a^b$≡$A^B$ (mod n)。

若 b=2q，更新後 r($a^2$)$^q$=r・$a^b$；若 b=2q+1，更新後 (ra)($a^2$)$^q$=r・$a^b$。每輪 b 減半，終止時 b=0，因此 r≡$A^B$。b 初始為 0 時直接回傳 1 mod n；特別地 n=1 時回傳 0。非互質底數同樣適用，因為只做乘法、不需逆元。負指數不是此演算法的輸入；若要定義，需另要求底數可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 右到左模冪：逐輪核對

計算 $7^{19} \bmod 23$，初態 (r,a,b)=(1,7,19)。每列由上一列依不變量更新；最後得到 11。

| 處理前 (r,a,b) | 處理後 (r,a,b) |
|---|---|
| (1,7,19) | (7,3,9) |
| (7,3,9) | (21,9,4) |
| (21,9,4) | (21,12,2) |
| (21,12,2) | (21,6,1) |
| (21,6,1) | (11,13,0) |

例如第三列 b=4 為偶數，r 保持 21，a 平方為 $9^2 \equiv 12$，b 除以 2 得 2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有限閉合子集與元素階

有限群 G 的非空子集 S 若對群運算封閉，就是子群。任取 $a \in S$；序列 $a, a^2, \cdots$ 都在 S 中，有限性保證 $a^i = a^j$（$1 \le i < j$），在 G 中消去 $a^i$ 得 $a^{j-i} = e \in S$。若 $j-i>1$，$a^{j-i-1} \in S$ 是 a 的逆；若 $j-i=1$ 則 $a=e$，其逆也在 S。這裡「有限」不能刪除。

元素 a 的 order 是使 $a^t = e$ 的最小正整數 t；$e, a, \cdots, a^{t-1}$ 互異，所有冪按指數模 t 重複。因此 $|\langle a \rangle| = t$，由 Lagrange 得 $t \mid |G|$，進而 $a^{|G|} = e$。例在加法群 $Z_8$ 中，2 生成 {0,2,4,6}，order 是 4；在乘法群 U(7) 中，2 生成 {1,2,4}，order 是 3。兩者運算不同，符號 $\langle 2 \rangle$ 的意義也不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Lagrange 定理：陪集如何分組？

設 $G$ 是有限群、$H$ 是子群。對 $a\in G$，左陪集 $aH=\{ah:h\in H\}$。映射 $h\mapsto ah$ 是雙射：左乘 $a^{-1}$ 可還原 $h$，所以每個左陪集都有 $|H|$ 個元素。

不同左陪集不是完全相同就是不相交：若 $ah_1=bh_2$，則 $b^{-1}a=h_2h_1^{-1}\in H$，因而 $aH=bH$。每個元素 $a$ 都屬於 $aH$（因單位元在 $H$），所以左陪集把 $G$ 分割成大小相同的區塊。故 $|G|=[G:H]|H|$，特別是 $|H|$ 整除 $|G|$。對 $H=\langle g\rangle$，便得 $\operatorname{ord}(g)\mid|G|$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Euler $\phi$ 的質數冪公式與積性

對質數 $p$、整數 $e\geq 1$，$1,\cdots,p^e$ 中恰有 $p^{e-1}$ 個 $p$ 的倍數，其餘均與 $p^e$ 互質，因此 $\phi(p^e)=p^e-p^{e-1}=p^{e-1}(p-1)$。

若 $\gcd(m,n)=1$，CRT 給出剩餘類的雙射；$x$ 與 $mn$ 互質，當且僅當它模 $m$、模 $n$ 的分量分別是 unit。故 $U(mn)\cong U(m)\times U(n)$，取數量得 $\phi(mn)=\phi(m)\phi(n)$。對一般 $n$ 可先分解成互質的質數冪再相乘。例 $72=8\cdot 9$，$\phi(72)=\phi(8)\phi(9)=4\cdot 6=24$；約定 $\phi(1)=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 有限群、Euler 定理與離散對數

模 n 的可逆元素 U(n) 在乘法下成群，大小為 φ(n)。對 a∈U(n)，乘以 a 是 U(n) 的置換；相乘並消去所有可逆因子，得到 $a^{\varphi(n)} \equiv 1 \pmod{n}$。更精確地，a 的階 $\mathrm{ord}_n(a)$ 整除 φ(n)，因其生成的子群大小整除群大小。

若 g 的階恰為 φ(n)，g 是模 n 的 primitive root，且 U(n)=⟨g⟩。給定 h∈⟨g⟩，離散對數是滿足 $g^x=h$ 的 x，唯一模 ord(g)；若 h 不在子群內則無解。不是每個模數都有 primitive root。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 群與離散對數例子

模 7 的 U(7)={1,2,3,4,5,6}。3 的冪依次為 3、2、6、4、5、1，因此 $\mathrm{ord}_7(3)=6$，3 為 primitive root；$\log_3 5 \equiv 5 \pmod 6$。相反，2 的冪為 2、4、1，生成子群 {1,2,4}；$2^x \equiv 3 \pmod 7$ 無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何模質數一定有生成元？

令 $G=F_p$ 的非零乘法群。對 G 中所有元素的 order 取最小公倍數 L。對 L 每個質因數 $\ell$ 的最高冪 $\ell^a$，挑一個 order 含此冪的元素，再取適當次方得到 order 恰為 $\ell^a$ 的元素。

不同質數冪的 order 互質。因 G 交換，這些元素相乘，其 order 為它們 order 的乘積 L。證明：若乘積的 k 次方為 1，每個因子的 k 次方同時位於自己所生成的子群及其餘因子所生成的子群。兩群的階互質，交集只有單位元，故每個 order 都整除 k；反向 L 次方顯然為 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 用多項式根數完成證明

所有 $x \in G$ 均滿足 $x^L=1$。體上非零 L 次多項式 $X^L-1$ 最多有 L 個根，因此 $p-1 \le L$。另一方面，由 Lagrange 定理每個元素的 order 整除 $p-1$，所以 L 也整除 $p-1$，特別是 $L \le p-1$。故 $L=p-1$，上一頁構造的元素生成 G。

根數上界依賴係數所在結構是「體」；不能把這項證明直接套用到任意模合數的剩餘環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 將生成元從 p 提升到 $p^2$

p 為奇質數，g 是模 p 的 primitive root。比較 g 與 g+p；二項式定理給 $(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p \pmod{p^2}$。第二項恰為非零的 p 倍數模 $p^2$，因此兩者至少一個 h 滿足 $h^{p-1} \not\equiv 1 \pmod{p^2}$。又 $h^{p-1} \equiv 1 \pmod{p}$，可寫成 1+cp，其中 p∤c。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 提升至所有奇質數冪

若 p 為奇質數且 $v_p(z-1)=j \geq 1$，二項式展開 $z^p-1=(1+(z-1))^p-1$：首項的 p-adic valuation 是 j+1，所有其餘項 valuation 更高，因此 $v_p(z^p-1)=j+1$。歸納得到 $v_p(h^{(p-1)p^k}-1) = k+1$。

h 在模 $p^e$ 的 order 是 (p−1) 的倍數，且整除 $\phi(p^e)=(p-1)p^{e-1}$。上式排除少於 e−1 個 p 因子，所以 order 恰為 $(p-1)p^{e-1}$，故 $U(p^e)$ 是 cyclic。例 p=3，g=2；$2^2=4 \not\equiv 1 \pmod 9$，所以 2 生成 $U(3^e)$ 的所有元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 哪些模數有 primitive root？

對 $n>1$，$U(n)$ 是 cyclic 當且僅當 $n=2$、$4$、$p^e$ 或 $2p^e$，其中 $p$ 為奇質數、$e\geq 1$。$U(2)$、$U(4)$ 可直接列舉；$U(p^e)$ 已證 cyclic，而 CRT 給 $U(2p^e)\cong U(2)\times U(p^e)$，前一分量只有一個元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 排除其餘模數

若 $8|n$，寫 $n=2^a m$，其中 $a\geq 3$ 且 $m$ 為奇數。任取 $b\in U(8)$，選 $b$ 的奇整數代表，先視為模 $2^a$ 的 unit。當 $m>1$，用互質的模數 $2^a$ 與 $m$ 以 CRT 解 $x\equiv b \pmod{2^a}$、$x\equiv 1 \pmod m$；當 $m=1$，直接取 $x=b$。所得 $x$ 與 $n$ 互質且 $x\equiv b \pmod 8$，故 $U(n)\to U(8)$ 的自然投影滿射。$U(8)=\{1,3,5,7\}$ 的每個元素平方皆為 1，卻有四個元素，因此非 cyclic。cyclic 群的同態像仍 cyclic，故 $U(n)$ 不可能 cyclic。

若 $n$ 有兩個相異奇質因數，CRT 可各自選 $\pm 1$，得到至少四個平方為 1 的 unit；若 $4|n$ 且 $n$ 另有奇質因數，$U(4)$ 與奇質數冪分量亦產生至少四根。有限 cyclic 群的 $x^2=1$ 至多有兩個解，故這些情況均被排除，完整涵蓋分類以外的 $n$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 模 121 的生成元：不能只看模 11

模 11 時，2 的 order 為 10。但要提升到 $11^2=121$，還需檢查 $2^{10}=1024\equiv 56\not\equiv 1 \pmod{121}$。模 121 的 order 是模 11 的 order 10 的倍數，且整除 $\phi(121)=110$；因此只可能是 10 或 110。10 已被排除，故 order=110，2 確實生成 U(121)。

這次檢查在模 $p^2$ 不等於 1，才可搭配前述 valuation 推導提升至更高奇質數冪；不能把「模 p 有生成元」直接當成任意選取的整數代表在模 $p^2$ 仍是生成元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 奇質數冪的平方根只有 ±1

設 p 為奇質數、$e\geq 1$，若 $x^2\equiv 1 \pmod{p^e}$，則 $p^e \mid (x-1)(x+1)$。由 $\gcd(x-1,x+1)$ 整除 2 可知 p 不可能同時整除兩個因子。因此 $p^e$ 的全部 p 因子都落在同一側，得到 $x\equiv 1$ 或 $-1 \pmod{p^e}$。反向這兩個值確實都是根；因 p 為奇數，兩者不同。

例模 25 僅有 1、24 兩個平方根。模合數可能有更多：模 15 的 1、4、11、14 都平方等於 1，這正是 CRT 各分量獨立取符號的結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 非平凡平方根如何給出因數？

給定奇數 n>1，若 $x^2 \equiv 1 \pmod n$ 但 $x \not\equiv 1、-1 \pmod n$，令 $d_- = \gcd(x-1,n)$、$d_+ = \gcd(x+1,n)$。兩者都小於 n，否則 $x \equiv \pm 1$；若 $d_- = 1$，從 $n \mid (x-1)(x+1)$ 可消去 x−1，推得 $n \mid (x+1)$，矛盾。$d_+ = 1$ 亦同理。故兩者皆為非平凡因數。

例 n=77、x=34：$34^2 = 1156 \equiv 1 \pmod{77}$，而 gcd(33,77)=11、gcd(35,77)=7。以 CRT 看，$34 \equiv 1 \pmod{11}$、$34 \equiv -1 \pmod 7$；不同分量選了不同符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 質數與合數

試除至 $\sqrt{n}$ 可判定整數 n≥2，但對長輸入所需步數可能很大。例 91=7・13 是合數；97 只需試除不超過 $\sqrt{97}$ 的質數 2、3、5、7，皆不整除，故為質數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Miller–Rabin 前置條件

$n<2$ 判定不是質數；$n=2$ 或 3 判質數；偶數 $n>2$ 判合數。對奇數 $n\geq 5$ 寫 $n-1=2^s d$，$d$ 為奇數，選底數 $a\in\{2,\cdots,n-2\}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 見證測試

算 $x=a^d \bmod n$。若 $x=1$ 或 $n-1$，此輪通過；否則重複平方至多 $s-1$ 次，若遇 $n-1$ 則通過；否則 $a$ 是合數見證。例 $n=15$，$a=2$，$d=7$、$s=1$，$2^7 \bmod 15=8$，未通過。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 質數不會被誤判

奇質數 p 中，$x^2 \equiv 1 \pmod p$ 只有 $\pm 1$，因 $p \mid (x-1)(x+1)$。結合費馬定理 $a^{p-1} \equiv 1$，可知合法底數必通過測試。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 隨機錯誤率與限制

對固定奇合數 $n \geq 5$，每輪從 $2 \cdots n-2$ 獨立均勻抽底數，通過 $t$ 輪仍為合數的機率至多 $4^{-t}$。這是對固定合數與隨機選擇的條件機率，不是任意輸入為合數的後驗機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fermat 偽質數：341

341=11・31 是合數，但 $2^{10}=1024\equiv 1 \pmod{341}$，故 $2^{340}=(2^{10})^{34}\equiv 1$。對底數 2 的 Fermat 測試會接受 341，卻不能因此認定它是質數。

Miller–Rabin 取 $340=2^2\cdot 85$；$2^{85}=(2^{10})^8\cdot 2^5\equiv 32$，下一次平方 $32^2=1024\equiv 1 \pmod{341}$。初值 32 既非 1 亦非 340，平方前也未見 −1，所以測試拒絕 341。這裡 32 是非平凡平方根；$\gcd(32-1,341)=31$，$\gcd(32+1,341)=11$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Carmichael 數：561

561=3・11・17 是合數，且 560 同時是 3−1=2、11−1=10、17−1=16 的倍數。若 $\gcd(a,561)=1$，由 Fermat 定理在三個質數模數下分別有 $a^{560}\equiv 1$；CRT 因而給 $a^{560}\equiv 1 \pmod{561}$。因此每一個與 561 互質的底數都通過 Fermat 測試；若底數不互質，這項結論不適用。

對 Miller–Rabin，$560=2^4\cdot 35$，底數 2 的序列為 $2^{35}\equiv 263$，接著平方依次得 166、67、1（mod 561）。初值不是 1，任何可檢查的項也不是 560，因此底數 2 揭露 561 是合數。注意 Fermat 全部互質底數均會受騙，不代表 strong test 也全部受騙。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strong pseudoprime：2047

2047=23・89 是合數，2046=2・1023，s=1、d=1023。由 $2^{11}=2048\equiv 1 \pmod{2047}$ 且 1023=11・93，得到 $2^{1023}\equiv 1$。因此 Miller–Rabin 以底數 2 的這一輪會接受 2047；2 是此合數的 strong liar。

三例對照的是固定底數的一輪結果。Miller–Rabin 的隨機誤判界來自依規定獨立、均勻抽取多個底數；固定使用底數 2 並不具備該機率保證。

**同一底數 2，三個合數的結果**

| 數字 | Fermat | Miller–Rabin | 依據 |
|---|---|---|---|
| 341 = 11·31 | 接受 | 拒絕 | 32 → 1 |
| 561 = 3·11·17 | 接受 | 拒絕 | 263 → 166 → 67 → 1 |
| 2047 = 23·89 | 接受 | 接受 | $2^{1023} \equiv 1$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 65 的 strong-liar 精確清單

65=5・13，$64=2^6\cdot 1$，所以 s=6、d=1；$\phi(5)=2^2\cdot 1$、$\phi(13)=2^2\cdot 3$。兩分量各有 $\gcd(1,u_i)=1$、$t_i=2$，故 L=1+1+4=6。

初值為 1 只給 a=1；初值為 −1 給 a=64。下一層需 $a^2\equiv -1 \pmod{65}$：模 5 的根是 2、3，模 13 的根是 5、8，四組 CRT 解為 8、18、47、57。完整 liar 集合為 {1,8,18,47,57,64}，其中實際底數區間 2,⋯,63 只留下四個，通過比例為 4/62=2/31。底數 2 的六個檢查值為 2、4、16、61、16、61，既未在初值得到 1，也未出現 64，故它是 witness。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Miller–Rabin 的 1/4 界：計數準備

令奇合數 $n=\prod_i p_i^{e_i}$，n−1=2ˢd（d 奇）。對每個 i，$\varphi(p_i^{e_i})=2^{t_i}u_i$（$u_i$ 奇），令 A=∏gcd(d,$u_i$)、t=min(s,$t_1$,⋯,$t_k$)。奇質數冪的乘法群是循環群；在大小為 $2^t u_i$ 的循環群，$z^d=1$ 有 gcd(d,$u_i$) 個解，

而 $z^{2^r d}=-1$（0≤r<$t_i$）有 $2^r$gcd(d,$u_i$) 個解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 強偽質數底數的精確計數

非互質底數一定不是 liar。CRT 把可逆底數分解成各質數冪座標；強測試接受當且僅當所有座標的 d 次冪均為 1，或存在同一 r∈[0,s) 使所有座標的 $2^r$d 次冪均為 −1。這些情況互斥，因此 liar 數 $L=A\left(1+\sum_{r=0}^{t-1}2^{kr}\right)$。其中 k 是相異質因數的個數。這是數量，不是機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 計數界：三個以上的質因數

當 $k\geq 3$，$\phi(n)=\prod 2^{t_i}u_i\geq 2^{kt}A$。且 $\left(1+\sum_{r=0}^{t-1}2^{kr}\right)/2^{kt}\leq 1/4$：$t=1$ 時為 $2/2^k\leq 1/4$，$t>1$ 時分子比對應幾何級數更小。故 $L\leq\phi(n)/4\leq(n-1)/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 計數界：恰有兩個質因數

若兩個 $t_i$ 不同，$\phi(n)\geq 2^{2t+1}A$，且 $1+\sum_{r=0}^{t-1}4^r\leq 4^t/2$，因此 $L\leq\phi(n)/4$。若兩個 $t_i$ 相同，而某個 $\gcd(d,u_i)<u_i$，或某個 $e_i>1$，因損失的奇因子至少為 3，$\phi(n)\geq 3\cdot 2^{2t}A$，得到 $L\leq\phi(n)/4$。

剩下的情況是不同奇質數 $p,q$ 各一次方、$t_1=t_2=t$ 且 $u_1,u_2$ 均整除 $d$。此時 $s\geq t$：$p=1+2^tu_1$、$q=1+2^tu_2$，所以 $pq-1$ 可被 $2^t$ 整除。故 $p-1$、$q-1$ 都整除 $pq-1$；模 $p-1$ 和模 $q-1$ 分別得 $p-1\mid q-1$、$q-1\mid p-1$，迫使 $p=q$，矛盾。故該情況不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 計數界：奇質數冪

若 $n=p^e$、$e\ge 2$，因 $p$ 不整除 $d$，$A=\gcd(d,p^{e-1}(p-1)$ 的奇部) 最多是 $p-1$ 的奇部。因 $t\le v_2(p-1)$，$L=A\cdot 2^t\le p-1\le (p^e-1)/4$，最後一步對奇 $p\ge 3$、$e\ge 2$ 成立。

綜合三種情況，$L\le (n-1)/4$。若從 $1,\cdots,n-1$ 均勻抽取，單輪誤判率 $\le 1/4$；若從 $2,\cdots,n-2$ 均勻抽取，去掉必為 liar 的 $1$、$n-1$ 後誤判率仍 $\le 1/4$。獨立重複 $r$ 輪才得到 $\le 4^{-r}$。質數不會被此測試誤判為合數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 固定合數誤判與事後機率

對每個固定奇合數，若每輪獨立且均勻取合法底數，$k$ 輪均通過的機率不超過 $\varepsilon=4^{-k}$。要使 $\varepsilon\le 10^{-9}$，因 $4^{14}=268435456<10^9\le 4^{15}=1073741824$，依此通用上界保證目標時，最小可選 15 輪；這不是說所有輸入的實際誤判率都需要 15 輪才能達標。

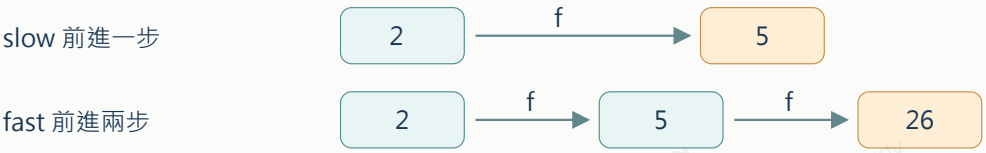
假設候選數分布的先驗質數率 $\pi=1/100$，測試對質數必通過，且對每個合數的通過率均 $\le\varepsilon$。由 Bayes 公式，$P(\text{合數}\mid\text{通過})\le (1-\pi)\varepsilon/[\pi+(1-\pi)\varepsilon]$。$k=15$ 時上界為 $99/(1073741824+99)\approx 9.22\times 10^{-8}$。這是該先驗下的後驗上界，不是固定合數的假陽性機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pollard ρ 與 Floyd 碰撞

選 $f(x)=(x^2+1) \bmod n$，令 slow=f(slow)、fast=f(f(fast))，每輪 $d=\gcd(|slow-fast|,n)$。若 $1<d<n$，得到非平凡因數；若 d=n，本次未分離因數，重新選種子或函數；d=1 則繼續。找到 d 不表示 d 一定是質數，仍須遞迴分解或檢測。

**Floyd 的雙速追蹤：$n=91$，$f(x)=x^2+1 \bmod 91$**




$\gcd(|5-26|,91)=7$，找到因數 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Pollard ρ 數例與啟發式

$n=91$，$x_0=2$。第一輪 slow=f(2)=5；fast=f(f(2))=f(5)=26；$\gcd(|5-26|,91)=\gcd(21,91)=7$，故 $91=7\cdot13$。這是固定 f 與種子的一個成功例，不保證其他輸入或種子都成功。

如果序列模未知最小質因數 p 的行為像隨機映射，生日碰撞啟發給約 $O(\sqrt{p})$ 次迭代；這依賴模型，不是最壞情況保證，也不含每次模乘與 gcd 的 bit 成本。可設定迭代上限並重啟，不能把啟發式寫成必然終止的定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Floyd 的三輪追蹤：8051

固定 n=8051、$x_0=2$、$f(x)=x^2+1 \bmod 8051$，slow 每輪走一步，fast 走兩步。每輪計算 $d=\gcd(|slow-fast|,8051)$。

| 輪 | slow | fast | 差的絕對值 | d |
|---|---|---|---|---|
| 1 | 5 | 26 | 21 | 1 |
| 2 | 26 | 7474 | 7448 | 1 |
| 3 | 677 | 871 | 194 | 97 |

第三輪 $677 \equiv 871 \equiv 95 \pmod{97}$；但它們模 83 分別是 13、41。因而 $\gcd(194,8051)=97$，且 $8051=97 \cdot 83$。最後兩因數可用試除至各自平方根確認為質數。

**第三輪比較兩個模數分量**

| slow = 677 | | fast = 871 |
|---|---|---|
| 677 mod 97 = 95 | = | 871 mod 97 = 95 |
| 677 mod 83 = 13 | ≠ | 871 mod 83 = 41 |

同餘模 97，但不相同模 83；因此 gcd = 97



## Checkpoint 是另一種排程

定義 $x_{i+1}=f(x_i)$。對 j=0,1,2,…，保存 $y=x_{2^j}$，讓 x 按序走過 $x_{2^j+1},\cdots,x_{2^{j+1}}$，每一步算 $\gcd(|x-y|,n)$，比較後才更新下一個 checkpoint。若得到 $1<d<n$ 則返回；若 d=n，重新選函數或種子；達工作上限仍無因數，只能回報本次未找到。

對有限函數圖，從 checkpoint 開始的序列終會進入長度 λ 的 cycle。若 y 已在 cycle 且隨後區間至少走 λ 步，就會遇到與 y 相同的分量。兩倍成長排程使用常數個整數狀態，與 Floyd 的 slow/fast 不是同一條追蹤表。

## Checkpoint 的具體操作

同樣取 n=8051、$x_0$=2。依序得到 $x_1$=5、$x_2$=26、$x_3$=677、$x_4$=7474、$x_5$=2839、$x_6$=871、$x_7$=1848。

| checkpoint y | 依次比較的 x | gcd 結果 |
|---|---|---|
| $x_1$=5 | $x_2$=26 | 1 |
| $x_2$=26 | $x_3$=677、$x_4$=7474 | 1、1 |
| $x_4$=7474 | $x_5$=2839、$x_6$=871、$x_7$=1848 | 1、1、97 |

在 $x_7$ 找到 97；因 97 是 8051 的非平凡因數，停止。本例的比較次序不同於 Floyd，雖然底層序列 $x_i$ 相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 生日碰撞界從何而來？

模型：從大小為 p 的集合獨立、均勻抽取 k 次；若 k≤p，前 j 次互異時，第 j+1 次避開既有值的機率為 1−j/p。連乘得 $\Pr(\text{無碰撞}) = \prod_{j=0}^{k-1}(1 - j/p)$。利用 1−u≤$e^{-u}$，得到 $\Pr(\text{無碰撞}) \le e^{-k(k-1)/(2p)}$；k>p 時無碰撞機率為 0。

令 k 約為 c$\sqrt{p}$，指數約為 $-c^2/2$，故典型碰撞尺度為 $\sqrt{p}$。這是獨立均勻抽樣的精確模型；隨機映射的初次碰撞過程也有相同連乘式。固定多項式 $f(x)=x^2+c$ 不能保證符合模型，因此 $\sqrt{p}$ 是啟發式，並非無條件的最壞時間或每次重啟成功率保證。平衡半質數 p≈$\sqrt{n}$ 時，對應 $n^{1/4}$ 次迭代的啟發尺度；每次模乘與 gcd 的 bit 成本須另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 獨立性與 Floyd 相遇的精確界

重複固定底數不會讓同一事件獨立；2047 對 base 2 每次均通過，再測多少次也如此。只有各輪依規定獨立抽樣，才可把每輪上界相乘。

有限函數圖從 $x_0$ 出發，尾長 $\mu \geq 0$、環長 $\lambda \geq 1$。Floyd 第 $k$ 輪比較 $x_k$ 與 $x_{2k}$。取 $k$ 為不小於 $\max(\mu,1)$ 的最小 $\lambda$ 倍數，則兩者均在環上、索引差 $k$ 是 $\lambda$ 的倍數，因此相遇，且 $k \leq \mu+\lambda$。這是函數圖上必然的相遇界；遇到相同值不必然得到非平凡因數。要估為 $O(\sqrt{p})$ 還需近似隨機映射的額外假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## gcd=n 的失敗案例

$n=15$，$x_0=0$，$f(x)=x^2+2 \bmod 15$。第一輪 slow=2、fast=6，$\gcd(|2-6|,15)=1$。第二輪 slow=6、fast=f(f(6))=f(8)=6，$\gcd(0,15)=15$。兩指標在模 $n$ 的整個狀態相遇，回傳 $n$ 不是成功的非平凡因數。

此時須換種子或函數重新嘗試，或在工作預算用盡時回報本次未找到因數；不能將失敗推論為 $n$ 是質數，也不能回報 15 作為 15 的非平凡分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## RSA 建鑰

取不同質數 p=11、q=13，n=143，φ(n)=120。選 e=7 且 gcd(7,120)=1，求 d=103，因 7・103=721≡1 (mod 120)。公開 (n,e)，保密 d、p、q。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## RSA 加密與解密

訊息代表元 0≤M<n。以 M=9 為例，C=$9^7$ mod 143=48；解密 $48^{103}$ mod 143=9。這是教學用小數值，無安全性。

- p=11, q=13, n=143
- e=7, d=103
- 9 → 48 → 9
- encrypt: $9^7$ mod 143 = 48

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## RSA 正確性

ed=1+kφ(n)。對 p，若 p|M 則兩邊皆為 0；否則由費馬定理 $M^{ed} \equiv M \pmod{p}$。對 q 同理；互質 CRT 得 $M^{ed} \equiv M \pmod{n}$，包含非互質訊息。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## RSA 對非 unit 也能還原

取不同質數 p=11、q=17；n=187，φ(n)=160，e=7，d=23，因為 7・23=161≡1 (mod 160)。選訊息 m=11，雖 gcd(11,187)=11，仍是合法的模 n 訊息。

加密：$11^2 \equiv 121$、$11^4 \equiv 55$，故 $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$。解密的平方鏈為 $88^2 \equiv 77$、$88^4 \equiv 132$、$88^8 \equiv 33$、$88^{16} \equiv 154$；23=16+4+2+1，154・132≡132，132・77≡66，66・88≡11。故 $88^{23} \equiv 11 \pmod{187}$。

**非 unit 訊息：逐質數分量驗證**

| | | | | | |
|---|---|---|---|---|---|
| 模 11：訊息是 0 | m=0 | —7次方→ | c=0 | —23次方→ | m′=0 |
| 模 17：訊息可逆 | m=11 | —7次方→ | c=3 | —23次方→ | m′=11 |

兩個分量均還原原值，CRT 唯一還原 11 mod 187

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何非 unit 的 RSA 等式成立？

設 ed≡1 (mod (p−1)(q−1))。對每個質數因子 r∈{p,q}：若 r|m，則 $m^{ed} \equiv m \equiv 0 \pmod r$；否則由 Fermat 定理及 r−1 | ed−1，得 $m^{ed} \equiv m \pmod r$。兩個分量均相同，由 CRT 得 $m^{ed} \equiv m \pmod{pq}$。

本例模 11 分量是 11→0、88→0；模 17 分量是 11→11、88→3，而 $3^{23} \equiv 11$。兩分量組合唯一還原 11 (mod 187)。上述論證假設 p、q 是相異質數、e,d 為正整數且乘積符合條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 裸 RSA 的可塑性

不加編碼的 E(m)=mᵉ mod n 是確定性映射，而且 E(m₁)E(m₂)≡E(m₁m₂) (mod n)。因此若 c=E(m)，任何 r∈U(n) 都可計算 c′=c・rᵉ mod n；它會解密成 mr mod n。這個代數可塑性不依賴知道 m 或私鑰。

例 n=187、e=7、c=88、r=2：$2^7 \equiv 128$，c′≡88・128≡44；$44^{23} \equiv 22$，正是 2・11。這不表示攻擊者一定在任何實際協定中得到明文，但揭示了直接使用裸冪無法提供所需安全性；實務加密必須採用經標準化的隨機化編碼與適當安全模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 裸 RSA 簽章的另一種缺陷

若簽章定義成 $\sigma = m^d \bmod n$，驗證只檢查 $\sigma^e \equiv m \pmod n$，攻擊者可先任選 $\sigma$，再設 $m = \sigma^e \bmod n$，得到可通過驗證的訊息／簽章對，無需私鑰。此構造讓攻擊者選擇 $\sigma$，通常不能讓他同時指定任意目標訊息；它證明的是在此訊息空間與裸驗證規則下可產生一組未經簽署的有效對。

例 $n = 55$、$e = 3$，選 $\sigma = 12$，則 $m = 12^3 \bmod 55 = 23$，因此驗證 $12^3 \equiv 23$ 成功。這是簽章驗證缺陷，與前頁討論的加密密文可塑性是不同的安全性問題；實際簽章還需經過合適的雜湊、編碼與嚴格驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 範圍與實務

本章為數學演算法，實際 RSA 需標準化編碼與隨機填充、安全參數及防側通道實作。質數測試的機率保證依隨機抽樣條件成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 31. MIT 6.046J Fall 2004 Lecture 7, p. 33, and Fall 2005 Lecture 8, p. 13: inverses modulo a prime.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 線性規劃：模型與證書

以線性目標與限制描述決策，並分辨最優、不可行和目標無界三種結果。

本章從形式轉換與建模開始，再以單形法、對偶與 Phase I 建立可檢查的證書。

## 學習路線

| 問題 | 方法 |
| --- | --- |
| 如何保留原問題的解？ | 轉換標準形與鬆弛形。 |
| 如何移動並維持可行？ | 基底、pivot 與比例測試。 |
| 如何判定結果？ | 對偶證書、不可行證書或改善射線。 |

前置知識包括線性代數、集合與基本最佳化概念。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 標準形與鬆弛形

本章的standard form固定為 $\max\ c^{\top}x \cdot Ax \le b \cdot x \ge 0$，$A \in \mathbb{R}^{m \times n}$。所有限制非嚴格，決策變數為連續實數；有理係數時才能以有限binary輸入談bit complexity。其他書也可能把等式形式叫standard，須先確認慣例。

Feasible point滿足全部限制；optimal point在可行點中達最大目標。Infeasible是沒有可行點；unbounded是可行且目標可任意大，不是「可行區域無界」的同義詞。例如$x \ge 0$、$\max(-x)$的區域無界，最優值仍是0。

有限個線性半空間的交集是凸多面體：若$Ax \le b$、$Ay \le b$，則$A(\lambda x+(1-\lambda)y) \le b$，$0 \le \lambda \le 1$。凸性不保證整數解，也不容許把非線性乘積當成線性限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 四種轉換都要能還原原變數

最小化f改最大化−f，輸出目標記得取負；$a^T x \geq b$乘−1改≤；等式拆成≤與≥。無符號限制的u改 $u=u^+-u^-$、$u^+,u^- \geq 0$：任意u可取(max(u,0),max(−u,0))，反向取差也恢復原點，雖不是一對一，最優值仍對應。

例 $\min\ 2u-v \cdot u+v=3 \cdot u-2v \geq -3 \cdot v \geq 0 \cdot u$ unrestricted，改最大化$-2u^+ +2u^- +v$，限制

$u^+ -u^- +v \leq 3$；$-u^+ +u^- -v \leq -3$；$-u^+ +u^- +2v \leq 3$。

代u=3−v得v≤2，原目標6−3v，故v=2,u=1、最小0；新目標最大0。若原有n變數、m限制，拆符號至多2n變數、拆等式至多2m限制，係數位元長僅增加常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Slack Form、Basis 與可行基本解

對Ax≤b引入s=b−Ax≥0，等式矩陣M=[A $I_m$]、全變數q=(x,s)≥0，Mq=b。Slack有唯一值，不是另一組可任意分配的決策。

選m個線性獨立columns作basis B，令其餘N非基本。由可逆$M_B$得

$$q_B = M_B^{-1} b - M_B^{-1} M_N q_N, \qquad z = v + \rho_N^T q_N.$$

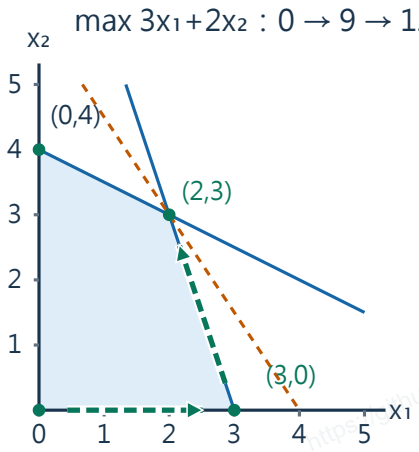
非基本全設0為basic solution；可行當且僅當$M_B^{-1} b$逐項≥0。初始slack basis是I，故初始點可行當且僅當b≥0。Basic不等於正：值0的基本變數是退化的來源。給定B後字典唯一，因解出線性方程所用的inverse唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 幾何、等值線與 Basis 是同一個例子

以下全部使用 $\max\ 3x_1+2x_2$，$x_1+2x_2\le 8$，$3x_1+x_2\le 9$，$x\ge 0$。可行四邊形頂點(0,0)、(3,0)、(2,3)、(0,4)。目標沿頂點路徑0→9→12改善，最後(2,3)。

初始字典：$x_3=8-x_1-2x_2$；$x_4=9-3x_1-x_2$；$z=3x_1+2x_2$。Basis={$x_3$,$x_4$}，初始slack=(8,9)。




藍線：兩個資源限制

$x_1+2x_2=8$

$3x_1+x_2=9$

箭頭虛線：simplex 的兩次移動

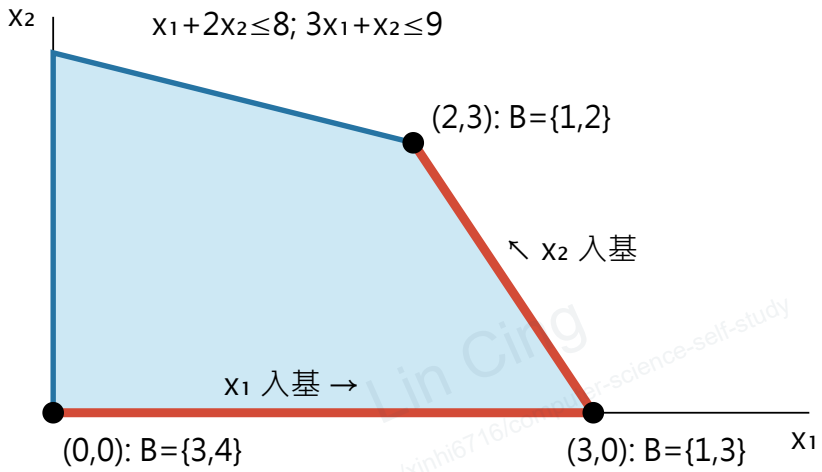
橘虛線：$3x_1+2x_2=12$

交點 (2,3)，目標 12

Lin Cing https://github.com/xinhi6716/computer-science-self-study



# 幾何、等值線與 Basis 是同一個例子 （續）




最優點也可能形成整條edge，例如改目標$x_1+2x_2$，邊(0,4)到(2,3)都同值8。頂點法會回其中一個最優基本解，不宣稱最優解唯一。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 線性規劃建模

建立模型先列「變數代表什麼、單位、允許範圍」，再逐項把可行解映到原問題，也把每個原可行方案映回變數。只有目標相似而漏了容量或守恆，並不是等價模型。

最短路使用potential $d_v$，固定$d_s$=0，對每邊uv限制$d_v - d_u \le w_{uv}$，最大化$d_t$。d沒有非負限制，負權路徑可能使$d_t$<0。

先取全部頂點從s可達且無負cycle、t可達的情形。沿任意s-t路把不等式相加，得$d_t$≤該路長，所以任何可行d是最短距離的下界；真距離δ(s,v)由三角不等式滿足所有限制，達到上界δ(s,t)。故最大化而非最小化恰得到最短距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Potential 的邊界不是程式中的 Infinity

若只研究s可達子圖，無負cycle時所有δ有限，可用前頁模型；t不可達要另外回報，不能把∞當實數LP變數。若保留不可達子圖，當中負cycle會使模型不可行，即使s到t距離完全正常，所以不能漏這個前提。

對無任何負cycle的全圖，可加0權supersource求一組全圖可行potential；若t不從s可達，可在不可達集合整體加任意正K，仍保持所有差限制，因無可達→不可達邊，故$d_t$無上界。這時LP unbounded對應t不可達，不是「存在負長路」。

數例s→a權2、a→t權−1、s→t權4，限制$d_a \le 2$、$d_t \le d_a - 1$、$d_t \le 4$。取(0,2,1)可行，且兩邊路給$d_t \le 1$，答案1；若錯加d≥0在別的負總長例會排除正確解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Flow 模型：每個 Arc 的非負量

此節用非負arc flow $g_{uv}$，而非上一章的反對稱淨流。最大流：對每個原邊設$0 \le g_{uv} \le c_{uv}$，內點滿足$\sum_{\text{out}} g - \sum_{\text{in}} g = 0$，最大化source淨流出。允許antiparallel，只要每個原arc各有自己的變數與容量；不能再加$g_{uv}$=−$g_{vu}$。

給原flow就滿足模型；給模型解，其容量與守恆正是flow定義。原邊變數m，限制O(n+m)，不必配置$n^2$個非邊變數。

最小成本流另固定source淨流出D、sink淨流入D，最小化$\sum_{(u,v)} a_{uv} g_{uv}$。D是「恰好需求」，不可只最大化總流再把成本當附帶值。原邊有有限容量時，負成本cycle可送循環流但不會因而無界；禁止循環需要另外建模。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

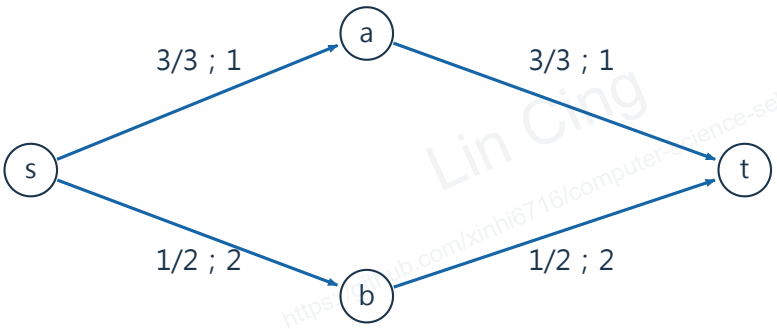


## 最小成本流的完整數值模型

原邊s→a容量3、單位成本1；a→t容量3、成本1；s→b容量2、成本2；b→t容量2、成本2。需求D=4。守恆令上下兩路流分別u,v，得到u+v=4，0≤u≤3、0≤v≤2，最小化2u+4v。

代v=4−u，成本16−2u；可行2≤u≤3，故u=3,v=1，成本10。這同時交付每邊flow、需求、可行區間與最優性，不只列出一條看似便宜的路。

需求 D=4；標記為「流 / 容量；單位成本」




u+v=4

2≤u≤3

成本=16−2u

最優 u=3,v=1

上路成本 3×(1+1)=6；下路成本 1×(2+2)=4；總 10

若把需求錯寫u+v≤4，零流成本0會被誤認最佳；若省容量u≤3，會錯選u=4，成本8。

## Multicommodity 不能拆成互不相干的最大流

對commodity k給$(s_k,t_k,D_k)$，每原邊有$g_{uv}^k \geq 0$。每種貨物各自滿足內點守恆、端點淨供需$\pm D_k$；共享容量是$\sum_k g_{uv}^k \leq c_{uv}$。若只問可行，目標可設0；可另加總成本。

同一瓶頸u→v容量1，兩種貨都唯一經過此邊，需求各3/4。各別都可送，但合併需求3/2>1，不可行。兩條獨立最大流會漏掉共享資源。

k種貨、m邊、n點有km變數、O(kn+m)限制，對給定有理D與容量為多項式大小LP。這是可分割貨流；要求每種貨只能選一條路或要求整數量，會改成不同的離散問題，不能沿用一般LP的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 單形法的字典與比例測試

固定使用以下符號慣例；$a_{ij}$ 是字典中「被減去」的係數。

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j, \qquad z = v + \sum_{j \in N} c_j x_j.$$

由 $b_i \geq 0$ 的可行字典選 $c_e > 0$ 入基，令 $x_e = \theta \geq 0$，其餘非基本變數維持零。各基本變數變為 $x_i = b_i - a_{ie}\theta$。

若 $a_{ie} > 0$，可行性要求 $\theta \leq b_i / a_{ie}$；若 $a_{ie} \leq 0$，此列不限制增加。取所有正分母的**最小非負比值，包含零**。

沒有正分母時，取 $q_e = 1$、$q_i = -a_{ie} \geq 0$、其他 $q_j = 0$。射線 $x(\theta) = x(0) + \theta q$ 保持可行，且目標 $v + c_e\theta \to \infty$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pivot：先解出入基變數

選 $e$ 入基、$l$ 離基，假設 $a_{le} \neq 0$。從第 $l$ 列解出

$$x_e = \frac{b_l}{a_{le}} - \frac{x_l}{a_{le}} - \sum_{j \in N \setminus \{e\}} \frac{a_{lj}}{a_{le}} x_j.$$

把這一列代回其他列與目標，再交換基本集合 $B$、非基本集合 $N$。$x_l$ 變成非基本變數，但仍出現在字典中；漏掉 $-x_l/a_{le}$ 就會改變可行集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Pivot：逐項更新係數

新入基列的係數與常數為

$$a'_{el} = \frac{1}{a_{le}}, \quad b'_e = \frac{b_l}{a_{le}}, \quad a'_{ej} = \frac{a_{lj}}{a_{le}}.$$

對其餘 $i \neq l$ 與 $j \in N \setminus \{e\}$，

$$b'_i = b_i - a_{ie} b'_e, \quad a'_{il} = -\frac{a_{ie}}{a_{le}}, \quad a'_{ij} = a_{ij} - \frac{a_{ie} a_{lj}}{a_{le}}.$$

目標列同樣代入得

$$v' = v + c_e b'_e, \quad c'_l = -\frac{c_e}{a_{le}}, \quad c'_j = c_j - \frac{c_e a_{lj}}{a_{le}}.$$

這是可逆代入，保留所有滿足原等式的點與目標值；再加上非負限制，得到同一個 LP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 比例測試為何保持可行？

新的入基值為 $b'_e = b_l / a_{le} = \theta \geq 0$。其餘基本值為

$$b'_i = b_i - a_{ie}\theta.$$

若 $a_{ie} > 0$，最小比值保證 $\theta \leq b_i / a_{ie}$，所以 $b'_i \geq 0$；若 $a_{ie} \leq 0$，則 $b'_i \geq b_i \geq 0$。新非基本變數全為零，因此新基本解可行。

目標增加 $c_e\theta \geq 0$。$\theta > 0$ 時嚴格改善；$\theta = 0$ 時，點與目標可能不變，只是基底改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Simplex 的控制流程

```
while some reduced cost c[e] > 0:
    choose entering e
    candidates = {i in B : a[i,e] > 0}
    if candidates empty: return feasible point + improving ray
    l = minimum-ratio row, using an anti-cycling rule
    pivot(l,e)
return current basic solution
```

比例測試不能排除零比值；沒有候選離基列時，改善射線才構成無界證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一二維例的兩次 Pivot

初始B={3,4}。先讓$x_1$入基，比值8/1=8、9/3=3，故$x_4$離基、θ=3：

$x_1=3-x_2/3-x_4/3$；$x_3=5-5x_2/3+x_4/3$；$z=9+x_2-x_4$。

再選$x_2$，$x_1$列比值3/(1/3)=9、$x_3$列5/(5/3)=3，故$x_3$離基：

$x_2=3-3x_3/5+x_4/5$；$x_1=2+x_3/5-2x_4/5$；$z=12-3x_3/5-4x_4/5$。

令非基本$x_3=x_4=0$，得到$(x_1,x_2)=(2,3)$，兩限制恰等號。所有reduced costs≤0，任意可行點的目標都≤12，所以此點最優。不必再數值搜尋整個可行多邊形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

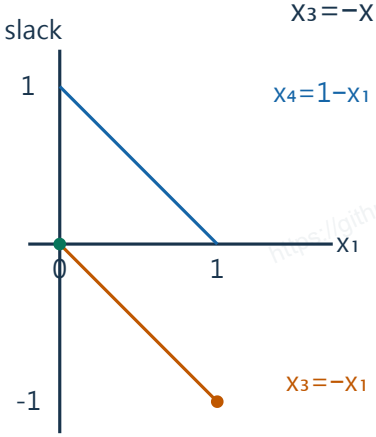


## 零步長是合法 Pivot

max $z=x_1$；$x_3=-x_1$；$x_4=1-x_1$，所有變數非負，初始B={3,4}、N={1}、基本值(0,1)。

| 基本列 | 正分母 | 比值 | θ=0 | 錯取θ=1 |
|---|---|---|---|---|
| $x_3$ | 1 | 0 | 0 | −1，不可行 |
| $x_4$ | 1 | 1 | 1 | 0 |

令$x_3$離基，得$x_1=-x_3$、$x_4=1+x_3$、$z=-x_3$。新基本點仍(0,0,1)，目標仍0，卻已能由非正reduced cost證最優。若只取positive ratio就立即越出可行域。




$x_3=-x_1$，$x_4=1-x_1$：零比值不能略過

$x_1=0$：$x_3=0$、$x_4=1$，可行

ratio：0/1=0 與 1/1=1

合法步長 θ=0，$x_3$ 離基

錯取 θ=1：$x_3=-1$，不可行

可行的 $x_1$ 只有 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 有限 Basis 本身不能排除 Cycling

M=[A I]有m行，任何basis選m個獨立columns；至多C(n+m,m)種。給定basis後字典唯一，所以若永不終止，必重複basis。但退化時目標可以不變，僅說「目標不下降」不能排除重複。

本章用精確lexicographic perturbation給出一個可自足證明的防循環版本。給當前可行起始字典 $q_B = \beta - Dq_N$，把RHS改$\beta_i$+$\varepsilon^i$，i=1,⋯,m；ε是正無窮小，**不是**某個任意浮點小數。常數項非負且每行有正的第一個ε係數，所以初始字典在此有序域嚴格可行。

計算與比較改用係數向量：先比常數，再比ε、$\varepsilon^2$,⋯；首個非零係數決定正負。後續pivot仍用正分母的最小比值，若實數比值相同，用後面的係數分出順序。

Bland 規則：固定變數編號，入基選正 reduced cost 中最小編號；最小非負比值平手時，離基也選最小編號。精確運算且兩側皆依此規則，才能套用有限終止定理；僅在平手時選最小編號並不足以宣稱防循環。



## Lexicographic 規則為何有限終止

把初始字典等式矩陣記$M_0$=[I D]，perturbed RHS $\beta + \sum_i e_i\epsilon^i$。任何可逆basis K的基本值是

$$M_{0,K}^{-1}\beta + \sum_{i=1}^{m}(M_{0,K}^{-1}e_i)\epsilon^i.$$

Inverse每行不全零，因此每個基本值作為ε多項式都不恒等於0；可行性又保證它非負，故其實嚴格正。兩個正分母ratio不可能完全相同：否則inverse兩行成比例，違反可逆。每次合法pivot的θ因此嚴格正，$c_e$>0使目標在lex順序嚴格增加。

同一basis的字典／目標唯一，不能重訪。有限basis保證終止。取所有係數的常數項，就回到原LP的可行基本解；reduced costs與RHS無關，若全≤0則常數解也最優。若出現改善ray，其係數同樣不依ε，從常數可行點出發便證原LP無界。故不是靠挑一個「夠小」的浮點ε猜測正確性。

## Lexicographic 平手的可重算例子

$x_3=2-x_1$；$x_4=2-x_1-x_2$；max $x_1+x_2$。初始把兩個RHS改$2+\varepsilon$、$2+\varepsilon^2$。$x_1$入基的兩比值是$2+\varepsilon$與$2+\varepsilon^2$，後者較小，所以$x_4$離基。

新列 $x_1=2+\varepsilon^2-x_2-x_4$、$x_3=\varepsilon-\varepsilon^2+x_2+x_4$、$z=2+\varepsilon^2-x_4$。後者已無正reduced cost，常數解$x_1=2$、$x_2=x_3=x_4=0$，目標2，符合原限制。$x_3$雖常數0，perturbed值$\varepsilon-\varepsilon^2$嚴格正，正是退化被符號分開的地方。

另一常用防循環法Bland規則是選最小索引的正reduced-cost入基，最小比值平手時選最小索引離基；兩處規則都要給。以上完整終止證明採lex版本，不把任意選取也說成必終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 每次 Pivot 的成本與全程成本要分開

普通dense字典一次更新O(mn)個係數，不能由此稱整個simplex多項式；basis數C(n+m,m)對維度可能指數大。Lex版本另存每個RHS的m+1個係數，更新與比較增加多項式因子，仍只有有限／指數的全程最壞界。

有理輸入用精確分數做pivot。Basis解由矩陣子行列式的比值表示；清除原分母後，行列式有至多m!項，bit長受維度與輸入bit長的多項式界控制。這說明單一basis的精確表示不必無限制膨脹，不代表pivot次數變多項式。

浮點實作要另訂feasibility／reduced-cost容許值、ratio近零與重算殘差；把負的b無條件截成0可能改題。Ellipsoid／interior-point屬不同演算法，其多項式理論不由上述pivot計數推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 對偶

Primal `max cᵀx · Ax≤b · x≥0` 對偶為 `min bᵀy · Aᵀy≥c · y≥0`。m個資源限制對應m個價格y；n個原變數對應n個價格下界。A需轉置，不能只交換max/min。

給兩側可行解，依x≥0、Aᵀy≥c與y≥0、Ax≤b依序得

$$c^T x \leq x^T A^T y = y^T A x \leq y^T b.$$

這是weak duality。任一可行dual給primal上界；若上下界相等，兩者同最優。只有相同目標值而不檢查兩側可行，不能當最佳性證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從終止 Basis 構造 Dual，一般證明

令M=[A I]、d=(c,0)，終止basis B的矩陣可逆。定義row vector $y^T = d_B^T M_B^{-1}$。代入$q_B = M_B^{-1} b - M_B^{-1} M_N q_N$得

$$z = y^T b + \sum_{j \in N} (d_j - y^T M_j) q_j.$$

所有columns的reduced cost $\rho_j$=$d_j$−yᵀ$M_j$，基本columns恰0。最優字典的ρ≤0：對slack column $e_i$有$\rho_{n+i} = -y_i \leq 0$，所以y≥0；對decision column $A_j$有$\rho_j$=$c_j$−yᵀ$A_j$≤0，所以Aᵀy≥c。

目前可行基本解的目標是yᵀb，故達weak-duality等號。這證明有限最優primal必有同值dual；也得到讀取規則「$y_i$=−原slack變數的reduced cost」，若該slack基本則成本0。證明依完整字典恆等式，並非只在單一最優點比較係數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一數例的 Dual 與 Complementary Slackness

二維例dual是 $\min\ 8y_1+9y_2$，$y_1+3y_2\ge3$、$2y_1+y_2\ge2$、$y\ge0$。終止目標$z=12-3x_3/5-4x_4/5$給$y=(3/5,4/5)$。兩個dual限制均等號，值$24/5+36/5=12$，與$x=(2,3)$吻合。

一般gap精確分解為

$$b^Ty - c^Tx = y^T(b - Ax) + x^T(A^Ty - c).$$

對兩側可行，右側每項非負，所以gap=0當且僅當每項都0：$y_i(b_i - A_ix) = 0$、$x_j((A^Ty)_j - c_j) = 0$。這就是complementary slackness，必要與充分都來自此分解；有正slack則對應價格0，有正變數則對應dual限制緊束。

另例$\max\ 2x_1+x_2$，$x_1+x_2\le4$，$x_1\le2$，$x\ge0$，dual可取$y=(1,1)$，上界6。$x=(2,2)$值6、全部互補乘積0；$x'=(1,3)$值5，唯一非零乘積為$y_2(2-x'_1)=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一數例的 Dual 與 Complementary Slackness （續）

$$\text{gap} = y^T(b-Ax) + x^T(A^Ty-c) \ge 0$$

$x=(2,2)$

$y=(1,1)$，dual 值 6

primal 6

$y_2(2-x_1) = 0$

其餘三個乘積皆 0

$x'=(1,3)$

$y=(1,1)$，dual 值 6

primal 5

$y_2(2-x_1) = 1$

其餘三個乘積皆 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 無界與不可行不能隨便互換

Primal無界時dual不可能可行，否則固定dual值由weak duality限制所有primal值；同理dual向下無界則primal不可行。逆向單靠這句不成立：primal不可行不代表dual無界，兩者可能都不可行。

例如 $\max\ -x \cdot 0\cdot x\le -1 \cdot x\ge 0$ 的primal不可行，dual $\min\ -y \cdot 0\cdot y\ge -1 \cdot y\ge 0$ 可行但向下無界。改為 $\max\ x \cdot 0\cdot x\le -1 \cdot x\ge 0$，dual限制$0\ge 1$也不可行，兩側皆不可行。

對一般符號可先轉standard再建dual；原primal等式限制的乘子可正可負，free primal變數對應dual等式。這些符號規則由拆成兩個非負變數或兩個相反不等式合併而來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 建立初始基本可行解

初始b有負項時，不能直接跑只維持可行性的ratio test。對$Ax\le b, x\ge 0$加入$a\ge 0$，解auxiliary：$\max\ -a \cdot Ax-a\cdot 1\le b$。它總可行：取x=0、a=max(0,$-\min_i b_i$)。目標$\le 0$，所以不會向上無界。

原LP可行⇒同一x配a=0可達目標0，已是auxiliary最優。反向auxiliary最優0⇒a=0⇒同一x滿足原限制。若最優<0，原LP不可行。不能以「目前人工變數仍正」提早判不可行，必先確定auxiliary已最優。

有m=0時直接取原點為可行基本解；若某$c_j$>0便有無界ray，否則原點最優，不必求空集合的min。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一次特殊 Pivot 建立 Auxiliary 可行基底

令$s_i$=$b_i$−$A_i$ x+a，挑r使$b_r$=$\min_i b_i$<0。讓a入基、$s_r$離基；此步pivot係數−1，**不是**從可行基底選改善方向的ratio步。

解出 $a = -b_r + A_r x + s_r$，代其他列 $s_i = b_i - b_r + (A_r - A_i)x + s_r$。設新非基本x=$s_r$=0，得到a=−$b_r$>0、$s_i$=$b_i$−$b_r$≥0，故建立可行基底。現在才可使用前述simplex迴圈及lex防循環。

Lex防循環從這個可行字典開始：可只為迴圈加入符號RHS，再取常數項；reduced costs不依RHS，因此得到原auxiliary的真最優值及可行基本解，不會把ε目標誤判成原問題的不可行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 人工變數為零，仍要清理 Basis

Auxiliary最優0時，若a非基本，直接刪a欄。若a基本且值0，其列為 $a = -\sum_j a_{aj} x_j$；若存在非零係數，任選該$x_j$入基、a離基。這個θ=0的代數pivot不改目前點，係數可正可負，不是改善目標的ratio選步。

若整列係數皆0，則a=0是冗餘等式，刪此列與a即可。一般等式／多人工變數實作必處理此分支；對獨立的完整slack矩陣可另由row rank排除，但不應除以0。

最後把**原目標**中的基本變數逐一用目前列代入，得到只含非基本變數的目標；不可只改標題或把auxiliary目標−a當原答案。這時完成Phase I，才用新的可行basis跑Phase II。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Phase I 與原目標的完整操作例

原題 $\min\ z=2x_1+x_2\cdot x_1+x_2\geq 5\cdot x\geq 0$。Surplus $x_3\geq 0$給$x_1+x_2-x_3=5$；引人工a：$a=5-x_1-x_2+x_3$，Phase I最小化w=a（等價最大化−a）。

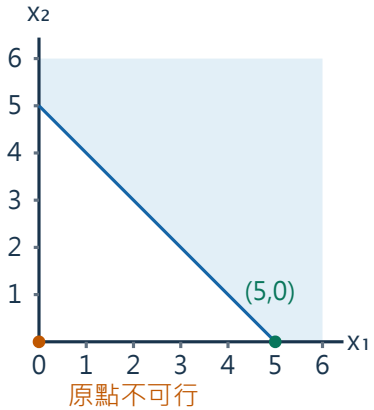
| 狀態 | 基本列 | 非基本0時 | 目標 |
|---|---|---|---|
| 初始 | $a=5-x_1-x_2+x_3$ | $a=5$ | $w=5$ |
| $x_1$入、a離 | $x_1=5-x_2+x_3-a$ | $x_1=5$ | $w=a$，最小0 |
| 刪a、代原目標 | $x_1=5-x_2+x_3$ | $x_1=5$ | $z=10-x_2+2x_3$ |
| $x_2$入、$x_1$離 | $x_2=5-x_1+x_3$ | $x_2=5$ | $z=5+x_1+x_3$ |

最小化終止時係數≥0，故最優z=5。起始$(x_1,x_2)=(0,0)$對原題不可行；配a=5後只對輔助題可行。這個單等式例用一個人工欄；上一頁的共用a模型則直接適用所有$Ax\leq b$列。

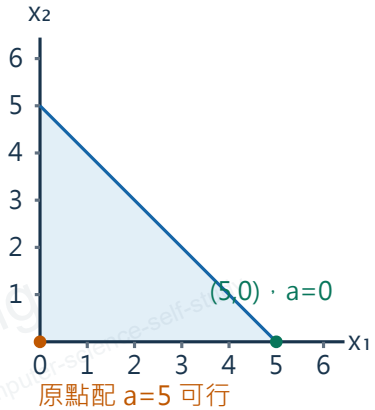


# Phase I 與原目標的完整操作例 (續)

原題：$x_1+x_2\geq 5$




輔助切片 $x_3=0$：$a=5-x_1-x_2$




加入人工變數只改輔助問題；a=0 才回到原限制

## 不可行也要有算完的證書

原題 $x\le 1$、$x\ge 2$、$x\ge 0$，加共用a後為$x-a\le 1$；$-x-a\le -2$。相加得$-2a\le -1$，故$a\ge 1/2$；取$x=3/2$、$a=1/2$兩式恰等號，所以auxiliary最大$-a=-1/2<0$，原題不可行。

完整字典：$s_1=1-x+a$、$s_2=-2+x+a$。先讓a入、$s_2$離，得$a=2-x+s_2$、$s_1=3-2x+s_2$、$w=-2+x-s_2$。x入基兩ratio為2、3/2，$s_1$離基，得

$x=3/2-s_1/2+s_2/2$、$a=1/2+s_1/2+s_2/2$、$w=-1/2-s_1/2-s_2/2$。

兩個reduced costs非正，確已最優，不是因尚有人工量就猜不可行。亦可直接把原兩個相反限制相加得到$0\le -1$作獨立矛盾證書。一般地，若能給出$y\ge 0$、$A^Ty\ge 0$且$b^Ty<0$，則任何假設的可行x都滿足$0\le x^TA^Ty=y^TAx\le y^Tb<0$，矛盾。本例$y=(1/2,1/2)$，$A^Ty=0$、$b^Ty=-1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 三種結果與基本定理的依賴關係

Phase I先有可行auxiliary基底，lex迴圈有限終止且auxiliary不向上無界；最優$<0$回infeasible，最優0清理人工欄並恢復原目標。

Phase II仍以可行字典啟動，lex規則保證有限終止。若沒有正分母，交付可行點與改善ray，證unbounded；否則終止時reduced costs$\le 0$，交付可行基本解與同值dual，證finite optimum。

所以每個本章標準形式LP恰屬不可行、可行而目標無界、可行且達有限最優之一。非嚴格限制與有限維LP結構很關鍵；若改成$\max\ x\cdot 0\le x<1$，上確界1不被任何點達到，便不是這個三分結論的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, Chapter 29. MIT 6.046J Fall 2004, Lecture 15; Fall 2005, Lecture 18.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 多執行緒演算法

動態多執行緒以依賴 DAG 分開工作總量、關鍵路徑與排程。本章從 spawn、sync 和競爭修復出發，推導矩陣乘法與平行 merge/sort 的演算法、證明和空間條件。

先備知識為 DAG、遞迴分析與並行成本模型；我們也會區分平行度上界與增加處理器後的實際收益。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 動態多執行緒基礎

動態多執行緒模型讓程式描述哪些計算可以並行，由排程器決定何時、在哪個處理器執行。邏輯task不是固定綁定的OS thread；spawn不承諾立刻取得另一個核心。

本章採相同速度的P個理想處理器與循序一致的共享記憶體：每條指令可看成某個符合各依賴的交錯順序，load/store各自是單一步驟；讀、加、寫的合成操作不自動具有原子性。單位成本算術、索引、spawn/sync為常數工作，排程器本身的時間、頻寬、cache及通訊成本不計入此理想界。

非空有限計算用DAG表示。節點可是一個strand（不含平行控制的一段順序指令），帶自己的正執行時間；空strand可刪除，要用單位節點證明時，可再拆成單位指令鏈。控制與資料依賴決定有向邊，而不是把畫面上同層的節點一律當成可並行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Spawn、call、sync 的確切語義

`x = spawn F(...)`允許F與呼叫者後續程式並行；普通`y = G(...)`則呼叫者須等待G返回。`sync`等待本程序此前spawn的child全部完成，才執行後面的指令；它不是全系統的barrier。

spawn參數先在parent求值。child的區域變數屬該次呼叫，指向共享陣列的參數仍可能讓多task存取同一格；不能只看變數名稱不同就判獨立。讀spawn回傳值之前必須有相應依賴，通常明寫sync。

本章程序返回前隱含sync，但若先求`return x+y`的值才返回，隱含等待不足以允許提早讀x；因此先sync再算回傳式。刪掉spawn/sync並將parallel for改普通for得到serialization；它能解出正確答案，不代表所有平行交錯也正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fibonacci 範例的程式與正確性

```
P-FIB(n):                 // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

兩個child只讀自己的n並回傳各自結果，沒有共享寫入。n下降使呼叫終止；基例F0=0、F1=1，加上歸納假設與sync後的x+y，得到正確Fn。若把兩個child都spawn，仍必等兩者後才能相加，漸近work/span不變，只改常數。

這個版本故意沒有memoization，同一參數會算多次。高平行度不等於work有效率：順序迭代法只需O(n)次算術，不能因遞迴版本可以占用更多核心，就忽略其指數工作量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一 strand 模型的 work 與 span

為了逐點驗算，將每個非基例呼叫分為三個單位strand：pre（到spawn）、cont（到普通第二呼叫）、join（sync後相加返回）；每個基例為一個單位strand。pre先於左child及cont，cont先於右child，兩個child結束都先於join。

令W為全部strand成本，D為最長依賴路徑成本，則

$$W_n = W_{n-1} + W_{n-2} + 3, \quad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \quad D_0 = D_1 = 1.$$

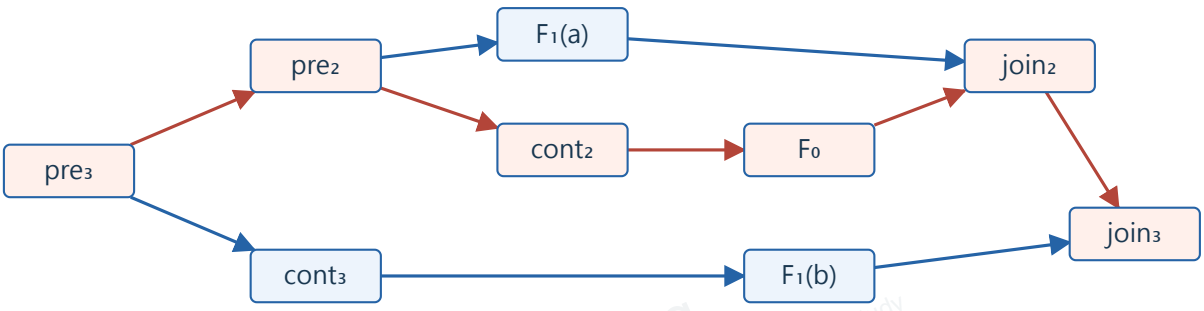
| n | Fn | Wn | Dn |
|---|---|---|---|
| 2 | 1 | 5 | 4 |
| 3 | 2 | 9 | 6 |
| 4 | 3 | 17 | 8 |

Wn=4F(n+1)−3可直接代入歸納，Dn=2n對n≥2成立。令 $\varphi = (1+\sqrt{5})/2$；由φ²=φ+1，歸納得φ^(n−2)≤Fn≤φ^(n−1)（n≥2）：n=2、3可核對，兩界的相鄰和仍是下一個φ次方。因此W=Θ(φ^n)、D=Θ(n)。這是明列的單位strand模型，不把一個完整遞迴呼叫誤算成一個O(1)節點。



## 九個 strand 的依賴可逐條追蹤

P-FIB(3) 的完整九個 strand：箭頭是先後依賴




每格恰為一個單位 strand；W=9。紅色最長路徑含6格，因此 D=6。

圖的pre3先分到pre2及cont3；cont3才呼叫右側F1。pre2也先分到其F1與cont2，cont2再呼叫F0。兩個基例都完成才可做join2，join2與右側F1都完成才可做join3。

紅路徑pre3→pre2→cont2→F0→join2→join3共6個節點，W=9、D=6。另一條pre3→cont3→F1→join3只有4個節點。圖上箭頭是依賴方向，不是把同一呼叫的所有節點排成一條長鏈。

## Work law、span law 與可達的加速比

記 $W = T_1$、$D = T_\infty$、實際P處理器時間 $T_P$。每個時間單位最多完成P單位工作，且關鍵路徑不可並行，故

$$T_P \geq \max\{W/P, D\};$$

單位節點離散模型還有 $T_P \geq \lceil W/P \rceil$。加速比 $S_P = W/T_P \leq \min(P, W/D)$，效率 $S_P/P \leq 1$。平均平行度A=W/D限制加速比；P>A便無法有完美的P倍加速，不代表第P+1顆處理器必然沒有收益。

順序合成兩段時，W相加、D也相加；若兩段獨立地fork後join，W仍相加，D取兩段最大值，另加明列的fork/join成本。空計算W=D=0直接時間0，不把0/0稱為平行度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Greedy 排程的完整上界證明

每個時間步，只要有至少P個ready節點便執行P個；不足P時全部執行，這是greedy。令完整步C、不完整步I，總時間T=C+I。計算尚未結束時，每個不完整步至少執行一個節點，故W≥PC+I。

不完整步會移除剩餘DAG的所有source。步後的最長路徑，可在該步開始前的DAG中向前延伸到至少一個被移除的source；若步後已空，步前span也至少1。因此每個不完整步使剩餘span至少減1，最多有D個不完整步，I≤D。代入得

$$T \leq \frac{W - I}{P} + I \leq \frac{W - D}{P} + D \leq W/P + D.$$

最後一步使用P≥1使I的係數1−1/P非負。單位節點證明可對整數長度strand拆鏈後套用；它不是無成本處理任意不可切割、長度未知task的另一個模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 兩倍近似與 slackness 的量化

最優排程也受 $T_P^* \geq \max(W/P, D)$ 限制，故greedy的T≤W/P+D≤2T*。更精確地可得 $(2 - 1/P)T_P^*$ 上界，但無需預知未來DAG就有常數比保證。

定義slackness $\sigma = W/(PD) = A/P$。對D>0，上下界給

$$1 \leq \frac{T_P}{W/P} \leq 1 + \frac{1 - 1/P}{\sigma} \leq 1 + 1/\sigma.$$

所以σ很大時，理想greedy時間接近W/P；這是數學界，不是對真機頻寬或排程器開銷的實測。最優排程可以停用多餘處理器，所以其時間隨P不增；任意實作的測得時間另受開銷與排程選擇影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 平行度30，第31顆仍有收益

697個互相獨立的單位工作全部完成後，才能開始23個單位工作的依賴鏈。W=697+23=720，D=1+23=24，A=30。

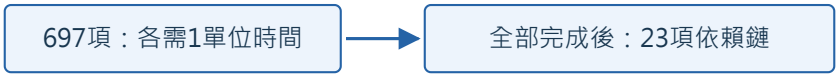
| P | 獨立階段的完整配置 | 階段時間 | 鏈時間 | 精確最短時間 |
|---|---|---|---|---|
| 30 | 23輪各30項，再7項 | 24 | 23 | 47 |
| 31 | 22輪各31項，再15項 | 23 | 23 | 46 |

所有前驅都完成前鏈不能開始，所以ceil(697/P)+23同時是可行排程與下界。
S30=720/47≈15.32、S31=720/46≈15.65，均≤30，但增加處理器確實仍有收益。P≥697才可把第一階段縮成1輪、達到D=24。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

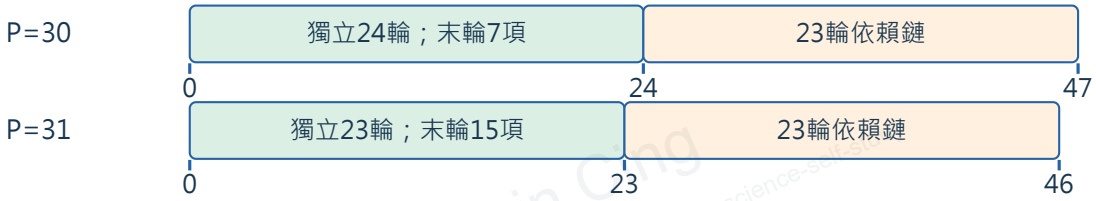
## 平行度30，第31顆仍有收益 （續）

相同 DAG：31個處理器仍比30個快




W=697+23=720
D=1+23=24

平均平行度 W/D=30；以下兩列使用相同時間比例尺。




$T_{30}=47$、$T_{31}=46$：加速比至多30，不代表第31個處理器沒有收益。



## 減 work 但增加 span，可能改變優劣

兩個正確版本分別有(W,D)=(1024,4)、(768,16)，都用理想greedy排程。

P=8時，第一版時間至少1024/8=128，第二版時間至多768/8+16=112，因此第二版一定較快。
P=128時，第一版至多1024/128+4=12，第二版至少16，因此第一版反而一定較快。

這個判定用到不重疊的上下界，不把W/P+D的上界誤當精確時間。若兩版本的可行時間區間重疊，單靠W,D未必能判定哪個實際較快。少量核心上的單次時間，也不足以識別work與span各自的改變。

## Parallel for 如何以 fork/join 實作

以下與合併算法均採0-based半開區間[l,r)，空區間l=r。

```
PAR-FOR(l,r,body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l,m,body)
    PAR-FOR(m,r,body)
    sync
```

n個非空迭代恰對應n葉、n−1內節點；每個內節點常數spawn/sync工作，所以額外work O(n)，不會憑空多出n lg n。最長呼叫與join路徑深度Θ(lg n)，因此總span是Θ(lg n)+各迭代span最大值，前提是各迭代真的獨立。

若用普通for依序spawn n個task，parent本身就有Θ(n)長的發出鏈，不能宣稱同樣對數span。平行迴圈結尾的sync也確保下一段讀取所有結果時已完成。



## Matrix-vector：只平行不同 row 是安全的

```
MAT-VEC(A,x):
    n = A.rows
    allocate y[n]
    parallel for i = 0 to n-1:
        y[i] = 0
        for j = 0 to n-1:
            y[i] += A[i,j]*x[j]
    return y
```

每個i獨占y[i]，A,x只讀；內圈第j步後是前j+1個乘積的和，所以正確。work $\Theta(n^2)$，span Θ(lg n)+Θ(n)=Θ(n)，平均平行度Θ(n)。n=0回空向量。

不能只把內圈也改parallel for：各j對同一y[i]做讀改寫，會產生determinacy race（未由依賴排序的衝突存取）。兩個邏輯並行存取同格、至少一個寫入便有此衝突；即使碰巧寫相同值，也不能用「跑過一次正確」證明獨立。

## 循序一致仍會遺失更新

遺失更新：逐指令讀值、暫存值與最後寫回

初始 x=0；每個 task 內部仍依 load → add → store。

| 時間步 | Task 1 | Task 2 | 共享 x |
|---|---|---|---|
| 1 | load $r_1 \leftarrow x = 0$ | | 0 |
| 2 | | load $r_2 \leftarrow x = 0$ | 0 |
| 3 | add $r_1 \leftarrow r_1+1 = 1$ | | 0 |
| 4 | store $x \leftarrow r_1$ | | 1 |
| 5 | | add $r_2 \leftarrow r_2+1 = 1$ | 1 |
| 6 | | store $x \leftarrow r_2$ | 1 |

兩者結尾 sync 後讀到1：等待完成不能把三條指令變成不可分割操作。

圖的交錯尊重各task的load→add→store，最後x卻為1；如果第一個task全部完成才換第二個，會得到2。循序一致只限制單條指令的交錯，不把三條指令合成原子操作。

本章以互不衝突的分工或reduction消除此類競爭。原子加或鎖也可建立依賴，但其同格序列化、工作量與span必須另算；不能保留「全部加法同時完成」的分析。



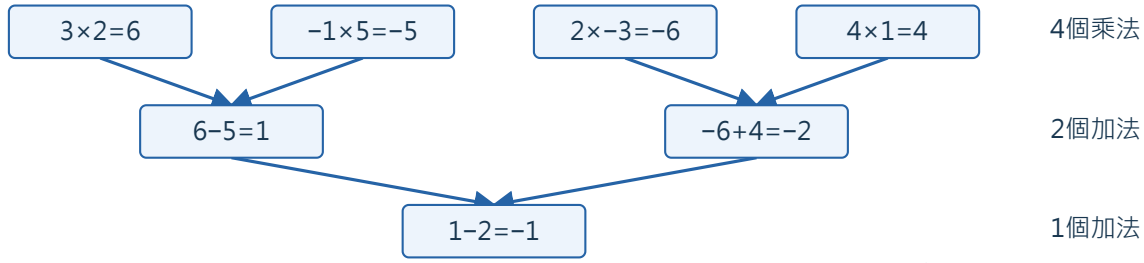
## 用 reduction 正確地平行內積

```
DOT(Arow,x,l,r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m = floor((l+r)/2)
    left = spawn DOT(Arow,x,l,m)
    right = DOT(Arow,x,m,r)
    sync
    return left+right
```

每個child只回傳自己的值，沒有共同累加格；結合律與歸納保證結果是區間內積。work為$\Theta(n)$，span $\Theta(\lg(n+1))$。把MAT-VEC每個row改成這個DOT，得到$\Theta(n^2)$work、$\Theta(\lg n)$span（$n \geq 2$）。例row=[3,−1,2,4]、x=[2,5,−3,1]，乘積[6,−5,−6,4]，第一層和[1,−2]，最後−1。

## 用 reduction 正確地平行內積 （續）

內積不是共寫一個總和：四個私有乘積沿樹合併




算術 DAG：work=7、span=3；fork/join 另有 Θ(log n) 控制深度。

對浮點加法，固定的樹可能和順序累加有不同捨入結果；正確性證明先在精確、可結合算術下成立，不能順手承諾逐bit相同。



## 粒度與記憶體配置的模型

將n個常數成本迭代分成每批最多g個、以平衡PAR-FOR發出各批，1≤g≤n，work Θ(n)，span Θ(lg⌈n/g⌉+g)。較大g減少task常數開銷，也增加每葉序列成本；是否值得依P及slackness而定。

本章allocate取得未初始化、互不重疊儲存空間的描述符，抽象為常數span；若實作需要逐格初始化，必須計入相應工作並以parallel loop初始化才保留對數span。不是把一個串行清零$n^2$格的操作藏進O(1)。

矩陣區塊用(row起點、column起點、stride、階數)的view表示，不隱藏整塊複製。任務控制區、分配器、cache與記憶體頻寬在實作時另有成本；以下數學界皆在已明列的理想模型內。

## 多執行緒矩陣乘法

```
P-MULTIPLY(A,B):
    n = A.rows                  // both n by n
    allocate C[n,n]
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1:
            C[i,j] = 0
            for k = 0 to n-1:
                C[i,j] += A[i,k]*B[k,j]
    return C
```

每個(i,j)獨占一格輸出，內圈仍依序累加。work $\Theta(n^3)$，span兩層發出樹$\Theta(\lg n)+\Theta(\lg n)$，再加一條內圈$\Theta(n)$，總$\Theta(n)$，平均平行度$\Theta(n^2)$。兩個parallel關鍵字沒有消除每格的n項依賴。

若每個Cij改由前述DOT類型的二分內積回傳，work仍$\Theta(n^3)$，span則$\Theta(\lg n)$，平均平行度$\Theta(n^3/\lg n)$。直接並行寫入Cij不是這個reduction算法。

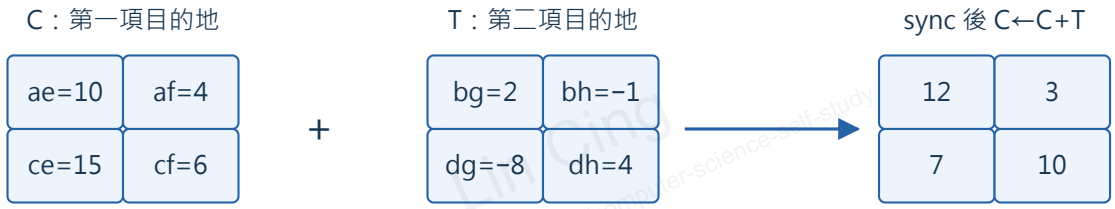


## 八次子乘法必須寫八個不同目的地

令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，此處字母是同階區塊。先計C=[[ae,af],[ce,cf]]與暫存T=[[bg,bh],[dg,dh]]，八個乘積各有獨占目的地。sync後才做C=C+T。

數例A=[[2,−1],[3,4]]、B=[[5,2],[−2,1]]：C暫時[[10,4],[15,6]]，T=[[2,−1],[−8,4]]，相加得[[12,3],[7,10]]。

八個獨立乘法：目的地分開，回收時才相加




八個乘法各寫 C 或 T 的一格；先等待全部乘法，再逐格相加。

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]；同格的兩項沒有同時做 +=。

若把ae與bg兩個task都寫成C11+=product，兩者便衝突。分出T是在用額外空間解除寫入依賴，不是只加spawn就能平行。

## 遞迴矩陣乘法的完整程序

```
P-REC(C,A,B):
    n = A.rows
    if n == 1: C[0,0] = A[0,0]*B[0,0]; return
    allocate T[n,n]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    view C and T into four n/2 by n/2 blocks
    spawn P-REC(C11,a,e); spawn P-REC(C12,a,f)
    spawn P-REC(C21,c,e); spawn P-REC(C22,c,f)
    spawn P-REC(T11,b,g); spawn P-REC(T12,b,h)
    spawn P-REC(T21,d,g); P-REC(T22,d,h)
    sync
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1: C[i,j] += T[i,j]
```

前提為同階方陣、階數是2的冪，A與B只讀，C不與它們重疊。任意正階數先補零到下一個2冪再取回結果，n=0直接回空矩陣。T及每個child工作區也獨立；八個spawn/call數量是固定常數，不是n長的發出鏈。



## 八路矩陣乘法的歸納與遞迴成本

基例寫出正確單乘積。歸納假設各n/2子乘積正確，八個child因目的地不同而無競爭；sync確保全部乘積可讀，再逐格相加，依區塊乘法恆等式得到AB。只有共同讀A、B不構成衝突。

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

span沿八個child中一條最長路徑，不是八者相加；每層回收的nested parallel for又有Θ(lg n)發出／join深度，不能當O(1)。n=2^L時Σr=1..L r=Θ($L^2$)，同時給上、下界。n=4有$8^2$=64個純量乘法葉；平行化沒有少做乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen 平行化的三個同步階段

本章以 $M_i$ 表示第 13 章的 $P_i$，兩者是同一個乘積。先以平行逐格加減準備(a+d)、(e+h)、(c+d)、(f−h)、(g−e)、(a+b)、(c−a)、(e+f)、(b−d)、(g+h)。輸入準備完成後，七乘積同時可執行：

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

七者完成後sync，再平行回收C11=M1+M4−M5+M7，C12=M3+M5，C21=M2+M4，C22=M1−M2+M3+M6。不能在輸入和差未完成前spawn消費它們，也不能在乘積未完成前回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Strassen 的 work、span 與平行工作區

每層七個同階child，常數個矩陣加減：

$$\begin{aligned} W_S(n) &= 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \\ D_S(n) &= D_S(n/2) + \Theta(\lg n) = \Theta(\lg^2 n). \end{aligned}$$

正確性由第 13 章已展開的四個區塊恆等式及遞迴歸納，平行化另需上述三階段依賴。其平均平行度Θ(n^(lg7)/lg²n)小於八乘法版本，卻具有較低work；平行度的比值不是單獨選算法的目標。

順序深度優先的O(n²)暫存不能直接移給所有child同時展開的版本。若每個呼叫分配自己的Θ(n²)工作區並保持到join，八路的完全展開空間滿足Q(n)=8Q(n/2)+Θ(n²)=Θ(n³)；七路則為Θ(n^(lg7))。有限P與工作區重用可降低峰值，但必同時核對排程與span，不能沿用一個未實作的空間界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 平行 Strassen 的可執行規格

定義PADD(X,Y,s)配置獨立R，以兩層parallel for寫R[i,j]=X[i,j]+s・Y[i,j]，等待迴圈完成再回R。取s=±1；其work $\Theta(n^2)$、span $\Theta(\lg n)$。以下A、B同階且先補到2冪，n=0由外層直接回空矩陣。

```
P-STRASSEN(A,B):
    n = A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    S1 = PADD(a,d,+1); S2 = PADD(e,h,+1)
    S3 = PADD(c,d,+1); S4 = PADD(f,h,-1)
    S5 = PADD(g,e,-1); S6 = PADD(a,b,+1)
    S7 = PADD(c,a,-1); S8 = PADD(e,f,+1)
    S9 = PADD(b,d,-1); S10 = PADD(g,h,+1)
    M1 = spawn P-STRASSEN(S1,S2)
    M2 = spawn P-STRASSEN(S3,e)
    M3 = spawn P-STRASSEN(a,S4)
    M4 = spawn P-STRASSEN(d,S5)
```



## 平行 Strassen 的可執行規格 （續）

```
    M5 = spawn P-STRASSEN(S6,h)
    M6 = spawn P-STRASSEN(S7,S8)
    M7 = P-STRASSEN(S9,S10)
    sync
    allocate C[n,n], viewed as four blocks
    parallel for i = 0 to n/2-1:
        parallel for j = 0 to n/2-1:
            C11[i,j] = M1[i,j]+M4[i,j]-M5[i,j]+M7[i,j]
            C12[i,j] = M3[i,j]+M5[i,j]
            C21[i,j] = M2[i,j]+M4[i,j]
            C22[i,j] = M1[i,j]-M2[i,j]+M3[i,j]+M6[i,j]
    return C
```

十個PADD在此依序呼叫，數量固定，故總span仍$\Theta(\lg n)$；先全部完成才spawn七乘法，與三階段依賴一致。沒有把任意數量的串行發出忽略掉。

## 多執行緒合併排序

只把兩個遞迴sort並行、仍用順序MERGE，並不充分：

```
PAR-SORT-SERIAL-MERGE(A,l,r):
    if r-l <= 1: return
    m = floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A,l,m)
    PAR-SORT-SERIAL-MERGE(A,m,r)
    sync
    serial MERGE the two sorted halves using an auxiliary buffer
```

work為2W(n/2)+Θ(n)=Θ(n lg n)，span卻是D(n/2)+Θ(n)=Θ(n)，平均平行度僅Θ(lg n)。合併前的sync必要，否則會讀到未排序完成的run；要進一步縮短span，必須平行化merge本身。



## 二分切點先定義半開區間與平手規則

兩個輸入run位於只讀陣列T的[l1,r1)、[l2,r2)，輸出O[o,o+N)不得覆寫T，N為兩段長度和。為保證穩定性，每個record以(key,原始位置)作總序比較；原始位置唯一，相同key仍按原先順序。

```
LOWER-BOUND(T,l,r,x):
    while l < r:
        m = floor((l+r)/2)
        if T[m] < x: l = m+1
        else: r = m
    return l
```

不變量是原區間中小於x者的切點位於當前[l,r]；在原輸入run內，當前l左方都<x、r右方（含r）都≥x。每次依中點比較保留可能切點，長度嚴格縮小；空run直接回l，永遠不讀不存在的T[l]。工作及span O(1+lg(n2+1))，明列空run的常數處理。

## 可處理空 run 的完整 Parallel Merge

```
P-MERGE(T,l1,r1,l2,r2,O,o):
    n1 = r1-l1; n2 = r2-l2
    if n1 < n2: swap (l1,r1) with (l2,r2); swap n1,n2
    if n1 == 0: return
    m = floor((l1+r1)/2)
    j = LOWER-BOUND(T,l2,r2,T[m])
    q = o+(m-l1)+(j-l2)
    O[q] = T[m]
    spawn P-MERGE(T,l1,m,l2,j,O,o)
    P-MERGE(T,m+1,r1,j,r2,O,q+1)
    sync
```

先把較長run放第一描述符；交換的是view，不移動或重新標記record。若第一段空，第二段也空；若僅第二段空，仍從第一段取median逐步定位，不讀空段。每次拿走一個pivot，兩個child總長N−1，因此有限輸入必終止。



## 輸出位置、正確性與穩定性的一般證明

左側應有m−l1個第一run元素、j−l2個第二run元素，總長q−o。它們都在總序上小於pivot；其餘元素都大於pivot。故pivot的全域輸出位置是q，兩個child輸出區間恰為[o,q)與[q+1,o+N)，彼此及pivot均不重疊。

對N歸納：N=0空合併正確；兩child均較小，由歸納假設正確合併相應元素。左答案、pivot、右答案串接已排序且每個輸入record恰出現一次，得到完整merge。T只讀、O的寫入區間互斥，所以沒有determinacy race，sync後全部結果可用。

相同key按全域原始位置比較，輸出保持原先相對次序，因此穩定。若只比key且一律使用同一lower_bound規則，長度交換可能把相等的右run元素放到左run之前；普通排序正確不自動等於穩定。

# 八個 record 的根切點與二分步驟

T=[1a,4b,6c,9d | 2e,3f,6g,8h]，字母依a..h表示原始位置0..7；比較先key再原始位置。l1=0,r1=4,l2=4,r2=8,o=0。第一run長4，m=2、pivot=6c。

| 當前搜尋[l,r) | mid | T[mid]與6c比較 | 更新 |
|---|---|---|---|
| [4,8) | 6 | 6g>6c（key相等、位置較後） | r=6 |
| [4,6) | 5 | 3f<6c | l=6 |
| [6,6) | — | 空搜尋區間 | 回j=6 |

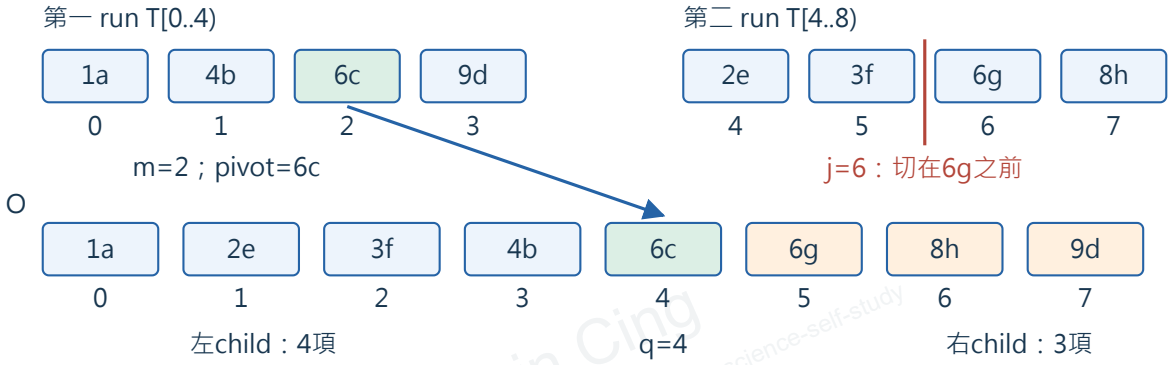
q=0+(2−0)+(6−4)=4；左child共4元素，右child共3元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 八個 record 的根切點與二分步驟 （續）

中位 pivot 的秩：$q=o+(m-l_1)+(j-l_2)=4$



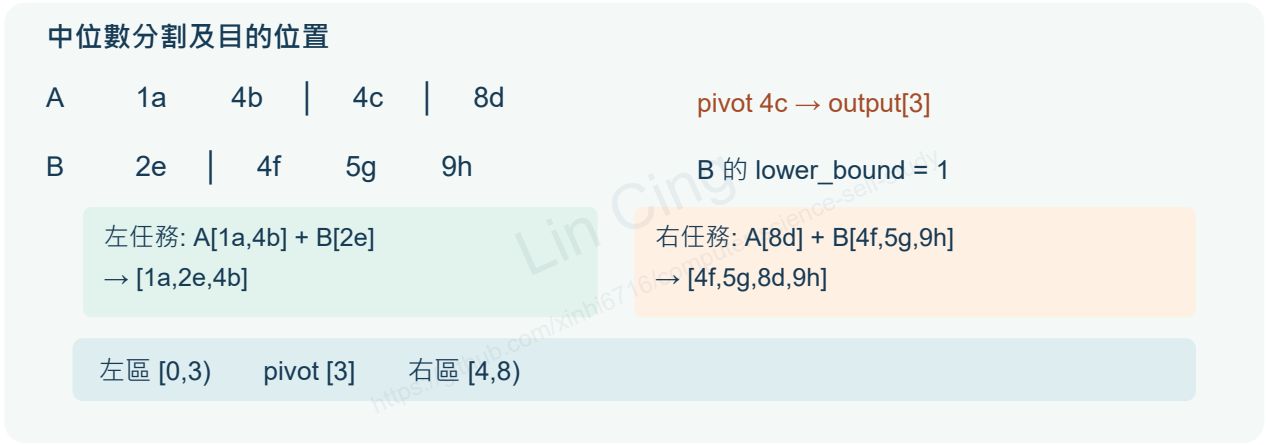

比較 (key, 原始位置)：6c < 6g；左區、pivot、右區的輸出互不重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同鍵案例：每個位置從哪裡來？

`A=[1a,4b,4c,8d]`，`B=[2e,4f,5g,9h]`；字母代表原始位置且 b < c < f。選 `x=4c`，A 左側長度 2；B 中第一個不小於 `4c` 的位置是 1，故輸出位置 `q=2+1=3`。

左任務合併 `[1a,4b]` 與 `[2e]`，右任務合併 `[8d]` 與 `[4f,5g,9h]`。兩側完成後得到 `[1a,2e,4b,4c,4f,5g,8d,9h]`。






## 同一數例的全部輸出寫入

以下為一個合法深度優先serialization的pivot寫入順序；平行執行可以改交錯，但目的格相同、各寫一次。

| 寫入序 | pivot | q（0-based） | 來源／輸出意義 |
|---|---|---|---|
| 1 | 6c | 4 | 根的左右分界 |
| 2 | 4b | 3 | 根左段的最後元素 |
| 3 | 3f | 2 | 較長run交換後的median |
| 4 | 2e | 1 | 小run的切點 |
| 5 | 1a | 0 | 另一run已空 |
| 6 | 8h | 6 | 根右段交換後的median |
| 7 | 6g | 5 | 右段左child |
| 8 | 9d | 7 | 右段右child |

最終O=[1a,2e,3f,4b,6c,6g,8h,9d]，兩個6仍c在g之前。即使改成先跑右child，只讀T及唯一輸出格也保證相同答案；不是靠整個陣列寫入順序維持排序。

## Merge 最壞 span 的上界與下界

較長run長n1≥n2，取median後任一child至多⌊n1/2⌋+n2≤(N+n2)/2≤3N/4。即使一側切點落在端點，這個上界仍成立。二分搜尋至多O(lg(N+1))，所以

$$D_M(N) \le D_M(3N/4) + O(\lg(N+1)) = O(\lg^2(N+1)).$$

沿路每次固定比例縮小，有O(lg N)層，第r層剩餘log大小O(lg N−cr)，等差和給平方對數。

下界用近等長交錯run，例如[0,2,⋯,2m−2]與[1,3,⋯,2m−1]，m是2冪。左child持續成為同型的兩個等長交錯run，長度每層減半，且搜尋另一run的median切點仍需Θ(lg m)比較。一路累加lg m+(lg m−1)+⋯給Ω(lg²m)，所以最壞span為Θ(lg²N)。若另一run始終空，搜尋只需常數、span則Θ(lg N)；不能把最壞界稱為每個輸入的精確成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 線性 work：一次寫出不等於已付清搜尋

每個非空呼叫挑走一個不同pivot，所以有N個非空呼叫；各生兩child，含空返回共2N+1個呼叫。N次輸出給Ω(N)，但O(N)上界還必須支付每個非空節點的二分搜尋，不能只說「每元素寫一次」。

大小m的節點非遞迴工作≤c lg(m+1)。把此成本平均分攤給它仍含有的m個輸入record，每個收 c・lg(m+1)/m。固定record x只在一條ancestor鏈上被收費，直到自己被選成pivot為止。

從最後一次含x的呼叫向根反數r層，因child≤3parent/4，該層大小至少(4/3)^r。下一頁將此一般不等式求和，連空child也可處理，不需要取lg0或聲稱小規模的兩個child都至少N/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 每個 record 的搜尋費總和有常數界

$g(u)=\lg(u+1)/u$ 在u>0遞減，因導數分子的符號等於u/(u+1)−ln(u+1)<0；後者由積分 ∫ 0^u 1/(1+t)dt>u/(1+u)得到。因此反數r層的分攤費至多

$$c\frac{\lg((4/3)^r+1)}{(4/3)^r}\le c\,[1+r\lg(4/3)](3/4)^r.$$

幾何和Σq^r=1/(1−q)，以及由有限和求導／取極限所得Σr q^r=q/(1−q)²，對q=3/4都收斂。因此每個record總費O(1)，N個record共O(N)；再加N+1個空呼叫的常數費，總work O(N)。與輸出下界合併，W_M(N)=Θ(N)。

這是對任意合法切點的一般計費證明；空run與不平衡的小child已包含，不以有限例子取代上界。



## 兩個 buffer 的完整 Parallel Merge Sort

先將輸入record附上原始位置，令X與Y兩個獨立長n陣列都含相同輸入（一次parallel copy），呼叫SORT(X,Y,0,n)，最後取Y。

```
SORT(src,dst,l,r):
    if l == r: return
    if r-l == 1: dst[l] = src[l]; return
    m = floor((l+r)/2)
    spawn SORT(dst,src,l,m)
    SORT(dst,src,m,r)
    sync
    P-MERGE(src,l,m,m,r,dst,l)
```

每個子呼叫交換buffer角色；parent的兩個已排序run會留在src，sync後merge到dst。P-MERGE輸入與輸出必須是不同實體buffer。這個雙buffer寫法避免每個sort呼叫再配置長度n的暫存，不把原地寫入與讀取尚未消費的輸入混用。

## 配置成本與實務取捨

上界假設未初始化的緩衝區能以 O(1) 配置，並計入每次實際讀寫。如果語言的配置動作必須序列初始化 n 個元素，單次初始化就可能引入 Θ(n) span，不能照搬前述界。

簡單版本各層同時可能存在 Θ(n) 個緩衝位置，最多 O(n log n) 額外空間；也可預先配置兩個長度 n 的緩衝陣列，在遞迴層間交換來源與目的，達 O(n) 額外空間，但必須另證各層讀寫無衝突。

小問題可改用序列 merge/sort 作為 cutoff，以降低工作排程開銷。這會改變常數及 span，需要按 cutoff 重新分析；理論上界不等於實測加速。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Buffer 角色交換的正確性與競爭檢查

每個SORT進入時，兩個buffer的當前區間都仍含該區間的原始輸入。根由copy建立；parent在啟動兩child前未改自身區間，兩個child又只碰各自互斥的半區，故每個child進入時也成立。

基例把唯一record複製到dst。由歸納假設，兩child結束後，parent的src左右兩run分別排序且元素不變；sync後的P-MERGE把兩者穩定合併到dst。歸納完成排序規格；parent無需承諾src最後仍與dst相同，只需在呼叫入口滿足條件。

兩child即使交換buffer角色，也仍在相同的一對全域陣列中寫互斥索引區間；parent只在sync後合併。總資料workspace O(n)，峰值task描述符O(n)、每條呼叫鏈O(lg n)；初始化copy工作Θ(n)、span Θ(lg n)，不改後述排序階數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 排序的緊界不能只把最壞成本相加

每層merge work Θ(n)，有Θ(lg n)層，故W_S(n)=Θ(n lg n)。span上界是

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\lg^2 n) = O(\lg^3 n).$$

要證最壞下界，需同一輸入在沿途各層都產生昂貴merge。對n=2^L，按L位元反轉索引的順序放入鍵；n=8例[0,4,2,6,1,5,3,7]。根的兩半排序後是偶數與奇數交錯run；每個子問題的相對秩亦是相同構造。故每層都有前述Ω(lg²(size))的merge span，且parent merge在兩child都完成後才開始。

沿一條child鏈相加Σr=1..L r²=Θ(L³)，得到二冪規模的最壞D_S(n)=Θ(lg³n)。平均平行度為Θ(n/lg²n)，不是把merge的平方對數錯當sort的span。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 非二冪規模也能構造同時昂貴的各層

任意n可遞迴安排各區間的相對秩：n偶數時，前半放偶秩、後半放奇秩；n奇數時，較短的前半放奇秩、較長的後半放偶秩。再對兩半各自的有序鍵集合做相同安排。每個sort節點排序後的兩run都互相交錯，長度差至多1；二冪時就是前頁的位元反轉族。

對這種merge，較長run的median左方有m個本run元素，另一run左方有m或m+1個。左child仍是長度差至多1的交錯run，總長介於N/2−1與N/2+1；N≥8時，兩run的長度也都為Θ(N)。所以沿該路徑有Θ(lg N)層，每層二分搜尋須Θ(lg N)比較（規模隨層數固定比例下降），總span為Θ(lg²N)。

同一遞迴秩配置讓每個sort節點都滿足此條件。沿較大的child走，規模每層約減半，且須等它完成後才開始parent merge，故可串接各層關鍵路徑，得到Ω(lg³n)。這補足任意n的緊下界，不只在不同層各自選互不相容的最壞輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 同一模型下比較 work 與最壞 span

| 算法 | Work | 最壞 span | 平均平行度 |
|---|---|---|---|
| 遞迴P-FIB | $\Theta(\varphi^n)$ | $\Theta(n)$ | $\Theta(\varphi^n/n)$ |
| row平行matrix-vector | $\Theta(n^2)$ | $\Theta(n)$ | $\Theta(n)$ |
| reduction matrix-vector | $\Theta(n^2)$ | $\Theta(\lg n)$ | $\Theta(n^2/\lg n)$ |
| 雙外圈平行matrix product | $\Theta(n^3)$ | $\Theta(n)$ | $\Theta(n^2)$ |
| 八路遞迴matrix product | $\Theta(n^3)$ | $\Theta(\lg^2 n)$ | $\Theta(n^3/\lg^2 n)$ |
| 平行Strassen | $\Theta(n^{\lg 7})$ | $\Theta(\lg^2 n)$ | $\Theta(n^{\lg 7}/\lg^2 n)$ |
| DOT reduction matrix product | $\Theta(n^3)$ | $\Theta(\lg n)$ | $\Theta(n^3/\lg n)$ |
| Parallel merge | $\Theta(n)$ | $\Theta(\lg^2 n)$ | $\Theta(n/\lg^2 n)$ |
| Parallel merge sort | $\Theta(n\lg n)$ | $\Theta(\lg^3 n)$ | $\Theta(n/\lg^2 n)$ |

表中漸近式採n充分大；空與單元素依程序基例。Work有效率、span小、空間少是不同目標；對固定P，還要把W/P、D及記憶體需求一起比較，不以一個平行度比值代替整個分析。



## 參考資料與署名

Cormen, Leiserson, Rivest, Stein, Introduction to Algorithms, third edition, Chapter 27. MIT 6.046J Fall 2004, Lecture 3, matrix multiplication and Strassen. MIT 6.046J Fall 2005, A Minicourse on Dynamic Multithreaded Algorithms, Handout 29.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

# A 求和

*Part VIII Appendix*

## 本章定位

從有限恆等式推導等差、冪和、幾何、消去與乘積，再用固定常數歸納、尾比率、dyadic切分與積分雙界分析迴圈；明列無限級數及換序的收斂前提。

## 前置知識

等差、等比、對數與積分估計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章路線

*學習目標*

- 使用求和公式與線性性。
- 以積分、分割與幾何級數建立界限。
- 避免把有限和與無窮級數混用。
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.1 Summation formulas and properties

$\sum_{k=1}^{n} a_k$是有限加總，n=0時定義為0；$\prod_{k=1}^{n} a_k$是有限乘積，n=0時為1。空sum與空product是加法／乘法單位元素，使迴圈零次執行的公式也成立。有限sum可任意重排，並有$\sum(ca_k + b_k) = c\sum a_k + \sum b_k$。

無限級數$\sum_{k=1}^{\infty} a_k$定義為partial sums $S_n$的極限，不是「把∞代進有限公式」。若極限不存在或無有限值即發散。$a_k \to 0$是收斂必要條件（$a_k = S_k - S_{k-1}$），卻非充分；1/k趨0但harmonic sum發散。

若$\sum_{k=1}^{\infty} |a_k|$收斂，稱絕對收斂，可以任意重排而不改和。條件收斂如$\sum_{k=1}^{\infty} (-1)^{k+1}/k$不保此性：重排會改partial sums的抵消模式。以下無限sum的逐項微積分須有收斂區間，不能把有限線性性無條件擴成任何無限操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 等差和的配對與歸納

$S_n = 1 + 2 + \ldots + n$，倒序同和n+(n−1)+⋯+1；逐項相加有$2S_n = n(n+1)$，故$S_n = n(n+1)/2$，含n=0。

歸納也能核：$S_0$=0；若$S_n = n(n+1)/2$，則$S_{n+1} = S_n + (n+1) = (n+1)(n+2)/2$。這裡歸納假設的係數固定，不隨n悄悄變動。

一般a+(a+d)+⋯+(a+(n−1)d)=na+d n(n−1)/2。例迴圈j=3..7每次2j−1次工作，和為$(2 \cdot \sum_{j=1}^{7} j - 7) - (2 \cdot \sum_{j=1}^{2} j - 2) = 49 - 4 = 45$；直接5+7+9+11+13=45。改起點要扣掉正確prefix，不是把n=7誤當項數5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 平方與立方和從消去式推導

由(k+1)³−k³=3k²+3k+1，k=0..n加總：

$(n+1)^3 = 3\sum_{k=0}^{n} k^2 + 3n(n+1)/2 + (n+1)$。整理得Σk²=n(n+1)(2n+1)/6。

由(k+1)⁴−k⁴=4k³+6k²+4k+1再加總，代入平方和與等差和，得到Σk³=n²(n+1)²/4=[n(n+1)/2]²。兩式n=0也成立。

例n=4：平方0+1+4+9+16=30；公式4・5・9/6=30。立方0+1+8+27+64=100；(4・5/2)²=100。公式是由整段telescoping恆等式推得，核小n只用來排除抄算失誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 等比和：有限、無限、邊界分開

$G_n(x) = \sum_{k=0}^{n} x^k$。把$xG_n - G_n$展開，中間項消去，得$(x-1)G_n = x^{n+1} - 1$；x≠1時$G_n = (x^{n+1} - 1)/(x-1)$。x=1須獨立給$G_n = n+1$，不可除0。

|x|<1時$x^{n+1} \to 0$，$\sum_{k=0}^{\infty} x^k = 1/(1-x)$。x=1時partial sums無界；x=−1時partial sums在1與0間振盪；|x|>1時terms不趨0，所以無有限和。

例x=−1/2、n=3，G=1−1/2+1/4−1/8=5/8；有限公式同值。無限和則2/3，不能把四項和直接寫成無限和。截尾誤差$\sum_{k=n+1}^{\infty} x^k = x^{n+1}/(1-x)$，絕對值$\le |x|^{n+1}/(1-|x|)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 權重等比和與微積分的合法區間

有限多項式$G_n$可逐項微分。更直接，$W_n=\sum_{k=0}^{n} kx^k$，計$W_n-xW_n$，得

$(1-x)W_n=x+x^2+\ldots+x^n-nx^{n+1}$，所以$W_n=[x-(n+1)x^{n+1}+nx^{n+2}]/(1-x)^2$（x≠1）。x=1時$W_n=n(n+1)/2$。

|x|<1時$n|x|^n\to 0$（固定底數指數超過多項式），取極限得$\sum_{k=0}^{\infty} kx^k=x/(1-x)^2$。不必未證明就交換無限sum與微分。power series在收斂半徑內可逐項微分／積分；本例半徑1，端點須另查。

再次對收斂半徑內的式子作用x・d/dx，得$\sum_{k=0}^{\infty} k^2x^k=x(1+x)/(1-x)^3$。x=1/2時$\sum_{k=0}^{\infty} k/2^k=2$、$\sum_{k=0}^{\infty} k^2/2^k=6$；不是由「每項趨0」直接推出收斂。



## Telescoping：只留下兩端

$\sum_{k=1}^{n}(a_k-a_{k-1})=a_n-a_0$，每個內部$a_k$恰加、減各一次。計算前先對齊索引及端點。

例如1/[k(k+2)]=(1/2)(1/k−1/(k+2))，故

$\sum_{k=1}^{n} 1/[k(k+2)]=(1/2)(1+1/2-1/(n+1)-1/(n+2))$。

n=3時1/3+1/8+1/15=21/40；右式(1/2)(3/2−1/4−1/5)=21/40。取n→∞和為3/4，這一步還用了末端分母發散至∞。

乘積也可消去：$\prod_{k=2}^{n}(1-1/k^2)=\prod[(k-1)/k]\cdot\prod[(k+1)/k]=(1/n)\cdot((n+1)/2)=(n+1)/(2n)$，n≥2；空乘積n=1時公式也為1。

## 乘積轉sum與幾個不能省的前提

若每個$a_k > 0$，$\ln \prod a_k = \sum \ln a_k$；若有0，ln不定義；若有負值，也不能直接用實數ln。例$\prod_{k=1}^{n} 2^{3k+1} = 2^{\sum(3k+1)} = 2^{3n(n+1)/2+n}$。

對$0 \le u_k < 1$，$\ln(1-u_k)$介於$-u_k/(1-u_k)$與$-u_k$（由ln界），可把乘積變成和來控制，但$u_k$接近1時下界分母會變大，不能當固定常數。

若對所有k≥1有0<c₁f(k)≤h(k)≤c₂f(k)，加總立即得c₁Σf≤Σh≤c₂Σf。若只有尾段成立，須把有限prefix單獨保留；當Σf→∞且prefix有限時不改Θ階。常數必對整段k一致，不可每個k各挑一個再宣稱統一Θ。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.2 Bounding summations

先定要精確值、上界或雙邊階，再選方法。可用歸納、逐項界、固定比率界、切分區間、積分；一個粗上界不等於已找到緊界。

例$S_n = \sum_{k=0}^{n} 4^k$。欲證$S_n \le c4^n$，n0要求c≥1；若$S_n \le c4^n$，則$S_{n+1} \le c4^n + 4^{n+1} = (c/4+1)4^{n+1}$，要$\le c4^{n+1}$需c≥4/3。取固定c=4/3對全部n成立。末項$4^n$給下界，所以$S_n = \Theta(4^n)$。

錯誤歸納「$\sum_{k=1}^{n} k = O(n)$，加n+1仍O(n+1)」隱藏了會長大的常數。若假設Σk≤cn，下一步需cn+n+1≤c(n+1)，即n+1≤c，不可能對任意n成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 最大項上界與尾段下界

對非負$a_k$，$\sum_{k=1}^{n} a_k \le n \cdot \max_{1\le k\le n} a_k$；對任何子集合J，$\sum a_k \ge \sum_{k\in J} a_k$。上界可用全體項，下界常只保留夠多的大項。

Σk≤n²。取k=ceil(n/2)..n，至少n/2筆，每項≥n/2，得Σk≥n²/4。對奇偶n同時成立，故Θ(n²)，不必先求精確公式。

固定實數r≥0，$\sum k^r \le n^{r+1}$；同一尾段給$\ge (n/2)(n/2)^r = 2^{-r-1}n^{r+1}$，故Θ$(n^{r+1})$。r固定很重要；若r=n，上下界常數$2^{-r-1}$不再與n無關，不能用此證明宣稱統一階。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 固定比例才有geometric尾界

假設$a_k \ge 0$且存在固定0<ρ<1，使$a_{k+1} \le \rho a_k$對所有k≥K成立。歸納得$a_{K+j} \le a_K \rho^j$，所以$\sum_{k=K}^{\infty} a_k \le a_K/(1-\rho)$。用乘法不等式寫可涵蓋$a_k = 0$，不需除0。

例$a_k = k^2/3^k$，k≥2時$a_{k+1}/a_k = (1/3)(1+1/k)^2 \le 3/4$。保留k0,1的prefix和1/3，從a₂=4/9起的尾和≤(4/9)/(1/4)=16/9，總和≤19/9；精確加權等比式給3/2，較小但同為有限界。

harmonic的比率k/(k+1)<1，卻趨1；不存在同一ρ<1控制全尾，不能套此法推收斂。若prefix本身依n，例如前兩項各n²，不能把「只有兩項」誤當O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

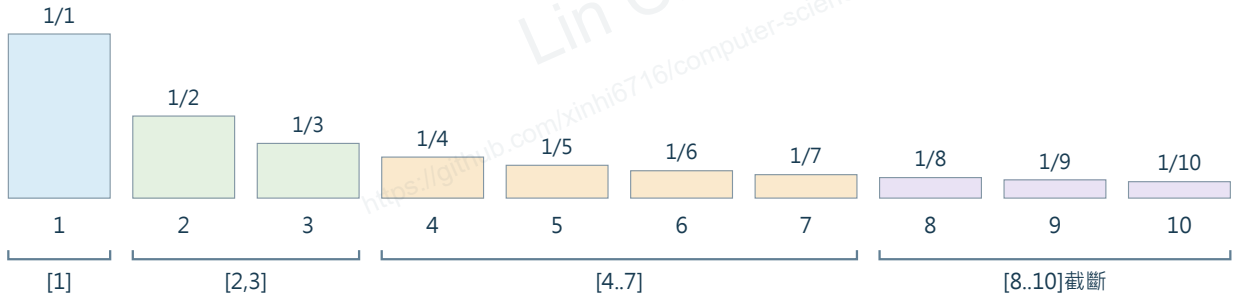
## Dyadic分塊看出harmonic的log層數

$H_n = \sum_{k=1}^{n} 1/k$。第j塊$k = 2^j .. 2^{j+1} - 1$有$2^j$項，每項介於$1/2^{j+1}$與$1/2^j$，所以完整塊貢獻介於1/2與1。

上界補齊最後不完整塊：共有floor(lg n)+1塊，每塊≤1，$H_n \le \text{floor}(\lg n) + 1$。下界取前floor(lg n)個完整塊，$H_n \ge \text{floor}(\lg n)/2$，n≥2時為Ω(lg n)。兩界一起給Θ(lg n)，也證無限harmonic發散。

例n=10：區塊[1]、[2,3]、[4,5,6,7]、[8,9,10]，實際貢獻1、5/6、319/420、121/360；總$H_{10}$=7381/2520≈2.928968。最後一塊只有3項，不應誤列為8項。

n=10：每個完整dyadic塊的和介於1/2與1






## 積分界由每個單位區間證明

若f在[m−1,n+1]可積且單調不減，對每整數k有$\int_{k-1}^{k} f(x)dx \le f(k) \le \int_{k}^{k+1} f(x)dx$；加總得$\int_{m-1}^{n} f \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f$。

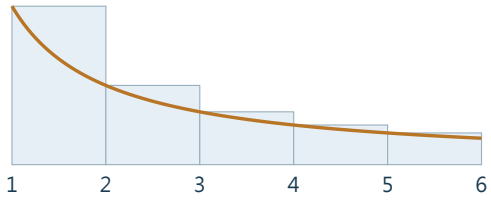
若f單調不增，各不等式反向：$\int_{m}^{n+1} f \le \sum f(k) \le \int_{m-1}^{n} f$。這裡不是把sum任意換成integral；左右端點的位移是控制誤差所需。

例如f=1/x在x>0遞減。下界$H_n \ge \int_{1}^{n+1} dx/x = \ln(n+1)$。上界不能從0積起，因1/x在0奇異；先取首項1，再對k2..n積分，得$H_n \le 1 + \int_{1}^{n} dx/x = 1 + \ln n$。

## 積分界由每個單位區間證明 （續）

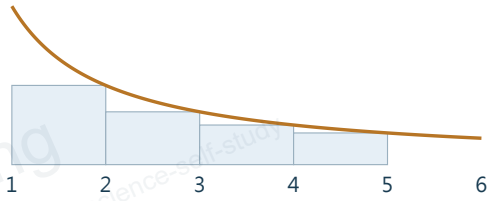
遞減f(x)=1/x：矩形高是1/k，兩邊的區間不同

下界：$\int_1^6 1/x\,dx \le H_5$




每個[k,k+1]：曲線≤1/k

上界：$H_5 - 1 \le \int_1^5 1/x\,dx$




每個[k−1,k]：曲線≥1/k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 可重算的積分上下界

n=5時$H_5$=137/60≈2.283333，ln6≈1.791759≤$H_5$≤1+ln5≈2.609438。夾界有可量化寬度，不是宣稱sum等於ln n。一般$\ln(n+1) \le H_n \le \ln n + 1$，所以$H_n = \ln n + O(1)$。

對p>1，$\sum_{k=1}^{n} 1/k^p \le 1 + \int_1^n x^{-p}dx \le 1 + 1/(p-1)$，單調有界partial sums收斂。p=1發散如前；0≤p<1，積分下界隨$n^{1-p}$增長，和為$\Theta(n^{1-p})$，不是所有terms趨0都收斂。

對$f = x^r$、固定r≥0，積分給$n^{r+1}/(r+1) \le \sum_{k=1}^{n} k^r \le [(n+1)^{r+1} - 1]/(r+1)$，再次得$\Theta(n^{r+1})$，也說明模型中的常數可依固定r。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### 雙重迴圈與改變加總次序

程式for i=1..n、for j=i..n，每次常數工作，次數$\sum_{i=1}^{n}(n-i+1)=n(n+1)/2$。也可依j計：每j有i=1..j，共$\sum_{j=1}^{n} j$。兩者數的是同一個索引集合{(i,j):1≤i≤j≤n}。

若每次工作j，則$\sum_{i=1}^{n}\sum_{j=i}^{n} j=\sum_{j=1}^{n}\sum_{i=1}^{j} j=\sum j^2=n(n+1)(2n+1)/6$。n=3各row和6、5、3，共14；按column1・1+2・2+3・3亦14。

這裡有限集合任意重排安全；無限雙重sum若含正負項，必先證絕對收斂，不能只因有限情況可換序就直接照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



### 資料來源

*書目*

### 章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# B 集合、關係、函數、圖與樹

*Part VIII Appendix*

## 本章定位

以逐元素邏輯與一般證明串聯sets、equivalence partitions、functions、graphs與trees；明列圖型/路徑/方向/空集合及binary slots慣例，完整證tree六等價與二分圖判準。

## 前置知識

基本離散數學符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章路線

*學習目標*

- 正確使用集合、關係與函數的定義。
- 辨識 graph 與 tree 的結構性質。
- 連結笛卡兒積、等價關係與遞迴結構。
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## B.1 Sets

set是不重複、無順序的元素集合，{u,v,u}={v,u}；multiset另記multiplicity，sequence另記位置，三者不能互換。x∈A表示元素，A⊆B表示對每個x，x∈A⇒x∈B；A⊊B另要求不相等。∅⊆A對任何A成立，∅∈A則未必；{∅}有一個元素，並非空集合。

集合相等須A⊆B且B⊆A。自然數本章採N={0,1,⋯}，Z為整數，R為實數。寫{x∈Z:x可被3整除}是用性質限定既有集合，不是排序後的list。

集合運算的定義是逐元素邏輯：x∈A∪B iff (x∈A or x∈B)，交集用and，差A\B用(x∈A and x∉B)。complement必指定universe U，$A^c$=U\A；更換U會更換complement。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 集合恆等式是對任意元素的命題

| 性質 | 兩個對偶版本 |
|---|---|
| 單位／吸收 | A∪∅=A；A∩∅=∅ |
| 冪等 | A∪A=A；A∩A=A |
| 交換／結合 | A∪B=B∪A；A∪(B∪C)=(A∪B)∪C，交集同理 |
| 分配 | A∩(B∪C)=(A∩B)∪(A∩C)；∪/∩互換亦成立 |
| 吸收 | A∩(A∪B)=A；A∪(A∩B)=A |
| De Morgan | $(B\cap C)^c=B^c\cup C^c$；$(B\cup C)^c=B^c\cap C^c$ |

例如$x \in A \setminus (B \cap C)$ iff x∈A且非(x∈B且x∈C)，iff (x∈A且x∉B)或(x∈A且x∉C)，iff $x \in (A \setminus B) \cup (A \setminus C)$。這是任意x的雙向等價；只畫一個有限Venn例子不代替全稱證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Partition、cardinality與重疊的計數

partition是非空blocks的集合：兩兩不交且聯集為S，使每個x∈S恰屬一塊。S=∅時允許空blocks集合，條件皆成立。例S={0,⋯,7}按mod3分為{0,3,6}、{1,4,7}、{2,5}。

對有限A,B，將A∪B拆成互斥的A\B、A∩B、B\A，可得|A∪B|=|A|+|B|−|A∩B|。例A={0,2,4,6}、B={2,3,6}，4+3−2=5；不能只把cardinalities相加而重複計交集。

三集合公式加三個單集合、減三個pair intersections、加triple intersection。一般inclusion-exclusion可逐元素證：一個落在r≥1個集合的元素係數為$\sum_{j=1}^{r}(-1)^{j+1}\binom{r}{j}=1$，因$(1-1)^r=0$。未落任何集合者係數0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Power set與Cartesian product的不同分母

power set P(S)={T:T⊆S}。若S有n個可區分元素，一個subset對每個元素只選入／不入，與n-bit字串一一對應，故$|P(S)|=2^n$；P(∅)={∅}仍有1筆。

ordered pair(a,b)保留位置，一般不同於(b,a)。A×B={(a,b):a∈A,b∈B}；固定a有|B|個pair，|A|個互斥rows給|A×B|=|A||B|。n-fold product計ordered n-tuples，有限集合時大小為各大小乘積；$A^0$含唯一空tuple，大小1。

例如A={p,q}、B={0,1,2}，A×B有6筆；P(A)只有∅,{p},{q},{p,q}四筆。sequence是由位置到值的function；相同值在不同位置可重複，不會像set自動合併。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 無限集合的大小要靠bijection

兩集合等cardinality指存在bijection，不是「兩個都無限就一樣多」。N到Z可設f(0)=0，f(2k−1)=k、f(2k)=−k（k≥1）；每個正負整數各有唯一原像，所以Z可數。N到偶數的2n也是bijection，無限集合可與自己的真子集等大。

R不可數可用diagonal argument：若(0,1)能列為$x_1,x_2,\cdots$，每個取不以無窮9結尾的十進位表示。建y的第i位：若$x_i$第i位不是1取1，否則取2。y∈(0,1)，只用1/2而無表示歧義，卻與每個$x_i$至少在第i位不同，矛盾。

有限集合的「注入且同大小即滿射」不能原封套到無限集合；f(n)=n+1在N上注入卻漏0。定理要清楚寫finite前提。



## B.2 Relations

binary relation R⊆A×B；在單一A上則R⊆A×A。aRb表示(a,b)∈R，沒有先要求每個a都有對象，也沒有要求唯一對象。

| 性質 | 量詞：適用所有a,b,c∈A |
|---|---|
| reflexive | aRa |
| symmetric | aRb⇒bRa |
| antisymmetric | aRb且bRa⇒a=b |
| transitive | aRb且bRc⇒aRc |

symmetric與antisymmetric不是互為否定：相等關係同時滿足兩者；含a↔b與b→c卻沒有c→b的relation可兩者都不滿足。空relation在非空A上對稱、反對稱、遞移皆真，卻不reflexive；不能憑「對稱且遞移」擅自假設aRb存在。

## Equivalence relation如何產生partition

equivalence relation同時reflexive、symmetric、transitive。定[a]={b∈A:aRb}。由aRa，[a]非空且各a被覆蓋。若c∈[a]∩[b]，則aRc、bRc，對稱及遞移給aRb；任意x∈[a]有bRaRx，故x∈[b]；反向同理，所以兩類只可能相同或不交。去掉重複類後便是partition。

反向給partition，定aRb iff a,b在同一block。每a恰在一塊，故reflexive；同塊是symmetric；a,b同塊且b,c同塊，由block唯一性三者同塊，故transitive。a所在block正是[a]，兩個建構互相對應。

例如mod3等價：a−b為3的倍數。0=3・0證反身，負號證對稱，相加證遞移；classes是餘數0,1,2三類，限制到有限S時得到前述三blocks。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Partial order、total order與極大／最大

partial order是reflexive、antisymmetric、transitive；total order再要求每對a,b可比較。total preorder只要求reflexive、transitive與任兩者可比較，允許不同元素互相等級相同，不必antisymmetric。

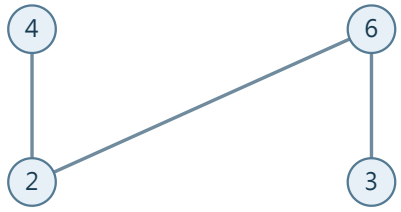
在poset內，maximum M滿足每個x≤M；maximal m只要求沒有嚴格更大的元素。minimum／minimal對偶。maximum若存在必唯一且maximal：若M,N都是maximum，M≤N且N≤M，由反對稱得相等；反向maximal不保證maximum。

S={2,3,4,6}以整除為≤。minimal={2,3}、maximal={4,6}，沒有minimum或maximum。Hasse圖只畫cover：x<y且沒有z使x<z<y；邊向上表示較大，省掉反身與傳遞邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Partial order、total order與極大／最大 （續）

S={2,3,4,6}：整除偏序的Hasse圖




向上：被整除的較大元素

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

沒有minimum / maximum

線段只表示cover，不代表數值距離；反身與傳遞關係不另畫邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.3 Functions

function f:A→B是在relation中要求每a∈A**恰有一個**b∈B的特例。A是domain，B是codomain；range f(A)可能只是B的子集。partial function若有a未定義，就不符合這裡的total function定義。

injection要求f(a)=f(a′)⇒a=a′；surjection要求對每b∈B存在a使f(a)=b；bijection同時滿足。例 f:N→N、f(n)=2n，注入非滿射；把codomain改成非負偶數，便是bijection。f(n)=floor(n/2)在N→N滿射非注入（0,1同映0）。

有限A,B，注入的不同輸入占不同codomain元素，故|A|≤|B|；滿射須每b至少一個原像，故|A|≥|B|。若有限且同大小，一個條件便推出另一個；無限時前頁已有反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inverse、image與composition的條件

對bijection f:A→B，$f^{-1}$(b)是唯一a使f(a)=b，並有$f^{-1} \circ f = id_A$、$f \circ f^{-1} = id_B$。若只injection，inverse只能定義在range；若非injection，relational inverse一般不是function。

set preimage $f^{-1}$(T)={a:f(a)∈T}對任意function及T⊆B都可定義，不要求可逆。它與「inverse function」同符號但不是同一個物件。

f:A→B、g:B→C的composition為(g∘f)(a)=g(f(a))。若f,g均注入，g(f(a))=g(f(a′))先取消g再取消f，得a=a′；若均滿射，給c先找b再找a，得composition滿射。兩者均bijection時，$(g \circ f)^{-1} = f^{-1} \circ g^{-1}$，順序反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.4 Graphs

本章有限directed graph G=(V,E)，E⊆V×V，可有self-loop但不記parallel edges；undirected simple graph的E是不同頂點的unordered pairs，沒有self-loop。若用multigraph須另保每條edge identity及loop的度數計法。

directed edge(u,v)從u出、到v入；outdegree數出邊，indegree數入邊，self-loop各計一次。undirected degree數incident edges。每無向邊恰貢獻兩個端點，故Σdeg=2|E|；有向Σoutdeg=Σindeg=|E|。因此無向奇度頂點個數必偶數。

同一張圖換頂點位置或畫成曲線不改E。兩條線交叉處除非明畫頂點，並非可轉向的節點；edge方向、頂點identity與數值須由圖例或edge list確定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Walk、simple path與cycle的約定

walk是一串$v_0, \ldots, v_k$，每相鄰一對有edge，長度k數邊不數點；CLRS的path允許重複，對應這裡walk。simple path要求所有頂點不同；u到自己的長0路徑存在。

本章談acyclic時，undirected cycle指長≥3、除起終點外頂點不重複的封閉路徑；只沿同一邊u→v→u往返不是cycle。directed simple cycle可長1（self-loop）或2（雙向邊），方向須逐邊成立。一般closed walk可有重複，不能與simple cycle混為同一物件。

若u到v有walk，遇重複頂點$v_i = v_j$，刪去兩次出現之間的閉段仍連成walk且更短；反覆刪除，有限長度降到simple path。這個一般消去論證使「可達」不必靠無限列出walk判定。

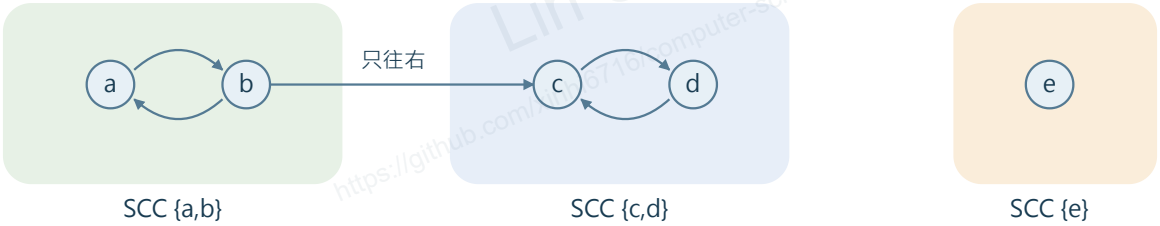


## 連通分量、SCC與方向

無向可達關係reflexive（長0）、symmetric（倒走）、transitive（串接），所以components是equivalence classes。有向可達仍反身、遞移，但可能不對稱；mutual reachability則三性皆具，classes稱SCC。

原創digraph：a→b、b→a、b→c、c→d、d→c，另e孤立。SCC為{a,b}、{c,d}、{e}；存在a→d卻沒有d→a，故不能把兩個SCC合併。忽略方向的版本只有{a,b,c,d}與{e}兩個components。

可達不等於互相可達；每條箭頭都有指定方向




a可達d，d不可達a。忽略方向後a/b/c/d同一component，e仍孤立。

## Subgraph、isomorphism與contraction

subgraph可取部分頂點與部分原edges，edges兩端仍須在所取頂點。induced subgraph G[W]則必保留所有兩端都在W的原edges，不能挑掉不喜歡的一條。例前圖W={a,b,c}的induced edges恰a→b,b→a,b→c。

isomorphism是bijection f:V→V′且(u,v)∈E iff (f(u),f(v))∈E′；只保一方向不足。頂點數、邊數、度數multiset相同是必要條件，不是充分：六頂點cycle $C_6$與兩個互斥三角形皆6邊、各degree2，但一個連通、一個不連通。

contract無向edge{u,v}時將端點合為新x，其他incident edges改接x，刪被縮edge；若保持simple graph表示，同一鄰居產生的重邊合一、自loop刪除。若演算法需保parallel edges（如某些隨機收縮），必明用multigraph，不能默默去重。



## 常見圖類與二分圖的一般判準

complete graph $K_n$每對不同頂點相鄰，|E|=n(n−1)/2。bipartite有V=L⊔R且每條edge跨兩側，等價可用兩色proper coloring。DAG是directed acyclic graph；tree／forest是無向圖概念，不能只看「箭頭沒有環」就叫tree。

二分圖不可能有奇cycle：沿cycle每跨一邊換側，奇次不能回到同一側。反向若無奇cycle，在每個component取根，依最短無向距離的奇偶分兩側。若edge(u,v)同奇偶，根到u、該edge、v回根構成奇長closed walk；任意奇closed walk若有重複頂點可拆為兩個closed walks，至少一個奇，反覆縮短會得到奇simple cycle，矛盾。

hypergraph的一條hyperedge可含多個頂點；可建incidence bipartite graph，一側原頂點、一側hyperedges，membership成為跨側edge。這個圖的節點種類與原圖不同，不能把一條hyperedge直接當普通pair。

## B.5 Trees

本節free tree要求n=|V|≥1、simple undirected、connected且沒有simple cycle。forest只要求acyclic，可有多個components與isolates；空forest允許，空圖不作本節tree六等價條件的輸入。

一個n≥2的tree必有leaf：取最長simple path的一個端點v。若v另有不在路徑的鄰居，可延長；若另接路徑內非相鄰頂點，便成cycle，皆矛盾。故v只有路徑上的一個鄰居，degree1。

移除leaf及其唯一邊後仍connected（其他點間simple path不用leaf作內部點）且acyclic。歸納n1有0邊，tree n筆有n−1邊。forest若有c個components，每個component是tree，逐塊相加得$|E| = \sum(n_i - 1) = n - c$（包含空forest n=c=0）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 六個條件為什麼等價：先看路徑與邊

對非空有限simple無向圖，下列六項等價：①tree；②每對頂點恰有一條simple path；③connected且刪任何edge都斷；④connected且m=n−1；⑤acyclic且m=n−1；⑥acyclic且加任何缺少的edge都成cycle。

①⇒②：連通給至少一條；若兩條不同simple paths，取第一次分歧與之後第一次會合的兩段，內部不交，合起來成cycle，矛盾。②⇒③：edge{u,v}本身就是唯一u-v路徑，刪後無路。

③⇒①：若有cycle，刪cycle一邊仍可沿其餘部分替代任何使用該邊的路徑，所以不斷，矛盾。①⇒④與⑤由前頁n−1邊證明。這些推論不把「有一條路」誤當「唯一一條路」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 六個條件的其餘方向與可重建構造

任何connected graph可建spanning tree：從一頂點S開始，只要S≠V，由連通性必有跨S與V\S的edge；加該edge與一個新頂點。每步只接一個新點，不成cycle；n−1步後得到n−1邊的tree，故一般connected graph m≥n−1。

④⇒①：上述spanning tree已用盡G的n−1邊，G無額外邊，故就是tree。⑤⇒①：由forest的m=n−c及m=n−1得c=1。①⇒⑥：任一新edge接u,v，與tree中唯一u-v路徑形成cycle。⑥⇒①：若有兩個不同components，加入跨component edge不可能成cycle，與條件矛盾；故connected。

六個方向已封閉。注意「m=n−1」單獨不夠：三角形加一個isolated vertex有n4,m3卻不是tree。相反「acyclic」也不保connected。

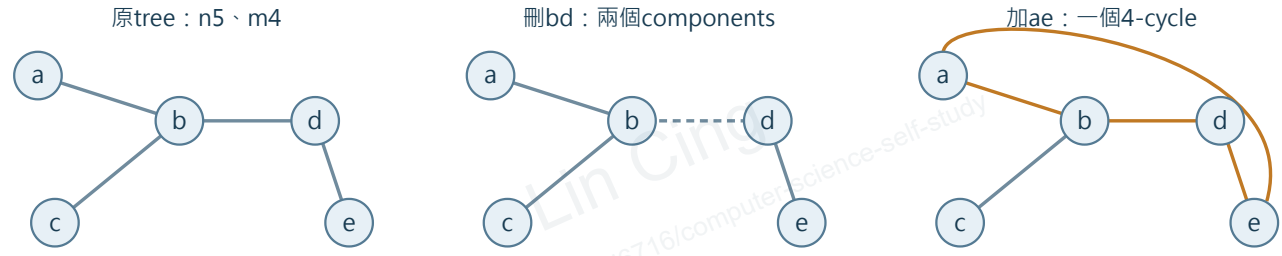
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一張圖的tree、非tree與新增cycle

原創tree V={a,b,c,d,e}，E={ab,bc,bd,de}，n5,m4。唯一a到e的simple path為a,b,d,e，長3。刪bd切成{a,b,c}與{d,e}；加入ae則形成a,b,d,e,a的4-cycle。

同一組頂點：刪tree邊必斷，加缺邊必成cycle




中圖虛線是被刪掉的邊；右圖橙色cycle沿a–b–d–e再走新邊e–a返回。

這裡有cycle時仍可能有spanning tree，只是圖本身不再是tree；minimum spanning tree的加權最佳化還需額外目標，並不是tree定義的一部分。

## Rooted與ordered：高度、深度與degree要說慣例

從free tree指定root r後，每個非根x的parent是唯一r-x simple path中前一點；其餘向下鄰居是children。ancestor／descendant本章包含自己，proper才排除自己；subtree rooted at x取x全部descendants。

depth(x)=根到x的邊數，height(x)=x到最深leaf的邊數；leaf height0，root depth0，tree height=max depth。rooted degree數children；free-tree graph degree還包含parent，故非根兩者差1。

若前頁tree以b為root，children a,c,d，d另有child e；depth(a,c,d)=1、depth(e)=2，height(b)=2、height(d)=1，其他0。b graph degree與children均3，d graph degree2但children1。

ordered tree另外定每節點children先後；把b的(a,c,d)改成(c,a,d)，rooted關係同但ordered tree不同。



## Binary／positional tree與空child

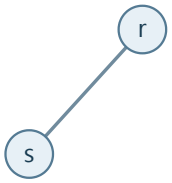
binary tree遞迴定義為空樹NIL，或一個root加互斥的left/right binary subtrees；只一個child時left與right仍不同。ordered tree只有一個child時沒有左slot／右slot之別，所以「degree≤2的ordered tree」不足以完全描述binary tree。

positional k-ary tree有固定slots1..k，空slot也帶位置資訊。將binary tree每個缺少child替成external NIL leaf，可得到每個internal node恰有兩個children的full binary tree，保留原left/right位置。

## Binary／positional tree與空child （續）

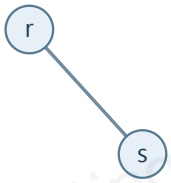
空slot也有位置：單一child不能省掉left/right資訊

只有left child



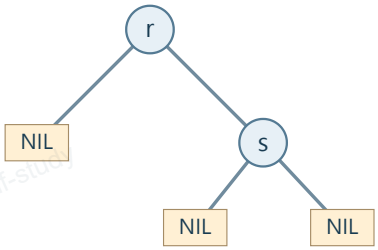

binary位置不同

只有right child




binary位置不同

右圖擴充：I2、L3




若原binary tree非空且有n≥1個實體nodes，總child slots2n，實體parent-child edges n−1，缺少slots=2n−(n−1)=n+1；擴充表示就有n個internal與n+1個external leaves。NIL sentinel若共用單一物件，邏輯空位置數仍是n+1，不等於實際配置n+1物件。空樹沒有實體父節點，故上述(parent,side) child slots為0；若另外表示空根的NIL位置，那是另一種計數。



## Full、complete與height的精確計數

full k-ary tree每個internal恰k個children，leaves可不同深度。以I internal、L leaves計edge：kI=I+L−1，故L=(k−1)I+1。k2時L=I+1。

CLRS此附錄的complete k-ary另要求全部leaves同深h（常有文獻稱perfect）；對k≥2，第d層$k^d$個nodes，leaves數 $L = k^h$，internal數 $I = \sum_{d=0}^{h-1} k^d = (k^h - 1)/(k - 1)$。例k3,h2：levels1,3,9，I4,L9、總13，edges12=3I。k1是鏈，須另算I=h、L1，不可除k−1=0。

heap章節常稱complete binary為「末層可不滿、由左填滿」，與這裡all-leaves-same-depth慣例不同，使用時須看定義。一般非空binary tree高h至多$\sum_{d=0}^{h} 2^d = 2^{h+1} - 1$個nodes，所以h≥ceil(lg(n+1))−1；最斜鏈h=n−1，沒有平衡前提不能聲稱所有binary tree高Θ(lg n)。

## 資料來源

*書目*

## 章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# C 計數與機率

*Part VIII Appendix*

## 本章定位

從sample object與統一分母建立計數、條件機率、Bayes及指標分析，再推完整geometric/binomial矩與尾界；明定獨立層級、零機率/支撐邊界及中心化r和總門檻t的區別。

## 前置知識

排列組合、機率公理與隨機變數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章路線

*學習目標*

- 用乘法、加法與補集計數。
- 計算期望與變異數，處理幾何與二項分布。
- 以 Chernoff bound 約束二項尾機率。
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## C.1 Counting

先定「一個結果」包含哪些資訊，再計數。sum rule要求互斥分類；product rule要求每個prefix後的選擇數已知。選擇集合可依先前結果改變，只要每層分支數固定，仍可相乘；若分支數不固定就逐branch加總，不能硬乘平均。

原創例：從6名學生選兩個不同職位，第一職6選、第二職5選，共30；若只是兩人committee、不分職位，每組被計兩個先後，除2得15。若允許同一人兼任，則變$6^2$=36。三個答案對應三種不同sample object。

長m、alphabet大小q的strings有$q^m$，允許重複且保位置。substring須連續，subsequence可跳位置；同一內容在不同起點是否算不同，必先約定。長n字串的長k≥1位置型substrings有max(0,n−k+1)個，不等於不同內容種類數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Permutations、combinations與不重複計數

相異n筆全部排列有n!；ordered選k筆無重複有P(n,k)=n!/(n−k)!，0≤k≤n。unordered k-subset每組恰對應k!個ordered結果，故$\binom{n}{k}$=n!/[k!(n−k)!]。k0唯一空選擇，$\binom{n}{0}$=1；k<0或k>n可擴充定為0。

若records可區分但keys重複，排列records仍n!；若只看key字串、各multiplicity為$m_1, \ldots, m_s$，則每key字串對應$\prod m_i!$種identity排列，故有$n!/\prod m_i!$個不同key字串。除法前須證每個結果被重複相同次數。

$\binom{n}{k}$=$\binom{n}{n-k}$由取補集bijection得。Pascal identity $\binom{n}{k}$=$\binom{n-1}{k}$+$\binom{n-1}{k-1}$，依某個指定元素不在／在subset分為兩個互斥且完備的類別。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 二項式展開與entropy界的來源

展開$(x+y)^n$時，每個factor選x或y；恰k個位置選x共有$\binom{n}{k}$種，故$(x+y)^n=\sum_{k=0}^{n}\binom{n}{k}x^k y^{n-k}$。x=y=1得$\sum\binom{n}{k}=2^n$，這也是按subset大小分類的計數。

對1≤k≤n，$\binom{n}{k}=\prod_{j=0}^{k-1}(n-j)/(k-j)\geq(n/k)^k$，因n≥k使每個factor≥n/k。上界$\binom{n}{k}\leq n^k/k!\leq(en/k)^k$，最後用$k!\geq(k/e)^k$。

另令α=k/n∈(0,1)，展開$(\alpha+1-\alpha)^n=1$，其中一項非負且≤1，故$\binom{n}{k}\leq\alpha^{-k}(1-\alpha)^{-(n-k)}=2^{nH(\alpha)}$，H(α)=−α lgα−(1−α)lg(1−α)。端點以0lg0=0連續延伸，$\binom{n}{0}=\binom{n}{n}=1$。H′=lg((1−α)/α)，最大在1/2，值1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## C.2 Probability

sample space S的elementary outcome是一個完整可區分結果，event是合法可測子集。有限／可數離散情況本章取所有subsets為events；不可數情況須指定對補集與可數聯集封閉的event族，不能無條件把每個subset都當event。

機率滿足P(A)≥0、P(S)=1，以及對兩兩互斥的有限／可數events，有$P(\cup A_i)=\sum P(A_i)$。因此P(∅)=0、P($A^c$)=1−P(A)、A⊆B⇒P(A)≤P(B)。離散時$P(A)=\sum_{s\in A}P(s)$。

有限S只有在每個outcome等機率時才可P(A)=|A|/|S|。例如丟兩個公平且獨立的0/1 bits，四個ordered結果各1/4；把它們壓成「0、1、2個1」後三種計數不是等機率，而是1/4、1/2、1/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 聯集、union bound與不需要的獨立性

由互斥分解A∪B=(A\B)⊔(A∩B)⊔(B\A)，P(A∪B)=P(A)+P(B)−P(A∩B)≤P(A)+P(B)。多事件定B₁=A₁、$B_i = A_i \setminus (\bigcup_{j<i} A_j)$，$B_i$互斥、聯集相同且$B_i \subseteq A_i$，故$P(\cup A_i) = \sum P(B_i) \leq \sum P(A_i)$，可數版本也成立。

此union bound不需獨立；右側可大於1，實用時再取min(1,右側)。相反若要把$P(\cap A_i)$寫成$\prod P(A_i)$，必須有mutual independence，不能因為union bound不需獨立就混用。

例三個failure events各≤0.01，不論相關性，至少一個失敗機率≤0.03。若三個事件其實完全相同，聯集機率0.01而不是1−0.99³；後式需要獨立前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Conditional probability與chain rule

P(A|B)=P(A∩B)/P(B)，要求P(B)>0；這是在保留B內outcomes後重新normalize。P(A∩B)=P(B)P(A|B)。反覆應用得$P(\cap_{i=1}^n A_i) = P(A_1) \prod_{i=2}^n P(A_i | A_1 \cap \ldots \cap A_{i-1})$，各conditional使用時須分母正；若某prefix機率0，整交集已為0，不能再寫未定義的比值乘積。

原創抽樣：袋中3藍2紅，無放回連抽兩球，P(兩藍)=(3/5)(2/4)=3/10；第二次藍的邊際仍3/5，但在第一次藍的條件下只有1/2，故兩次不獨立。若放回並重新均勻抽取，才是(3/5)²。

知道「至少一事件發生」與知道「指定那一次發生」通常不同。描述觀測或篩選規則是機率模型的一部分，不可只看到剩兩種文字敘述就自動各給1/2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Independence、互斥與pairwise的差異

A,B獨立指P(A∩B)=P(A)P(B)，這個定義也涵蓋機率0 events；若P(B)>0才可等價寫P(A|B)=P(A)。互斥指交集空，若兩者機率皆正則不獨立。空event則與任意event獨立。

mutual independence要求每個有限子集合的交集機率都等於邊際乘積；pairwise只核每一對，不足以乘三個以上。

取U,V為公平獨立bits，W=U XOR V，outcomes(0,0,0),(0,1,1),(1,0,1),(1,1,0)各1/4。U,V,W每對的四種組合都各1/4，故pairwise independent；但P(U=V=W=1)=0，而三邊際乘積=1/8，非mutual。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Independence、互斥與pairwise的差異 (續)

同一張完整表：pairwise independent，非mutually independent

| U | V | W=U XOR V | P(outcome) |
|---|---|---|---|
| 0 | 0 | 0 | 1/4 |
| 0 | 1 | 1 | 1/4 |
| 1 | 0 | 1 | 1/4 |
| 1 | 1 | 0 | 1/4 |

任一對：00/01/10/11各1/4

三個邊際P(=1)都為1/2

但111根本不在sample space

$0 \neq (1/2)^3 = 1/8$

每對核對成立，不代表三者交集可直接乘邊際；這裡variance可加，三重乘積不可分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Total probability與Bayes的分母

若$B_i$形成S的可數partition且$P(B_i)>0$，由$A=\sqcup(A\cap B_i)$，$P(A)=\sum P(B_i)P(A|B_i)$。Bayes再給$P(B_j|A)=P(B_j)P(A|B_j)/P(A)$，須P(A)>0。

原創監測器模型：故障prior P(F)=0.02；有故障時發alarm機率0.9，無故障時誤報0.1。則P(alarm)=0.02・0.9+0.98・0.1=0.116；posterior P(F|alarm)=0.018/0.116=9/58≈0.155172。

alarm的高sensitivity0.9不等於posterior0.9；大量正常樣本提供0.098誤報分母。兩次測試若只在給定F／$F^c$後獨立，雙alarm機率=0.02・$0.9^2$+0.98・$0.1^2$=0.026，posterior=0.0162/0.026=81/130≈0.623077；沒有先給條件獨立，不能平方likelihood。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Continuous uniform不給每點正機率

X在[a,b]連續uniform、a<b時，P(c≤X≤d)=(d−c)/(b−a)，a≤c≤d≤b。單點c=d機率0，但整區間機率1；可數可加性不要求對不可數個單點直接相加。

例[a,b]=[2,8]，P(3≤X≤5)=2/6=1/3，去掉兩端點結果相同。若Z=floor X，Z的值2..7各機率1/6，而Z=8只在單點X8出現，機率0；不得把七種整數標籤平均分配。

接下來主講離散random variables及pmf。離散機率質量P(X=x)與連續density不同：density可大於1，單點機率仍0。對連續分布應積分區間，不能把density當成每點機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## C.3 Discrete random variables

random variable X:S→R是outcome的數值函數，event{X=x}是其preimage。pmf $p_X(x) = P(X = x)$，非負且$\sum_x p_X(x) = 1$；本章用「probability mass function」，避免把離散質量與連續density混淆。

joint pmf p(x,y)=P(X=x,Y=y)，marginals為$p_X(x) = \sum_y p(x, y)$、$p_Y(y) = \sum_x p(x, y)$。給$p_Y(y) > 0$，conditional pmf為$p(x, y)/p_Y(y)$。X,Y獨立當且僅當所有x,y都有$p(x, y) = p_X(x)p_Y(y)$。

原創：兩個獨立公平四面骰A,B∈{1,2,3,4}，令M=max(A,B)。P(M≤m)=$m^2/16$，P(M=m)=[$m^2$−$(m-1)^2$]/16=(2m−1)/16，m1..4質量1,3,5,7除16；不同M值不是等機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Expectation是加權和，不必是可能結果

$E[X] = \sum_x x p_X(x)$。有限支撐必可算；可數支撐要E|X|<∞才有有限且可重排的有號期望。非負X允許E[X]=∞；不能把∞−∞當合法期望運算。

對四面骰最大值M，E[M]=(1・1+2・3+3・5+4・7)/16=50/16=25/8；E[$M^2$]=(1+12+45+112)/16=170/16=85/8。期望3.125不在整數支撐，仍是正確平均。

任意g，只要E|g(X)|<∞，按每個X值的preimage分類得到$E[g(X)] = \sum_x g(x)p_X(x)$，不用先重新列g(X)的所有不同值。非線性時一般E[g(X)]≠g(E[X])；平方的差就是variance。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 線性期望的完整推導與indicator

若X,Y可積，從joint pmf有$E[X+Y]=\sum_x\sum_y(x+y)p(x,y)=\sum_x x\sum_y p(x,y)+\sum_y y\sum_x p(x,y)=E[X]+E[Y]$；絕對可積允許拆與換序，沒有使用獨立。E[aX+b]=aE[X]+b。有限多變數逐次套用。

indicator $I_A$在A發生為1，否則0，$E[I_A]=P(A)$。若X計某些events發生次數，$X=\sum I_i$，故$E[X]=\sum P(A_i)$，即使相依也成立。例uniform排列n個相異keys，inversion指標每對期望1/2，總n(n−1)/4。

若X,Y獨立且可積，joint分解後$E[XY]=(\sum xp_X)(\sum yp_Y)=E[X]E[Y]$。這次用到獨立；若X=Y是公平bit，E[XY]=E[X]=1/2≠1/4。多項乘積通常需mutual independence，而非只pairwise。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Tail-sum與Jensen的理由

非負整數X逐outcome有$X=\sum_{i\geq 1}IX\geq i$。有限截尾先求期望再單調取極限（每項非負），得$E[X]=\sum_{i\geq 1}P(X\geq i)$，可含∞。若X可以負值，不能直接用這個式子。

convex f滿足f(λx+(1−λ)y)≤λf(x)+(1−λ)f(y)，0≤λ≤1。有限支撐以此二點定義歸納：把前m−1個權重normalize成一點，與最後一點再套凸性，得到f(E[X])≤E[f(X)]。

一般可積情況，若f在包含支撐及μ=E[X]的區間凸，μ有supporting line f(x)≥f(μ)+a(x−μ)，取期望即E[f(X)]≥f(μ)；端點退化情況X必集中該端點。需兩側期望有定義；這不是任意函數都可把f移入／移出E的規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Variance、covariance與相依成本

若$E[X^2]<\infty$，$\mu=E[X]$，$Var(X)=E[(X-\mu)^2]=E[X^2]-\mu^2\geq 0$；$\sigma=\sqrt{Var}$為standard deviation。$Var(aX+b)=a^2Var(X)$，不是乘a。前例M的$Var=85/8-(25/8)^2=55/64$，$\sigma=\sqrt{55/8}$。

$Cov(X,Y)=E[XY]-E[X]E[Y]$，展開平方得$Var(X+Y)=VarX+VarY+2Cov$。n項時$Var(\sum X_i)=\sum VarX_i+2\sum_{i<j}Cov(X_i,X_j)$。pairwise independent且二次矩有限足以使covariances全0；不需mutual independent。

但只同分布不足。若$X_1=\ldots=X_n=$同一公平bit，sum只有0或n，各1/2，$variance=n^2/4$；逐項variance相加只有n/4。缺少covariance會把不確定性嚴重低估。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Markov與Chebyshev：各自的適用條件

若$X\geq 0$且$E[X]<\infty$，對$t>0$逐點有$X\geq t\,I\{X\geq t\}$，取期望得$P(X\geq t)\leq E[X]/t$（Markov）。不需獨立，但不能套負值變數；有正負抵消時平均小不表示正尾小。

對二次矩有限Y，取非負$X=(Y-\mu)^2$與$t=a^2$，$a>0$，得到$P(|Y-\mu|\geq a)\leq Var(Y)/a^2$（Chebyshev）。它控制雙尾；若只要一側也可用同上界，但可能寬鬆。

例如四面骰最大值M的μ25/8、variance55/64，a=2，雙尾≤55/256≈0.214844；真事件只有M1，機率1/16。上界不是宣稱真機率等於bound；超過1的界可取1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## C.4 The geometric and binomial distributions

Bernoulli(p)取1（success）機率p、0機率q=1−p。$X^2=X$，故E[X]=p，Var=p−$p^2$=pq。說一串Bernoulli trials時，以下另要求mutual independence與相同p；只有各次邊際p不夠。

geometric本章與CLRS採**包含首次成功那次的trial數**G∈{1,2,⋯}，0<p≤1。G=k要求先k−1次失敗、再成功，$P(G=k)=q^{k-1}p$；p0沒有成功，G=∞，不是這個有限值geometric分布。

若別處採「成功前失敗次數」F=G−1，支撐從0開始、pmf為$q^f p$、E[F]=q/p；不能把兩種mean都寫1/p。p1時G≡1、F≡0，公式與variance0仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Geometric的和、期望與variance

$\sum_{k\ge1} q^{k-1}p = p/(1-q) = 1$。$P(G>t)=q^t$（整數t≥0），所以$P(G\ge i)=q^{i-1}$；tail-sum得E[G]=1/(1−q)=1/p。

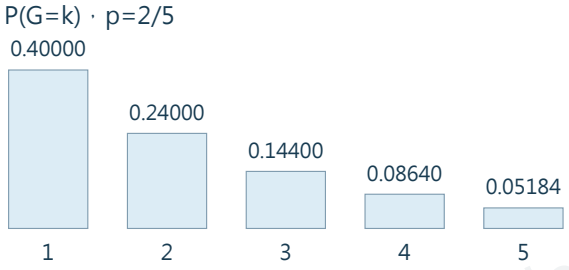
由加權等比公式可得$E[G^2]=(1+q)/p^2$：例如$\sum_{k\ge1} k^2q^{k-1} = (1+q)/(1-q)^3$，乘p即得。Var=$(1+q)/p^2-1/p^2=q/p^2$。q0情況直接由G1處理，不做除q。

原創p=2/5，k1..5的pmf依次2/5、6/25、18/125、54/625、162/3125；總2882/3125，剩尾$P(G>5)=(3/5)^5=243/3125$。E[G]=5/2、Var=15/4。期望不是「第2.5次真的成功」，而是多次重做此完整實驗的平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Geometric的和、期望與variance （續）

先固定計數慣例，再計pmf、尾端與期望




一個實現：F → F → F → S

G=4：包含最後成功trial

F=G−1=3：只數失敗

五次仍未成功：$q^5$=243/3125

此尾端沒有被前五根柱吃掉

橫軸是包含首次成功的trial index；E[G]=2.5，E[F]=1.5，兩種計數不可交換。



## Memoryless與重試的模型邊界

對0<p<1、整數s,t≥0，$P(G > s+t | G > s) = q^{s+t}/q^s = q^t = P(G > t)$。已失敗s次不改未來的success率，因各trial獨立且同p；不是「已經失敗很久，下一次更容易成功」。

若retry成功率隨次數改、系統有同一持續故障、或抽樣無放回，都不符合相同獨立trials。此時尾端可能是$\prod_{i=1}^{t}(1-p_i)$（獨立但不同p），或需要conditional probabilities，不能硬寫$q^t$。

若每trial固定成本c，總成本cG期望c/p；若成本依成功／失敗不同，總成本是$c_{\text{fail}}(G-1)+c_{\text{success}}$，期望$c_{\text{fail}}q/p + c_{\text{success}}$。先定成本與trial的關係，再套線性期望。

## Binomial的pmf、mean與variance

互相獨立同p的n次trial，$X = \sum_{i=1}^{n} I_i \in 0, \ldots, n$。固定一組k個成功位置，其機率$p^k q^{n-k}$；$\binom{n}{k}$個互斥位置集合給$b(k; n, p) = \binom{n}{k} p^k q^{n-k}$。總和$(p+q)^n = 1$。

線性期望給μ=np，不需獨立；variance要各對covariance0才可Σpq=npq。完整binomial pmf則用了mutual independence，pairwise只夠mean／variance，不保整個分布。n0退化X0；p0退化0、p1退化n，0⁰在pmf端點依空乘積視為1。

不同$p_i$但mutually independent時是Poisson-binomial，mean為$\sum_i p_i$、variance為$\sum_i p_i(1-p_i)$，不應把平均p塞進binomial pmf就說相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一張能逐項核對的binomial表

n=8,p=1/4,q=3/4，各pmf的共同分母4⁸=65536；分子$\binom{8}{k}3^{8-k}$如下。

| k | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| 分子 | 6561 | 17496 | 20412 | 13608 | 5670 | 1512 | 252 | 24 | 1 |

分子總65536，Σk・分子=131072給E[X]=2；Σk²・分子=360448給E[X²]=11/2，所以Var=11/2−4=3/2。

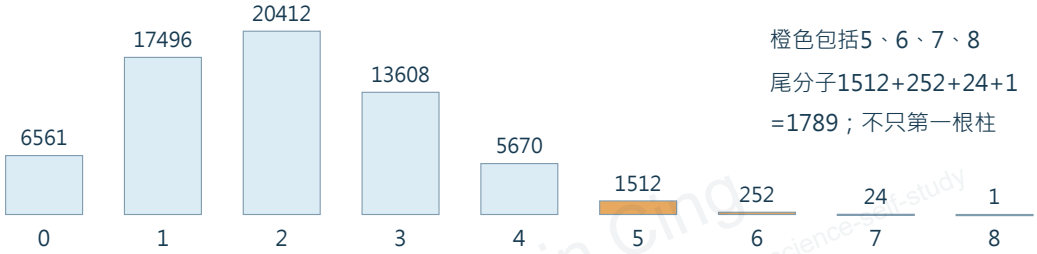
精確尾P(X≥5)=(1512+252+24+1)/65536=1789/65536≈0.027298。不是只算b(5)=1512/65536，也不是把「五次以上」漏掉k8。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一張能逐項核對的binomial表 （續）

單一機率柱與尾端總和不同：門檻5含5

每柱高度 = 分子 / 65536；n8、p1/4




橙色包括5、6、7、8

尾分子1512+252+24+1

=1789；不只第一根柱

Σ分子=65536；mean2、variance1.5；P(X≥5)=1789/65536≈0.027298。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Mode由相鄰比率定，不能直接當mean

對0<p<1、1≤k≤n，有b(k)/b(k−1)=[(n−k+1)p]/(kq)。比率>1 iff k<(n+1)p；等於1 iff k=(n+1)p。

若(n+1)p不是整數，唯一mode=floor((n+1)p)；若是整數m，m−1與m都是modes。n8,p1/4時(n+1)p=2.25，mode2，mean2恰巧相同；n4,p1/2時m=2.5，mode2；n3,p1/2時m2，modes1,2，mean1.5沒有整數機率柱。

對0≤k≤n，前述entropy/binomial coefficient界給$b(k) \le (np/k)^k(nq/(n-k))^{n-k}$，端點以連續或0⁰慣例處理。這是單一pmf上界，不是整個tail上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## C.5 The tails of the binomial distribution

先對齊嚴格／非嚴格尾：整數X的P(X≥a)從ceil(a)加到n；P(X>a)從floor(a)+1起。P(X<k)（整數k）只加0..k−1，別和P(X≤k)混用。

對X~Bin(n,p)，事件X≥k意味存在一組k個指定trials全部success。每組機率$p^k$，共$\binom{n}{k}$組；用union bound（組與組不必互斥）得$P(X \geq k) \leq \binom{n}{k} p^k$。k0界1，k>n真事件空。左尾用failures Y=n−X~Bin(n,q)：$P(X \leq k) \leq \binom{n}{k} q^{n-k}$。

數例n8,p1/4,k5，右尾≤56/1024=7/128≈0.054688，真尾1789/65536≈0.027298。重複計入同一次出現6個以上success的outcomes，正是上界通常不相等的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 左尾的geometric比率界：推導而非背公式

假設0<p<1、整數0<k<np，q=1−p。對1≤i≤k，b(i−1)/b(i)=iq/[(n−i+1)p]<kq/[(n−k)p]=ρ；因k<np，ρ<1。

由後往前，$b(k-j) < \rho^j b(k)$，j1..k。因此$P(X < k) < b(k) \sum_{j \geq 1} \rho^j = [\rho/(1-\rho)]b(k) = [kq/(np-k)]b(k)$。分母為正是前提，不可在k=np直接代入。

若0<k≤np/2，係數kq/(np−k)≤q≤1，故$L_k = P(X < k) < b(k)$，於是$L_k / L_{k+1} = L_k/(L_k + b(k)) < 1/2$。這說遠左尾再往左縮一個整數門檻時，機率至少減半。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 右尾由failures換元，門檻也要換

令Y=n−X~Bin(n,q)。當np<k<n，n−k<nq，將左尾界套到整數門檻n−k：

P(X>k)=P(Y<n−k)<[(n−k)p/(nq−(n−k))]b(n−k;n,q)=[(n−k)p/(k−np)]b(k;n,p)。

若k>(np+n)/2，則n−k<nq/2，左尾減半性給P(X>k)<(1/2)P(X>k−1)。success與failure的角色互換，不能只把原公式中的<改成>。

n8,p1/4,k4：係數(8−4)(1/4)/(4−2)=1/2，故P(X>4)<2835/65536≈0.043259；真尾1789/65536。它比本例union界7/128小，但不同參數時不必總是較緊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exponential moment把尾事件轉成非負量

令$I_i$為mutually independent Bernoulli trials（參數$p_i$），$X=\sum I_i$，$\mu=\sum p_i$。對任意λ>0及t，exp單調與Markov給$P(X\ge t)\le e^{-\lambda t}E[e^{\lambda X}]$。獨立性在下一步不可省：

$E[e^{\lambda X}]=\prod E[e^{\lambda I_i}]=\prod[1+p_i(e^{\lambda}-1)]\le \exp[\mu(e^{\lambda}-1)]$，因$1+z\le e^{z}$。故$P(X\ge t)\le \exp[\mu(e^{\lambda}-1)-\lambda t]$。

若μ>0、t>μ，對λ微分$\mu e^{\lambda}-t=0$，最小在λ=ln(t/μ)>0，二階$\mu e^{\lambda}>0$。代入得$P(X\ge t)\le e^{-\mu}(e\mu/t)^{t}$。此界允許各$p_i$不同；不需同分布，但需mutual independence。μ0時全部$I_i=0$幾乎必然，正尾直接0，不代ln(t/0)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 原書中心化r界與較精細的t界不能混用

CLRS所列較鬆形式：對r>μ>0，$P(X-\mu \geq r) \leq (e\mu/r)^r$。可直接證：$E[e^{\lambda(I_i-p_i)}] = p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda})$。相乘再用Markov，得$P(X-\mu \geq r) \leq \exp(\mu e^{\lambda} - \lambda r)$；取λ=ln(r/μ)即得。

本式r是**超過mean的幅度**，事件門檻是t=μ+r；前頁t則是**總success門檻**。若μ10,r40，中心化事件X≥50，較鬆bound為$(e/4)^{40} \approx 1.947 \times 10^{-7}$；精細t界則$e^{-10}(e/5)^{50} \approx 2.650 \times 10^{-18}$。兩者都是上界，不能把40與50交換後仍稱同一推導。

r只略大μ時$(e\mu/r)^r$可超1，仍合法卻無資訊，需取min(1,bound)。若是整數binomial，真sum起點為ceil(μ+r)；不把非整數門檻向下捨。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 同一binomial尾端比較不同保證

原創X~Bin(1000,0.01)，μ10，關心X≥50：Markov≤10/50=0.2；Chebyshev以Var9.9及偏差40給≤9.9/1600=0.0061875；中心化r40界約1.947e−7；精細t50界約2.650e−18。

精確值仍是$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}$，可用相鄰比率遞推或log-pmf避免直接算大factorial後浮點溢出。這些界用到的條件不同：Markov只需非負與mean；Chebyshev需variance；最後兩個用mutual independent indicators。

若所有$I_i$共用一個Bernoulli(.01)，X只可能0或1000，P(X≥50)=.01，仍符合Markov，但原variance與exponential-moment乘積均不適用。相同marginals不是使用binomial／Chernoff的充分理由。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 資料來源

*書目*

## 章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix C。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# D 矩陣

*Part VIII Appendix*

## 本章定位

以entry、尺寸及field前提重建矩陣運算，完整追蹤消去、rank因式分解與inverse；證明row/column rank、det運算律與Gram正定條件，明確區分實數、複數與有限field。

## 前置知識

向量空間、線性映射與矩陣乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章路線

*學習目標*

- 使用矩陣運算與轉置。
- 辨識 rank、determinant、inverse 的關係。
- 理解 SPD 與特徵值的必要條件。
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## D.1 Matrices and matrix operations

本附錄除另註外取輸入維度為正整數、entries在R，vector預設column；rank0的factorization允許0個中間columns／rows，空加總定為0。$A \in \mathbb{R}^{m \times n}$有m個rows、n個columns，$a_{ij}$先row再column。矩陣相等須尺寸相同且每個entry相同；矩陣加法與scalar multiplication逐entry定義。

例$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}$是2×3，$A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$是3×2，$(A^T)_{ji} = a_{ij}$。轉置不是把同一串記憶體不加定義就改形狀；row-major／column-major是儲存慣例，數學transpose由索引對應定義。

向量$x = (1, -1, 2)^T$是3×1，$x^T$是1×3。$e_i$是第i筆1、其他0的standard basis vector，維度由使用處指定；0矩陣亦須尺寸相符。scalar0與2×3零矩陣不是相同型別。



## 常見square結構的精確零位置

| 類型 | n×n矩陣的entry條件 |
|---|---|
| diagonal | $i \neq j \Rightarrow a_{ij} = 0$ |
| identity $I_n$ | 對角1、其他0 |
| tridiagonal | $\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$ |
| upper triangular | $i > j \Rightarrow a_{ij} = 0$ |
| lower triangular | $i < j \Rightarrow a_{ij} = 0$ |
| unit triangular | 對應triangle且全部對角1 |
| permutation | 每個row及每個column恰一個1、其餘0 |
| real symmetric | $A = A^T$ |

## 常見square結構的精確零位置 (續)

這些類別可重疊：identity同時diagonal、上下triangular、symmetric、permutation。tridiagonal不保symmetric，因上下對角線數值可不同。unit triangular不表示整張矩陣都是1；unit vector也不是全部entries1的向量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 乘法先核維度，再核每個收縮索引

若$A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times p}$，才有$C = AB \in \mathbb{R}^{m\times p}$，$c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$。中間索引k被加總，外側i,j留下；不是逐entry相乘。

沿用A，取$B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$，則$AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$。四筆計算依序：1・2+2・(−1)+0・4=0；1・1+2・3+0・0=7；0・2+1・(−1)+1・4=3；0・1+1・3+1・0=3。

反向BA也有定義但為3×3：$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}$，與AB連尺寸都不同。即使兩者同為square也通常AB≠BA；交換律不能由scalar算術照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 乘法先核維度，再核每個收縮索引 (續)

先看尺寸，再沿同一個k對齊的三項相加

A：2 rows × 3 columns

| | | |
|---|---|---|
| 1 | 2 | 0 |
| 0 | 1 | 1 |

×

B：3 rows × 2 columns

| | |
|---|---|
| 2 | 1 |
| -1 | 3 |
| 4 | 0 |

=

C=AB：2×2

| | |
|---|---|
| 0 | 7 |
| 3 | 3 |

C[1,2] = A的row1 · B的column2 = 1×1 + 2×3 + 0×0 = 7

黃色兩段各長3，收縮同一個k；輸出保留row1、column2，不能逐entry相乘。



## 矩陣乘法的可實作版本與成本

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

n≥1時共mnp次scalar multiplies、mp(n−1)次scalar additions，另有mp次輸出寫入及迴圈控制。單次scalar算術O(1)的RAM模型下為Θ(mnp)；n×n為Θ($n^3$)。若先s=0並做n次s+=，則多做mp次加法，是另一個精確計數但同階。

輸出C不得alias尚待讀的A/B；否則寫C可能破壞後續輸入。任意精度整數或field元素的bit成本須另計；「矩陣乘法做$n^3$次乘法」不是「任意bit長皆$n^3$時間」的保證。

## 結合、分配、轉置：每個等式都要尺寸相容

令A為m×n、B為n×p、C為p×q。第(i,j)筆的$(AB)C$為$\sum_{k=1}^{p}\sum_{\ell=1}^{n}a_{i\ell}b_{\ell k}c_{kj}$；有限sum換序得$\sum_{\ell=1}^{n}a_{i\ell}\sum_{k=1}^{p}b_{\ell k}c_{kj}=(A(BC))_{ij}$，故結合成立。分配律由scalar分配逐entry得；單位元須寫$I_mA=AI_n=A$。

$(AB)^T=B^TA^T$：左邊第(j,i)筆是$\sum_k a_{ik}b_{kj}$；右邊同一位置$\sum_k(B^T)_{jk}(A^T)_{ki}=\sum_k b_{kj}a_{ik}$，由scalar交換相等。矩陣乘法本身不交換，轉置後順序反轉不可漏。

若A,B都real symmetric，A+B與A−B仍對稱，但AB對稱 iff AB=BA，因$(AB)^T=BA$。不能把「兩個對稱矩陣」直接當成「其積也對稱」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Ax、inner product與outer product的三個輸出型別

將A的columns記$a_1,\dots,a_n$，$Ax=\sum_j x_ja_j$是m-vector。沿用A與$x=(1,-1,2)^T$，$Ax=(1,0)^T-(2,1)^T+2(0,1)^T=(-1,1)^T$，與逐row點積相同。

同長real vectors u,v的$u^Tv$是scalar；$uv^T$是matrix。例$u=(1,2)^T$、$v=(3,-1)^T$，$u^Tv=1$，$uv^T=\begin{bmatrix}3&-1\\6&-2\end{bmatrix}$。outer product每個column為$v_ju$，故rank≤1；u,v都非零時至少一個column為非零倍u，rank=1。任一為0則rank0。

$\|x\|_2=\sqrt{x^Tx}$只對real x直接如此寫；complex需$x^*x=\sum_i|x_i|^2$，其中*是conjugate transpose，不是普通transpose。例x=[i]，$x^Tx=-1$，不是長度平方。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Permutation左乘換rows，右乘換columns

取$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，則$Px = (x_3, x_1, x_2)^T$。若A有3個rows，PA把rows改為舊row3、row1、row2；因$(PA)_{ij} = \sum_k p_{ik} a_{kj}$，每row恰選一筆。

若M有3個columns，MP的columns則是舊column2、column3、column1，因P的三columns依序$e_2, e_3, e_1$。左乘與右乘的重新排列方向要從entry式或basis columns讀取，不能只記「都照3,1,2」。

不同columns of P是不同basis vectors，$P^T P = I = P P^T$，故$P^{-1} = P^T$。兩個permutation matrices相乘仍是permutation matrix，因它們的basis重排composition仍恰是一個bijection。



## D.2 Basic matrix properties

vectors $v_1, \ldots, v_n$線性獨立指$\sum_j c_j v_j = 0$只可能所有$c_j = 0$；若存在非全0係數就是dependent。A的column rank是最大獨立column數，row rank類似，zero matrix rank0。

因$Ax = \sum_j x_j a_j$，columns獨立 iff Ax=0只有x=0；若有非零x使Ax=0，本文稱nonzero null vector。整個null space含0，不能把「沒有非零null vector」誤寫成「null space沒有任何向量」。

rank不是非零entries個數，也不是只數原始主對角非零數。例$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$有8個非零entries，但column3=column1+column2，且前兩column獨立，所以rank2；null vector為$(-1, -1, 1)^T$。

## 消去法的操作與完整中間狀態

三種elementary row operations都可逆：交換兩rows；一row乘非零scalar；一row加另一row的倍數。每種可表示左乘invertible elementary matrix，所以不改column之間的linear dependence；rows本身的span也不改。

對M：$R_2 \leftarrow R_2 - 2R_1$，得 $\begin{bmatrix}1&2&3\\0&0&0\\0&1&1\end{bmatrix}$；交換$R_2, R_3$，得 $\begin{bmatrix}1&2&3\\0&1&1\\0&0&0\end{bmatrix}$；$R_1 \leftarrow R_1 - 2R_2$，得$R = \begin{bmatrix}1&0&1\\0&1&1\\0&0&0\end{bmatrix}$。

R有pivots columns1,2，第三變數free，解$Rx=0$為$x_1=-x_3$、$x_2=-x_3$，所以null space為$\{t(-1,-1,1)^T : t \in \mathbb{R}\}$。非零row2個，彼此leading positions不同而獨立；非pivot column是pivot columns之和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 消去法的操作與完整中間狀態 （續）

M=BC：中間維度2，從消去結果追回原columns

原M：3×3、rank2

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 4 | 6 |
| 0 | 1 | 1 |

=

B：原pivot columns

| | |
|---|---|
| 1 | 2 |
| 2 | 4 |
| 0 | 1 |

×

C：各column的座標

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 1 |

M的column3 = column1 + column2，所以C的column3=$(1,1)^T$。

B保留原向量；消去後的unit columns只用於讀取座標，不能直接取代B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 不預設主對角可用的Gauss–Jordan

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

每步pivot前，前r個pivot columns已是distinct unit columns；未用rows在它們上皆0。新pivot取剩餘row、normalization後消去其他rows，不破壞舊pivot columns，因新row在舊pivot位置0。若剩餘rows該column全0才跳過。迴圈有限，得到r個pivots與zero rows，field operations O(mn・min(m,n))。

精確算術中!=0依field判定；浮點則需pivoting與尺度／容許值，不能把固定|pivot|<$10^{-12}$當成對所有矩陣普遍正確的rank定義。



## 為什麼row rank等於column rank

設R=EA為row-reduced形式，E是invertible。E乘在每個column上是同一可逆線性映射，所以columns的相依係數在A與R相同；R恰r個pivot columns獨立，其他column是它們線性組合，故column rank(A)=r。

row operations每步以可逆組合改rows，因此row span(A)=row span(R)。R非零rows r個，依pivot位置可逐係數消去證獨立，故row rank(A)=r。於是可統稱rank(A)，介於0與min(m,n)。

nullity=n−r：pivot variables由n−r個free variables唯一線性決定；每次只令一個free variable1、其餘0得到一個null basis。它們獨立（看free coordinates），也生成所有null solutions，完成dimension計數，不只在M一例成立。

## Rank factorization與乘積rank界

取A原本的r個pivot columns組成B（m×r），每個原column在B下的coordinates排成C（r×n），則A=BC。B columns獨立；C含r×r identity位置。反向若A=UV且U只有s個columns，A的columns全在span(U)，rank(A)≤s，所以r也是最小可因式分解的中間維度。

M的原pivot columns給$B=\begin{bmatrix}1&2\\2&4\\0&1\end{bmatrix}$，$C=\begin{bmatrix}1&0&1\\0&1&1\end{bmatrix}$，逐column乘回恰M。不可拿R的pivot columns當成原M的columns，因row operation改變了向量本身。

對相容A,D，若$A=B_rC_r$，則$AD=B_r(C_rD)$，rank(AD)≤rank(A)；若$D=U_sV_s$，$AD=(AU_s)V_s$，亦≤rank(D)。故rank(AD)≤min(rank A,rank D)。若左因子P可逆，配合A=P⁻¹(PA)反向界得到rank(PA)=rank(A)；右側可逆同理。



## Inverse存在與唯一性的全部條件

n×n的inverse B要求AB=I=BA。若B,C皆inverse，B=B(AC)=(BA)C=C，故唯一。可逆A,B有(AB)⁻¹=B⁻¹A⁻¹；左右各乘檢查都回I。$(A^{-1})^T=(A^T)^{-1}$亦由轉置兩個inverse等式得。

square A full rank iff無非零null vector，等價linear map x↦Ax injective。n個獨立columns在n維空間成basis，故每$e_j$有唯一解$Ab_j=e_j$；把$b_j$排成B得AB=I。再由A(BA−I)=0及A injective，逐column得BA=I，故full rank iff invertible。反向若Ax=0，乘inverse即x=0。

rectangular時只一側inverse不保另一側：A=[1,0]、$B=(1,0)^T$有AB=[1]，但BA=diag(1,0)≠I₂。不能把square定理不改尺寸套用。

## 一個可逐步核對的inverse

取$H=\begin{bmatrix}2&1\\3&2\end{bmatrix}$，從augmented matrix[H|I]做同一組row operations：

$$\left[\begin{array}{cc|cc}2&1&1&0\\3&2&0&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1/2&-3/2&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1&-3&2\end{array}\right].$$

依序為$R_2-(3/2)R_1$、$R_2$乘2；再$R_1-R_2$，最後$R_1$除2，得到$[I\mid H^{-1}]$，$H^{-1}=\begin{bmatrix}2&-1\\-3&2\end{bmatrix}$。檢查$HH^{-1}=\begin{bmatrix}4-3&-2+2\\6-6&-3+4\end{bmatrix}=I$，反向乘也為I。

消去法對左半做E，右半同時變E；當EH=I，E就是inverse。若左半無法產生n個pivots，不能假裝最後右半就是inverse。



## Determinant、minor與cofactor

對square A，minor matrix A[ij]刪row i及column j；cofactor為$C_{ij}=(-1)^{i+j}\det A[ij]$。n1時det[a]=a；若採0×0 convention，det(empty)=1以統一cofactor式。沿row i展開$\det A=\sum_{j=1}^{n}a_{ij}C_{ij}$。

原創$N=\begin{bmatrix}2&1&0\\1&3&2\\0&-1&1\end{bmatrix}$，第一row signs為+−+，所以detN=2・(3・1−2・(−1))

−1・(1・1−2・0)+0=10−1=9。minor是矩陣；其det是scalar；cofactor再加正負號，三者不混同。

一般permutation式為$\det A=\sum_{\sigma\in S_n}\operatorname{sgn}(\sigma)\prod_{i=1}^{n}a_{i,\sigma(i)}$。每項在每row／column各選一筆，sign由permutation parity決定；按固定row選到的column分組便得到cofactor展開。

## Determinant運算律的原因

permutation式對每個column各自linear；兩columns相同時，把交換這兩column的permutations配對，項互消，故det0；交換兩columns使sign反號。對rows同理，且σ↦σ⁻¹使$\det(A^T) = \det(A)$。因此一row乘α，det乘α；一row加另一row的倍數，multilinearity加出的重複row項為0，det不變。

任一對columns交替multilinear的scalar函數F，把每column依basis $e_i$展開；重複basis index的項為0，留下各permutation，且$F(e_{\sigma(1)}, \ldots, e_{\sigma(n)}) = sgn(\sigma)F(I)$，所以F(B)=F(I)det(B)。

固定A，令F(B)=det(AB)；AB每column為A乘該column，故F仍交替multilinear，F(I)=det(A)。由唯一展開得det(AB)=det(A)det(B)。這證明不要求A預先可逆；若det A=0，F亦處處為0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Determinant與singularity，不只背2×2公式

upper triangular的非零permutation項須σ(i)≥i對每i成立；但Σσ(i)=Σi，所以只能全部σ(i)=i，故det是對角乘積。lower triangular同理，或轉置化成upper。row echelon若有zero row，det0；若全部n個pivots非零，triangular對角乘積非零。

消去過程的row swap、非零scaling只乘非零因子，row-add不改det，故原det為0 iff echelon缺pivot iff rank<n iff singular。於是det≠0是可逆判準，不代表數值上well-conditioned。

例diag(1,ε)，ε≠0時det ε且可逆；ε很小仍是精確full rank，inverse含1/ε，誤差可能放大。實際解Ax=b通常以穩定factorization求解，不必先顯式形成inverse；數學存在性與數值可靠性是不同問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Real symmetric正定與Gram的一般證明

本附錄正定採**real symmetric** S且對每個非零real x有$x^T Sx > 0$；semidefinite把>改≥。若不要求對稱，quadratic form只看$(S + S^T)/2$，不能由$x^T Sx > 0$自動推出$S = S^T$。

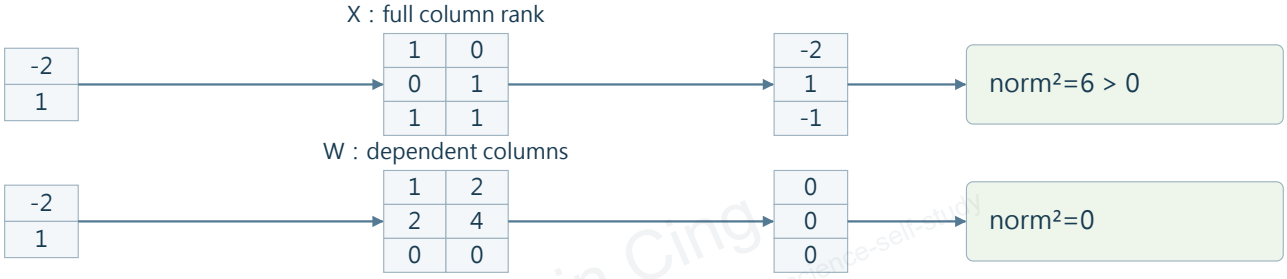
對任意real m×n A，$G = A^T A$為symmetric，且$x^T Gx = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0$。等於0 iff Ax=0。因此G正定 iff A無非零null vector iff A full column rank；若n>m就不可能。只說「A非零」不足。

原創$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$，$X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$；對$x = (s,t)^T$，$x^T Gx = s^2 + t^2 + (s+t)^2 > 0$除非s=t=0。對$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}$，$W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix}$，取$x = (-2,1)^T$使Wx=0，只有semidefinite。



## Real symmetric正定與Gram的一般證明 (續)

比較$x^T A^T Ax = \text{norm}(Ax)^2$：同一輸入，兩種rank




同一非零x=(−2,1)ᵀ。一般正定結論要證全部非零x；W只需一個null vector即可反證。

## 複數與有限field：哪些定理仍可用

前述加乘、inverse、rank及det的field論證可改用C或prime field $F_p$；但R的正負及euclidean內積不能照搬。composite模數的ring不一定每非零元素可除，例如mod6的2非零卻無inverse，所以pivot normalization與field rank定理須重查。

complex版本以$A^*$為conjugate transpose，Hermitian要求$S^* = S$。Gram $A^*A$ semidefinite，full column rank時positive definite，因$x^*A^*Ax = \sum_i |(Ax)_i|^2$。普通$A^TA$不保此結論：A=[i]時$A^TA = [-1]$，$A^*A = [1]$。

對real或complex square A，分別有$\det(A^TA) = (\det A)^2$，與$\det(A^*A) = |\det A|^2 \geq 0$；第一式只在real情況可推≥0。代數中的transpose恆等式和內積中的conjugation須依field清楚區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 資料來源

*書目*

## 章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix D。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study